ECE 408

目录

第 1 讲 Introduction **1**
1.1 课程框架与技术背景 . . . 1
1.2 课程运行框架、作业流程与学术规范 . . . 3
1.3 从摩尔定律时代到后登纳德时代 . . . 10
1.4 中央处理器与图形处理器的设计取向 . . . 15
1.5 图形处理器的起源、统一计算设备架构与应用扩张 . . . 20
1.6 并行化工作流与核心性能挑战 . . . 26
1.7 阿姆达尔定律：整体加速比的串行上界 . . . 30
1.8 综合框架与关键结论 . . . 34

第 2 讲 Introduction to CUDA C and Data Parallel Programming **37**
2.1 核心内容 . . . 37
2.2 从线程到大规模数据并行 . . . 37
2.3 CUDA 的主机-设备执行模型 . . . 40
2.4 线程层次结构与一维索引映射 . . . 41
2.5 向量加法：从顺序循环到线程网格 . . . 44
2.6 核函数实现、启动配置与边界检查 . . . 47
2.7 函数限定符与 CUDA 编译流程 . . . 50
2.8 异步执行与错误检查 . . . 52
2.9 完整、可检查的向量加法程序 . . . 54
2.10 例题与完整推导 . . . 57
2.11 常见错误、检查清单与公式表 . . . 58
2.12 综合框架与关键结论 . . . 61

第 3 讲 CUDA Parallel Execution Model: Multidimensional Grids and Data **62**
3.1 核心内容 . . . 62
3.2 多维执行层次与索引系统 . . . 64
3.3 多维启动配置与数据布局 . . . 68
3.4 实例一：RGB 彩色图像转灰度 . . . 72
3.5 实例二：邻域平均模糊 . . . 76
3.6 实例三：矩阵与矩阵乘法 . . . 80
3.7 例题与完整推导 . . . 85
3.8 统一方法与关键结论 . . . 88

第 4 讲 Compute Architecture and Scheduling **92**
4.1 核心内容 . . . 92
4.2 GPU 计算架构与线程块驻留 . . . 93
4.3 线程块内同步、跨线程块同步与可伸缩性 . . . 96
4.4 线程束、处理块与单指令多线程执行 . . . 99
4.5 控制发散与有效执行通道 . . . 102
4.6 线程束调度与延迟隐藏 . . . 105
4.7 占用率、资源约束与线程块大小 . . . 108

4.8 运行时资源查询与例题 . . . 112
4.9 架构、调度与性能机制的统一关系 . . . 115

第 5 讲 CUDA Memory Model 117
5.1 课程安排与核心内容 . . . 117
5.2 从处理器执行到 CUDA 存储层次 . . . 118
5.3 基本矩阵乘法：从公式到线程映射 . . . 124
5.4 访存量、算术强度与 Roofline 性能分析 . . . 130
5.5 例题解析与知识归纳 . . . 136

第 6 讲 Data Locality and Tiled Matrix Multiply 140
6.1 课程事项与核心内容 . . . 140
6.2 数据局部性与分块矩阵乘法 . . . 142
6.3 线程块同步与正确执行顺序 . . . 148
6.4 边界条件与一般尺寸矩阵 . . . 151
6.5 分块收益与资源约束 . . . 158
6.6 共享内存接口与分块思想的推广 . . . 162
6.7 关键结论与公式汇总 . . . 165

第 7 讲 DRAM Bandwidth and other Performance Considerations 167
7.1 课程安排与核心内容 . . . 167
7.2 DRAM 组织与带宽 . . . 168
7.3 Warp 访存与共享内存分块 . . . 172
7.4 线程粒度、指令调度与同步 . . . 179
7.5 优化权衡与例题 . . . 187

第 8 讲 Convolution Concept; Constant Cache 194
8.1 课程安排与核心内容 . . . 194
8.2 卷积的计算结构与边界处理 . . . 195
8.3 缓存层次与常量存储器 . . . 200
8.4 分块的数据复用与三种策略 . . . 204
8.5 分块核函数与索引推导 . . . 207
8.6 综合算例与关键结论 . . . 212

第 9 讲 2D Tiled Convolution Kernel; Reuse Analysis 215
9.1 核心概念与分块策略 . . . 215
9.2 二维分块卷积的线程组织与实现 . . . 218
9.3 数据复用与全局访存计数 . . . 223
9.4 带宽、算术强度与计算上限 . . . 228
9.5 分块规模与存储层次 . . . 231
9.6 例题与关键结论 . . . 234

第 10 讲 Introduction to ML; Inference and Training in DNNs **237**
10.1 机器学习与表示学习 . . . 237
10.2 从感知机到多层网络 . . . 242
10.3 前向计算与分类输出 . . . 247
10.4 损失函数、反向传播与参数更新 . . . 251
10.5 训练组织与并行计算 . . . 258
10.6 泛化、模型容量与任务适配 . . . 261
10.7 关键公式与结构关系 . . . 264

第 11 讲 Computation in Convolutional Neural Networks **265**
11.1 卷积网络的结构与局部计算 . . . 265
11.2 卷积与池化的前向计算 . . . 268
11.3 直接卷积的并行实现与数据复用 . . . 273
11.4 卷积的矩阵表示与输入展开 . . . 282
11.5 工程规模与综合例题 . . . 290

第 12 讲 Computation in Large Language Models **294**
12.1 自回归生成与模型输入 . . . 294
12.2 因果多头注意力与层内计算 . . . 297
12.3 注意力的 CUDA 实现 . . . 301
12.4 两阶段推理与键值缓存 . . . 307
12.5 计算规模与端到端性能 . . . 312

第 13 讲 Atomic Operations and Histogramming **317**
13.1 并发更新与原子操作 . . . 317
13.2 直方图与并行数据划分 . . . 321
13.3 原子操作的延迟与吞吐 . . . 324
13.4 线程块私有化与共享内存 . . . 326
13.5 线程粗化与性能权衡 . . . 331
13.6 例题与完整推导 . . . 335

第 14 讲 Profiling on Nvidia GPU **338**
14.1 性能分析与 GPU 硬件 . . . 338
14.2 矩阵乘法基准与可信测量 . . . 341
14.3 系统时间线与内核指标采集 . . . 348
14.4 吞吐率、屋顶线与存储流量 . . . 352
14.5 线程束调度、占用率与指令定位 . . . 357
14.6 矩阵乘法的比较与优化 . . . 361

第 15 讲 Parallel Computation Patterns: Reduction Trees **367**
15.1 归约的运算结构与并行代价 . . . 367
15.2 分块归约与内存访问 . . . 370
15.3 线程束内通信与两阶段归约 . . . 375
15.4 线程粗化与完整归约实现 . . . 381
15.5 综合算例与关键结论 . . . 386

第 1 讲

Introduction

1.1 课程框架与技术背景

内容概览

课程运行框架包括课程目标、资源平台、作业流程、评分方式、计算环境以及学术诚信要求。单核频率不再持续快速增长之后，现代计算系统转向**并行性**(*parallelism*)、**异构性**(*heterogeneity*)与**处理器专用化**(*processor specialization*)，图形处理器也成为这一转向中的核心加速器。

相关硬件趋势与软件责任构成以下因果链：

晶体管密度持续增加 + 频率缩放趋于停滞
$\Longrightarrow$ 更多晶体管无法自动转化为更快的单线程执行
$\Longrightarrow$ 多核、向量化与专用加速器成为主要性能路径
$\Longrightarrow$ 软件必须显式暴露并行性，并控制负载、访存与串行瓶颈.

1.1.1 课程的技术定位

课程名称为应用并行编程，核心对象是**大规模并行处理器**(*massively parallel processor*)，主要以支持统一计算设备架构的英伟达图形处理器为实验平台。课程并不把处理器结构本身作为最终目标；架构知识只学习到足以解释程序性能的深度。换言之，处理器结构、编程接口、并行算法模式和性能测量在本课程中始终服务于同一个目标：把一个具有足够并行性的计算映射到实际硬件上，并使映射后的程序获得可解释、可复现且可扩展的性能。

课程目标包含以下五类技术内容：

- 并行编程的基本模型与术语；
- 常见并行算法的原则与模式；

- 编程接口、工具链与实现技巧；
- 影响性能的处理器结构特征与约束；
- 以人工智能负载为代表的高价值应用。

关键结论

并行程序的质量不能只用“能否运行”衡量。一个合格的并行实现至少同时满足四项要求：结果正确、性能足够高、代码可维护、算法与实现能适应后续硬件代际。仅通过增加线程数得到一次性的速度提升，并不等于获得了可扩展的并行程序。

1.1.2 课程形成背景与计算平台脉络

美国国家超级计算应用中心的图形处理器系统建设构成了本课程的计算平台背景。课程教师长期从事高性能计算、计算加速器以及机器学习系统研究。三代具有代表性的系统分别是：2008 年建设的加速集群使用较早期的图形处理器；2018 年建设的面向人工智能的集群使用 V100 图形处理器；2024 年建设的 DeltaAI 系统采用基于 GH200 超级芯片的 Grace–Hopper 节点。课程实验实际使用较早的 Delta 超级计算机，而不是 DeltaAI。

图形处理器属于更大的系统演化趋势：高性能计算系统逐步从以通用中央处理器为绝对中心的结构，转向由中央处理器负责控制与串行工作、由一个或多个加速器承担大规模并行计算的异构结构。

1.1.3 课程规则与技术导论的逻辑关系

课程运行规则与技术导论看似是两组独立内容，实际存在直接联系：

- 课程要求在 Delta 上编译并通过作业调度系统运行，是因为真实超级计算机由大量用户共享，计算资源必须被隔离和调度。
- 实验强调自己编写代码，是因为期中考试要求在纸面上完成与实验相近的程序片段；依赖自动代码生成无法形成独立实现能力。
- 项目同时评价功能与性能，是因为并行程序的价值来自正确性与性能的共同满足。
- 硬件代际快速变化，因此并行程序需要具备可移植性、可扩展性和可维护性，不能只适用于某一张图形处理器。

评估并行化方法时需要同时回答两个问题：第一，该方法为什么能提高吞吐量；第二，该方法在真实共享系统、未来硬件和完整应用中是否仍然成立。

1.2 课程运行框架、作业流程与学术规范

1.2.1 授课形式与支持体系

课程以线下授课为主，课堂同时录制并上传至学校媒体平台。实验又称**机器问题**(*Machine Problem, MP*)，项目由学生在课外独立或按指定团队形式完成；两次期中考试均在线下进行。助教团队分别承担实验、项目和答疑工作，办公时间从第二周开始，其时间表由课程管理平台公布。

课程支持体系由多个平台共同构成。每个平台只承担一部分功能，因此应按任务类型选择入口。

平台	主要用途	使用要点
课程管理平台(*Canvas*)	教学大纲、课件、录像链接、课程文件、项目报告与成绩	课件通常在上课前上传；最终成绩在此汇总。打开课程页面。
学校媒体平台(*Illinois Media Space*)	课堂录像	录像发布到课程专用频道。打开录像频道。
代码托管平台(*GitHub*)	实验与项目的基础代码、开发仓库和最终代码提交	先同步发布代码，再在 Delta 上编辑、编译和运行，最后把待评分版本提交到仓库。打开发布仓库。
在线评测平台(*PrairieLearn*)	实验配套测验与概念问题	应在代码部分完成之后作答，因为部分问题依赖实际运行结果。
课程讨论平台(*Campuswire*)	公告、学生问答、教师与同学共同讨论	面向班级显示时可采用匿名形式；不能把它当作提交受保护代码的渠道。打开讨论区。
课程人工智能问答系统(*course AI chatbot*)	课程规则、统一计算设备架构概念、接口、编译与运行错误等问题	推荐先询问该系统；若结果不充分，再使用讨论区或办公时间。链接以课程管理平台的当期入口为准。

1.2.2 教材与技术文档

首选教材为 Wen-mei Hwu、David Kirk 与 Izzat El Hajj 编写的《大规模并行处理器编程：实践方法》(*Programming Massively Parallel Processors: A Hands-on Approach*) 第五版，2026 年出版。第四版于 2022 年出版，也被课程接受，并可通过学校订阅在线阅读。第三版及更早版本由于内容老化，不建议使用。

统一计算设备架构的主要文档包括：

- 《统一计算设备架构 C 编程指南》(*CUDA C Programming Guide*)；
- 《统一计算设备架构 C++ 最佳实践指南》(*CUDA C++ Best Practices Guide*)。

Delta 环境安装 CUDA 13.2，文档参考涉及 13.2 与 13.3 两个版本。阅读文档时需核对文档版本与安装版本；实际使用时应以课程管理平台的当期公告为准。

1.2.3 典型周节奏与提交时间

前三周的示例安排如下：

表 1.2 前三周典型节奏

周次	课堂内容	课外任务与截止项
第 1 周	周二：第 1 讲 “导论”；周四：第 2 讲 “统一计算设备架构导论”	配置代码仓库与计算机访问权限。
第 2 周	周二：数据并行模型；周四：统一计算设备架构内存模型	实验 0：设备查询示例。
第 3 周	周二：继续讲解内存模型；周四：性能考虑	实验 1：向量加法示例。

部分周次会用项目检查点替代实验提交；项目被拆分为三个检查点，并采用逐步完成的交付方式。

所有实验和项目里程碑的正式截止时间均为**美国中部时间周五晚 8:00**。课程同时提供三天默认宽限期：在周一晚 8:00 之前提交仍可获得满分，且不需要提前通知。宽限期的制度含义必须准确理解：

- 正式截止时间仍是周五晚 8:00，周一不是新的常规截止时间；
- 如需超过三天的额外延长，必须在周五晚 8:00 之前给负责该作业的助教发邮件；

- 周五晚 8:00 之后提出的额外延长请求不予考虑，是否批准由课程团队根据特殊情况决定；
- 周一晚 8:00 时仓库中的代码被视为最终版本。即使较早提交过正确版本，评分也会取该时刻检出的仓库状态。

需要特别注意

共享超级计算机可能因系统维护或队列拥堵而延迟作业启动。宽限期为这类不可控情况提供缓冲，但它不能替代提前开始。若大量学生在周五集中提交，等待计算节点本身就可能消耗数小时。

1.2.4 评分结构与考核方式

课程总评由三部分组成：

表 1.3 总评构成

组成	权重	具体规则
期中考试	50%	两次期中考试各占 25%；第一次约覆盖前 11 讲，第二次约覆盖其余讲次。课程没有期末考试；期中考试为三小时纸笔考试，并要求手写与实验相近的代码。
实验	25%	自动数据集通过情况约占实验成绩的 90%，同时考查配套问题的正确性；最低的一个计分实验成绩被舍弃；实验 0 和实验 1 不计入总评。实验数量与各项权重以课程团队发布的详细量规为准。
项目	25%	演示、功能与编码风格约占项目分数的 50%；在完整功能成立的前提下，性能约占另外 50%。详细量规将在之后公布。

项目形式包括两种：一种围绕卷积神经网络若干层的实现；另一种由两至三人组队，实现大语言模型或变换器中的若干层。无论选择哪类项目，功能完整性都先于性能评分。一个输出错误但运行很快的程序不构成高性能实现，因为其计算语义已经改变。

纸笔考试与实验之间形成直接约束。实验中出现的程序结构可能在考试中以手写代码形式再次出现，因此完成实验的目标包含两层：一是获得正确输出，二是建立不依赖代码生成器或他人实现的独立编程能力。

1.2.5 实验的端到端工作流

实验采用下列端到端流程：

1. 从课程发布仓库取得基础代码，并按说明把发布内容合并到个人仓库；
2. 在 Delta 的登录节点上编辑与编译代码；
3. 通过作业调度系统把程序提交到计算节点执行，不能在登录节点上直接运行重计算任务；
4. 根据正确性测试与运行结果继续调试，直到代码满足作业要求；
5. 将最终版本提交到代码托管仓库，供评分流程检出；
6. 代码部分完成后，再回答在线评测平台中的配套问题。

最后一步的顺序不是形式要求。实验问题往往询问实际输出、性能现象或实现细节；在程序尚未完成时作答，会把推测当成测量结果，也更容易忽略题目中隐含的前提。

1.2.6 Delta 超级计算机与作业调度

Delta 是由美国国家超级计算应用中心运行、通过高等科学与工程研究计算环境分配资源的超级计算机。课程使用其中配备 A40 图形处理器的子系统。四卡计算节点规格如下。

NCSA Delta Supercomputer

- Delta is a dedicated, ACCESS-allocated resource designed by HPE and NCSA, delivering a highly capable GPU-focused compute environment for GPU and CPU workloads.



4-Way **A40 GPU Compute Node** Specs	
Specification	**Value**
Number of nodes	100
GPU	NVIDIA A40
GPUs per node	4
GPU Memory (GB)	48 DDR6
CPU	AMD Milan
CPU sockets per node	1
Cores per socket	64
Cores per node	64
Clock rate (GHz)	~ 2.45
RAM (GB)	256

ELECTRICAL & COMPUTER ENGINEERING GRAINGER ENGINEERING

图 1.1 Delta 四卡 A40 子系统

表 1.4 四卡 A40 计算节点规格

项目	数值
节点数量	100
图形处理器型号	英伟达 A40
每节点图形处理器数量	4
每卡显存	48 GB
中央处理器	AMD Milan
每节点中央处理器插槽数	1
每插槽核心数	64
每节点核心总数	64
近似时钟频率	2.45 GHz
主存容量	256 GB

Delta 是多用户共享系统。用户首先连接到**登录节点** (*login node*) 或**头节点** (*head node*)，在这里管理文件、编辑代码和编译程序。实际计算通过**作业调度系统** (*job scheduler*) Slurm 提交到**计算节点** (*compute node*)。调度器根据请求的资源、队列状态与系统策略决定作业何时启动。

该共享系统具有以下运行特性：

- 用户可能登录到多个登录节点中的任意一个；这些节点共享文件系统与调度队列，因此登录节点编号变化不影响文件可见性。

- 系统繁忙时，作业可能等待数分钟、数小时，项目级大任务甚至可能等待更久。
- 课程作业通常请求一张图形处理器，而一个计算节点包含四张 A40，因此同一节点上最多可同时存在若干用户作业。
- 调度器对分配给不同作业的图形处理器进行隔离；一个作业不能使用分配给另一个作业的设备。
- 登录节点面向许多并发用户，不能把它当作绕过调度器的计算节点。

关键结论

在共享超级计算环境中，“程序运行时间”与“从提交到拿到结果的墙钟时间”不是同一个量。前者是作业实际执行时间，后者还包含排队等待。课程截止时间约束的是完整工作流，因此排队风险必须通过提前提交来管理。

1.2.7 Delta 账户与登录步骤

账户建立需要经过高等科学与工程研究计算环境和 Delta 两层系统，不能在截止日前临时完成。具体步骤如下：

1. 在账户注册页面创建用户；已有账户者继续使用原用户名，不应重复注册。
2. 新建账户时把“伊利诺伊大学厄巴纳-香槟分校”选作**身份提供方** (*Identity Provider*)，以学校凭据完成认证。
3. 记录由系统分配的用户标识，并通过课程表单提交给课程团队。
4. 等待课程团队把用户加入课程资源组，再等待 Delta 端完成账户创建。典型等待时间为 24–48 小时，完整传播过程可能更长。
5. 收到确认邮件后，通过安全外壳协议连接：

```
ssh yourusername@login.delta.ncsa.illinois.edu
```

6. 登录需要 Duo 双因素认证；随后克隆课程仓库并按仓库说明配置环境。

1.2.8 课程人工智能问答系统的技术机制

课程问答系统采用**检索增强生成**(*Retrieval-Augmented Generation, RAG*)。它不只依赖语言模型内部参数回答问题，而是先从课程材料中检索相关片段，再把这些片段作为上下文交给语言模型。

建模说明

其基本数据流可表示为

$$\begin{aligned}&\text{课程文档} \longrightarrow \text{分段} \longrightarrow \text{嵌入向量} \longrightarrow \text{向量数据库},\\&\text{学生问题} \longrightarrow \text{问题嵌入} \longrightarrow \text{相似片段检索},\\&\qquad\longrightarrow \text{上下文拼接} \longrightarrow \text{大语言模型} \longrightarrow \text{回答与来源指向}.\end{aligned}$$

其中，嵌入向量 (embedding) 用数值向量表示文本语义，向量数据库 (vector database) 用于相似性检索，大语言模型 (large language model) 根据检索上下文生成回答。向提示词中放入大量课程上下文称为**上下文填充**(*context stuffing*) 或**超长提示构造**(*mega-prompting*)。系统可通过应用程序编程接口调用模型，也可使用本地运行的开源模型。

检索增强只能提高回答与课程材料的一致性，不能保证每个答案都正确。系统可能遗漏题目隐含假设、检索到不完整片段，或在生成过程中引入错误。课程明确规定：学生对最终代码与答案承担责任，不能把“问答系统给出了该答案”作为重新评分的依据。

1.2.9 允许与禁止的协作和人工智能使用

课程允许概念层面的讨论，也鼓励学生利用问答系统学习接口与技术；同时严格禁止把实现责任转移给他人或人工智能工具。边界可按下表理解。

允许	禁止
与同学讨论思路、算法方向、所用函数以及调试策略；接受修过课程者的口头建议。	查阅往年作业实现或网络上发布的本课程作业答案。
询问统一计算设备架构接口的含义，例如内存复制函数的作用、内核函数的调用方式。	要求人工智能系统直接生成某个实验或项目的实现代码。
询问编程技术，例如共享内存数组如何声明、运行时如何指定动态共享内存大小。	复制超过一行左右的非平凡代码；让他人提供代码同样会受到处罚。

允许	禁止
询问并行编程概念与性能优化原理，例如分块为什么能改善全局内存带宽。	阅读他人代码或人工智能生成的代码后，再离开原文“独立重写”；课程把这种行为也视为复制。
询问编译错误、运行时错误的含义，并学习系统化调试方法。	把完整程序粘贴给问答系统，让系统代替自己定位并修复错误。
进行期中复习与一般课程规则查询。	把实验测验题直接粘贴给问答系统，或在考试中给予、接受帮助。
逐行理解并按当前问题改写课堂明确提供的代码示例。	故意规避实验要求，或只做表面变量改名、语句重排来掩盖复制。

“非平凡代码”通常指超过一行左右的实现。代码相似性检测不只比较变量名，也能识别重排、改名等表面变化。允许引用课堂示例的前提是逐行理解并针对当前程序进行实质性改写，而不是整段搬运。

学术不端的后果为：第一次在相应作业或考试上记零分，并通过学校的 FAIR 系统向学院报告；重复违规将导致课程自动不及格。提供代码者与接收代码者都可能受到相同处罚，因为相似性检测并不能可靠判断谁最先写出该实现。

需要特别注意

课程对人工智能工具的限制不是“完全禁止使用”。允许用途集中在概念、接口、错误解释与复习；禁止用途集中在替代学生完成受评分实现。判定边界时应问：该工具是在帮助理解问题，还是在产生本应由学生独立完成的答案或代码？

1.3 从摩尔定律时代到后登纳德时代

1.3.1 两条经常被混淆的缩放规律

现代并行计算的硬件动因必须从两条历史趋势区分开来。

表 1.6 摩尔定律与登纳德缩放的作用区分

概念	核心含义	对性能的历史影响
摩尔定律(*Moore's Law*)	制造特征尺寸不断缩小，使单位面积可容纳的晶体管数量约每 18–24 个月翻倍。它是经验性“定律”，也可视为产业目标。	提供更高的元件密度，使芯片能够容纳更多功能单元、缓存和控制逻辑。它本身并不等价于单线程性能翻倍。
登纳德缩放(*Dennard Scaling*)	随器件尺寸缩小，晶体管能够在更高时钟频率下工作，是推动频率上升的物理基础。	使单个处理器核心能够更快执行同一指令流，是早期“相同代码自动变快”的重要来源。

大众叙述常把处理器性能增长统称为“摩尔定律”，但历史上的快速性能增长实际来自摩尔定律与登纳德缩放的共同作用。晶体管数量增加提供了更复杂的处理器结构，时钟频率提高则直接缩短许多指令的执行时间。两者叠加后，通用计算机在约三十年内表现出大致每 18 个月翻倍的性能趋势。

1.3.2 “自动性能增长”对软件的影响

在这一时期，软件常能在不改变算法与并行结构的情况下获得显著加速。一个今天运行过慢的串行程序，经过一到两个处理器代际后，只需重新编译或迁移到新机器，就可能明显变快。由此形成了三项长期软件预期。

1.3.2.1 同一份软件随硬件代际自动加速

程序不需要显式使用更多线程，也能受益于更高频率、更好的流水线与更强的单核结构。在这种历史环境中，“代码不够快，等待两年”一度可以成立；它不是现代系统中的可靠开发策略。

某些程序较早暴露了这一策略的边界。若程序主要受**内存访问延迟**(*memory-access latency*)限制，例如部分稀疏、非结构化或图算法，处理器算术速度提高并不会按相同比例缩短运行时间。此类程序需要的数据访问不规则，缓存难以持续命中，因此内存系统可能先成为瓶颈。

1.3.2.2 内存从稀缺资源变为多数应用中的充足资源

芯片密度增加也扩大了可用内存容量。早期程序常需要显式把数据分块，在磁盘与主存之间来回搬运；随着系统能够提供数百吉字节乃至更大容量，许多应用可以把完整数据集常驻内存。这一变化直接改变了算法设计：过去为节省内存而使用的复杂外存策略，在新的容量条件下可能不再必要。

1.3.2.3 软件可移植性成为默认期待

如果相同程序可以在多个处理器代际上继续运行并自动加速，那么软件稳定性和**可移植性**(*portability*)自然具有很高价值。应用代码可以长期演进，而不必每次为了新硬件彻底重写。可扩展且可移植的软件能够跨越多个硬件代际；现代并行软件必须通过更主动的设计才能维持这种寿命。

1.3.3 专用计算机在早期为何难以长期保持优势

专用处理器可以针对某类计算取得初始性能优势，但在通用处理器性能约每 18 个月快速增长的时期，该优势可能在数年内被下一代通用微处理器追平。专用芯片的设计、制造与软件生态成本很高，如果其目标市场不足以支撑持续迭代，就难以与通用处理器的规模经济竞争。

这种现象称为“杀手级微处理器的冲击”(*Attack of the Killer Micros*)：许多试图领先于摩尔定律与登纳德缩放组合速度的计算机公司，最终被快速演化的通用微处理器压缩了生存空间。图形处理器之所以比许多其他专用加速器更容易形成长期生态，也与这一背景有关。

1.3.4 轻松性能增长的结束

约在 2005–2006 年，登纳德频率缩放停止延续原有趋势。过去可以忽略的**漏电流**(*leakage current*)变得重要，功耗与散热约束使时钟频率无法继续按历史速度增加。处理器频率大体停留在约 2.5 GHz to 3.5 GHz 的范围；近二十年后，频率数量级仍没有发生根本变化。

与此同时，晶体管数量仍在增加，尽管摩尔定律本身也在放缓。于是出现了新的设计矛盾：芯片可以容纳更多晶体管，却不能简单地把它们全部用于提高一个核心的时钟频率。图 1.2 中，晶体管数量继续上升，而频率曲线在 2000 年代中期后明显趋平；逻辑核心数量随后开始增长。

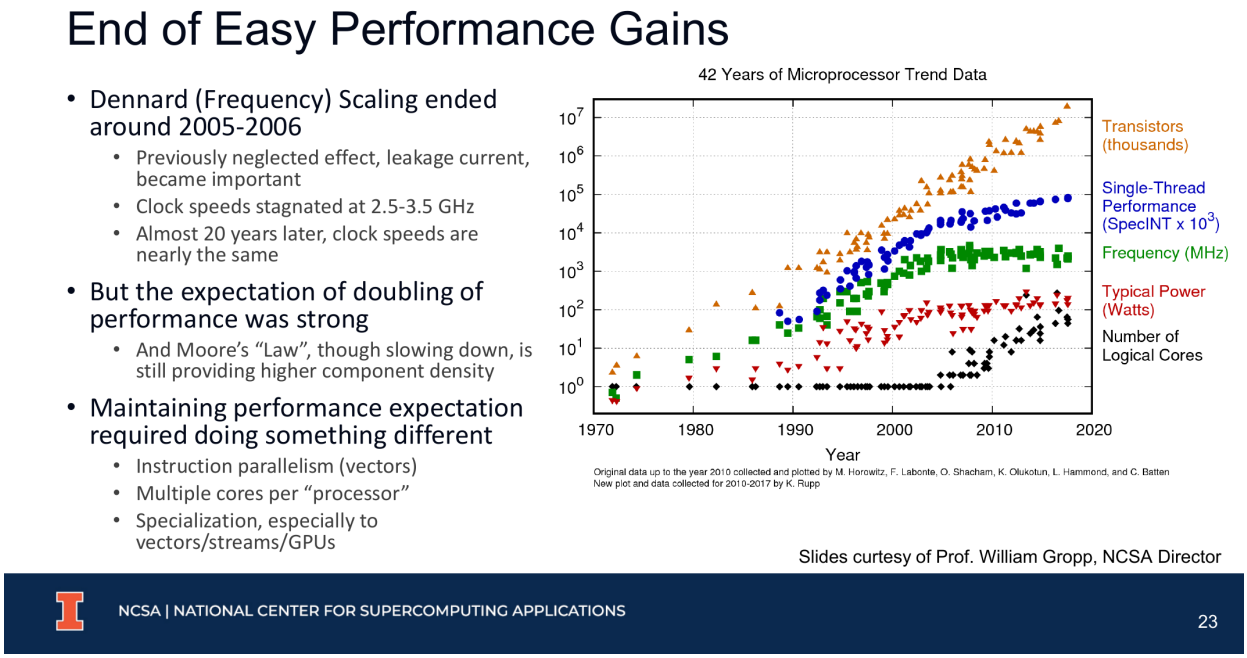


图 1.2 单纯依靠频率获得性能增长的时代结束

关键结论

登纳德缩放结束后的关键变化是：硬件仍可能提供更多计算资源，但串行程序不会自动使用这些资源。新增晶体管被组织为更多核心、更多向量执行单元或专用加速器之后，软件必须暴露与之匹配的并行工作，才能把硬件规模转化为应用性能。

1.3.5 后登纳德时代的三条主要性能路径

维持性能增长主要有三种方向。

1.3.5.1 指令级与向量并行性

向量执行（*vector execution*）让一条指令对多个数据元素执行相同运算。其基本思想是把规则、重复的数据运算打包，使同一控制流驱动多个算术通道。后续图形处理器设计继承并扩大了这种“相同操作作用于不同数据”的思想。

1.3.5.2 多核处理器

当单个核心无法继续大幅提高频率时，可以在同一处理器中放置多个核心。每个核心执行自己的指令流，共享部分缓存、内存与互连资源。多核设计提高了芯片可并行执行的工作量，但单线程程序通常仍只使用一个核心，因此程序需要被改写为多线程或多进程形式。

现代处理器可包含上百个核心。即使硬件核心数从 1 增长到 100，一个只含单一串行指令流的程序也不会自动获得 100 倍加速。核心数量是可用资源，不是无条件性能保证。

1.3.5.3 处理器专用化

通用中央处理器追求“几乎所有代码都能运行，并在多数工作负载上获得尚可到优秀的表现”。专用处理器则主动放弃一部分通用性，把更多芯片面积用于某类操作或数据流。处理器专用化包括以下方向：

- 使用更简单的轻量级核心并增加核心数量，例如高性能计算中的蓝色基因系统与至强融核；
- 针对向量、流式数据或图形运算构建专用结构；
- 在移动芯片中同时配置中央处理器、图形处理器和**神经网络处理单元**（*Neural Processing Unit, NPU*），分别承担通用控制、图形与特定机器学习运算；
- 在超级计算机中由通用中央处理器协调，由图形处理器承担大规模数值计算。

1.3.6 并行性与异构性的共同转向

异构计算（*heterogeneous computing*）是指同一系统包含多种结构不同、性能特征不同的处理器，并把应用的不同部分映射到最合适的设备。它与并行性相互依赖：专用加速器往往通过大量并行执行单元取得吞吐量优势，而应用必须提供足够规则的并行任务才能利用这些单元。

移动芯片 Apple A14 Bionic 与橡树岭国家实验室 Frontier 百亿亿次级系统都采用加速器架构，说明这种结构从最小规模的移动设备一直延伸到最大规模的超级计算机。

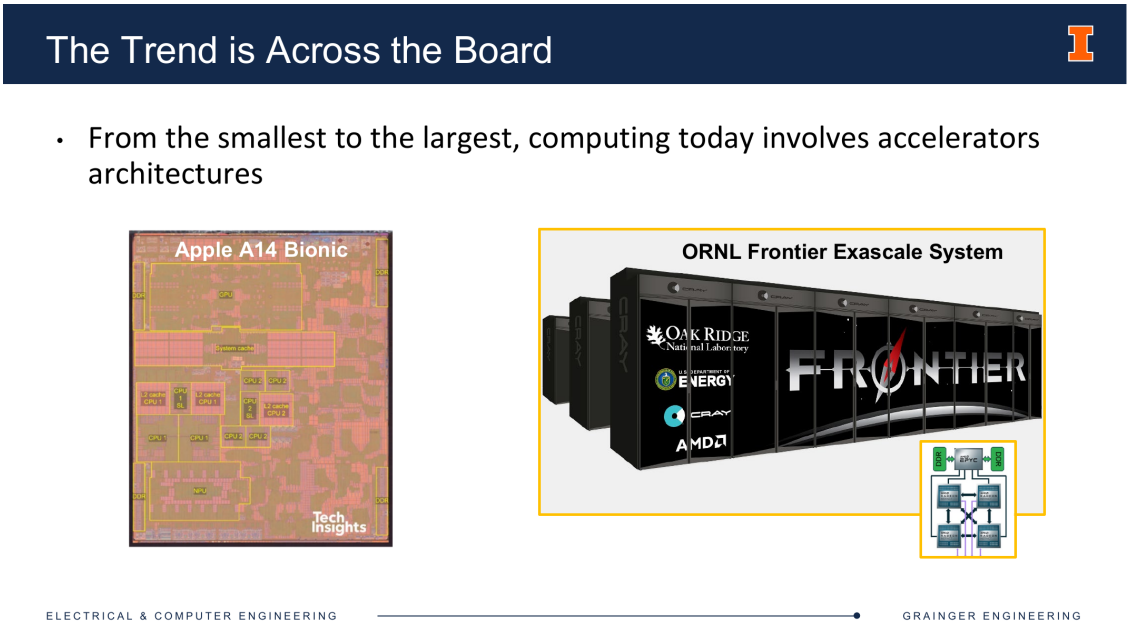


图 1.3 加速器架构贯穿移动设备与超级计算机

这一转向改变了软件责任分配。在早期“自动性能增长”环境中，硬件升级可以替软件完成大量加速工作；在后登纳德时代，硬件只提供并行资源与专用能力，程序员必须识别可并行部分、选择可扩展算法、组织数据并消除瓶颈。

1.4 中央处理器与图形处理器的设计取向

1.4.1 先区分延迟与吞吐量

比较中央处理器和图形处理器之前，必须先区分两个性能指标。

- **延迟**（*latency*）描述单个任务从开始到完成所经历的时间。若任务在时刻 t_{start} 开始、在 t_{finish} 完成，则

$$L = t_{\text{finish}} - t_{\text{start}}.$$

  延迟越低，单个请求越快得到结果。

- **吞吐量**（*throughput*）描述单位时间内完成的任务数量。若时间区间 Δt 内完成 N 个任务，则平均吞吐量为

$$R = \frac{N}{\Delta t}.$$

  吞吐量越高，系统在一段时间内处理的总工作越多。

低延迟与高吞吐量并不等价。一个流水线可以让单个任务经过多个阶段，因此单任务延迟较长；当流水线填满后，各阶段同时处理不同任务，系统仍可在每个周期完成一个或多个结果，从而获得高吞吐量。

建模说明

设一条理想流水线包含 d 个阶段，时钟频率为 f。单个任务的流水线延迟约为 d/f，但流水线达到稳态后，每条执行通道的理想结果产生速率可以接近 f。增加流水线深度可能不降低单任务延迟，却能支持更高的稳态工作速率，从而形成图形处理器“长延迟、高吞吐量”的组合。

1.4.2 中央处理器：面向低延迟的设计

中央处理器采用**面向延迟的设计**(*latency-oriented design*)，主要优化少量指令流的快速完成，其芯片面积通常分配给以下结构。

1.4.2.1 少量但功能强的算术逻辑单元

算术逻辑单元(*Arithmetic Logic Unit, ALU*)执行整数、逻辑及部分数值运算。中央处理器核心包含较少但较复杂的执行单元，目标是降低单条指令或短指令序列的完成时间。复杂的乱序执行与多种执行端口也属于同一低延迟取向。

1.4.2.2 较大的缓存层次

主存访问延迟远高于算术操作。中央处理器投入大量芯片面积构建多级**缓存**(*cache*)，把频繁访问的数据保留在靠近执行单元的位置。缓存命中时，原本较长的主存访问被转换为较短的缓存访问，从而降低单线程停顿。

1.4.2.3 复杂控制逻辑

中央处理器使用复杂控制结构减少依赖与分支带来的等待：

- **分支预测**(*branch prediction*)提前猜测控制流方向，尽量避免处理流水线因等待分支结果而空闲；
- **数据转发**(*data forwarding*)把某条指令刚产生的结果直接送给后续相关指令，减少等待数据写回再读取的时间；

- 适度多线程用于隐藏相对较短的延迟，但线程数量通常远少于图形处理器。

中央处理器还通常采用较高时钟频率。综合这些机制，它非常适合控制密集、分支复杂、串行依赖明显或任务规模较小的程序部分。

1.4.3 图形处理器：面向高吞吐量的设计

图形处理器采用**面向吞吐量的设计**(*throughput-oriented design*)，主要优化大量相似任务的总完成速率。

1.4.3.1 大量较小的算术单元

图形处理器把更多芯片面积用于计算通道，因而能同时保持大量算术操作在执行流水线中。单个操作或单个线程的延迟可能高于中央处理器，但当工作规模足够大时，所有线程的总完成时间可以显著缩短。

1.4.3.2 深流水线与海量线程

图形处理器通过深流水线提高稳态吞吐量，并维护大量可执行线程。当一组线程等待长延迟事件，例如全局内存数据返回时，调度器可以选择另一组已经具备操作数的线程执行。

这个机制称为**延迟隐藏**(*latency hiding*)。它没有缩短那次内存访问本身的物理延迟；它通过让其他工作占用原本会空闲的执行周期，减少延迟对总体吞吐量的影响。若程序没有足够多的独立线程，或所有线程同时等待同一瓶颈，延迟就无法被有效隐藏。

1.4.3.3 相对较小的缓存与较简单的控制

图形处理器也包含缓存，但相对于中央处理器，它把更大比例的面积用于算术单元、寄存器与线程执行资源。控制结构也偏向规则执行，因为最有效的工作负载通常让许多线程对不同数据执行相同或相近的指令序列。其时钟频率通常适中，性能主要来自并行宽度与总吞吐量，而不是极高频率。

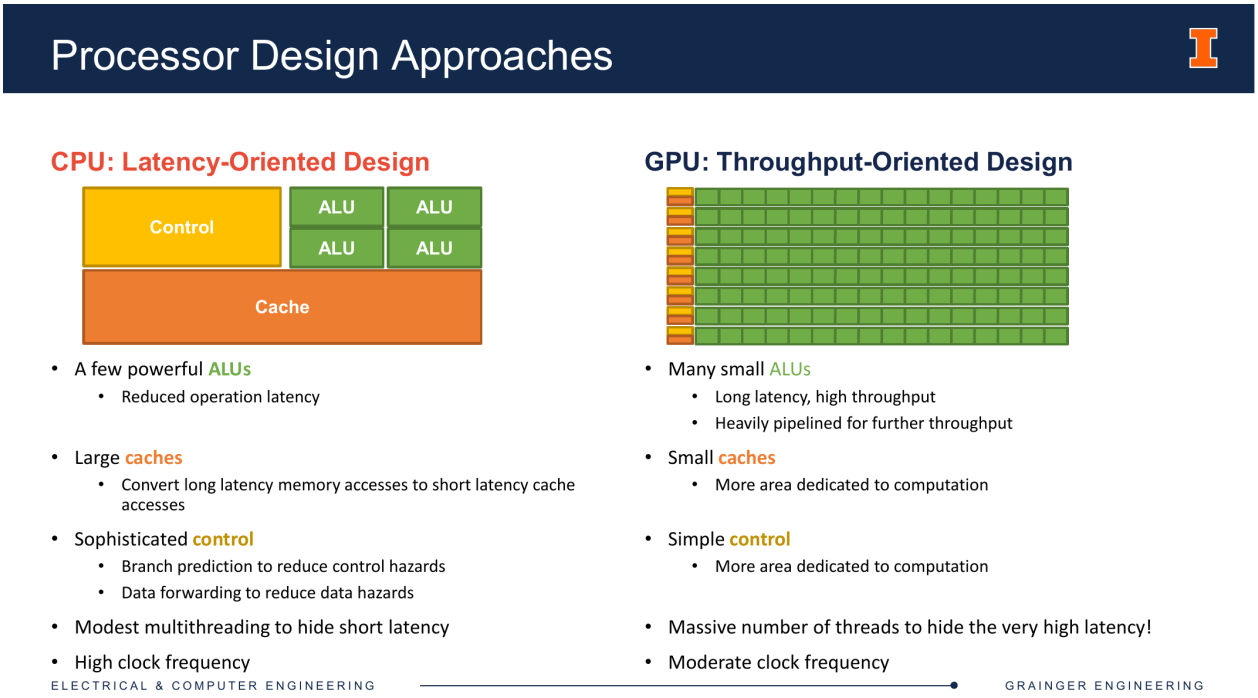


图 1.4 中央处理器的低延迟取向与图形处理器的高吞吐量取向

1.4.4 芯片面积分配反映了不同目标

图 1.4 的方块图不是精确的晶体管布局，而是面积预算的概念图。中央处理器把大量面积用于缓存和复杂控制，以减少少量线程的等待；图形处理器把更多面积复制为大量计算通道，以提高规则并行工作负载的总处理速率。

这种权衡可以从机会成本理解。若增加缓存和控制逻辑，就会减少可放置的算术通道；若增加算术通道，就必须接受较少的单线程优化资源。两类处理器都不是绝对“更快”，它们针对不同目标函数优化。

两张芯片照片可以进一步说明粒度差异：第十代英特尔酷睿处理器 (10th Gen Intel Core processor) 在 14 nm 工艺下包含 10 个中央处理器核心；英伟达 GK110 在 28 nm 工艺下标有 2880 个统一计算设备架构核心。较老工艺的 GK110 仍能容纳数量大得多的“核心”，正是因为这里的统一计算设备架构核心远小于完整中央处理器核心。该对比用于说明结构粒度，不能仅按数字推断两张芯片的通用性能。

表 1.7 中央处理器与图形处理器的高层对比

维度	中央处理器	图形处理器
主要目标	降低单个线程或请求的延迟	提高大批相似任务的总吞吐量
算术资源	少量、复杂、功能强的执行单元	大量、较小、深流水化的执行单元
缓存	较大，用于降低平均访存延迟	相对较小，把更多面积留给计算资源
控制	分支预测、数据转发等复杂机制	偏向规则控制和大规模线程调度
并发方式	少量强线程，适度多线程	海量线程，通过切换隐藏长延迟
典型优势部分	串行依赖、复杂控制、低延迟响应	数据并行、规则数值计算、高吞吐批处理

1.4.5 “核心”一词在两类处理器中并不等价

核心(*core*)在两类处理器中具有不同的结构粒度和语义。

中央处理器核心是相对完整的处理单元，通常包含多个执行单元、寄存器、至少一级私有缓存和复杂控制逻辑。它能够独立推进一个或多个通用指令流。

英伟达所称的**统一计算设备架构核心**(*CUDA core*)则是更细粒度的算术数据通路，可以近似理解为流式处理器中的一个算术执行单元。它不具备中央处理器核心那样完整的缓存与控制结构。因此，“10 个中央处理器核心”与“2880 个统一计算设备架构核心”不能按核心数量直接比较；两者的粒度、能力和组织方式完全不同。

1.4.6 高性能系统通常同时使用中央处理器与图形处理器

异构系统中的任务划分原则是：

- 对延迟敏感、难以并行化的串行部分由中央处理器执行；
- 对吞吐量敏感、含有大量相互独立工作的并行部分由图形处理器执行。

两类处理器在各自擅长的代码上可能相差十倍以上。该数值是概念性数量级说明，不是对任意程序的固定保证。实际加速取决于可并行比例、数据传输、内存访问、分支规则性、问题规模和实现质量。

一个典型异构应用的执行过程是：中央处理器读取与准备数据、决定计算阶段和发起图形处理器任务；图形处理器执行规模最大的并行计算；结果再被中央处理器用于后处理、控制或输出。两种设备之间的协同本身会引入数据移动与同步开销，因此正确的任务划分不仅

要比较计算速度，还要判断把任务迁移到加速器是否足以抵消这些额外成本。

关键结论

中央处理器与图形处理器的根本差异可以概括为：中央处理器用更多硬件资源缩短少数线程的关键路径，图形处理器用更多并行执行资源扩大单位时间内完成的工作总量。图形处理器编程的首要条件不是“有很多算术”，而是“有很多能够同时推进、且执行结构足够规则的算术”。

1.5 图形处理器的起源、统一计算设备架构与应用扩张

1.5.1 图形工作负载天然具有大规模数据并行性

图形处理器最初为图形渲染而设计，因此名称中的“图形”并不是历史偶然。屏幕包含大量像素，许多像素可以在相互独立或弱依赖的条件下执行相近的颜色、几何与纹理计算。把相同计算作用于大量不同数据元素，正符合高吞吐并行硬件的设计取向。

2007 年以前，图形处理器主要通过图形应用程序编程接口使用，例如**开放图形库**（*OpenGL*）与**直接三维图形接口**（*Direct3D*）。研究者若想让图形处理器完成非图形数值计算，必须把问题重新表述为图形流水线能够接受的形式，即把一般计算改写成“为某个像素着色的函数”，再把渲染输出解释为所需数值结果。

这种方法证明了图形处理器具有通用数值计算潜力，但编程过程高度间接，算法表达受图形接口约束。它促使硬件与软件接口共同演化。

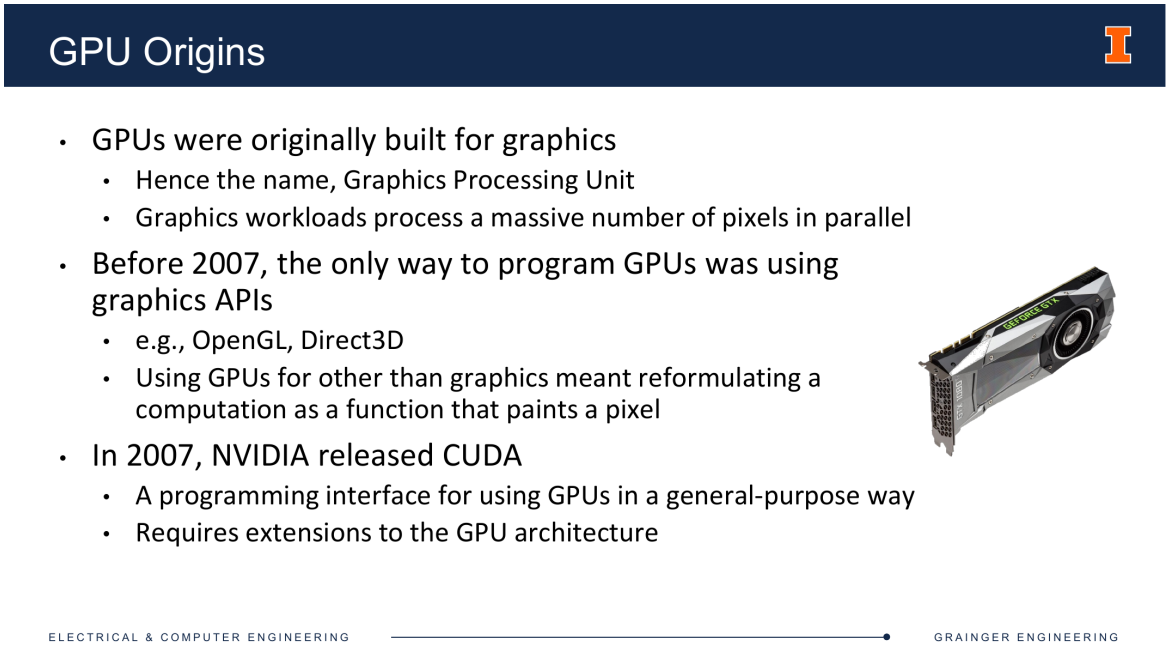


图 1.5 图形处理器从图形接口走向通用计算的历史转折

1.5.2 2007 年的统一计算设备架构转折

2007 年，英伟达发布**统一计算设备架构**（*Compute Unified Device Architecture, CUDA*）。它为图形处理器提供面向通用计算的编程接口，使程序员可以直接表达线程、内核函数、设备内存与数据传输，而不必把数值问题伪装成图形渲染。

这一转折通常称为**通用图形处理器计算**（*General-Purpose Computing on Graphics Processing Units, GPGPU*）。它包含两方面变化：

- 软件层面提供适合 C/C++ 程序员使用的语言扩展、运行时接口与工具链；
- 硬件层面增加或调整支持通用加载、存储、控制与数值计算的结构，使图形处理器不再只适合固定图形流水线。

统一计算设备架构 C/C++ 程序通常同时包含中央处理器代码和图形处理器代码：中央处理器端创建数据、分配设备资源并发起计算；图形处理器端执行由大量线程组成的**内核函数**（*kernel*）。这种主机端与设备端的协作构成统一计算设备架构程序的高层执行模型。

1.5.3 为什么沿用图形处理器，而不是从零设计另一种吞吐处理器

先进芯片的设计、验证、制造与软件生态建设成本极高，需要足够大的销量分摊固定成本。完全新建一种大规模并行处理器，即使技术上可行，也必须同时建立市场、编译器、工具、库和开发者生态。

图形处理器在并行计算进入主流之前已经拥有庞大的游戏与图形市场。这个既有装机基础带来三项先发优势：

1. 规模化销量能够分摊芯片研发与制造成本；
2. 图形工作负载已经推动硬件形成大规模并行执行能力；
3. 既有产品迭代、驱动与开发工具可以逐步扩展到科学计算和人工智能，而不必从零建立完整生态。

因此，图形处理器的成功来自技术结构与市场结构的共同作用。它既具备适合吞吐计算的硬件基础，也拥有其他专用加速器往往缺乏的持续商业规模。

1.5.4 数据中心与人工智能推动通用图形处理器计算扩张

英伟达 2019–2024 年按市场部门划分的收入曲线反映了通用图形处理器计算的扩张。图中数据中心收入在后期快速上升，这一变化与人工智能驱动的训练和推理需求密切相关。

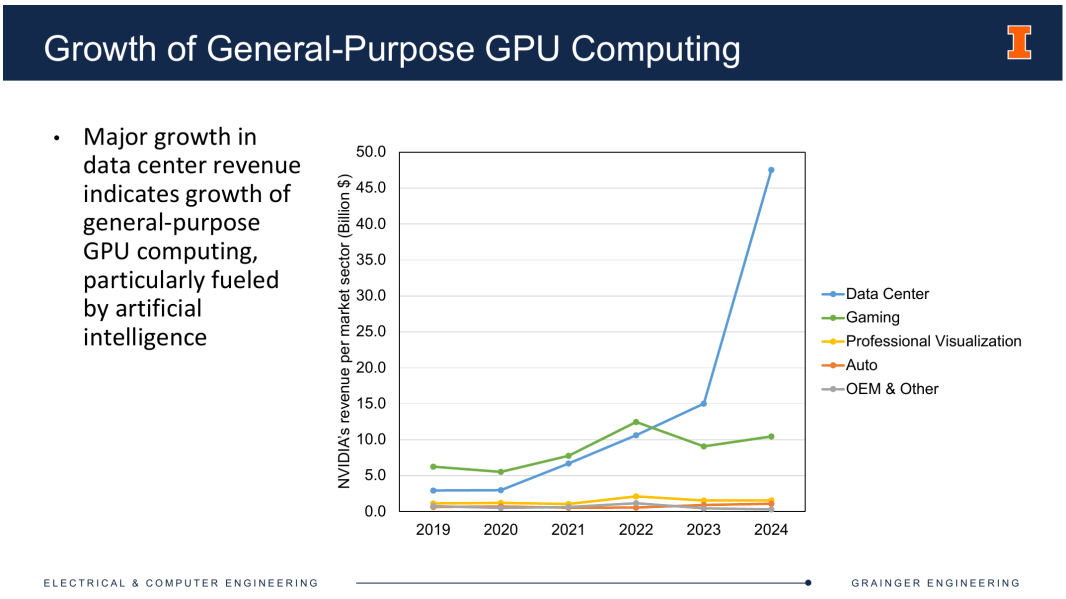


图 1.6 数据中心收入增长与通用图形处理器计算的扩张

该图反映了应用重心的变化：图形处理器仍服务于游戏与可视化，但数据中心、高性能计算和人工智能已成为其重要使用场景。模型训练需要执行大规模矩阵与张量运算；模型推理则把类似计算转化为持续服务需求。两者都可产生规模巨大且结构相对规则的并行工作。

1.5.5 人工智能计算需求的数量级增长

若干机器学习系统所需的浮点运算数量表明，从早期模型到大型语言与多模态模型，计算量跨越了许多数量级，最高约为 10^{24} 次浮点运算。

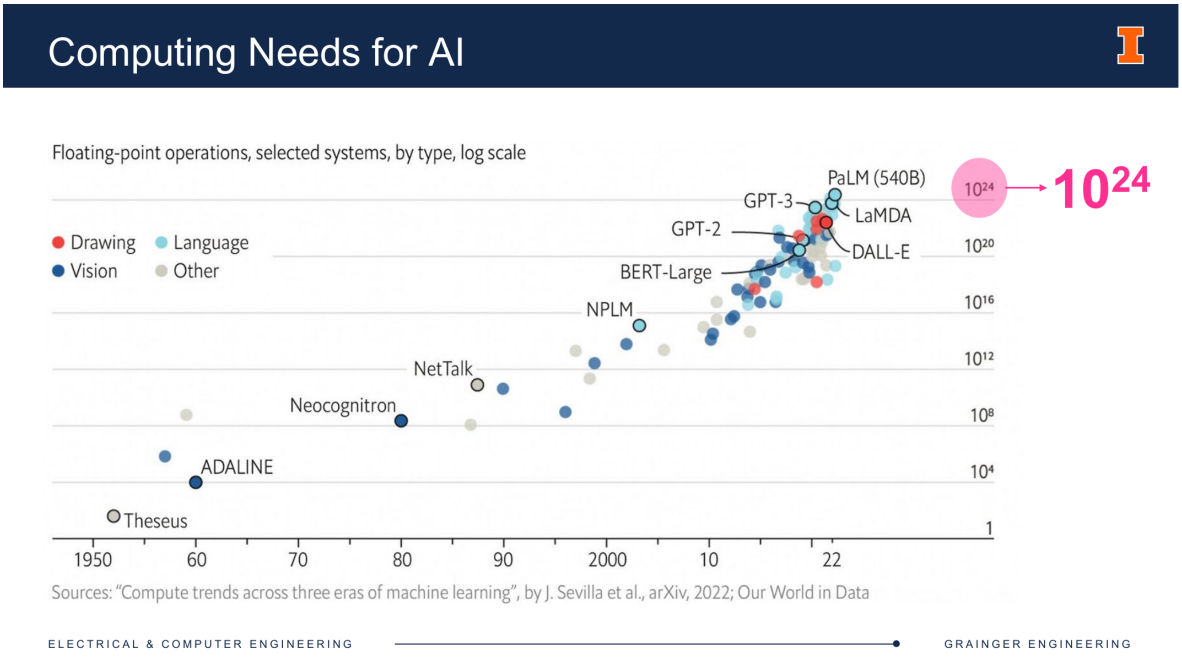


图 1.7 人工智能系统计算需求增长趋势

对数坐标上的纵向小幅移动代表乘法级增长。例如从 10^{16} 增长到 10^{20} 不是增加四个单位，而是增加 10^4 倍。大模型计算需求的这种数量级变化，使单纯依靠单核性能提升不可行，也使高吞吐加速器、多设备并行与大规模系统调度成为必要条件。

1.5.6 超级计算机中的加速器普及

全球超级计算机五百强榜单依据大规模线性代数基准对系统进行排序，通常每半年更新一次，分别对应年中和年末的主要超级计算会议。2026 年 6 月榜单的前五名中，第一名使用自定义多核处理器；第二至第五名分别明确使用 AMD Instinct MI300A、AMD Instinct MI250X、英特尔数据中心图形处理器 Max 和英伟达 GH200 超级芯片。这个样本说明，顶级系统通常依靠大量加速器获得主要浮点计算能力。

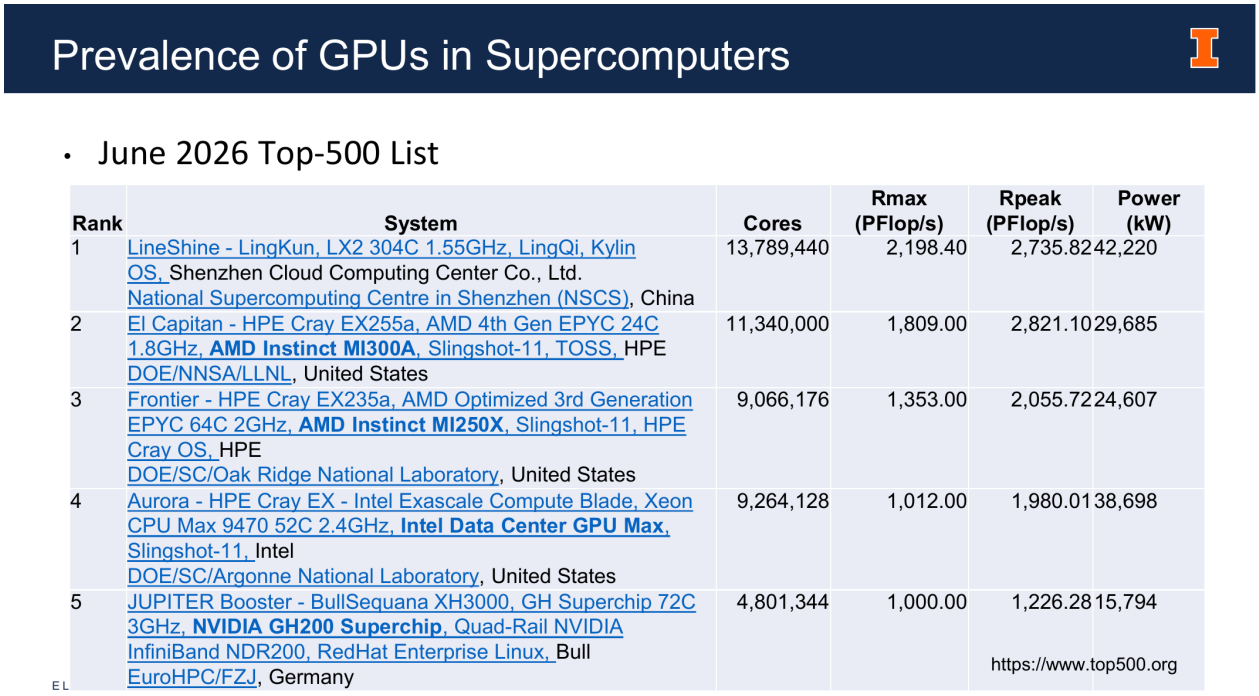


图 1.8 2026 年 6 月全球超级计算机五百强前五名

中央处理器仍负责操作系统、作业管理、控制流和部分串行计算；加速器负责高密度并行数值工作。要让几十年前为串行处理器编写的科学代码利用这些系统，必须重构算法、数据布局和并行分解。硬件峰值只是潜在能力，应用性能来自软件对硬件的有效映射。

1.5.7 并行编程框架之间的关系

统一计算设备架构是并行编程体系中的一种平台；其他常见框架分别提供跨设备编程、指令式卸载、分布式通信或共享内存并行等抽象。

表 1.8 常见并行编程框架

框架	主要目标	与统一计算设备架构的关系
统一计算设备架构	英伟达图形处理器的通用并行编程平台	本课程的主要编程环境。
开放计算语言 (*OpenCL*)	面向多类处理器与加速器的开放并行编程接口	提供另一类图形处理器并行编程框架。
开放加速器编程指令 (*OpenACC*)	通过编译指令把循环与数据区域卸载到加速器	可用于较高层次的加速器编程，本课程不以它为主线。
消息传递接口 (*Message Passing Interface, MPI*)	分布式内存系统中进程之间的显式通信	可与统一计算设备架构组合，在多个计算节点和多张图形处理器之间分配工作。
开放式多处理 (*Open Multi-Processing, OpenMP*)	共享内存系统中的线程级并行与编译指令	可与统一计算设备架构组合，协调单节点内的中央处理器线程及多设备工作。

单张图形处理器只解决一个设备内部的并行执行。若应用要扩展到多个节点，就需要额外机制完成进程间数据交换、任务划分和同步；消息传递接口承担这类分布式协调。若应用在单个共享内存节点内使用多个中央处理器线程或多个加速器，开放式多处理可以参与上层协调。统一计算设备架构、消息传递接口和开放式多处理因此可以处于同一应用的不同层次，而不是相互排斥的替代方案。

关键结论

图形处理器成为通用加速器的完整原因链为：图形渲染提供天然大规模数据并行性；游戏市场提供持续销量与研发投入；统一计算设备架构把硬件能力暴露为通用编程模型；科学计算与人工智能又提供了规模更大的规则并行工作负载。四个条件共同形成了今天的图形处理器计算生态。

1.6 并行化工作流与核心性能挑战

1.6.1 从“程序太慢”到可验证的并行实现

当已有程序运行过慢时，需要找出耗时部分，把适合的部分放到图形处理器上，并持续优化直到性能满足目标。并行化应形成测量驱动的闭环，而不能从“直接重写全部代码”开始。

1.6.1.1 建立正确的串行基线

首先保留一个能够产生正确结果的基线版本，并记录代表性输入上的运行时间。基线提供两种参照：一是验证并行版本是否保持相同计算语义，二是定义加速比的分母。没有可靠基线，后续的“更快”可能来自减少计算、降低精度或产生错误结果。

1.6.1.2 性能剖析并识别热点

使用**性能剖析**(*profiling*)工具测量各函数、循环或阶段的时间占比，找出真正主导总运行时间的**热点**(*hotspot*)。如果某段代码只占总时间的 1%，即使把它加速 100 倍，对整体性能的贡献也很有限；第 1.7 节将用阿姆达尔定律严格表达这一点。

剖析还应区分计算、内存访问、数据传输和同步等待。某一函数“耗时长”不一定表示算术量大，也可能表示它在等待数据或共享资源。

1.6.1.3 判断原算法能否暴露可扩展并行性

有些算法可以直接把独立迭代分配给不同线程；另一些算法包含长依赖链，原始形式不适合并行执行。此时需要采用或重新设计**可扩展算法**(*scalable algorithm*)。可扩展性意味着：问题规模和处理资源增加时，可同时利用的有用工作也能增加，而串行协调、通信与资源争用不会迅速吞噬收益。

因此，并行化可能只是实现层面的映射，也可能要求算法层面的替换。若现有算法不适合并行执行，程序员需要选择结构完全不同的算法。

1.6.1.4 映射工作并优化数据布局

把工作划分为大量线程后，还要决定数据如何存放、何时移动以及哪些线程访问哪些数据，其目标是优化数据排列以最大化局部性。**局部性**(*locality*)表示程序在较短时间内重复访问相近或相关数据的倾向。良好的数据布局可以减少高延迟访问、提高缓存或片上存储的复用，并让大量线程更有效地使用全局内存带宽。

1.6.1.5 测量驱动的性能调优

性能调优(*performance tuning*)是迭代过程：修改实现、测量、解释瓶颈，再决定下一项修改。每次优化必须同时检查正确性和性能，不能仅依据代码看起来“更并行”就断言加速成立。

完整工作流可概括为：

$$\text{正确基线} \rightarrow \text{性能剖析} \rightarrow \text{热点与并行性分析} \rightarrow \text{算法或实现重构},$$
$$\rightarrow \text{数据布局优化} \rightarrow \text{测量与验证} \rightarrow \text{继续迭代}.$$

最后还要检查代码的可移植性、可扩展性和可维护性。一个依赖特定设备偶然参数、只在单一输入上有效的实现，可能在下一代硬件或更大数据集上失效。

1.6.2 挑战一：大规模并行要求规则性

图形处理器最有效的执行形态是许多线程在同一阶段执行相同或相近的操作，只是处理不同数据。这种要求称为**规则性**(*regularity*)。规则性包含至少三个层面：

- 工作结构规则：线程执行路径相近，不出现少数线程承担大量额外步骤；
- 数据访问规则：线程以可预测方式访问数据，避免大量随机、冲突或低复用访问；
- 阶段推进规则：线程组能在相近时间完成当前阶段，减少等待与闲置。

规则性不是说所有线程必须处理完全相同的数据，而是说硬件可以用共享的控制与调度机制高效驱动大量相似工作。若每个线程都执行截然不同的控制流，图形处理器复制大量算术通道的优势就难以兑现。

1.6.3 挑战二：负载均衡

设一个并行阶段包含 n 个任务，第 i 个任务的执行时间为 T_i。如果下一阶段必须等所有任务完成，则该阶段的完成时间为

$$T_{\text{phase}} = \max_{1 \le i \le n} T_i. \tag{1.1}$$

式 (1.1) 表明，最快线程提前完成并不能缩短阶段时间；最慢线程决定何时能够继续。这就是**负载均衡** (*load balance*) 的核心。

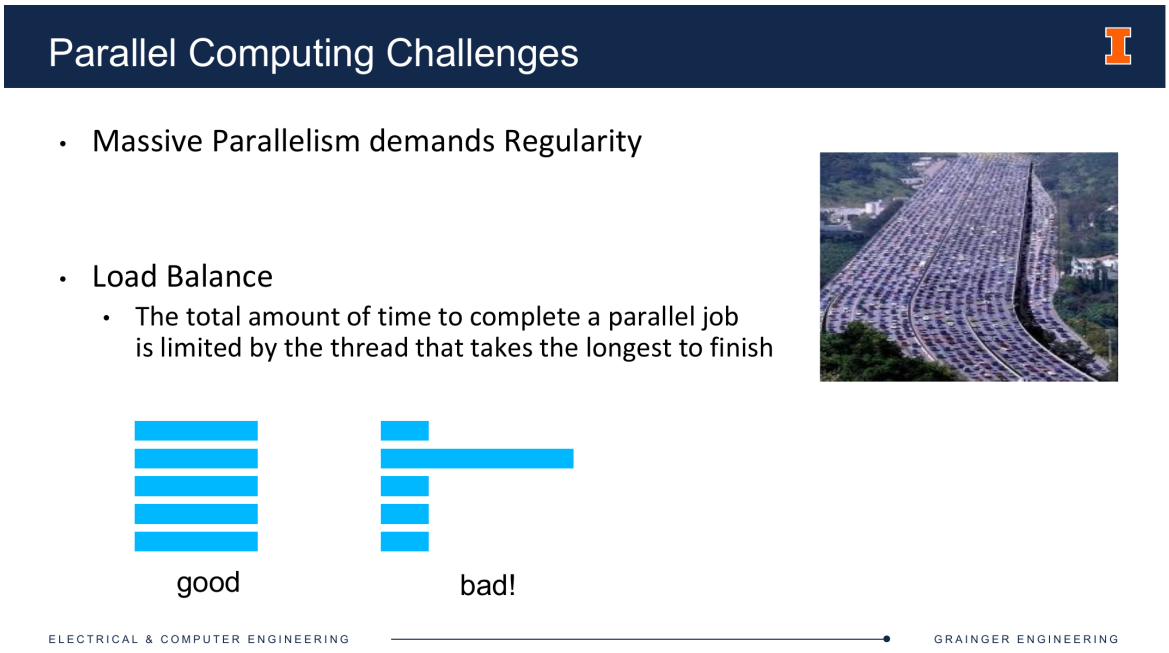


图 1.9 规则并行与负载不均衡

为了量化不均衡，可以定义一个解释性指标

$$E_{\text{load}} = \frac{\sum_{i=1}^{n} T_i}{n \max_i T_i}, \qquad 0 < E_{\text{load}} \le 1. \tag{1.2}$$

若所有任务耗时相同，则 $E_{\text{load}} = 1$；若少数任务远慢于其余任务，该比值会降低，表示大量并行资源在等待最慢任务时处于空闲状态。式 (1.2) 是用于理解负载不均衡的解释性指标；性能也可以用其他指标量化。

负载不均衡可能来自输入数据差异、分支路径差异、任务划分粒度过粗或某些任务争用更多资源。解决方向是重新分割任务、细化工作粒度或选择更规则的算法，使各执行单元承担的有效工作接近。

1.6.4 挑战三：全局内存带宽的理想值与实际值

全局内存带宽（*global memory bandwidth*）表示设备内存与处理器之间单位时间能够传输的数据量。硬件规格给出的峰值带宽是在理想访问模式下的上限，应用实际获得的**有效带宽**（*effective bandwidth*）可写为

$$B_{\text{eff}} = \frac{D_{\text{useful}}}{T}, \tag{1.3}$$

其中 D_{useful} 是完成目标计算所需的有效数据量，T 是相应时间。若程序重复搬运数据、访问模式分散或不能充分并行发起内存请求，则 B_{eff} 可能远低于峰值。

水坝与吸管的类比可以区分“理想”与“现实”：设备可能具备很宽的总带宽，但单个程序若组织不当，只能利用其中很小一部分。提高有效带宽依赖正确的内存模型、数据复用和访问模式。

1.6.5 挑战四：冲突访问导致串行化与延迟

当大量线程同时访问某个受限或关键资源时，请求必须排队，原本并行的工作被迫部分**串行化**（*serialization*）。国际机场入境检查可以类比这种瓶颈：许多航班可以同时到达，但若所有旅客必须通过数量有限的检查窗口，检查阶段就成为窄口，前面的并行到达能力无法转化为端到端吞吐量。

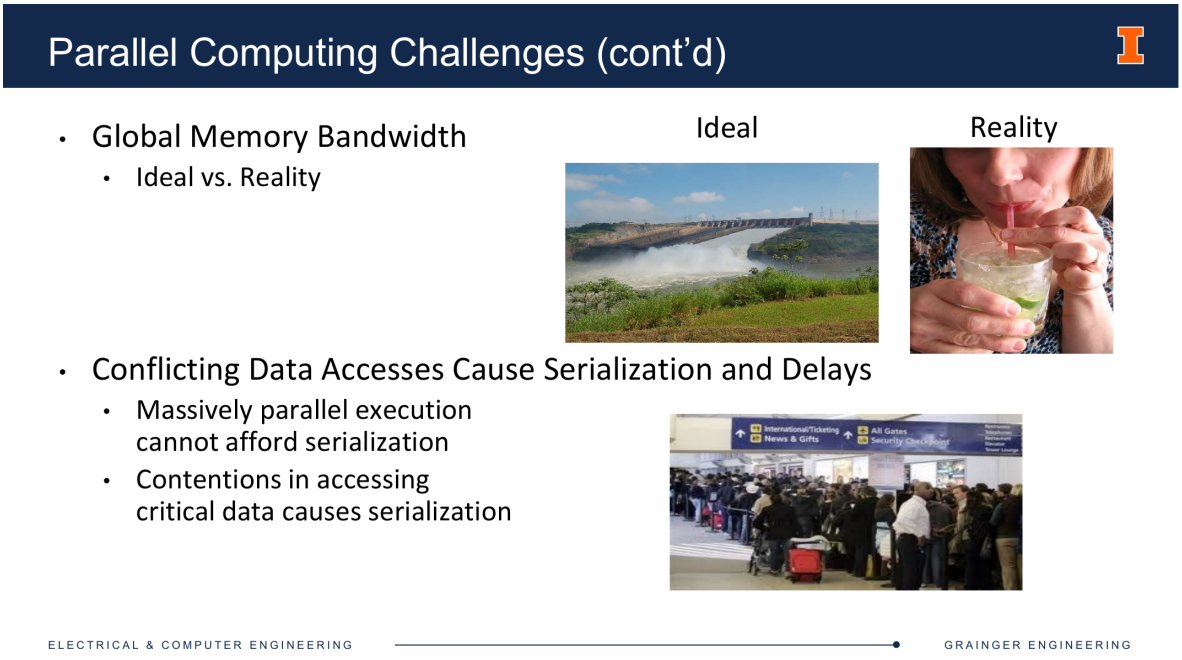


图 1.10 全局内存带宽与关键资源争用

从系统角度看，争用会产生两类损失：

1. 请求排队，单个访问的等待时间增加；
2. 等待资源的线程不能推进，执行通道可能空闲，整体吞吐量下降。

海量并行系统对串行化尤其敏感。若只有一个线程，某个小的串行步骤只是该线程的一部分；若有成千上万线程同时汇聚到同一关键资源，该步骤会把大量并行执行压缩为一条队列。同步、共享数据更新与内存访问设计都必须控制这种汇聚效应。

1.6.6 四类挑战之间的耦合

规则性、负载均衡、带宽和争用不是相互独立的问题。一个不规则算法可能同时造成：不同线程工作量差异、数据访问失去局部性、多个线程争用热点数据，以及难以隐藏的等待。相反，规则的数据并行结构通常更容易均匀划分工作，也更容易形成高效访存。

关键结论

并行化的目标不是把串行语句机械地复制到更多线程，而是构造一种硬件能够持续推进的工作形态：线程数量足够多，执行路径足够规则，任务负载相近，数据访问能够利用带宽，并且不存在把全部线程重新压回串行队列的关键资源。

1.7 阿姆达尔定律：整体加速比的串行上界

1.7.1 问题设定与符号

考虑一个固定规模的应用，其原始串行执行时间为 t。把原始时间按能否被目标并行化方法加速分成两部分：

- 可并行部分占比为 p，原始耗时为 pt；
- 不可并行部分占比为 $1-p$，原始耗时为 $(1-p)t$；
- 可并行部分经过并行化后获得局部加速比 s，因此其新耗时为 pt/s。

这里 $0 \le p \le 1$，$s \ge 1$。该模型假设问题规模固定，并暂时不单独计入内核启动、数据传输、同步与通信等额外开销。

1.7.2 总执行时间的推导

不可并行部分没有被加速，因此仍需 $(1-p)t$；可并行部分缩短为 pt/s。新程序的总执行时间为

$$T_{\text{new}} = (1-p)t + \frac{pt}{s} = \left(1 - p + \frac{p}{s}\right)t. \tag{1.4}$$

整体加速比定义为原始执行时间与新执行时间之比：

$$S(p,s) = \frac{t}{T_{\text{new}}} = \frac{t}{\left(1 - p + \frac{p}{s}\right)t} = \boxed{\frac{1}{1 - p + \frac{p}{s}}}. \tag{1.5}$$

原始时间 t 在分子与分母中约去，因此在这一固定比例模型中，整体加速比只由可并行比例 p 和局部加速比 s 决定。

1.7.3 并行部分无限快时的上界

即使可并行部分被理想化为瞬间完成，即 $s \to \infty$，仍有

$$\lim_{s\to\infty} S(p,s) = \frac{1}{1-p}. \tag{1.6}$$

式 (1.6) 是阿姆达尔定律最重要的含义：整体加速比由未被加速的串行比例限制。可并行部分继续变快时，它在总时间中的占比越来越小，最终总时间几乎全部由串行部分构成。

Amdahl's Law

- In general, if an application has:
  - t sequential execution time
  - p fraction of execution that is parallelizable
  - s speedup achieved on the parallelizable part
- What is the overall speedup of the application?

$$t_{parallel} = (1-p) * t + \frac{p * t}{s} = \left(1 - p + \frac{p}{s}\right) * t$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{t}{\left(1 - p + \frac{p}{s}\right) * t} = \frac{1}{1 - p + \frac{p}{s}} \xrightarrow{s \to \infty} \frac{1}{1-p}$$

ELECTRICAL & COMPUTER ENGINEERING GRAINGER ENGINEERING

图 1.11 阿姆达尔定律推导

1.7.4 示例：90% 可并行，局部加速 1000 倍

例题

原程序串行运行时间为 $t = 100\,\mathrm{s}$，可并行比例为 $p = 0.9$，可并行部分在图形处理器上获得 $s = 1000$ 的加速。求新执行时间与整体加速比。

串行部分原本占 10%，因此保持为

$$(1-p)t = (1-0.9) \times 100\,\mathrm{s} = 10\,\mathrm{s}.$$

可并行部分原本耗时 90 s，加速 1000 倍后为

$$\frac{pt}{s} = \frac{0.9 \times 100\,\mathrm{s}}{1000} = 0.09\,\mathrm{s}.$$

所以

$$T_{\mathrm{new}} = 10\,\mathrm{s} + 0.09\,\mathrm{s} = 10.09\,\mathrm{s},$$

$$S = \frac{100\,\mathrm{s}}{10.09\,\mathrm{s}} \approx 9.91.$$

尽管局部加速比达到 1000，整体加速比仍只有约 9.91。若把可并行部分变为无限快，整体

上界也只是

$$S_{\max} = \frac{1}{1 - 0.9} = 10.$$

1.7.5 串行比例对上界的高度敏感性

表 1.9 不同可并行比例对应的理想整体加速上界

可并行比例 p	串行比例 $1 - p$	$s \to \infty$ 时的上界 $1/(1 - p)$
0.50	0.50	2
0.90	0.10	10
0.95	0.05	20
0.99	0.01	100
0.999	0.001	1000

要获得数量级更高的整体加速，串行比例必须按同样数量级下降。理想情况下，若希望整体加速达到 S_{target}，至少需要

$$1 - p \le \frac{1}{S_{\text{target}}}.$$

例如，100 倍加速要求串行比例不超过 1%；有限的局部加速和额外开销会使实际要求更严格。许多大数据集应用的可并行比例可以超过 99%，因此超过 100 倍的加速在合适条件下可以实现。

1.7.6 为什么先做性能剖析

假设某函数占总时间的比例为 p，对该函数的任何局部优化都受式 (1.5)约束。因此，性能剖析的作用不是只找“看起来复杂”的代码，而是确定哪些部分具有足够大的时间占比。优化低占比部分，即使局部加速很高，整体收益也会被阿姆达尔定律压缩。

反过来，当并行部分已经极快时，原先不起眼的串行初始化、数据准备、输出或同步会成为新的热点。性能优化因此具有阶段性：每消除一个瓶颈，时间分布都会改变，需要重新剖析，而不能永久沿用最初的热点排序。

1.7.7 模型边界与现实开销

实际异构程序通常还包含图形处理器内核启动、中央处理器与图形处理器之间的数据移动、同步、资源争用和调度开销。若把这些额外时间记为 T_{overhead}，则更现实的表达为

$$S_{\text{real}} = \frac{t}{(1-p)t + \frac{pt}{s} + T_{\text{overhead}}}. \tag{1.7}$$

式 (1.7) 在基本模型中加入了显式开销。它说明，即使 p 很大，问题规模过小也可能无法抵消固定开销；提高局部内核速度之后，数据移动与同步可能主导总时间。这与第 1.3 式所描述的有效带宽问题以及资源争用问题相互关联。

关键结论

阿姆达尔定律给出的优化顺序是：先找到占比最大的可加速部分，同时持续削减不可并行与协调开销；局部加速达到很高后，必须重新测量，因为瓶颈会迁移到之前占比很小的部分。整体性能永远由完整执行路径决定。

1.8 综合框架与关键结论

1.8.1 硬件演化到软件责任的完整链条

硬件演化与软件责任构成以下统一框架：

1. 摩尔定律长期提高晶体管密度，登纳德缩放长期提高时钟频率；两者共同形成早期单核性能快速增长。
2. 约在 2005–2006 年，漏电流、功耗与散热约束使频率增长趋于停滞，但晶体管数量仍能继续增加。
3. 新增晶体管被组织为多核、向量执行资源和专用加速器。现代系统因此具有显著并行性与异构性。
4. 中央处理器面向低延迟，适合串行依赖和复杂控制；图形处理器面向高吞吐量，适合大量规则且相互独立的计算。
5. 图形处理器原有的游戏市场和图形并行结构提供了经济与技术基础；统一计算设备架构进一步把它转化为通用计算平台。

6. 应用必须显式暴露并行性。性能依赖算法可扩展性、线程规则性、负载均衡、数据布局、有效带宽和争用控制。

7. 任意局部加速最终都受完整应用中未加速部分限制，阿姆达尔定律给出这一上界。

1.8.2 选择中央处理器或图形处理器时的判断顺序

面对一个待加速代码段，可以按下列问题逐步判断：

1. 它在完整程序中占多少时间？占比过低时，局部优化的整体收益受限。

2. 是否存在大量相互独立或弱依赖的任务？任务数量必须足以填充图形处理器。

3. 各任务是否执行相同或相近的操作？高度不规则控制流会降低并行执行效率。

4. 每个任务的工作量是否接近？最慢任务会决定并行阶段完成时间。

5. 数据访问是否规则、具有局部性，并能利用全局内存带宽？

6. 把数据移动到图形处理器并启动内核的开销，是否小于并行计算所节省的时间？

7. 实现能否在更大输入和后续硬件上保持正确性、性能与可维护性？

只有当这些问题的总体答案支持加速器映射时，图形处理器才可能产生端到端收益。对于串行依赖明显、任务规模很小或数据移动占主导的代码，中央处理器可能更合适。

1.8.3 高频概念辨析

容易形成的误解	准确理解
摩尔定律等同于处理器性能翻倍。	历史性能增长来自晶体管密度、频率缩放和处理器结构改进的共同作用；摩尔定律主要描述晶体管密度趋势。
处理器核心数量增加会自动加速串行程序。	串行指令流通常只使用一个核心；程序必须暴露并行任务并承担调度、通信和同步成本。
图形处理器的单个线程一定比中央处理器线程快。	图形处理器优化大批线程的总吞吐量，单线程延迟可能更高。
统一计算设备架构核心与中央处理器核心可以按数量直接比较。	两者粒度不同；统一计算设备架构核心更接近细粒度算术执行通道。
硬件峰值内存带宽就是程序能够获得的带宽。	实际有效带宽取决于数据布局、访问模式、复用、并发程度和争用。

容易形成的误解	准确理解
并行部分加速 1000 倍会让整个程序加速 1000 倍。	整体加速由 $S = 1/(1 - p + p/s)$ 决定，未加速部分最终成为上界。
问答系统使用课程材料，因此答案必然正确。	检索增强生成只能改善相关性；检索遗漏、隐含假设与生成错误仍可能导致错误答案。

1.8.4 “高性能”的操作性定义

高性能需要放在完整软件工程目标中理解。并行实现应满足：

- **功能正确**：与参考语义一致，能够通过规定的数据集与问题检查；
- **端到端加速**：完整应用时间缩短，而不是只报告某个局部内核的理想时间；
- **可解释**：能够用并行度、访存、负载、串行比例等因素解释性能变化；
- **可扩展**：问题规模或硬件资源增加时仍能利用更多并行能力；
- **可维护与可移植**：实现结构清晰，不依赖难以复现的偶然参数，并能适应硬件代际变化。

这五项要求也解释了项目评分为何同时关注功能、编码风格和性能。单独追求某一项会产生失衡：正确但极慢的实现没有利用加速器价值；快速但错误的实现改变了问题；高度手工特化但不可维护的实现难以跨代延续。

1.8.5 阅读与操作任务

需要完成以下事项：

- 阅读教材第 1 章；
- 阅读《统一计算设备架构编程指南》第 1.1 节“导论”；
- 尽快创建高等科学与工程研究计算环境账户并取得 Delta 访问权限；
- 配置个人代码仓库，完成课程发布代码的同步流程；
- 在账户可用后验证安全外壳协议登录、编译节点与计算节点作业提交流程。

关键结论

软件可持续性要求算法与库具备可扩展性和可移植性，从而跨越多代硬件；其价值来自对并行结构和性能瓶颈的清晰建模，而不是依赖某一代设备的偶然峰值。

第 2 讲

Introduction to CUDA C and Data Parallel Programming

2.1 核心内容

内容概览

- **数据并行计算**(*data-parallel computing*)的基本概念；
- **CUDA C 编程接口**(*CUDA C programming interface*)的基本特性。

2.2 从线程到大规模数据并行

2.2.1 线程是计算的基本单位

线程(*thread*)是一条正在执行的顺序计算。线程包含程序或指令序列、指出当前执行位置的**程序计数器**(*program counter, PC*)，以及执行上下文（context），例如内存状态与寄存器状态。单个线程内部仍按顺序执行；并行性来自许多线程在硬件上同时推进。

线程的最小心智模型

设线程 T 的状态写成

$$T = (P,\ PC,\ R,\ M_{\text{visible}}),$$

其中 P 是程序，PC 是下一条待执行指令的位置，R 表示该线程可见的寄存器状态，M_{visible} 表示该线程可以访问的内存状态。CUDA 中同一**核函数**(*kernel*)产生的线程执行相同程序 P，但它们看到的线程索引不同，因此访问的数据位置不同。

2.2.2 “多个线程”与“海量线程”

传统多核 CPU 往往运行数量相对有限、行为较复杂的线程；GPU 的设计目标则是让数量极大的线程并发执行。当线程数量达到成千上万甚至更多时，任意而频繁的线程间通信会迅速造成等待与串行化。因此，最容易映射到 GPU 的工作负载通常具备以下特征：

- 线程之间的依赖很少；
- 每个线程能够独立确定自己的输入与输出；
- 大量线程执行相同或相近的计算；
- 线程之间即使合作，也尽量限制在较小的局部范围内，例如同一线程块。

CUDA 线程仍可通过**共享内存**（*shared memory*）、**原子操作**（*atomic operation*）与**屏障同步**（*barrier synchronization*）进行合作。设计并行程序时，应**先寻找独立工作，再加入必要合作**。

2.2.3 任务并行与数据并行

任务并行（*task parallelism*）把不同操作分配给不同执行单元；**数据并行**（*data parallelism*）则让大量执行单元在不同数据上执行相同操作。GPU 最擅长后者，因为大型数据集常能释放出极高的并行度。

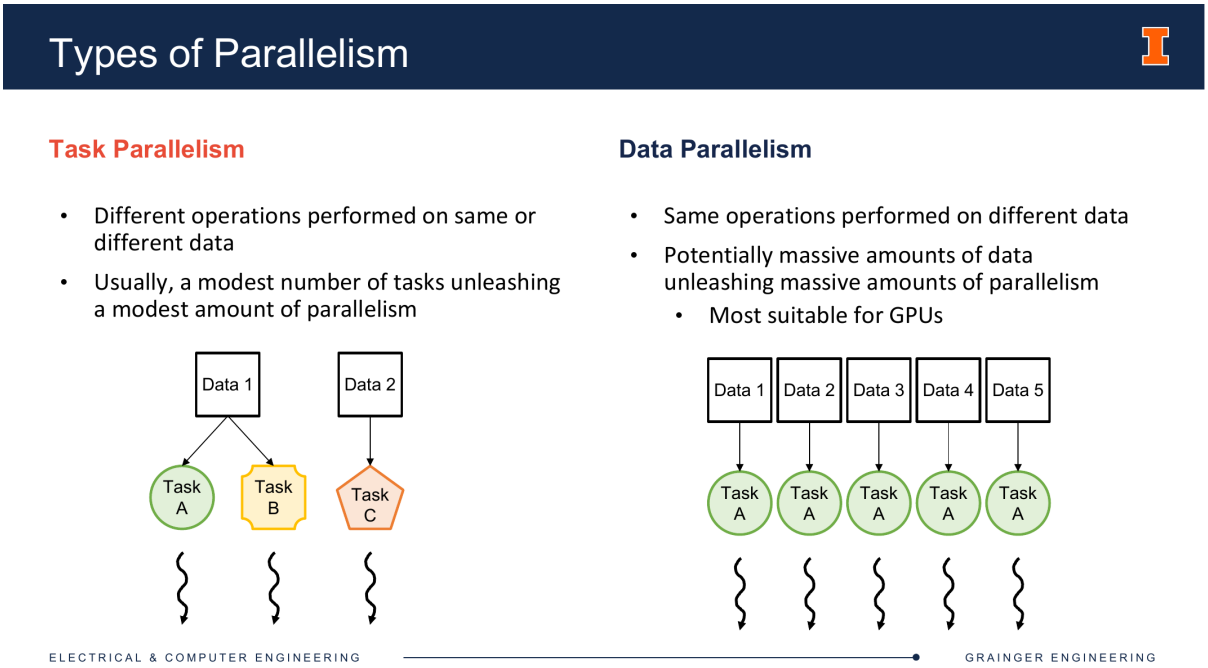


图 2.1 任务并行（task parallelism）与数据并行（data parallelism）的对比

可用一个统一的数学形式描述数据并行。设输入数据为

$$X = \{x_0, x_1, \dots, x_{n-1}\},$$

每个输出元素由同一个函数 f 产生：

$$y_i = f(x_i), \qquad 0 \le i < n.$$

若 y_i 的计算不依赖尚未生成的 y_j，也不修改其他线程负责的输出位置，则所有 y_i 可以并行计算。

2.2.3.1 彩色图像转灰度图像

每个像素具有红、绿、蓝三个分量 (r_i, g_i, b_i)。常用的亮度近似为

$$L_i = 0.299 r_i + 0.587 g_i + 0.114 b_i.$$

对任意两个不同像素 $i \neq j$，L_i 的计算不读取或修改像素 j 的输出，因此像素转换彼此独立。顺序循环

$$\texttt{for each pixel:}\ \ L_i \leftarrow f(r_i, g_i, b_i)$$

可以被解释成“一个线程对应一次循环迭代”。这正是 CUDA 入门中最重要的**循环并行化** (*loop parallelism*) 视角。

判断数据并行时必须问的三个问题

1. 每次迭代写入的位置是否唯一？
2. 每次迭代读取的数据是否已经可用，而非由其他并行迭代临时产生？
3. 改变不同迭代的执行顺序，是否仍能得到相同结果？

三个答案均为“是”时，通常可以直接采用一线程一元素的映射。

2.3 CUDA 的主机-设备执行模型

2.3.1 一个源程序，两类执行位置

CUDA 程序是**异构应用** (*heterogeneous application*)：同一个工程同时包含在 CPU 上运行的**主机代码** (*host code*)，以及在 GPU 上运行的**设备代码** (*device code*)。主机代码通常负责串行或并行度较低的工作，例如输入输出、内存管理与核函数调度；设备代码负责数据量大、并行度高的计算阶段。

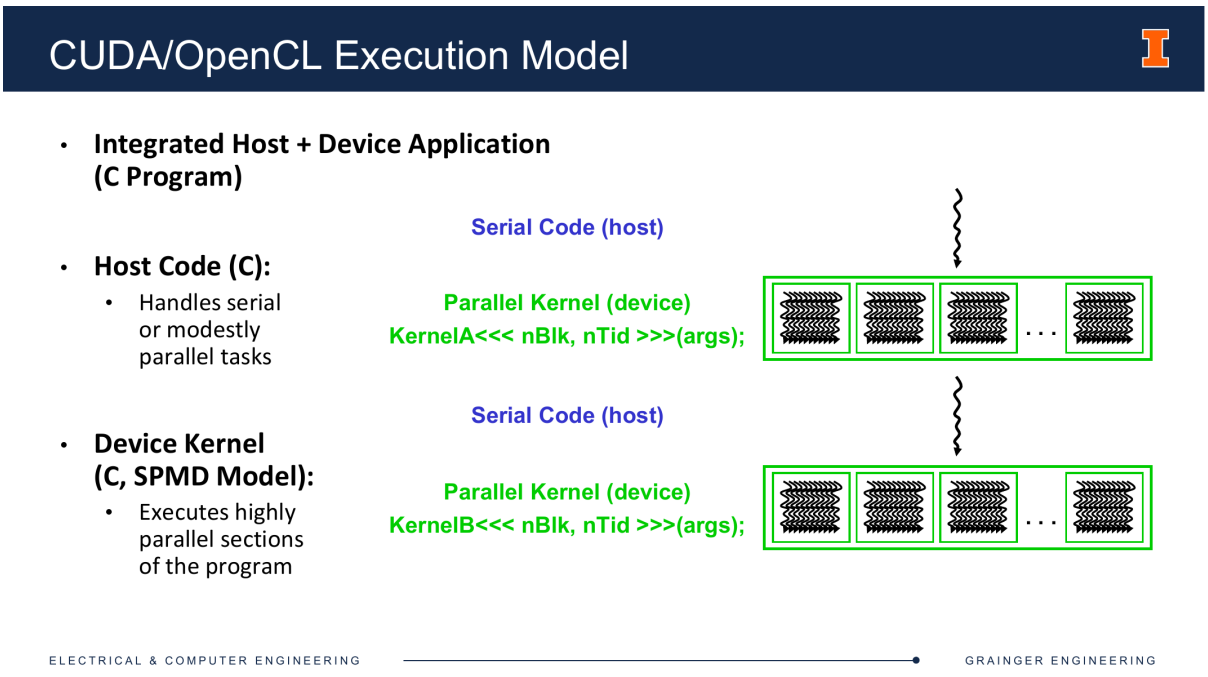


图 2.2 CUDA/OpenCL 的集成式主机-设备执行模型

一个典型的时间顺序可以写成：

主机准备数据 → 启动核函数 A → 主机继续执行或等待 → 启动核函数 B → 取回结果.

2.3.2 核函数与单程序多数据模型

核函数(*kernel*)是由许多 GPU 线程执行的设备函数。一次核函数启动（kernel launch）创建一个线程网格。网格中的所有线程运行同一份核函数代码，但每个线程使用自己的索引选择不同数据。这种编程方式称为**单程序多数据**(*Single Program Multiple Data, SPMD*)。

SPMD 的“同一程序”不意味着所有线程在每一时刻必然完成相同工作。线程可以依据索引进行条件判断；只是它们的代码文本相同。最简单的向量加法核函数逻辑是

$$C[i] = A[i] + B[i],$$

其中线程索引 i 决定该线程访问哪一对输入元素。

主机与设备的职责边界

对向量加法而言，主机代码负责准备 A、B、C，把输入移动到设备可访问的位置，配置线程组织并启动核函数；设备线程只负责执行元素级加法。把数据移动、调度和计算区分开，是阅读任何 CUDA 程序的第一步。

2.4 线程层次结构与一维索引映射

2.4.1 网格、线程块与线程

一次核函数启动创建一个**网格**(*grid*)。网格由若干**线程块**(*thread block*)组成，每个线程块再由若干**线程**(*thread*)组成。对于一维网格：

- `gridDim.x`：网格在 x 方向包含的线程块数；
- `blockIdx.x`：当前线程块在网格中的索引；
- `blockDim.x`：每个线程块在 x 方向包含的线程数；
- `threadIdx.x`：当前线程在线程块内部的索引。

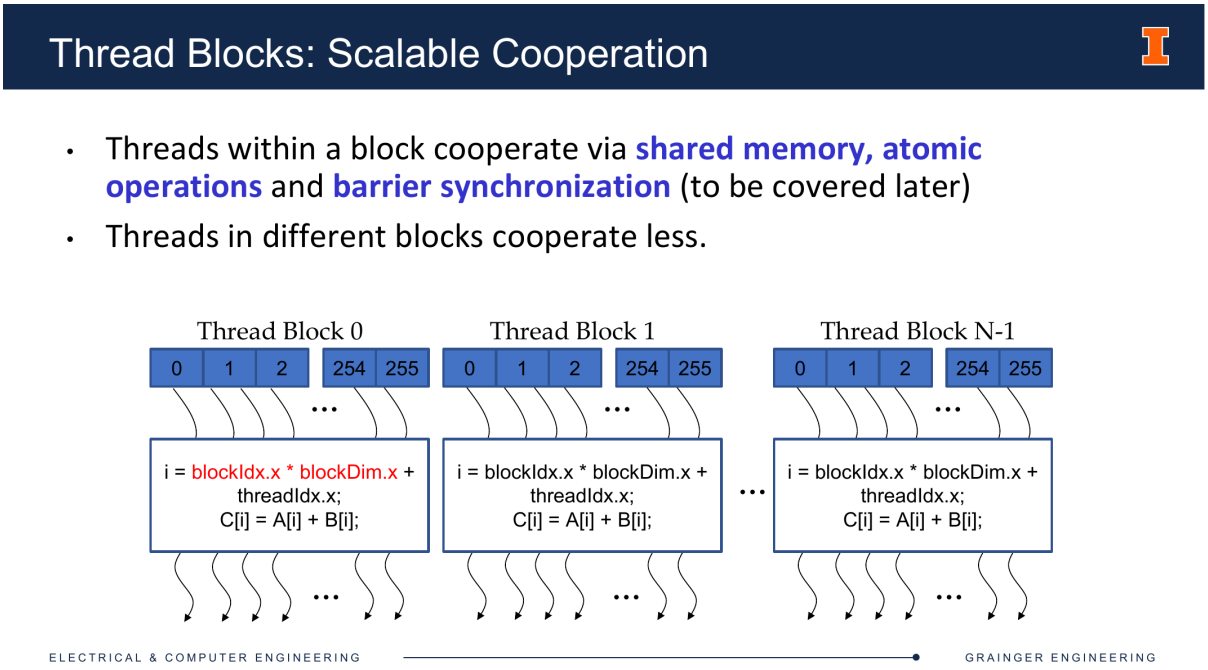


图 2.3 线程块内部可进行较强合作，不同线程块之间的合作更受限制

同一线程块内的线程可通过共享内存、原子操作与屏障同步进行局部合作。不同线程块必须能够以任意顺序调度，因此各线程块通常应设计为彼此独立。这个限制换来了**可扩展性**(*scalability*)：同一核函数可以在执行资源不同的 GPU 上运行，而无需把线程块绑定到特定硬件位置。

2.4.2 全局线程索引的推导

设每个线程块有 $T = \texttt{blockDim.x}$ 个线程，当前线程块索引为

$$b = \texttt{blockIdx.x},$$

线程在线程块内的局部索引为

$$t = \texttt{threadIdx.x}.$$

在当前线程块之前共有 b 个完整线程块，每个线程块贡献 T 个线程，所以当前线程块之前一共有 bT 个线程。把局部偏移 t 加上去，得到全局一维索引

$$\boxed{i = bT + t}$$

即

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

2.4.2.1 覆盖范围证明

对固定的线程块 b，局部索引满足 $0 \le t < T$，因此该线程块产生的全局索引范围是

$$bT \le i \le bT + T - 1.$$

相邻线程块 $b+1$ 的起始索引为

$$(b+1)T = bT + T,$$

恰好接在前一线程块的末尾之后。因此所有线程块共同形成连续、不重叠的一维索引区间。

例：线程块大小为 256

若 $T = 256$：

$$\begin{aligned} \text{线程块0}：i &= 0, 1, \dots, 255, \\ \text{线程块1}：i &= 256, 257, \dots, 511, \\ \text{线程块3中的线程0}：i &= 3 \times 256 + 0 = 768. \end{aligned}$$

由此得到的全局索引与图示中的线程分配一致。

2.4.3 线程索引是“数据归属规则”

索引公式本身只是编号。真正的并行设计问题是：**编号为 i 的线程应该负责哪一块数据？**对一维向量加法，最自然的规则是线程 i 负责元素 i。对二维图像、卷积、归约或扫描，线程到数据的映射会更复杂，但方法相同：先定义工作单元，再用内置索引计算该工作单元的位置。

关键结论

给定 B 个线程块、每块 T 个线程：

$$\text{启动线程总数} = BT.$$

每个线程拥有自己的自动局部变量副本；因此若核函数中声明一个局部变量 `i`，整个网格生命周期中最多会为 BT 个线程各产生一个逻辑副本。

2.5 向量加法：从顺序循环到线程网格

2.5.1 顺序基线与并行分解

向量加法（vector addition）是数据并行程序中的“Hello, World!”。给定长度均为 n 的向量 A 与 B，目标是计算

$$C[i] = A[i] + B[i], \qquad 0 \le i < n.$$

顺序 C++ 实现用一个循环依次完成 n 次加法：

```
void vecAddCPU(const float* A, const float* B, float* C, int n) {
    for (int i = 0; i < n; ++i) {
        C[i] = A[i] + B[i];
    }
}
```

并行化时，不需要保留这个循环的顺序执行结构。把第 i 次循环迭代定义为一个独立工作单元，再把工作单元 i 分配给全局线程 i：

$$\text{线程} i \quad \longleftrightarrow \quad C[i] = A[i] + B[i].$$

因此，原来的循环被整个线程网格替代。并行化前应先回答“每个线程究竟独立负责什么工作”，因为这一选择会同时决定线程数量、数据布局与后续优化空间。

唯一写入者映射

输出元素 $C[i]$ 由唯一线程 i 负责写入。不同线程不会同时修改同一输出位置，因此不需要原子操作或同步。

2.5.2 CPU 与 GPU 的分离内存空间

经典的离散 GPU 系统模型由彼此分离的主机与设备内存空间组成：CPU 连接**主机内存**（*host memory*），GPU 连接**设备全局内存**（*device global memory*）。主机指针默认指向主机地址空间，设备指针指向设备地址空间；在基础 CUDA 编程模型中，二者不能被对方代码直接解引用。

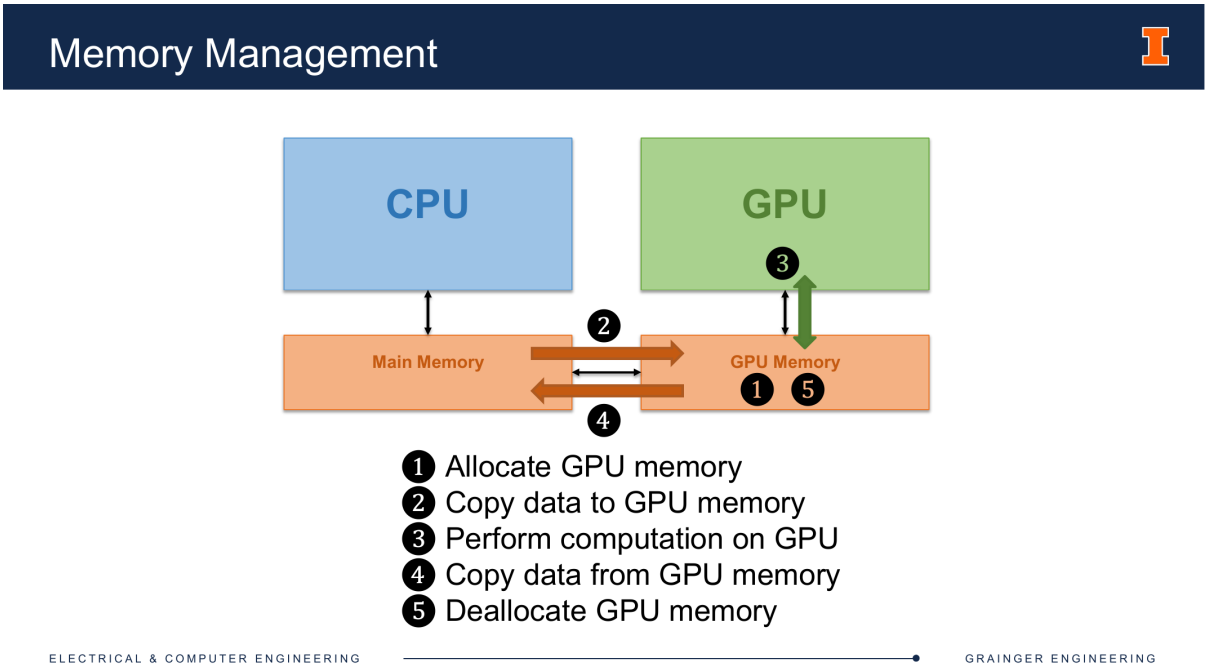


图 2.4 设备内存管理的五个阶段

完整数据流由五步构成：

1. 在设备端分配空间；
2. 把输入数据从主机复制到设备；
3. 在设备端启动核函数并计算；
4. 把结果从设备复制回主机；
5. 释放设备端空间。

即使系统提供**统一内存**（*unified memory*）或某些物理共享内存机制，数据仍需要在合适时机出现在 GPU 可高效访问的位置。显式分配与显式复制能够清楚呈现数据移动的成本与依赖关系。

2.5.3 设备内存管理 API

2.5.3.1 分配与释放

CUDA API	作用与关键参数
`cudaMalloc(void **devPtr, size_t size)`	在设备全局内存中分配 `size` 字节，并把起始地址写入 `*devPtr`。返回值类型为 `cudaError_t`。
`cudaFree(void *devPtr)`	释放由 `devPtr` 指向的设备内存区域。

对 n 个 `float` 元素，字节数应写成

$$S = n \cdot \texttt{sizeof(float)}.$$

实际代码宜用 `size_t` 保存 S，避免大规模数组下的整数溢出。

```
size_t bytes = static_cast<size_t>(n) * sizeof(float);
float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;

cudaMalloc(reinterpret_cast<void**>(&A_d), bytes);
cudaMalloc(reinterpret_cast<void**>(&B_d), bytes);
cudaMalloc(reinterpret_cast<void**>(&C_d), bytes);
// ... use the allocations ...
cudaFree(A_d);
cudaFree(B_d);
cudaFree(C_d);
```

设备指针的语义

`A_d` 的数值是设备地址。主机代码可以把它传给 CUDA 运行时 API 或核函数，但不应在普通主机表达式中写 `A_d[i]`。设备端数据访问应发生在核函数中。

2.5.3.2 主机-设备数据复制

`cudaMemcpy` 的接口为

```
cudaError_t cudaMemcpy(void* dst, const void* src,
                       size_t count, cudaMemcpyKind kind);
```

第四个参数明确复制方向：

枚举常量	方向
`cudaMemcpyHostToHost`	主机到主机（host to host）
`cudaMemcpyHostToDevice`	主机到设备（host to device）
`cudaMemcpyDeviceToHost`	设备到主机（device to host）
`cudaMemcpyDeviceToDevice`	设备到设备（device to device）

向量加法的输入与输出复制分别为

```
cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice);
// kernel launch
cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost);
```

读代码时可先检查 `dst` 与 `src` 是否和复制方向一致。例如，`cudaMemcpyDeviceToHost` 要求目标指针位于主机端、源指针位于设备端。

2.6 核函数实现、启动配置与边界检查

2.6.1 核函数声明与线程工作

核函数使用 `__global__` 限定符声明。每个线程执行同一函数体，并根据内置索引计算自己的元素编号：

```
__global__ void vecAddKernel(const float* A_d,
                             const float* B_d,
                             float* C_d,
                             int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        C_d[i] = A_d[i] + B_d[i];
    }
}
```

`i` 是**自动局部变量**（*automatic local variable*）。逻辑上，每个线程拥有自己的 `i`，一个线程对它的赋值不会改变其他线程的副本。

2.6.2 三重尖括号与启动参数

主机通过**三重尖括号语法**(*triple-chevron syntax*)启动核函数：

```
vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
```

第一个执行配置参数是网格中的线程块数，第二个参数是每个线程块的线程数。二者的位置不能互换。

2.6.3 向上取整除法的推导

设数据元素数为 n，每个线程块有 T 个线程。若每个线程只处理一个元素，需要的线程块数量为

$$B = \left\lceil \frac{n}{T} \right\rceil .$$

在整数算术中常写为

$$\boxed{B = \left\lfloor \frac{n + T - 1}{T} \right\rfloor},$$

对应代码

```
int numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock;
```

证明如下。令 $n = qT + r$，其中 $0 \le r < T$。

- 若 $r = 0$，则 n 可被 T 整除，公式得到 $B = q$；
- 若 $r > 0$，则至少还需要一个线程块，公式得到 $B = q + 1$。

因此该整数表达式与向上取整完全等价。

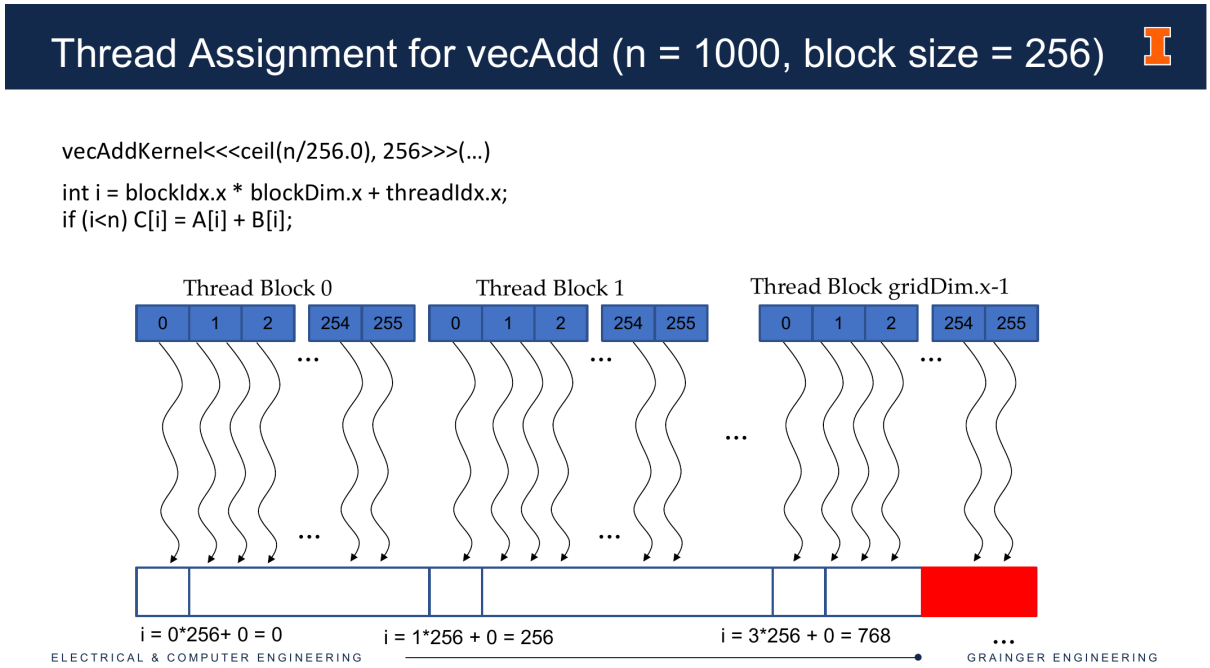


图 2.5 $n = 1000$、线程块大小为 256 时的线程分配

2.6.4 为什么必须做边界检查

总启动线程数为

$$N_{\text{launch}} = BT.$$

由于 B 经过向上取整，通常有 $N_{\text{launch}} \geq n$。多出的线程数

$$E = BT - n$$

满足 $0 \leq E < T$。这些额外线程没有对应的数据元素，必须在访问数组前检查

```
if (i < n) { /* valid work */ }
```

否则它们可能越界读取或写入设备内存。

例：$n = 1000$，$T = 256$

$$B = \left\lceil \frac{1000}{256} \right\rceil = 4, \qquad N_{\text{launch}} = 4 \times 256 = 1024.$$

前 1000 个线程处理有效元素，最后 24 个线程满足 $i \geq 1000$，被边界条件屏蔽。

2.6.5 向量加法的主机端骨架

把内存管理与核函数启动串联起来，得到如下流程：

```
void vecAdd(const float* A_h, const float* B_h,
            float* C_h, int n) {
    size_t bytes = static_cast<size_t>(n) * sizeof(float);
    float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;

    cudaMalloc(reinterpret_cast<void**>(&A_d), bytes);
    cudaMalloc(reinterpret_cast<void**>(&B_d), bytes);
    cudaMalloc(reinterpret_cast<void**>(&C_d), bytes);

    cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice);

    const int threadsPerBlock = 256;
    const int numBlocks = (n + threadsPerBlock - 1)
                          / threadsPerBlock;
    vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);

    cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost);

    cudaFree(A_d);
    cudaFree(B_d);
    cudaFree(C_d);
}
```

这段代码展示了结构，但尚未加入系统化的错误检查。下一轮将补齐可运行版本，并解释异步启动、编译流程与错误传播。

2.7 函数限定符与 CUDA 编译流程

2.7.1 三类函数限定符

CUDA C++ 用**函数限定符**（*function qualifier*）说明函数从哪里调用、在哪里执行。

限定符	可调用位置	执行位置	含义
`__host__`	主机（host）	主机（host）	普通主机函数；省略限定符时的默认情况。
`__global__`	主机，或受支持的设备端启动	设备（device）	核函数；调用时创建新的线程网格，返回类型必须为 `void`。
`__device__`	设备（device）	设备（device）	由核函数或其他设备函数调用，不创建新网格。

同一个普通函数也可以同时标记为

```
__host__ __device__ float add(float a, float b) {
    return a + b;
}
```

编译器会为主机和设备分别生成版本。调用端仍需满足各自执行环境的限制。

2.7.2 从.cu 源文件到 CPU/GPU 机器代码

CUDA 源文件通常使用 `.cu` 扩展名，并交给 **NVIDIA CUDA 编译器**（*NVIDIA CUDA Compiler, NVCC*）。NVCC 根据函数限定符区分主机代码与设备代码：

1. 主机部分交给常规 C/C++ 编译器，生成 CPU 目标代码；
2. 设备部分先生成**并行线程执行中间表示**（*Parallel Thread Execution, PTX*）；
3. PTX 在安装或运行阶段由**即时编译器**（*just-in-time compiler, JIT compiler*）转化为目标 GPU 的设备机器码，常以 SASS 表示。

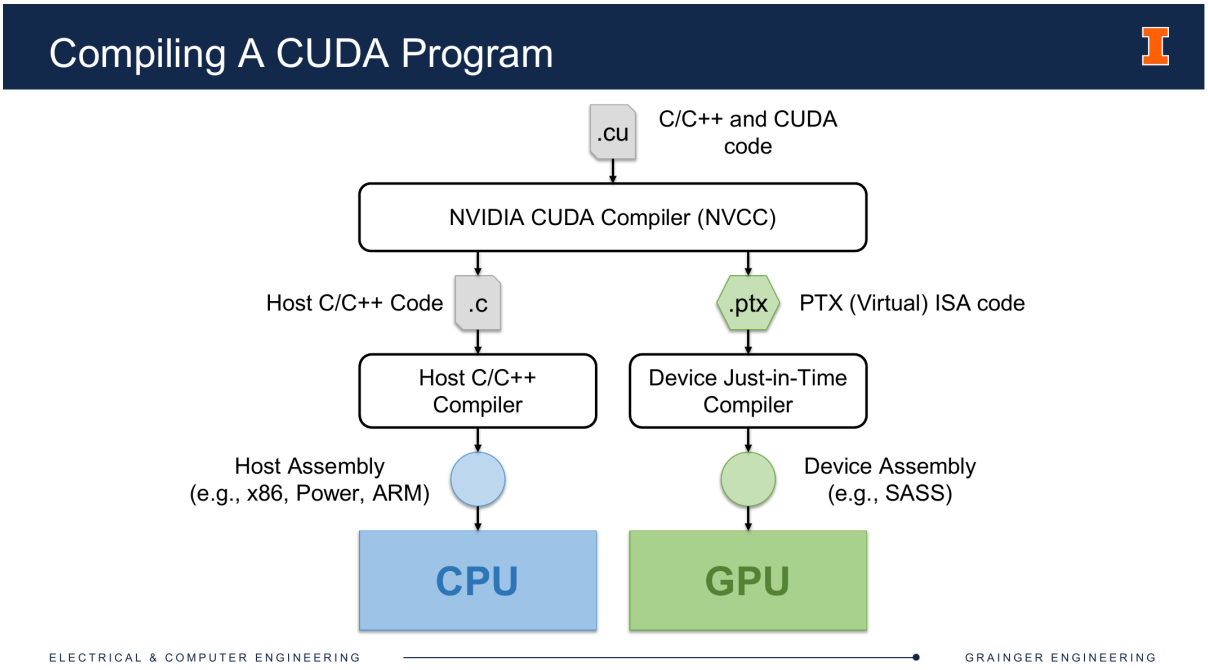


图 2.6 CUDA 程序的主机代码与设备代码编译路径

一个典型编译命令为

```
nvcc -O2 -std=c++17 vec_add.cu -o vec_add
```

其中 `-O2` 请求优化，`-std=c++17` 选择 C++ 语言标准。构建脚本或作业框架可以进一步封装这一步。

2.8 异步执行与错误检查

2.8.1 核函数启动默认是异步的

一次核函数启动通常先把工作提交给 GPU，然后让主机线程继续执行。此行为称为**异步调用**（*asynchronous call*）。因此，下面的 CPU 计时方式主要测到“提交请求”的时间，不一定覆盖 GPU 完成计算所需的时间：

```
start_timer();
vecAddKernel<<<B, T>>>(A_d, B_d, C_d, n);
stop_timer();      // usually too early
```

要让主机等待此前提交的设备工作完成，可调用

```
cudaDeviceSynchronize();
```

该函数提供主机与设备之间的**同步点**（*synchronization point*），并返回此前异步工作中暴

露的错误状态。

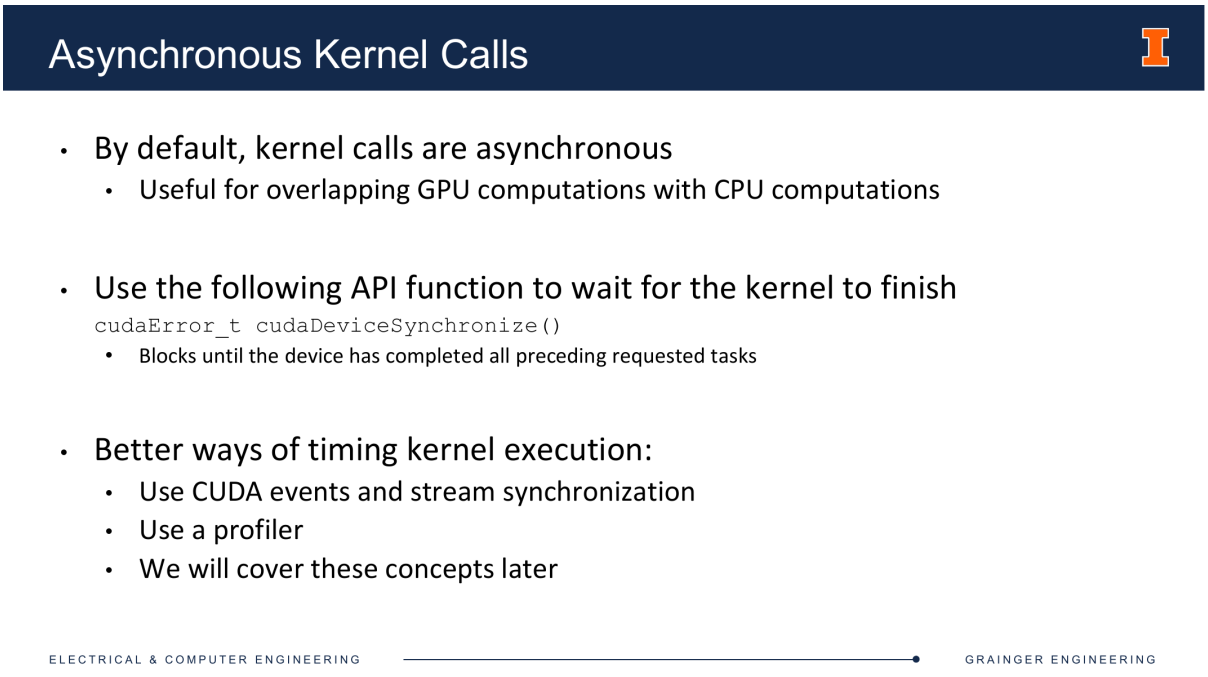


图 2.7 异步核函数调用与 `cudaDeviceSynchronize()`

当核函数后紧跟设备到主机的 `cudaMemcpy` 时，复制不能在产生结果的核函数完成之前得到最终输出。更精确的性能测量通常使用 CUDA **事件**（*event*），并在需要时配合**流同步**（*stream synchronization*）或**性能分析器**（*profiler*）。

2.8.2 两类错误：启动错误与执行错误

绝大多数 CUDA 运行时 API 都返回 `cudaError_t`。`cudaSuccess` 表示成功。其他错误码可交给 `cudaGetErrorString`，转换为可读消息。

核函数调用本身没有普通返回值，因此常分两步检查：

1. `cudaGetLastError()`：检查非法启动配置等立即可见的**启动错误**（*launch error*）；
2. `cudaDeviceSynchronize()`：等待执行完成，并暴露越界访问等**异步执行错误**（*asynchronous execution error*）。

```
vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

简化示例与工程代码的区别

用于突出概念的短代码通常会省略错误检查，也可能包含简化写法。工程代码应检查内存分配、数据复制、核函数启动与核函数执行的返回状态。

2.9 完整、可检查的向量加法程序

以下完整示例把线程索引、内存管理、核函数启动、同步和错误检查串联起来。代码使用 CUDA **运行时 API**（*CUDA Runtime API*），并用宏统一检查错误。

```cpp
#include <cuda_runtime.h>

#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <vector>

#define CUDA_CHECK(call) \
    do { \
        cudaError_t err__ = (call); \
        if (err__ != cudaSuccess) { \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
            __FILE__, __LINE__, cudaGetErrorString(err__)); \
            std::exit(EXIT_FAILURE); \
        } \
    } while (0)
```

```cpp
__global__ void vecAddKernel(const float* A_d,
                             const float* B_d,
                             float* C_d,
                             int n) {
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        C_d[i] = A_d[i] + B_d[i];
    }
}
```

```
void vecAddGPU(const float* A_h, const float* B_h,
               float* C_h, int n) {
    if (n <= 0) {
        return;
    }

    const size_t bytes = static_cast<size_t>(n) * sizeof(float);
    float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;

    CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&A_d), bytes));
    CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&B_d), bytes));
    CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&C_d), bytes));

    CUDA_CHECK(cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice));

    constexpr int threadsPerBlock = 256;
    const int numBlocks =
        (n + threadsPerBlock - 1) / threadsPerBlock;

    vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost));

    CUDA_CHECK(cudaFree(A_d));
    CUDA_CHECK(cudaFree(B_d));
    CUDA_CHECK(cudaFree(C_d));
}
```

```
int main() {
    constexpr int n = 1 << 20;
    std::vector<float> A(n), B(n), C(n);

    for (int i = 0; i < n; ++i) {
        A[i] = static_cast<float>(i);
        B[i] = 2.0f * static_cast<float>(i);
    }

    vecAddGPU(A.data(), B.data(), C.data(), n);

    for (int i = 0; i < n; ++i) {
        const float expected = A[i] + B[i];
        if (std::fabs(C[i] - expected) > 1.0e-5f) {
            std::fprintf(stderr, "Mismatch at %d\n", i);
            return EXIT_FAILURE;
        }
    }

    std::printf("Vector addition passed for n=%d.\n", n);
    return EXIT_SUCCESS;
}
```

2.9.1 程序结构

1. `main` 在主机端创建并初始化三个向量；
2. `vecAddGPU` 为三个向量分配设备内存；
3. 两次主机到设备复制把输入送入 GPU；
4. 启动配置根据 n 自动计算线程块数；
5. 每个设备线程计算一个 $C[i]$，越界线程被条件屏蔽；
6. 主机等待核函数完成并检查错误；
7. 设备到主机复制取回结果，随后释放设备资源；
8. `main` 在 CPU 上验证每一个结果元素。

正确性与性能是两个层次

本程序首先保证结果正确、资源被释放、错误能被报告。它仍会为每次函数调用重新分配内存并往返复制数据；对于包含多个 GPU 阶段的真实程序，通常应让数据在设备端保留更久，减少重复分配与传输。

2.10 例题与完整推导

2.10.1 例题一：读取核函数启动配置

给定

```
kernel<<<VECTOR_N, ELEMENT_N>>>(d_C, d_A, d_B, ELEMENT_N);
```

三重尖括号中的第一个参数是线程块数，第二个参数是每块线程数。因此

$$\boxed{\text{每块线程数} = \texttt{ELEMENT_N}}$$

且

$$\boxed{\text{网格总线程数} = \texttt{VECTOR_N} \times \texttt{ELEMENT_N}}.$$

变量名不会改变语法位置的含义；即使命名不理想，也必须按 CUDA 启动规则解释。

2.10.2 例题二：每线程处理 8 个元素

已知向量长度 $n = 16000$，每个线程处理 $k = 8$ 个输出元素，线程块大小 $T = 256$。

第一步，求逻辑上需要的工作线程数：

$$L = \left\lceil \frac{n}{k} \right\rceil = \frac{16000}{8} = 2000.$$

第二步，求线程块数：

$$B = \left\lceil \frac{L}{T} \right\rceil = \left\lceil \frac{2000}{256} \right\rceil = 8.$$

第三步，求实际启动线程数：

$$N_{\text{launch}} = BT = 8 \times 256 = 2048.$$

因此答案为

$$\boxed{2048 \text{ 个线程}}.$$

其中 48 个线程没有完整工作。实现**线程粗化**（*thread coarsening*）时，线程 q 可令起始元素为 kq，并对其负责的每个元素分别检查是否小于 n。

2.10.3 例题三：局部变量有多少份

核函数启动 $B = 512$ 个线程块，每块 $T = 256$ 个线程。总线程数为

$$BT = 512 \times 256 = 131072.$$

若核函数中声明一个普通自动局部变量，则每个线程拥有自己的逻辑副本，因此整个执行生命周期中共有

$$\boxed{131072 \text{ 份}}$$

线程私有版本。这里计算的是逻辑变量实例数；编译器可能把变量放入寄存器，也可能因资源压力使用局部内存，但线程私有语义不变。

2.10.4 特殊长度输入

$$B = \left\lceil \frac{4097}{256} \right\rceil = 17, \quad N_{\text{launch}} = 17 \times 256 = 4352, \quad E = 4352 - 4097 = 255.$$

该例达到单个线程块大小下“额外线程数”的最大可能值 $T - 1$，说明边界检查不是罕见特例，而是通用模板的一部分。

2.11 常见错误、检查清单与公式表

2.11.1 常见错误

错误	后果与修正
把 `<<<B,T>>>` 的顺序记反	第一个参数是线程块数，第二个参数是每块线程数。
写成 `ceil(n/T)`，但 n, T 都是整数	整数除法先截断，`ceil` 已无作用。改用 `(n+T-1)/T` 或浮点除法后取整。
遗漏 `if (i<n)`	最后一个线程块中的额外线程可能越界访问。
在主机端解引用设备指针	主机普通代码不应直接访问设备地址；通过 CUDA API 或核函数使用。

错误	后果与修正
不检查 CUDA 返回码	内存不足、非法配置或设备执行错误会被延迟发现，调试困难。
直接用 CPU 计时包围核函数启动	异步启动导致测到的主要是提交开销。计时结束前必须同步，后续宜使用 CUDA 事件。
假设线程块按编号顺序执行	CUDA 可按任意顺序调度线程块；不同线程块应在没有全局同步的情况下保持独立。
忘记释放设备内存	长时间运行或共享 GPU 环境中会造成资源泄漏与后续分配失败。

2.11.2 写一维 CUDA 核函数前的检查清单

1. 明确一个线程负责的输出元素或数据片段；
2. 写出全局索引公式 $i = bT + t$；
3. 计算 $B = \lceil n/T \rceil$；
4. 对不整除情况加入边界检查；
5. 明确每个指针位于主机还是设备；
6. 检查分配、复制、启动、执行与释放的错误状态；
7. 不对不同线程块的执行先后作假设；
8. 验证结果后再讨论性能。

2.11.3 核心公式表

问题	公式
一维全局线程索引	$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$
需要的线程块数	$B = \lceil n/T \rceil = \lfloor (n+T-1)/T \rfloor$
实际启动线程数	$N_{\text{launch}} = BT$
额外线程数	$E = BT - n$，且 $0 \le E < T$
k 元素/线程的逻辑线程数	$L = \lceil n/k \rceil$
k 元素/线程的线程块数	$B = \lceil L/T \rceil$
n 个 `float` 的字节数	$S = n \cdot \texttt{sizeof(float)}$
局部变量逻辑副本数	网格总线程数 BT

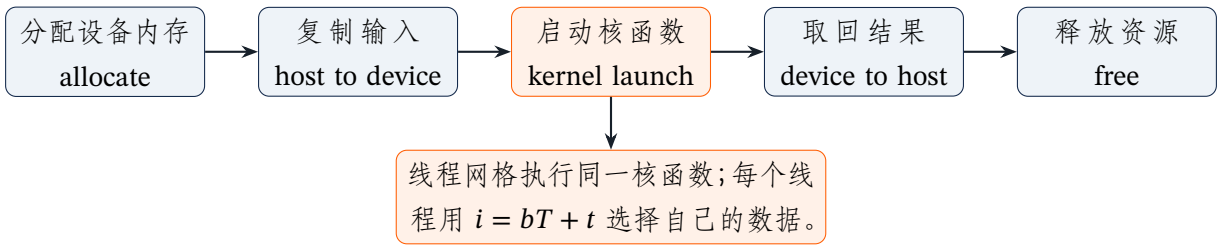
2.11.4 术语速查

中文术语	英文标准名称
数据并行	data parallelism
任务并行	task parallelism
线程 / 线程块 / 网格	thread / thread block / grid
核函数 / 核函数启动	kernel / kernel launch
单程序多数据	Single Program Multiple Data (SPMD)
主机 / 设备	host / device
主机内存 / 设备全局内存	host memory / device global memory
边界检查	boundary check
向上取整除法	ceiling division
异步调用 / 同步点	asynchronous call / synchronization point
函数限定符	function qualifier
即时编译	just-in-time compilation (JIT compilation)

2.12 综合框架与关键结论

数据并行 CUDA 程序的基本结构可以归纳为以下完整链条：

1. 从数据依赖出发，定义每个线程的独立工作；
2. 用主机代码管理数据与调度，用设备核函数执行高并行计算；
3. 用网格-线程块-线程层次和 $i = bT + t$ 建立执行到数据的映射；
4. 用显式内存管理完成分配、传输、计算、回传与释放；
5. 用向上取整启动和边界检查覆盖任意长度输入；
6. 用 NVCC 编译主机与设备代码，并正确处理异步执行与错误状态；
7. 先验证正确性，再进行性能优化。



关键公式

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}, \qquad B = \frac{n + T - 1}{T},$$

`if (i<n)` 保护边界，　`<<<B,T>>>` 依次表示线程块数与每块线程数.

第 3 讲

CUDA Parallel Execution Model: Multidimensional Grids and Data

3.1 核心内容

内容概览

- 理解 CUDA 线程的多维逻辑组织；
- 了解如何在核函数中使用循环等控制结构。

3.1.1 并行线程数组与单程序多数据模型

CUDA 核函数启动后，由一个线程网格执行。网格中的所有线程运行同一份核函数代码，但每个线程都能读取一组与自身位置有关的只读内建变量，从而确定自己负责的数据位置。该执行方式属于单程序多数据 (single-program, multiple-data, SPMD) 模型：

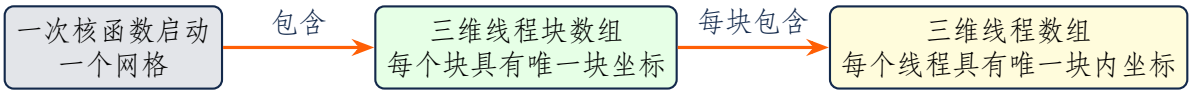
$$\text{同一程序} + \text{不同线程标识} \Longrightarrow \text{不同数据上的并行实例}.$$

定义

网格是一次核函数启动所创建的全部线程块的逻辑集合；线程块是能够使用块内共享机制进行协作的一组线程。因而，一次常规核函数启动形成两级层次：

$$\text{网格} \longrightarrow \text{线程块} \longrightarrow \text{线程}.$$

每个网格内的线程块具有相同的块尺寸，每个线程块内的线程执行相同的核函数代码。



未使用的维度尺寸取 1；所有线程执行同一核函数，并通过坐标选择不同数据。

图 3.1 CUDA 的逻辑执行层次

3.1.2 块内协作与块间独立性

线程块既是逻辑索引单位，也是协作边界。块内线程可通过共享内存 (shared memory)、原子操作 (atomic operation) 和屏障同步 (barrier synchronization) 等机制协作。不同线程块不共享块内资源，通常主要通过全局内存 (global memory) 交换数据，因此块间协作更受限制。

这一约束直接解释了线程块设计的两个基本原则：

- 同一块内的线程应对应一组存在局部数据关系、未来可能需要协作的工作项；
- 各线程块应尽量能够独立完成工作，使硬件可以按任意次序调度线程块。

3.1.3 多维组织的目的

CUDA 将网格和线程块统一表示为至多三维的逻辑数组。即使应用只使用一维或二维，未使用的维度仍存在，其尺寸取为 1。多维组织的主要价值是让线程坐标直接对应数据坐标：

- 一维向量通常使用全局下标 i；
- 二维图像或矩阵通常使用行列坐标 (r, c)；
- 三维体数据通常使用坐标 (z, y, x) 或 (d, r, c)。

这种对应关系降低了地址计算的复杂度，也使边界判断与任务划分更清晰。多维网格并未改变存储器本身的一维线性地址空间；它提供的是更自然的*逻辑坐标系*。逻辑坐标仍需在线程内部转换为线性内存偏移。

关键结论

多维线程组织解决的是“线程负责哪个逻辑数据元素”；行主序等数据布局解决的是“该逻辑元素位于线性内存的哪个位置”。二者需要连续完成：先求全局逻辑坐标，再求线性地址。

3.2 多维执行层次与索引系统

3.2.1 四组只读内建变量

每个正在执行的线程都可读取四组 CUDA 只读内建变量。它们共同给出“网格有多大、当前块在哪里、块有多大、当前线程在块内哪里”。

表 3.1 多维线程组织中的四组内建变量

变量	含义	三个分量及取值范围
`gridDim`	网格在各方向上的线程块数量	`x`、`y`、`z`；每个分量都是相应方向上的块数
`blockIdx`	当前线程块在网格中的坐标	$0 \le$ `blockIdx.d` $<$ `gridDim.d`，其中 $d \in \{x, y, z\}$
`blockDim`	每个线程块在各方向上的线程数量	`x`、`y`、`z`；三者乘积为每块线程总数
`threadIdx`	当前线程在线程块内部的坐标	$0 \le$ `threadIdx.d` $<$ `blockDim.d`，其中 $d \in \{x, y, z\}$

其中，`gridDim` 与 `blockDim` 描述尺寸，`blockIdx` 与 `threadIdx` 描述位置。对一维配置，未使用的 y, z 维尺寸为 1，对应索引只能取 0；对二维配置，未使用的 z 维尺寸为 1。

三维尺寸示例

若

$$\texttt{gridDim} = (8, 3, 2), \qquad \texttt{blockDim} = (5, 4, 3),$$

则网格含有

$$8 \times 3 \times 2 = 48$$

个线程块，每块含有

$$5 \times 4 \times 3 = 60$$

个线程，总启动线程数为

$$48 \times 60 = 2880.$$

块坐标的最大值为 $(7, 2, 1)$，块内线程坐标的最大值为 $(4, 3, 2)$。

3.2.2 一维全局索引

仅使用线程的块内下标 `threadIdx.x` 无法唯一标识整个网格中的线程，因为不同块内可以存在相同的块内下标。设每块包含 $B_x = $ `blockDim.x` 个线程，当前块编号为 $b_x = $ `blockIdx.x`，当前线程在块内的编号为 $t_x = $ `threadIdx.x`。当前块之前共有 b_x 个完整线程块，因此它们共覆盖 $b_x B_x$ 个工作项；再加上块内偏移 t_x，得到一维全局工作下标

$$i = b_x B_x + t_x.$$

对应的 CUDA 表达式为

代码 3.1 一维全局工作下标

```
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

若网格在 x 方向有 $G_x = $ `gridDim.x` 个块，则启动线程覆盖的全局下标范围为

$$0 \le i < G_x B_x.$$

数据长度通常并不恰好等于 $G_x B_x$，因此实际内存访问仍需检查 i 是否小于数据长度。

3.2.3 二维全局坐标

二维数据通常用行列坐标 (r, c) 表示。CUDA 习惯让 x 方向对应列，让 y 方向对应行。设

$$b_x = \texttt{blockIdx.x},\quad B_x = \texttt{blockDim.x},\quad t_x = \texttt{threadIdx.x},$$
$$b_y = \texttt{blockIdx.y},\quad B_y = \texttt{blockDim.y},\quad t_y = \texttt{threadIdx.y}.$$

则线程负责的全局行列坐标为

$$\boxed{r = b_y B_y + t_y, \qquad c = b_x B_x + t_x}$$

即

代码 3.2 二维线程坐标到全局数据坐标的映射

```
unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
```

这两个式子彼此独立：x 方向只决定列坐标，y 方向只决定行坐标。其几何意义与一维情形相同，都是“前面完整块的跨度加上块内偏移”。

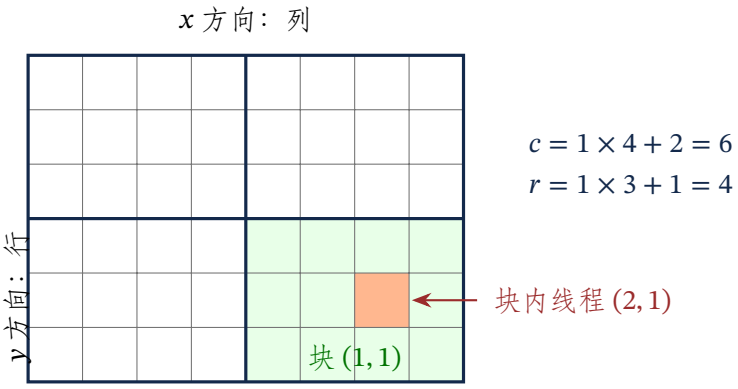


图 3.2 二维索引示例：块尺寸为 4×3，高亮线程的全局坐标为 $(r, c) = (4, 6)$

关键结论

在二维图像或矩阵核函数中，最常见的坐标约定是

$$\mathsf{x} \leftrightarrow \text{列}, \qquad \mathsf{y} \leftrightarrow \text{行}.$$

因此，宽度与 x 方向相关，高度与 y 方向相关。将二者交换会导致转置式访问、错误边界判断或越界。

3.2.4 三维全局坐标

对体数据或三维张量，可逐维应用同一构造。若把 z 方向解释为深度，则

$$\begin{aligned} c &= \texttt{blockIdx.x}\,\texttt{blockDim.x} + \texttt{threadIdx.x}, \\ r &= \texttt{blockIdx.y}\,\texttt{blockDim.y} + \texttt{threadIdx.y}, \\ d &= \texttt{blockIdx.z}\,\texttt{blockDim.z} + \texttt{threadIdx.z}. \end{aligned}$$

每个线程因此对应一个逻辑体素 (d, r, c)。各维度的计算互不干扰，边界条件也应逐维检查。

3.2.5 块数、每块线程数与总线程数

记网格尺寸为

$$(G_x, G_y, G_z) = (\texttt{gridDim.x}, \texttt{gridDim.y}, \texttt{gridDim.z}),$$

块尺寸为

$$(B_x, B_y, B_z) = (\texttt{blockDim.x}, \texttt{blockDim.y}, \texttt{blockDim.z}).$$

则

$$\begin{aligned} N_{\text{blocks}} &= G_x G_y G_z, \\ N_{\text{threads/block}} &= B_x B_y B_z, \\ N_{\text{threads,total}} &= G_x G_y G_z B_x B_y B_z. \end{aligned}$$

这些乘积计算的是*启动*的线程数量。有效工作项数量由数据尺寸决定，边界处多出的线程会通过条件判断退出。

三维逻辑坐标的线性化

多维线程索引可逐维使用；在需要给全部启动线程构造一个线性序号时，可进一步线性化。令

$$N_x = G_x B_x, \qquad N_y = G_y B_y,$$

并用 (g_z, g_y, g_x) 表示线程的三维全局坐标，则按 x 最快变化的顺序，线性线程坐标可写为

$$q = (g_z N_y + g_y) N_x + g_x.$$

该式只是坐标表示的补充。处理二维数据时，直接保留 (r, c) 通常更清晰，无需先构造 q 再反解行列。

3.2.6 每次核函数启动拥有独立配置

同一应用中的多个核函数可以使用彼此独立的网格尺寸与块尺寸。配置属于某一次核函数启动，不是整个程序的固定属性。图像处理阶段可以采用二维组织，向量阶段可以采用一维组织，体数据计算可以采用三维组织。只要每次启动时传入的配置与该核函数的工作项形状一致即可。

3.3 多维启动配置与数据布局

3.3.1 使用 dim3 指定块与网格尺寸

核函数启动语法中的两个配置参数分别表示网格尺寸和线程块尺寸：

代码 3.3 二维网格的典型启动配置

```
dim3 numThreadsPerBlock(32, 32);

dim3 numBlocks(
    (width  + numThreadsPerBlock.x - 1) / numThreadsPerBlock.x,
    (height + numThreadsPerBlock.y - 1) / numThreadsPerBlock.y);

kernel<<<numBlocks, numThreadsPerBlock>>>(/* kernel arguments */);
```

内建类型 dim3 用三个分量保存 x, y, z 方向的尺寸。上例只显式给出两个分量，因此 z 分量取 1。核函数内部读到的 blockDim 等于启动时传入的 numThreadsPerBlock，读到的 gridDim 等于 numBlocks。

二维数据的启动配置遵循以下对应关系：

$$B_x = \text{线程块宽度}, \quad B_y = \text{线程块高度},$$
$$G_x = \text{水平方向块数}, \quad G_y = \text{竖直方向块数}.$$

其中 $B_x B_y$ 是每块线程数，$G_x G_y$ 是网格中的块数。

3.3.2 向上取整除法

若数据宽度为 W，每块在 x 方向覆盖 B_x 个元素，则所需块数必须满足

$$G_x B_x \geq W.$$

取满足该条件的最小整数，得到

$$\boxed{G_x = \left\lceil \frac{W}{B_x} \right\rceil}, \qquad \boxed{G_y = \left\lceil \frac{H}{B_y} \right\rceil}.$$

对正整数，可用整数运算实现为

$$\left\lceil \frac{W}{B_x} \right\rceil = \frac{W + B_x - 1}{B_x},$$

右侧除法按整数除法截断。

整数向上取整公式为何成立

令 $W = qB_x + r$，其中 $0 \leq r < B_x$。

- 若 $r = 0$，则 $(W + B_x - 1)/B_x = q + (B_x - 1)/B_x$，整数除法结果为 q；
- 若 $r > 0$，则 $W + B_x - 1 = (q+1)B_x + (r-1)$，整数除法结果为 $q+1$。

两种情形分别对应整除与非整除，恰好等于 $\lceil W/B_x \rceil$。

3.3.3 覆盖矩形与多余线程

向上取整保证所有有效元素都被覆盖，但通常会启动一部分落在数据范围外的线程。以宽 $W = 76$、高 $H = 62$ 的图像和 16×16 线程块为例：

$$G_x = \left\lceil \frac{76}{16} \right\rceil = 5, \qquad G_y = \left\lceil \frac{62}{16} \right\rceil = 4.$$

因此启动 $5 \times 4 = 20$ 个线程块，共有

$$20 \times 16 \times 16 = 5120$$

个线程，而图像只有

$$76 \times 62 = 4712$$

个像素。多出的 $5120 - 4712 = 408$ 个线程必须通过边界条件排除。

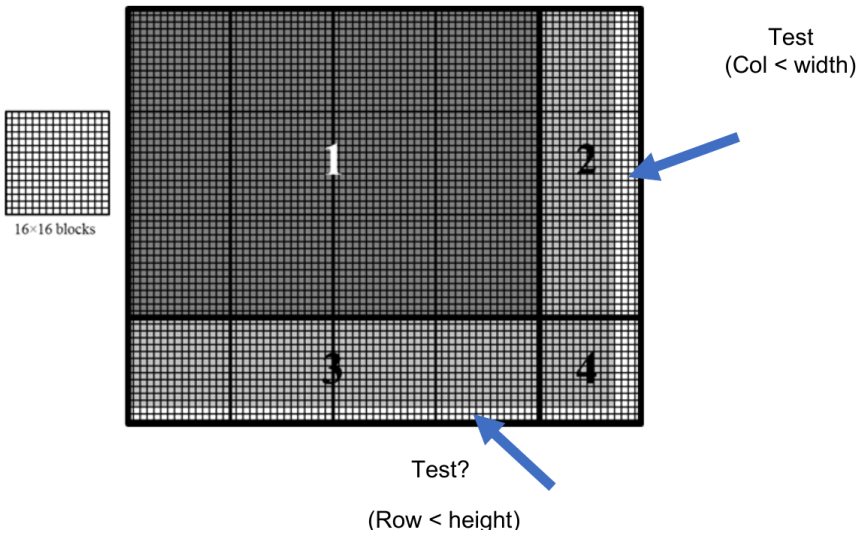


图 3.3 76×62 图像由 16×16 线程块向上取整覆盖。右侧、底部以及右下角区域包含越过图像边界的线程。

对应的二维有效性条件为

代码 3.4 二维输出坐标的边界保护

```
if (row < height && col < width) {
    // All output memory accesses using (row, col) are valid here.
}
```

需要特别注意

向上取整只保证“线程数量足够”，不保证每个线程都对应有效数据。只要数据尺寸可能不是块尺寸的整数倍，就必须在访问内存前检查每个相关维度。

3.3.4 二维数据的行主序布局

C 与 C++ 按行主序 (row-major order) 存放普通二维数组：同一行的元素在线性内存中连续，整行存完后才存下一行。设矩阵或图像共有 H 行、W 列，元素的逻辑坐标为 (r, c)，其中

$$0 \le r < H, \qquad 0 \le c < W.$$

第 r 行之前共有 r 个完整行，每行含 W 个元素，因此该行起点的线性偏移为 rW；再加上行内偏移 c，得到

$$\boxed{\text{index}(r, c) = rW + c}.$$

若数组首地址为 p、元素类型为 T，对应字节地址为

$$\mathrm{addr}(r,c) = p + (rW + c)\,\mathrm{sizeof}(T).$$

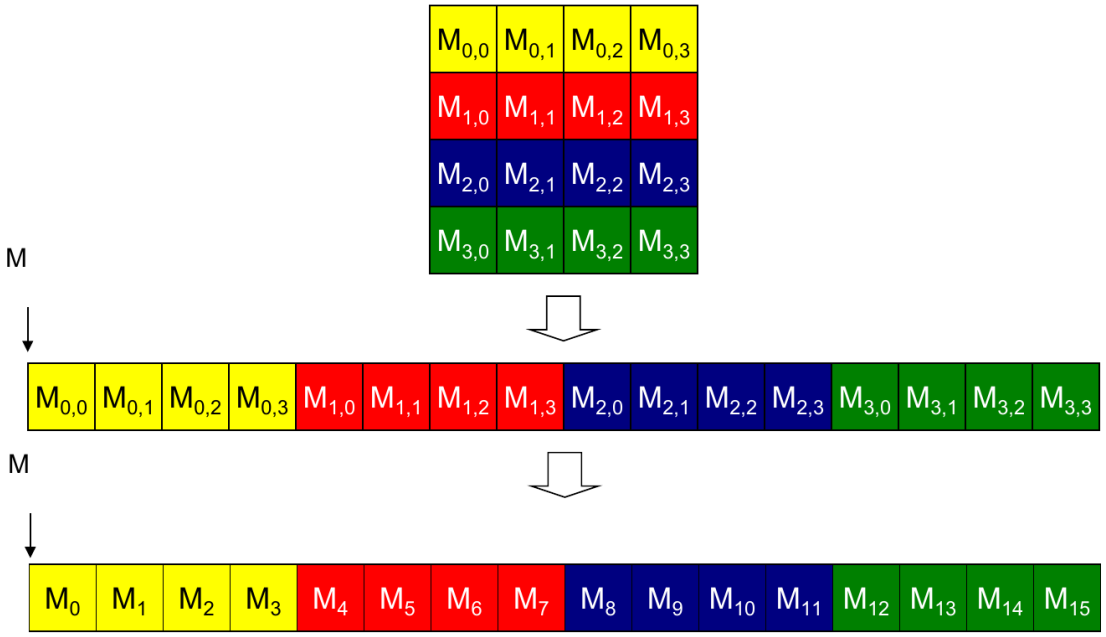


图 3.4 4×4 矩阵按行主序展开为一维连续序列。同行元素相邻，下一行紧随上一行。

从二维坐标到线性下标

对宽度 $W = 4$ 的矩阵，元素 $M_{2,1}$ 位于第 2 行、第 1 列，采用从 0 开始的下标。其线性下标为

$$\mathrm{index}(2,1) = 2 \times 4 + 1 = 9.$$

这意味着它是从线性下标 0 开始计数的第 10 个元素。

行主序公式还直接给出两个局部关系：

$$\mathrm{index}(r, c+1) = \mathrm{index}(r,c) + 1,$$

说明同一行相邻列在内存中相邻；而

$$\mathrm{index}(r+1, c) = \mathrm{index}(r,c) + W,$$

说明同一列相邻行之间相隔 W 个元素。

3.3.5 三维数据的线性化补充

若三维数据尺寸为深度 D、高度 H、宽度 W，并采用 x 对应列、y 对应行、z 对应深度的约定，则可先将 (d,r,c) 看成第 d 个二维平面中的位置。每个平面含 HW 个元素，因此

$$\boxed{\text{index}(d,r,c) = (dH + r)W + c = dHW + rW + c}.$$

该式是二维行主序的逐层扩展。核函数仍应先验证

$$0 \le d < D, \qquad 0 \le r < H, \qquad 0 \le c < W,$$

再使用线性下标访问内存。

关键结论

二维核函数的标准地址链条为

$$(\texttt{blockIdx}, \texttt{threadIdx}) \longrightarrow (r,c) \longrightarrow rW + c \longrightarrow \text{内存访问}.$$

每一步使用不同信息：线程层次给出逻辑坐标，数据布局给出线性偏移，边界条件保证该偏移有效。

3.4 实例一：RGB 彩色图像转灰度

3.4.1 数据并行分解

红绿蓝 (red-green-blue, RGB) 图像的每个像素包含三个颜色分量。三个独立的一维数组 `red`、`green` 和 `blue` 分别保存三个输入分量，数组 `gray` 保存灰度输出。四个数组的长度均为 WH，其中 W 为图像宽度，H 为图像高度。

每个输出像素只依赖同一位置的三个颜色分量，不依赖其他像素。因此可把输出空间直接作为并行工作空间：

$$\boxed{\text{一个线程} \longleftrightarrow \text{一个输出像素}}.$$

线程在二维网格中先求自己负责的 (r, c)，再用 $p = rW + c$ 访问三个输入通道并写入一个输出值。

设某像素的红、绿、蓝分量分别为 R_p, G_p, B_p，灰度计算为

$$\boxed{Y_p = \frac{3}{10}R_p + \frac{6}{10}G_p + \frac{1}{10}B_p}.$$

三个系数之和为 1，因此当三个输入均位于 $[0, 255]$ 时，浮点中间结果也位于该区间内。

单个像素的灰度计算

若某像素满足

$$R_p = 100, \qquad G_p = 150, \qquad B_p = 200,$$

则

$$Y_p = 0.3 \times 100 + 0.6 \times 150 + 0.1 \times 200 = 30 + 90 + 20 = 140.$$

该像素的三个输入值由一个线程读取，结果 140 由同一线程写入对应的灰度位置。

3.4.2 完整核函数

核函数使用三个独立通道数组与上述权重。线性下标 `pix` 在边界判断通过后计算，使边界保护与内存访问的对应关系更直接。

代码 3.5 带二维边界保护的 RGB 转灰度核函数

```
__global__
void rgb2gray_kernel(unsigned char* red,
                     unsigned char* green,
                     unsigned char* blue,
                     unsigned char* gray,
                     unsigned int width,
                     unsigned int height)
{
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < height && col < width) {
        unsigned int pix = row * width + col;

        gray[pix] = red[pix]   * 3.0f / 10.0f
                  + green[pix] * 6.0f / 10.0f
                  + blue[pix]  * 1.0f / 10.0f;
    }
}
```

后缀 `f` 表示单精度浮点字面量。乘加表达式先产生浮点中间结果，再在赋给 `unsigned char` 时转换为字节值。源实现未加入额外舍入步骤，因此若结果带小数部分，赋值转换会去除其小数部分。

3.4.3 启动配置

采用 16×16 线程块时，二维启动配置可写为

代码 3.6 RGB 转灰度核函数的二维启动配置

```
dim3 threadsPerBlock(16, 16);
dim3 numBlocks(
    (width  + threadsPerBlock.x - 1) / threadsPerBlock.x,
    (height + threadsPerBlock.y - 1) / threadsPerBlock.y);

rgb2gray_kernel<<<numBlocks, threadsPerBlock>>>(
    red, green, blue, gray, width, height);
```

该配置创建的矩形线程区域至少覆盖整个图像。右边界与下边界多出的线程仍会执行到条件判断，但不会读取或写入任何像素。

3.4.4 一次线程索引的完整追踪

考虑宽 $W = 76$、高 $H = 62$ 的图像，块尺寸为 16×16。某线程满足

$$\texttt{blockIdx} = (3, 2), \qquad \texttt{threadIdx} = (5, 7).$$

其全局列、行坐标分别为

$$c = 3 \times 16 + 5 = 53,$$
$$r = 2 \times 16 + 7 = 39.$$

因为 $53 < 76$ 且 $39 < 62$，该线程对应有效像素。线性下标为

$$p = 39 \times 76 + 53 = 3017.$$

于是该线程读取

$$\texttt{red[3017]},\ \texttt{green[3017]},\ \texttt{blue[3017]},$$

并写入 `gray[3017]`。

若同一块中的线程改为 `threadIdx.y=15`，则行坐标变为

$$r = 2 \times 16 + 15 = 47,$$

仍然有效；而位于最后一行块中的部分线程会得到 $r \geq 62$，这些线程由外层条件排除。

关键结论

该实例包含一条完整的二维逐元素模式：

二维线程坐标

⇓

像素行列坐标(r, c)

⇓

线性下标$p = rW + c$

⇓

三个输入读取与一个输出写入.

所有四次内存访问共享同一个有效性条件 $r < H \wedge c < W$。

3.5 实例二：邻域平均模糊

3.5.1 从逐元素映射到邻域映射

灰度图像模糊仍采用“一个线程生成一个输出像素”的并行分解，但一个输出不再只依赖一个输入位置。设模糊半径为

$$R = \texttt{BLUR_SIZE}.$$

输出像素 (r, c) 的名义邻域为

$$\mathcal{N}_R(r,c) = \{(i,j) \mid r - R \le i \le r + R,\ c - R \le j \le c + R\}.$$

若该邻域完全位于图像内部，其中包含

$$(2R+1)^2$$

个输入像素。当 $R = 1$ 时，内部像素使用 3×3 邻域，共 9 个输入。

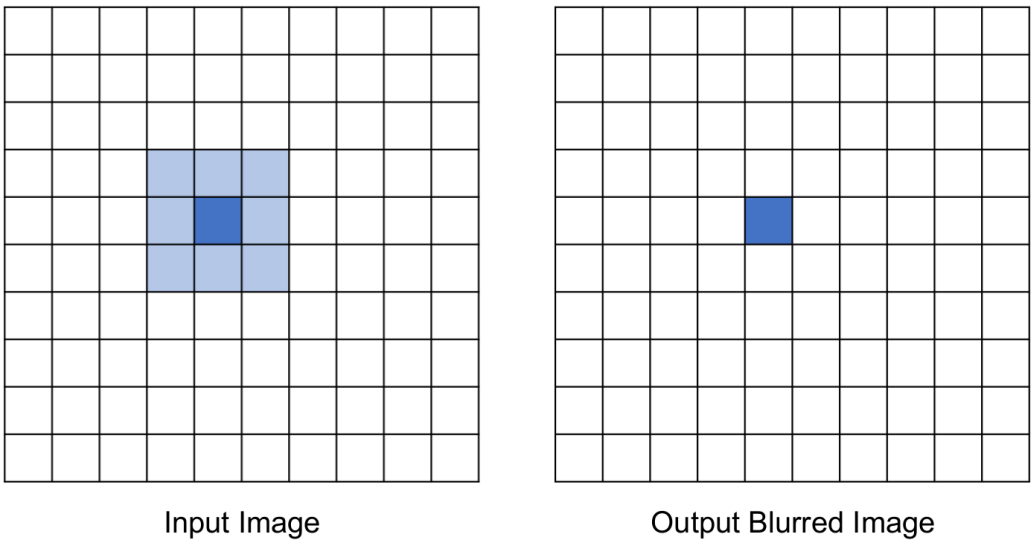


图 3.5 一个线程负责一个输出像素，并读取该位置周围的多个输入像素。图中半径为 1。

设图像有效区域为

$$\Omega = \{(i,j) \mid 0 \le i < H,\ 0 \le j < W\}.$$

真正参与边界像素计算的集合是

$$\mathcal{V}_R(r,c) = \mathcal{N}_R(r,c) \cap \Omega.$$

因此输出应定义为有效邻域的平均值：

$$\boxed{B_{r,c} = \frac{1}{|\mathcal{V}_R(r,c)|} \sum_{(i,j)\in\mathcal{V}_R(r,c)} I_{i,j}}.$$

3.5.2 核函数中的双重循环

每个线程知道自己的输出坐标 (outRow, outCol) 后，需要枚举邻域的所有候选输入坐标。二维邻域自然对应两层循环：外层改变输入行，内层改变输入列。

代码 3.7 未处理输入边界的基本邻域遍历

```
unsigned int sum = 0;

for (int inRow = outRow - BLUR_SIZE;
     inRow < outRow + BLUR_SIZE + 1;
     ++inRow) {
    for (int inCol = outCol - BLUR_SIZE;
         inCol < outCol + BLUR_SIZE + 1;
         ++inCol) {
        sum += image[inRow * width + inCol];
    }
}
```

该代码只对完全位于图像内部的输出像素安全。即使输出坐标本身有效，邻域中的 inRow 或 inCol 仍可能越界。例如，左上角输出 $(0,0)$ 在 $R=1$ 时会尝试访问行 -1 或列 -1。

3.5.3 两层边界条件

模糊核函数中存在两类不同的内存访问，因而需要两层保护：

1. **输出保护**：验证线程负责的输出坐标满足 $0 \le \texttt{outRow} < H$ 且 $0 \le \texttt{outCol} < W$，从而保证最终写入有效；
2. **输入保护**：对循环中每一个候选邻域坐标验证 $0 \le \texttt{inRow} < H$ 且 $0 \le \texttt{inCol} < W$，从而保证每次读取有效。

内存访问规则可形式化为：

每次内存访问都必须有对应的索引保护

对任意数组访问 `array[index]`，必须能够指出一条在该访问之前成立的条件，并由该条件推出 `index` 位于数组合法范围内。模糊核函数的输出写入由外层条件保护，输入读取由内层条件保护。

邻域索引使用有符号整数 `int`。这样，顶边和左边附近的候选坐标可以自然取负值，并由 `inRow >= 0` 与 `inCol >= 0` 排除。若这些变量使用无符号类型，减去模糊半径可能发生下溢，使边界逻辑难以按预期工作。

3.5.4 固定分母为何会产生错误

只跳过越界输入还不够。若始终除以名义邻域大小 $(2R+1)^2$，边界处没有被加入求和的像素仍被隐含地计入分母，相当于把缺失像素按 0 处理，输出会偏小。

当 $R = 1$ 时：

- 内部像素有 9 个有效邻居，分母为 9；
- 非角点边缘像素通常有 6 个有效邻居，分母应为 6；
- 角点像素有 4 个有效邻居，分母应为 4。

因此必须在累加有效像素时同步维护计数 `count`，最后使用 `sum / count`。

角点像素的有效邻域

考虑 3×3 灰度块

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}.$$

当 $R = 1$ 时，中心像素的九点平均为 50。左上角输出只使用

$$10,\ 20,\ 40,\ 50,$$

所以

$$\texttt{sum} = 120, \qquad \texttt{count} = 4, \qquad B_{0,0} = 120/4 = 30.$$

若错误地除以 9，则得到 13，明显把边界亮度压低。

3.5.5 完整核函数

代码 3.8 对有效邻域求平均的完整灰度模糊核函数

```
#define BLUR_SIZE 1

__global__
void blur_kernel(unsigned char* image,
                 unsigned char* blurred,
                 unsigned int width,
                 unsigned int height)
{
    int outRow = blockIdx.y * blockDim.y + threadIdx.y;
    int outCol = blockIdx.x * blockDim.x + threadIdx.x;

    if (outRow < height && outCol < width) {
        unsigned int sum = 0;
        unsigned int count = 0;

        for (int inRow = outRow - BLUR_SIZE;
             inRow < outRow + BLUR_SIZE + 1;
             ++inRow) {
            for (int inCol = outCol - BLUR_SIZE;
                 inCol < outCol + BLUR_SIZE + 1;
                 ++inCol) {
                if (inRow >= 0 && inRow < height &&
                    inCol >= 0 && inCol < width) {
                    sum += image[inRow * width + inCol];
                    ++count;
                }
            }
        }

        blurred[outRow * width + outCol]
            = (unsigned char)(sum / count);
    }
}
```

对任何通过外层条件的线程，中心输入 (outRow, outCol) 一定有效，因此 count 至少为 1，最终除法不会除以 0。

3.5.6 控制流与工作量

所有线程执行同一双重循环结构，但边界线程的内层条件只有部分迭代成立。对半径 R，每个有效输出线程检查 $(2R+1)^2$ 个候选位置；实际读取次数等于 $|\mathcal{V}_R(r,c)|$。这说明核函数内部可以包含普通控制结构：循环负责枚举一个线程的局部工作集，条件判断负责过滤无效位置。

关键结论

RGB 转灰度与模糊使用相同的输出并行分解，但单线程输入工作集不同：

计算	每线程输出	每线程输入读取
RGB 转灰度	1 个灰度像素	3 个同位置通道值
邻域模糊	1 个模糊像素	$\lvert\mathcal{V}_R(r,c)\rvert$ 个邻域像素

线程数量由输出空间决定，单线程内部的循环由输入依赖范围决定。

3.6 实例三：矩阵与矩阵乘法

3.6.1 输出元素的定义

设方阵 $A, B \in \mathbb{R}^{N\times N}$，其乘积记为

$$C = AB.$$

采用从 0 开始的下标，输出元素 $C_{r,c}$ 是 A 的第 r 行与 B 的第 c 列的内积：

$$\boxed{C_{r,c} = \sum_{k=0}^{N-1} A_{r,k} B_{k,c}}.$$

固定 (r,c) 后，求和变量 k 枚举共享维度。不同输出位置写入 C 的不同元素，因此可以并行计算：

$$\boxed{\text{一个线程} \longleftrightarrow \text{一个输出矩阵元素}}.$$

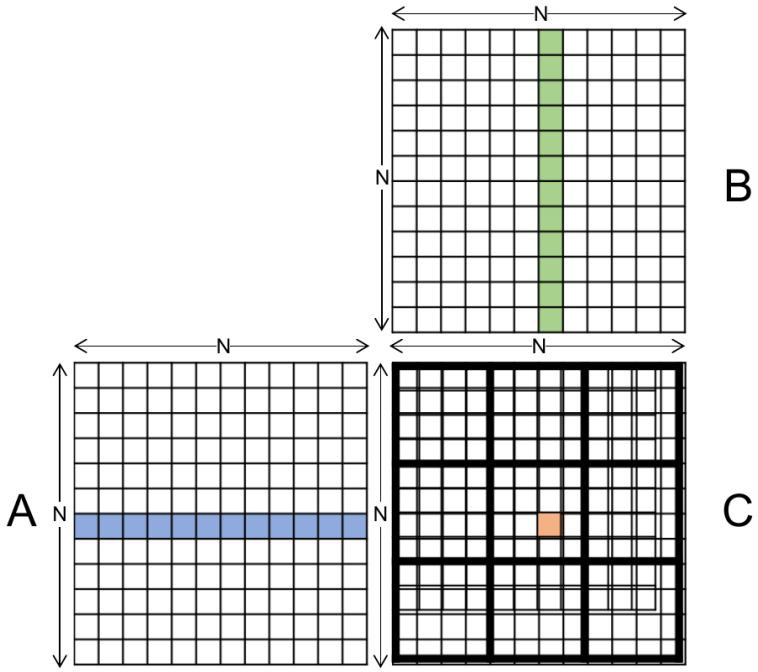


图 3.6 输出元素由 A 的一行与 B 的一列做逐项乘积并求和得到；二维线程网格覆盖输出矩阵。

这种分解与图像模糊具有相同的外层结构：线程坐标决定一个输出位置，线程内部再用循环遍历生成该输出所需的多个输入。区别在于，模糊遍历二维局部邻域，矩阵乘法遍历长度为 N 的共享维度。

3.6.2 简化方阵核函数

方阵情形可使用以下简化实现：

代码 3.9 简化的方阵乘法核函数

```
__global__
void mm_kernel(float* A, float* B, float* C,
               unsigned int N)
{
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for (unsigned int k = 0; k < N; ++k) {
        sum += A[row * N + k] * B[k * N + col];
    }

    C[row * N + col] = sum;
}
```

代码中三个线性下标分别对应

$$A_{r,k} \longleftrightarrow \texttt{A[row * N + k]},$$
$$B_{k,c} \longleftrightarrow \texttt{B[k * N + col]},$$
$$C_{r,c} \longleftrightarrow \texttt{C[row * N + col]}.$$

该简化版本未包含边界条件。只有在启动线程区域恰好不越过 $N \times N$ 输出范围，或调用者另有保证时，所有访问才有效；一般实现必须显式处理边界。

3.6.3 一般矩形矩阵

矩阵不必为方阵。设

$$A \in \mathbb{R}^{M \times K}, \qquad B \in \mathbb{R}^{K \times N}.$$

两者共享维度均为 K，输出为

$$C = AB \in \mathbb{R}^{M \times N},$$

并满足

$$\boxed{C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}}, \qquad 0 \le r < M, \quad 0 \le c < N.$$

这里三个矩阵的行主序宽度不同：A 的每行宽度为 K，B 与 C 的每行宽度为 N。因此正

确线性下标为

$$\text{index}_A(r,k) = rK + k,$$
$$\text{index}_B(k,c) = kN + c,$$
$$\text{index}_C(r,c) = rN + c.$$

把所有宽度都写成同一个 N 只适用于方阵情形。

带边界条件的矩形版本

将维度关系推广为 $M \times K$、$K \times N$ 和 $M \times N$，并加入边界条件，可得到：

```
__global__
void mm_kernel_rect(float* A, float* B, float* C,
                    unsigned int M,
                    unsigned int K,
                    unsigned int N)
{
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float sum = 0.0f;
        for (unsigned int k = 0; k < K; ++k) {
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

外层条件同时保护 A 的行索引、B 的列索引与 C 的写入位置；循环条件 $k < K$ 保护共享维度上的两次读取。

3.6.4 网格尺寸由输出矩阵决定

由于每个线程生成一个 C 元素，线程网格应覆盖 $M \times N$ 输出空间。对块尺寸 (B_x, B_y)，启动块数为

$$G_x = \left\lceil \frac{N}{B_x} \right\rceil, \qquad G_y = \left\lceil \frac{M}{B_y} \right\rceil.$$

注意 x 方向对应输出列数 N，y 方向对应输出行数 M；共享维度 K 决定每个线程的循环长度，不决定二维网格的外部尺寸。

代码 3.10 矩形矩阵乘法的启动配置

```
dim3 threadsPerBlock(16, 16);
dim3 numBlocks(
    (N + threadsPerBlock.x - 1) / threadsPerBlock.x,
    (M + threadsPerBlock.y - 1) / threadsPerBlock.y);

mm_kernel_rect<<<numBlocks, threadsPerBlock>>>(A, B, C, M, K, N);
```

输出空间决定启动空间

若 A 为 5×3，B 为 3×4，则 C 为 5×4。采用 2×2 线程块时，

$$G_x=\left\lceil\frac{4}{2}\right\rceil=2,\qquad G_y=\left\lceil\frac{5}{2}\right\rceil=3.$$

共启动 2×3 个块、24 个线程，其中 20 个线程对应有效输出元素，最后一行块中有 4 个线程因 $r\geq5$ 被排除。每个有效线程执行 $K=3$ 次乘加迭代。

3.6.5 单线程工作量与访问方式

每个输出元素的内积包含 K 次乘法和 $K-1$ 次数学求和，核函数循环执行 K 次累加。全部 MN 个输出元素的算术规模随 MNK 增长。

从行主序下标可直接观察两种输入访问：

- 当 k 增加 1 时，`A[row * K + k]` 的下标也增加 1，线程沿 A 的一行连续读取；
- 当 k 增加 1 时，`B[k * N + col]` 的下标增加 N，线程沿 B 的一列跨行读取。

相邻输出线程会重复需要 A 的同一行片段或 B 的同一列片段，但简化核函数直接从全局内存逐次读取，没有显式利用这些重用关系。因此，该实现的主要问题是性能：计算单元可能花费大量时间等待数据，形成以内存访问为主的瓶颈。

关键结论

矩阵乘法的三个维度承担不同角色：

维度	作用
M	输出行数，也决定网格的 y 向覆盖范围
N	输出列数，也决定网格的 x 向覆盖范围
K	内积长度，也决定每个线程的循环次数

混淆这些维度会直接造成错误的启动配置或错误的行主序地址。

3.7 例题与完整推导

3.7.1 统一计算框架

对三维网格尺寸

$$(G_x, G_y, G_z)$$

与三维块尺寸

$$(B_x, B_y, B_z),$$

依次计算

$$N_{\text{threads/block}} = B_x B_y B_z,$$
$$N_{\text{blocks}} = G_x G_y G_z,$$
$$N_{\text{threads,total}} = N_{\text{threads/block}} N_{\text{blocks}}.$$

未使用的维度取 1，不能把它们从概念上混成 0；尺寸为 0 会意味着不启动相应工作，而一维或二维配置的剩余维度实际为 1。

3.7.2 例题一：一维网格与二维线程块

题目

一个 CUDA 核函数启动 32 个线程块，每个线程块包含 32×32 个线程。求每块线程数与总启动线程数。

网格可写为

$$\texttt{gridDim} = (32, 1, 1),$$

线程块可写为

$$\texttt{blockDim} = (32, 32, 1).$$

因此每块线程数为

$$N_{\text{threads/block}} = 32 \times 32 \times 1 = 1024.$$

网格中的块数为

$$N_{\text{blocks}} = 32 \times 1 \times 1 = 32.$$

总线程数为

$$N_{\text{threads,total}} = 1024 \times 32 = 32768.$$

答案

每个线程块包含 1024 个线程；整个网格共启动 32768 个线程。

3.7.3 例题二：二维网格与二维线程块

题目

线程块配置为 16×16，网格尺寸为 32×16 个块。求每块线程数、块总数与整个网格的线程总数。

对应尺寸为

$$\texttt{blockDim} = (16, 16, 1), \qquad \texttt{gridDim} = (32, 16, 1).$$

每块线程数为

$$N_{\text{threads/block}} = 16 \times 16 \times 1 = 256.$$

块总数为

$$N_{\text{blocks}} = 32 \times 16 \times 1 = 512.$$

因此总线程数为

$$N_{\text{threads,total}} = 256 \times 512 = 131072.$$

也可一次性验证：

$$(32 \times 16) \times (16 \times 16) = 131072.$$

答案

每块包含 256 个线程；网格含 512 个块；整个网格共启动 131072 个线程。

3.7.4 例题三：三维网格与三维线程块

题目

网格尺寸为 $8 \times 4 \times 2$ 个块，每个线程块尺寸为 $4 \times 8 \times 8$。求每块线程数与整个网格的线程总数。

每块线程数为

$$N_{\text{threads/block}} = 4 \times 8 \times 8 = 256.$$

网格中的块数为

$$N_{\text{blocks}} = 8 \times 4 \times 2 = 64.$$

总线程数为

$$N_{\text{threads,total}} = 256 \times 64 = 16384.$$

答案

每块包含 256 个线程；网格含 64 个块；整个网格共启动 16384 个线程。

3.7.5 综合例题：最后一个有效像素与第一个无效线程

考虑 76×62 图像，块尺寸为 16×16，因此

$$\texttt{gridDim} = (5, 4, 1).$$

图像最后一个有效像素坐标为

$$(r, c) = (61, 75).$$

取最后一个线程块

$$\texttt{blockIdx} = (4, 3),$$

并取块内线程

$$\texttt{threadIdx} = (11, 13).$$

其坐标为

$$c = 4 \times 16 + 11 = 75,$$

$$r = 3 \times 16 + 13 = 61.$$

因此该线程恰好负责最后一个有效像素，线性下标为

$$p = 61 \times 76 + 75 = 4711,$$

与长度 $76 \times 62 = 4712$ 的数组最后下标一致。

若只把块内 x 坐标增加 1，即取 `threadIdx.x=12`，则

$$c = 4 \times 16 + 12 = 76.$$

此时 $c < 76$ 不成立，该线程是最靠近右边界的无效线程之一。这个例子说明，线程的块内索引本身完全合法，并不意味着它对应的数据坐标合法；数据边界必须单独判断。

3.7.6 常见计算错误

- 把“网格中的块数”与“每块线程数”混为同一个量；
- 只乘二维图中显式写出的分量，却忘记未使用维度应乘以 1；
- 把 `gridDim` 当成线程总数，或把 `blockDim` 当成块总数；
- 认为启动线程总数必然等于有效数据元素数，忽略向上取整产生的边界线程；
- 在二维索引中把 x 当作行、y 当作列，导致宽度与高度的判断互换。

3.8 统一方法与关键结论

3.8.1 从输出空间建立并行映射

三个实例都可用同一建模步骤构造。

1. **确定输出空间。**向量输出是一维，图像与矩阵输出通常是二维，体数据输出可以是三维。
2. **确定单线程输出。**一个线程生成一个输出元素，使不同线程写入互不重叠的位置。
3. **选择线程块形状。**块的逻辑维度与输出坐标轴对应，二维情形通常令 x 对应列、y 对应行。

4. **向上取整计算网格。**每一维都用数据长度除以对应块尺寸并向上取整。
5. **计算全局逻辑坐标。**逐维使用“块坐标乘块尺寸加块内线程坐标”。
6. **建立边界保护。**输出写入与每一类输入读取都必须由可证明其下标有效的条件保护。
7. **线性化并计算。**按行主序把逻辑坐标转换为线性偏移，再执行逐元素运算或线程内部循环。

二维输出核函数的通用骨架为

代码 3.11 二维逐输出元素核函数骨架

```
__global__
void kernel(/* arrays and parameters */, unsigned int width,
            unsigned int height)
{
    unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < height && col < width) {
        unsigned int out = row * width + col;

        // Read the valid input working set for (row, col).
        // Compute one output value.
        // Write the result to output[out].
    }
}
```

该骨架只规定输出映射。输入工作集由具体算法决定：可以是同位置的若干通道、一个局部邻域，也可以是一整行与一整列。

3.8.2 三个实例的结构对照

表 3.2 RGB 转灰度、模糊与矩阵乘法的并行结构

计算	输出空间	单线程输入工作集	线程内控制结构
RGB 转灰度	$H \times W$ 灰度图像	同一像素位置的红、绿、蓝三个分量	固定数量的乘加，无循环
邻域模糊	$H \times W$ 模糊图像	以输出像素为中心的有效邻域 $\mathcal{V}_R(r,c)$	两层循环与输入边界条件
矩阵乘法	$M \times N$ 输出矩阵	A 的第 r 行与 B 的第 c 列，共享长度为 K	一层长度为 K 的累加循环

三者的线程数量都由输出空间决定，而单线程工作量由生成一个输出所需的输入数量决定。这一区分有助于判断某个维度应放在网格中，还是应由线程内部循环遍历：

- 输出坐标通常映射到并行线程；
- 单个输出内部的归约维度通常映射到线程内循环。

3.8.3 关键公式汇总

一维全局下标：
$$i = \texttt{blockIdx.x}\,\texttt{blockDim.x} + \texttt{threadIdx.x},$$

二维全局坐标：
$$r = \texttt{blockIdx.y}\,\texttt{blockDim.y} + \texttt{threadIdx.y},$$
$$c = \texttt{blockIdx.x}\,\texttt{blockDim.x} + \texttt{threadIdx.x},$$

二维行主序：
$$p = rW + c,$$

二维块数：
$$G_x = \left\lceil \frac{W}{B_x} \right\rceil, \qquad G_y = \left\lceil \frac{H}{B_y} \right\rceil,$$

总启动线程数：
$$N_{\text{threads,total}} = (G_x G_y G_z)(B_x B_y B_z),$$

矩形矩阵乘法：
$$C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}.$$

3.8.4 正确性检查表

1. 网格的每个维度是否按对应输出尺寸向上取整？
2. 是否始终使用 x 对应列、y 对应行的同一约定？
3. 全局坐标是否包含块偏移与块内偏移两部分？
4. 行主序公式中的宽度是否属于正在访问的那个数组？
5. 输出写入之前是否检查了所有输出坐标？
6. 每次输入读取之前是否检查了该输入访问涉及的全部坐标？
7. 邻域循环是否使用能够表示负候选坐标的有符号索引？

8. 边界处的平均值是否除以实际参与求和的元素数？

9. 矩阵乘法中，输出维度 M, N 与归约维度 K 是否各自使用在正确位置？

关键结论

多维网格的核心作用是把数据的几何结构直接编码到线程坐标中。正确核函数由三条关系组成：

线程层次 → 逻辑输出坐标，　　逻辑坐标 → 线性内存地址，　　有效性条件 → 安全访问.

循环与条件控制单个线程的局部工作集，不改变所有线程共享同一核函数代码这一单程序多数据模型。

第 4 讲

Compute Architecture and Scheduling

4.1 核心内容

内容概览

- 理解 GPU 架构与组织的基础；
- 理解线程调度、延迟容忍与硬件占用率；
- 区分驻留线程束与当前实际发射指令的线程束。

4.1.1 软件执行层次

CUDA 的软件执行层次由网格、线程块和线程组成。网格中的所有线程运行同一份核函数代码，每个线程通过线程索引和线程块索引选择自身负责的数据与工作。这种执行方式属于**单程序多数据** (*Single Program, Multiple Data, SPMD*) 模型。

线程块可以组织成一维、二维或三维形状。多维组织使线程索引能够自然对应多维数据，从而简化地址计算；维度本身并不会自动提高访存效率或计算性能。例如，二维图像既可以使用二维线程块，也可以使用一维线程块，后一种方式需要显式完成一维索引到二维坐标的映射。

关键结论

线程块的形状用于表达问题结构；硬件执行方式取决于线程块如何驻留到流式多处理器、如何拆分为线程束，以及线程束如何被调度。

4.1.2 线程块的三个基本语义

1. **线程块是资源分配与驻留的基本单位。**同一线程块的全部线程被分配到同一个流式多处理器；一个线程块不会跨越多个流式多处理器执行。
2. **同一线程块内部可以高效协作。**线程可以使用共享内存，并通过线程块级屏障进行同步。
3. **不同线程块的执行顺序没有保证。**核函数必须在不依赖线程块调度顺序的前提下得到正确结果；同一程序由此能够在硬件并行度不同的 GPU 上透明伸缩。

4.1.3 软件层次到硬件层次的映射

软件执行层次与硬件执行资源之间的基本关系为

$$\boxed{\text{网格} \to \text{线程块} \to \text{流式多处理器驻留} \to \text{线程束} \to \text{处理块发射与执行}}.$$

线程束(*warp*)是硬件调度的基本单位。线程块驻留到某个流式多处理器后，其线程按照线性化线程索引划分为若干线程束；截至 Hopper 的所列 NVIDIA 架构均采用 32 线程的线程束。控制发散、延迟隐藏与占用率都以线程束为核心进行分析。

4.2 GPU 计算架构与线程块驻留

4.2.1 流式多处理器是主要硬件组织单元

一块 GPU 由多个**流式多处理器**(*Streaming Multiprocessor, SM*)组成。每个流式多处理器内部包含若干执行核心、控制资源和片上存储资源；多个流式多处理器通过更高层次的存储系统访问全局内存(global memory)。以 Hopper H100 的架构示意为例，144 个流式多处理器、每个流式多处理器 128 个核心，对应

$$144 \times 128 = 18\,432$$

个核心。

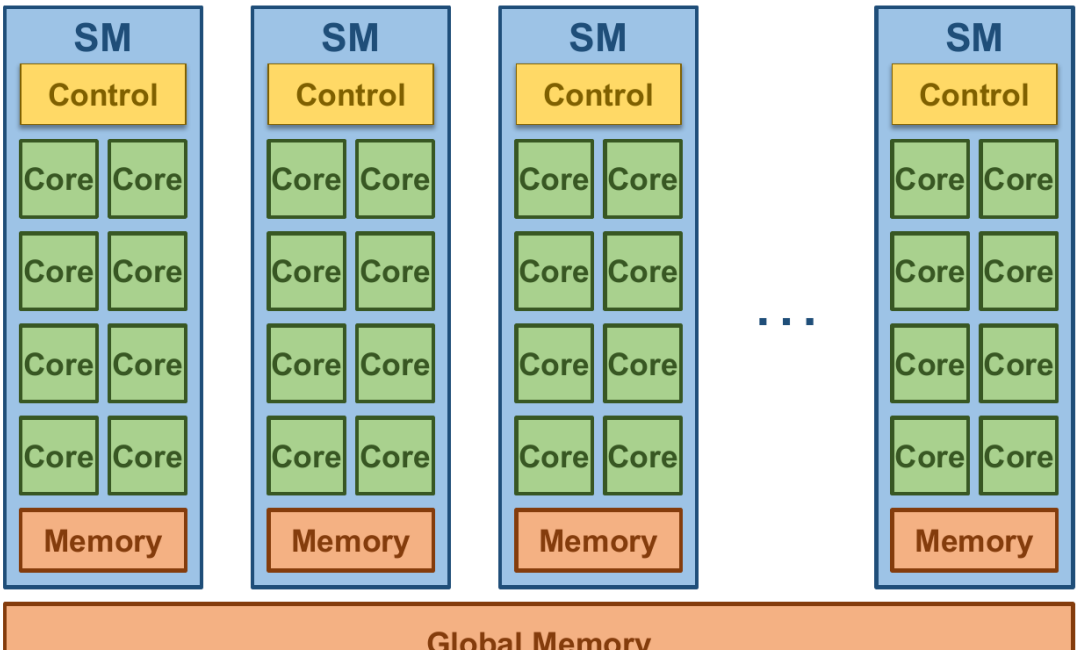


图 4.1 多个流式多处理器共享更高层次的全局内存；每个流式多处理器内部具有控制、执行核心和片上存储资源

定义

流式处理器(*Streaming Processor, SP*) 或 CUDA 核心表示流式多处理器内部的标量执行核心。流式多处理器负责维护大量驻留线程的执行状态并调度线程束；流式处理器是实际完成相应算术工作的执行资源之一。两者不能用同一个“核心数”概念替代。

4.2.2 线程块以整体形式分配到一个流式多处理器

CUDA 线程不是逐个、独立地被分配到任意流式多处理器。硬件以**线程块粒度**完成驻留分配：一个线程块的全部线程都属于同一个流式多处理器，并在该流式多处理器上保持驻留直到线程块完成。一个线程块不能拆到多个流式多处理器上。

多个线程块可以同时驻留在同一个流式多处理器上，但前提是它们共同占用的硬件资源没有超过该流式多处理器的限制。尚未获得驻留资源的线程块处于等待状态；当已有线程块完成并释放资源后，等待线程块才可能获得驻留机会。

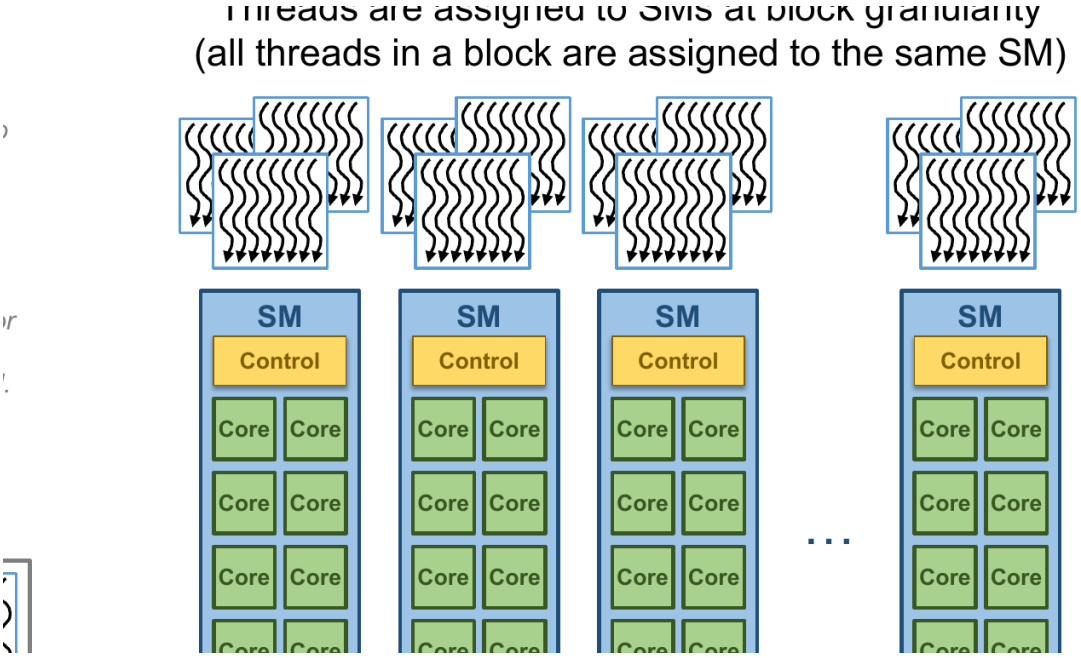


图 4.2 线程块以整体形式分配到流式多处理器；超出当前驻留能力的线程块必须等待资源释放

这里必须区分**驻留**和**正在执行**。假设某个流式多处理器能够容纳 2048 个驻留线程，并不意味着这 2048 个线程在同一个时钟周期中都各自占用一个物理执行核心。一个流式多处理器的物理执行核心数通常远小于可驻留线程数；驻留线程会进一步组成线程束，由调度器选择其中已经具备执行条件的线程束发射指令。

关键结论

“线程块被分配到某个流式多处理器”表示该线程块所需的执行状态和资源已在该流式多处理器上建立，并不表示线程块中的所有线程同时占用独立核心执行。

4.2.3 为什么同一线程块必须留在同一个流式多处理器

同一线程块中的线程依赖两类低开销协作资源：

- **共享内存**（*shared memory*）：物理资源位于流式多处理器上，但逻辑上按线程块分配；同一线程块中的线程可以访问该线程块的共享内存。
- 线程块级屏障：同一线程块的线程可以使用 `__syncthreads()` 等机制等待彼此达到同步点。

如果一个线程块被拆分到不同流式多处理器，这些低开销机制就失去统一的本地硬件基础。因此，线程块边界既是编程模型中的协作边界，也是硬件资源分配的重要边界。

不同线程块之间如果需要交换数据，通常必须依赖全局可见的存储与更高层次的同步机制。其成本和约束都显著高于线程块内部协作，因此 CUDA 的基本执行模型要求普通线程块之间不依赖具体的调度顺序。

4.2.4 驻留需要同时满足多类资源限制

线程块获得驻留位置之前，流式多处理器必须能够为整个线程块预留足够资源。主要约束包括：

- 每线程块最大线程数；
- 每流式多处理器最大驻留线程数；
- 每流式多处理器最大驻留线程块数；
- 寄存器文件容量；
- 共享内存容量；
- 最大驻留线程束数等架构限制。

因此，“线程数还放得下”只是必要条件之一。例如，一个线程块可能在线程数方面合法，但其寄存器或共享内存需求使得同一个流式多处理器只能驻留很少的线程块。后面的占用率分析将把这些约束统一起来。

4.3 线程块内同步、跨线程块同步与可伸缩性

4.3.1 线程块级屏障的语义

同一线程块内最基本的屏障同步由 `__syncthreads()` 提供。其核心语义是：到达该屏障的线程必须等待，直到线程块中所有尚未退出且应参与该屏障的线程都一致地到达同一个屏障实例，然后才能继续执行。

机制

线程块级屏障之所以可以高效实现，是因为同一线程块的全部驻留状态位于同一个流式多处理器中。屏障除了协调控制进度，还会对相关的共享内存和全局内存访问建立必要的顺序关系，使屏障之前的相关写入在屏障之后按照该同步语义被观察。

4.3.2 发散条件中的屏障为什么危险

下面的结构存在根本问题：

代码 4.1 发散条件中的两个不同屏障实例

```
if (predicate) {
    __syncthreads();
} else {
    __syncthreads();
}
```

表面上看，两条分支都出现了 `__syncthreads()`；但如果线程块中的不同线程沿不同分支前进，它们到达的可能是两个不同的静态屏障实例。这样无法保证全部参与线程在同一个屏障上会合，结果可能造成错误甚至死锁。

正确性要求可以概括为：对于某一个线程块级屏障实例，所有应参与的线程必须以兼容、可达且一致的方式到达它。屏障不能依赖会让同一线程块中的线程分成互不相交路径的控制结构。

需要特别注意

“每条分支里都写一个屏障”不等价于“所有线程到达同一个屏障”。判断同步是否正确，必须看线程是否会合到同一个屏障实例，而不是只看源代码中是否都出现了同名函数。

4.3.3 为什么线程块必须一次性获得完整驻留资源

线程块不能只让一部分线程先驻留，剩余线程等以后再分配。原因可以用屏障直接解释：假设线程块前半部分已经驻留并执行到屏障，而后半部分因为资源不足根本无法开始执行，那么前半部分会永久等待后半部分；同时前半部分占据的资源又可能阻止后半部分获得驻留机会，从而形成死锁。

所以，线程块只有在流式多处理器能够为**整个线程块**提供所需驻留资源时，才会被接纳。资源分配的整体性是线程块级同步语义能够成立的硬件条件之一。

4.3.4 普通线程块之间为何不能依赖执行顺序

如果不同线程块之间没有同步依赖，那么这些线程块可以：

- 以任意顺序开始；
- 在不同流式多处理器上并行执行；
- 在硬件资源较少时部分顺序执行；
- 在硬件资源较多时让更多线程块同时执行。

这形成了**透明可伸缩性**（*transparent scalability*）：同一份核函数代码可以在具有不同流式多处理器数量的 GPU 上运行，而正确性不依赖具体有多少线程块同时驻留。

关键结论

线程块执行顺序不受程序员控制，因此普通核函数中的正确性条件必须对所有可能的线程块调度顺序成立。

4.3.5 跨线程块屏障的死锁风险

若程序试图让整个网格中的线程块通过普通自制屏障互相等待，问题在于部分线程块可能尚未获得驻留。已经驻留的线程块先到达屏障并占用流式多处理器资源；未驻留的线程块却必须先运行到同一屏障才能解除等待。如果驻留资源被等待中的线程块占满，后续线程块就永远没有机会启动。

受约束的跨线程块同步机制主要包括：

- **协作组**（*cooperative groups*）可以在满足特定驻留约束的条件下，支持线程块簇或整个网格范围的屏障；关键前提是相关参与者能够被同时调度到硬件上。
- 某些单向生产者—消费者同步可以通过更严格的执行结构建立先后关系，但普通线程块的调度顺序本身仍不能被程序假定。

因此，跨线程块同步属于比线程块内同步更高成本、约束更多的机制。除非算法明确需要并满足相应条件，否则应优先保持线程块之间的顺序独立性。

4.4 线程束、处理块与单指令多线程执行

4.4.1 线程块进一步划分为线程束

线程块驻留到流式多处理器后，硬件按照线程的**线性化索引**把它拆分成若干**线程束**(*warp*)。线程束大小由设备实现决定；截至 Hopper 的所列 NVIDIA 架构均采用 32 个线程。

若令线程束大小为 $W = 32$，线程块中线性化索引为 t 的线程属于

$$\text{warpId} = \left\lfloor \frac{t}{W} \right\rfloor .$$

因此，索引 $0 \sim 31$ 属于线程束 0，索引 $32 \sim 63$ 属于线程束 1，依此类推。一个包含 T_b 个线程的线程块需要

$$W_b = \left\lceil \frac{T_b}{W} \right\rceil$$

个线程束。

例题：三个 256 线程的线程块

每个线程块包含

$$256/32 = 8$$

个线程束。若三个这样的线程块同时驻留在同一个流式多处理器上，则共有

$$8 \times 3 = 24$$

个驻留线程束。这里的 24 表示可供调度的线程束状态数量，不表示 24 个线程束在同一时钟周期同时发射指令。

4.4.2 处理块与线程束调度器

流式多处理器内部的执行核心还会组织成若干**处理块**(*processing block, PB*)。在简化的流式多处理器模型中，每个处理块具有自己的控制与线程束调度资源；线程束被调度到这些处理块上执行。

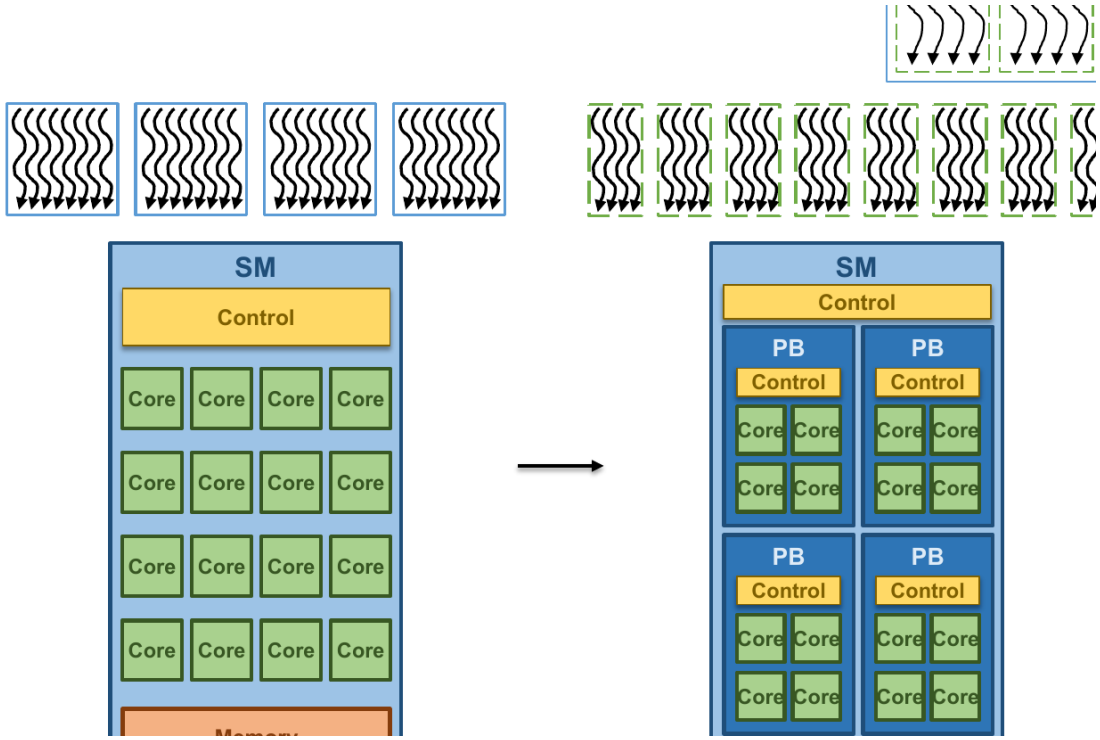


图 4.3 线程块先拆分成线程束；流式多处理器内部再由多个处理块及其控制资源调度这些线程束

以 Hopper H100 为例，128 个 FP32 核心组织成 4 个处理块，每个处理块包含 32 个 FP32 核心并共享相应的指令分派资源。该图是吞吐结构的简化表示，因为一个流式多处理器内还存在整数单元、加载/存储单元、FP64 单元与张量核心等其他执行资源。

4.4.3 从单程序多数据到单指令多线程

软件层次的**单程序多数据**(*Single Program, Multiple Data, SPMD*)描述的是：大量线程运行同一个核函数程序，每个线程依据自己的索引和数据完成不同实例的工作。

硬件调度层次则使用**单指令多线程**(*Single Instruction, Multiple Threads, SIMT*)：当某个线程束被选中时，调度器为该线程束发射一条指令，各个活动线程在各自的数据和线程状态上执行这条指令。该组织与**单指令多数据**(*Single Instruction, Multiple Data, SIMD*)具有相同的控制复用思想，即让多个执行通道共享指令获取和分派成本。

关键结论

SPMD 是程序员看到的核函数编程模型；SIMT 是线程束层次的硬件执行模型。二者的连接点是线程束：来自同一核函数的线程被成组调度，同一时刻的指令发射以线程束为单位。

这种结构的主要硬件收益是摊薄控制成本。若 32 个执行通道都要执行同一条指令，就不需要为每个通道复制完全独立的指令获取与分派逻辑。代价则是：如果线程束内部的线程需要走不同控制路径，执行通道不能始终保持同时有效，这就产生控制发散。

4.4.4 独立线程调度不改变性能分析的基本单位

从 Volta 开始，NVIDIA 引入**独立线程调度**(*independent thread scheduling*)。线程束中的线程可以维护更独立的执行状态，在发散后沿不同路径前进并在之后重新会合，不再要求所有线程在所有情况下表现为严格锁步。

性能分析仍把线程束视为关键单位，原因有两点：第一，执行资源和指令发射仍围绕线程束组织；第二，线程束内部的控制不一致依然会降低有效通道利用率。独立线程调度还意味着某些过去依赖隐含锁步行为的线程间协作必须使用更明确的同步机制。

4.4.5 驻留线程束、就绪线程束与实际发射线程束

对一个流式多处理器中的线程束，需要区分三个状态概念：

- **驻留线程束**：其线程状态、寄存器等资源已经分配在流式多处理器中；
- **就绪或可选线程束**：下一条指令所需操作数与执行条件已经满足，可以被调度器选择；
- **实际发射线程束**：当前获得发射机会、正在向执行资源送出指令的线程束。

占用率统计的是驻留规模，而不是“这一周期有多少线程束正在发射”。理解这一差别，是后续延迟隐藏和占用率分析的前提。

4.5 控制发散与有效执行通道

4.5.1 控制发散是线程束内部的问题

当同一线程束中的线程对分支条件得到不同结果，或执行不同数量的循环迭代时，就出现**控制发散**（*control divergence*）。线程束需要依次处理不同控制路径；处理某一路径时，只启用需要执行该路径的通道，其他通道处于非活动状态。

如果两个不同线程束分别选择不同路径，但每个线程束内部的 32 个线程都一致，则不存在该意义下的线程束内发散。因而判断发散时，关键问题是：**同一线程束的活动通道是否对当前控制条件意见一致。**

4.5.2 分支发散示例

考虑以下条件：

代码 4.2 线程束内的分支发散

```
if (threadIdx.x < 24) {
    A;
} else {
    B;
}
C;
```

对第一个线程束，线程索引为 0 ~ 31。条件成立的线程有 24 个，条件不成立的线程有 8 个。因此执行路径 A 时有 24 个活动通道、8 个非活动通道；执行路径 B 时则相反。两条路径都完成后，线程束重新会合并共同执行 C。

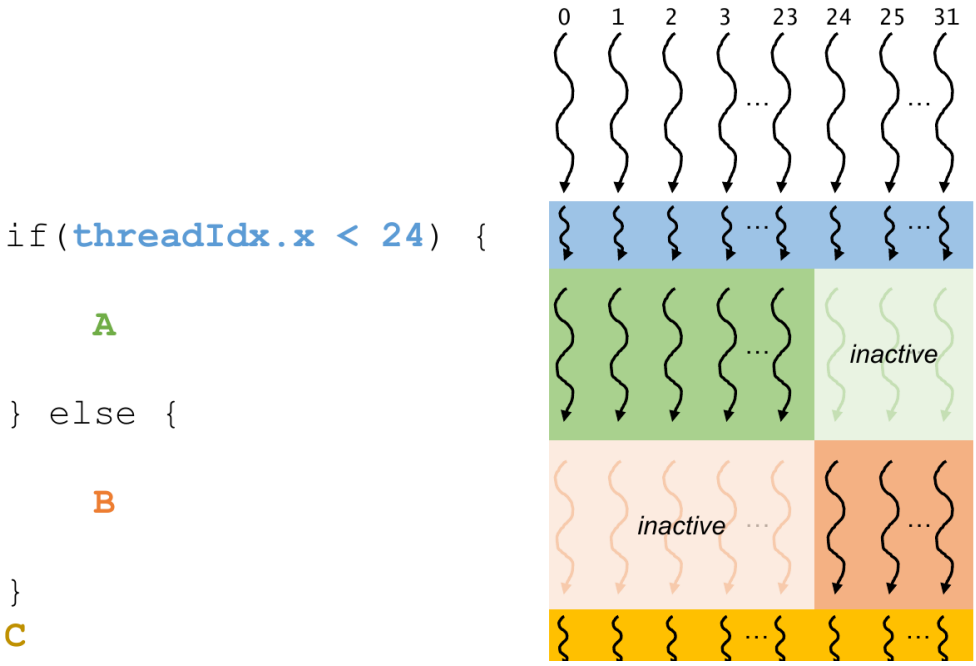


图 4.4 同一线程束中的线程在条件分支上分成两组时，两条路径对该线程束串行处理，未选择当前路径的通道保持非活动

4.5.3 活动通道比例与发散区间效率

活动通道比例（*active-lane fraction*）是一次 SIMT/SIMD 执行中启用的线程或执行通道所占比例。对于线程束大小 W，某条路径上有 n 个活动线程时，活动通道比例为

$$f_{\text{active}} = \frac{n}{W}.$$

在上例中，路径 A 的活动通道比例是 $24/32 = 75\%$，路径 B 的活动通道比例是 $8/32 = 25\%$。若两条路径拥有相同数量的动态指令，则仅考虑分支区间时，两个串行阶段一共提供 $2W = 64$ 个通道-指令槽，其中实际有用的通道-指令数为 $24 + 8 = 32$，因此平均有效比例为

$$\eta = \frac{24 + 8}{2 \times 32} = 50\%.$$

若不同路径长度不同，可把这一思路写成加权形式。设路径 p 有 n_p 个活动线程，并执

行 I_p 条动态指令，则发散区间的通道利用率可写成

$$\eta = \frac{\sum_p n_p I_p}{W \sum_p I_p}.$$

该表达式只描述“每条路径串行执行、非参与通道关闭”造成的通道利用率，没有包含存储延迟、流水线吞吐或其他瓶颈。

4.5.4 循环也会产生控制发散

若每个线程的循环次数来自线程私有数据，例如

代码 4.3 每线程循环次数不同造成的发散

```
int N = a[threadIdx.x];
for (int i = 0; i < N; ++i) {
    A;
}
```

则线程束中的线程可能在不同迭代结束。只要还有某个活动线程尚未达到自己的 N，线程束就必须继续执行循环体；已经完成的线程通道则保持非活动。随着越来越多线程退出，活动通道比例会逐步下降。

因此，发散不仅来自显式的 `if/else`。任何依赖每线程数据的控制结构，包括循环退出条件，都可能让同一线程束内部的动态执行路径不同。

4.5.5 通过线程束粒度组织条件减少发散

一种减少分支发散的方法是让条件在一个线程束内部保持恒定。例如令条件依赖

$$\left\lfloor \frac{\text{threadIdx.x}}{\text{warpSize}} \right\rfloor,$$

则同一线程束内的线程具有相同商值。不同线程束仍然可以选择不同分支，但每个线程束内部不会因为该条件产生发散。

代码 4.4 线程束粒度的分支条件

```
if (threadIdx.x / warpSize > 2) {
    // 路径 A
} else {
    // 路径 B
}
```

这里仍有两个程序路径；优化点在于路径选择发生在线程束粒度，而不是在线程粒度。

4.5.6 边界检查中的少量发散通常不可避免

处理向量或图像时，问题规模经常不是线程块大小的整数倍。常见写法会向上取整线程块数量，然后在核函数中执行边界检查：

代码 4.5 向量尾部的边界检查

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < N) {
    // 有效工作
}
```

最后一个线程块中可能只有部分线程有效，因此最后若干线程束会出现发散。这类发散通常只影响计算尾部的一小部分线程束，代价有限。实际目标不是机械消除所有分支，而是避免让大量线程束长期处于严重的通道失活状态。

关键结论

减少控制发散的核心原则是让控制条件尽量在线程束内部保持一致。判断性能影响时，应关注发散路径的长度以及每条路径上的活动通道比例，而不是只统计源代码里有多少个分支语句。

4.6 线程束调度与延迟隐藏

4.6.1 线程束调度器选择可执行的线程束

流式多处理器同时维护多个驻留线程束。在线程束已有驻留状态之间切换发射机会，可近似视为**零开销线程束调度**(*zero-overhead warp scheduling*)：当前线程束无法继续时，调度器可以迅速选择另一个已经就绪的驻留线程束，无需执行传统 CPU 线程上下文切换所需的完整保存与恢复。

线程束只有在下一条指令具备执行条件时才能成为候选对象，其中一个关键条件是所需操作数已经可用，即操作数已 “ready for consumption”。

机制

每个驻留线程已经在流式多处理器的寄存器文件中占有相应状态；线程束还具有程序计数器等控制状态。在线程束之间切换发射机会时，不需要把整个线程上下文先写回内存再恢复。调度器主要是在已有驻留状态之间选择下一批要推进的线程束。

CUDA 不向程序员暴露具体的线程束优先级策略。程序可以依赖的是：满足执行条件的驻留线程束会被硬件调度以持续推进，而不能编写依赖某个固定线程束优先顺序的程序。

4.6.2 长延迟操作如何造成线程束停顿

算术指令所需操作数通常要先进入寄存器。若线程束执行高延迟的全局内存加载，而后续指令立即依赖加载结果，那么在数据返回之前，该线程束无法继续执行依赖指令，因而处于等待状态。

类似地，某些需要多个周期的执行资源也可能造成依赖停顿。关键并不在于停顿来源是内存还是计算，而在于：**当前线程束的下一条指令暂时无法发射。**

4.6.3 用其他线程束覆盖等待时间

延迟隐藏（*latency hiding*）的核心是把一个线程束的等待区间与其他线程束的有用执行重叠。设线程束 W_0 发出高延迟操作后必须等待；只要同一流式多处理器中存在另一个已经就绪的线程束 W_1，调度器就可以让 W_1 获得执行机会。若 W_1 之后也停顿，则继续寻找 W_2、W_3 等可执行线程束。

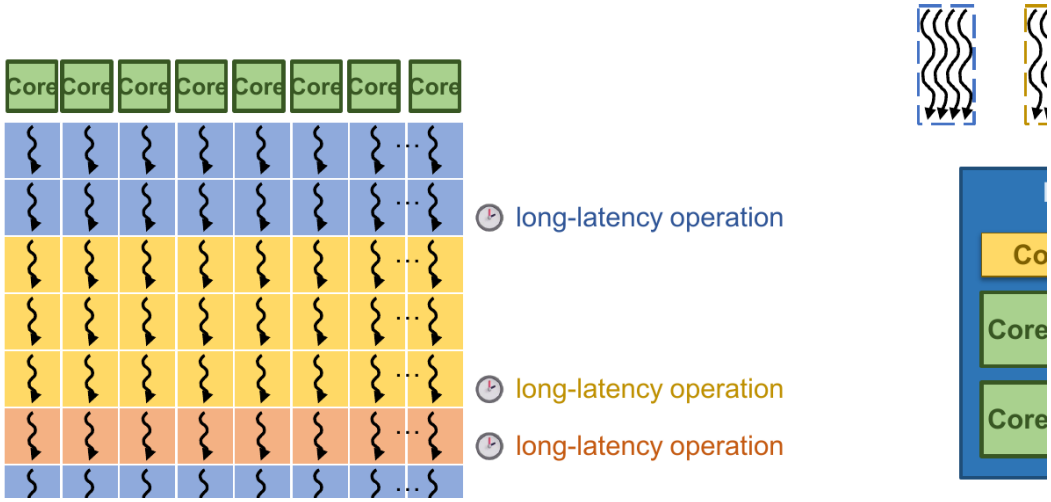


图 4.5 某个线程束遇到高延迟操作后，处理块可以选择其他就绪线程束继续执行，从而把等待时间与有用工作重叠

延迟隐藏不缩短一次全局内存访问本身需要的时间；它通过在这段时间内执行其他独立工作来减少执行资源的空闲时间。因而判断它是否有效时，应问“等待期间有没有别的线程束可以推进”，而不是只问“单次操作的延迟是多少”。

4.6.4 为什么驻留线程数远大于物理核心数

一个流式多处理器通常支持的最大驻留线程数明显大于其物理执行核心数，这使硬件能够保持较大的候选工作池。

假设流式多处理器中只有很少线程束。如果这些线程束同时发出长延迟加载，调度器就可能找不到任何就绪线程束，执行资源只能空等。若驻留线程束数量更多，则在部分线程束等待时，其他线程束已经准备好操作数的概率通常更高。

因此可以把关系概括为

$$\text{更多驻留线程束} \Longrightarrow \text{更大的就绪候选集合} \Longrightarrow \text{更强的延迟隐藏机会}.$$

这条关系解释了为什么占用率常常重要：占用率提高意味着流式多处理器上同时驻留的线程束更多，从而更可能在某个线程束停顿时找到替代工作。

4.6.5 延迟隐藏失效的典型情形

延迟隐藏只有在存在其他可执行线程束时才有作用。以下情况会使“增加线程束”无法继续解决问题：

- **内存带宽已经饱和。**更多线程束只会增加等待请求，不能提高已经受带宽上限约束的吞吐；
- **所有线程束都受到同类未解决依赖阻塞。**如果没有任何线程束的下一条指令已经就绪，调度器仍然无事可做；
- **缺少足够的指令级并行。指令级并行**(*instruction-level parallelism*)不足时，各线程束可能都很快进入依赖停顿，无法形成可交错执行的独立工作。

关键结论

延迟隐藏的必要条件不是“高占用率”这个数字本身，而是驻留工作中持续存在可发射的独立线程束。高占用率通常提高这一机会，但不能越过带宽、依赖或执行资源的硬瓶颈。

4.7 占用率、资源约束与线程块大小

4.7.1 占用率的定义

占用率(*occupancy*)描述一个流式多处理器上实际驻留的线程束数量，相对于该流式多处理器允许的最大驻留线程束数量的比例。记实际驻留线程束数为 W_{res}，最大驻留线程束数为 $W_{\max}$，则

$$\boxed{\mathcal{O} = \frac{W_{\text{res}}}{W_{\max}}}.$$

由于同一架构上的线程束大小固定，若只考虑线程数约束，也可以等价地用驻留线程数与最大驻留线程数之比理解。但正式定义以驻留线程束为中心更直接，因为调度器的候选单位就是线程束。

需要特别注意

占用率中的“驻留/活动”不等于“当前周期正在执行”。一个线程束可以已经驻留，却正在等待内存；也可以已经就绪，却暂时没有得到发射机会。占用率衡量的是可驻留并保持状态的工作池大小。

4.7.2 流式多处理器资源示例

不同 GPU 架构具有不同的资源参数。最大驻留线程数通常显著高于每个流式多处理器的核心数，同时还受到线程块数、寄存器和共享内存等约束。

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Compute Capability	3.5	5.2	6.0	7.0	8.0	9.0
Number of SMs	15	24	56	80	108	132
Cores per SM	192	128	64	64	64	128
Max threads per SM	2048	2048	2048	2048	2048	2048
Max blocks per SM	16	32	32	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	228KB

图 4.6 多代 GPU 的流式多处理器资源对比；红框突出每个流式多处理器的核心数与最大驻留线程数

GPU	K40	M40	P100	V100	A100	H100
微架构	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
计算能力	3.5	5.2	6.0	7.0	8.0	9.0
流式多处理器数量	15	24	56	80	108	132
每个流式多处理器核心数	192	128	64	64	64	128
最大驻留线程数/流式多处理器	2048	2048	2048	2048	2048	2048
最大驻留线程块数/流式多处理器	16	32	32	32	32	32
最大线程数/线程块	1024	1024	1024	1024	1024	1024
寄存器/流式多处理器	64K	64K	64K	64K	64K	64K
共享内存/流式多处理器	48KB	96KB	64KB	96KB	164KB	228KB

表 4.1 多代 GPU 的流式多处理器资源示例

4.7.3 把多重驻留限制写成统一形式

设一个线程块有 T_b 个线程，线程束大小为 W，则每个线程块需要

$$W_b = \left\lceil \frac{T_b}{W} \right\rceil$$

个线程束。设设备限制分别为：最大驻留线程块数 $B_{\max}$、最大驻留线程数 $T_{\max}$、最大驻留线程束数 $W_{\max}$。若每个线程块还需要 R_b 个寄存器资源单位与 S_b 字节共享内存，而流式多处理器总量分别为 R_{SM} 和 S_{SM}，则多重限制可概念化为

$$B_{\mathrm{res}} \le \min\left(B_{\max}, \left\lfloor \frac{T_{\max}}{T_b} \right\rfloor, \left\lfloor \frac{W_{\max}}{W_b} \right\rfloor, \left\lfloor \frac{R_{\mathrm{SM}}}{R_b} \right\rfloor, \left\lfloor \frac{S_{\mathrm{SM}}}{S_b} \right\rfloor\right).$$

该表达式用于统一理解各类驻留约束；实际硬件还可能按特定粒度分配寄存器或共享内存，因此精确驻留数应由设备属性与占用率工具计算。确定 B_{res} 后，驻留线程束数为

$$W_{\text{res}} = B_{\text{res}} W_b,$$

再代入占用率定义即可。

4.7.4 H100 风格限制下的三个线程块大小

以下例题采用一组简化限制：每线程块最多 1024 个线程，每流式多处理器最多 2048 个驻留线程、最多 32 个驻留线程块，并假设寄存器和共享内存不形成额外限制。

4.7.4.1 每块 256 个线程

线程数约束允许

$$\left\lfloor \frac{2048}{256} \right\rfloor = 8$$

个线程块，且 $8 < 32$。所以可驻留 $8 \times 256 = 2048$ 个线程，即 64 个线程束；在这一简化模型中占用率为

$$\mathcal{O} = \frac{64}{64} = 100\%.$$

4.7.4.2 每块 32 个线程

若只看线程数，需要

$$2048/32 = 64$$

个线程块才能填满线程容量，但设备最多只允许 32 个驻留线程块。因此实际只能驻留

$$32 \times 32 = 1024$$

个线程，也就是 32 个线程束，对应简化占用率

$$\mathcal{O} = \frac{32}{64} = 50\%.$$

这里的瓶颈是**最大驻留线程块数**。

4.7.4.3 每块 768 个线程

线程数约束允许

$$\left\lfloor \frac{2048}{768} \right\rfloor = 2$$

个线程块，驻留线程数为

$$2 \times 768 = 1536.$$

剩余 $2048 - 1536 = 512$ 个线程容量无法再容纳完整的第三个线程块，因此简化占用率为

$$\mathcal{O} = \frac{1536}{2048} = 75\%.$$

关键结论

线程块驻留必须以完整线程块为单位，所以资源利用会出现“装箱”效应。即使总资源还有余量，只要余量不足以容纳另一个完整线程块，这部分资源就不能被该核函数的额外线程块使用。

4.7.5 二维线程块大小：彩色转灰度核函数

另一个例题假设某 GPU 的每个流式多处理器最多驻留 1536 个线程和 8 个线程块，比较 8×8、16×16 与 32×32 三种二维线程块。

线程块形状	每块线程数	可驻留块数	驻留线程数	简化占用率
8×8	64	8	512	33.3%
16×16	256	6	1536	100%
32×32	1024	1	1024	66.7%

推导过程如下：

- 8×8：线程数允许 $1536/64 = 24$ 个线程块，但块数上限为 8，所以只能驻留 512 个线程；
- 16×16：$1536/256 = 6$ 个线程块，既不超过 8 块限制，又正好使用全部 1536 个线程容量；
- 32×32：每块 1024 线程，第二块会使线程数达到 2048，因此只能驻留一块。

在该题给定的简化限制下，16×16 获得最高理论占用率。

4.7.6 占用率高并不保证性能最高

较高占用率通常有利于增加延迟隐藏机会，但占用率只在能够帮助隐藏延迟的范围内有用，并不单独保证性能。

原因与上一节一致：若内存带宽已经饱和、所有线程束都受共同依赖阻塞，或其他执行资源成为瓶颈，即使继续增加驻留线程束，也未必增加有效指令吞吐。因此，占用率最适合作为**资源驻留与延迟隐藏能力的诊断量**，而不是完整的性能模型。

4.8 运行时资源查询与例题

4.8.1 运行时查询设备属性

不同 GPU 架构的资源上限不同，因此程序不应把所有驻留限制都永久写死。CUDA 运行时提供 `cudaGetDeviceProperties` 查询设备属性：

代码 4.6 `cudaGetDeviceProperties` 函数原型

```
cudaError_t cudaGetDeviceProperties(cudaDeviceProp* prop, int device);
```

基本用法如下：

代码 4.7 查询设备线程资源上限

```
int devID = 0;
cudaDeviceProp devProp;
cudaGetDeviceProperties(&devProp, devID);

int maxThreadsPerBlock = devProp.maxThreadsPerBlock;
int maxThreadsPerMultiProcessor =
    devProp.maxThreadsPerMultiProcessor;
```

设备属性结构还可以提供线程束大小以及其他架构限制。程序可以据此选择更合适的启动配置，而不是假设未来设备与当前设备拥有完全相同的线程数、线程块数或线程束限制。

4.8.2 占用率计算工具

NVIDIA Nsight Compute 提供**占用率计算器**(*Occupancy Calculator*)，能够根据目标 GPU、每线程块线程数、每线程寄存器使用量、共享内存使用量等信息估算多处理器占用率。该工具替代了较早的 CUDA 占用率计算表格。

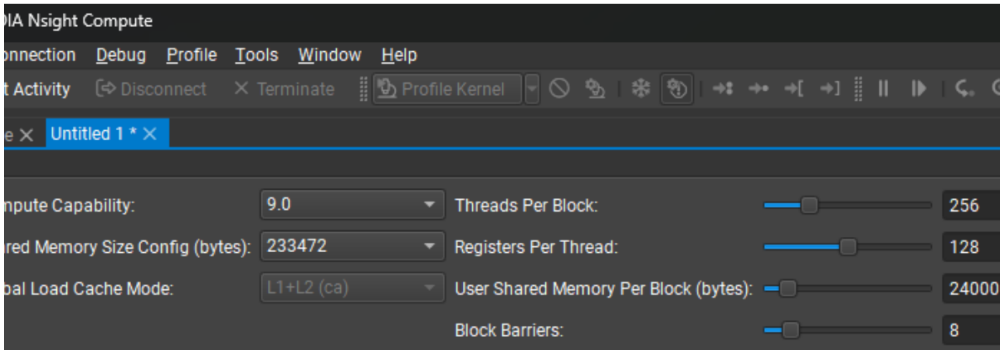


图 4.7 占用率计算器把线程块大小、寄存器和共享内存等静态资源需求映射为驻留线程束与线程块限制

占用率工具的价值在于快速识别明显的资源浪费或驻留瓶颈。它不能替代性能测量，因为高占用率只说明潜在的延迟隐藏能力较强，并不能说明实际瓶颈一定来自驻留线程束不足。

4.8.3 例题一：1536 线程与 4 线程块上限

某 CUDA 设备的每个流式多处理器最多驻留 1536 个线程，并且最多驻留 4 个线程块。比较每块 256、384、576 个线程，哪一种配置能在仅考虑这两个限制时填满线程容量？

完整推导

每块 256 线程：

$$\frac{1536}{256} = 6.$$

线程数允许 6 个线程块，但块数上限只有 4，因此实际最多驻留

$$4 \times 256 = 1024$$

个线程，不能填满 1536 线程容量。

每块 384 线程：

$$\frac{1536}{384} = 4.$$

恰好需要 4 个线程块，既满足线程数限制，也满足线程块数限制，驻留线程数为

$$4 \times 384 = 1536.$$

因此该配置在题目给定的简化限制下达到满线程容量。

每块 576 线程：

$$\left\lfloor \frac{1536}{576} \right\rfloor = 2,$$

只能驻留

$$2 \times 576 = 1152$$

个线程；第三个线程块会使线程总数超过 1536，所以不能填满容量。

因此答案是 **384 线程/块**。

4.8.4 例题二：只知道流式多处理器和核心数，能否选出最佳配置

题目给定一个一维数组，每个元素由一个线程处理。目标 GPU 有 8 个流式多处理器，每个流式多处理器有 16 个流式处理器，问“最佳执行配置”是什么。

仅凭这些信息无法确定。流式多处理器数量和核心数只描述部分物理执行资源；至少还缺少：

- 每流式多处理器最大驻留线程数和最大驻留线程块数；
- 每线程块最大线程数；
- 寄存器与共享内存容量；
- 该核函数每线程寄存器使用量和每线程块共享内存使用量；
- 实际性能测量，用于判断瓶颈是否来自占用率、带宽或其他资源。

关键结论

从“核心数”直接推导线程块大小是不充分的。CUDA 的启动配置同时受到驻留限制和核函数资源足迹约束，而最高理论占用率也不等价于最高实测性能。

4.8.5 例题三：统一分支条件是否造成控制发散

考虑核函数：

代码 4.8 统一分支条件的核函数

```
__global__ void do_work(int q, int *A) {
    int result = 0;
    if (q < 5)
        result = threadIdx.x;
    A[threadIdx.x] = result;
}
```

问题是：条件 `q < 5` 是否造成控制发散？

答案是**不会**。变量 `q` 是统一传入的标量参数，同一线程束中的线程对该条件得到相同结果。因此整个线程束要么都执行赋值语句，要么都跳过赋值语句。虽然赋值结果使用了每线程不同的 `threadIdx.x`，但这只意味着各线程处理不同数据，并不意味着控制路径不同。

更一般地说，控制发散要求同一线程束的活动线程对分支谓词得到不同结果。分支条件不必显式包含线程索引才可能发散；只要它依赖的每线程数据不同，就可能造成发散。反之，即使分支语句存在，只要谓词在线程束内一致，就没有线程束内控制发散。

4.9 架构、调度与性能机制的统一关系

4.9.1 从网格到执行通道的完整映射

从软件层次到硬件执行资源的完整映射链为

$$\text{网格} \rightarrow \text{线程块} \rightarrow \text{流式多处理器驻留} \rightarrow \text{线程束} \rightarrow \text{处理块调度} \rightarrow \text{执行通道}$$

线程块决定资源分配和协作范围；线程束决定调度与控制发散的粒度；处理块和执行资源决定线程束指令如何获得实际执行机会。把这三个粒度区分开，就能避免把“线程块同时驻留”“线程束被调度”和“线程在核心上执行”混为同一件事。

4.9.2 四个性能概念之间的因果链

1. **资源限制决定驻留规模。**线程数、线程块数、寄存器和共享内存等限制共同决定一个流式多处理器能容纳多少线程块与线程束。
2. **驻留规模决定占用率。**驻留线程束数与架构最大线程束数之比就是占用率。
3. **占用率影响延迟隐藏机会。**驻留线程束越多，某个线程束停顿时找到另一个就绪线程束的机会通常越高。
4. **控制发散降低单个线程束的有效通道利用。**即使有足够多驻留线程束，如果每个线程束内部大量通道因为分支而失活，执行资源的有效利用仍会下降。

因此，线程块大小不是一个只影响“线程有多少”的参数。它同时改变每块线程束数、流式多处理器可驻留线程块数、资源装箱效率和可用于延迟隐藏的线程束池；如果算法中的控制结构又与线程索引相关，还要进一步考虑线程束内部的发散模式。

4.9.3 概念边界总结

概念	主要粒度	回答的问题
线程块驻留	线程块/流式多处理器	一个线程块能否进入某个流式多处理器，以及同时能驻留多少块
线程束调度	线程束/处理块	哪个已经驻留且就绪的线程束现在获得指令发射机会
控制发散	单个线程束内部	同一线程束的活动线程是否沿不同控制路径执行
延迟隐藏	多个驻留线程束	当前线程束等待时，是否有其他就绪线程束可以使用执行资源
占用率	流式多处理器	驻留线程束池相对架构最大容量有多大
线程块内同步	单个线程块	同一线程块中的线程如何在共享资源上建立协作和屏障
跨线程块协作	多线程块/更高作用域	如何在不依赖普通线程块调度顺序的前提下建立受约束的协作

第 5 讲

CUDA Memory Model

5.1 课程安排与核心内容

5.1.1 学习目标与技术范围

CUDA 存储模型连接了两类问题：线程能够访问哪些数据，以及这些访问如何影响计算速度。基本矩阵乘法提供了一个完整例子：为输出元素分配线程，建立正确的地址映射，再根据浮点运算量与访存量分析性能。

学习目标为：了解 CUDA 线程可访问的各类存储器的基本特点；为 Lab 2 的基本矩阵乘法实现做准备；学习评估全局内存访问对性能的影响；了解 GPU 性能指标。对应的技术内容依次为存储层次、基本矩阵乘法，以及算术强度和 Roofline 性能模型。

5.1.2 实验、交流与课程项目

5.1.2.1 实验与支持渠道

以下安排对应 **2026 年 9 月 8 日**的课程信息。Lab 1 的截止时间为当周周五，**9 月 11 日**；需要先完成 Lab 0，Lab 1 不计入最终成绩。

课程问题通过 Campuswire 发布，课程工作人员只回复在 Campuswire 上发布的问题。提问前应先尝试自行查找答案，包括询问课程聊天机器人。答疑时间同时包含线下与线上形式，需要查看相应时间表。课后事项为阅读教材第 3 章、取得 Delta 账户、设置 GitHub，以及提交 Labs 0 和 1。

5.1.2.2 课程项目与暂定里程碑

两个项目方向都只实现**推理的前向计算**(*inference / forward pass*)，区别在于实现对象与组织形式。

方向	实现内容	组织形式
CNN	LeNet-5 卷积神经网络中的卷积层	个人项目
GPT-2	GPT-2 模型中的各层	团队项目，每组 3–4 人

暂定时间为：Milestone 1 在第 7–8 周，Milestone 2 在第 10–11 周，Milestone 3 在第 15–16 周。以上里程碑为暂定周次。

5.2 从处理器执行到 CUDA 存储层次

5.2.1 数据为什么必须先进入寄存器

5.2.1.1 冯 · 诺依曼模型中的计算与存储

冯 · 诺依曼模型 (*Von Neumann model*) 将计算机划分为处理单元、存储器、输入输出子系统和控制单元。**处理单元** (*processing unit, PU*) 中的**算术逻辑单元** (*arithmetic logic unit, ALU*) 执行算术与逻辑操作，**寄存器堆** (*register file*) 保存正在参与运算的数据。存储器同时保存程序指令和数据；**输入输出子系统** (*I/O subsystem*) 负责与外部环境通信；**控制单元** (*control unit, CU*) 协调指令执行。

在这一执行模型下，运算的操作数位于寄存器中。存储器里的数据需要先装入寄存器，指令也需要从存储器取出并装入**指令寄存器** (*instruction register, IR*)。因此，一条高级语言表达式中的“取值”和“计算”，在执行层面对应不同的工作。

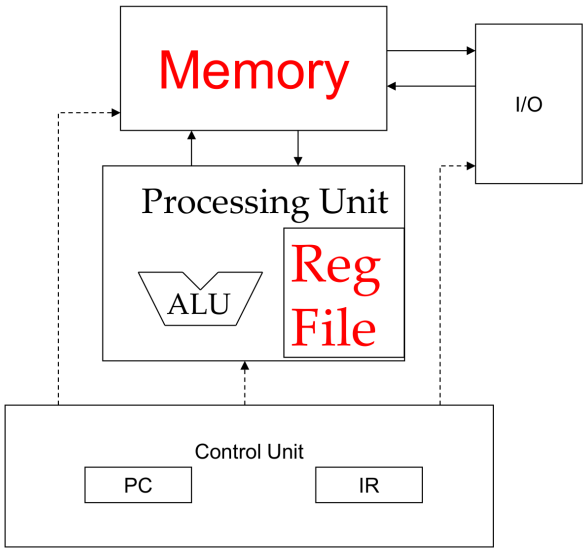


图 5.1 处理单元通过寄存器保存工作数据，控制单元协调存储、运算与输入输出。图中的 PC 为程序计数器，IR 为指令寄存器。

5.2.1.2 ADD 与 LOAD 的区别

指令处理可分为**取指** (*fetch*)、**译码** (*decode*)、**执行** (*execute*) 和**访存** (*memory*) 几个阶段。三类指令分别承担运算、数据传输与控制流功能。下面两条 LC-3 指令用于说明寄存器运算与数据装载的差别。

```
ADD R1, R2, R3
LDR R4, R6, #3
```

`ADD R1, R2, R3` 读取 R2 和 R3，进行加法，再把结果写入 R1。两个输入都已在寄存器中，其数据运算不需要再从数据存储器读取操作数。用寄存器内容 R_i 表示，可写为

$$R_1 \leftarrow R_2 + R_3.$$

`LDR R4, R6, ##3` 先读取 R6，将其内容加上偏移量 3，形成有效地址，再从该地址读出数据并写入 R4。若 $\mathrm{Mem}[a]$ 表示地址 a 处的内容，则

$$a \leftarrow R_6 + 3, \qquad R_4 \leftarrow \mathrm{Mem}[a].$$

两条指令本身都需要被取出和译码；差别在于 LOAD 还显式访问其计算出的数据地址。这里的 3 保留 LC-3 示例中的地址偏移含义，不将其转换为 CUDA 数组的字节步长。

5.2.1.3 延迟与容量的差别

寄存器接近运算单元，适合保存当前工作值；主存容量大，适合保存完整数据集。简化的数量级对照为：寄存器访问约 1 个周期，内存访问约数百个周期；CPU 的寄存器数量为数百量级，GPU 的每个**流式多处理器**(*streaming multiprocessor, SM*) 拥有 $O(10^4)$ 量级的寄存器，而内存容量可达到 GB 或更大。

从计算式看数据流

对于“从数组读取两个数并相乘累加”，数据先被读入可供运算使用的寄存器，再进行乘法和累加。反复使用已经取到的工作值，可以减少再次读取远端存储器的需求。CUDA 的不同存储空间，为线程私有工作值、块内协作和大规模数据集提供了不同的位置。

5.2.2 线程、线程块与设备的可见范围

存储空间可以从两个角度理解：**作用域**(*scope*) 决定哪些线程能够访问同一份数据；**物理位置**(*physical location*) 决定数据处于 SM 内部、设备内存，还是相应的缓存路径。变量的**生命周期**(*lifetime*) 则表示这份存储随线程、线程块或应用程序存在的时间范围。

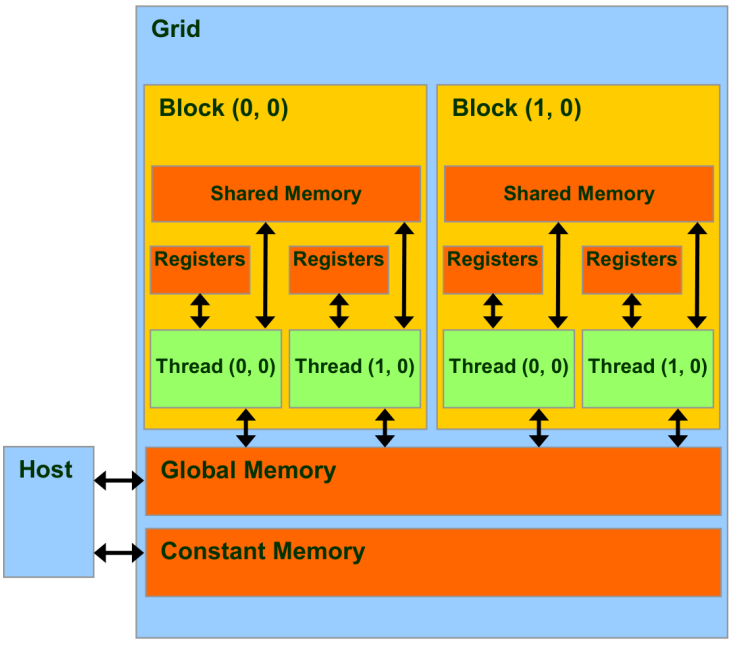


图 5.2 线程持有自己的寄存器，同一线程块的线程共同访问该块的共享内存；全局内存与常量内存对网格中的线程可见。

5.2.2.1 线程私有：寄存器与局部内存

寄存器(*registers*)保存单个线程的私有工作值，访问延迟最低。例如，矩阵乘法中的累加器只需要由当前线程更新，适合用一个自动标量变量表示，其典型位置就是寄存器。

局部内存(*local memory*)同样属于单个线程的私有地址空间，但物理上由设备内存支持，并通常经过 L1/L2 缓存。较大的自动对象，以及**寄存器溢出**(*register spill*)所需的存储，都可能使用这一空间。这里的 local 描述的是线程私有的*作用域*，物理位置仍然可能远离运算单元。

这一区分直接影响对代码的理解：两个变量都只在一个线程中使用，并不代表它们具有相同的访问代价。自动标量通常使用寄存器；线程私有数组可能使用寄存器，也可能使用局部内存，具体分配由编译器处理。

5.2.2.2 块内共享：共享内存

共享内存(*shared memory*)位于 SM 上，由同一**线程块**(*thread block*)中的线程共同访问，生命周期与线程块对应。不同线程块各自拥有独立的共享内存存储。

共享内存适合承担块内协作和数据复用：某个线程把数据写入本块的共享内存后，其他需要该数据的块内线程可以从这份共享存储中读取。因而，当多个线程需要反复使用相同输入时，共享内存提供了显式组织复用的位置。共享内存分块矩阵乘法利用这一机制组织输入复用。

5.2.2.3 设备可见：全局内存与常量内存

全局内存(*global memory*)对设备上的线程可见，适合保存主数据集，例如输入矩阵与输出矩阵。它支持线程读取和写入，容量大，但访问延迟高。代码中的全局地址访问也可能由缓存提供数据，因此其请求与最终到达设备内存的流量需要分开计数。

常量内存(*constant memory*)保存设备线程只读的数据，并通过**常量缓存**(*constant cache*)加速访问。**广播或一致访问**(*broadcast / uniform access*)，即多个线程读取相同的数据，是它特别适合的访问情形。全局内存和常量内存的可见范围都可以覆盖一个网格；对设备变量而言，其生命周期在声明表中列为应用程序级。

5.2.3 物理层次、缓存与访问代价

5.2.3.1 从 SM 到设备内存

在硬件视图中，每个 SM 内部具有寄存器、共享内存、L1 缓存与常量缓存；多个 SM 通过下一级 L2 缓存访问全局内存。**缓存** (*cache*) 保存已访问数据的副本，使后续访问有机会在更接近计算单元的位置得到响应。局部内存与全局内存的访问流量都可以经过缓存。

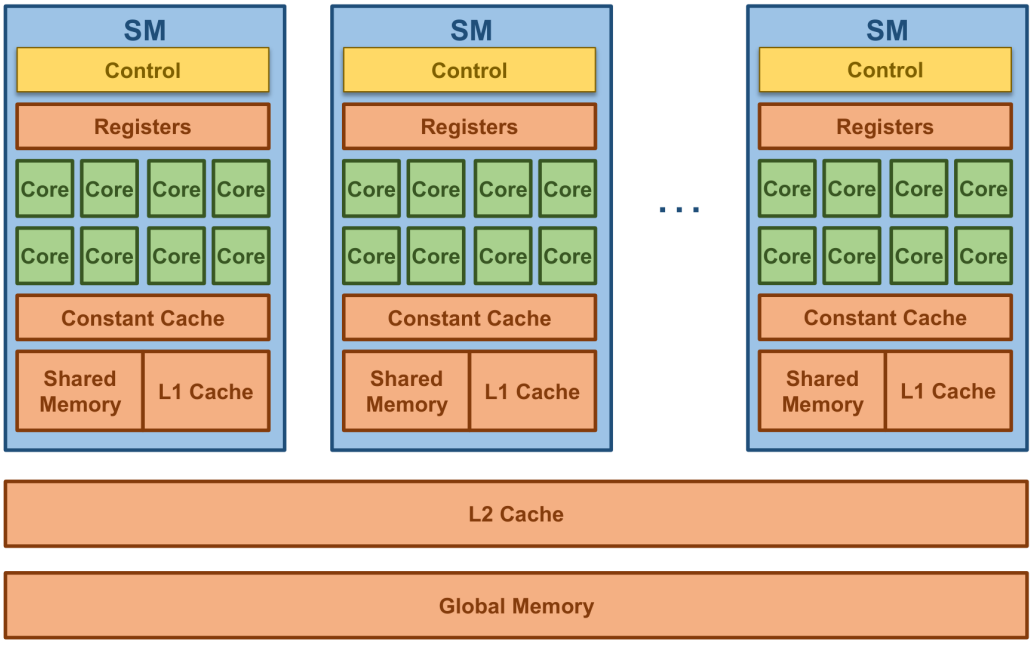


图 5.3 寄存器与块内共享存储位于 SM 内部，L2 缓存位于多个 SM 与全局内存之间。寄存器、共享内存与缓存分别承担不同的数据保存与复用任务。

表 5.1 CUDA 存储层次的作用域、位置与管理方式。

类型	作用域	典型位置	管理方式	主要用途
寄存器	线程	SM	编译器	私有工作值
局部内存	线程	设备内存及缓存	编译器	溢出、较大自动对象
共享内存	块	SM	程序员	块内协作与复用
全局内存	设备/网格	HBM / DRAM	程序员	主数据集
常量内存	设备/网格	DRAM 与常量缓存	程序员	一致的只读数据
L1 / L2	缓存层	片上	主要由硬件	缓存全局/局部访问

这里“程序员管理”指需要在程序中安排相应存储及数据使用；寄存器与局部内存之间的自动变量分配则主要由编译器处理。

5.2.3.2 周期数的使用范围

简化的程序员视图给出寄存器约 1 周期、共享内存约 5 周期、全局内存约 500 周期，以及命中缓存的常量内存约 5 周期。硬件层次图另将共享内存和常量缓存标为数十周期、全局内存标为数百周期。

这些示意值用于建立层次之间的代价差别：寄存器访问最快，片上共享存储与缓存提供较低延迟，设备内存访问具有更高延迟。两个图的数值口径分别保留，不能从中得到某一指定 GPU 上各类访问的固定延迟。

5.2.4 声明限定符与变量生命周期

5.2.4.1 从声明读出存储空间

CUDA **类型限定符**（*type qualifiers*）让变量声明表达其存储属性。下表中的设备变量声明对应设备侧存储；函数内没有限定符的自动变量则按线程独立产生。

表 5.2 变量声明、典型存储空间与生命周期。

声明形式	存储空间	作用域	生命周期
`__device__ int globalVar;`	全局	设备/网格	应用程序
`__device__ __constant__ int constantVar;`	常量	设备/网格	应用程序
`__device__ __shared__ int sharedVar;`	共享	块	块
`int localVar;`	通常为寄存器	线程	线程
`int localArr[N];`	寄存器或局部内存	线程	线程

声明中若已有 `__shared__` 或 `__constant__`，可以省略 `__device__`。

单独使用 `__device__` 的变量声明不能放在函数内部。表中的 N 表示数组长度。

5.2.4.2 自动变量与线程私有状态

设一个 kernel 中声明了 `int Row;` 和 `float Pvalue;`。每个线程都有自己的行索引和累加器：线程 A 更新其 `Pvalue`，不会因此更新线程 B 的 `Pvalue`。这正是用一个线程独立计算一个输出元素所需的私有状态。

当一个线程块中声明了共享变量，该块内的线程共同使用这份存储；另一个块中的同名共享变量属于另一个独立实例。因而，变量名相同与物理上访问同一份数据之间的关系，取决于变量所属的存储空间和作用域。

判断变量的三个问题

先确定谁需要访问它，再确定它的典型物理位置，最后检查它需要存在多久。线程私有工作值、块内共享中间结果、设备可见的大型输入输出，在这三个问题上分别对应不同的选择。

5.3 基本矩阵乘法：从公式到线程映射

5.3.1 一个输出元素对应一个点积

5.3.1.1 矩阵、下标与行主序地址

设输入矩阵 M, N 和输出矩阵 P 均为 $W \times W$，其中 W 对应代码中的 `Width`。采用与 C/CUDA 代码一致的零起始下标，行、列和求和下标分别为 i, j, k，取值范围均为 $0, \ldots, W-1$。矩阵乘法定义为

$$P_{ij} = \sum_{k=0}^{W-1} M_{ik} N_{kj}. \tag{5.1}$$

一个 P_{ij} 取决于 M 的第 i 行和 N 的第 j 列：将两者逐项相乘，再把这些乘积相加。数学公式与数组寻址均使用零起始下标。

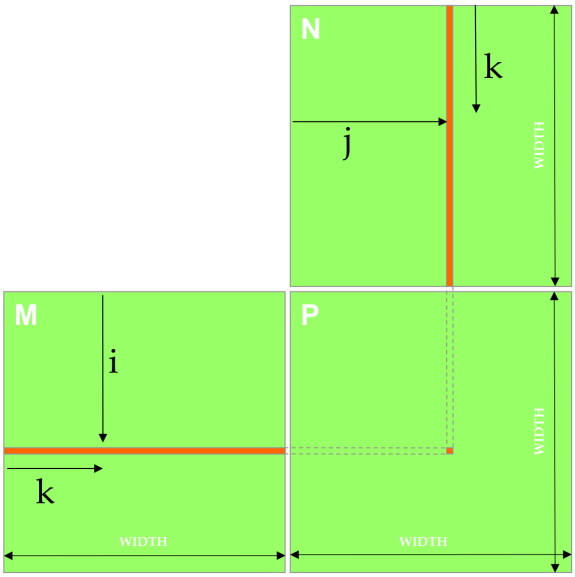


图 5.4 沿 M 的一行和 N 的一列取数，计算它们在 P 中交会位置的元素。

行主序 (*row-major order*) 将同一行的元素连续存储。在每行包含 W 个元素时，跳过前 i

行需要跳过 iW 个元素，再在当前行向右移动 j 个元素，因此二维位置与一维偏移之间的对应关系为

$$(i, j) \longleftrightarrow iW + j. \tag{5.2}$$

对公式(5.1)中的三个矩阵，这给出

$$M_{ik} \leftrightarrow \texttt{M[i * Width + k]}, \qquad N_{kj} \leftrightarrow \texttt{N[k * Width + j]},$$

$$P_{ij} \leftrightarrow \texttt{P[i * Width + j]}.$$

这些表达式的下标以元素为单位；数组元素为 `float` 时，访存字节量另按每个元素 4 B 计算。

5.3.1.2 CPU 三重循环

固定 i 与 j 后，内层循环用一个私有累加器计算点积。外层的两重循环则遍历所有输出位置。

代码 5.1 基本 CPU 矩阵乘法，保留行主序寻址和三重循环结构。

```
void MatrixMul(float *M, float *N, float *P, int Width)
{
    for (int i = 0; i < Width; ++i) {
        for (int j = 0; j < Width; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < Width; ++k) {
                float a = M[i * Width + k];
                float b = N[k * Width + j];
                sum += a * b;
            }
            P[i * Width + j] = sum;
        }
    }
}
```

5.3.1.3 并行化输出元素

不同的 P_{ij} 使用不同的累加器，并写入不同的输出位置；它们共同读取 M, N，但计算一个输出元素不依赖另一个输出元素的完成。因此，可以将外层的 (i, j) 遍历映射为并行线程，让**一个线程负责一个输出元素**，而该线程内部仍顺序执行 k 循环。

把内层求和本身并行化会涉及**归约**（*reduction*）。浮点加法通常不满足结合律，改变求和顺序可能改变舍入结果；每个线程内部的顺序累加保持固定的求和次序。

CPU 循环与 GPU 线程的对应关系

CPU 的两个外层循环负责“选择输出位置”，GPU 用线程的二维索引完成这项选择。内层循环负责“计算该位置上的完整点积”，仍由被分配到该位置的线程执行。

5.3.2 把输出矩阵划分给二维线程块

5.3.2.1 线程块、输出分块与网格大小

令 T 为 `TILE_WIDTH`，即一个方形线程块在两个方向上的宽度。一个 $T \times T$ 的线程块包含 T^2 个线程，负责输出矩阵中的一个 $T \times T$ 子区域，称为一个**分块**（*tile*）。

当 W 能被 T 整除时，两个方向各需要 W/T 个块。一般情况下需要向上取整，使边缘位置也被覆盖：

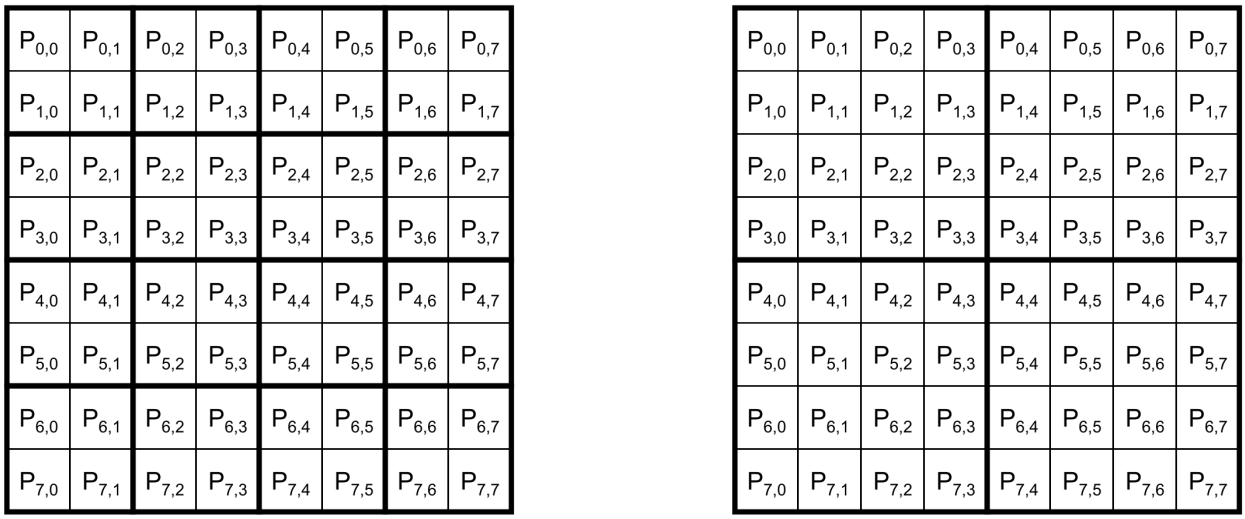
$$G_x = G_y = \left\lceil \frac{W}{T} \right\rceil, \qquad N_{\text{blocks}} = G_x G_y = \left\lceil \frac{W}{T} \right\rceil^2. \tag{5.3}$$

这里的分块首先用于分配*输出计算*。共享内存分块矩阵乘法进一步把 M, N 的输入子矩阵显式装入共享内存，以组织输入数据复用。

5.3.2.2 三个网格配置例子

矩阵宽 W	块宽 T	每块线程数	网格尺寸	总块数
4	2	$2^2 = 4$	2×2	4
8	2	$2^2 = 4$	4×4	16
8	4	$4^2 = 16$	2×2	4

表 5.3 输出分块大小决定单块工作量及网格中线程块的数量。



(a) $W = 8, T = 2$：16 个块。 **(b)** $W = 8, T = 4$：4 个块。

图 5.5 相同的 8×8 输出矩阵可以使用不同大小的输出分块；粗线表示块之间的边界。

对这两个 $W = 8$ 的整除例子，总线程数都是 64，每个线程仍计算一个输出元素。改变 T 调整了线程分组，但基本 kernel 的每个有效线程仍然执行长度为 W 的点积。

5.3.3 线程坐标如何确定输入与输出地址

5.3.3.1 从块索引与块内索引得到行列

记块索引为 (b_x, b_y)，块内线程索引为 (t_x, t_y)，块尺寸为 (B_x, B_y)。它们分别对应 `blockIdx`、`threadIdx` 与 `blockDim` 的两个分量。线程负责的全局行、列为

$$\text{Row} = b_y B_y + t_y, \qquad \text{Col} = b_x B_x + t_x. \tag{5.4}$$

x 方向沿矩阵的列变化，y 方向沿行变化。乘以块尺寸给出当前块的起点，再加上块内偏移，得到整个输出矩阵中的位置。

当 $T = 2$ 且 $(b_x, b_y) = (0, 0)$ 时，该块负责行 $0, 1$ 和列 $0, 1$，四个线程与输出的对应关系如下。

(t_x, t_y)	Row	Col	负责的输出
$(0, 0)$	0	0	$P_{0,0}$
$(1, 0)$	0	1	$P_{0,1}$
$(0, 1)$	1	0	$P_{1,0}$
$(1, 1)$	1	1	$P_{1,1}$

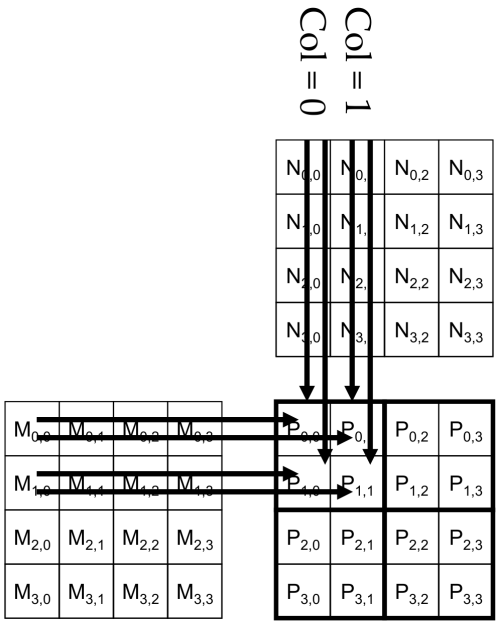


图 5.6 左上角线程块中的四个线程读取 M 的前两行与 N 的前两列，各自形成一个完整点积。

向右相邻的块满足 $(b_x, b_y) = (1, 0)$，其行仍为 $0, 1$，列变为 $2, 3$。CUDA 块坐标按 (b_x, b_y) 排列，b_x 表示水平方向，b_y 表示垂直方向。

5.3.3.2 固定一个线程，追踪整个点积

取 $W = 4$、Row $= 1$、Col $= 2$。线程最终写入 $P_{1,2}$，对应一维位置 $1 \times 4 + 2 = 6$。内层循环的四次访问为

k	M 的元素与偏移	N 的元素与偏移	对累加器的贡献
0	$M_{1,0}$，偏移 4	$N_{0,2}$，偏移 2	$M_{1,0}N_{0,2}$
1	$M_{1,1}$，偏移 5	$N_{1,2}$，偏移 6	$M_{1,1}N_{1,2}$
2	$M_{1,2}$，偏移 6	$N_{2,2}$，偏移 10	$M_{1,2}N_{2,2}$
3	$M_{1,3}$，偏移 7	$N_{3,2}$，偏移 14	$M_{1,3}N_{3,2}$

沿 k 增长时，M 的地址每次增加 1，N 的地址每次增加 W。这是一个线程内部沿一行和一列取数所产生的地址序列。每次读入两个元素并相乘累加，四次循环结束后，只需将累加器写入一次输出数组。

5.3.4 kernel、启动配置与边界保护

5.3.4.1 完整的基本 kernel

代码 5.2 一个线程计算一个输出元素的基本矩阵乘法 kernel。

```
__global__
void MatrixMulKernel(float *d_M, float *d_N,
                     float *d_P, int Width)
{
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    if ((Row < Width) && (Col < Width)) {
        float Pvalue = 0.0f;
        for (int k = 0; k < Width; ++k) {
            Pvalue += d_M[Row * Width + k]
                    * d_N[k * Width + Col];
        }
        d_P[Row * Width + Col] = Pvalue;
    }
}
```

`Row`、`Col`、`k` 与 `Pvalue` 均为线程自己的状态。`d_M`、`d_N` 和 `d_P` 指向设备数组，其中循环内两处下标访问读取全局数据，循环后的下标访问写回输出。`Pvalue` 在整个循环中保留累加结果，避免每做一次乘加就把部分和写回输出数组。

5.3.4.2 主机侧启动配置

令 `TILE_WIDTH` 为预先定义的正整数常量，三个设备数组已经准备好，`Width` 为矩阵宽度。启动片段如下；`ceil` 对应向上取整，C++ 中可在包含 `<cmath>` 后使用 `std::ceil`。

代码 5.3 二维网格与二维线程块的启动片段。

```
dim3 dimGrid(std::ceil((1.0 * Width) / TILE_WIDTH),
             std::ceil((1.0 * Width) / TILE_WIDTH), 1);
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);

MatrixMulKernel<<<dimGrid, dimBlock>>>(
    d_M, d_N, d_P, Width);
```

`1.0 * Width` 使除法在浮点数范围内进行，从而在向上取整前保留余数对应的小数部分。对正整数 W, T，同一数量也可用整数算术表达为

$$\left\lceil \frac{W}{T} \right\rceil = \left\lfloor \frac{W + T - 1}{T} \right\rfloor,$$

因此可以用 `(Width + TILE_WIDTH - 1) / TILE_WIDTH` 计算每个方向的块数。两个表达式都表示“有余数时再增加一个块”。

5.3.4.3 向上取整后为什么需要边界判断

取 $W = 5, T = 2$，每个方向需要 $\lceil 5/2 \rceil = 3$ 个块，共 9 个块、36 个线程。它们覆盖一个 6×6 的线程坐标区域，而有效输出只包含 $5 \times 5 = 25$ 个位置。因此，有 11 个线程对应超出矩阵边缘的坐标。

`if ((Row < Width) && (Col < Width))` 同时检查行与列，使这些边缘线程跳过整个点积和输出写入。有效线程内部有 $0 \le k < W$，因此 M 的列索引与 N 的行索引也位于范围内。

任一有效位置 (r, c) 都可以唯一分解为

$$b_y = \lfloor r/T \rfloor, \quad t_y = r \bmod T, \qquad b_x = \lfloor c/T \rfloor, \quad t_x = c \bmod T.$$

这说明向上取整的网格覆盖了所有有效输出，并且每个输出都被唯一一个线程负责。不同输出可以重复读取相同输入，但写入目标彼此不同。

5.4 访存量、算术强度与 Roofline 性能分析

5.4.1 基本矩阵乘法为什么容易受带宽限制

5.4.1.1 逐次迭代统计操作数与运算量

基本矩阵乘法将输出位置分配给大量线程，但每个线程在每次 k 循环中都要读取 `d_M[Row * Width + k]` 和 `d_N[k * Width + Col]`。两个 `float` 元素各为 4 B，因此每次迭代请求 8 B 输入数据，同时执行一次浮点乘法和一次浮点加法。

浮点运算（*floating-point operation, FLOP*）是这里的运算计数单位：一次乘法与一次加法合计 2 FLOP。将数据需求除以运算量，得到

$$R_{\text{req}} = \frac{8\ \text{B}}{2\ \text{FLOP}} = 4\ \text{B/FLOP}. \tag{5.5}$$

其倒数是每读取一字节所完成的运算量，即**算术强度**（*arithmetic intensity*）。用 I 表示，基本

kernel 在不计缓存复用、忽略最终写回的口径下有

$$I_{\text{req}} = \frac{2}{8} = 0.25 \text{ FLOP/B}. \tag{5.6}$$

设总浮点运算量为 F，源代码请求的总字节量为 Q_{req}。保留最后一次输出写回时，计数可逐层展开如下。

计数范围	浮点运算量	请求的字节量
一个线程的一次 k 迭代	2 FLOP	8 B 输入
一个有效线程的完整点积	$2W$ FLOP	$8W + 4$ B
整个 $W \times W$ 输出	$F = 2W^3$ FLOP	$Q_{\text{req}} = 8W^3 + 4W^2$ B

由此得到更完整的请求级算术强度

$$I_{\text{req}} = \frac{2W^3}{8W^3 + 4W^2} = \frac{W}{4W + 2} \xrightarrow[W\to\infty]{} \frac{1}{4} \text{ FLOP/B}. \tag{5.7}$$

输出写入在总请求流量中的比例随 W 增大而减小，因此大矩阵分析中忽略它，可以得到简洁的 0.25 FLOP/B 估计。

请求流量与设备内存流量

上述 Q_{req} 统计各线程在代码中请求的操作数。缓存复用可以让多个请求由已缓存数据响应，使实际到达所建模存储层的字节量 Q 减少。用设备内存带宽估计性能时，算术强度应按该层的 $I = F/Q$ 计算；忽略缓存复用时，才把上述请求计数作为该层流量的近似。

5.4.1.2 150 GB/s 如何得到 37.5 GFLOP/s

考虑给定的示例处理器：计算吞吐上限为 1000 GFLOP/s，全局内存带宽为 150 GB/s。若每执行 1 FLOP 都需要从该层取得 4 B 数据，则内存每秒能支撑的运算量为

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s}. \tag{5.8}$$

也可以写成 150 GB/s × 0.25 FLOP/B。字节单位消去后，结果为运算吞吐率。

这解释了瓶颈：计算单元每秒最多能够完成 1000 G 次浮点运算，但当前的数据需求让带宽只能支撑其中的 37.5 G 次。大量输出线程提供了并行任务，却没有减少每单位计算需要搬运的数据。

在这一示例中，37.5 GFLOP/s 是按给定访存模型得到的带宽上限；原材料给出的执行表

现约为 25 GFLOP/s，并解释为内存并非始终处于满负荷状态。提高性能的方向是减少重复的全局数据访问，使每次搬入的数据参与更多运算。上述数值均属于该教学示例。

5.4.2 从两个峰值建立 Roofline 模型

5.4.2.1 运算吞吐率与内存带宽

浮点运算吞吐率(*FLOPS rate*)表示每秒能够完成多少浮点运算，单位为 FLOP/s。**内存带宽**(*memory bandwidth*)表示每秒能够提供多少数据，单位为 B/s。前者度量计算供给，后者度量数据供给。

用 C_{peak} 表示峰值计算吞吐率，B_{peak} 表示所建模存储层的峰值带宽；设一个 kernel 的总工作量为 F FLOP，在该层传输 Q B 数据，执行时间为 t。则算术强度和实际运算吞吐率分别为

$$I = \frac{F}{Q} \quad \text{(FLOP/B)}, \qquad \mathcal{P} = \frac{F}{t} \quad \text{(FLOP/s)}. \tag{5.9}$$

F 是运算总量，$\mathcal{P}$ 是运算速率；相应地，Q 是数据总量，Q/t 是平均数据传输速率。计算例子中的 G 和 T 按十进制量级 10^9、10^{12} 使用。

5.4.2.2 性能上限与时间下限

计算单元提供了 C_{peak} 的吞吐上限；内存以 B_{peak} B/s 供给数据，每字节可支持 I FLOP，因而给出 $B_{\text{peak}}I$ 的另一条上限。两项限制必须同时满足：

$$\boxed{\mathcal{P} \le \min\bigl(C_{\text{peak}}, B_{\text{peak}}I\bigr).} \tag{5.10}$$

这就是 **Roofline 模型 (*roofline model*)** 的基本关系。

用执行时间表达同一限制，计算所需时间至少为 F/C_{peak}，数据传输所需时间至少为 Q/B_{peak}，因此

$$\boxed{t \ge t_{\min} = \max\!\left(\frac{F}{C_{\text{peak}}}, \frac{Q}{B_{\text{peak}}}\right).} \tag{5.11}$$

这里取两项的最大值，表达两条下限必须同时成立；该式没有把两个阶段假定为必然串行。即便达到较小的一项，仍须满足较大的时间需求。

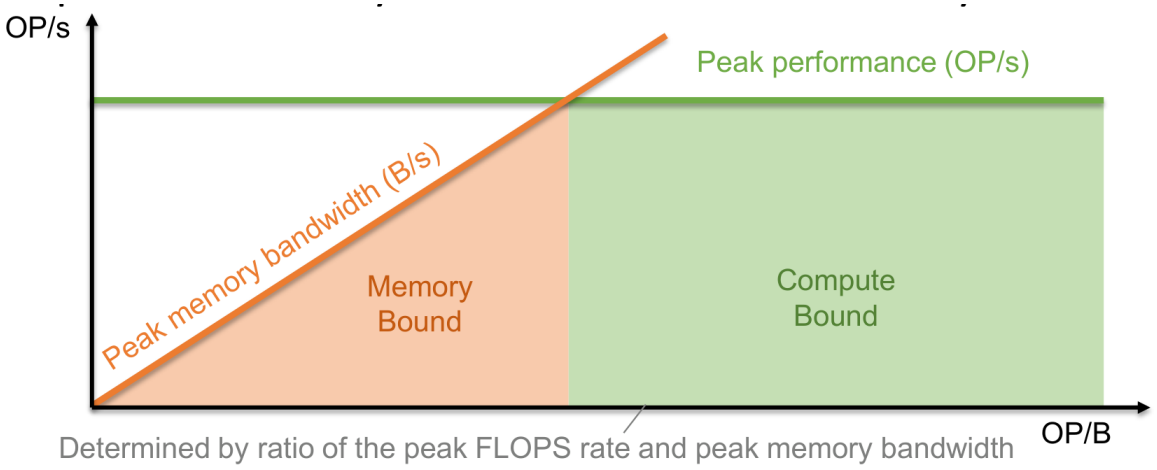


图 5.7 横轴为算术强度，纵轴为运算吞吐率。斜线给出带宽限制，水平线给出计算限制；可达到的性能位于两条限制的下方。

5.4.2.3 屋脊点与受限类型

两条上限相交的位置称为**屋脊点**(*ridge point*)。令 $C_{\text{peak}} = B_{\text{peak}} I_*$，得到

$$I_* = \frac{C_{\text{peak}}}{B_{\text{peak}}}. \tag{5.12}$$

当 $I < I_*$ 时，带宽提供的性能上限较低，对应**内存受限**(*memory-bound*)区域：计算核心可能等待内存提供数据。当 $I > I_*$ 时，计算吞吐上限较低，对应**计算受限**(*compute-bound*)区域。这里讨论的是 Roofline 模型中的限制关系，实际执行性能可以位于上限之下。

沿斜线区域向右移动，表示每字节数据参与更多运算，带宽所能支撑的 FLOP/s 随之提高；到达屋脊后，继续提高算术强度也不会抬高计算单元本身的吞吐上限。

表 5.4 给定 GPU 峰值参数与屋脊点。单位分别为 GFLOP/s、GB/s 和 FLOP/B。

GPU	微架构	C_{peak}	B_{peak}	I_*
K40	Kepler	5,040	288	17.5
M40	Maxwell	6,844	288	23.8
P100	Pascal	9,340	732	12.8
V100	Volta	14,028	900	15.6
A100	Ampere	19,500	1,555	12.5
H100	Hopper	66,900	3,352	20.0

同一个 I 在不同硬件上，需要与不同的 I_* 比较。峰值参数是给定计算条件下的性能上限，不能直接用作任意 kernel 的预期实测速率。

以 H100 的示例参数为例，采用近似值计算，有

$$I_* \approx \frac{67\ \text{TFLOP/s}}{3.35\ \text{TB/s}} \approx 20\ \text{FLOP/B}.$$

基本矩阵乘法的请求级估计仅约 0.25 FLOP/B，在不计缓存复用的带宽模型中远低于这个屋脊点。

5.4.3 向量加法与矩阵乘法的对照

5.4.3.1 向量加法：每个输出都要读二写一

设向量长度为 n，每个元素为 4 B 的单精度浮点数，计算为

```
z[i] = x[i] + y[i];
```

每个输出需要读取两个输入元素、写入一个输出元素，共 $4+4+4=12$ B，只进行一次浮点加法。因而

$$F = n, \qquad Q = 12n, \qquad I_{\text{vec}} = \frac{1}{12} \approx 0.083\ \text{FLOP/B}. \tag{5.13}$$

这里输出写入占每个元素数据流量的三分之一，必须计入。矩阵乘法每个线程做 W 次乘加而只写一次输出，向量加法则每做一次加法就写一次输出，两者写回成本在总流量中的比重不同。

对 $n = 10^9$，运算量为 1 GFLOP，数据量为 12 GB。使用 H100 示例中的 3.35 TB/s 带宽，以及舍入后的 $I = 0.083$ FLOP/B，可得理论极限时间

$$t_{\min} \approx \frac{1 \times 10^9\ \text{FLOP}}{(0.083\ \text{FLOP/B})(3.35 \times 10^{12}\ \text{B/s})} \approx 3.60\ \text{ms}. \tag{5.14}$$

直接用未舍入的数据量计算 $12\times10^{9}/(3.35\times10^{12})$，得到约 3.58 ms；差别来自将 1/12 提前舍入为 0.083。

理论极限性能（*speed of light*）提供比较实测结果的参照。若测得 kernel 时间 t，则该模型下的平均有效带宽为 Q/t，与带宽上限的比值为

$$\eta_B = \frac{Q/t}{B_{\text{peak}}} = \frac{Q/B_{\text{peak}}}{t}.$$

对这一内存受限例子，它等于带宽时间下限与实测时间之比。比较时应沿用相同的数据量、存储层和计时范围。

5.4.3.2 矩阵乘法：理想复用下的算术强度

矩阵乘法包含 W^2 个点积，每个点积执行 W 次乘法与 W 次加法，因此

$$F = 2W^3.$$

在**理想复用**假设下，两个输入矩阵在所考察存储层各读入一次，输出矩阵写回一次。每个矩阵有 W^2 个 4 B 元素，于是

$$Q_{\text{ideal}} = 3\times W^2\times 4 = 12W^2 \text{ B},$$

$$I_{\text{ideal}} = \frac{2W^3}{12W^2} = \frac{W}{6} \approx 0.167W \text{ FLOP/B}. \tag{5.15}$$

计算量按 W^3 增长，而理想数据量按 W^2 增长，因此算法具有随规模增大的复用潜力。以 $I_{*}\approx 20$ FLOP/B 为例，理想强度与屋脊相交的条件为 $W/6\approx 20$，即 $W\approx 120$。这个交点属于理想复用模型，实际 kernel 的算术强度仍由实现形成的数据流量决定。

5.4.3.3 相同算法为何得到不同的强度

表 5.5 矩阵乘法的算法级复用潜力与基本实现的请求级计数。

分析口径	计数的访存行为	算术强度
理想复用	两个输入各读一次，输出写一次	$W/6$ FLOP/B
基本 kernel	每次乘加重新请求两个输入元素	约 0.25 FLOP/B
计入缓存复用	按实际到达所建模层的字节量计数	F/Q，取决于复用

固定 M_{ik}，它会参与同一输出行的所有 P_{ij}，即被不同的 j 使用；固定 N_{kj}，它会参与同一输出列的所有 P_{ij}，即被不同的 i 使用。因此，一个输入元素在数学上可以服务多个输出点积。

基本 kernel 将这些点积分给不同线程，但仍在每个线程内部独立请求操作数。缓存可以自然保留部分数据，显式组织复用则进一步利用上述行列关系。共享内存分块的方向是把一组输入先搬入块内可共享的存储，再让多个线程使用这组数据完成各自的计算。

两类例子指向的性能结论

向量加法的强度约为 0.083 FLOP/B，基本矩阵乘法的请求级估计约为 0.25 FLOP/B，都远低于 H100 示例的约 20 FLOP/B 屋脊点。矩阵乘法额外具有显著的输入复用潜力；实现这种复用，可以减小所建模内存层的数据流量，使算术强度向屋脊点移动。

5.5 例题解析与知识归纳

5.5.1 三道例题的条件与推导

5.5.1.1 256 行、4 列矩阵需要多少个线程块

题目。输入矩阵有 256 行、4 列，线程块为 16×16，每个线程处理一个输入元素。需要启动多少个线程块？

在忽略原矩阵二维几何形状、通过**展平或重映射**（*flattening / remapping*）自行定义索引的假设下，需要 **4 个块**。其数量计算为

$$\text{元素数} = 256 \times 4 = 1024, \qquad \text{每块线程数} = 16 \times 16 = 256,$$

$$N_{\text{blocks}} = 1024/256 = 4.$$

一个实现这种假设的映射为：将 4 个块沿网格的 x 方向排布，块内仍使用 16×16 个线程。先将块内二维线程索引展平，再映射回原矩阵：

$$\ell = t_y \times 16 + t_x, \qquad 0 \le \ell < 256,$$

$$g = b_x \times 256 + \ell, \qquad 0 \le g < 1024,$$

$$\text{row} = \lfloor g/4 \rfloor, \qquad \text{col} = g \bmod 4.$$

对应的索引计算片段为

代码 5.4 由原题的展平假设展开得到的索引片段。

```
int localIndex = threadIdx.y * blockDim.x + threadIdx.x;
int threadsPerBlock = blockDim.x * blockDim.y;
int index = blockIdx.x * threadsPerBlock + localIndex;

int row = index / 4;
int col = index % 4;
```

二维几何覆盖的对照。若继续使用矩阵乘法中 x 对应列、y 对应行的直接二维映射，则

$$G_x = \left\lceil \frac{4}{16} \right\rceil = 1, \qquad G_y = \left\lceil \frac{256}{16} \right\rceil = 16,$$

需要 $1 \times 16 = 16$ 个块。每块只有 4 列线程对应有效元素，其余 12 列需由边界判断跳过。这样共启动 4096 个线程，其中 1024 个有效。

块数随索引方案变化

展平重映射下，4 个块可让全部 1024 个线程各处理一个有效元素。直接二维覆盖下，需要 16 个块覆盖 256 行，并产生无效边缘线程。判断块数时，需要先确定线程如何对应数据坐标。

5.5.1.2 同一线程块中的线程如何交换数据

题目。某些线程需要读取同一个线程块中其他线程写入的数据。在寄存器、共享内存和全局内存中，哪一种最适合且访问最快？

答案：共享内存。所需存储必须同时满足“同一个块内的线程能够访问同一份数据”和“具有较低访问代价”。共享内存的作用域为线程块，典型位置为 SM，正好匹配这两项需求。

寄存器用于每个线程的私有工作值，不能作为这道题中的普通块内共享存储；全局内存支持跨线程访问，但在给定的存储层次中具有更高的访问代价。因此，在三个选项中，共享内存最合适。

5.5.1.3 两个线程块写同一地址会得到什么结果

题目。设备端分配一个 `char` 元素，初始值为 3。执行下列 kernel 后，`*dst` 的可能值是什么？

代码 5.5 两个线程块各用一个线程写入同一全局地址。

```
__global__ void kernel(char *dst)
{
    dst[0] = blockIdx.x;
}

// dst[0] was initialized to 3 on the device.
kernel<<<2, 1>>>(dst);
```

按独立写入先后解释的结果：0 或 1。启动配置包含两个线程块，每个块只有一个线程。块 0 向 `dst[0]` 写入 0，块 1 向同一地址写入 1；二者都覆盖原来的值 3。由于块的执行顺序没有保证，在这一写入顺序模型中，最终写入者可以不同。

多个线程块在缺少确定次序的情况下写同一个位置，形成**数据竞争**(*data race*)。全局内存的设备可见性允许线程访问该地址，访问权限本身没有确定谁先写、谁后写。

基本矩阵乘法的输出映射让每个有效位置只有一个写入线程，因此其输出写入不具有这道题的同址竞争。判断并行代码时，需要同时检查输入可以共享读取，以及输出是否被多个线程重复写入。

5.5.2 关键公式与术语

5.5.2.1 从地址映射到性能上限

下表集中给出前述推导中的关系。W 为方阵宽度，T 为方形线程块宽度；F 为浮点运算量，Q 为所建模存储层的传输字节量。

问题	关系与条件
二维行主序寻址	$(r,c)\mapsto rW+c$。
线程负责的位置	$\mathrm{Row}=b_yB_y+t_y$；$\mathrm{Col}=b_xB_x+t_x$。
方阵网格大小	$G_x=G_y=\lceil W/T\rceil$；总块数为 G_xG_y。
基本矩阵乘法计数	$F=2W^3$；$Q_{\mathrm{req}}=8W^3+4W^2$ B。
算术强度	$I=F/Q$，单位为 FLOP/B。
基本矩阵乘法强度	$I_{\mathrm{req}}=W/(4W+2)\approx 0.25$ FLOP/B。
向量加法强度	读二写一，$I=1/12\approx 0.083$ FLOP/B。
矩阵乘法理想强度	输入各读一次、输出写一次，$I_{\mathrm{ideal}}=W/6$。
Roofline 吞吐上限	$\mathcal{P}\le\min(C_{\mathrm{peak}},B_{\mathrm{peak}}I)$。
屋脊点	$I_*=C_{\mathrm{peak}}/B_{\mathrm{peak}}$。
执行时间下限	$t_{\min}=\max(F/C_{\mathrm{peak}},Q/B_{\mathrm{peak}})$。

存储空间决定数据由哪些线程访问、典型位置在哪里；索引映射决定线程实际读写哪些元素；访存量和算术强度把实现与硬件上限连接起来。分析一个 kernel 时，这三个层次应使用相互一致的数据与计数口径。

5.5.2.2 中英文术语对照

中文	英文	对应含义
寄存器 / 寄存器堆	Register / register file	线程当前工作值所在的低延迟存储。
流式多处理器	Streaming multiprocessor (SM)	包含运算核心及片上存储资源。
共享内存	Shared memory	同一线程块共同访问的存储。
全局内存	Global memory	保存设备可见的大型输入输出。
常量内存	Constant memory	设备线程只读，适于一致访问。
局部内存	Local memory	线程私有，由设备内存支持。
缓存	Cache	保留数据副本以支持后续访问。
作用域 / 生命周期	Scope / lifetime	谁能访问，以及存储存在多久。
行主序	Row-major order	同一行的元素连续存储。
分块	Tile / tiling	将矩阵划分为线程块负责的区域。
归约	Reduction	将多个值汇合为一个结果，例如求和。
展平 / 重映射	Flattening / remapping	在数据与线程坐标之间重新建立对应。
浮点运算 / 吞吐率	FLOP / FLOPS rate	运算次数 / 每秒运算次数。
内存带宽	Memory bandwidth	每秒可传输的数据字节数。
算术强度	Arithmetic intensity	每传输一字节对应的运算量。
屋脊点	Ridge point	计算上限与带宽上限的交点。
内存受限 / 计算受限	Memory-bound / compute-bound	由数据供给 / 计算供给主导的限制。
理论极限性能	Speed of light	给定模型下最好可能达到的性能。
数据竞争	Data race	原题中多个线程块无确定次序地写同一地址。

第 6 讲

Data Locality and Tiled Matrix Multiply

6.1 课程事项与核心内容

6.1.1 学习目标与课程事项

学习目标为：为 Lab 3 的分块矩阵乘法做准备，处理分块算法的边界条件，以及评估分块带来的收益。技术内容围绕同一个计算问题展开：将矩阵乘法中的输入复用转化为较少的全局内存访问，同时保证线程协作的正确性。

6.1.1.1 作业与阅读安排

以下课程事项对应 2026 年 9 月 10 日。Lab 1 在当周周五截止，要求先完成 Lab 0，且 Lab 1 不计入最终成绩。阅读材料为教材第 5 章；待办事项包括获取 Delta 账号、设置 GitHub，以及提交 Labs 0 和 1。

实验和项目的通常截止时间为周五 20:00，默认提供三天宽限期：周一 20:00 前提交仍可获得完整分数，无需提前通知。若需要超过三天的宽限期，必须在周五 20:00 前，通过邮件向负责相应作业的助教提出申请；是否批准取决于具体特殊情况，超过这一申请时间的请求不予考虑。最低分的一次实验会从总评中剔除；在上述延期政策之外，实验和项目不再接受逾期提交。

6.1.1.2 概念关系

全局内存带宽限制了数据供给速度；共享内存使线程块可以显式保存并复用一小块输入。矩阵乘法沿内积维度逐块累加，形成“协作加载—同步—局部计算—同步”的循环。边界处理维持这一循环的统一结构，性能分析则比较数据复用与线程块资源消耗。

6.1.2 Delta 命令行支持工具

6.1.2.1 启动与用途

命令行支持代理可协助排查 Delta 使用问题，例如实验无法运行、SLURM 作业状态查询，也可辅助实验代码编写。使用前查看 `lab1` 目录下的 `hpcGPT.md`。启动命令为：

```
module load hpc-gpt/1.18.23-l
opencode
```

该模块版本对应 2026 年 9 月 10 日的支持工具入口。

内存分配示例涉及三个数组：两个输入和一个输出都需要对应的设备端空间。分配量由元素个数乘以 `sizeof(float)` 得到，单位是字节；CUDA 分配函数需要设备指针的地址，以便把分配结果写回该指针。

6.1.2.2 内置命令

输入 `/` 可以调出内置命令列表。下表按用途归类保留命令及其含义。

用途	命令与含义
会话与目录	`/new` 新会话；`/sessions` 切换会话；`/move` 更换项目目录；`/exit` 退出。
代理与连接	`/agents` 切换代理；`/models` 切换模型；`/connect` 连接提供方；`/mcps` 切换 MCP。
作业与状态	`/jobs` 显示或隐藏侧栏 SLURM 作业；`/jobs-refresh` 立即刷新作业；`/status` 查看状态。
编辑与检查	`/editor` 打开编辑器；`/diff` 查看差异；`/review` 检查变更，支持 commit、branch、pr，默认检查未提交变更。
配置与帮助	`/init` 引导设置 `AGENTS.md`；`/skills` 查看技能；`/themes` 主题；`/help` 帮助。
诊断与反馈	`/debug` 调试信息；`/feedback` 附带当前对话提交反馈；`/report` 向 Delta 支持团队报告问题。

6.2 数据局部性与分块矩阵乘法

6.2.1 从内积计算到输入复用

6.2.1.1 矩阵、线程与下标约定

先考虑两个 $N \times N$ 矩阵相乘，$C = AB$。采用与数组一致的零起始下标，有

$$C_{r,c} = \sum_{j=0}^{N-1} A_{r,j} B_{j,c}, \qquad 0 \le r, c < N. \tag{6.1}$$

每个输出元素是 A 的一行与 B 的一列的内积。基本线程映射是：一个线程负责一个 $C_{r,c}$，在线程私有的累加变量中保存部分和，内积完成后写回一次。

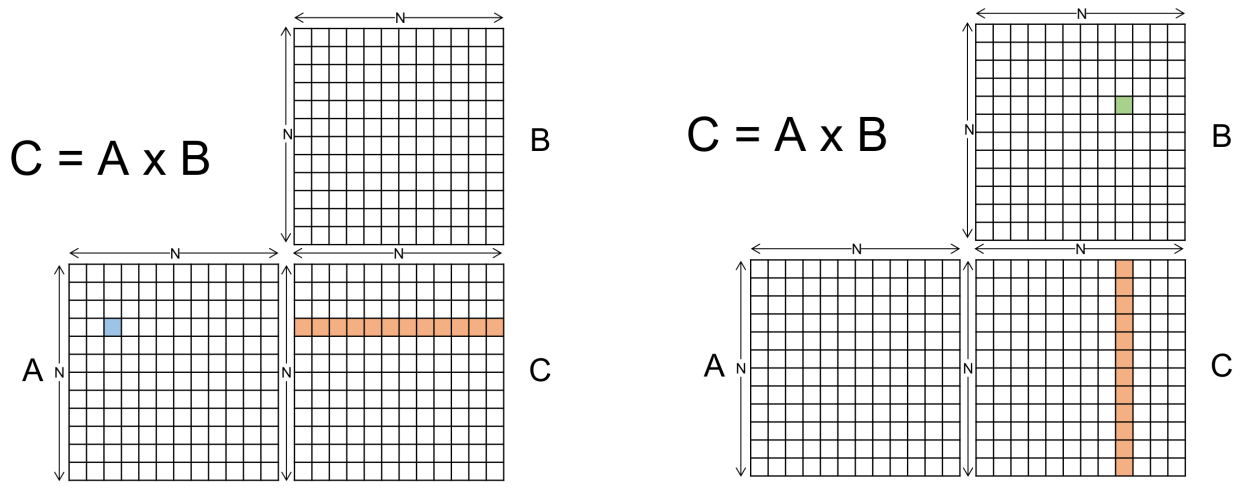
令 T 表示方形分块的边长，对应代码中的 `TILE_DIM` 或 `TILE_WIDTH`。一个线程块含 $T \times T$ 个线程，负责一个同样大小的输出分块。线程块坐标为 (b_x, b_y)，块内线程坐标为 (t_x, t_y)；横坐标对应列，纵坐标对应行：

$$r = b_y T + t_y, \qquad c = b_x T + t_x, \qquad 0 \le t_x, t_y < T. \tag{6.2}$$

后续使用 q 表示沿内积方向的分块编号，k 表示某一分块内部的内积下标，因此完整内积下标为 $j = qT + k$。这三个层次分别回答“哪个输出块”“哪个输出元素”和“当前使用哪一段输入”。

6.2.1.2 同一个输入服务于多个输出

固定 r, j，元素 $A_{r,j}$ 参与 C 的第 r 行全部 N 个输出；固定 j, c，元素 $B_{j,c}$ 参与 C 的第 c 列全部 N 个输出。因此，输入矩阵中每个元素都具有重复使用的机会。



(a) A 的一个元素服务于一行输出。

(b) B 的一个元素服务于一列输出。

图 6.1 矩阵乘法的输入复用关系。

当一个线程块负责 $T \times T$ 个输出时，同一行的 T 个线程会使用相同的 A 元素，同一列的 T 个线程会使用相同的 B 元素。将这些共同需要的输入保存在块内共享的存储区域，就能让一次全局加载支撑多次计算。

6.2.2 局部性与共享内存

6.2.2.1 带宽为什么会限制运算速度

全局内存（*global memory*）由 DRAM 实现。线程读取的数据有时已经位于 L1 缓存中；也可能在另一个线程再次使用之前被逐出。**共享内存**（*shared memory*）为线程块提供一个显式管理的、较小而较快的数据区域，可以直接组织所需的复用。

数据局部性（*data locality*）在这里体现为：把一组即将反复访问的数据集中起来，在它们仍然可用时完成尽可能多的相关计算。按一次乘法和一次加法合计两个浮点运算计数，每个内积步读取两个 `float` 输入，需要 8 字节，得到

$$\frac{8\ \text{B}}{2\ \text{FLOP}} = 4\ \text{B/FLOP}. \tag{6.3}$$

暂时忽略输出写回和缓存复用。对于计算峰值 1000 GFLOP/s、内存带宽 150 GB/s 的示例 GPU，内存只能支撑

$$\frac{150\ \text{GB/s}}{4\ \text{B/FLOP}} = 37.5\ \text{GFLOP/s}. \tag{6.4}$$

计算单元即使具有更高峰值，也需要足够快的数据供给才能发挥作用。这组参数说明了带宽

对数据供给的限制。

6.2.2.2 分块使数据复用成为显式协作

分块 (tiling) 将数据划分为能够放入共享内存的子集。线程块协作把当前子集从全局内存搬入共享内存，随后反复使用它完成计算，再处理下一组输入。在基本分块矩阵乘法中，每个线程每轮加载一个 A 元素和一个 B 元素，整个线程块共同填满两个 $T \times T$ 的共享数组。

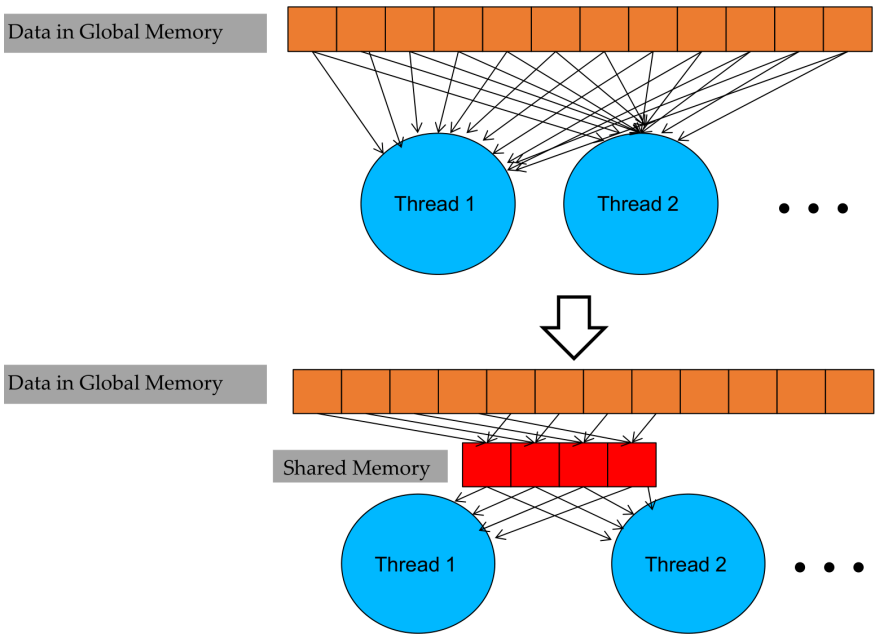


图 6.2 多线程直接重复读取全局数据，以及经共享内存组织复用的两种访问方式。

数据的存放位置

A、B 的完整输入和 C 的完整输出位于全局内存；当前两个输入分块位于共享内存；每个线程的部分和 `sum` 位于寄存器中。一个输出块的结果由该块全部线程的私有部分和共同构成。计算结束时，各线程把自己的最终结果直接写回 C。

6.2.3 沿内积维度推进分块

6.2.3.1 固定输出块，依次更换输入块

先假定 N 是 T 的整数倍。输出块的 (b_x, b_y) 在整个核函数执行中保持不变，而输入分块随 $q = 0, 1, \ldots, N/T-1$ 推进。第 q 轮使用 A 的列区间 $[qT, (q+1)T)$ 和 B 的行区间 $[qT, (q+1)T)$。

把式 (6.1) 的求和下标写成 $j = qT + k$，得到与分块循环直接对应的等式：

$$C_{r,c} = \sum_{q=0}^{N/T-1} \left(\sum_{k=0}^{T-1} A_{r,qT+k} B_{qT+k,c} \right). \tag{6.5}$$

内层求和产生一个部分内积，外层求和累加所有输入分块的贡献。此重排用于说明分块算法与原矩阵乘法的对应关系。

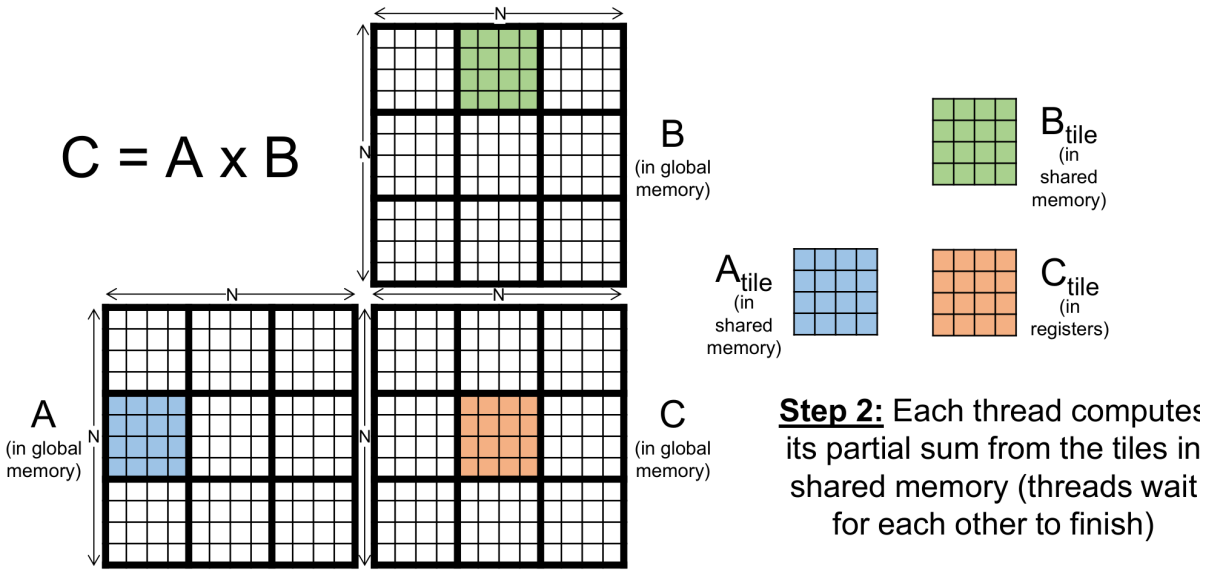


图 6.3 一个输出块对应一对当前输入块；输入块位于共享内存，输出部分和位于各线程寄存器。

6.2.3.2 协作加载的二维索引

线程 (t_x, t_y) 把两个输入元素分别写入共享数组的同一个局部位置 $[t_y][t_x]$：

$$A_s[t_y][t_x] \leftarrow A[r][qT + t_x], \tag{6.6}$$

$$B_s[t_y][t_x] \leftarrow B[qT + t_y][c]. \tag{6.7}$$

对于 A，全局行号固定在当前输出行，沿列方向推进分块；对于 B，全局列号固定在当前输出列，沿行方向推进分块。因此，A 的内积方向由 t_x 定位，B 的内积方向由 t_y 定位。

第 0 轮分别读取 $A[r][t_x]$、$B[t_y][c]$；第 1 轮分别读取 $A[r][T+t_x]$、$B[T+t_y][c]$。块内所有线程联合覆盖整对输入分块。加载阶段还应使同一 warp 中的访问便于合并访问 (coalesced access)。

6.2.3.3 行主序的一维地址

行主序 (row-major order) 将每一行连续存放。对于列数为 N 的数组，二维位置 (i, j) 的一维偏移为 $iN + j$。式 (6.7) 因而变为：

```
A_s[ty][tx] = A[row * N + q * TILE_DIM + tx];
B_s[ty][tx] = B[(q * TILE_DIM + ty) * N + col];
```

A 中的 $qT + t_x$ 已经是列偏移，无需再乘以行宽；B 中的 $qT + t_y$ 是行号，必须先乘以行宽。共享数组在声明时具有二维形状，可以继续使用二维下标。

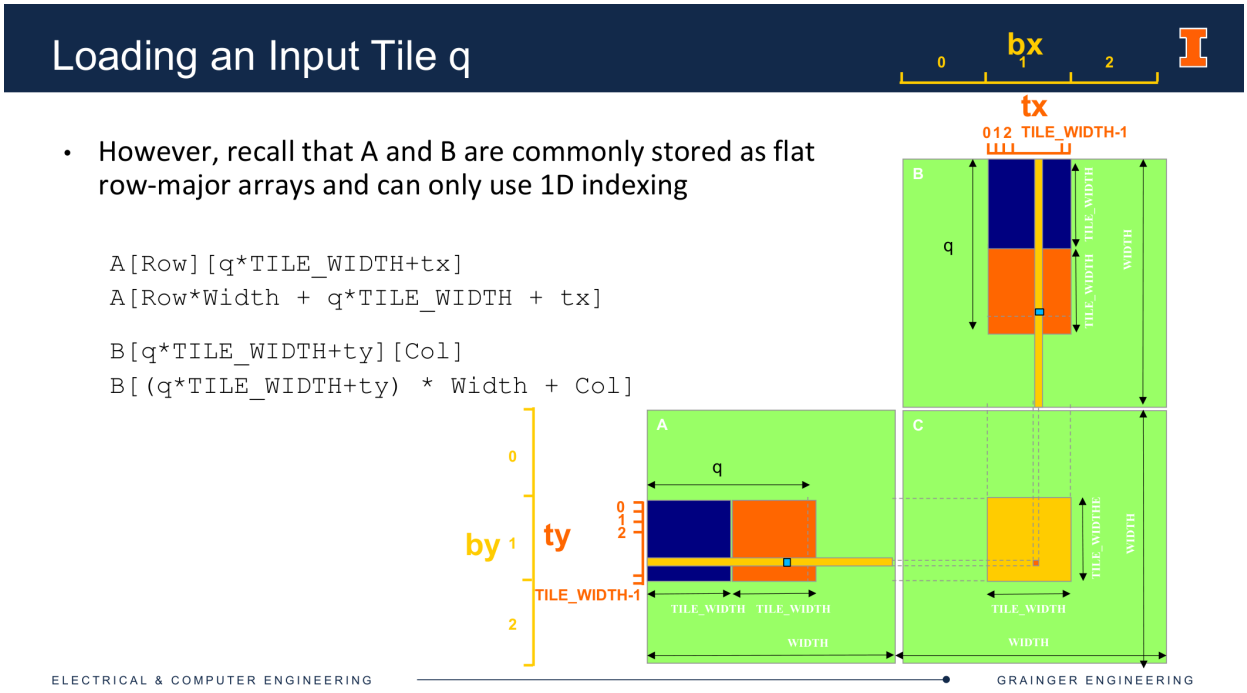


图 6.4 输出块坐标固定，A 的输入块向右移动，B 的输入块向下移动；两者由同一分块编号 q 控制。

6.2.3.4 计算时读取共享行与共享列

分块加载完成后，线程 (t_x, t_y) 按自己的输出位置读取共享数组的一行与一列：

```
for (int k = 0; k < TILE_DIM; ++k) {
    sum += A_s[ty][k] * B_s[k][tx];
}
```

将加载定义代入可验证，每一步使用的恰好是

$$A_s[t_y][k]B_s[k][t_x] = A_{r,qT+k}B_{qT+k,c}.$$

加载时，线程负责 $[t_y][t_x]$ 这个存储位置；计算时，它需要由多个线程加载的共享行和共享列。正是这一区分使协作加载产生复用，也使线程间同步成为必要条件。

例：一个线程在加载与计算中的不同职责

取 $N = 4$、$T = 2$，考虑线程块 $(b_x, b_y) = (0, 1)$ 中的线程 $(t_x, t_y) = (1, 0)$。由式 (6.2)，它负责 $C_{2,1}$。

在 $q = 0$ 时，该线程加载 $A_{2,1}$ 与 $B_{0,1}$，但其部分内积为

$$A_{2,0}B_{0,1} + A_{2,1}B_{1,1}.$$

其中 $A_{2,0}$ 和 $B_{1,1}$ 由其他线程加载。在 $q = 1$ 时，该线程加载 $A_{2,3}$ 与 $B_{2,1}$，并累加

$$A_{2,2}B_{2,1} + A_{2,3}B_{3,1}.$$

两轮得到完整的 $C_{2,1}$。该例用于逐项解释加载索引和计算索引，并沿用每线程一个输出的基本算法。

6.3 线程块同步与正确执行顺序

6.3.1 共享数据带来的两类依赖

6.3.1.1 使用之前，必须完成当前分块加载

每个线程只加载当前输入分块的一小部分，却会在内积循环中读取其他线程负责的元素。例如，线程 (t_x, t_y) 在第 k 步读取的 $A_s[t_y][k]$，由线程 (k, t_y) 加载；$B_s[k][t_x]$，由线程 (t_x, k) 加载。开始计算之前，必须保证这些加载已经完成。

6.3.1.2 覆盖之前，必须完成上一分块使用

两轮分块复用相同的共享数组 A_s、B_s。较快的线程如果提前进入下一轮，就可能把第 $q+1$ 轮数据写到仍被其他线程读取的位置。这会让某些乘积混用两轮输入。进入下一轮加载之前，需要确认当前轮的读取和计算已经结束。

这两类依赖分别保护“生产完成后再消费”和“消费完成后再覆盖”。只等待加载完成，不能保护共享数组的再次使用。

6.3.2 屏障与整体同步执行

6.3.2.1 屏障的含义

屏障（*barrier*）是线程集合共同到达的同步点。先到达的线程等待其余线程；参与线程全部到达后，才能继续执行屏障之后的代码。

整体同步执行（*bulk synchronous execution*）将并行计算划分为若干阶段：每个阶段内部，各线程并行完成自己的工作；阶段之间由屏障建立清晰的先后关系。线程在一个阶段内部的执行进度可以不同。

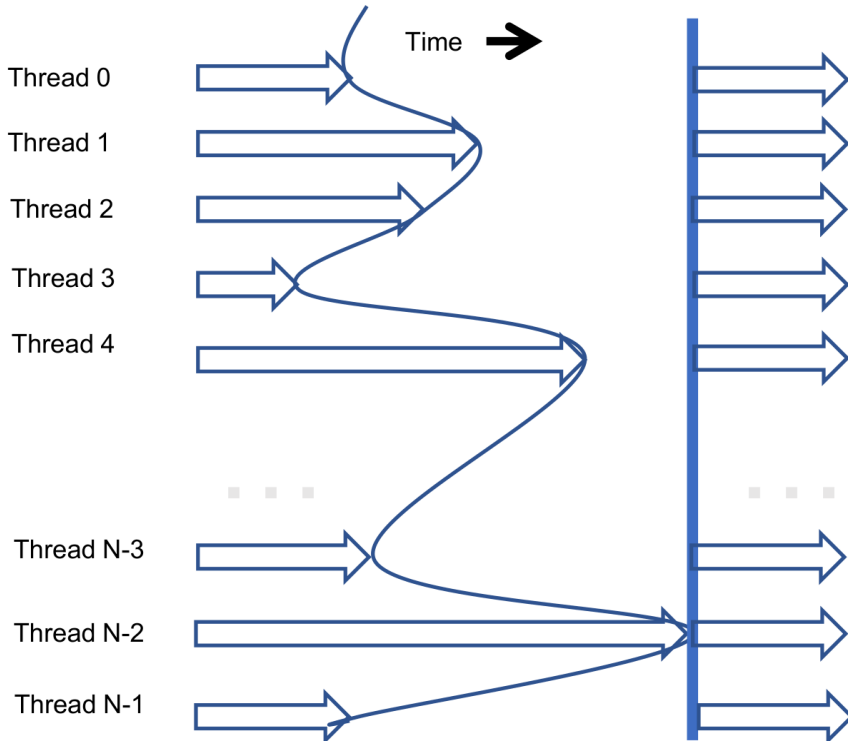


图 6.5 线程以不同进度到达屏障，在共同的同步点之后继续执行。

6.3.2.2 CUDA 的块内同步

`__syncthreads()` 是 CUDA 的设备端内置同步操作，用于同一线程块中的线程。线程块内的所有线程需要到达相应同步点，才能继续执行。

同步调用可以位于条件分支内，前提是该条件对整个参与线程块一致。如果只有一部分线程执行同步，执行可能挂起或产生错误行为。边界线程处理尤其需要保持这一点：线程的输入加载是否有效可以不同，但参与分块循环和同步的结构应保持一致。

每一轮的执行顺序

协作加载 A_s, B_s ⟶ `__syncthreads()`

当前分块的部分内积 ⟶ `__syncthreads()`

然后进入下一轮加载。

第一个屏障保证“数据已准备好”，第二个屏障保证“旧数据已使用完”。屏障作用于当前线程块；不同输出块具有各自的输入缓冲区和输出区域。

6.3.3 整除情形的完整核函数

下面将加载、两次同步、部分内积和写回组合起来。假设 $N > 0$ 且 T 整除 N，所有矩阵均按行主序存储；线程块大小固定为 $T \times T$，每线程负责一个输出。代码将 `TILE_DIM` 设为 16，这一取值仅用于具体化示例。

代码 6.1 带两次块内同步的方阵分块核函数。

```
#include <cuda_runtime.h>
#define TILE_DIM 16

__global__ void matmul_tiled_square(
    const float* A, const float* B, float* C, int N)
{
    __shared__ float A_s[TILE_DIM][TILE_DIM];
    __shared__ float B_s[TILE_DIM][TILE_DIM];
    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int row = blockIdx.y * TILE_DIM + ty;
    const int col = blockIdx.x * TILE_DIM + tx;
    float sum = 0.0f;

    for (int q = 0; q < N / TILE_DIM; ++q) {
        A_s[ty][tx] = A[row * N + q * TILE_DIM + tx];
        B_s[ty][tx] = B[(q * TILE_DIM + ty) * N + col];
        __syncthreads();  // Wait for all tile producers.

        for (int k = 0; k < TILE_DIM; ++k) {
            sum += A_s[ty][k] * B_s[k][tx];
        }
        __syncthreads();  // Wait for all tile consumers.
    }
    C[row * N + col] = sum;
}
```

对应的主机端启动代码为：

```
dim3 block(TILE_DIM, TILE_DIM);
dim3 grid(N / TILE_DIM, N / TILE_DIM);
matmul_tiled_square<<<grid, block>>>(d_A, d_B, d_C, N);
```

其中 `d_A`、`d_B`、`d_C` 是已经分配的设备端矩阵指针。启动片段只说明线程配置，沿用已完成输入准备和设备内存分配的前提。

6.3.3.1 正确性的阶段不变量

以下不变量用于形式化说明代码 6.1 的执行过程。第 q 轮开始时，线程 (t_x, t_y) 的累加值为

$$\texttt{sum} = \sum_{j=0}^{qT-1} A_{r,j} B_{j,c}.$$

第一次同步之后，当前所需的共享行与共享列均已加载。内积循环增加

$$\sum_{k=0}^{T-1} A_{r,qT+k} B_{qT+k,c},$$

所以本轮结束时，累加范围恰好扩展为 $0 \le j < (q+1)T$。第二次同步保证当前输入不会被过早覆盖。全部 N/T 轮结束后，累加范围覆盖 $0 \le j < N$，因而 sum 等于 $C_{r,c}$。

6.4 边界条件与一般尺寸矩阵

6.4.1 从矩阵形状推导三类有效性条件

6.4.1.1 一般矩阵尺寸与行宽

采用一般矩阵形状

$$A \in \mathbb{R}^{M \times K}, \qquad B \in \mathbb{R}^{K \times N}, \qquad C \in \mathbb{R}^{M \times N}. \tag{6.8}$$

其中 M 是输出行数，N 是输出列数，K 是内积长度。方阵情形对应 $M = K = N$。线程映射仍为 $r = b_y T + t_y$、$c = b_x T + t_x$。

A 的行宽为 K，B 和 C 的行宽为 N。因此，一维数组偏移分别为

$$A_{r,j} \longleftrightarrow rK + j, \qquad B_{j,c} \longleftrightarrow jN + c, \qquad C_{r,c} \longleftrightarrow rN + c.$$

各矩阵的行宽决定地址偏移；加载条件和输出条件需要分别使用相应矩阵的维度。

6.4.1.2 加载有效性与输出有效性分别判断

同一个线程在一轮中有三个独立的任务：加载一个 A 元素、加载一个 B 元素，以及更新自己的输出部分和。前两项是否对应有效输入，需要根据各自的输入坐标判断。

操作	目标位置	有效性条件
加载 A	$A_{r,qT+t_x}$	$r < M$ 且 $qT + t_x < K$。
加载 B	$B_{qT+t_y,c}$	$qT + t_y < K$ 且 $c < N$。
写回 C	$C_{r,c}$	$r < M$ 且 $c < N$。

所有下标由非负线程坐标生成，因此这里仅列上界条件。

一个输出列越界的线程，仍可能负责加载同块其他线程需要的 A 元素；一个输出行越界的线程，仍可能负责加载有效的 B 元素。输出有效性只能决定是否写回当前输出，不能直接代替输入加载条件。

6.4.1.3 三种方向的分块数量

输出网格覆盖 $M \times N$，所以横纵方向的线程块数分别为

$$n_x = \left\lceil \frac{N}{T} \right\rceil, \qquad n_y = \left\lceil \frac{M}{T} \right\rceil. \tag{6.9}$$

每个输出都需要遍历长度为 K 的内积方向，因此每个线程块执行

$$Q = \left\lceil \frac{K}{T} \right\rceil \tag{6.10}$$

轮输入加载。网格尺寸由输出矩阵决定，循环轮数由内积长度决定。

对于正整数 D，整数向上取整可以写成

$$\left\lceil \frac{D}{T} \right\rceil = \left\lfloor \frac{D-1}{T} \right\rfloor + 1. \tag{6.11}$$

这对应 `(D - 1) / TILE_DIM + 1`。若 D 为 T 的倍数，减一使商减少一，再加一恢复；若存在余数，则在完整分块数上加一，覆盖最后的不完整分块。公式用于 $D > 0$ 的情形。

6.4.2 零填充维持统一的计算循环

6.4.2.1 缺失输入在加载时写零

零填充(*zero padding*)在加载阶段处理边界：目标输入有效时正常加载，越界时向该线程负责的共享数组位置写入 0。随后，所有线程都可以执行相同的 T 步内积循环。超出内积长度的项因乘以零而不产生贡献。

每一轮都必须写入该共享位置，无论写入的是输入还是零。共享数组会在多轮之间复用；若越界分支仅跳过赋值，该位置可能保留上一轮的数据，使不应参与计算的项继续产生贡献。

零填充的三个动作

加载 A：使用 A 自身的边界条件，越界写零。
加载 B：使用 B 自身的边界条件，越界写零。
写回 C：仅当输出坐标有效时写入全局内存。
两个同步点仍由整个线程块共同执行。

6.4.2.2 例一：输出有效，但最后一轮输入不完整

取 $M = K = N = 3$、$T = 2$。线程块 $(b_x, b_y) = (0, 0)$ 负责左上角四个有效输出，共执行 $Q = 2$ 轮。第二轮 $q = 1$ 对应全局内积下标 $j = 2, 3$，其中 $j = 3$ 超出有效输入范围。共享数组应为

$$A_s = \begin{bmatrix} A_{0,2} & 0 \\ A_{1,2} & 0 \end{bmatrix}, \qquad B_s = \begin{bmatrix} B_{2,0} & B_{2,1} \\ 0 & 0 \end{bmatrix}. \tag{6.12}$$

在 $k = 0$ 时，四个线程分别增加 $A_{r,2}B_{2,c}$；在 $k = 1$ 时，共享行的第二个元素与共享列的第二个元素均为零，贡献为零。以 $C_{0,1}$ 为例，全部两轮累加为

$$\underbrace{A_{0,0}B_{0,1} + A_{0,1}B_{1,1}}_{q=0} + \underbrace{A_{0,2}B_{2,1} + 0 \cdot 0}_{q=1}.$$

这个线程的输出位置始终有效，但它仍需要对最后一轮输入进行边界处理。

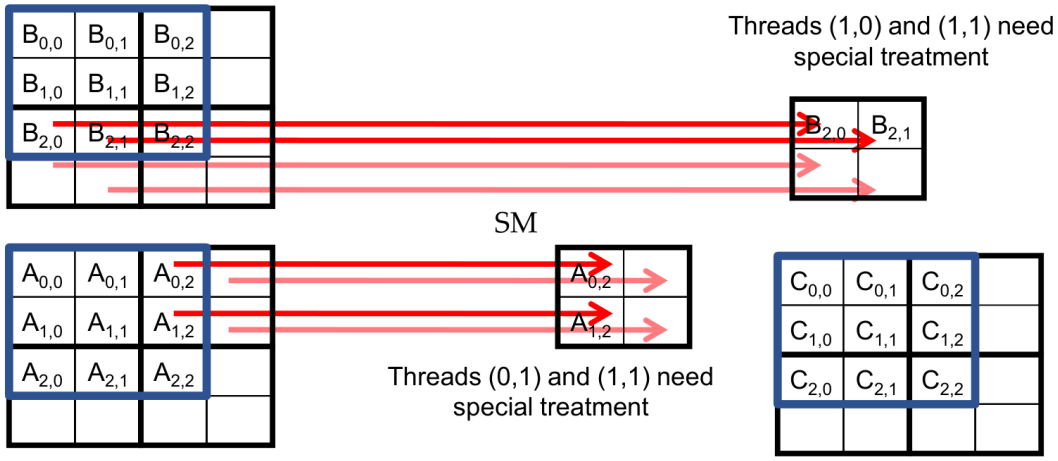


图 6.6 3×3 矩阵使用 2×2 分块时，第二轮输入块包含缺失位置。图中的线程编号采用（行，列）次序。

6.4.2.3 例二：输出越界的线程仍承担加载任务

仍取 3×3 矩阵与 2×2 分块。线程块 $(b_x, b_y) = (1,1)$ 覆盖输出坐标 $r, c \in \{2,3\}$，只有 $C_{2,2}$ 有效。在第一轮 $q=0$，四个线程的任务如下；线程坐标均按 (t_x, t_y) 书写。

线程	输出位置	加载 A	加载 B
$(0,0)$	$C_{2,2}$ 有效	$A_{2,0}$	$B_{0,2}$
$(1,0)$	$C_{2,3}$ 越界	$A_{2,1}$	越界，写零
$(0,1)$	$C_{3,2}$ 越界	越界，写零	$B_{1,2}$
$(1,1)$	$C_{3,3}$ 越界	越界，写零	越界，写零

于是共享数组为

$$A_s = \begin{bmatrix} A_{2,0} & A_{2,1} \\ 0 & 0 \end{bmatrix}, \qquad B_s = \begin{bmatrix} B_{0,2} & 0 \\ B_{1,2} & 0 \end{bmatrix}.$$

唯一负责有效输出的线程 $(0,0)$，需要线程 $(1,0)$ 加载的 $A_{2,1}$ 和线程 $(0,1)$ 加载的 $B_{1,2}$，才能计算

$$A_{2,0}B_{0,2} + A_{2,1}B_{1,2}.$$

第二轮再增加 $A_{2,2}B_{2,2}$，最终得到完整的 $C_{2,2}$。因此，应当保留输出越界线程的协作加载与同步，只在写回时排除其输出。

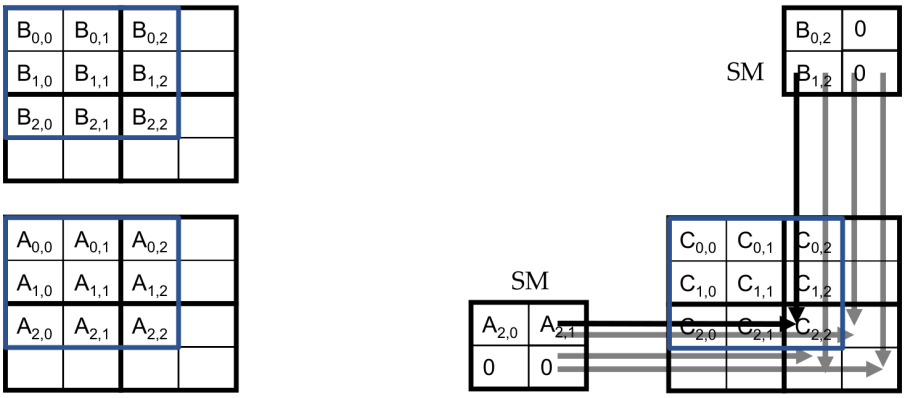


图 6.7 右下角输出块只含一个有效输出，但该输出仍需要其他线程协作加载输入。

6.4.3 支持一般尺寸的完整实现

6.4.3.1 核函数与启动配置

代码 6.2 把三类边界判断、零填充和向上取整组合起来。假设 M, K, N 为正整数，矩阵使用行主序，输入和输出数组已经按式 (6.8) 分配。

代码 6.2 一般尺寸的分块矩阵乘法；加载与同步由全部线程参与。

```cpp
#include <cuda_runtime.h>
#define TILE_DIM 16

__global__ void matmul_tiled(
    const float* A, const float* B, float* C,
    int M, int K, int N)
{
    __shared__ float A_s[TILE_DIM][TILE_DIM];
    __shared__ float B_s[TILE_DIM][TILE_DIM];
    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int row = blockIdx.y * TILE_DIM + ty;
    const int col = blockIdx.x * TILE_DIM + tx;
    const int numTiles = (K - 1) / TILE_DIM + 1;
    float sum = 0.0f;

    for (int q = 0; q < numTiles; ++q) {
        const int aCol = q * TILE_DIM + tx;
        const int bRow = q * TILE_DIM + ty;
        A_s[ty][tx] = (row < M && aCol < K)
            ? A[row * K + aCol] : 0.0f;
        B_s[ty][tx] = (bRow < K && col < N)
            ? B[bRow * N + col] : 0.0f;
        __syncthreads();

        for (int k = 0; k < TILE_DIM; ++k) {
            sum += A_s[ty][k] * B_s[k][tx];
        }
        __syncthreads();
    }
    if (row < M && col < N) {
        C[row * N + col] = sum;
    }
}
```

启动时，网格横向覆盖 N 列，纵向覆盖 M 行：

```cpp
dim3 block(TILE_DIM, TILE_DIM);
dim3 grid((N - 1) / TILE_DIM + 1,
          (M - 1) / TILE_DIM + 1);
matmul_tiled<<<grid, block>>>(d_A, d_B, d_C, M, K, N);
```

参数顺序为 M, K, N，与 A 的行数、内积长度、B 的列数一致。

6.4.3.2 为什么零填充保持结果正确

对输入在原矩阵范围外定义零扩展 $\widetilde{A}$、$\widetilde{B}$。当 $0 \le r < M$、$0 \le c < N$ 时，代码实际计算

$$\sum_{q=0}^{Q-1} \sum_{k=0}^{T-1} \widetilde{A}_{r,qT+k} \widetilde{B}_{qT+k,c} = \sum_{j=0}^{QT-1} \widetilde{A}_{r,j} \widetilde{B}_{j,c} \tag{6.13}$$

$$= \sum_{j=0}^{K-1} A_{r,j} B_{j,c} = C_{r,c}. \tag{6.14}$$

由于 $QT \ge K$，有效内积项全部被覆盖；多出来的项恰好为零。此推导解释了为何内积循环仍可固定执行 T 步。零扩展只是在共享加载阶段实现的逻辑含义，无需为完整矩阵额外分配补零后的全局数组。

6.4.3.3 可选的计算判断与同步位置

对于无有效输出的线程，零填充使其相应共享行或共享列为零，部分和不影响任何有效输出。可以继续执行内积循环，也可以仅让有效输出线程执行乘加，以减少这些线程的无用运算。该调整的实际性能收益取决于分支和省去运算的开销。
若增加计算条件，正确的局部结构如下：

```
__syncthreads();
if (row < M && col < N) {
    for (int k = 0; k < TILE_DIM; ++k) {
        sum += A_s[ty][k] * B_s[k][tx];
    }
}
__syncthreads();
```

两个同步点都在按线程变化的条件之外。将输出越界线程提前移出分块循环，会丢失它们仍可能承担的输入加载任务；将同步放入仅部分线程执行的分支，也不满足块内同步要求。

6.4.3.4 边界分支的工作量范围

边界条件可能造成**分支分歧**(*branch divergence*)。输出的末行、末列分块会包含输出越界线程；当 K 不能被 T 整除时，每个输出块的最后一轮也会处理内积方向的缺失输入。这一分类由三类边界谓词直接得到。

在矩阵各维度相对 T 都很大时，边缘区域和最后一轮输入只占较小比例；小矩阵或高度

不规则的尺寸则可能有明显边界开销。该判断延续大矩阵边界影响较小的定性分析，具体占比仍由 M, N, K, T 决定。

6.5 分块收益与资源约束

6.5.1 用数据流量解释计算收益

6.5.1.1 一个分块轮次的加载量与计算量

以下推导展开分块减少全局访问的数量关系。采用 $N \times N$ 方阵，且 T 整除 N；按代码中的全局输入加载计数，暂时忽略缓存效果、输出流量、同步和其他执行开销。

一个线程块的一轮计算，需要加载两个 $T \times T$ 输入块，共 $2T^2$ 个 `float`，即 $8T^2$ 字节。该轮计算 T^2 个输出部分和，每个包含 T 次乘加；按每次乘加两个浮点运算计，计算量为 $2T^3$ FLOP。于是，**计算强度** (*arithmetic intensity*) 为

$$I_T = \frac{2T^3 \text{ FLOP}}{8T^2 \text{ B}} = \frac{T}{4} \text{ FLOP/B}. \tag{6.15}$$

等价地，每个浮点运算需要 $4/T$ 字节的全局输入数据。T 增大时，同一输入被更多输出复用。

对比不使用共享复用的情况：T^2 个线程各自为 T 个内积步加载两次输入，共有 $2T^3$ 次输入元素加载；协作加载只需 $2T^2$ 次。因此，当前线程块这一轮的输入加载量减少为原来的 $1/T$。

6.5.1.2 整个矩阵乘法的流量

不使用共享复用时，N^2 个输出各需 $2N$ 次输入加载，总数为 $2N^3$。分块后，有 $(N/T)^2$ 个输出块，每块执行 N/T 轮，每轮加载 $2T^2$ 个输入元素，得到

$$L_T = \left(\frac{N}{T}\right)^2 \frac{N}{T} 2T^2 = \frac{2N^3}{T}. \tag{6.16}$$

当前方法的复用发生在线程块内部；不同输出块仍分别加载它们需要的输入块。上述计数保留了这一点。

将输出写回加入流量模型

每个有效输出最终写回一次，故输出流量为 $4N^2$ 字节。把输入与输出合并，可得到更完整的计算强度近似：

$$I_T^{\text{in+out}} = \frac{2N^3}{8N^3/T + 4N^2} = \frac{NT}{4N + 2T}\ \text{FLOP/B}.$$

当 N 相对 T 很大时，该式趋近 $T/4$。这一补充用于说明忽略输出写回的近似条件，其余缓存和执行开销仍未包含。

6.5.2 带宽瓶颈如何转向计算瓶颈

令 P_{peak} 表示计算峰值，B_{mem} 表示全局内存带宽。用计算峰值和带宽支持能力作为两项上界，可写成

$$P_T \le \min\bigl(P_{\text{peak}},\, B_{\text{mem}} I_T\bigr). \tag{6.17}$$

这是一种用于解释瓶颈转移的简化模型。代入示例的 $P_{\text{peak}} = 1000\,\text{GFLOP/s}$、$B_{\text{mem}} = 150\,\text{GB/s}$，有

$$B_{\text{mem}} I_T = 37.5T\ \text{GFLOP/s}.$$

T	输入字节/FLOP	带宽支持能力	同时考虑计算峰值
1	4	37.5 GFLOP/s	37.5 GFLOP/s
16	0.25	600 GFLOP/s	600 GFLOP/s
32	0.125	1200 GFLOP/s	1000 GFLOP/s

16×16 分块把带宽支持能力提高到 600 GFLOP/s。32×32 分块的输入供给可以支持 1200 GFLOP/s，已经超过示例 GPU 的计算峰值，所以该简化模型的限制转为计算能力。

这些数值描述带宽可以支撑的运算量及理论上界。实际执行还受到输出流量、缓存效果、同步、占用率和寄存器压力影响；这些因素正是从流量模型走向实际性能时需要补充的项。

6.5.3 分块尺寸与驻留资源

6.5.3.1 每个线程块需要多少资源

每线程一个输出、两个共享输入块的基本实现中，线程数和共享内存用量为

$$n_{\text{threads}} = T^2, \qquad S_{\text{block}} = 2T^2 \,\text{sizeof}(\texttt{float}) = 8T^2 \text{ B}. \tag{6.18}$$

较大的分块带来更多复用，也同时增大线程块和共享内存占用。资源分配会限制一个**流多处理器**(*streaming multiprocessor, SM*)上可同时驻留的线程块数量，进而影响**占用率**(*occupancy*)。

占用率描述驻留 warp 相对于硬件可容纳 warp 数的比例。对这里的 16×16、32×32 规则线程块，可通过驻留线程数与线程上限比较其资源占用水平。资源容量给出驻留数量的上界，实际驻留状态还取决于核函数的资源需求。

6.5.3.2 示例：共享容量限制与线程容量限制

采用第一代 Maxwell 的示例条件：每个 SM 有 64 kB 共享内存，每线程块最多使用 48 kB，每个 SM 最多驻留 2048 个线程。以下存储容量换算使用 $1\,\text{kB} = 1024\,\text{B}$。

资源或计数	$T = 16$	$T = 32$
每块线程数 T^2	256	1024
每块共享内存 $8T^2$	$2048\,\text{B} = 2\,\text{kB}$	$8192\,\text{B} = 8\,\text{kB}$
仅按共享容量限制的块数	$64/2 = 32$	$64/8 = 8$
仅按线程容量限制的块数	$2048/256 = 8$	$2048/1024 = 2$
上述两项合并的块数上限	8	2
相应驻留线程数上限	$8\times256 = 2048$	$2\times1024 = 2048$
每线程两次加载的计数	$8\times256\times2 = 4096$	$2\times1024\times2 = 4096$

两个分块尺寸在这一估算中都暴露出 4096 个待加载操作。32×32 分块虽然允许驻留的线程块更少，但每块线程更多，合计线程数相同；其输入复用倍数则更高。这里的加载数是按驻留线程和每线程两次加载得到的并行性估算。

6.5.3.3 寄存器与其他上限共同决定驻留数量

共享内存和线程数是两项直接可见的限制，寄存器数量及每 SM 的线程块数量上限也会约束驻留。令 S_{SM} 为每 SM 共享内存容量，R_{SM} 为寄存器数量，R_{thread} 为每线程寄存器用量，$n_{\max}$ 与 $B_{\max}$ 分别为每 SM 的线程数和块数上限。可用下面的理想化上界概括最紧资源约束：

$$B_{\mathrm{active}} \le \min\left\{B_{\max}, \left\lfloor \frac{n_{\max}}{T^2} \right\rfloor, \left\lfloor \frac{S_{\mathrm{SM}}}{8T^2} \right\rfloor, \left\lfloor \frac{R_{\mathrm{SM}}}{R_{\mathrm{thread}} T^2} \right\rfloor\right\}. \tag{6.19}$$

该式将资源容量除以每块需求，表达“由最先耗尽的资源限制驻留”的原则；实际资源分配粒度未包含在内。每块资源需求还必须满足设备对单个线程块的限制。

表 6.1 GPU 资源规格示例；M40 属于第二代 Maxwell。

GPU 型号	K40	M40	P100	V100	A100	H100
微架构	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
计算能力	3.5	5.2	6.0	7.0	8.0	9.0
SM 数量	15	24	56	80	108	132
每 SM 核心数	192	128	64	64	64	128
每 SM 最大线程数	2048	2048	2048	2048	2048	2048
每 SM 最大块数	16	32	32	32	32	32
每块最大线程数	1024	1024	1024	1024	1024	1024
每 SM 寄存器数	64K	64K	64K	64K	64K	64K
每 SM 共享内存	48KB	96KB	64KB	96KB	164KB	228KB

6.5.3.4 如何理解分块尺寸的取舍

分块边长 T 同时影响两组数量：复用次数随 T 增长，而每块线程数与共享内存需求随 T^2 增长。选择尺寸时，需要把输入复用、可驻留线程块数量、寄存器消耗和同步代价放在同一个执行模型中考虑。

基本实现中，16×16 对应 256 个线程，32×32 对应 1024 个线程；后者已经达到表 6.1 中列出的每块最大线程数。提高复用的空间因此也受到线程映射方式约束。具体核函数在实际设备上的较优尺寸仍需由测量确定。

6.6 共享内存接口与分块思想的推广

6.6.1 动态共享内存

6.6.1.1 将容量配置推迟到核函数启动时

静态共享数组的大小写在声明中，例如 `A_s[TILE_DIM][TILE_DIM]`。当所需容量需要在启动时确定，可以使用**动态共享内存**（*dynamic shared memory*）：

```
extern __shared__ float A_s[];
```

核函数启动配置的第三个参数 `smemPerBlock` 指定每个线程块申请的动态共享内存字节数。配置形式为 `<<<nBlocks, nThreadsPerBlock, smemPerBlock>>>`。该容量按线程块分别提供，单位是字节。

6.6.1.2 将一段共享区域划分为两个输入块

以下局部片段用于把动态共享内存接口连接到两个输入块的存储需求。设运行时的分块边长为 T，可以在核函数内部声明一段共享区域，再把它分成前后两段：

```
extern __shared__ float shared[];
float* A_s = shared;
float* B_s = shared + T * T;
```

两个输入块各占 T^2 个 `float`。采用一维共享指针后，局部位置 (t_y, t_x) 使用偏移 $t_y T + t_x$；计算时的两个偏移分别为 $t_y T + k$ 和 $kT + t_x$。对应的主机端容量计算为：

```
size_t smemPerBlock = 2 * T * T * sizeof(float);
```

分区方式只改变共享数组的表示和容量配置；协作加载、两次同步、零填充和输出写回的职责保持相同。该局部片段为存储布局说明，完整矩阵乘法实现采用代码 6.2 的静态共享数组版本。

6.6.2 Hopper 的相关功能

6.6.2.1 分布式共享内存

分布式共享内存(*distributed shared memory*) 允许同一**线程块集群**(*thread block cluster*) 内的线程块访问彼此的共享内存。它从 Hopper 架构开始提供，将访问范围扩展到集群中的线程块。前面的基本分块核函数使用各线程块自己的共享数组。

6.6.2.2 张量内存加速器

张量内存加速器(*Tensor Memory Accelerator, TMA*) 支持全局内存与共享内存之间的数据异步传输。它可用于分块算法的输入搬运阶段。

6.6.3 CPU 缓存分块

6.6.3.1 相同的局部性，不同的存储管理方式

CPU 上也可以通过分块重排计算顺序。**缓存分块**(*cache blocking*) 使一小组数据在较短的执行区间内被反复使用，从而建立**时间局部性**(*temporal locality*)，提高数据仍在缓存中时被再次访问的机会。

GPU 基本实现由程序把输入块显式放入共享内存，并通过线程块同步协调使用；CPU 示例依靠缓存复用，通过循环顺序集中对当前分块的访问。二者都将计算组织为“小块数据，多次使用”。

6.6.3.2 三层分块循环与三层元素循环

CPU 示例对输出行、输出列和内积方向分别划分分块。外层三重循环选择输出块与当前输入块，内层三重循环计算该输入块对输出块的贡献。代码 6.3 是该循环结构的函数实现；假设 $N > 0$、$T > 0$，且 T 整除 N。

代码 6.3 CPU 缓存分块矩阵乘法，保持每个内积块的部分和累加顺序。

```
void matmul_cpu_tiled(
    const float* A, const float* B, float* C, int N, int T)
{
    for (int rt = 0; rt < N / T; ++rt) {
        for (int ct = 0; ct < N / T; ++ct) {
            for (int qt = 0; qt < N / T; ++qt) {
                for (int row = rt * T;
                     row < (rt + 1) * T; ++row) {
                    for (int col = ct * T;
                         col < (ct + 1) * T; ++col) {
                        float sum = 0.0f;
                        for (int j = qt * T;
                             j < (qt + 1) * T; ++j) {
                            sum += A[row * N + j]
                                 * B[j * N + col];
                        }
                        if (qt == 0) {
                            C[row * N + col] = sum;
                        } else {
                            C[row * N + col] += sum;
                        }
                    }
                }
            }
        }
    }
}
```

6.6.3.3 初始化与部分和的归属

对于固定的 $C_{r,c}$，每个内积块产生一个新的局部变量 `sum`。第一块通过赋值初始化 $C_{r,c}$，后续块通过 `+=` 累加。因此，初始的 C 数组内容无需参与计算。

GPU 基本核函数把一个输出的 `sum` 保持在线程寄存器中，跨越全部输入分块，最后统一写回；这里的 CPU 循环先计算当前内积块对整个输出块的贡献，跨内积块的部分和保存在 C 中。两种写法都实现式 (6.5)，但部分和保留的位置与循环组织不同。

6.7 关键结论与公式汇总

6.7.1 机制之间的联系

6.7.1.1 复用决定分块的收益

矩阵乘法的同一个输入会服务于多个输出。一个 $T \times T$ 线程块先协作加载 $2T^2$ 个输入，再计算 $2T^3$ 个浮点运算，将块内输入复用转化为约 T 倍的全局加载量缩减。

6.7.1.2 同步决定共享缓冲区的正确使用

共享数组在空间上由多个线程共同使用，在时间上由多个分块轮次复用。第一次屏障保护当前数据的完整加载，第二次屏障保护当前数据的完整使用。两次屏障分别对应这两类依赖。

6.7.1.3 边界处理决定协作任务能否保留

每个线程的 A 加载、B 加载和 C 写回具有不同的有效性条件。加载阶段写零，使内积循环保持统一；输出阶段做有效性判断，使越界线程保留协作职责而不写入非法输出位置。

6.7.1.4 资源约束决定复用能否转化为性能

分块增大使计算强度线性增长，同时使每块线程数和共享内存需求按平方增长。带宽、计算峰值、驻留线程块数量以及寄存器需求，共同影响最终的执行速度。

6.7.2 关键公式

以下使用 $A \in \mathbb{R}^{M \times K}$、$B \in \mathbb{R}^{K \times N}$、$C \in \mathbb{R}^{M \times N}$，分块边长为 T。

对象	表达式
线程对应输出	$r = b_y T + t_y,\quad c = b_x T + t_x$。
输出网格尺寸	$n_x = \lceil N/T \rceil,\quad n_y = \lceil M/T \rceil$。
内积方向轮数	$Q = \lceil K/T \rceil$。
A 加载地址与条件	$rK + qT + t_x$；$r < M$ 且 $qT + t_x < K$。
B 加载地址与条件	$(qT + t_y)N + c$；$qT + t_y < K$ 且 $c < N$。
共享内积	$\sum_{k=0}^{T-1} A_s[t_y][k] B_s[k][t_x]$。
输出地址与条件	$rN + c$；$r < M$ 且 $c < N$。
每块线程与共享容量	T^2 个线程，$8T^2$ 字节共享内存。
理想输入计算强度	$I_T = T/4$ FLOP/B。
简化性能上界	$P_T \le \min(P_{\text{peak}}, B_{\text{mem}} I_T)$。

计算强度的单精度假设、浮点计数与流量模型见第 6.5 节。

第 7 讲

DRAM Bandwidth and other Performance Considerations

7.1 课程安排与核心内容

7.1.1 课程安排与阅读材料

以下课程事项对应 2026 年 9 月 15 日。

Lab 2 提交要求 Lab 2 在授课当周的星期五截止。需要完成全部要求的代码，包括主机内存的分配与释放；将工作推送到 GitHub；代码完成之后再提交 Lab 2 quiz。

课程辅助资源 课程聊天机器人可用于询问设备内存分配、主机到设备的数据复制、GPU **内核**（*kernel*）的调用、共享内存数组声明，以及向量加法代码的逐行解释。复习时也可要求它围绕 Lecture 2 和 Lecture 3 逐题提问、等待回答并检查正确性。阅读任务是教材第 6 章和 CUDA Best Practices Guide 的 *Memory Optimizations*；待办事项是提交 Lab 2。

7.1.2 学习目标与知识范围

学习目标 理解动态随机存取存储器的组织方式；理解突发模式和多个存储体如何提高数据传输率；理解连接 GPU kernel 性能与 DRAM 组织的合并访存；掌握多种 kernel 优化手段及其相互权衡。

优化机制之间的关系 每块线程数、每块共享内存量和每线程寄存器数共同影响**占用率**（*occupancy*）；减少控制分歧可提高 SIMD 效率，共享内存分块和寄存器分块则减少内存流量。访存与执行优化包括合并访存、面向内存延迟的占用率调节、线程粗化、循环展开和双缓冲。私有化与 warp 级原语分别用于减少访问冲突和线程间协调开销。

全局内存的标称带宽与程序实际获得的有效数据率之间可能存在差距。并发访问若争用同一资源，会产生串行化与延迟；为计算核心提供足够多的线程，还需要让这些线程以合适的方式请求数据。

三种互补的改进方向

数据复用减少需要搬运的数据量；合并访存提高必要传输中有效数据的比例；并发请求使内存系统在等待某次访问时仍能处理其他工作。线程粗化、指令调度与双缓冲进一步调整每线程工作量和等待关系。

7.2 DRAM 组织与带宽

7.2.1 从存储单元到二维阵列

7.2.1.1 电容保存比特，晶体管控制连接

随机存取存储器（*Random Access Memory, RAM*）允许直接访问任意地址，无须先顺序经过前面的存储位置。**动态随机存取存储器**（*Dynamic RAM, DRAM*）用电容上的电荷保存一个比特，并通过晶体管把该电容接到**位线**（*bit line*）上进行读写。**选择信号**（*SELECT*）决定这个连接是否导通。

电容会漏电，保存的信息因此需要周期性**刷新**（*refresh*）；典型时间尺度为数十毫秒。每个比特只需要一个晶体管和一个电容，因此可以在有限面积内布置大量存储单元。

7.2.1.2 高密度与慢访问来自同一组织方式

在示意组织中，一根位线连接约 1,000 个单元，地址译码选择其中一个单元与位线接通。存储单元的电容很小，长位线的电容却很大；微小电荷改变位线状态需要时间，因此还要使用**感应放大器**（*sense amplifier*）来加速信号检测。

一个 DRAM **存储体**（*bank*）由二维单元阵列组成：先激活一行，再按列选择数据。一条行选择线连接约 1,000 根位线，这与每根位线上的约 1,000 个单元一起形成约 10^6 比特量级的示意阵列。这些数值用于说明组织规模。

各部件的分工 **行译码器**（*row decoder*）根据行地址选择一行；感应放大器检测单元数据；**列锁存器**（*column latches*）保存已取得的行数据；**多路选择器**（*multiplexer, Mux*）再根据列地址选择输出。访问阵列和从已取得的数据中选择输出，构成快慢不同的两个阶段。

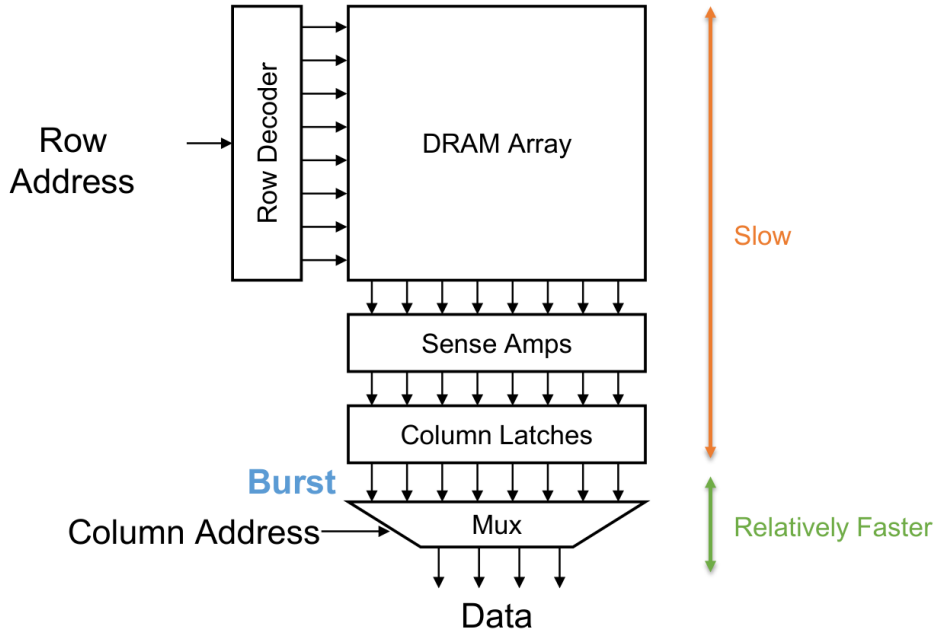


图 7.1 DRAM bank 的行译码、阵列、感应放大器、列锁存器与多路选择器。上部阵列访问较慢，已取得数据的列选择与输出相对较快。

7.2.2 选行、选列与突发传输

7.2.2.1 一次访问的三个阶段

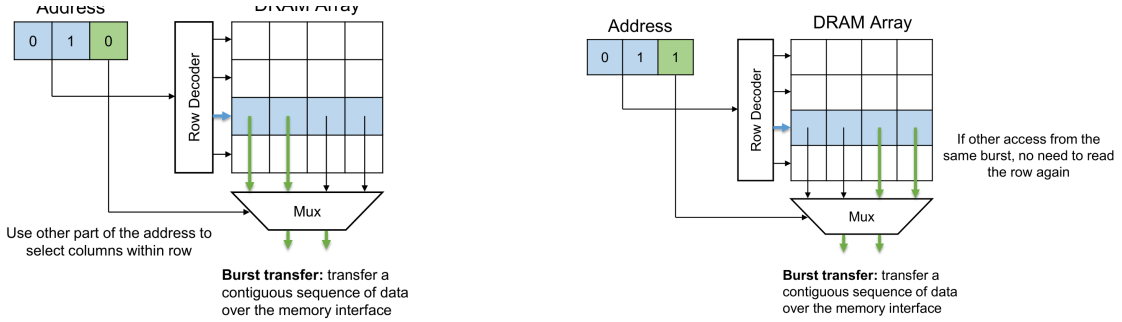
行激活 (*row activation*) 把选定行的数据打开到**行缓冲区** (*row buffer*)；**列访问** (*column access*) 从已打开的行中选择所需数据；**突发传输** (*burst transfer*) 通过内存接口传送一串连续数据。

行地址与列地址承担不同的选择任务。行访问先承担较慢的阵列操作，后续数据可以从已经取得的内容中选出。这样，一次较慢的准备工作可支持多个相邻数据的输出。

7.2.2.2 8×2 比特小存储体示例

“8×2 bit”表示 8 个可寻址位置、每个位置 2 bit，总计 16 bit。按图中的地址划分，地址的前两位控制行选择，最后一位控制行内数据选择。示意阵列画成 4 行、每行 4 bit。

地址从 `010` 变为 `011` 时，仅最后一位改变。两次访问复用已取得的行数据，只改变后续的选择与输出，从而共用较慢的前期工作。



(a) 地址 `010`：选行后选择一组数据。 **(b)** 地址 `011`：保持行选择，改变行内选择。

图 7.2 同一批已取得的数据支持相邻地址的输出。地址前两位以蓝色标记，最后一位以绿色标记。

7.2.2.3 把慢阶段的成本分摊到多个数据

小存储体的时序模型用橙色表示较慢的阵列访问，用绿色表示较快的数据输出。访问同一 burst 中的数据，可以共用前面的橙色阶段；图中不同 burst 的示意则重复了这个阶段。

示意时序的形式化 设 L 为一次慢阶段的时间，t 为一次快输出的时间，q 为所需输出的数据个数。在“每个数据单独重复慢阶段”和“q 个数据共用慢阶段”这两个示意情形下，分别有

$$T_{\text{separate}} = q(L + t), \tag{7.1}$$

$$T_{\text{burst}} = L + qt, \tag{7.2}$$

$$T_{\text{separate}} - T_{\text{burst}} = (q - 1)L. \tag{7.3}$$

L 和 t 分别表示模型中的慢阶段与输出阶段耗时；时间式通过比较两阶段的重复次数解释 burst 的收益。

突发传输的带宽含义

连续数据能够分摊一次较慢的准备成本。若一次传输带来的数据大部分都会被使用，接口传输的数据就能更多地转化为程序需要的数据；若只使用其中少量位置，其余传输占用的带宽就没有直接贡献。

7.2.3 多个存储体与内存级并行

7.2.3.1 通过重叠操作填补传输间隙

单个 bank 即使使用 burst，两次突发传输之间仍可能存在等待时间。多个 bank 分别处理独立请求时，控制器可以把它们的内部操作重叠起来，使一个 bank 准备数据的时间与另一个 bank 输出数据的时间交错，从而保持数据接口忙碌。

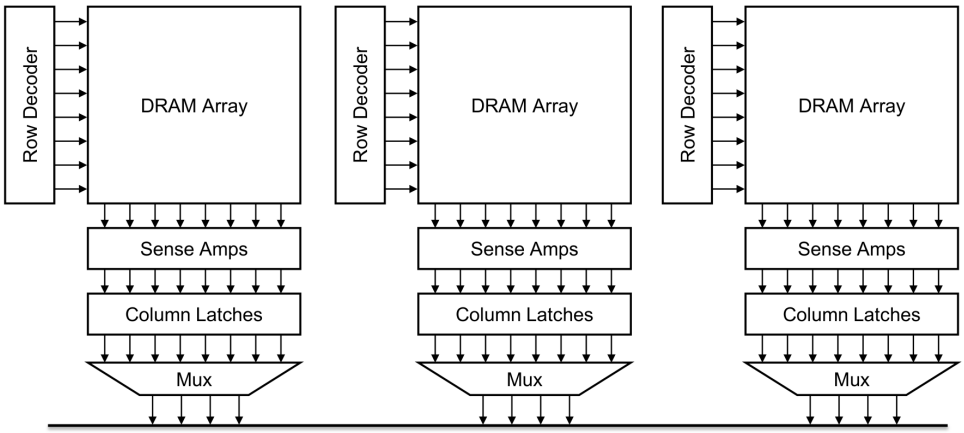


图 7.3 多个 DRAM bank 具有各自的阵列、译码与数据选择部件，并连接到下方的数据接口。不同 bank 上的独立请求提供可重叠的内部操作。

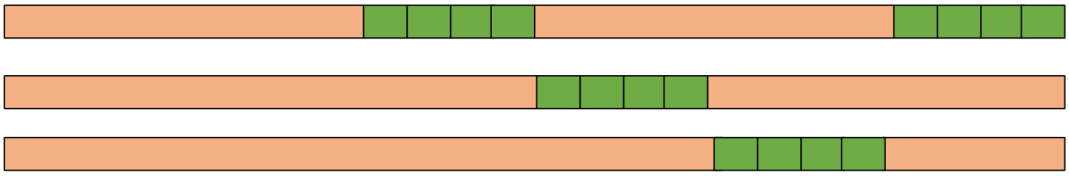


图 7.4 三个 bank 的慢阶段与突发输出交错排列。绿色输出段依次出现，使整体数据传输能比单个 bank 更连续。

7.2.3.2 硬件并行需要足够多的独立请求

内存级并行(*memory-level parallelism, MLP*) 是多个独立内存操作同时处于进行中的状态。多个 bank 提供并行处理的条件，程序则需要持续发出足够多的请求，让这些 bank 有工作可做。

较高的**流多处理器**(*Streaming Multiprocessor, SM*) 占用率可以提供更多线程，从而增加可并发的内存访问。这与使用多个**线程束**(*warp*) 隐藏计算核心中的流水线延迟具有相同的基本思路：一部分工作正在等待时，仍有其他工作可以推进。

与单次访存效率的关系 合并访存改善一次 warp 访问的数据分布，MLP 改善不同请求在时间上的重叠程度。共享内存分块则通过复用减少需要发出的请求。三者分别作用于传输量、传输效率和请求并发程度。

占用率的权衡 为了提高占用率，需要同时考虑每块线程数、每块共享内存和每线程寄存器。更多驻留线程有助于隐藏延迟，也会共享并竞争这些有限资源；优化目标是提供足够的独立工作，同时保留每个线程或线程块需要的复用空间。

7.3 Warp 访存与共享内存分块

7.3.1 合并访存的判断单位

7.3.1.1 固定一条访存指令，比较同一 warp 的地址

合并访存（*memory coalescing*）是把同一**线程束**（*warp*）内多个线程对相邻位置的访问组合起来服务。连续访问能够以较少的**内存事务**（*memory transaction*）传送所需数据；分散到不同位置的请求往往需要更多事务，也会增加 warp 等待数据的时间。这类分散访问也称为**内存分歧**（*memory divergence*）。

判断时需要固定同一次访存，再观察 warp 内各线程的地址。例如，在矩阵乘法内积循环中固定 k，列出线程 $T_0, T_1, \ldots$ 此时分别访问哪里。一个线程随 k 变化的访问顺序，与多个线程在固定 k 时的地址分布，是两个不同的观察方向。

访存分析的三个要素

线程块形状决定一个 warp 中哪些线程坐标相邻；地址表达式把线程坐标映射成元素下标；元素类型再把下标差换算成字节距离。确定这三项之后，才能判断同一 warp 的地址是连续、相同，还是带有步长。

7.3.1.2 二维线程块如何组成 warp

对二维线程块，线程首先沿 x 维连续排列，再进入下一条 y 行。用 $B_x = \texttt{blockDim.x}$、$B_y = \texttt{blockDim.y}$ 表示块形状，用 t_x, t_y 表示块内坐标，可将这个排列规则写为

$$t = t_y B_x + t_x, \qquad t_x = t \bmod B_x, \quad t_y = \left\lfloor \frac{t}{B_x} \right\rfloor. \tag{7.4}$$

式中 t 为块内线性线程编号。每 32 个相邻线性线程构成一个 warp；用 $\ell = 0, \ldots, 31$ 表示 warp 内的**线程槽位**(*lane*)，某个完整 warp 的线程编号可写成 $t = 32w + \ell$，其中 w 为该 warp 在块内的编号。这是对给定排列规则的索引展开。

因此，`dim3(32, 8)` 中的 warp 可以沿完整的一条 x 行排列；`dim3(1, 32)` 的 warp 沿 y 变化；`dim3(16, 16)` 的一个 warp 则由两条各含 16 个线程的 x 行组成。

7.3.1.3 四种典型地址分布

下面的地址均为字节地址。连续访问中的每个线程读取一个 4 字节元素。

表 7.1 同一次访存中，不同 lane 的地址关系。

模式	lane 0、1、2、3 等的地址	主要特征
连续合并 *coalesced*	$0, 4, 8, 12, \ldots, 124$	相邻线程请求相邻元素，形成连续的 128 字节需求。
广播 *broadcast*	$40, 40, 40, 40, \ldots$	多个线程请求同一数据，表现为同址复用。
步长访问 *strided*	$0, 16, 32, 48, \ldots$	每次只取相隔 16 字节的位置，地址间留有空隙。
分散访问 *scattered*	$84, 0, 132, 16, \ldots$	地址分散，需逐项检查其覆盖的传输区域。

广播可以满足多个线程对同一数据的需求；连续合并则让多个不同元素一起被有效传输。广播与连续合并对应不同的数据关系，分析复用收益时应分别处理。

在简化模型中，一组连续访问可由一次 burst 或事务服务。更一般地，合并访存减少满足同一 warp 请求所需的事务数量；具体数量还取决于硬件事务粒度与地址对齐。

7.3.2 向量访问与行主序地址

7.3.2.1 向量加法：每条指令内的地址都连续

对长度为 n 的单精度向量，线程的全局下标为

$$i = \texttt{blockIdx.x}\,\texttt{blockDim.x} + \texttt{threadIdx.x}. \tag{7.5}$$

核心运算为 $z[i] = x[i] + y[i]$，边界判断保证只有有效下标参与计算。

代码 7.1 向量加法：相邻线程处理相邻元素。

```
__global__ void vectorAdd(const float* x, const float* y,
                          float* z, unsigned int n) {
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        z[i] = x[i] + y[i];
    }
}
```

设一个完整 warp 的第一个全局下标为 i_0，则对数组 x 的读取地址为

$$a_{x,\ell} = a_x + 4(i_0 + \ell), \qquad \ell = 0, \dots, 31, \tag{7.6}$$

其中 a_x 是数组起始字节地址。相邻 lane 的地址差是 4 B，整个 warp 需要 32 个 float，即 128 B 的连续数据。对 y 的读取与对 z 的写入也分别具有同样的关系。三组数组访问分别分析。

7.3.2.2 二维 C 数组按行线性存储

行主序（*row-major order*）把同一行的元素连续放入内存，再接下一行。设矩阵列数为 $W = \texttt{Width}$，行列下标分别为 r, c，则元素下标与字节地址为

$$\mathrm{index}(r, c) = rW + c, \qquad a(r, c) = a_0 + s(rW + c), \tag{7.7}$$

其中 a_0 为矩阵起始地址，s 为每个元素的字节数。固定 r 而令 c 增加 1，地址增加 s；固定 c 而令 r 增加 1，地址增加 sW。这是对行主序图的形式化。

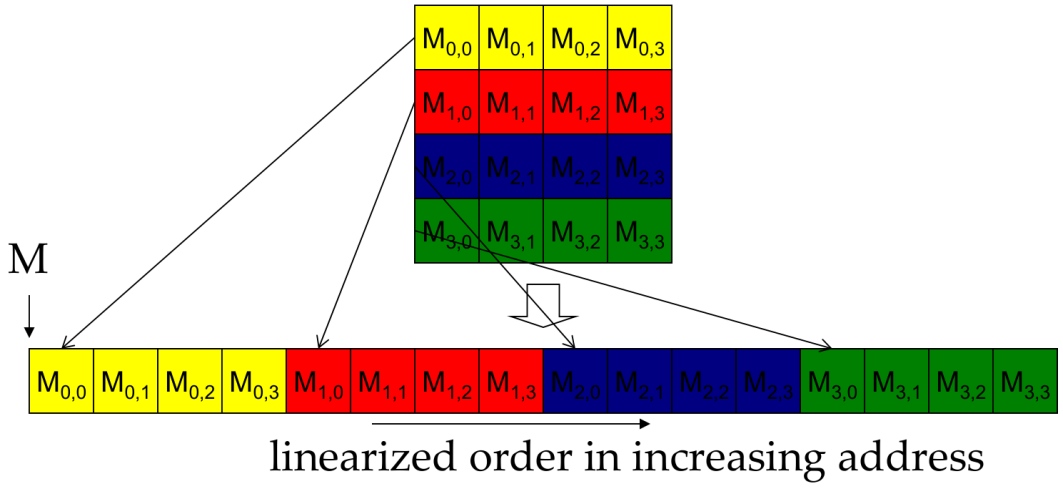


图 7.5 四行矩阵依次线性存储。每一行以同一颜色表示，内存地址沿下方箭头增大。

矩阵乘法中的 `M[row*Width+k]` 沿单个线程的 k 循环读取一行，`N[k*Width+col]` 沿单个线程的 k 循环读取一列。是否合并还取决于同一 warp 内的 `row` 和 `col` 如何变化。

7.3.3 矩阵乘法：连续访问、广播与跨行步长

7.3.3.1 固定输出元素与固定内积迭代

设 M, N, P 均为 $W \times W$ 矩阵，$P = MN$，则

$$P_{r,c} = \sum_{k=0}^{W-1} M_{r,k} N_{k,c}. \tag{7.8}$$

每个线程负责一个输出元素，`threadIdx.x` 影响输出列，`threadIdx.y` 影响输出行。下列片段在 kernel 内使用，M, N 为输入指针，`Width` 为 W；假设该线程的行列坐标有效。

代码 7.2 矩阵乘法的内积访问，Width 表示矩阵维度。

```
int row = blockDim.y * blockIdx.y + threadIdx.y;
int col = blockDim.x * blockIdx.x + threadIdx.x;
float sum = 0.0f;
for (int k = 0; k < Width; ++k) {
    sum += M[row * Width + k] * N[k * Width + col];
}
```

7.3.3.2 沿同一输出行排列的线程

考虑 `blockDim.x=32`，一个 warp 位于同一输出行 r，列下标为 $c_0 + \ell$。固定 k 后，两个输入的元素下标分别为

$$I_M(\ell) = rW + k, \tag{7.9}$$

$$I_N(\ell) = kW + c_0 + \ell. \tag{7.10}$$

I_M 与 ℓ 无关，32 个线程读取同一个 $M_{r,k}$，对应广播或复用；I_N 随 ℓ 每次增加 1，线程读取相邻的 N 元素，对应经典连续合并访问。

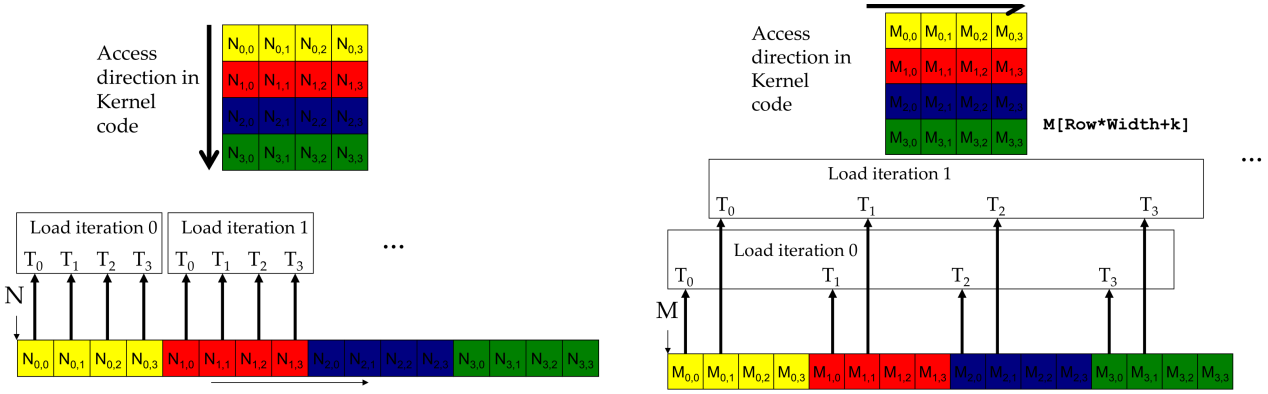
沿着单个线程观察，N 的读取会随着 k 增加而跳到下一行；沿着同一次 k 的 warp 观察，线程却一起读取 N 的同一行。这解释了“单个线程纵向访问，但 warp 仍可合并”的现象。

7.3.3.3 跨输出行排列的线程

若所比较的线程具有 $r = r_0 + \ell$，则固定 k 时

$$I_M(\ell) = (r_0 + \ell)W + k, \qquad I_M(\ell+1) - I_M(\ell) = W. \tag{7.11}$$

相邻线程访问不同行的同一列，相差 W 个元素。单精度数据的字节距离为 $4W$，这就是跨行步长访问。



(a) N：固定 k 后，线程访问相邻列。 **(b)** M：固定 k 后，所画线程跨不同的行。

图 7.6 两个访问方向示意。上方箭头描述单线程沿 k 的移动，下方连线描述固定 k 时线程之间的地址关系。

线程映射决定访问模式 跨输出行的线程读取 M 时形成跨行步长；同一输出行中的线程则对 M 广播。判断访存模式时，需要先明确所比较线程的坐标关系。

块宽为 16 时的索引展开 一个 warp 包含两条输出行、每行 16 个线程。固定 k 时，对 M 的请求形成两组同址访问；对 N 的请求则是同一组 16 个连续列各被访问两次。这可直接由式(7.4)和内积代码得到，进一步说明应按实际 warp 布局判断。

7.3.4 共享内存分块：协作加载与数据复用

7.3.4.1 将数据搬运方式与计算访问方式分开

共享内存分块(*shared-memory tiling*)让一个线程块协作加载输入子块，再在共享内存中反复使用这些值。全局内存加载按便于合并的顺序分工，乘法运算则按内积所需的行列关系读取共享内存。

设子块边长为 $T = \texttt{TILE_DIM}$，阶段编号为 p，块内线程坐标为 t_x, t_y。每个线程在阶段 p 加载

$$M_s[t_y][t_x] \leftarrow M[rW + pT + t_x], \tag{7.12}$$

$$N_s[t_y][t_x] \leftarrow N[(pT + t_y)W + c]. \tag{7.13}$$

在同一条块内 y 行上，r 与 t_y 固定，t_x 连续变化，c 也随 t_x 连续变化，因此两个输入的加载地址都连续。

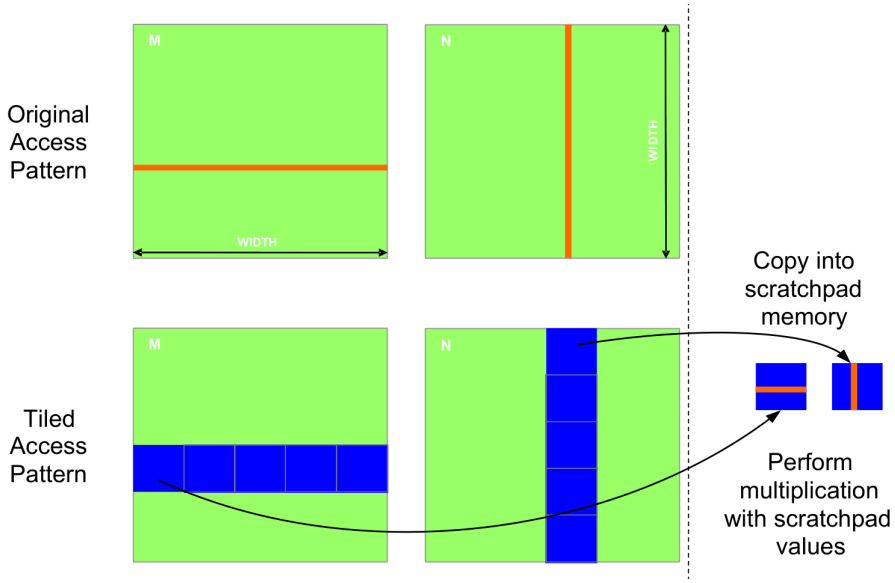


图 7.7 输入子块先复制到**暂存区**(*scratchpad memory*)，再使用暂存值执行乘法。蓝色区域表示分阶段加载的子块，橙色线表示内积所需的行列方向。

7.3.4.2 完整计算阶段与同步位置

方阵分块核函数依次执行协作加载、同步、计算和再次同步。T 为编译期常量，`Width` 是 T 的正整数倍；线程块为 $T \times T$，网格为 $(\texttt{Width}/T) \times (\texttt{Width}/T)$。这些约定使所有加载下标有效。

代码 7.3 包含协作加载、两次同步和输出写回的分块矩阵乘法。

```
template<int T>
__global__ void matmulTiled(const float* M, const float* N,
                            float* P, int Width) {
    __shared__ float M_s[T][T];
    __shared__ float N_s[T][T];
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int row = blockIdx.y * T + ty;
    int col = blockIdx.x * T + tx;
    float sum = 0.0f;

    for (int tile = 0; tile < Width / T; ++tile) {
        M_s[ty][tx] = M[row * Width + tile * T + tx];
        N_s[ty][tx] = N[(tile * T + ty) * Width + col];
        __syncthreads();

        for (int k = 0; k < T; ++k) {
            sum += M_s[ty][k] * N_s[k][tx];
        }
        __syncthreads();
    }
    P[row * Width + col] = sum;
}
```

第一次同步确保两个输入子块已经由所有线程加载完成；第二次同步确保所有线程都用完当前子块，随后才允许下一阶段覆盖这两块共享内存。

7.3.4.3 合并收益与复用收益分别计算

加载顺序 在 $T = 32$ 的示意下，warp 中的 $t_x = 0, \ldots, 31$，两个加载式分别请求 32 个连续元素。在简化事务模型中，这类连续加载可由一次事务服务。块宽变化后仍按实际 warp 布局分析，例如 $T = 16$ 时每个 warp 包含两段各 16 个元素的行内加载。

同一输出行直接读取 M 时，整个 warp 取得一个共同元素；协作加载阶段则一次取得一段不同的 M 元素，随后供块内重复使用。这样既提高传输中不同有效数据的利用程度，也保留后续的复用机会。

源代码层面的读取次数 一个 $T \times T$ 输出子块在一个内积阶段需要 T^3 次乘法。逐输出直接读取两项输入的写法，对应 $2T^3$ 次逻辑元素读取；协作加载两个 $T \times T$ 子块，则只在加载阶段执行 $2T^2$ 次全局元素读取。于是

$$\frac{\text{逐输出读取次数}}{\text{协作加载次数}} = \frac{2T^3}{2T^2} = T. \tag{7.14}$$

这项计数用于解释复用机会：同一输入元素在块内参与多次乘法。广播和缓存会影响实际 DRAM 流量，所以该比值描述代码中的请求计数，不直接等于实际带宽或加速比。

资源代价 代码中两个共享数组共占 $2T^2$ 个 float。按每个 float 为 4 字节，共享内存需求为

$$S_{\text{block}} = 8T^2\,\text{B}. \tag{7.15}$$

例如 $T = 16$ 时为 2,048 B，$T = 32$ 时为 8,192 B；对应线程数分别为 256 与 1,024。这是由代码得到的资源计数。更大的子块增加复用机会，同时消耗更多每块资源，因而会与占用率产生权衡。

7.4 线程粒度、指令调度与同步

7.4.1 从细粒度线程到线程粗化

7.4.1.1 细粒度划分与透明可扩展性

细粒度线程划分（*fine-grain thread granularity*）把尽可能小的可并行工作单元分给每个线程：向量加法中一个线程处理一个向量元素，RGB 转灰度或模糊操作中一个线程处理一个输出像素，矩阵乘法中一个线程处理一个输出元素。

这种划分向硬件暴露尽可能多的线程。资源充足时，可以同时推进较多工作；线程数超过硬件当时能容纳的数量时，其余工作可由硬件以较低开销分批执行。未来设备具有更多资源时，同一工作划分还能直接提取更多并行性，这就是**透明可扩展性**（*transparent scalability*）。

过细划分的代价 若把工作拆给更多线程会额外引入重复读取、重复计算或同步，这些开销会随着细粒度划分一起出现。只有一部分线程能同时执行时，程序可能已经承担了更多并行化开销，却仍需要等待多批工作依次完成。

7.4.1.2 一个线程承担多个工作单元

线程粗化(*thread coarsening*)把多个可并行工作单元交给同一线程。**粗化因子**(*coarsening factor*)表示每个线程负责的工作单元数。例如，粗化因子为 4 时，每个线程处理四个工作单元。

下列片段在 kernel 内使用，`foo(i)` 表示已经定义好的第 i 个独立工作单元的处理函数。未粗化时，每个线程只调用一次：

代码 7.4 细粒度映射。

```
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
foo(i);
```

粗化之后，每个线程顺序处理四个单元：

代码 7.5 连续四单元分配给同一线程。有效下标范围在该概念片段中作为前提。

```
unsigned int iStart =
    4 * (blockIdx.x * blockDim.x + threadIdx.x);
for (unsigned int c = 0; c < 4; ++c) {
    unsigned int i = iStart + c;
    foo(i);
}
```

粗化把一部分跨线程的工作关系改为线程内部的顺序关系，为复用数据或中间结果、减少重复工作和同步提供机会；具体收益取决于原来为并行化付出了什么开销。

7.4.2 粗化后的地址布局与资源代价

7.4.2.1 连续分配给线程可能产生跨线程步长

设一个 warp 中相邻线程的全局编号为 $q_0 + \ell$。在上面的连续四单元分配中，固定粗化循环下标 c 后，第 ℓ 个 lane 的元素下标为

$$i_{\ell,c} = 4(q_0 + \ell) + c, \qquad i_{\ell+1,c} - i_{\ell,c} = 4. \tag{7.16}$$

若 `foo(i)` 访问数组第 i 个 float，相邻 lane 相隔 4 个元素，即 16 B。每个线程自己的四次访问相邻，但同一次循环迭代中，warp 的地址带有步长。

7.4.2.2 按线程块宽度跨步，保留每轮连续访问

另一种映射让相邻线程在同一轮处理相邻元素，让同一线程在不同轮之间跨过一个线程块宽度。令 $B = \texttt{blockDim.x}$，代码为：

代码 7.6 保持每次迭代内 warp 连续访问的四倍粗化。

```
unsigned int base =
    blockIdx.x * (4 * blockDim.x) + threadIdx.x;
#pragma unroll
for (unsigned int c = 0; c < 4; ++c) {
    unsigned int i = base + c * blockDim.x;
    if (i < n) {
        foo(i);
    }
}
```

用 b 表示线程块编号，t 表示块内线程编号，则该映射为

$$i(b,t,c) = 4bB + t + cB, \qquad 0 \le t < B, \quad 0 \le c < 4. \tag{7.17}$$

固定 b,c，相邻线程的下标差为 1；固定 b,t，同一线程下一轮的下标增加 B。一个块覆盖从 $4bB$ 到 $4(b+1)B-1$ 的连续 $4B$ 个位置。

例如取 $b=0, B=32$，一个 warp 的映射如下。表中的数字为**元素下标**：

表 7.2 四轮粗化中的线程分工，由式(7.17)计算。

循环下标 c	lane 0	lane 1	lane 2	lane 31
0	0	1	2	31
1	32	33	34	63
2	64	65	66	95
3	96	97	98	127

因此，对 n 个工作单元，线程块数量应相应改成

$$G = \left\lceil \frac{n}{4B} \right\rceil. \tag{7.18}$$

最后一个块中，`i<n` 使超过数组长度的位置不执行处理。

7.4.2.3 粗化因子与占用率一起选择

粗化可能减少冗余内存访问、冗余计算、同步开销或控制分歧；同时，一个线程也可能需要更多寄存器或其他资源，进而降低占用率。粗化因子过大，还会使可并发线程过少而无法充分利用设备。

相同的粗化因子在不同设备上可能表现不同，因此需要重新调节。影响性能的关系包括：每线程增加了多少有效工作、减少了多少重复开销、每线程资源增加多少，以及仍有多少独立线程或指令可以隐藏延迟。

粗化的两个联合检查

一方面检查工作分配：每轮相邻 lane 是否仍访问相邻元素，网格是否覆盖全部工作；另一方面检查资源代价：增加的线程内工作是否值得以更少的驻留线程来交换。

7.4.3 循环展开、指令调度与寄存器使用

7.4.3.1 展开循环体，减少循环控制

循环展开（*loop unrolling*）把循环体复制若干次，同时按相同倍数减少循环迭代次数。循环体执行的总工作量保持一致，循环计数、判断和跳转出现的次数则减少。
设 `A(i)`、`B(i)` 是已定义的操作，原循环为：

代码 7.7 原始循环执行 16 次，每次处理一个下标。

```
for (unsigned int i = 0; i < 16; ++i) {
    A(i);
    B(i);
}
```

按因子 4 展开之后，每次循环处理四个下标，外层循环只执行 4 次：

代码 7.8 按因子 4 展开的循环。

```
for (unsigned int i = 0; i < 16; i += 4) {
    A(i);     B(i);
    A(i + 1); B(i + 1);
    A(i + 2); B(i + 2);
    A(i + 3); B(i + 3);
}
```

循环展开减少停顿的两条路径是：减少循环分支，以及暴露更多独立指令供后续调度。当缺少分支预测、循环分支具有较长延迟时，减少分支也能减少相应等待。

7.4.3.2 拉开依赖指令之间的距离

指令调度(*instruction scheduling*)在保持程序依赖关系的条件下重排指令，把相互依赖的指令隔得更远，从而让中间穿插的独立指令填补等待时间。
示例明确假设：`B(i)` 依赖 `A(i)`，而不同 i 之间没有冲突依赖或副作用。也就是每个下标内部存在 $A(i) \to B(i)$，不同下标的操作链相互独立。于是展开后的指令可以排列为：

代码 7.9 在各迭代独立的前提下先安排 A，再安排依赖它们的 B。

```
for (unsigned int i = 0; i < 16; i += 4) {
    A(i);
    A(i + 1);
    A(i + 2);
    A(i + 3);
    B(i);
    B(i + 1);
    B(i + 2);
    B(i + 3);
}
```

这样，`A(i)` 与 `B(i)` 之间插入了另外三个独立的 `A` 操作。循环展开扩大了可见的指令序列，指令调度再利用其中的独立性。这种单个线程内部可利用的指令并行性称为**指令级并行**(*instruction-level parallelism, ILP*)。

若不同迭代之间存在必须保留的依赖，能够重排的范围就由这些依赖限定。例如，示意中的调度依赖于“不同迭代没有冲突”这一明确假设。

7.4.3.3 编译器指令与编译期常量

循环展开和指令调度通常由编译器自动完成，也可以使用指令控制展开。例如，使用 `##pragma unroll 4`：

代码 7.10 使用编译器指令指定展开因子。

```
#pragma unroll 4
for (unsigned int i = 0; i < 16; ++i) {
    A(i);
    B(i);
}
```

固定循环范围有利于编译器识别迭代次数。线程粗化常会引入这种短循环；粗化因子为编译期常量时，粗化循环可以完全展开。

7.4.3.4 常量下标使局部数组更容易提升为寄存器

粗化之后，一个线程可能用小数组保存多个工作单元的状态。例如，从一个标量变为四个元素：

代码 7.11 具有变量下标的线程局部数组。

```
int x[4] = {0, 0, 0, 0};
for (unsigned int c = 0; c < 4; ++c) {
    foo(x[c]);
}
```

完全展开后，各次访问的下标成为常量：

代码 7.12 展开后各次访问具有固定下标。

```
int x[4] = {0, 0, 0, 0};
foo(x[0]);
foo(x[1]);
foo(x[2]);
foo(x[3]);
```

固定下标使每个数组元素可分别对应一个寄存器值，从而提供**寄存器提升**（*register promotion*）的机会。变量下标可能使数组需要可寻址的存储，常量下标则便于编译器把各元素分别分配到寄存器；实际分配取决于编译器优化和资源限制。

三种变换的关系 线程粗化改变“一个线程负责多少工作”；循环展开改变“同一线程怎样表达这些工作”；指令调度改变“在依赖允许范围内，先执行哪条指令”。三者可以结合，以减少重复开销、循环控制和依赖等待，同时仍需考虑增加的寄存器需求。

7.4.4 双缓冲与同步依赖

7.4.4.1 单缓冲为何需要两次屏障

双缓冲（*double buffering*）用一个缓冲区读取当前数据，用另一个缓冲区写入新数据，消除因共享同一存储位置而产生的**假依赖**（*false dependence*）。

设一个线程块内的线程编号为 `tid`，`other` 是该线程要读取的有效块内下标。每轮从其他线程对应的旧状态取得数据，再更新自己的状态。用已定义函数 `updateValue` 表示状态更新，单缓冲循环可写为：

代码 7.13 单缓冲迭代：读完旧值后才能覆盖，写完新值后下一轮才能读取。

```
for (int j = 0; j < steps; ++j) {
    float next = updateValue(buffer[other]);
    __syncthreads();
    buffer[tid] = next;
    __syncthreads();
}
```

第一次屏障：保护尚未完成的旧值读取 一些线程可能已经读完并准备写入，而另一些线程仍在读取旧数据。第一次 `__syncthreads()` 要求整个块先完成旧值读取，随后再覆盖同一数组。它保护的是**写后于读**（*write-after-read, WAR*）的顺序。这个约束源于读写复用了同一存储位置，因此属于假依赖。

第二次屏障：确保下一轮读到已完成的新值 下一轮需要使用本轮其他线程产生的新数据，所以所有写入都要完成，线程才能进入下一轮读取。这个**读后于写**（*read-after-write, RAW*）的关系来自计算的数据流，属于**真依赖**（*true dependence*）。

假依赖仍决定单缓冲实现的正确性。改变存储安排、让旧值与新值分别保存之后，才具备移除第一道屏障的条件。

7.4.4.2 两个缓冲区交替承担输入与输出

假设 buffer0、buffer1 是两个大小相同、彼此分离的块内共享缓冲区，初始状态已放入 buffer0，并已完成必要的初始化同步。所有参与线程执行相同轮数。上述初始化前提用于明确执行语义，指针交换在代码中显式展开。

代码 7.14 双缓冲迭代：只保留每轮末尾的屏障。两个指针交换输入与输出缓冲区的角色。

```
float* inBuffer = buffer0;
float* outBuffer = buffer1;
for (int j = 0; j < steps; ++j) {
    float next = updateValue(inBuffer[other]);
    outBuffer[tid] = next;
    __syncthreads();

    float* tmp = inBuffer;
    inBuffer = outBuffer;
    outBuffer = tmp;
}
```

读取 inBuffer 时，其他线程写入的是 outBuffer，因此写入新数据不会覆盖本轮仍需读取的旧值。每轮结束的屏障仍然保留，确保所有新数据都已完成写入。交换两个指针后，刚写好的数组成为下一轮输入，原输入数组成为下一轮输出。

表 7.3 双缓冲的角色随迭代交替，由指针交换代码直接得到。

迭代编号	读取旧状态	写入新状态	交换后的输入指针
$j = 0$	buffer0	buffer1	指向 buffer1
$j = 1$	buffer1	buffer0	指向 buffer0
$j = 2$	buffer0	buffer1	指向 buffer1

循环状态的检查 每次交换之后，inBuffer 都指向最新结果，outBuffer 指向可供下一轮覆盖的旧缓冲区。交换只改变指针代表的角色，不搬运数组内容。最后一轮也完成交换，因此循环退出后的最新状态仍由 inBuffer 指向。以上结论直接来自代码中的赋值顺序。

7.4.4.3 减少屏障与增加存储之间的交换

按上述两个循环计数，执行 $J = \texttt{steps}$ 轮时，单缓冲的循环内屏障调用次数为 $2J$，双缓冲为 J；初始化同步不计入这项比较。减少的是用于约束旧值读取与覆盖写入的那一道屏障。

若一个缓冲区占用 S 字节，两块缓冲区需要 $2S$ 字节。在共享内存中实现时，增加的每块存储需求还要与占用率一起考虑。同步开销在总时间中所占比例，以及额外存储对并行执行的影响，共同决定实际收益。

双缓冲保留的数据依赖

旧状态与新状态分别存储，使本轮读取和本轮写入可以不经中间屏障直接衔接；本轮写入与下一轮读取之间仍有真依赖，所以每轮末尾的屏障保留。

7.5 优化权衡与例题

7.5.1 常见优化的功能清单

7.5.1.1 计算利用率

计算利用率优化主要处理可执行工作不足、循环和依赖停顿，以及 SIMD 执行中的空闲线程。

表 7.4 计算利用率优化的收益与实现方向。

优化	对计算与访存的收益	实现方向
占用率调节	用更多工作隐藏流水线延迟；用更多并行内存请求隐藏 DRAM 延迟。	调节每块线程数、每块共享内存及每线程寄存器等 SM 资源用量。
循环展开	减少循环分支；暴露独立指令；可能促进局部数组进入寄存器而减少内存访问。	利用编译器展开，在适用处使用编译期可知的固定循环范围。
减少控制分歧	提高 SIMD 效率，减少统一执行过程中空闲的计算资源。	重新组织线程与工作、数据的映射，或调整数据布局。

7.5.1.2 内存利用率

内存利用率优化分别减少无效传输、跨层重复取数和冲突串行化。共享内存分块主要减少全局内存与 SM 之间的重复传输；寄存器分块进一步减少共享内存到寄存器的重复取数。

表 7.5 内存利用率优化。私有化用于减少共享访问冲突。

优化	对计算与访存的收益	实现方向
合并全局访存	减少等待全局数据的停顿；减少无效流量，提高 burst / 缓存行利用率。	调整线程映射或数据布局；合并地搬入共享内存，再在其中执行所需访问，例如 corner turning。
共享内存分块	减少全局内存流量及相应等待。	把线程块内重复使用的数据保存在共享内存，减少跨全局内存层的重复传输。
寄存器分块	减少共享内存流量及相应等待。	把线程或 warp 内复用的数据放入寄存器，减少共享内存到寄存器的重复取数。
私有化 *privatization*	减少原子更新引起的等待、竞争与串行化。	先在私有副本上做局部更新，完成后再更新公共副本。

7.5.1.3 同步延迟与一般工作粒度

Warp 级原语 (*warp-level primitives*) 先在 warp 内完成需要协调的操作，再在块内合并各 warp 的结果，可减少块级屏障和共享内存流量。

双缓冲 把写入和此前的读取放到不同缓冲区，消除写后于读的假依赖，从而删除用于保护该假依赖的屏障。

线程粗化 把多个并行单元分配给同一线程，用于减少不必要的并行化开销。计算和内存两方面的收益均取决于被减少的开销类型。

7.5.2 优化顺序、资源权衡与瓶颈

7.5.2.1 从数据搬运到线程内工作

优化目标可以按以下顺序组织。这个顺序先检查传输量，再检查传输效率与并发程度，最后结合每线程工作和实际测量。

1. **减少不必要的数据流量。**通过复用和分块，避免反复搬运已经取得的数据。
2. **提高必要流量的传输效率。**通过合并访存，让一次传输覆盖尽可能多的有效数据。
3. **暴露足够多的并行工作。**通过延迟隐藏和 MLP，使不同请求能够重叠处理。
4. **增加每线程的有效工作。**结合线程粗化和 ILP，减少重复开销并提高独立指令的可见性。

5. **比较优化前后的测量结果。**依据应用在目标设备上的表现评价变换。

7.5.2.2 优化会交换不同资源

占用率、数据复用与每线程工作量共享同一组有限的 SM 资源。增加驻留线程可以隐藏更多延迟，也会增加寄存器、共享内存和缓存的竞争；增加共享内存子块可以提高复用，但可能减少同时驻留的线程块；增加粗化因子可以减少并行化开销，但提高每线程资源需求。

这些关系使单个指标的最大值与实际性能之间存在权衡。合理的配置需要同时保留足够独立的工作，以及支撑每线程有效计算和数据复用的资源。

7.5.2.3 以应用在目标设备上的瓶颈为依据

瓶颈(*bottleneck*)是限制应用在某个设备上继续提高性能的约束。它同时取决于应用与设备：相同代码在不同设备上的限制可能不同，同一设备处理不同应用也可能受到不同资源约束。

优化通过增加一种资源的使用，缓解另一种资源的限制。诊断出主要瓶颈之后，才有依据判断应优先减少数据量、改善访存布局、增加并行请求，还是减少同步和线程内停顿。

性能上限与停止条件　将应用性能与对应的**速度极限**(*speed-of-light performance*)比较，有助于判断还有多大的改进空间，以及何时停止继续优化。比较时需使用相同设备和计算条件下的性能上限。

7.5.3 例题一：线程块形状改变访存优势

7.5.3.1 题目与已知信息

下面两个写法使用相同的全局线程坐标。比较它们的全局内存访问效率：

代码 7.15 两种行列下标映射。A、B 表示数组名；注释中的 A/B 表示待比较的两种写法。

```
int tx = blockIdx.x * blockDim.x + threadIdx.x;
int ty = blockIdx.y * blockDim.y + threadIdx.y;

// Form A
B[ty * Width + tx] = 2 * A[ty * Width + tx];

// Form B
B[tx * Width + ty] = 2 * A[tx * Width + ty];
```

只给出地址表达式时，答案是**尚不能判断**，因为线程块的启动形状没有给出。两种写法的输入读取和输出写入都使用各自相同的下标，因此分析该下标在一个 warp 内如何变化即可。

7.5.3.2 32×8 与 1×32 的相反结果

记 $W = \texttt{Width}$。忽略所有线程共有的起始偏移，只比较相邻 lane 之间的下标变化。

线程块为 `dim3(32, 8)` 一个 warp 内的 `ty` 固定，`tx` 随 lane 增加 1。写法 A 的下标增加 1，写法 B 的下标增加 W，因此原例给出的预期优势是 A：

$$I_{\mathrm{A}}(\ell) = I_{\mathrm{A}}(0) + \ell, \tag{7.19}$$

$$I_{\mathrm{B}}(\ell) = I_{\mathrm{B}}(0) + \ell W. \tag{7.20}$$

线程块为 `dim3(1, 32)` 一个 warp 内的 `tx` 固定，`ty` 随 lane 增加 1。此时写法 A 的下标增加 W，写法 B 的下标增加 1，原例给出的预期优势变为 B：

$$I_{\mathrm{A}}(\ell) = I_{\mathrm{A}}(0) + \ell W, \tag{7.21}$$

$$I_{\mathrm{B}}(\ell) = I_{\mathrm{B}}(0) + \ell. \tag{7.22}$$

同一段下标表达式的表现，会随着哪一个线程坐标在 warp 内连续变化而改变。

7.5.3.3 16×16：分别展开两组 16 个线程

考察线程块内第一个 warp，去掉全局起点偏移后，前 16 个 lane 的坐标为 $(t_x, t_y) = (0,0), \ldots, (15,0)$，后 16 个 lane 为 $(0,1), \ldots, (15,1)$。因此两种写法的相对元素下标分别为：

$$I_{\mathrm{A}}: \quad \underbrace{0, 1, \ldots, 15}_{t_y=0}, \quad \underbrace{W, W+1, \ldots, W+15}_{t_y=1}, \tag{7.23}$$

$$I_{\mathrm{B}}: \quad \underbrace{0, W, \ldots, 15W}_{t_y=0}, \quad \underbrace{1, W+1, \ldots, 15W+1}_{t_y=1}. \tag{7.24}$$

写法 A 形成两段各 16 个元素的行内连续访问；写法 B 在每组 16 个线程内部具有步长 W，两组之间在对应位置形成相邻的两个元素。具体传输区域的覆盖还要代入矩阵宽度与起点，因此源例要求分别分析两组线程，而不是直接套用“整个 warp 都沿 x 变化”的情形。

表 7.6 启动形状与 warp 地址分布的对应关系。

线程块形状	warp 内坐标变化	原例的判断
`dim3(32, 8)`	同一行内 x 连续变化	A 形成连续行访问。
`dim3(1, 32)`	x 固定，y 连续变化	交换索引后的 B 形成连续访问。
`dim3(16, 16)`	两条 x 行，各 16 个线程	对两个半 warp 分别展开地址。

7.5.4 例题二：物理带宽与有效输入吞吐率

7.5.4.1 保留题目中的全部假设

每个 float 为 4 字节；输入起点按 512 字节对齐；流式输入数据起初不在缓存中；两次加载所需的每个 512 字节区域总共只读取一次；并发程度足以持续达到 240 GB/s 的物理读取流量；忽略其他流量。求理想的有效输入吞吐率。

代码 7.16 每个线程使用四个连续元素中的前两个。

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
float temp = A[4 * i] + A[4 * i + 1];
```

512 字节区域与 240 GB/s 均是这个题目的给定条件。有效吞吐率只统计上面两次加载真正使用的输入字节。

7.5.4.2 先在四元素组内计算有效比例

线程 i 涉及的四元素组为

$$A[4i],\ A[4i+1],\ A[4i+2],\ A[4i+3]. \tag{7.25}$$

实际只使用前两个元素，因此每 $4\times4=16\,\mathrm{B}$ 的地址区间中，使用 $2\times4=8\,\mathrm{B}$，另有 $8\,\mathrm{B}$ 未用于该运算。

表 7.7 一个 16 字节组中的有效输入位置。

元素	$A[4i]$	$A[4i+1]$	$A[4i+2]$	$A[4i+3]$
相对字节范围	0–3	4–7	8–11	12–15
是否用于 `temp`	是	是	否	否

7.5.4.3 再用一个 512 字节区域核对

取 $i=0,\dots,31$。32 个线程覆盖 32 组、每组 16 字节，共

$$D_{\text{physical}} = 32\times16 = 512\,\mathrm{B}. \tag{7.26}$$

每个线程使用两个 float，对应的有效输入为

$$D_{\text{useful}} = 32\times2\times4 = 256\,\mathrm{B}. \tag{7.27}$$

于是有效字节比例为

$$\eta = \frac{D_{\text{useful}}}{D_{\text{physical}}} = \frac{256}{512} = \frac{1}{2}. \tag{7.28}$$

这里的两次加载共同使用同一个区域，题目规定这个区域只读取一次，所以物理字节数按 $512\,\mathrm{B}$ 计。

7.5.4.4 将有效字节比例乘以物理带宽

设 B_{physical} 为物理读取带宽，B_{useful} 为有效输入吞吐率，则在题设条件下

$$B_{\text{useful}} = \eta B_{\text{physical}} = \frac{1}{2}\times240\,\mathrm{GB/s} = \boxed{120\,\mathrm{GB/s}}. \tag{7.29}$$

这个结果保留了题目的“理想有效输入吞吐率”口径：物理接口仍按每秒 240 GB 读取，但其中一半字节用于指定的两个输入元素。

两道例题的共同计算方法

先根据实际线程布局展开同一次访问的地址，再对照题目给出的传输区域统计有效字节。第一题强调 launch 决定 warp 坐标关系；第二题强调所用字节数与所传字节数共同决定有效输入吞吐率。

第 8 讲

Convolution Concept; Constant Cache

8.1 课程安排与核心内容

8.1.1 核心内容与学习目标

授课日期为 2026 年 9 月 17 日。**卷积**(*convolution*)是信号、图像和视频处理中的常见并行计算模式，也与科学计算中的**模板计算**(*stencil computation*)及神经网络有关。学习目标是理解卷积，并利用缓存提高 GPU 数据访问效率。一维与二维邻域加权和确定了输入访问模式，常量缓存与共享内存分别利用系数一致性和输入窗口重叠提高数据复用。

8.1.2 课程安排与阅读材料

以下安排对应上述授课日期。

事项	安排
Lab 2	授课当周的星期五截止。
Midterm 1 时间	2026 年 10 月 6 日，19:00—22:00。
地点与形式	具体考场分配发布在 Canvas；纸笔考试。
考试范围	Lectures 1—11，Labs 1—4。
时间冲突安排	有有效时间冲突者须在 2026 年 9 月 30 日前邮件联系教师，另行安排考试。
复习与阅读	复习资料位于 Canvas 的 Files/MT1；指定阅读为教材第 7 章。

课程安排与阅读材料。

核心关系

每个输出依赖一个输入邻域，而不同输出使用相同的滤波器。前一种重叠带来输入数据的复用机会，后一种一致性适合常量存储器。分块时，输出覆盖范围确定了需要载入的输入核心区与两侧 halo；线程如何分担这些载入工作，决定了三种策略的区别。

8.2 卷积的计算结构与边界处理

8.2.1 邻域加权和与滤波器方向

一个输出元素由其附近输入元素的加权和得到。权重数组称为**卷积掩模**(*convolution mask*)或**滤波器**(*filter*)，通常也称卷积核；为避免与 CUDA 核函数混淆，以下用“滤波器”指权重数组，用“核函数”指设备端并行程序。同一个滤波器通常用于整个输入数组。相邻位置使用不同的输入窗口，但使用同一组权重，因此改变滤波器就能改变对局部信号的处理方式。

连续函数与离散序列的数学卷积分别写为

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau) g(x-\tau)\,\mathrm{d}\tau, \qquad (f * g)[x] = \sum_{k=-\infty}^{\infty} f[k]\,g[x-k]. \tag{8.1}$$

其中 x 指定输出位置，τ 或 k 遍历参与计算的输入位置。表达式中的 $x-\tau$、$x-k$ 对应其中一个函数或序列的翻转与平移。

互相关(*cross-correlation*)采用不翻转滤波器的滑动匹配。对实值函数，可写为 $(f \star g)(t) = \int f(\tau) g(t+\tau)\,\mathrm{d}\tau$。卷积常用于物理系统与滤波，互相关常用于模式匹配与机器学习。CNN 中通常计算互相关，但历史上仍沿用“卷积”的名称；当权重通过训练获得时，是否预先规定翻转可通过学习到的权重方向吸收。

后续数组与 CUDA 示例采用**滤波器按原顺序读取**的约定。这个约定直接决定代码中使用 `M[j]`，也便于对应每一个乘加项。给定一组非对称权重时，翻转与不翻转通常会得到不同结果；下面一维示例的权重是对称的，因此该示例的数值无法单独区分这两种方向。

8.2.2 一维窗口、中心与零填充

输入数组记为 N，输出数组记为 P，长度均为 W；滤波器记为 M，宽度为 K。代码使用 `Width`、`Mask_Width` 表示这两个宽度。取奇数 $K = 2r+1$，则半径 $r = (K-1)/2$ 表示输出中心左右各涉及多少个输入元素；代码中的整数除法 `Mask_Width / 2` 在这个条件下等于 r。

为形式化说明边界判断，定义零延拓输入 $\widetilde{N}[q]$：有效下标 $0 \le q < W$ 时等于 $N[q]$，其

他位置等于零。按滤波器原顺序读取，一个输出为

$$P[i] = \sum_{j=0}^{K-1} \widetilde{N}[i-r+j]M[j], \qquad 0 \le i < W. \tag{8.2}$$

这个表达式把三个下标作用分开：i 决定输出位置，j 决定滤波器内的位置，$i-r+j$ 决定当前乘加读取哪个输入。$j=0$ 对应窗口最左端，$j=r$ 对应输入中心 $N[i]$，$j=K-1$ 对应最右端。

以 $N=[1,2,3,4,5,6,7]$、$M=[3,4,5,4,3]$ 为例，$K=5$、$r=2$。计算 $P[2]$ 时读取 $N[0]$ 到 $N[4]$，而计算 $P[3]$ 时整个窗口向右移动一格：

$$\begin{aligned} P[2] &= 1\cdot 3 + 2\cdot 4 + 3\cdot 5 + 4\cdot 4 + 5\cdot 3 = 57, \\ P[3] &= 2\cdot 3 + 3\cdot 4 + 4\cdot 5 + 5\cdot 4 + 6\cdot 3 = 76. \end{aligned} \tag{8.3}$$

每一个输出都需要五次乘加，但这两个窗口共同使用了 $N[1]$ 到 $N[4]$。这种重叠正是后续共享内存分块的依据。

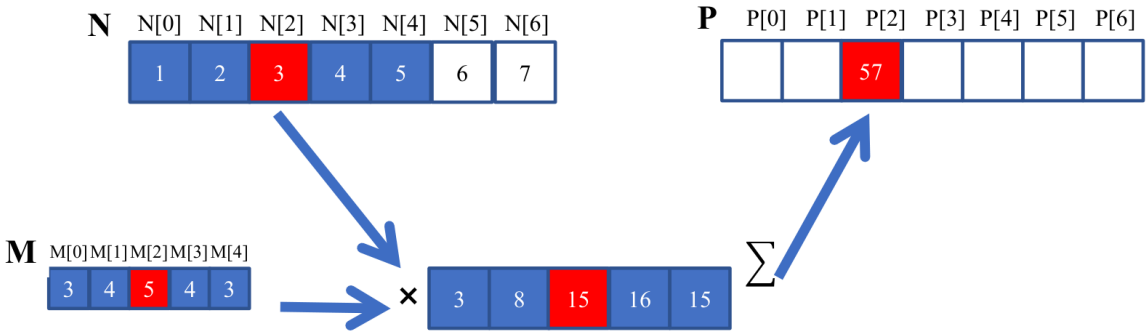


图 8.1 一维窗口与滤波器逐项相乘，再归约到输出 $P[2]=57$。

当窗口跨出数组边界时，会出现没有对应实际数组元素的位置，称为**幽灵元素**（*ghost elements*）。边界规则可以采用零填充，也可以复制边缘值；本讲代码采用零填充。输出 $P[1]$ 的输入窗口为 $[0,1,2,3,4]$，因此 $P[1]=0\cdot 3+1\cdot 4+2\cdot 5+3\cdot 4+4\cdot 3=38$。

零填充可以通过条件判断实现：遇到无效输入下标就跳过乘加。由于该项应为零，跳过与显式读取一个零具有相同的数学作用，因此无须为基本核函数实际分配数组外的元素。按式(8.2)复算全部位置，可得到 $P=[22,38,57,76,95,90,74]$，可作为边界实现的参考结果。

8.2.3 二维邻域与图像平滑

二维计算将一维窗口扩展为一个矩形邻域。设输入有 H 行、W 列，滤波器为 $K \times K$，两个方向的半径均为 r。在与一维相同的读取约定下，零延拓输入为 $\tilde{N}$，输出行列为 (y, x)，则

$$P[y, x] = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} \tilde{N}[y - r + u, x - r + v] M[u, v]. \tag{8.4}$$

两个求和分别遍历滤波器的行与列；每个输出将整个局部窗口压缩为一个数，输入窗口与滤波器先逐项相乘，再将全部乘积相加。

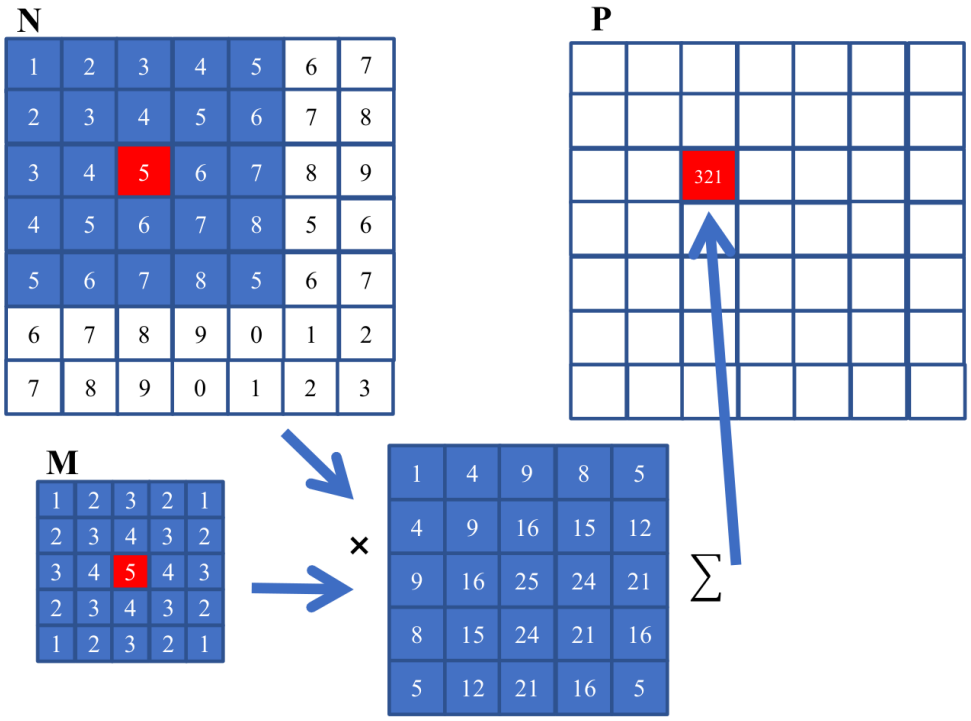


图 8.2 5×5 输入邻域与权重逐项相乘，得到二维内部输出 $P[2, 2] = 321$。

在图中的零起始坐标下，中心为 $(2, 2)$，使用输入的前五行、前五列。乘积矩阵各行之和依次为 $27, 56, 95, 84, 59$，合计 321。在边界例中，输出 $(1, 0)$ 的窗口跨过上边界和左边界，填零后五行乘积之和为 $0, 16, 34, 34, 28$，合计 112。这些行和来自图中数值，可用于把二维索引与实际求和对应起来。

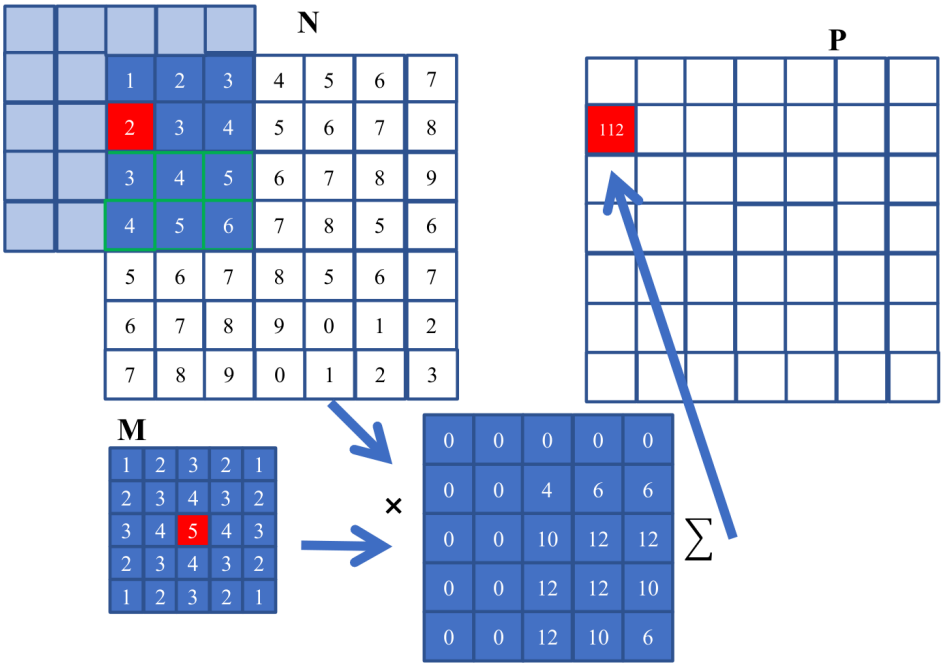


图 8.3 二维零填充：窗口上侧与左侧的无效位置取零，得到边界输出 112。

幽灵单元也称 apron cells 或 halo cells。例如，另一个零填充边界窗口的五行分别为 $[0,0,0,0,0]$、$[0,3,4,5,6]$、$[0,2,3,4,5]$、$[0,3,5,6,7]$、$[0,1,1,3,1]$。使用图中相同的整数权重，五行乘积之和依次为 $0, 52, 54, 61, 12$，输出为 179。这些名称都与邻域的外围数据有关；进入分块实现后，需要进一步区分“落在全局数组外的补零位置”与“仍在全局数组内、但落在当前块核心区外的邻域元素”。后一类是真实输入，后续分块必须读取其值。

一个具体的图像滤波器为

$$M = \frac{1}{273}\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}. \tag{8.5}$$

平滑作用与边界条件。所有系数非负，矩阵元素之和为 $17 + 66 + 107 + 66 + 17 = 273$，归一化后权重和为 1。每个输出是附近像素的加权平均，中心像素的权重较大、远处像素的权重较小，因此它起到平滑、模糊局部变化的作用。若整个窗口内的像素都等于常数 c，则输出仍为 c。采用零填充时，图像边缘窗口含有零，因而这一“保持常数”结论限于完整有效窗口；边界值可能被拉低。

8.2.4 直接 CUDA 实现：一个线程计算一个输出

输出之间只共享输入读取，没有相互依赖的输出写入，所以可让每个线程独立计算一个 $P[i]$。一维网格中的全局下标为 $i = \texttt{blockIdx.x}\,\texttt{blockDim.x} + \texttt{threadIdx.x}$。最后一个线程块可能超过数组长度，因此先检查输出下标，再在循环中检查输入下标。

代码 8.1 直接一维卷积：每个线程计算一个输出，越界输入按零处理。

```
__global__ void convolution_1D_basic_kernel(
    const float* N, const float* M, float* P,
    int Mask_Width, int Width)
{
    int i = int(blockIdx.x) * int(blockDim.x)
          + int(threadIdx.x);
    if (i >= Width) return;
    float Pvalue = 0.0f;
    int N_start_point = i - Mask_Width / 2;
    for (int j = 0; j < Mask_Width; ++j) {
        int q = N_start_point + j;
        if (q >= 0 && q < Width) {
            Pvalue += N[q] * M[j];
        }
    }
    P[i] = Pvalue;
}
```

两个范围检查各自保护不同的数组：入口处的 `i >= Width` 保护输出 P，循环内的检查保护输入 N。即使 i 合法，$i - r + j$ 仍可能越界，因此两者都需要。局部标量 `Pvalue` 是每个线程自己的累加器；每个有效线程恰好写一个输出。

实现说明。代码将只读输入写为 `const float*`，并显式把 CUDA 索引转为有符号整数，便于表达减去半径后可能出现的负值。直接版没有共享内存协作，因此无效输出线程可在入口直接返回。分块版的线程还承担输入搬运任务，返回位置需要与这些任务一起分析。

假设每块 B 个线程且 $W > 0$，网格块数取 $\lceil W/B \rceil$。在整数代码中可以写作 `1 + (Width - 1) / B`。随后，每个线程沿着滤波器执行顺序乘加，线程之间沿输出坐标并行；这同时解释了计算并行性和输入访问的重叠。

8.3 缓存层次与常量存储器

8.3.1 从主存访问到缓存命中

GPU 的计算需要持续获得输入。如果每次读取都访问全局存储器中的底层数据，带宽就可能限制执行速度。分块矩阵乘法通过共享内存显式复用输入；**缓存**（*cache*）则在硬件管理下保留近期访问数据，为后续读取提供另一条复用路径。

缓存由若干**缓存行**（*cache line*）组成，每行可存放一段连续地址的数据。当请求的数据尚未在缓存中时，硬件把包含该地址的一行数据带入缓存；缓存中还保存**标签**（*tag*），记录该行对应的内存地址。后续请求找到对应数据时称为缓存命中，否则需要继续访问后面的存储层次。缓存数据是原数据的副本。

连续地址成组传输，使一次读取也可能为后续的邻近读取准备好数据。例如，一次突发传输约 1024 bit，即 128 B 时，相邻数据可以随请求位置一起进入缓存。这个示例以一次突发对应一行说明成组搬运；具体缓存行与事务大小取决于存储层次和硬件实现。

空间局部性与时间局部性。空间局部性（*spatial locality*）指相近的时间里访问连续或邻近地址；**时间局部性**（*temporal locality*）指一个数据在短时间内被多次访问。对于固定线程，一维卷积沿着窗口依次读取 $N[i-r], N[i-r+1], \ldots$，体现空间局部性；相邻输出窗口重叠，使一个输入可能在较短时间内再次被使用，体现时间局部性。两种局部性都能提高缓存发挥作用的机会。

缓存容量小于全局存储器，不能保留全部输入。当需要为新缓存行腾出空间时，已有行会被替换；一种典型的替换思路是**最近最少使用**（*least recently used, LRU*），即优先替换最久没有被访问的缓存行。能否命中还取决于相关数据在再次访问前是否仍留在缓存中，因此“存在复用机会”与“每次都能缓存命中”需要分开判断。

8.3.2 共享内存、L1/L2 与常量缓存

CUDA 的存储空间在可见范围、读写方式及用途上不同。下表给出用于直观比较的近似访问周期。实际延迟取决于器件、缓存命中及访问模式，这些示例量级不构成统一的运行时间预测模型。

存储空间	可见范围	线程权限	示例访问周期
寄存器	单线程	读、写	~ 1 cycle
共享内存	同一线程块	读、写	~ 5 cycles
全局存储器	网格内线程	读、写	~ 500 cycles
常量存储器	网格内线程	只读	缓存条件下 ~ 5 cycles

表 8.1 CUDA 存储空间的程序员视图与教学访问量级。

共享内存与缓存都利用靠近 SM 的片上存储减少主存压力，但**内容由谁管理**决定了它们的编程方式。共享内存是程序显式管理的暂存区（scratchpad）：程序决定把什么值写入哪个位置，并在需要时同步线程。缓存的内容则由微体系结构根据内存访问自动选择和替换，程序通常不显式指定某个数据应放入哪一条缓存行。共享内存还可以保存中间结果，不要求内容始终对应全局存储器中的一个副本。

在 Volta 及之后的相关设计中，共享内存和 L1 可以使用统一的物理资源，并在这些用途之间分配。物理资源可以共享，访问与管理接口仍保持各自特点。图8.4中的局部片上存储靠近各个 SM，L2 位于多个 SM 与全局存储器之间。

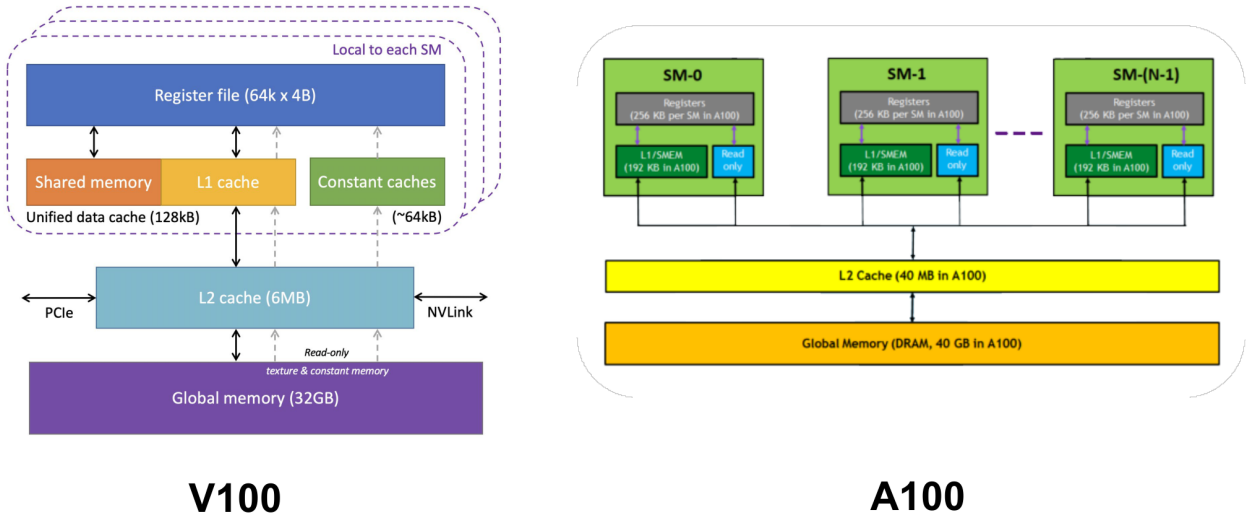


图 8.4 V100 与 A100 的缓存层次示例：SM 附近的局部存储、共享 L2 及其后的全局存储器。图中容量保留对应示例器件的标注。

对于可修改的缓存数据，硬件需要处理修改状态及与后备内存的一致性。**常量缓存**（*constant cache*）服务于核函数执行期间保持不变的数据，读取路径可以针对这类模式优化。常量和纹理的只读访问路径，与一般全局访问使用的 L1 路径具有不同用途。滤波器系数的实现使用常量存储器。

8.3.3 滤波器的访问一致性

滤波器 M 有三个相关性质：计算所有输出都需要它，它在一次网格执行期间保持不变，而且各线程按照相同的顺序读取系数。在循环第 j 次迭代，输出位置不同的线程读取各自的 $N[i-r+j]$，同时读取同一个 $M[j]$。小而重复使用、且访问顺序一致的滤波器因此适合放入**常量存储器**(*constant memory*)。

同一轮乘加的两种地址关系

对相邻输出线程 $i, i+1, \ldots$，固定 j 时，输入地址为 $i-r+j, i+1-r+j, \ldots$，沿数组相邻分布；滤波器地址均为 j。输入复用与系数复用具有不同结构，分别对应后续共享内存分块与常量存储器优化。

同地址访问与广播。同一 warp 的线程访问同一常量地址时可以共享该次读取；访问多个不同地址时，请求按不同地址串行处理，代价随不同地址数增加。因此 `Mc[j]` 的一致访问有利，而按线程编号访问各不相同的常量位置就不具备同样的优势。

输入 N 在本次卷积中也保持只读，但往往比滤波器大得多。常量存储器总容量上限为 64 KB，因此是否只读只是选择条件之一；还要考虑容量与访问结构。64 KB 指常量地址空间的容量约束，不能从这个数字直接推出每个 SM 的常量缓存大小。常量缓存未命中时仍需取得后备数据。

8.3.4 声明、初始化与常量版核函数

常量数组通过 `__constant__` 在所有主机函数与设备核函数之外声明。主机用 `cudaMemcpyToSymbol` 将权重复制到这个符号；主机端数据可以运行时生成，设备端执行网格期间按只读方式使用这些系数。

代码 8.2 常量滤波器声明及主机初始化；五个系数沿用一维数值例。

```
constexpr int MASK_WIDTH = 5;
__constant__ float Mc[MASK_WIDTH];
// Inside a host function, before launching the kernel:
float Mask[MASK_WIDTH] = {3, 4, 5, 4, 3};
cudaError_t status = cudaMemcpyToSymbol(
    Mc, Mask, MASK_WIDTH * sizeof(float));
if (status != cudaSuccess) {
    // Report or propagate the error; do not launch on failure.
    return;
}
```

复制长度的单位是字节，所以必须乘以 `sizeof(float)`。这次调用采用默认偏移 0 与

默认方向 cudaMemcpyHostToDevice。权重也可以由随机数生成器产生；核函数启动之前，都需要先完成权重初始化和复制。

常量版只把原先的 M[j] 改为 Mc[j]，窗口起点、边界条件与求和次序均保持不变：

代码 8.3 使用常量滤波器的一维卷积。

```
__global__ void convolution_1D_constant_kernel(
    const float* N, float* P, int Mask_Width, int Width)
{
    int i = int(blockIdx.x) * int(blockDim.x)
          + int(threadIdx.x);
    if (i >= Width) return;
    float Pvalue = 0.0f;
    int N_start_point = i - Mask_Width / 2;
    for (int j = 0; j < Mask_Width; ++j) {
        int q = N_start_point + j;
        if (q >= 0 && q < Width) {
            Pvalue += N[q] * Mc[j];
        }
    }
    P[i] = Pvalue;
}
```

参数与初始化条件。常量版直接读取设备符号 Mc[j]，因此核函数参数只包含输入、输出及两种宽度，无须传入滤波器指针。调用应传入已经初始化的有效宽度，满足 $0 < K \le$ MASK_WIDTH；后续完整示例固定采用奇数 $K = 5$。

假设 Nd、Pd 已完成设备端分配、Nd 已存有输入，$W > 0$，取每块 B 个线程，则调用形式为

代码 8.4 与常量版核函数匹配的启动参数。

```
dim3 dimBlock(B);
dim3 dimGrid(1 + (Width - 1) / B);
convolution_1D_constant_kernel<<<dimGrid, dimBlock>>>(
    Nd, Pd, MASK_WIDTH, Width);
```

常量版改善了系数的读取路径，但相邻线程仍会反复请求重叠的输入窗口。共享内存分块进一步利用 N 的复用。

8.4 分块的数据复用与三种策略

8.4.1 访问计数、算术强度与复用机会

对于不接触边界的一个一维输出，直接实现读取 K 个输入与 K 个系数，共 $2K$ 次标量读取，并执行 K 次乘法和 K 次加法，共 $2K$ 次浮点运算。二维 $K\times K$ 滤波器对应 $2K^2$ 次标量读取与 $2K^2$ 次浮点运算。因此增加滤波器宽度时，直接算法的读取量和计算量一起增长，其复用前的算术强度较低。

算术强度（*arithmetic intensity*）用计算量与数据搬运量之比描述计算对带宽的需求。按每个 `float` 为 4 B，将标量读取次数换成字节：一维内部输出读入 $8K$ B、写出 4 B。忽略复用时，仅以读入量为分母得到 $2K/(8K)=0.25$ FLOP/B；把输出写入也计入，得到

$$I_{\text{direct,1D}}=\frac{2K}{8K+4},\qquad I_{\text{direct,2D}}=\frac{2K^2}{8K^2+4}. \tag{8.6}$$

较低的算术强度说明直接算法容易受到带宽限制；常量缓存与共享内存的作用，就是减少同一份数据在较慢存储层次上的重复获取。

计数口径。这里的“读取次数”首先指代码层面的标量访问。缓存命中、常量同地址访问以及成组内存事务会改变实际 DRAM 流量，所以不能直接把 $2K$ 次读取等同于 $2K$ 次独立 DRAM 事务。边界线程又会在直接核函数中跳过无效项，此时实际执行的读取与乘加比内部输出少。后面的精确例题始终先说明统计的是输入请求、字节还是乘加次数。

8.4.2 输出分块、输入覆盖范围与 halo

设一个线程块生成 B 个连续输出，块号为 b，输出起点为 $s=bB$。输出范围是 $[s,s+B-1]$。最左输出需要从 $s-r$ 开始读取，最右输出需要读到 $s+B-1+r$，因此整个块需要的输入范围为

$$[s-r,\ s+B+r-1],\qquad T_{\text{in}}=B+2r=B+K-1. \tag{8.7}$$

这里 B 是输出块宽度，T_{in} 是包含边缘邻域的输入块宽度。输入核心区与输出对齐，左右各多出的 r 个位置称为**晕区**（*halo*）。

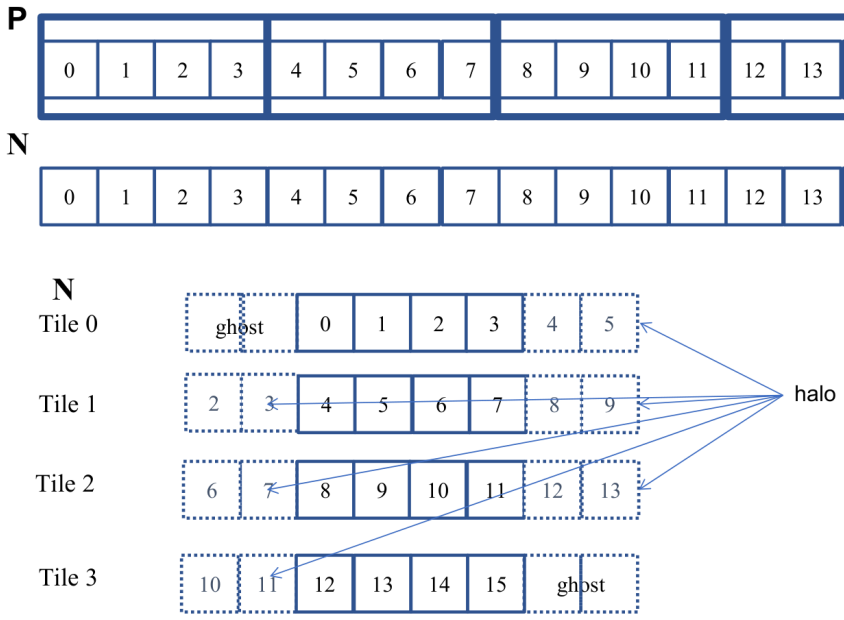


图 8.5 $W = 16$、$B = 4$、$K = 5$ 时的四个输出块及其输入邻域。中间块的 halo 是真实输入，首末块还包含越界的幽灵位置。

以 $b = 1$ 为例，这个块生成 $P[4]$ 到 $P[7]$，输入需要 $N[2]$ 到 $N[9]$。核心区是 $N[4:7]$，左 halo 是 $N[2:3]$，右 halo 是 $N[8:9]$。相邻块的**输出不重叠、输入可以重叠**，所以线程块可以独立写各自的输出，但边缘输入可能被多个块读取。第一个块需要的 $N[-2]$ 与 $N[-1]$ 才属于全局数组外的幽灵位置，按边界规则填零。

块内复用的数量。对于 $B = 4, K = 5$，四个输出分别需要五个输入，共产生 20 次输入使用，但覆盖的不同输入只有八个。$N[2]$ 到 $N[9]$ 的块内使用次数依次为 $1, 2, 3, 4, 4, 3, 2, 1$，总和为 20。一旦把这八个值放入共享内存，块内后续使用就可以从共享内存读取。

对于远离全局边界的完整块，直接方法的输入请求数为 BK；把完整输入块载入一次则为 $B + 2r$。因此只比较 N 的块内读取请求，可写出复用因子

$$R_N = \frac{BK}{B + 2r} = \frac{BK}{B + K - 1}. \tag{8.8}$$

这给出示例中的 $20/8 = 2.5$。固定 K 而增大 B 时，halo 相对占比 $2r/B$ 下降，R_N 趋近于 K。这些是输入覆盖关系推得的请求量比较，没有计入系数缓存、同步或硬件资源对实际速度的影响。

8.4.3 线程数量与 halo 放置的组合

分块之后仍有两个决定：线程数与输出块匹配，还是与输入块匹配；halo 是否也放入共享内存。由此形成三种主要策略。

比较项	策略 1	策略 2	策略 3
线程块覆盖	B 个输出	$B+2r$ 个输入	B 个输出
每块线程数	B	$B+2r$	B
共享内存内容	核心区与两侧 halo	核心区与两侧 halo	仅核心区
载入方式	部分线程额外载入	每线程载入一个输入	每线程载入一个核心输入
计算方式	每线程计算一个输出	仅部分线程计算输出	每线程计算一个输出
计算时 halo	从共享内存读取	从共享内存读取	从全局地址读取
主要取舍	载入阶段任务不均	计算阶段部分线程闲置	计算循环选择两种读取路径

表 8.2 以相同的 B 个输出为比较对象，线程数与输入存放方式的差异。

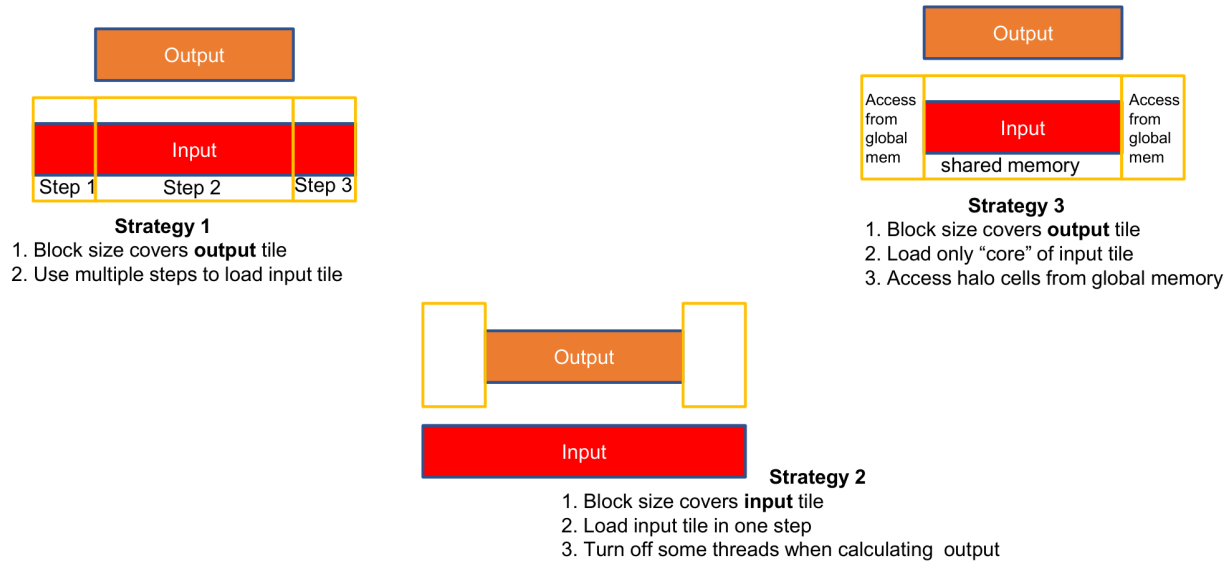


图 8.6 三种分块策略的输入、输出和线程工作范围。策略 1 分步载入完整输入块；策略 2 为输入配置线程；策略 3 将 halo 留给全局读取路径。

策略 1 的取舍。将 halo 一并载入共享内存后，卷积计算循环的输入都来自同一种位置，不需要在每次乘加时选择共享或全局路径。连续位置可以按组载入，也有利于合并访问。代价是共享内存由 B 个元素增加到 $B+2r$ 个元素，而且部分线程在载入阶段必须多做工作。载入任务按线程编号分配时，可能出现部分线程执行分支、其他线程不执行的情况。

策略 3 的取舍。只缓存核心区，把共享内存优先用于块内复用较高的输入。计算每一个乘加项时判断输入是否位于本块核心区：位于核心区就读共享内存，位于 halo 就按全局地

址读取。同一 warp 内的线程可能选择不同路径；窄 halo 的访问也可能只利用一次成组内存传输的一小部分。

按 $r \le B$ 的完整内部块展开计数，左侧各输出对左 halo 的读取数为 $r, r-1, \dots, 1$，右侧对称，因此两侧一共 $r(r+1)$ 次全局 halo 请求。再加 B 次核心区载入，策略 3 对 N 的请求数为 $B + r(r+1)$。在 $B = 4, r = 2$ 的例子中，这个数为 10，策略 1 则是 8。该比较按程序请求计数；策略 3 的 halo 请求可能由缓存满足，不能只靠这两个数判定实测速度。

策略 2 的取舍。让每个线程载入一个输入，载入阶段无需由少量线程额外承担 halo；随后仅有与输出对应的线程执行乘加。按相同输出宽度 B 比较，计算输出的线程比例为 $B/(B+2r)$。这种分配减少高延迟载入阶段的工作分歧，同时让部分线程不参与较低延迟的计算阶段。全局数组首末处的边界填零仍然需要处理。

8.5 分块核函数与索引推导

8.5.1 策略 1：完整输入块与三段载入

策略 1 为一个输出块配置 B 个线程。局部线程号记为 t，块起点为 $s = bB$，线程的输出下标为 $i = s + t$。共享数组 `N_ds` 需要容纳 $B + 2r$ 个元素。在这个共享数组中，最左位置从整个输入窗口的左端开始，因而有统一映射

$$\boxed{N_{\mathrm{ds}}[q] = \widetilde{N}[s - r + q], \qquad 0 \le q < B + 2r.} \tag{8.9}$$

右侧的 $\widetilde{N}$ 已包含零填充约定。这个关系同时描述了左 halo、核心区与右 halo，是核对所有搬运代码的共同依据。

线程 t 计算的输出为 $P[s+t]$，其第 j 项需要 $\widetilde{N}[s+t-r+j]$。代入式(8.9)可知，该项的共享内存下标恰好是 $q = t + j$。因此乘加使用 `N_ds[threadIdx.x + j]`，不再额外减去半径：半径偏移已经包含在共享数组起点的选择中。

三段载入的分工。左 halo 由局部编号最大的 r 个线程搬运，即 $t \ge B - r$ 的线程。其全局下标为 $(b-1)B + t = s - B + t$，共享下标为 $t - (B - r)$。当 $t = B - r$ 时，这两个下标分别为 $s - r$ 与 0；当 $t = B - 1$ 时，分别为 $s - 1$ 与 $r - 1$，正好覆盖左 halo。

所有线程将核心值 $N[s+t]$ 放入共享位置 $r + t$。右 halo 则由局部编号最小的 r 个线程搬运：全局下标为 $(b+1)B + t = s + B + t$，共享下标为 $r + B + t$。三个目标区间 $[0, r-1]$、$[r, r+B-1]$、$[r+B, r+B+r-1]$ 连续且不重叠，共同填满 $B + 2r$ 个位置。

载入区域	负责的局部线程 t	全局输入下标	共享数组下标
左 halo	$B-r,\dots,B-1$	$s-B+t$	$t-(B-r)$
核心区	$0,\dots,B-1$	$s+t$	$r+t$
右 halo	$0,\dots,r-1$	$s+B+t$	$r+B+t$

表 8.3 策略 1 的三段搬运映射；要求 $r \le B$，并对无效全局下标填零。

在 $b=1, B=4, r=2$ 的例子中，线程 $2,3$ 把 $N[2], N[3]$ 放入共享位置 $0,1$；所有线程把 $N[4],\dots,N[7]$ 放入位置 $2,\dots,5$；线程 $0,1$ 把 $N[8], N[9]$ 放入位置 $6,7$。相同线程可以同时承担核心区和 halo 的载入，且这些工作写入不同的共享地址。

完整的三段载入核函数如下。滤波器存放在常量数组 `Mc` 中，宽度固定为 $K=5$，由 `MASK_WIDTH` 指定。块宽 B 为模板参数，启动时每块必须恰好配置 B 个线程。共享数组在核函数内声明；有符号索引与双侧范围检查保证左右边界按零填充处理。

代码 8.5 策略 1：核心区与两侧 halo 分段载入。

```
template<int B>
__global__ void convolution_1D_tiled_three_loads(
    const float* N, float* P, int Width)
{
    constexpr int radius = MASK_WIDTH / 2;
    static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
    static_assert(B >= radius, "B must cover each halo");
    __shared__ float N_ds[B + MASK_WIDTH - 1];
    int t = int(threadIdx.x);
    int s = int(blockIdx.x) * B;
    int i = s + t;
    if (t >= B - radius) {
        int q = s - B + t;
        N_ds[t - (B - radius)] =
            (0 <= q && q < Width) ? N[q] : 0.0f;
    }
    N_ds[radius + t] = (i < Width) ? N[i] : 0.0f;
    if (t < radius) {
        int q = s + B + t;
        N_ds[radius + B + t] =
            (0 <= q && q < Width) ? N[q] : 0.0f;
    }
    __syncthreads();
    float Pvalue = 0.0f;
    for (int j = 0; j < MASK_WIDTH; ++j) {
        Pvalue += N_ds[t + j] * Mc[j];
    }
    if (i < Width) P[i] = Pvalue;
}
```

为何载入后需要同步。一个线程计算时会读取其他线程写入的共享位置。屏障 `__syncthreads()` 将共同载入与后续读取分开，使参与线程在开始乘加前可见已完成的共享内存写入。三段载入本身写入互不重叠的位置，中间不需要互相消费刚写出的数据，因此一次位于载入之后的屏障即可。

输出越界不等于搬运任务消失。最后一个块中，某个线程的 i 即使超过 $W-1$，它负责的共享位置仍可能被其他有效输出使用，包括对幽灵位置写零。因此完整代码先让所有线程完成分工、经过屏障，再用 $i < W$ 保护写出。直接版入口处的提前返回不能原样搬到这段代码的载入前；具体缺失哪些位置，可在第8.6节的末尾块例子中核对。

代码采用“全部线程计算、只保护输出写入”的结构，所以末尾无有效输出的线程也会执行乘加。若将计算限制为有效输出线程，需要把这一限制放在共同载入与屏障之后，且不得跳过仍被其他线程需要的搬运任务。

8.5.2 策略 1 的连续载入写法

三段写法按“左、核心、右”分配输入。另一种写法直接把完整共享数组看成一段连续的长度 $B+2r$ 的区间：第一轮由所有线程填入前 B 个位置，第二轮由部分线程填入剩余 $2r = K-1$ 个位置。两种写法最终都满足式(8.9)，所以计算循环完全相同。

第一轮让线程 t 读取 `start = i - radius`，写入 `N_ds[t]`。第二轮令 `start += B`，由 $t < 2r$ 的线程读取该地址，写入 `N_ds[B+t]`。这里第二轮补齐的是“完整输入块的末尾”，其中可能同时包含核心区的一部分和右 halo。

例如 $B = 4, r = 2, s = 4$ 时，第一轮载入 $N[2:5]$，即左 halo 加核心区的前半部分；第二轮载入 $N[6:9]$，即核心区后半部分加右 halo。与三段写法相比，区别在于每个线程负责哪些搬运，而非最终共享数组的内容。

代码 8.6 策略 1：两轮连续载入；需要 $K-1 \le B$。

```
template<int B>
__global__ void convolution_1D_tiled_two_loads(
    const float* N, float* P, int Width)
{
    constexpr int radius = MASK_WIDTH / 2;
    static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
    static_assert(B >= MASK_WIDTH - 1, "Two loads need B >= K-1");
    __shared__ float N_ds[B + MASK_WIDTH - 1];
    int t = int(threadIdx.x);
    int i = int(blockIdx.x) * B + t;
    int start = i - radius;
    N_ds[t] = (0 <= start && start < Width)
            ? N[start] : 0.0f;
    if (t < MASK_WIDTH - 1) {
        start += B;
        N_ds[t + B] = (0 <= start && start < Width)
                    ? N[start] : 0.0f;
    }
    __syncthreads();
    float Pvalue = 0.0f;
    for (int j = 0; j < MASK_WIDTH; ++j) {
        Pvalue += N_ds[t + j] * Mc[j];
    }
    if (i < Width) P[i] = Pvalue;
}
```

两轮载入的覆盖条件。第二轮最多有 B 个线程各填一个位置，而剩余位置数为 $2r$，因此要求 $2r \le B$。条件 `MASK_WIDTH - 1 > threadIdx.x` 在这个条件下可以覆盖所有剩余位置；当 $K-1 > B$ 时，仅有两轮就无法填满共享数组。这个限制由载入分工推得，不影响卷积计算公式本身。

由同一映射导出的循环形式。需要支持更多载入轮次时，让线程 t 依次负责 $q = t, t+B, t+2B, \ldots$，直到 $q \ge B + K - 1$。每个合法共享下标都有唯一的模 B 余数，因此恰好由一个线程负责：

代码 8.7 连续载入的索引推广；替换载入阶段，之后仍需相同屏障与计算循环。

```
int s = int(blockIdx.x) * B;
for (int q = t; q < B + MASK_WIDTH - 1; q += B) {
    int g = s - radius + q;
    N_ds[q] = (0 <= g && g < Width) ? N[g] : 0.0f;
}
__syncthreads();
```

该循环将两轮载入推广到任意所需轮数。当输入块比线程块更宽时，各线程以步长 B 分摊载入任务，仍保持共享位置恰好初始化一次。

8.5.3 策略 3：核心区缓存与按需 halo 读取

策略 3 的共享数组只容纳 B 个核心输入。线程 t 将 $N[s+t]$ 放入 `N_ds[t]`，所以共享映射变为 $N_{\text{ds}}[q] = \widetilde{N}[s+q]$，$0 \le q < B$。载入完成后经过屏障，再进入卷积循环。

对输出 $i = s+t$ 的第 j 项，输入全局下标为 $g = i - r + j$。先判断它是否属于全局有效区间 $[0, W)$；有效时再判断是否属于本块核心区 $[s, s+B)$。位于核心区的输入，其共享下标为 $g - s = t - r + j$；位于 halo 的输入直接使用全局下标 g 读取。

代码 8.8 策略 3：缓存核心区，并检查全局输入范围。

```
template<int B>
__global__ void convolution_1D_core_shared(
    const float* N, float* P, int Width)
{
    constexpr int radius = MASK_WIDTH / 2;
    static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
    __shared__ float N_ds[B];
    int t = int(threadIdx.x);
    int s = int(blockIdx.x) * B;
    int i = s + t;
    N_ds[t] = (i < Width) ? N[i] : 0.0f;
    __syncthreads();
    float Pvalue = 0.0f;
    for (int j = 0; j < MASK_WIDTH; ++j) {
        int N_index = i - radius + j;
        if (0 <= N_index && N_index < Width) {
            if (s <= N_index && N_index < s + B) {
                Pvalue += N_ds[t - radius + j] * Mc[j];
            } else {
                Pvalue += N[N_index] * Mc[j];
            }
        }
    }
    if (i < Width) P[i] = Pvalue;
}
```

共享下标安全性。进入共享读取分支时，已有 $s \le g < s+B$，因此 $0 \le g - s < B$，等价于 $0 \le t - r + j < B$。尽管表达式中含有减去半径，分支条件保证实际读取时下标合法。外层条件负责全局数组边界，内层条件负责选择数据来源，两者不能相互替代。

以 $s = 4, B = 4, r = 2$ 的线程 $t = 0$ 为例，它计算 $P[4]$，依次读取 $N[2]$ 到 $N[6]$。前两项不属于核心区，从全局地址读取；后三项属于核心区，从共享位置 $0, 1, 2$ 读取。与此同时，计算 $P[6]$ 的线程可能在相同的循环迭代已经使用共享内存，因而同一 warp 内可能出现不同读取分支。

两种共享数组起点决定两种计算下标

策略 1 的共享位置 0 对应全局位置 $s-r$，因此读取下标为 $t+j$。策略 3 的共享位置 0 对应全局位置 s，因此读取下标为 $t-r+j$，且仅在输入属于核心区时使用。这一差异完全来自共享数组表示的输入范围。

核心区载入同样需要检查边界。`(i < Width) ? N[i] : 0.0f` 在第一次全局读取之前处理了末尾块；仅在最后给 `P[i]` 加条件，并不能保护此前已经执行的 `N[i]` 读取。策略 3 与策略 1 都要完成共同载入和屏障，随后才进入各自的计算路径。

8.6 综合算例与关键结论

8.6.1 末尾不足一个完整块时的计算

沿用一维输入和滤波器，逐项分析末尾块的载入与计算。取 $W=7$、$B=4$、$K=5$、$r=2$，输入 $N=[1,2,3,4,5,6,7]$，滤波器 $M=[3,4,5,4,3]$。网格需要两块，分别负责输出下标 0—3 与 4—7，其中 $P[7]$ 没有对应的有效输出位置。

第二块从 $s=4$ 开始。策略 1 的共享数组覆盖全局位置 2 到 9，依次填入 $[3,4,5,6,7,0,0,0]$。前五个值来自真实输入，后三个值按零填充规则生成。三段载入中，线程 $t=3$ 虽然对应无效输出 $i=7$，仍负责把 $N[3]=4$ 放入共享位置 1，并把其核心位置写为零。

如果线程 $t=3$ 在搬运前提前返回，共享位置 1 就没有被它初始化。有效线程 $t=0$ 计算 $P[4]$ 时需要共享位置 0 到 4，其中就包含这一位置。因此有效输出会依赖未完成的搬运；只查看“哪些线程要写输出”无法判断哪些线程可以提前结束。

局部线程	输出位置	策略 1 读取的共享值	结果与写入
$t=0$	$P[4]$	$[3,4,5,6,7]$	$9+16+25+24+21=95$
$t=1$	$P[5]$	$[4,5,6,7,0]$	$12+20+30+28=90$
$t=2$	$P[6]$	$[5,6,7,0,0]$	$15+24+35=74$
$t=3$	无有效输出	$[6,7,0,0,0]$	可执行乘加，但不写 $P[7]$

表 8.4 末尾块的完整窗口；零值也占据明确的共享位置。

两轮载入写法中，第二块第一轮载入 $[N[2],N[3],N[4],N[5]]$，第二轮载入 $[N[6],0,0,0]$，最终共享内容与三段写法相同。在不同写法中负责同一数据的线程可能不同，但只要式(8.9)成立，乘加结果就一致。

对策略 3，同一块的共享内容只有 $[N[4],N[5],N[6],0]=[5,6,7,0]$。计算 $P[4]$ 时从全局地址取得 $N[2],N[3]$，从共享数组取得 $N[4],N[5],N[6]$。不同数据放置方式由不同下标转换

衔接到同一个卷积窗口。

8.6.2 带边界条件的精确计数

对直接核函数，位置 i 的有效乘加项数就是窗口 $[i-r, i+r]$ 与数组 $[0, W-1]$ 的交集长度。将边界判断写成计数式，可得

$$v_i = \min(W-1, i+r) - \max(0, i-r) + 1, \qquad 0 \le i < W. \tag{8.10}$$

对于 $W=7, r=2$，v_i 依次为 $3,4,5,5,5,4,3$，总计 29。直接核函数每个有效项读一个输入和一个系数，因此一共执行 58 次标量读取，对应 232 B 的逻辑读入；有效输出写入 7 个 `float`，对应 28 B；乘法加法合计为 58 FLOPs。这些数字来自代码实际通过边界分支的项数。

若只比较策略 1 对输入 N 的请求，首块的输入覆盖 $[-2, 5]$，其中六个位置有效；第二块覆盖 $[2, 9]$，其中五个位置有效，共读入 11 个真实输入值，即 44 B。共享内存中总共写入 $2 \times 8 = 16$ 个位置，其余五个位置由线程直接写零。常量系数的读取和缓存事务没有包含在这 44 B 中。

完整输入块已在载入阶段填零，策略 1 的乘加循环就不再跳过零项。七个有效输出各执行五次乘加，需要 70 FLOPs；代码8.5与代码8.6采用全部线程计算的结构，八个已启动线程共执行 80 FLOPs，最后一个线程的结果不写出。**减少输入请求可以同时增加零项或无效输出上的算术工作**，这正是按计数对象分别核对的意义。

再考虑 $W=16, B=4, K=5$ 的布局。四个块的有效输入请求数依次为 $6,8,8,6$，策略 1 合计请求 28 个输入值。直接核函数的有效输入使用次数为 $3+4+12\times5+4+3=74$。该例按整个网格统计的输入请求减少因子为 $74/28 \approx 2.64$，而式(8.8)的 2.5 比较的是一个远离边界的完整内部块。两者采用了不同的统计范围。

8.6.3 关键公式与实现关系

卷积计算、缓存与分块可以通过以下关系一起核对。公式中的 $K = 2r + 1$，B 表示每块输出数，$s = bB$ 表示输出块起点；策略 2 若按输入块大小配置线程，线程数应相应增加为 $B + 2r$。

对象	关系
输出与输入窗口	$P[i] = \sum_{j=0}^{K-1} \widetilde{N}[i - r + j]M[j]$
一块所需输入数	$T_{\text{in}} = B + 2r = B + K - 1$
完整输入块的共享映射	$N_{\text{ds}}[q] = \widetilde{N}[s - r + q]$
策略 1 的计算下标	$q = t + j$
核心区的共享映射	$N_{\text{ds}}[q] = \widetilde{N}[s + q]$
策略 3 的共享计算下标	$q = t - r + j$，仅当 $s \le i - r + j < s + B$
内部块输入请求比较	直接法 BK；策略 1 为 $B + 2r$
策略 1 的线程覆盖条件	三段载入要求 $r \le B$；两轮载入要求 $2r \le B$
共享内存容量	策略 1、2 为 $4(B + 2r)$ B；策略 3 为 $4B$ B

表 8.5 卷积窗口、共享索引与资源用量。

从输出窗口出发确定完整输入范围，再选择共享数组的起点，即可推得载入与计算下标。常量存储器处理统一系数的读取，共享内存处理邻域输入的复用；二者连接到同一个卷积计算，而优化的对象各不相同。

第 9 讲

2D Tiled Convolution Kernel; Reuse Analysis

9.1 核心概念与分块策略

9.1.1 卷积与模板计算

模板算法（stencil algorithm）按照固定的邻域模式更新数组元素。邻域规定一个输出依赖哪些输入；计算规则规定这些输入如何组合。二维卷积是其中一种：每个输出由一个二维邻域内的输入与相应权重相乘，再求和得到。其他模板可以只使用少量特定方向的邻点，例如二维五点模板、三维十三点有限差分模板，以及四维 Wilson–Dslash 模板。分块所利用的共同性质，是相邻输出的输入邻域存在重叠。

设输入数组为 N，输出数组为 P，两者均有 H 行、W 列；权重矩阵为 M，边长为奇数 m，半径为 $r = (m-1)/2$。用 $\widetilde{N}$ 表示在数组范围外补零后的输入。为明确后续代码的下标关系，按其权重访问顺序写出计算式：

$$P[y,x] = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} M[i,j]\,\widetilde{N}[y-r+i,\,x-r+j]. \tag{9.1}$$

这里沿用权重按 $M[i,j]$ 直接访问的约定；公式的用途是说明计算邻域与数组下标。权重数组在代码中记为 `Mc`。例如 $m = 5$ 时，每个内部输出需要一个 5×5 输入窗口，共进行 25 次乘法和 25 次累加。

输出分块（output tile）是一组由同一线程块负责生成的相邻输出。输入分块（input tile）则包含计算这些输出所需的全部输入，其中超出输出分块对应核心区域的部分称为光环区（halo）。落到整个输入数组之外、由边界规则补出的值称为虚拟边界元素（ghost element）。光环描述块与块之间的邻域关系；虚拟边界元素描述整个数组以外的位置。内部块也有光环，但其光环通常仍是有效数组元素。

本讲的学习目标是理解分块卷积算法中的输入分块与输出分块，进一步分析卷积和模板计算中的数据复用，并为 Lab 4 做准备。

表 9.1 二维方形分块的统一记号

记号	含义	代码中的对应量
H, W	输入和输出的行数、列数	`Height`, `Width`
m, r	权重边长及半径，$m = 2r + 1$	`MASK_WIDTH`, `RADIUS`
T	输出分块的边长	`TILE_WIDTH`
S	输入分块的边长，$S = T + m - 1$	`INPUT_WIDTH`
t_x, t_y	线程块内的二维线程坐标	`threadIdx.x`, `threadIdx.y`
b_x, b_y	网格内的二维线程块坐标	`blockIdx.x`, `blockIdx.y`

9.1.2 三种分块策略

线程可以围绕输出计算分工，也可以围绕输入加载分工。输出分工使一个线程自然地对应一个输出元素；加载分工则使一个线程对应一个需要装入共享内存的输入元素。由于输入分块通常更大，这两种分工对应不同的线程块大小。

策略 1：线程块覆盖输出分块。用 T^2 个线程生成 T^2 个输出，但需要准备 S^2 个输入。因此部分线程要通过多次加载搬入光环元素。所有线程在计算阶段都可以承担输出，加载阶段则需要额外的索引安排。

策略 2：线程块覆盖输入分块。用 S^2 个线程分别写入共享内存中的一个位置，再由其中的 T^2 个线程计算输出。加载工作统一，计算阶段则有一部分线程不再执行乘加循环。这是后续二维内核的主要组织方式。

策略 3：共享内存只保存核心区。用 T^2 个线程加载与输出分块同样大小的核心输入区域。计算邻域落在核心区内时读取共享内存；落在光环区时读取全局内存。因此乘加循环内部需要判断数据位置，且光环仍可能产生重复的全局读取。

策略 2 的取舍是把更整齐的工作分配用于高延迟的输入加载，并把线程参与程度的差异留给乘加计算。它避免为了补光环而单独组织很窄的一段访问。这里“加载一致”针对块内各线程的职责；靠近整幅图像边界时，有效输入与补零位置仍需分别处理。

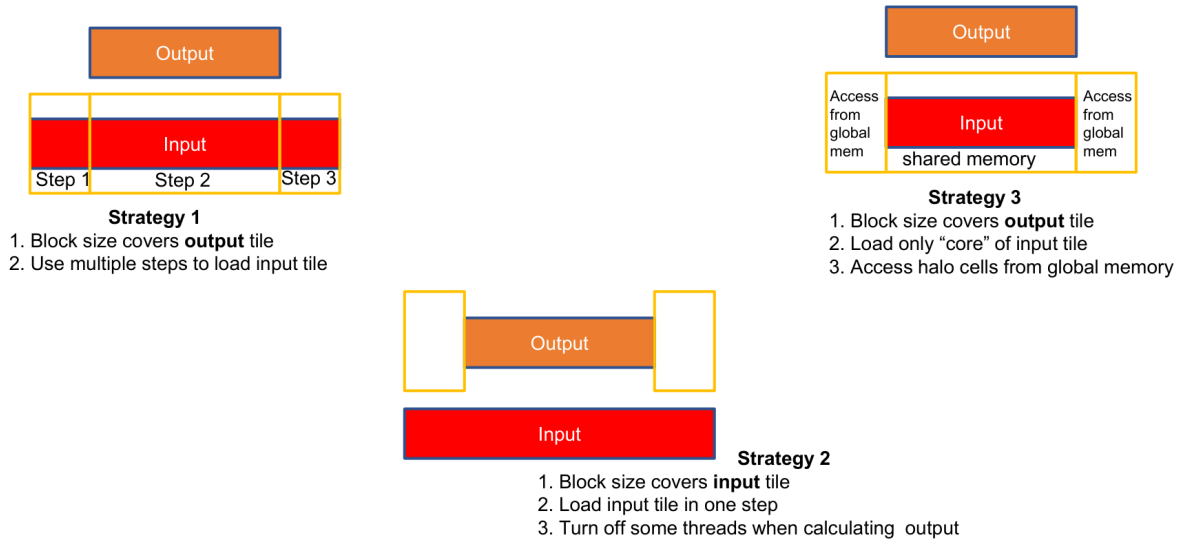


图 9.1 三种策略的加载范围与线程分工。策略 1 分多步装入完整输入，策略 2 为完整输入分配线程，策略 3 仅将核心区置于共享内存。

9.1.3 课程事项

表 9.2 2026 年 9 月 22 日公布的课程安排

事项	安排
实验与项目	Lab 3 于当周星期五截止；项目里程碑 1 于当周发布。CNN 为个人项目，GPT-2 为 3–4 人团队项目。
第一次期中考试	10 月 6 日 19:00–22:00，纸笔考试；范围为 Lectures 1–11 和 Labs 1–4。具体教室由 Canvas 公布。
考试时间冲突	有有效时间冲突者，须在 9 月 30 日前发邮件联系教师，另行安排考试时间。
复习与阅读	复习资料已在 Canvas 发布；阅读教材第 7 章，并提交 Lab 3。

9.2 二维分块卷积的线程组织与实现

9.2.1 输入范围与网格步长

一个边长为 T 的输出分块，在每个方向上都需要额外的左右或上下邻域。以列方向为例，输出列范围为 b_xT 到 b_xT+T-1；最左侧输出需要向左延伸 r 列，最右侧输出需要向右延伸 r 列，因此输入列范围为 b_xT-r 到 $b_xT+T-1+r$。含首尾两端，共有

$$S=(b_xT+T-1+r)-(b_xT-r)+1=T+2r=T+m-1 \tag{9.2}$$

列。行方向完全相同，故完整输入分块是 $S\times S$。当 $m=5$ 时，每侧增加 2 个元素，输入边长为 $T+4$。

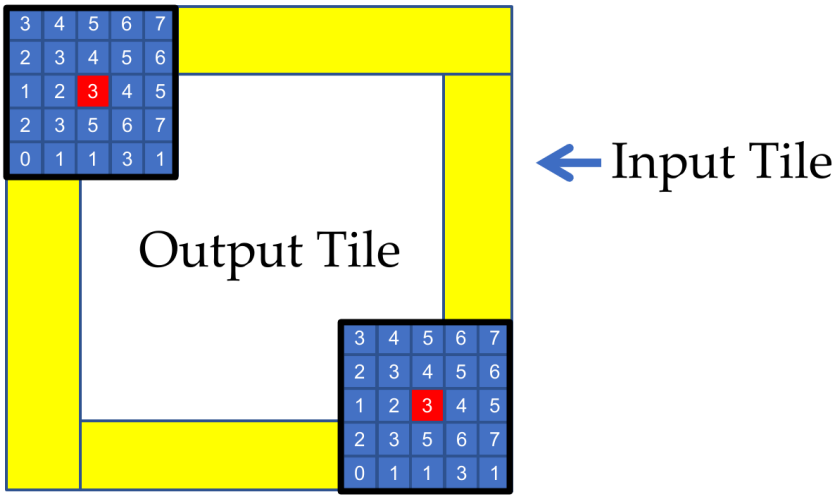


图 9.2 计算输出分块两个相对角点时，其输入窗口分别延伸到左上与右下的光环区域。白色输出区域外的黄色边框也是输入分块的一部分。

策略 2 的线程块大小为 $(S,S,1)$，每个线程负责输入分块中的一个位置。网格大小则由输出覆盖范围决定：

$$\text{gridDim.x}=\left\lceil\frac{W}{T}\right\rceil,\qquad \text{gridDim.y}=\left\lceil\frac{H}{T}\right\rceil. \tag{9.3}$$

相邻块的输出起点相差 T，恰好使输出分块相接；输入起点也相差 T，因此相邻输入分块重叠 $S-T=m-1$ 列或行。若用 S 作为输出步长，两个相邻输出分块之间会留下宽度为 $m-1$ 的缺口。

例如 $H=37, W=53, T=16, m=5$，应建立 4×3 个线程块，每块有 $20\times 20=400$ 个线程。右下角块的输出起点为 $(32,48)$，只有 5×5 个输出落在图像内，但该块仍按完整的输入分块组织加载与同步。

9.2.2 三组坐标的对应关系

线程局部坐标 (t_y, t_x) 同时参与输出坐标与输入加载坐标的构造，两者之间相差半径 r：

$$\begin{aligned} y_o &= b_y T + t_y, \quad x_o = b_x T + t_x, \\ y_i &= y_o - r, \qquad x_i = x_o - r. \end{aligned} \tag{9.4}$$

每个线程把 $N[y_i, x_i]$ 放入共享内存的 `tile[ty][tx]`。当 $t_y < T$ 且 $t_x < T$ 时，它还负责输出 $P[y_o, x_o]$。其余线程虽然可以算出形式上的 (y_o, x_o)，但这些坐标并不代表本块实际负责的输出。

共享内存中的位置 (s_y, s_x) 对应的全局输入坐标为

$$(b_y T - r + s_y,\ b_x T - r + s_x). \tag{9.5}$$

将 $s_y = t_y + i, s_x = t_x + j$ 代入，可得 $\text{tile}[t_y + i][t_x + j]$ 恰好对应 $\widetilde{N}[y_o - r + i, x_o - r + j]$。于是卷积窗口可直接从共享内存局部坐标 (t_y, t_x) 开始访问，不再在内层循环里减半径。

加载的元素与输出的中心

线程 (t_y, t_x) 加载的是其输出窗口的左上角输入。输出中心对应的输入位于共享内存 $(t_y + r, t_x + r)$，通常由另一个线程加载。因此参加输出计算的是线程坐标中的左上 $T \times T$ 区域；无需把计算条件改成 $r \le t_x, t_y < r + T$。

取 $T = 8, m = 5$ 和块 $(b_x, b_y) = (1, 2)$。输入分块覆盖行 14–25、列 6–17；输出分块覆盖行 16–23、列 8–15。线程 $(t_y, t_x) = (0, 0)$ 加载 $N[14, 6]$，计算 $P[16, 8]$；线程 $(7, 7)$ 加载 $N[21, 13]$，计算 $P[23, 15]$；线程 $(11, 11)$ 加载 $N[25, 17]$，只参与加载。最后这个输入正是右下角输出窗口需要的元素。

9.2.3 协作加载、补零与同步

所有 S^2 个线程都向共享内存写入一个值。若 $0 \le y_i < H$ 且 $0 \le x_i < W$，就读取有效输入；否则写入 `0.0f`。这样，共享内存中的整个输入分块都已初始化，乘加循环可以统一访问一个完整窗口。

输入越界与输出越界要分别判断。左上角线程可能加载一个负坐标位置，却仍要计算有效输出。例如第一块的线程 $(0, 0)$ 加载输入坐标 $(-r, -r)$，需要写入零，但它负责的输出坐标是 $(0, 0)$。反过来，右边缘某个线程的输出坐标可能超过 $W - 1$，而其左移 r 后的输入坐标仍

然有效，且可能被附近的有效输出使用。

加载完成后执行 `__syncthreads()`。其位置必须在所有线程的加载或补零之后，在任何线程读取他人写入的共享数据之前。由于一个输出窗口包含多个线程写入的元素，单个线程完成自己的写入，并不表示整个窗口都已经准备好。

两个判断、一个公共同步点

先根据输入坐标决定“读取输入”或“写入零”，随后让整个线程块经过同步点，最后根据线程角色及输出边界决定是否计算和写回。不能在同步之前仅因某个线程没有有效输出而提前退出，否则既可能漏掉其他线程需要的加载，也会破坏这一统一的协作结构。

9.2.4 输出计算与写回条件

参加计算的线程满足 $t_x < T$ 且 $t_y < T$。它将累加器初始化为零，执行 $m \times m$ 次乘加，然后只在 $y_o < H$ 且 $x_o < W$ 时写回。行优先存储（row-major storage）中，二维位置 (y, x) 的线性偏移是 $yW + x$。

共享内存访问的最大行下标为 $(T-1)+(m-1) = S-1$，列下标也相同。因此，只要加载分块确实分配为 $S \times S$，上述计算范围与共享内存范围就一致。这里的边界检查分别承担不同任务：线程坐标限制本块拥有的输出，输出坐标限制整幅图像内的写入位置。

矩形图像的边界条件。行坐标应与高度比较，即 `row_o < Height`；列坐标应与宽度比较，即 `col_o < Width`。混用高度与宽度会使矩形图像出现漏写或越界。权重边长由编译期常量 `MASK_WIDTH` 指定，两个循环均按这一边长遍历。

9.2.5 二维策略 3 的对应变化

策略 3 将共享内存缩为 $T \times T$，线程 (t_y, t_x) 加载其输出中心对应的核心输入。此时权重位置 (i, j) 需要访问的核心区局部坐标为 $(t_y - r + i, t_x - r + j)$。若两个局部坐标均落在 $[0, T)$，从共享内存读取；否则根据全局坐标读取光环，越过整幅图像边界时按零处理。

因此，策略 2 在加载时一次性解决输入分块边界，乘加循环统一读取共享内存；策略 3 在每次乘加时判断目标属于核心区还是光环区。后者不需要仅负责光环加载的线程，但同一线程束（warp）中的线程可能在内层循环选择不同的读取分支。这一差异同时影响全局访问次数和控制流，后续复用公式要与具体的加载方式对应。

9.2.6 完整内核与主机启动

下面将加载、同步、计算与写回连成一个完整内核。示例取 $T = 16, m = 5$，每块 400 个线程，分配 400 个单精度共享内存元素；其中 256 个线程具有输出角色。Mc 在启动前由主机填入，输入与输出使用互不重叠的设备缓冲区。

代码 9.1 二维策略 2：完整输入分块的协作加载与输出计算

```
#pragma once
#include <cuda_runtime.h>

constexpr int TILE_WIDTH = 16;
constexpr int MASK_WIDTH = 5;
constexpr int RADIUS = MASK_WIDTH / 2;
constexpr int INPUT_WIDTH = TILE_WIDTH + MASK_WIDTH - 1;
static_assert(TILE_WIDTH > 0 && MASK_WIDTH % 2 == 1);

__constant__ float Mc[MASK_WIDTH][MASK_WIDTH];

__global__ void convolution2DStrategy2(
    const float* N, float* P, int Height, int Width)
{
    __shared__ float tile[INPUT_WIDTH][INPUT_WIDTH];
    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int row_o = int(blockIdx.y) * TILE_WIDTH + ty;
    const int col_o = int(blockIdx.x) * TILE_WIDTH + tx;
    const int row_i = row_o - RADIUS;
    const int col_i = col_o - RADIUS;

    if (row_i >= 0 && row_i < Height &&
        col_i >= 0 && col_i < Width) {
        tile[ty][tx] = N[size_t(row_i) * Width + col_i];
    } else {
        tile[ty][tx] = 0.0f;
    }
    __syncthreads();

    if (ty < TILE_WIDTH && tx < TILE_WIDTH) {
        float Pvalue = 0.0f;
        for (int i = 0; i < MASK_WIDTH; ++i) {
            for (int j = 0; j < MASK_WIDTH; ++j) {
                Pvalue += Mc[i][j] * tile[ty + i][tx + j];
            }
        }
        if (row_o < Height && col_o < Width) {
            P[size_t(row_o) * Width + col_o] = Pvalue;
        }
    }
}
```

主机端的关键区别仍是“块尺寸用 S，网格覆盖用 T”。以下接口负责权重复制、参数检查和内核启动，并返回相应错误状态。h_Mc 指向行优先的 $m \times m$ 主机权重数组，d_N 与 d_P 已分别分配至少 HW 个单精度元素。

代码 9.2 主机端的启动接口与设备资源检查

```
#include "convolution2d_strategy2.cuh"

cudaError_t launchConvolution(const float* d_N, float* d_P,
                              const float* h_Mc,
                              int Height, int Width)
{
    if (!d_N || !d_P || !h_Mc || d_N == d_P ||
        Height <= 0 || Width <= 0)
        return cudaErrorInvalidValue;

    int device = 0;
    cudaError_t err = cudaGetDevice(&device);
    if (err != cudaSuccess) return err;
    cudaDeviceProp prop;
    err = cudaGetDeviceProperties(&prop, device);
    if (err != cudaSuccess) return err;
    if (INPUT_WIDTH * INPUT_WIDTH > prop.maxThreadsPerBlock ||
        INPUT_WIDTH > prop.maxThreadsDim[0] ||
        INPUT_WIDTH > prop.maxThreadsDim[1] ||
        sizeof(float) * INPUT_WIDTH * INPUT_WIDTH >
            prop.sharedMemPerBlock)
        return cudaErrorInvalidConfiguration;

    err = cudaMemcpyToSymbol(Mc, h_Mc, sizeof(Mc));
    if (err != cudaSuccess) return err;
    dim3 block(INPUT_WIDTH, INPUT_WIDTH, 1);
    dim3 grid((Width - 1) / TILE_WIDTH + 1,
              (Height - 1) / TILE_WIDTH + 1, 1);
    if (grid.x > unsigned(prop.maxGridSize[0]) ||
        grid.y > unsigned(prop.maxGridSize[1]))
        return cudaErrorInvalidConfiguration;
    convolution2DStrategy2<<<grid, block>>>(d_N, d_P, Height, Width);
    return cudaGetLastError();
}
```

此接口返回的是参数检查、权重复制或内核启动阶段的错误。调用方需要在取得结果前完成相应的设备同步或数据传回，并处理执行阶段可能返回的错误。代码中的资源检查也说明，S 不能仅由复用公式任意增大：线程块还必须满足实际设备的单块线程数、各维大小及共享内存容量限制。

9.3 数据复用与全局访存计数

9.3.1 复用因子的计数口径

相邻输出反复使用同一个输入元素。直接实现每次需要它时都从全局内存读取；完整分块实现先把它读入共享内存，再为多个输出提供数据。设直接实现对输入数组的读取次数为 A，分块后为准备这些输出而需要加载的不同有效输入数为 B，定义

$$R = \frac{A}{B}. \tag{9.6}$$

R 是输入全局读取量的缩减倍数，也是一次输入加载平均支持的有效使用次数。若 $R = 4$，分块后的这部分全局读取量是原来的四分之一。权重读取、输出写回以及访问被硬件合并或缓存后的物理事务量，不包含在这一计数口径中。

分块几何给出的是可利用的复用量。对于把完整输入分块逐元素装入共享内存的策略 1 和策略 2，B 可以直接取有效输入元素数；对于只保存核心区的策略 3，还要计入计算期间发生的光环全局读取。因而同一个几何复用公式，不能直接作为三种实现实际全局流量都相等的依据。

9.3.2 一维内部块：按输出与按输入两种计数

9.3.2.1 从输出需要多少输入开始

取 $T = 8, m = 5$，一个内部输出块为 $P[8]$ 到 $P[15]$。$P[8]$ 需要 $N[6]$ 到 $N[10]$，$P[9]$ 需要 $N[7]$ 到 $N[11]$，依次移动后，$P[15]$ 需要 $N[13]$ 到 $N[17]$。全部输出总共使用 $8 \times 5 = 40$ 个输入值，但不同的输入元素只有 $N[6]$ 到 $N[17]$ 这 12 个。

因此，40 次直接全局读取可由 12 次输入加载加共享内存读取完成，$R = 40/12 = 10/3 \approx 3.3$。若每个输入为 4 字节，这部分全局读入数据由 160 字节降至 48 字节。对于一般的内部块，输出共有 T 个，每个需要 m 个输入，不同输入数为 $T + m - 1$，于是

$$R_{1D}(T, m) = \frac{Tm}{T + m - 1}. \tag{9.7}$$

只要块完全位于输入内部，该比值由 T 和 m 决定，与整条输入数组的总长度无关。

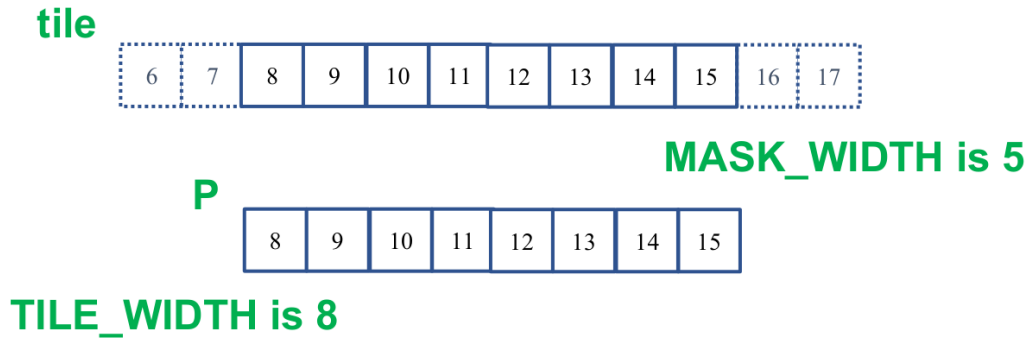


图 9.3 8 个连续输出对应 12 个不同输入。左右各两个光环元素保证最外侧输出也能获得完整的五点窗口。

9.3.2.2 从每个输入被使用几次开始

同一例子也可以按输入计数。$N[6]$ 仅供 $P[8]$ 使用，$N[7]$ 供两个输出使用，靠近中间的输入被五个输出使用，另一侧再逐渐减少。按 $N[6]$ 到 $N[17]$ 的顺序，使用次数为

$$1,\ 2,\ 3,\ 4,\ 5,\ 5,\ 5,\ 5,\ 4,\ 3,\ 2,\ 1.$$

这些次数之和仍为 40。光环附近的低复用输入拉低了平均值，所以有限分块的平均复用小于单个中央输入能达到的五次使用。

当 $T \geq m$ 时，两侧各形成一个从 1 到 $m-1$ 的阶梯，中间有 $T-m+1$ 个输入各使用 m 次。由此得到

$$\begin{aligned} A &= 2\sum_{k=1}^{m-1} k + m(T-m+1) \\ &= m(m-1) + m(T-m+1) = Tm. \end{aligned} \tag{9.8}$$

这是对式(9.7)分子的另一种验证。阶梯加平台的这个具体展开采用 $T \geq m$；即使 $T < m$，按输出计数所得的 $A = Tm$ 仍然成立。

表 9.3 一维内部块的输入读取缩减倍数

输出边长 T	16	32	64	128	256
$m = 5$	4.0	4.4	4.7	4.9	4.9
$m = 9$	6.0	7.2	8.0	8.5	8.7

将分母写成 $T[1 + (m-1)/T]$，可看出 $R_{1D} = m/[1 + (m-1)/T]$。固定 m 并增大 T，光环相对于核心区的开销下降，复用逐渐接近 m；继续扩大分块的收益也随之减小。

9.3.3 一维边界块：虚拟元素不产生全局读取

考虑最左侧输出块 $P[0]$ 到 $P[7]$，仍取 $m=5, r=2$，并假定右侧有足够多的有效输入。共享内存的 12 个位置对应输入坐标 -2 到 9；前两个位置写入零，实际从全局内存读取的只有 $N[0]$ 到 $N[9]$，共 10 个元素。

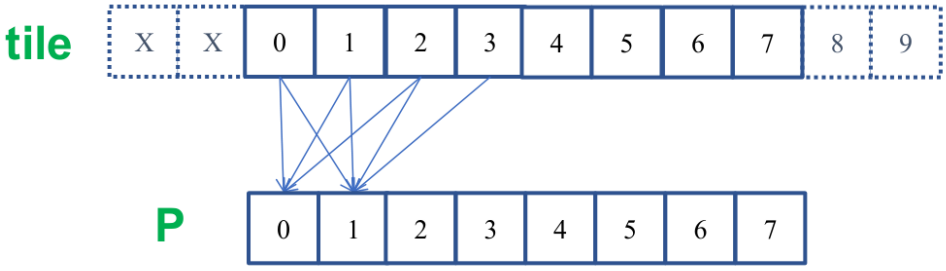


图 9.4 左边界输入分块中的两个虚拟位置。它们占用共享内存位置，但不需要从输入数组加载。

直接实现计算 $P[0]$ 时只需读取三个有效输入，计算 $P[1]$ 时读取四个，其余六个输出各读取五个。分子和分母都应只计有效的全局读取，因此

$$A = 3 + 4 + 6 \times 5 = 37, \qquad B = 8 + 2 = 10, \qquad R_{\mathrm{1D},edge} = \frac{37}{10} = 3.7. \tag{9.9}$$

若乘加循环仍按五个位置执行，分块内核仍可读取 40 次共享内存，只是其中三次读取的是已补入的零。这与“直接实现有 37 次有效全局读取”采用不同的计数对象。

为便于推广，对一侧接触边界、另一侧不接触边界的完整输出块进行形式化计数。设 $T \geq r$ 且输入至少含有 $T + r$ 个元素，最靠边的 r 个输出分别缺少 $r, r-1, \ldots, 1$ 个有效输入，故

$$R_{\mathrm{1D},edge} = \frac{Tm - r(r+1)/2}{T + r}. \tag{9.10}$$

边界例子的比值高于内部例子的 40/12，原因是低复用的最外侧输入变成了零，既减少直接读取，也减少分块加载。由此得到的是计数比例变化；实际执行还包含补零、边界分支等开销。

9.3.4 二维内部块：两个方向的复用相乘

二维输出块有 T^2 个输出，每个输出使用 m^2 个输入，直接实现需要 T^2m^2 次输入读取。不同输入组成边长为 $T+m-1$ 的完整方形区域，只需加载 $(T+m-1)^2$ 个元素。因此

$$R_{\mathrm{2D}}(T,m)=\frac{T^2m^2}{(T+m-1)^2}=\left(\frac{Tm}{T+m-1}\right)^2. \tag{9.11}$$

这个平方来自行、列两个方向的几何计数；权重仍可以是任意的稠密 $m\times m$ 矩阵。

当 $T=8, m=5$ 时，输入分块为 12×12，只需加载 144 个输入；64 个输出共使用 $64\times25=1600$ 个输入值。于是 $R=1600/144\approx11.1$，对应的输入全局读入字节数由 6400 降至 576。这里输出分块大小是 8×8，输入分块大小是 12×12，二者在分子与分母中承担不同角色。

表 9.4 二维内部块的输入读取缩减倍数

输出分块	8×8	16×16	32×32	64×64
$m=5$	11.1	16.0	19.7	22.1
$m=9$	20.3	36.0	51.8	64.0

固定 m 时，R_{2D} 随 T 增大而趋近 m^2。较大的权重窗口提供更大的潜在复用，但也扩大光环和计算量。表中的分块大小用于几何分析；能否按“一线程加载一个输入”的策略 2 直接启动，还要检查 $(T+m-1)^2$ 个线程是否满足设备限制。

9.3.5 二维边界的乘积计数

二维边界也可先分别统计行、列方向，再相乘。假设一个完整输出块仅在某一侧接触图像边界，且相反侧仍有足够多的输入。边界方向的直接读取计数为 $Tm-r(r+1)/2$，不同输入数为 $T+r$；内部方向则分别为 Tm 和 $T+2r$。所以

$$\begin{aligned} R_{\mathrm{2D},edge}&=R_{\mathrm{1D},edge}\,R_{\mathrm{1D}},\\ R_{\mathrm{2D},corner}&=R_{\mathrm{1D},edge}^2. \end{aligned} \tag{9.12}$$

角块的两个方向都靠边，边块只有一个方向靠边。以 $T=8, m=5$ 为例，三个位置的计数如下。

表 9.5 同一分块大小在内部、单边与角落处的计数差异

位置	直接读取 A	不同有效输入 B	缩减倍数 R
内部块	$40^2 = 1600$	$12^2 = 144$	11.11
单边块	$37 \times 40 = 1480$	$10 \times 12 = 120$	12.33
角块	$37^2 = 1369$	$10^2 = 100$	13.69

9.3.5.1 非整块与较小图像的统一写法

当最后一块只有部分有效输出，或图像小到一个分块同时接触两侧边界时，可以直接从有效区间计数。对长度为 D 的一维数组，位置 q 所需的有效邻域长度为

$$\ell_D(q) = \min(D-1, q+r) - \max(0, q-r) + 1, \quad 0 \le q < D. \tag{9.13}$$

有效输出区间为 $[a,b]$ 时，其输入并集长度是 $U_D(a,b) = \min(D-1, b+r) - \max(0, a-r) + 1$。设二维块的有效输出行区间为 $[y_0, y_1]$，列区间为 $[x_0, x_1]$，则

$$\begin{aligned} A &= \left(\sum_{y=y_0}^{y_1} \ell_H(y) \right) \left(\sum_{x=x_0}^{x_1} \ell_W(x) \right), \\ B &= U_H(y_0, y_1)\, U_W(x_0, x_1). \end{aligned} \tag{9.14}$$

这是把每个输出的有效输入数 $\ell_H(y)\ell_W(x)$ 对行列求和后分离所得的计数恒等式。它还说明，应先确定有效输出集合，再统计所需输入；若内核为越界候选输出仍执行乘加，这部分无效算术需要另外记录。

9.4 带宽、算术强度与计算上限

9.4.1 未分块卷积的字节 / 浮点操作比

一次卷积乘加读取一个单精度输入，产生一次乘法与一次加法，按两次浮点操作（floating-point operation，FLOP）计。输入的 4 字节来自全局内存；权重的 4 字节按常量缓存提供。只统计输入的全局读入量时，每次乘加对应

$$b_0 = \frac{4\,\mathrm{B}}{2\,\mathrm{FLOP}} = 2\,\mathrm{B/FLOP}. \tag{9.15}$$

b 表示字节 / 浮点操作比，数值越大，每完成一定算术工作所需的全局数据越多。它的倒数称为算术强度（arithmetic intensity），记为 $I = 1/b$，单位为 FLOP/B；未分块的基准值是 $I_0 = 0.5\,\mathrm{FLOP/B}$。

对于内部块，若每次输入加载平均支持 R 次使用，算术工作量不变，输入全局读取量降为原来的 $1/R$，所以

$$b_{\mathrm{tiled}} = \frac{b_0}{R} = \frac{2}{R}\,\mathrm{B/FLOP}, \qquad I_{\mathrm{tiled}} = \frac{R}{2}\,\mathrm{FLOP/B}. \tag{9.16}$$

这些式子连接了几何复用与带宽需求。后续峰值分析沿用这一口径：单精度输入、内部有效输出、权重按常量缓存供给，暂不计输出写回。

9.4.2 由内存带宽得到吞吐上限

设内存带宽为 $\mathcal{B}$，单位为字节每秒；计算峰值为 C_{peak}，单位为 FLOP 每秒。若每完成一 FLOP 需要 b 字节输入，则带宽最多支持 $\mathcal{B}/b$ FLOP/s。结合算术峰值，模型给出的上限为

$$C_{\mathrm{bound}} = \min\left(C_{\mathrm{peak}}, \frac{\mathcal{B}}{b}\right) = \min\left(C_{\mathrm{peak}}, \mathcal{B}I\right). \tag{9.17}$$

当 $\mathcal{B}/b < C_{\mathrm{peak}}$ 时，内存供给先达到上限；提高数据复用能增加带宽允许的算术吞吐。要使带宽至少能够供给峰值算术，需要

$$b \le b_{\mathrm{balance}} := \frac{\mathcal{B}}{C_{\mathrm{peak}}}, \qquad R \ge R_{\min} := \frac{b_0 C_{\mathrm{peak}}}{\mathcal{B}}. \tag{9.18}$$

$R_{\min}$ 是这一输入流量模型中的必要复用门槛。到达门槛后，线程调度、同步、共享内存访问及其他执行开销仍会影响实际吞吐。

以 $C_{\text{peak}} = 1000\,\text{GFLOP/s}$、$\mathcal{B} = 150\,\text{GB/s}$ 为例，未分块时带宽上限是 $150/2 = 75\,\text{GFLOP/s}$，仅为峰值的 7.5%。所需复用为 $1000/75 \approx 13.3$。这里 GB 与 GFLOP 都采用十进制数量级，因此相除时可以直接约去 10^9。

9.4.3 不同参数下的复用门槛

将下面三组硬件算例参数代入同一公式，可以比较计算能力增长与内存带宽增长之间的差异。峰值相对于带宽越大，每加载一个字节需要承担的算术工作就越多。

表 9.6 只计输入读取时的带宽上限与复用门槛

算例	C_{peak}	$\mathcal{B}$	未分块上限	峰值比例	$R_{\min}$
约 2010 年 GPU	1000 GFLOP/s	150 GB/s	75 GFLOP/s	7.50%	13.3
2013 年 GRID K520	5000 GFLOP/s	192 GB/s	96 GFLOP/s	1.92%	52.1
H100 PCIe	51 TFLOP/s	2 TB/s	1 TFLOP/s	1.96%	51.0

H100 算例的复用门槛。按所列 51 TFLOP/s、2 TB/s 和 $b_0 = 2\,\text{B/FLOP}$ 计算，$R_{\min} = 2 \times 51/2 = 51$；表9.6列出这一参数组合下的计算结果。

一维 $m = 5$ 卷积的内部块复用上限是 5，二维 $m = 5$ 的上限是 25。即使不断增大分块，这两种情形也达不到 51 的输入复用门槛。二维 $m = 9, T = 32$ 时，$R = (32 \times 9/40)^2 = 51.84$，在此模型中刚好超过该门槛。是否能用相应线程组织实现这个分块，还取决于资源容量。

9.4.4 权重窗口大小与资源平衡

由式(9.17)可得，分块后的模型峰值比例是

$$U(R) = \min\left(1, \frac{\mathcal{B}R}{b_0 C_{\text{peak}}}\right). \tag{9.19}$$

把 1000 GFLOP/s、150 GB/s 的组合记为模型 A，把 5000 GFLOP/s、192 GB/s 的组合记为模型 B，则分别有 $U_A = \min(1, 0.075R)$ 和 $U_B = \min(1, 0.0192R)$。下面固定输出分块大小，改变权重边长。

表中百分比为近似值；按未舍入公式计算，二维 $m = 5$ 在模型 B 下为 37.93%，二维 $m = 9$ 为 99.53%。这些百分比表示带宽模型允许的峰值比例，不能当作已测得的硬件利用率。

一维的内部块复用趋近 m，二维则趋近 m^2，因此二维卷积用较小的权重边长就可能接近同一个带宽门槛。同时，扩大权重窗口会增加完成一个输出的算术量；峰值比例上升，本身并没有给出处理同一问题所需时间的缩短幅度。

表 9.7 不同权重窗口对应的近似峰值比例

维度	输出分块	m	模型 A	模型 B
一维	$T = 1024$	5	37%	9.6%
		9	67%	17%
		15	100%	28%
		55	100%	100%
二维	$T \times T = 32 \times 32$	3	60%	15%
		5	100%	37%
		7	100%	67%
		9	100%	接近 100%

9.4.5 扩展计数：将输出写回计入流量

为了说明计数口径对结论的影响，可以在原有模型上增加输出写回。一个完整二维内部块加载 S^2 个输入，写回 T^2 个输出，进行 $2T^2m^2$ 次浮点操作。因此，只增加输出写回、不改变权重缓存假设时，得到

$$b_{\text{read+write}} = \frac{4S^2 + 4T^2}{2T^2m^2} = \frac{2}{R} + \frac{2}{m^2} \quad \text{B/FLOP.} \tag{9.20}$$

输入复用可以降低第一项，输出写回项则仍然存在。例如 $T = 16, m = 5$ 时 $R = 16$，只计输入得到 $b = 0.125\,\text{B/FLOP}$；加上输出后是 $0.125 + 0.08 = 0.205\,\text{B/FLOP}$。对应的未分块读写口径为 $2 + 2/m^2$。

因此，使用字节数比较实现、或将理论模型与测量结果对照时，应让输入、权重、输出及边界的统计口径保持一致。几何复用分析提供其中最直接的一部分：一次输入加载究竟支持了多少次邻域使用。

9.5 分块规模与存储层次

9.5.1 线程数量与共享内存容量

9.5.1.1 每块的线程预算

策略 2 为每个输入位置配置一个线程，因此每块的线程数、共享内存字节数及输出角色比例分别为

$$n_{\text{threads}} = S^2, \qquad q_{\text{shared}} = 4S^2\,\text{B}, \qquad \eta_{\text{compute}} = \frac{T^2}{S^2}. \tag{9.21}$$

η_{compute} 只表示具有输出计算角色的线程比例。$T = 8, m = 5$ 时它是 $64/144 \approx 44.4\%$；$T = 16, m = 5$ 时为 $256/400 = 64\%$。这些线程在加载阶段全部有工作，因而这一比例也不能直接当成整个内核的效率或 SM 占用率。

设设备报告的单块线程上限为 L_{block}，这种实现必须满足

$$(T + m - 1)^2 \le L_{\text{block}}, \qquad T \le \lfloor \sqrt{L_{\text{block}}} \rfloor - m + 1. \tag{9.22}$$

例如，若目标设备的单块线程上限为 1024，则 $m = 5$ 时允许的最大输出边长是 28，$m = 9$ 时是 24。32×32 输出配 5×5 权重需要 $36 \times 36 = 1296$ 个加载线程，已经超过这一示例上限；配 9×9 权重则需要 1600 个线程。几何表中的 32×32 或 64×64 分块仍可用于分析，但其实现必须采用不同的线程分工。

由式(9.11)还可以得到 $R_{2D} = m^2 \eta_{\text{compute}}$。这个关系把光环造成的线程角色开销与平均复用联系起来：固定权重边长时，核心区相对输入分块越大，平均复用越接近 m^2。

9.5.1.2 每个 SM 的驻留预算

流式多处理器（streaming multiprocessor，SM）可以同时容纳多个线程块。单块能否启动与多个块能否同时驻留，分别对应单块限制和 SM 总资源限制。设 SM 的线程容量为 N_{SM}，共享内存容量为 Q_{SM}；仅考虑这两项约束时，同时驻留的块数满足

$$n_{\text{resident}} \le \min\left(\left\lfloor \frac{N_{\text{SM}}}{S^2} \right\rfloor, \left\lfloor \frac{Q_{\text{SM}}}{4S^2} \right\rfloor\right). \tag{9.23}$$

共享内存容量尚有剩余，并不保证还能增加线程块；线程容量可能已经先被占满。

A100 的 SM 存储示例包含 256 KB 寄存器文件，以及 192 KB 的 L1 缓存 / 共享内存池，其中最多 164 KB 可划作共享内存。存储访问延迟可按粗略层级比较：寄存器约一个周期，共

享内存和缓存 / 常量存储约数个周期，全局内存约数百个周期。这些量级用于解释把反复访问的数据移到片上的动机。

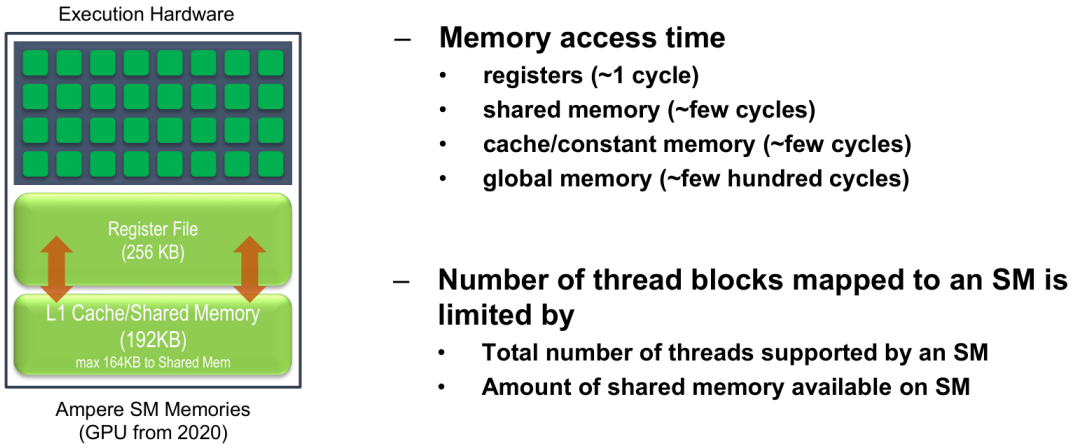


图 9.5 A100 示例中的片上存储容量及粗略访问层次。线程容量与共享内存容量共同限制可驻留线程块的数量。

若每个线程仅搬运一个 4 字节元素，2048 个线程总共只搬运 $2048 \times 4 = 8192$ 字节，约 8 KB。与一个 64 KB 共享内存容量的示例比较，线程数量与可用数据容量之间仍有很大差距。这个容量算式没有指定单块线程上限；实际 `blockDim` 仍需遵守式(9.22)。

9.5.2 更大数据块的多元素分工

共享内存中的数据分块可以比物理线程块大。让每个线程分多轮加载输入、承担多个输出，就能用较少的物理线程处理更大的输出分块。此时，线程块大小描述同时参与协作的线程数，数据分块大小描述这一组线程最终准备和处理的数据范围，两者可以分别设计。

一种组织方式是把二维输入分块展平。设共有 n 个线程，线程的线性编号为 t；让它依次加载 $t, t+n, t+2n, \ldots$ 这些位置，直到覆盖全部 S^2 个输入。加载和补零结束后统一同步，再按类似的循环分配 T^2 个输出。扩大数据分块后，光环占比降低，但每个线程的顺序工作也会增加。

更大分块的带宽模型。对模型 B 和 $m = 7$，$T = 32$ 时 $R = (224/38)^2 \approx 34.75$，对应峰值比例约 66.7%。取 $T = 80$，则

$$S = 86, \qquad R = \left(\frac{80 \times 7}{86}\right)^2 \approx 42.40, \qquad U_B = 0.0192R \approx 81.4\%. \tag{9.24}$$

该输入分块占 $86^2 \times 4 = 29584$ 字节。它需要多轮加载与多输出分工，不能直接建立 86×86 个线程。81.4% 是模型上限，尚未包括具体线程布局、同步和执行开销。

9.5.3 分银行存储与访问组织

寄存器文件和共享内存都分成多个存储体（bank），以提供并行的数据通路。多个访问争用同一存储体的服务能力时，可能发生存储体冲突（bank conflict），使流水线等待。数据已经位于片上，并不表示任意访问模式都能获得同样的吞吐。

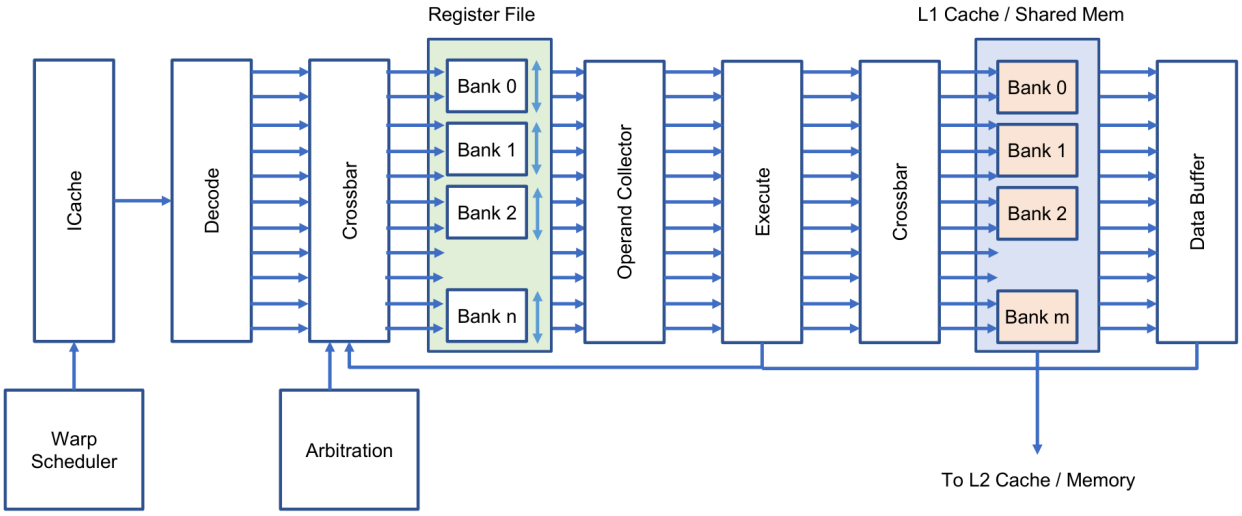


图 9.6 概念性的 SM 执行流水线。寄存器文件与 L1 / 共享内存均采用多存储体组织，交叉开关连接执行通路、操作数收集与存储通路。

图9.6中的线程束调度器决定待执行工作，经指令缓存与译码后，寄存器中的操作数通过交叉开关和操作数收集器送到执行单元。涉及存储访问时，数据通路还会经过 L1 / 共享内存一侧；访问更低层存储时连接到 L2 缓存和内存。寄存器与共享内存两处都可能成为数据供给的限制环节。

全局内存则包含多个通道、存储体和页，并依赖成组传输提高效率。合并访存（memory coalescing）关注相邻线程的访问如何组合成有效的数据传输。行优先图像中，同一行相邻列的地址连续，所以让相邻的横向线程加载相邻列，有利于形成成组访问。输入读取次数相同的两个实现，也可能因为地址分布不同而产生不同的传输效率。

L1 缓存的非一致性（non-coherence）意味着不能仅凭存在缓存，就假设不同缓存副本会随其他副本的写入自动保持一致。数据放置、访问组织和同步分别解决不同的问题。对分块卷积而言，完整的性能分析需要同时看输入复用、线程分工、共享内存访问及全局访问组织；单独提高 R 只改善其中的输入供给部分。

9.6 例题与关键结论

9.6.1 角块的全局读取缩减倍数

题目。对 32×32 输入图像使用 5×5 权重和 16×16 输出分块，采用策略 1。求线程块 $(0,0)$ 因共享内存复用而获得的输入全局读取缩减倍数。

输出范围与输入范围。本块负责输出行、列 0–15，半径 $r = 2$。完整输入分块的坐标范围为 -2–17，边长为 20；与有效图像相交后，实际读取的行、列范围都是 0–17。因此不同有效输入数为 $B = 18 \times 18 = 324$。其余 $20^2 - 18^2 = 76$ 个共享内存位置补零。

直接读取计数。输出 $(0,0)$ 需要 $3 \times 3 = 9$ 个有效输入，$(0,1)$ 需要 $3 \times 4 = 12$ 个，第 0 行的其余 14 个输出各需要 $3 \times 5 = 15$ 个。第 1 行有四行有效输入，后面 14 行各有五行有效输入。逐行统计为

$$\begin{aligned} A_0 &= 9 + 12 + 15 \times 14 = 231, \\ A_1 &= 12 + 16 + 20 \times 14 = 308, \\ A_y &= 15 + 20 + 25 \times 14 = 385, \quad 2 \le y \le 15, \\ A &= 231 + 308 + 14 \times 385 = 5929. \end{aligned} \tag{9.25}$$

最终结果为

$$\boxed{R = \frac{5929}{324} \approx 18.30.} \tag{9.26}$$

乘积验证。单个方向的有效邻域长度之和为 $3 + 4 + 14 \times 5 = 77$，二维直接读取数就是 $77^2 = 5929$。这与式(9.14)及逐行展开一致。采用策略 1 影响的是 256 个线程如何分多步装入这些输入；只要完整输入被正确装入且每个有效元素只读一次，计数结果仍由 324 个不同有效输入决定。

把本题当成内部块，会得到 $16^2 \times 25/20^2 = 16$。这一结果对应的是“400 个输入位置全部有效”的另一种位置条件。本题角块应同时排除直接实现中的越界读取和分块加载中的虚拟输入，得到 18.30。

9.6.2 输入型线程块中的控制分歧

题目。二维输出分块为 30×30，权重为 3×3，输入分块为 32×32。采用策略 2，计算输出时每个线程块中有多少个线程束发生控制分歧？

线程束与行的对应。这里每个线程束含 32 个线程。二维块按 x 坐标优先展平，线性编号为 $t=t_x+32t_y$，线程束编号是 $w=\lfloor t/32\rfloor=t_y$。因此每一行恰好构成一个线程束，总计 $1024/32=32$ 个线程束。

逐线程束检查。输出角色条件为 $t_y<30$ 且 $t_x<30$。对于 $w=0$–29，同一线程束中 $t_x=0$–29 的 30 个线程进入乘加循环，$t_x=30,31$ 的两个线程不进入，故这 30 个线程束都有控制分歧。对于 $w=30,31$，所有线程都不满足行条件，整个线程束统一跳过计算。

表 9.8 计算角色判断在各线程束中的结果

线程束编号	线程束数量	每束计算线程	每束不计算线程	是否分歧
0–29	30	30	2	是
30–31	2	0	32	否

因此，**有 30 个线程束发生计算角色分歧，2 个线程束没有该分歧**。全部线程统一不执行某个分支，仍然是统一的控制流。所有 32×32 个线程都参与输入分块的加载或补零；上述答案只统计计算角色条件，图像边缘的加载和写回判断应按实际位置另行检查。

当输入块的边长 S 不等于 32 时，一个线程束可能跨越多行。这时应先用 $t_x=t \bmod S$、$t_y=\lfloor t/S\rfloor$ 恢复局部坐标，再检查每组线程的条件取值，不能继续把一行直接当成一个线程束。

9.6.3 A40 算例中的带宽平衡与复用门槛

题目。采用 A40 算例给出的 FP32 峰值 37.4 TFLOP/s、内存容量 48 GB 和带宽 696 GB/s。求充分供给计算峰值所需的字节 / FLOP 比，并说明卷积输入需要多少复用。

平衡点的两种单位。这一比值由带宽与计算峰值决定，内存容量不参与计算。统一单位后，可得

$$\begin{aligned} b_{\text{balance}} &= \frac{696}{37400} \approx 0.0186096\,\text{B/FLOP} \approx 0.019\,\text{B/FLOP},\\ I_{\text{balance}} &= \frac{37400}{696} \approx 53.7356\,\text{FLOP/B}. \end{aligned} \tag{9.27}$$

也就是每从全局内存取得一个字节，需要支持约 53.7 次浮点操作，才与这一峰值计算 / 带宽比平衡。

输入复用门槛。上述 53.7 的单位是 FLOP/B。要得到相对于未分块卷积的输入复用倍数，还要使用其基准 $b_0 = 2\,\text{B/FLOP}$，或等价的 $I_0 = 0.5\,\text{FLOP/B}$：

$$\boxed{R_{\min} = \frac{b_0}{b_{\text{balance}}} = \frac{I_{\text{balance}}}{I_0} = \frac{2 \times 37400}{696} \approx 107.47.} \tag{9.28}$$

从单个输入元素看也一样：4 字节需要支持约 $4 \times 53.7356 = 214.94$ FLOP，而一个输入的一次卷积使用贡献两次 FLOP，故需要约 $214.94/2 = 107.47$ 次平均使用。推算时保留未舍入的带宽比，可以避免过早使用 0.019 带来的数值偏差。

三个量的区别

0.01861 B/FLOP 是峰值平衡所允许的字节 / 操作比；53.74 FLOP/B 是它的倒数；107.47 是在未分块卷积基准之上所需的无量纲输入复用倍数。前两个量由设备算例的带宽与峰值给出，第三个量还依赖算法的基准数据使用方式。

9.6.4 关键公式与实现关系

二维分块卷积的实现与分析可以按同一条依赖关系组织：先确定输出分块，向外扩展出输入分块，再把输入位置分配给线程，最后统计这些加载支撑的有效计算。表9.9汇总了其中的关键关系。

表 9.9 几何、线程、复用与吞吐之间的关系

问题	核心关系	适用条件或含义
输入分块	$S = T + m - 1 = T + 2r$	输出每侧扩展半径 r。
块与网格	块为 $S \times S$；网格按 $\lceil W/T \rceil \times \lceil H/T \rceil$	策略 2；输出步长为 T。
输入坐标	$y_i = y_o - r,\ x_i = x_o - r$	所有线程分别加载或补零。
输出计算	$t_y < T,\ t_x < T$；访问 $\text{tile}[t_y + i][t_x + j]$	同步后计算；写回还需输出边界判断。
内部块复用	$R_{1D} = Tm/S$；$R_{2D} = (Tm/S)^2$	完整加载；计输入全局读取。
边界块复用	$R = A/B$，A, B 均只计有效输入	虚拟位置补零，不产生输入全局读取。
带宽模型	$b = 2/R$；$I = R/2$	单精度乘加；只计输入读取。
计算平衡	$R_{\min} = b_0 C_{\text{peak}}/\mathcal{B}$	模型中的必要复用门槛。

核对代码时，输入边界、全块同步、计算角色和输出边界应依次成立；行坐标与高度比较，列坐标与宽度比较。核对计数时，先区分输入分块的存储位置数、真正发生的全局加载数，以及所有输出累计的输入使用次数。核对性能结论时，则将几何上的复用、资源允许的线程布局和带宽模型的上限分别算清，再建立它们之间的关系。

第 10 讲

Introduction to ML; Inference and Training in DNNs

10.1 机器学习与表示学习

10.1.1 前馈神经网络与课程事项

前馈神经网络（feedforward neural network）通过一系列有方向的层，将输入变换为输出。神经元模型与激活函数决定各层的计算方式，梯度下降则通过调整参数完成训练。与并行程序设计的联系在于：网络的前向计算需要大量点积与矩阵运算，训练还需要计算、传播并汇总这些运算对应的梯度。

课程事项：2026 年 9 月 24 日

Lab 3 于当周周五截止；项目里程碑 1（Project milestone 1）即将发布。期中考试 1 的时间为 10 月 6 日 19:00–22:00。课后事项为提交 Lab 3、选择项目；阅读范围为教材第 4 版第 16 章，或第 5 版第 19 章。

10.1.2 从输入输出规则到从样本中学习

机器学习（machine learning）从数据中提取模式，用这些模式建立输入与输出之间的关系。对于难以完整写出程序规则的任务，可以提供带标签的数据，即已经配对的输入和期望输出，再迭代调整模型参数，使模型的答案接近期望答案。这个调整过程称为训练（training）；参数确定以后，由新输入计算输出的过程称为推断（inference）。分类、回归、语音转写和翻译都是机器学习任务。

数据驱动的方法依赖三个条件：有足够的计算能力反复训练模型，有能够表达目标关系的数据，以及存在难以手工完整描述规则的应用需求。GPU 与 CUDA 缩短了深度神经网络的训练与实验周期；传感器、网络服务和存储设施增加了可获得的数据；自动驾驶、智能设备、安全和医疗等需求则推动了相关应用。

计算速度提高以后，网络结构、特征选择、输入输出形式和高质量标注仍然需要设计。训练数据的作用是提供可学习的关系；数据是否覆盖了任务中的关键变化，直接影响模型有机会学到什么。

10.1.3 穷举规则的规模与特征的作用

10.1.3.1 小规模布尔函数可以由真值表完整指定

对三个二进制输入 a, b, c，总共只有 $2^3 = 8$ 种输入。表 10.1 给出一个完整的输入输出关系。只要列出全部输出为 1 的输入组合，就能构造对应的逻辑表达式和逻辑电路。

表 10.1 三个输入的奇偶性函数。

a	b	c	输出	输入中 1 的个数
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	2
1	0	0	1	1
1	0	1	0	2
1	1	0	0	2
1	1	1	1	3

这张表还具有一个更紧凑的描述：输入中 1 的个数为奇数时输出 1，为偶数时输出 0。相应的布尔表达式是 $a \oplus b \oplus c$，其中 $\oplus$ 表示异或。**奇偶性就是能够解释全部观测的一个特征。**只有部分观测时，例如 $000 \mapsto 0$、$011 \mapsto 0$、$100 \mapsto 1$、$110 \mapsto 0$，还不能仅凭查表覆盖所有输入；能够提取这种规律，才有机会对未出现的组合给出答案。

10.1.3.2 图像输入使查表规模呈指数增长

一张 32×32 的图像包含 1024 个像素。按每像素 8 位计算，输入总共需要 8192 位，因此所有可能输入的数量是 2^{8192}。即使每种输入只对应一个类别，完整真值表仍然需要为这 2^{8192} 种输入分别记录输出。

因为 $10^9 \approx 2^{30}$，十亿个不同输入覆盖的比例约为

$$\frac{2^{30}}{2^{8192}} = 2^{-8162}. \tag{10.1}$$

这个数量级说明，收集更多样本仍然无法接近穷举整个输入空间。学习过程需要利用多个样

本共有的特征，把有限观测中的结构推广到未观测输入。小规模逻辑例子的奇偶性，正好说明了“从大量具体情况中提取简洁规律”这一作用。

10.1.4 规则的描述难度与计算难度

10.1.4.1 两个不同的困难来源

一个任务可以容易形式化，却需要很大的计算量；也可以由人较容易完成，却难以写出完整规则。国际象棋的棋盘、棋子与合法走法都很明确，其困难主要来自搜索规模。人脸检测与语音识别则说明了另一种困难：识别结果容易理解，但把所有输入条件转化为固定规则很困难。文学分析与国际外交被用作更复杂任务的示意例子。

这种区分与并行计算有关。对于运算规则已经明确的任务，可以继续研究算法复杂度、并行性与数据带宽；对于输入输出关系本身难以手工表达的任务，还需要解决模型如何从观测中获得这种关系的问题。

10.1.4.2 国际象棋：规则明确，搜索树很大

棋盘有 64 个位置，初始有 32 个棋子。根据合法走法生成后继局面、为搜索树叶节点评分、再根据这些评分选择路径，就形成了一种可明确描述的计算过程。如果每个局面平均约有 30 种合法走法，搜索 10 个半回合的叶节点数量约为

$$30^{10} = 5.9049 \times 10^{14}. \tag{10.2}$$

即使每秒处理 10^6 个局面，也需要约 5.9×10^8 秒，即约 18.7 年。因此，启发式方法、剪枝、并行搜索和更快的计算机共同影响这种方案是否实用。Deep Blue 在 1997 年击败 Garry Kasparov，是一个历史例子。

10.1.4.3 规则系统：常识需要一致的语义表示

规则系统用事实与规则驱动推理引擎（inference engine），产生可由已有知识推出的结论。Cyc 的例子把常识组织为本体和知识库，再对形式化陈述进行推理。例如，从“某人是一位美国总统”和“美国总统是美国公民”可以推出该人是美国公民。这说明规则能够显式表达一条推理链。

“Fred 正在刮胡子”的例子揭示了语义表示的难处：人没有电气部件，人刮胡子时有剃刀，剃刀有电气部件。若把“拥有某物”与“某物是身体的组成部分”混为同一种关系，规

则之间就可能产生错误关联。能够处理真实常识的系统需要大量事实、规则及一致的关系定义；这里的规则系统依赖预先给定的规则与陈述。

10.1.5 人工特征与学习得到的表示

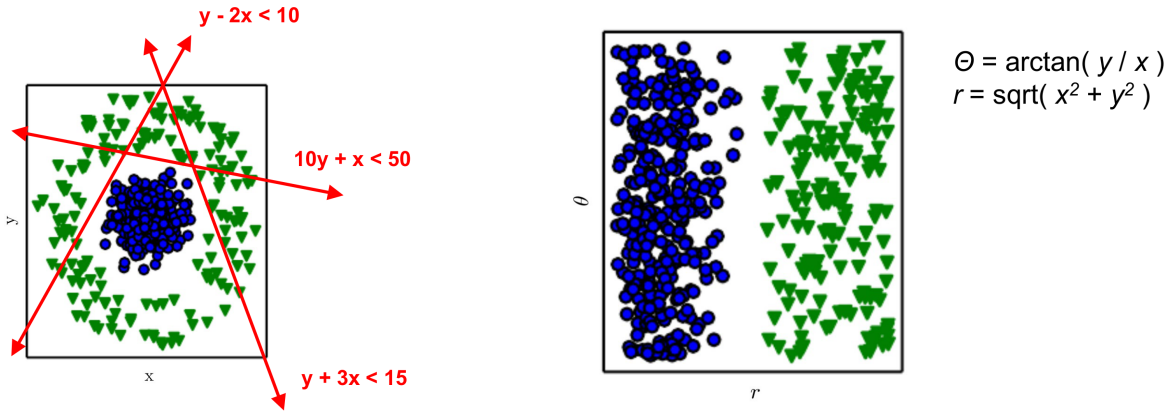
10.1.5.1 经典机器学习中的特征映射

经典机器学习通常先由人选定特征，再学习特征与输出之间的映射。例如，朴素贝叶斯（naïve Bayes）采用“给定类别后，各特征条件独立”的假设；逻辑回归（logistic regression）学习各特征对输出的权重，训练过程可以使用梯度下降。这两个例子说明，特征如何表示输入，与模型如何利用这些特征，是可以分别考虑的环节。

不同任务需要不同的特征。图像可以经过视觉特征提取再进行检测；音频可以转换成音频特征，用于识别说话人；文本特征可以用于文本分类、机器翻译和信息检索。特征必须保留与任务输出有关的信息，同时让后续映射更容易表达。

10.1.5.2 坐标表示能够改变分类边界的复杂度

图 10.1 中，蓝色点聚集在中心，绿色点分布在外围。在直角坐标 (x, y) 下，仅用一条直线难以将两者分开；图中的多条线性不等式是在尝试用若干半平面描述中心区域。



(a) 直角坐标表示。 **(b)** 半径与角度表示。

图 10.1 相同分类任务在不同特征坐标下具有不同的分离难度。

若改用半径 $r = \sqrt{x^2 + y^2}$ 和角度 θ，中心点与外围点的差异就主要表现为 r 的大小。于是，用一个半径阈值就能描述图示的分离关系。这里有用的特征是“到中心的距离”，它把几何结构直接交给后续分类器。

图中的 $\theta = \arctan(y/x)$ 是简写；完整角度表示还需要区分象限，并处理 $x = 0$ 的情形。该例用于说明半径特征的作用，不依赖具体的角度实现。

10.1.5.3 从变化因素到层次化表示

人工指定“有车轮”作为识别汽车的特征，仍然要回答如何从像素中识别车轮。圆形轮廓、较暗的边缘等规则又会受到遮挡、观察角度、阴影和轮胎外观的影响。因此，选定一个看似合理的语义特征，并没有自动解决其底层检测问题。

变化因素（factors of variation）指影响观测数据的潜在因素。例如，年龄、性别、口音和说出的词语会影响语音；位置、颜色和太阳角度会影响汽车图像。一个观测通常同时受到多个因素影响，学习表示需要从这些混合变化中提取与任务相关的结构。

表示学习(representation learning)把特征本身也纳入学习过程。深度学习(deep learning)进一步用其他表示来表达新的表示，形成层次化特征。图像中的示意层次可以是输入像素、边缘、角点与轮廓、对象部件，最后是对象类别；较复杂概念由较简单概念组合而成。

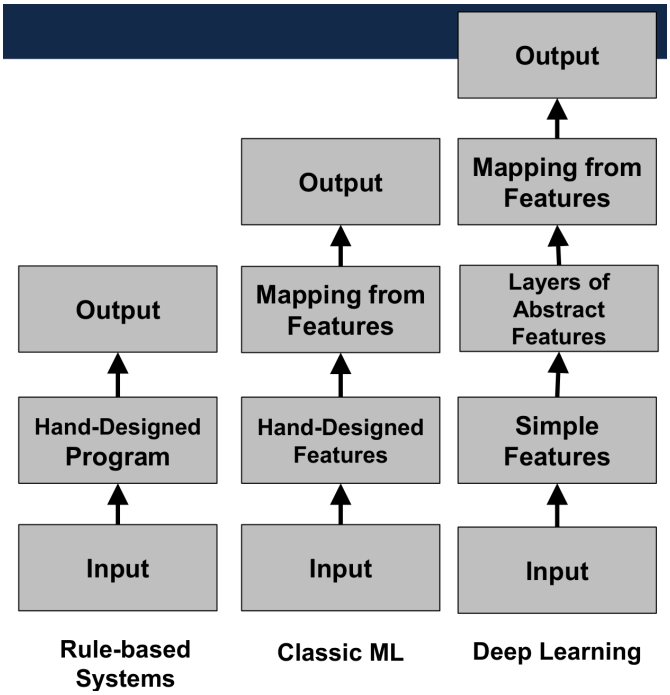


图 10.2 规则系统、经典机器学习与深度学习中的输入到输出路径。深度学习在中间引入多层特征表示。

输入输出关系与中间表示

规则系统直接编码输入输出关系；经典机器学习学习人工特征到输出的映射；表示学习还学习特征的构造方式。深度学习通过层次组合表达特征。多层感知机中的隐藏层，为这种“由输入构造新表示，再根据新表示做判断”的过程提供了一个具体模型。

10.2 从感知机到多层网络

10.2.1 分类函数与单个感知机

分类（classification）把输入向量对应到某个离散类别。设输入 $x \in \mathbb{R}^N$，类别数为 C，未知的目标关系可以写成 $F : \mathbb{R}^N \to \{1, \dots, C\}$。模型使用可调参数 Θ，构造近似这一关系的函数。对于后面的数字识别，类别索引使用 $0, \dots, 9$；这只是类别编号的变化。

一个感知机（perceptron）先计算输入的加权和，再通过符号函数给出二分类结果。设权重 $w \in \mathbb{R}^N$、偏置 $b \in \mathbb{R}$，则

$$u = w^{\mathrm{T}} x + b = \sum_{j=0}^{N-1} w_j x_j + b, \qquad \hat{y} = \operatorname{sign}(u). \tag{10.3}$$

其中 $\Theta = \{w, b\}$；w_j 决定输入第 j 个分量对加权和的贡献，b 决定阈值的平移。这里的中间量 u 与最终的类别输出 $\hat{y}$ 分开记号，便于区分点积和分类判断。

当 $u > 0$ 时取 $+1$，$u < 0$ 时取 -1。决策边界是 $w^{\mathrm{T}} x + b = 0$：在二维输入空间中是一条直线，在更高维中是一个超平面。本节的布尔例子使用 0、1 作为输入，用 -1、$+1$ 编码假、真；所选偏置保证样本不会恰好落在 $u = 0$ 上。

10.2.2 异或问题与隐藏表示

10.2.2.1 单条线性边界为什么无法表达异或

异或（XOR）在两个输入不同时为真，相同时为假。因此，$(0,1)$ 与 $(1,0)$ 属于正类，$(0,0)$ 与 $(1,1)$ 属于负类。四个点的几何位置使同类点位于正方形的对角线上，单条直线无法完成所需分离。

假设四个点均被严格正确分类，则权重与偏置必须同时满足下列条件。

$$b < 0, \qquad w_0 + w_1 + b < 0, \tag{10.4}$$

$$w_0 + b > 0, \qquad w_1 + b > 0. \tag{10.5}$$

后两式相加得到 $w_0 + w_1 + 2b > 0$；而前两式相加得到同一表达式小于 0。两者矛盾。因此，式 (10.3) 这种单个感知机不能完成该异或分类。

10.2.2.2 用 AND 与 OR 构造中间特征

AND 与 OR 分别可以由一个感知机实现。取如下权重与偏置：

$$h_{\text{OR}} = \text{sign}(x_0 + x_1 - 0.5), \tag{10.6}$$

$$h_{\text{AND}} = \text{sign}(x_0 + x_1 - 1.5). \tag{10.7}$$

两个感知机具有相同的输入权重 $(1,1)$，但使用不同偏置，因此具有不同判断阈值。原来的二维输入被转换成新坐标 $(h_{\text{AND}}, h_{\text{OR}})$：$(0,0)$ 变为 $(-1,-1)$，两个“恰好一个输入为 1”的点都变为 $(-1,+1)$，$(1,1)$ 变为 $(+1,+1)$。

这时，异或为真的样本已经集中到同一个表示位置。组合 $(+1,-1)$ 不可能出现，因为 AND 为真必然意味着 OR 为真。接下来的一个感知机只需要在这三个有效位置上完成分类。

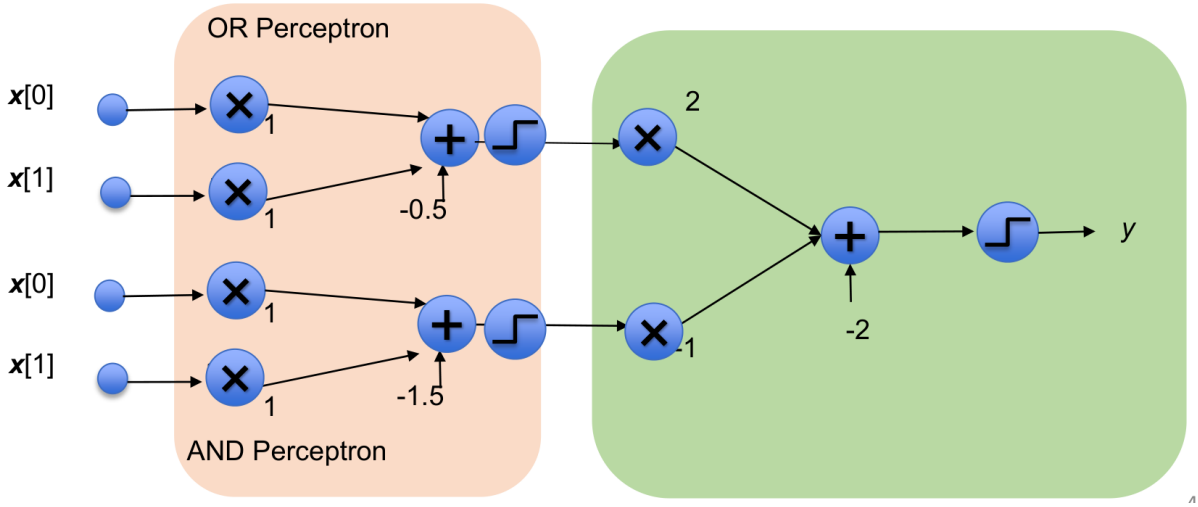


图 10.3 OR 与 AND 感知机构成隐藏层，再由带权组合和符号函数给出 XOR 输出。乘号旁的数值是连接权重，加法器旁的常数是偏置。

输出层取

$$\hat{y}_{\text{XOR}} = \text{sign}(2h_{\text{OR}} - h_{\text{AND}} - 2). \tag{10.8}$$

表 10.2 逐项核算这一表达式。对于只有一个输入为 1 的情形，输出层的加权和为 $2\times 1 - (-1) - 2 = 1$，所以得到正类；其余两个情形分别得到 -3 和 -1，所以得到负类。

表 10.2 两层感知机的完整 XOR 计算。

x_0	x_1	h_{AND}	h_{OR}	$2h_{\text{OR}} - h_{\text{AND}} - 2$	输出
0	0	-1	-1	-3	-1
0	1	-1	$+1$	$+1$	$+1$
1	0	-1	$+1$	$+1$	$+1$
1	1	$+1$	$+1$	-1	-1

10.2.2.3 非线性变换如何改变下一层的问题

隐藏层的符号函数把输入变为新的特征位置。下一层的直线边界是在这个新表示空间中起作用，映射回原始输入后就能够实现单个感知机无法表达的分类关系。这里体现的是表示变化与分类边界之间的配合。

若两层之间没有激活函数，两次仿射变换可以合并。设 $h = W_1 x + b_1$，输出加权和为 $W_2 h + b_2$，则

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2). \tag{10.9}$$

合并后的表达式仍只有一次仿射变换；若最终再接一个符号函数，仍然只产生一条线性决策边界。多层结构能够形成新表示，关键在于层间插入了合适的非线性函数。

XOR 例子的三个组成部分

第一层的权重与偏置定义 OR、AND 两个判断；隐藏层的符号函数生成新表示；输出层对这个表示加权、加偏置并再次判断。偏置与最终的符号函数共同构成式 (10.8) 的输出层。若仅用 $ah_{\text{OR}} + bh_{\text{AND}}$，不加偏置，则输入 00 与 11 对应的组合值互为相反数，无法同时得到 XOR 所需的负类输出。

10.2.3 全连接层、网络层次与参数维度

全连接层（fully-connected layer）让每个输出单元都连接到全部输入。若输入有 N 个分量、输出有 M 个分量，每个输出都需要一个长度为 N 的权重向量；把这些权重向量逐行排列，就得到权重矩阵 W。

矩阵与索引约定

后续计算采用列向量：$x \in \mathbb{R}^N$、$W \in \mathbb{R}^{M\times N}$、$b \in \mathbb{R}^M$。$W[i,j]$ 表示输入 j 到输出 i 的权重，即**行对应输出，列对应输入**。因此，$4 \to 3$ 层使用 3×4 权重矩阵；若按输入在行、输出在列排列权重，则须转置后参与这里的列向量乘法。连接数与参数量不因排列方向而改变。

在这一约定下，全连接层的加权和是 $\mathrm{fc} = Wx + b$，其第 i 个分量为

$$\mathrm{fc}[i] = \sum_{j=0}^{N-1} W[i,j]x[j] + b[i]. \tag{10.10}$$

每个输出再经过激活函数（activation function）ϕ，得到 $h[i] = \phi(\mathrm{fc}[i])$。对于逐元素激活，$h = \phi(Wx + b)$ 中的 ϕ 分别作用于向量各个分量。点积负责组合输入，偏置调节加权和的位置，激活函数给出非线性变换。

网络中的输入层（input layer）存放输入数据，隐藏层（hidden layer）产生中间表示，输出层（output layer）产生用于最终任务的输出。全连接网络包含多个这样的计算层时，就形成多层感知机（multilayer perceptron，MLP）。图中节点表示一个计算单元，连线表示参与加权求和的连接。

参数计数由全连接结构直接得到。输入为 N 维、输出为 M 维的一层有 MN 个权重与 M 个偏置，因此参数总数为 $M(N+1)$。以 $4 \to 4 \to 3$ 的网络为例，在列向量约定下，第一层为 $W_1 \in \mathbb{R}^{4\times 4}$、$b_1 \in \mathbb{R}^4$，第二层为 $W_2 \in \mathbb{R}^{3\times 4}$、$b_2 \in \mathbb{R}^3$。第二层的 3 行分别生成 3 个输出，每一行都读取隐藏层的 4 个分量。整个网络共有 $4\times 4 + 4 + 3\times 4 + 3 = 35$ 个可调参数。

10.2.4 手写数字识别网络的具体结构

MNIST 手写数字识别例子以 28×28 的灰度图像为输入，输出数字 0 至 9 中的一个类别。给出的训练数据有 60,000 张带有人类标注的图像。把一张图像的像素按固定顺序展开，就得到长度为 $28 \times 28 = 784$ 的输入向量。

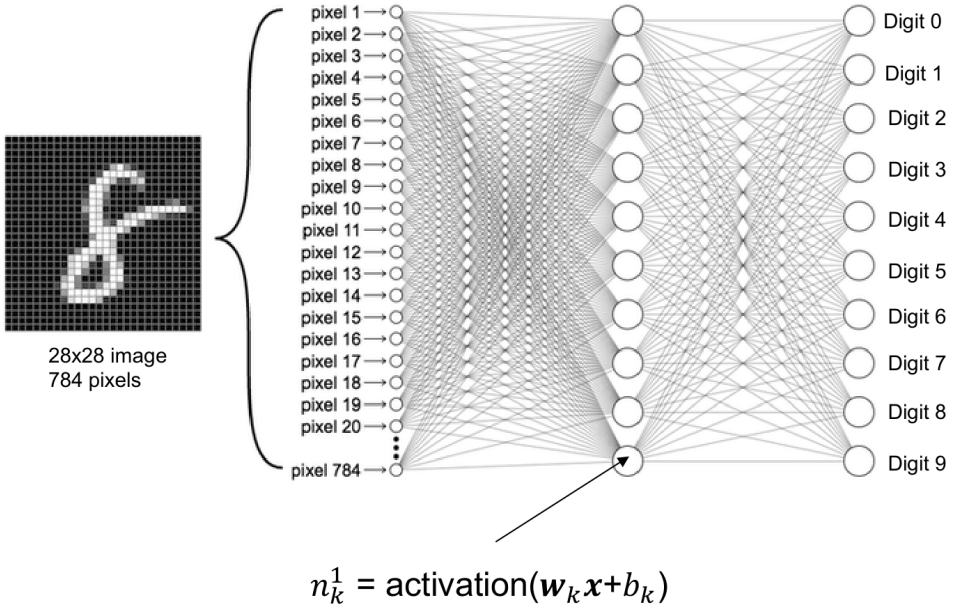


图 10.4 784 → 10 → 10 的手写数字识别网络。每个隐藏单元对 784 个像素加权求和，并经过激活函数；输出端有 10 个类别分量。

该例包含 784 个输入节点、10 个隐藏节点和 10 个输出节点。两层参数的形状与数量列在表 10.3 中。输入层不额外引入一组权重；参数属于输入到隐藏层、隐藏层到输出层这两次可调变换。

表 10.3 数字识别网络的参数维度与计数。

变换	权重形状	偏置形状	权重数	参数合计
784 → 10	10×784	10×1	7,840	7,850
10 → 10	10×10	10×1	100	110
总计			7,940	**7,960**

XOR 例子中，权重可以根据逻辑规则直接构造。数字识别中，需要用带标签的数据自动调整这 7,960 个参数，使输入像素与正确类别之间的关系编码到网络中。训练所需的两个进一步组成部分是：如何从连续输出获得类别信息，以及如何用输出误差计算参数的调整方向。

10.3 前向计算与分类输出

10.3.1 输入、分数、概率与类别的区别

前向传播（forward propagation）按照网络的连接方向依次求值。给定输入 x 和参数 Θ，先计算各层加权和与激活值，再获得输出分量。分类任务还需要根据这些输出选择一个类别。

反向传播（backpropagation）则需要真实标签，用损失函数衡量输出与目标的差异，并计算损失对参数的导数。训练包含前向求值、损失计算、反向求导和参数更新；前向求值也是训练的一部分。推断时使用已经确定的参数产生输出。

表 10.4 两层数字识别网络的主要符号及其含义。

符号	含义	数字识别例子的形状
x	一张图像展开后的输入向量	784×1
fc_1, y	第一层加权和与激活输出	10×1
fc_2, z	第二层加权和与激活输出	10×1
k	归一化后的类别概率向量	10×1
T, $\widehat{T}$	真实数字标签与最终预测标签	标量类别编号
Θ	两层的权重与偏置集合	共 7,960 个参数
E	一个带标签输入对应的损失	标量

两层数字识别网络在每层加权和之后均使用 sigmoid，再由 softmax 将第二层激活值转换为类别概率：

$$\mathrm{fc}_1 = W_1 x + b_1, \qquad y = \sigma(\mathrm{fc}_1), \tag{10.11}$$

$$\mathrm{fc}_2 = W_2 y + b_2, \qquad z = \sigma(\mathrm{fc}_2), \tag{10.12}$$

$$k = \mathrm{softmax}(z), \qquad \widehat{T} = \underset{0 \le i < C}{\arg\max}\, k[i]. \tag{10.13}$$

这里的 y 是隐藏层向量，z 是传入 softmax 的分数向量，k 才是满足归一化要求的类别概率；$\widehat{T}$ 是离散类别编号。$\arg\max$ 的作用是选出最大分量的索引，损失及其导数将在连续输出 k 上定义。

10.3.2 激活函数的形状与可求导性

10.3.2.1 符号函数与梯度传播

符号函数可以表达 XOR 构造中的硬阈值，但其输出在阈值两侧分别为常数。因此，在非零输入处导数为 0，在零点发生跳变。若直接使用普通链式法则传播梯度，大部分位置都会遇到这个零导数，难以提供连续调整权重所需的信号。

训练中的激活函数既决定前向变换，也决定反向传播时乘上的局部导数。激活值能否随输入的微小变化而改变，是理解这种梯度信号的重要出发点。

10.3.2.2 Sigmoid：输出值可以直接用于求导

Sigmoid，也称逻辑函数（logistic function），定义为

$$\sigma(u) = \frac{1}{1+e^{-u}}, \qquad \sigma : \mathbb{R} \to (0,1). \tag{10.14}$$

当 u 增大时，输出从接近 0 平滑增加到接近 1，并在 $u = 0$ 时取 1/2。其导数可以写成已计算输出的简单函数：

$$\sigma'(u) = \frac{e^{-u}}{(1+e^{-u})^2} \tag{10.15}$$

$$= \frac{1}{1+e^{-u}}\left(1 - \frac{1}{1+e^{-u}}\right) = \sigma(u)\big(1-\sigma(u)\big). \tag{10.16}$$

因此，前向已经算得 $s = \sigma(u)$ 时，反向计算只需要 $s(1-s)$，无需重新求一次指数函数。这也说明为什么保存前向激活值对训练有用。

由 $s \in (0,1)$ 可直接得到 $0 < \sigma'(u) \le 1/4$；在 s 接近 0 或 1 时导数接近 0。当激活值接近区间两端时，这一局部因子会减小传回的梯度。

10.3.2.3 ReLU 与 softplus

整流线性单元（rectified linear unit，ReLU）定义为

$$\mathrm{ReLU}(u) = \max(0, u). \tag{10.17}$$

负输入被映射为 0，正输入保持不变。它的前向计算不需要指数函数，因此具有较简单的计算形式。由分段定义可得，在 $u < 0$ 时导数为 0，在 $u > 0$ 时导数为 1；零点没有普通导数。

实现反向传播时，需要单独约定零点处采用的梯度值。

Softplus 是 ReLU 的一种平滑近似：

$$\operatorname{softplus}(u) = \ln(1 + e^{u}), \qquad \frac{d\,\operatorname{softplus}(u)}{du} = \frac{e^{u}}{1 + e^{u}} = \sigma(u). \tag{10.18}$$

这也解释了“softplus 是逻辑函数的一个原函数”的说法。Sigmoid、ReLU 与 softplus 的形状和计算代价不同；具体网络需要选择与其任务和训练方式相适应的函数。

一层中的两类计算

$Wx + b$ 通过矩阵运算组合输入，逐元素激活 $\phi(\cdot)$ 再改变每个输出分量。反向传播时，矩阵部分与激活部分分别贡献自己的导数，并按照链式法则相乘。激活函数的前向值与局部导数因此需要配套考虑。

10.3.3 Softmax 与互斥类别的概率

10.3.3.1 归一化的类别输出

对于 C 个互斥类别，模型输出的各个类别概率应当非负，且总和为 1。对每个分量独立使用 sigmoid，虽然能把各分量限制在 $(0,1)$，却不会自动保证总和为 1；ReLU 也不提供这种归一化。因此，需要把分数向量进一步转换为类别概率向量。

给定 $z = (z[0], \dots, z[C-1])^{\mathrm{T}}$，softmax 定义为

$$k[i] = \frac{e^{z[i]}}{\sum_{j=0}^{C-1} e^{z[j]}}, \qquad i = 0, \dots, C-1. \tag{10.19}$$

每个分量的分子都为正，分母又是全部分子的总和，因此 $k[i] > 0$ 且 $\sum_i k[i] = 1$。较大的分数对应较大的概率；最终分类可以取 $\widehat{T} = \arg\max_i k[i]$。

取 $z = (0.2, 0.4, 0.8)^{\mathrm{T}}$，则

$$e^{z} \approx (1.2214, 1.4918, 2.2255)^{\mathrm{T}}, \qquad k \approx (0.2473, 0.3021, 0.4506)^{\mathrm{T}}. \tag{10.20}$$

采用从 0 开始的类别编号时，预测类别为 2。分数 $z[2] = 0.8$ 和概率 $k[2] \approx 0.4506$ 是两个不同的量：概率需要把这一分数与全部其他类别的分数共同归一化。

归一化还带来平移不变性：对所有分数加同一个常数 c，分子和分母都乘以 e^{c}，因此 $\operatorname{softmax}(z + c\mathbf{1}) = \operatorname{softmax}(z)$。这表明输出取决于类别之间的相对分数。

有界分数对概率范围的限制

两层 sigmoid 网络的每个分数均满足 $0 < z[i] < 1$。当类别数 $C \geq 2$ 时，

$$k[i] = \frac{1}{1 + \sum_{j \neq i} e^{z[j]-z[i]}} < \frac{1}{1 + (C-1)e^{-1}} = \frac{e}{e + C - 1}.$$

因此，$C = 10$ 时任意类别的概率都小于约 0.232。这一上界来自 softmax 之前的 sigmoid 对分数范围的限制；它不限制通过最大概率索引得到的分类准确率。

10.3.3.2 Softmax 的导数为什么包含其他类别

改变一个分数 $z[m]$ 会同时改变它自己的分子，以及所有类别共享的分母。因此，$k[i]$ 不仅依赖 $z[i]$，还依赖其他类别的分数。这个耦合关系使 softmax 的导数成为一个矩阵。

令 $S = \sum_j e^{z[j]}$。用商法则展开，有

$$\frac{\partial k[i]}{\partial z[m]} = \frac{\delta_{i,m} e^{z[i]} S - e^{z[i]} e^{z[m]}}{S^2} \tag{10.21}$$

$$= k[i]\big(\delta_{i,m} - k[m]\big), \tag{10.22}$$

其中 $\delta_{i,m}$ 是克罗内克符号（Kronecker delta）：$i = m$ 时为 1，否则为 0。这就给出两种情况：

$$\frac{\partial k[i]}{\partial z[m]} = \begin{cases} k[i](1 - k[i]), & i = m, \\ -k[i]k[m], & i \neq m. \end{cases} \tag{10.23}$$

自身分数上升会提高自身概率；某个其他类别的分数上升，则会降低当前类别的概率。这种联动来自总概率固定为 1 的要求。

令雅可比矩阵 $J_s[i,m] = \partial k[i]/\partial z[m]$，则

$$J_s = \operatorname{diag}(k) - kk^{\mathrm{T}}. \tag{10.24}$$

若已经知道 $g_k[i] = \partial E/\partial k[i]$，链式法则给出

$$g_z[m] = \frac{\partial E}{\partial z[m]} = \sum_i g_k[i]\frac{\partial k[i]}{\partial z[m]} \tag{10.25}$$

$$= k[m]\left(g_k[m] - \sum_i k[i]g_k[i]\right), \tag{10.26}$$

$$g_z = k \odot \left(g_k - (k^{\mathrm{T}} g_k)\mathbf{1}\right). \tag{10.27}$$

符号 $\odot$ 表示相同形状向量的逐元素乘法，$\mathbf{1}$ 表示全 1 向量。式 (10.27) 只需要一个内积与逐元素运算，就能完成这个导数传播；它与显式构造整个 $C \times C$ 雅可比矩阵后相乘的结果相同。

激活函数与 softmax 的作用范围

Sigmoid 与 ReLU 在这里逐元素作用于各个单元；softmax 同时读取一个样本的所有类别分数，并把它们耦合成归一化概率。前者的局部导数可以逐元素相乘，后者的导数还包含类别之间的交叉项。

10.4 损失函数、反向传播与参数更新

10.4.1 数字识别例子中的损失函数

损失函数（loss function）把模型输出与目标之间的差异转化为一个标量，使训练能够比较不同参数所产生的结果。对于真实数字标签 T 和输出概率 k，一个候选损失是

$$E_1 = 1 - k[T]. \tag{10.28}$$

因为全部类别概率之和为 1，E_1 正好等于分配给所有错误类别的概率质量。它把全部错误类别同等对待，只关心正确类别分量的大小。

数字识别例子实际选用的另一种损失利用类别编号的数值差异：

$$\boxed{E = \sum_{j=0}^{C-1} k[j](j - T)^2.} \tag{10.29}$$

定义 $q[j] = (j - T)^2$，可将其简写为 $E = k^{\mathrm{T}} q$。正确类别的代价 $q[T] = 0$；距离目标数字更远的类别受到更大的惩罚。例如目标为 3 时，类别 2 的代价为 1，而类别 9 的代价为 36。这个数值距离是该例明确采用的损失设计。

式 (10.29) 可以解释为按 k 加权的数字平方距离。若随机变量 J 以概率 $k[j]$ 取值 j，则 $E = \mathbb{E}[(J-T)^2]$。因此，先求平方距离再求加权和，与先求预测数字的均值再平方，是不同的计算。例如 $k = (0.2, 0.5, 0.3)^{\mathrm{T}}$、$T = 1$ 时，该损失为 $0.2 \times 1 + 0.5 \times 0 + 0.3 \times 1 = 0.5$；预测数字的均值却是 1.1，其与目标之差的平方只有 0.01。这里需要保留式 (10.29) 中的运算顺序。

10.4.2 从损失求到 softmax 输入的梯度

目标标签固定时，$q[j] = (j-T)^2$ 是常数。损失对概率输出的导数首先是

$$\frac{\partial E}{\partial k[i]} = q[i] = (i-T)^2. \tag{10.30}$$

再用式 (10.26)，得到

$$\frac{\partial E}{\partial z[m]} = \sum_i q[i]k[i]\big(\delta_{i,m} - k[m]\big) \tag{10.31}$$

$$= k[m]q[m] - k[m]\sum_i k[i]q[i] \tag{10.32}$$

$$= \boxed{k[m]\big(q[m] - E\big).} \tag{10.33}$$

向量形式是 $g_z = k \odot (q - E\mathbf{1})$。对于目标类别 $m = T$，有 $g_z[T] = -k[T]E \le 0$；沿负梯度方向调整该分数，会推动其上升。对于其他类别，调整方向由该类别的代价 $q[m]$ 相对于当前平均代价 E 的大小决定。

继续使用 $z = (0.2, 0.4, 0.8)^{\mathrm{T}}$ 的三类别例子，并取 $T = 1$，则 $q = (1, 0, 1)^{\mathrm{T}}$，有

$$E \approx 0.697936, \qquad g_z \approx (0.074703, -0.210821, 0.136118)^{\mathrm{T}}. \tag{10.34}$$

梯度分量之和为 0，可以由公式直接核验：$\sum_m k[m](q[m]-E) = E - E = 0$。这与 softmax 对分数共同平移不敏感的性质一致。

10.4.3 梯度下降与链式法则

梯度（gradient）记录损失对各个参数的局部变化率。设迭代编号为 r，学习率为 $\epsilon > 0$，一次梯度下降（gradient descent）更新写成

$$\Theta^{(r+1)} = \Theta^{(r)} - \epsilon\,\Delta\Theta, \qquad \Delta\Theta = \nabla_{\Theta} E. \tag{10.35}$$

多个样本共同参与更新时，$\Delta\Theta$ 可以是相应样本梯度的累加量。学习率控制沿梯度方向移动的幅度。实际参数改变量是 $-\epsilon\Delta\Theta$，而 $\Delta\Theta$ 表示更新时所用的梯度。

链式法则把复合函数的导数拆成相邻计算环节的导数。例如 Θ 先影响中间量 u，u 再影响损失 E，就有 $\partial E/\partial\Theta = (\partial E/\partial u)(\partial u/\partial\Theta)$。在向量网络中，同一个变量可能通过多个后继输出影响损失，因此还需要把这些路径的贡献相加。

负梯度方向的局部下降性质来自一阶展开。令 $g = \nabla E(\Theta)$，在可微且步长足够小的情形下，$E(\Theta - \epsilon g) \approx E(\Theta) - \epsilon\|g\|^2$。这解释了更新式中的负号。

10.4.4 单层平方误差的完整更新

10.4.4.1 局部导数与参数导数

先考虑一个标量输入、标量输出的线性模型 $y = wx + b$，目标值为 t，损失取

$$E_{\text{sq}} = \frac{1}{2}(y - t)^2. \tag{10.36}$$

t 是该回归模型的数值目标。这个平方误差直接比较标量预测与目标，与前面的概率加权数字距离损失具有不同含义。因子 $1/2$ 抵消求导时产生的 2，使输出梯度为 $\partial E_{\text{sq}}/\partial y = y - t$。同时，$\partial y/\partial w = x$，$\partial y/\partial b = 1$，因此

$$\frac{\partial E_{\text{sq}}}{\partial w} = \frac{\partial E_{\text{sq}}}{\partial y}\frac{\partial y}{\partial w} = (wx + b - t)x, \tag{10.37}$$

$$\frac{\partial E_{\text{sq}}}{\partial b} = wx + b - t. \tag{10.38}$$

由此得到配套的权重与偏置更新：

$$w^{+} = w - \epsilon(wx + b - t)x, \qquad b^{+} = b - \epsilon(wx + b - t). \tag{10.39}$$

权重梯度包含输入 x，因为同样大小的权重变化，会在不同输入下造成不同大小的输出变化。偏置直接加到输出上，其局部导数是 1。

10.4.4.2 数值核算

取 $x=2$、$t=3$、$w=0.5$、$b=0$、$\epsilon=0.1$。更新前，$y=1$，$E_{\text{sq}}=2$；输出梯度为 -2，权重梯度为 -4，偏置梯度为 -2。同一次更新使用这些在旧参数处计算的梯度，得到 $w^{+}=0.9$、$b^{+}=0.2$。再次前向计算，有 $y^{+}=2$、$E_{\text{sq}}^{+}=0.5$。这个例子把前向求值、误差、局部导数、参数梯度与更新后的结果完整连接起来。

10.4.5 全连接层的三个梯度

10.4.5.1 每个权重如何影响损失

设一层输入 $x\in\mathbb{R}^{N}$，加权和 $\text{fc}=Wx+b\in\mathbb{R}^{M}$，并且已经从后续计算得到 $\delta[i]=\partial E/\partial \text{fc}[i]$。$\delta$ 与该层输出加权和具有相同形状。由式 (10.10)，权重 $W[i,j]$ 对相应输出的局部导数是 $x[j]$，所以

$$\frac{\partial E}{\partial W[i,j]}=\frac{\partial E}{\partial \text{fc}[i]}\frac{\partial \text{fc}[i]}{\partial W[i,j]}=\delta[i]x[j]. \tag{10.40}$$

因此，一个权重的梯度需要两项信息：该权重连接的输入值，以及与输出单元对应的反向梯度。这正是前向信息与反向信息在参数计算中的交汇点。

偏置梯度与输入梯度同样由加权和的局部导数得到。偏置 $b[i]$ 对 $\text{fc}[i]$ 的导数为 1，因此 $\partial E/\partial b[i]=\delta[i]$。一个输入 $x[j]$ 会影响全部输出，所以它的梯度需要汇总所有输出路径：

$$\frac{\partial E}{\partial x[j]}=\sum_{i=0}^{M-1}\delta[i]W[i,j]. \tag{10.41}$$

三类梯度的矩阵形式为

$$\boxed{\nabla_{W}E=\delta x^{\mathrm{T}},\qquad \nabla_{b}E=\delta,\qquad \nabla_{x}E=W^{\mathrm{T}}\delta.} \tag{10.42}$$

10.4.5.2 外积与维度核验

δ 的形状为 $M\times 1$，x^{T} 的形状为 $1\times N$，因此外积 δx^{T} 的形状为 $M\times N$，正好与 W 一致。$W^{\mathrm{T}}\delta$ 的形状为 $N\times 1$，正好与输入一致。这种检查能够直接发现漏写转置或把两个不同梯度

混淆的问题。

对于多个输入分量、多个输出分量，式 (10.40) 没有把全部权重的导数压缩成一个标量。“输出梯度乘输入”在逐分量层面得到一个数，在整层层面则是产生完整权重梯度矩阵的外积。

10.4.6 两层数字识别网络的完整反向链

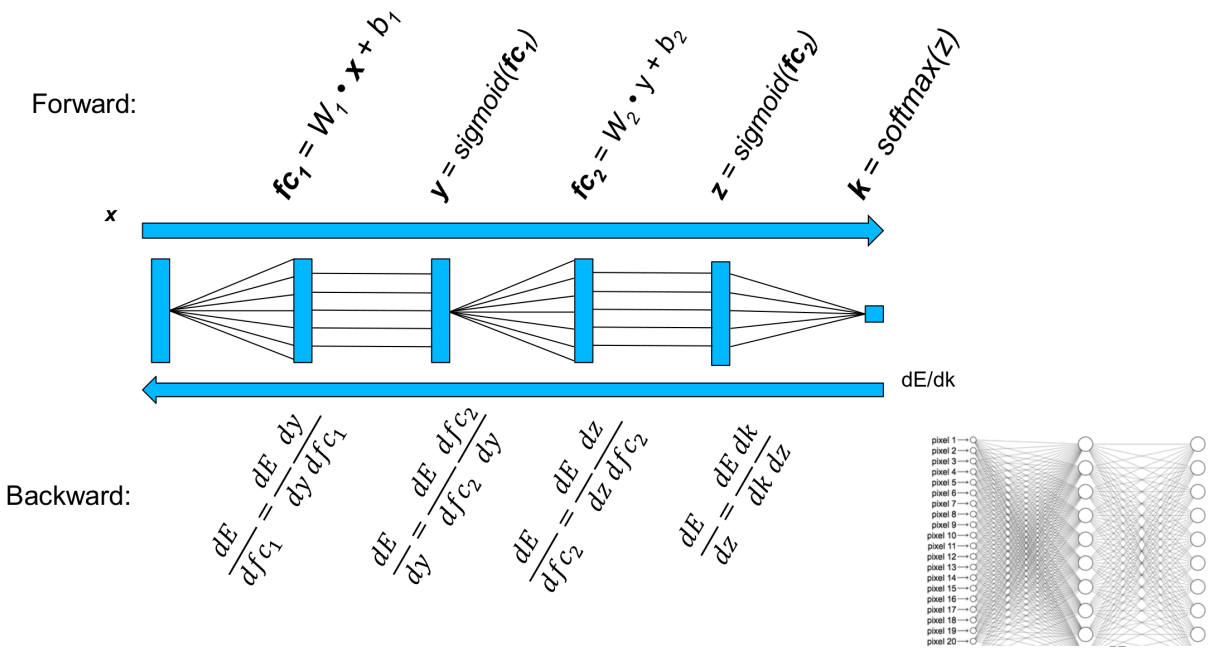


图 10.5 前向从输入逐层计算至概率输出，反向从损失对概率的导数逐层向前传播。图中的第二层也包含 sigmoid。

全部向量均为列向量，δ_ℓ 表示损失对第 ℓ 层加权和的梯度。先在输出端计算 $q[i] = (i - T)^2$、$E = k^{\mathrm{T}} q$。概率层和第二次 sigmoid 的导数依次给出

$$g_z = k \odot (q - E\mathbf{1}), \tag{10.43}$$

$$\delta_2 = g_z \odot z \odot (\mathbf{1} - z). \tag{10.44}$$

δ_2 已经是对第二层加权和 fc_2 的梯度，因此可以计算该层参数梯度：

$$\nabla_{W_2} E = \delta_2 y^{\mathrm{T}}, \qquad \nabla_{b_2} E = \delta_2. \tag{10.45}$$

同时，将梯度通过第二层传回隐藏输出，再通过隐藏层 sigmoid，得到

$$g_y = W_2^{\mathrm{T}} \delta_2, \tag{10.46}$$

$$\delta_1 = g_y \odot y \odot (\mathbf{1} - y), \tag{10.47}$$

$$\nabla_{W_1} E = \delta_1 x^{\mathrm{T}}, \qquad \nabla_{b_1} E = \delta_1. \tag{10.48}$$

在 $784 \to 10 \to 10$ 网络中，$\nabla_{W_2} E$ 为 10×10，$\nabla_{W_1} E$ 为 10×784，两个偏置梯度都为 10×1。所有梯度都与对应参数同形。

式 (10.44) 中的 $z \odot (\mathbf{1} - z)$ 来自第二次 sigmoid。只要采用式 (10.13) 所定义的网络结构，这个因子就属于完整的链式法则。损失直接作用于连续概率 k，反向链不经过输出离散类别编号的 $\arg\max$。

10.4.7 前向缓存、梯度传递与参数更新的时序

计算 $\nabla_{W_1} E$ 需要第一层输入 x，计算 $\nabla_{W_2} E$ 需要前向得到的隐藏输出 y。对于 sigmoid，局部导数也可以直接由 y 和 z 得到；输出概率 k 则用于损失及其初始梯度。反向传播因此依赖前向过程中生成的信息。

同一次更新的梯度应当对应同一组参数。先用当前参数完成前向，再由这些中间结果完成反向，得到各层的参数梯度，最后统一进行 $W_\ell \leftarrow W_\ell - \epsilon \nabla_{W_\ell} E$ 与 $b_\ell \leftarrow b_\ell - \epsilon \nabla_{b_\ell} E$。传播到前一层时使用的 W_2^{T}，仍是本次前向求值时的权重。

全连接层反向计算的分工

δx^{T} 计算当前层的权重梯度，δ 给出偏置梯度，$W^{\mathrm{T}}\delta$ 则把梯度传给前一层。激活函数的导数在相应的激活节点处逐元素相乘。参数更新需要前向输入，跨层传递还需要当前层权重。

10.4.8 两层网络的数值核算

采用 $2 \to 2 \to 3$ 的小网络，保留“全连接、sigmoid、全连接、sigmoid、softmax”的结构，以及式 (10.29) 的损失。以下数值用于逐项检验计算公式。取输入、真实类别与初始参数为

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad T = 1, \quad W_1 = \begin{bmatrix} 0.1 & -0.2 \\ 0.3 & 0.25 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.05 \\ -0.1 \end{bmatrix}, \tag{10.49}$$

$$W_2 = \begin{bmatrix} 0.4 & -0.3 \\ 0.1 & 0.2 \\ -0.2 & 0.5 \end{bmatrix}, \qquad b_2 = \begin{bmatrix} 0 \\ 0.1 \\ -0.05 \end{bmatrix}. \tag{10.50}$$

网络共有 $2\times 2 + 2 + 3 \times 2 + 3 = 15$ 个参数。第一层加权和为 $\mathrm{fc}_1 = (-0.25, 0.7)^{\mathrm{T}}$，继续逐层代入得到表 10.5。表中向量横向列出，均按列向量参与实际计算。

表 10.5 小网络一次前向计算的结果，保留六位小数。

中间量	分量
$y = \sigma(\mathrm{fc}_1)$	$(0.437823,\ 0.668188)$
$\mathrm{fc}_2 = W_2 y + b_2$	$(-0.025327,\ 0.277420,\ 0.196529)$
$z = \sigma(\mathrm{fc}_2)$	$(0.493669,\ 0.568914,\ 0.548975)$
$k = \mathrm{softmax}(z)$	$(0.318978,\ 0.343906,\ 0.337116)$
q 与 E	$q = (1, 0, 1)$，$E = 0.656094$

最大概率的类别已经是 1，但其他类别仍获得较多概率质量，因此损失仍明显大于 0。反向计算得到

$$g_z \approx (0.109698, -0.225635, 0.115936)^{\mathrm{T}}, \tag{10.51}$$

$$\delta_2 \approx (0.027420, -0.055337, 0.028706)^{\mathrm{T}}, \tag{10.52}$$

$$\delta_1 \approx (-0.00007552, -0.00109537)^{\mathrm{T}}. \tag{10.53}$$

相应的两个权重梯度是

$$\nabla_{W_2} E \approx \begin{bmatrix} 0.012005 & 0.018322 \\ -0.024228 & -0.036976 \\ 0.012568 & 0.019181 \end{bmatrix}, \qquad \nabla_{W_1} E \approx \begin{bmatrix} -0.00007552 & -0.00015104 \\ -0.00109537 & -0.00219074 \end{bmatrix}. \tag{10.54}$$

偏置梯度分别等于 δ_2 和 δ_1。以 $\epsilon = 0.1$ 同时更新全部权重与偏置后，重新前向计算得到

$E^{+} \approx 0.655334$。这个下降量来自全部 15 个参数的共同调整。

还可以对任意参数 θ_a 使用中心差分

$$\frac{\partial E}{\partial \theta_a} \approx \frac{E(\Theta + h e_a) - E(\Theta - h e_a)}{2h}, \tag{10.55}$$

其中 e_a 只在待检验参数位置为 1。用双精度、$h = 10^{-6}$ 检查全部 15 个参数，解析梯度与中心差分的最大绝对差约为 8.3×10^{-11}。该数值核验同时检查了两个 sigmoid 因子、softmax 的交叉项、权重外积与偏置梯度。

10.5 训练组织与并行计算

10.5.1 训练轮次与一次参数更新

一个训练轮次（epoch）表示遍历整个训练集一次。一次参数更新则表示计算一组梯度后，执行一次 $\Theta \leftarrow \Theta - \epsilon \Delta\Theta$。一个训练轮次中包含多少次更新，取决于每次更新使用多少个样本。

设训练集为 $\mathcal{D} = \{(x_s, T_s)\}_{s=1}^{D}$。采用梯度求和约定，全数据目标为

$$J_{\mathcal{D}}(\Theta) = \sum_{s=1}^{D} E_s(\Theta), \qquad \nabla J_{\mathcal{D}}(\Theta) = \sum_{s=1}^{D} \nabla E_s(\Theta). \tag{10.56}$$

这里 D 是样本数，N 仍表示一个输入的维度。各个样本共享同一组网络参数，但具有各自的输入、标签、前向中间量及参数梯度。

10.5.2 全批量与小批量梯度下降

10.5.2.1 全批量：累积整个训练集后更新

全批量梯度下降（full-batch gradient descent）先将梯度累加器置零，再依次处理所有带标签样本。对每个样本，读取输入、前向计算输出、根据标签计算损失、反向求参数梯度，并把梯度加入累加器。整个训练集处理完毕后，执行一次参数更新。

整个累加过程中参数保持不变。因此，累加量正好对应式 (10.56) 在这一组参数处的梯度。一个 epoch 对应一次参数更新。这里的“完整梯度”是给定有限训练集目标的梯度。

10.5.2.2 小批量：按样本组交替求梯度和更新

小批量随机梯度下降（mini-batch stochastic gradient descent）把数据分成若干批量，每个批量单独清零梯度累加器、完成前向与反向、累加梯度并更新参数。设某批样本索引组成集合 $\mathcal{B}$，批量大小为 B，对应梯度为

$$\Delta\Theta_{\mathcal{B}} = \sum_{s\in\mathcal{B}} \nabla E_s(\Theta), \qquad \Theta^{+} = \Theta - \epsilon\Delta\Theta_{\mathcal{B}}. \tag{10.57}$$

下一批样本使用更新后的参数继续计算。与全批量相比，这种方式在完整遍历训练集之前就会多次调整参数。批量大小还影响一次梯度估计利用的信息量，以及可同时处理的样本数量。

表 10.6 每次更新涉及的样本范围与更新次数。单样本情形作为小批量公式的 $B = 1$ 特例。

方式	一次更新使用的样本	一次完整遍历中的更新次数
全批量	全部 D 个	1
小批量	一组 B 个	D/B，当 B 整除 D 时
单样本	1 个	D

以 60,000 个样本、$B = 100$ 为例，一个 epoch 包含 600 个批量，因此有 600 次参数更新。若不整除且保留最后一个较小批量，更新次数为 $\lceil D/B \rceil$。

10.5.2.3 梯度求和与梯度求平均

上述更新采用梯度求和。若目标改为批量平均损失 $\bar{J}_{\mathcal{B}} = B^{-1}\sum_{s\in\mathcal{B}} E_s$，则梯度也需要除以 B。这是同一批样本目标的尺度变化。

采用相同学习率时，平均梯度产生的参数改变量是求和梯度的 $1/B$。若要求两种写法产生相同的一步更新，应配套调整学习率。反向公式与参数更新之间，只需按所定义的目标处理一次这个归一化因子。

10.5.3 把小批量写成矩阵运算

按列组织一批样本后，逐样本的前向与反向过程可写成矩阵乘法、逐元素运算和归约。以下采用梯度求和约定。

10.5.3.1 按列组织样本的前向计算

把同一批中的 B 个输入作为矩阵 $X \in \mathbb{R}^{N\times B}$ 的各列。对于隐藏维度 M、类别数 C 的两层网络，前向可以写成

$$F_1 = W_1 X + b_1 \mathbf{1}_B^{\mathrm{T}}, \qquad Y = \sigma(F_1), \tag{10.58}$$

$$F_2 = W_2 Y + b_2 \mathbf{1}_B^{\mathrm{T}}, \qquad Z = \sigma(F_2), \tag{10.59}$$

$$K_{:,s} = \mathrm{softmax}(Z_{:,s}), \qquad s = 1, \dots, B. \tag{10.60}$$

其中 $Y \in \mathbb{R}^{M\times B}$，$Z, K \in \mathbb{R}^{C\times B}$，$\mathbf{1}_B$ 是长度为 B 的全 1 向量。偏置项把同一个偏置向量加到每一列；softmax 对每个样本的类别列分别归一化。

10.5.3.2 多样本外积之和变成矩阵乘法

令 $G_2 \in \mathbb{R}^{C\times B}$ 的第 s 列是样本 s 的 δ_2，$G_1 \in \mathbb{R}^{M\times B}$ 的第 s 列是其 δ_1。将单样本外积按样本求和，可得

$$\nabla_{W_2} J_{\mathcal{B}} = G_2 Y^{\mathrm{T}}, \qquad \nabla_{b_2} J_{\mathcal{B}} = G_2 \mathbf{1}_B, \tag{10.61}$$

$$\nabla_{W_1} J_{\mathcal{B}} = G_1 X^{\mathrm{T}}, \qquad \nabla_{b_1} J_{\mathcal{B}} = G_1 \mathbf{1}_B. \tag{10.62}$$

例如，$G_1 X^{\mathrm{T}}$ 的第 (i, j) 个元素是 $\sum_s \delta_{1,s}[i] x_s[j]$，正好是这一权重对批内全部样本的梯度累加量。传向隐藏层的梯度同样可以一次写成 $W_2^{\mathrm{T}} G_2$，随后与 $Y \odot (\mathbf{1} - Y)$ 逐元素相乘。

表 10.7 批量网络中的主要计算形式。由前向与梯度矩阵公式直接归纳。

计算阶段	代表表达式	运算结构
全连接前向	$W_1 X$、$W_2 Y$	矩阵乘法
激活与局部导数	$\sigma(F_1)$、$Y \odot (\mathbf{1} - Y)$	逐元素计算
Softmax 与损失	每列指数和、加权代价和	按类别归约
跨层反向	$W_2^{\mathrm{T}} G_2$	矩阵乘法
权重梯度	$G_2 Y^{\mathrm{T}}$、$G_1 X^{\mathrm{T}}$	矩阵乘法
偏置梯度	$G_\ell \mathbf{1}_B$	按样本归约

批量把多个向量计算组织到同一个矩阵运算中，从而暴露样本之间的并行性。与此同时，激活值和反向梯度也多出一个大小为 B 的样本维度，需要保存更多中间数据。批量大小因此同时关联梯度估计、并行组织与中间存储；实际吞吐还取决于矩阵尺寸、存储访问与硬件资源。

10.6 泛化、模型容量与任务适配

10.6.1 训练集、验证集与测试集

泛化（generalization）是模型在新输入上仍能产生合适结果的能力。训练集用于计算参数更新；验证集用于选择设置和停止训练的位置；测试集用于检查模型对新输入的表现。三者在训练与评价过程中的用途不同。

训练损失持续下降，说明模型在参与参数更新的数据上拟合得更好。图 10.6 同时给出训练损失与验证损失随 epoch 变化的曲线：训练损失继续趋近于零，而验证损失先下降，随后趋于平缓并缓慢上升。这个变化说明，继续改进训练集上的拟合，可能无法同步改进新数据上的结果。

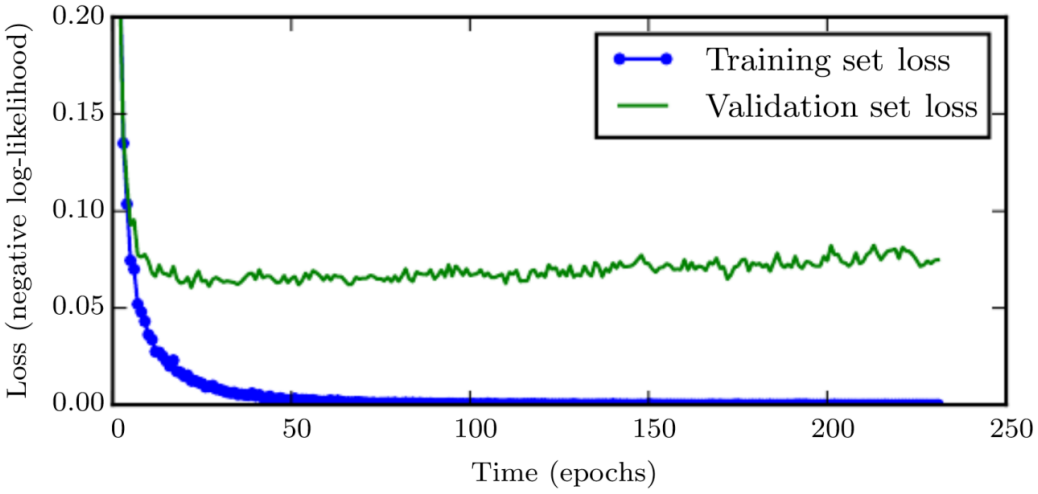


图 10.6 训练损失与验证损失随训练轮次的变化。纵轴为负对数似然损失；该曲线用于说明泛化现象，与前面数字距离损失的数值例题分开讨论。

验证数据为训练停止位置提供依据：参数更新继续使用训练集，停止点与设置则根据验证表现选择。测试数据用于最终检查泛化。由此，优化训练目标、选择模型和评估新输入表现形成相互关联而职责明确的环节。

10.6.2 模型容量、欠拟合与过拟合

10.6.2.1 多项式拟合的类比

模型容量（capacity）描述模型可以表达多复杂的关系。一个简单模型可能连训练样本中的主要变化都无法拟合，表现为欠拟合（underfitting）。增加表达能力后，模型可以捕捉这些变化；若过于贴合训练样本的具体细节，则可能在未见输入上出现较大误差，表现为过拟合（overfitting）。

多项式拟合提供了一个直观例子。过低阶的曲线难以追随数据趋势；合适的阶数能够表达主要关系；过高阶的曲线可以更加贴近已知点，同时在已知点之间或样本范围之外产生较大振荡。这个类比说明，训练误差的改善需要结合新数据表现来判断。

10.6.2.2 训练轮次与模型容量是两个观察轴

图 10.7 的横轴是模型容量，纵轴是误差。示意曲线中，容量增加时训练误差下降，而泛化误差在合适容量附近较低，继续增大容量后又上升。两条曲线的差异称为泛化差距（generalization gap）。

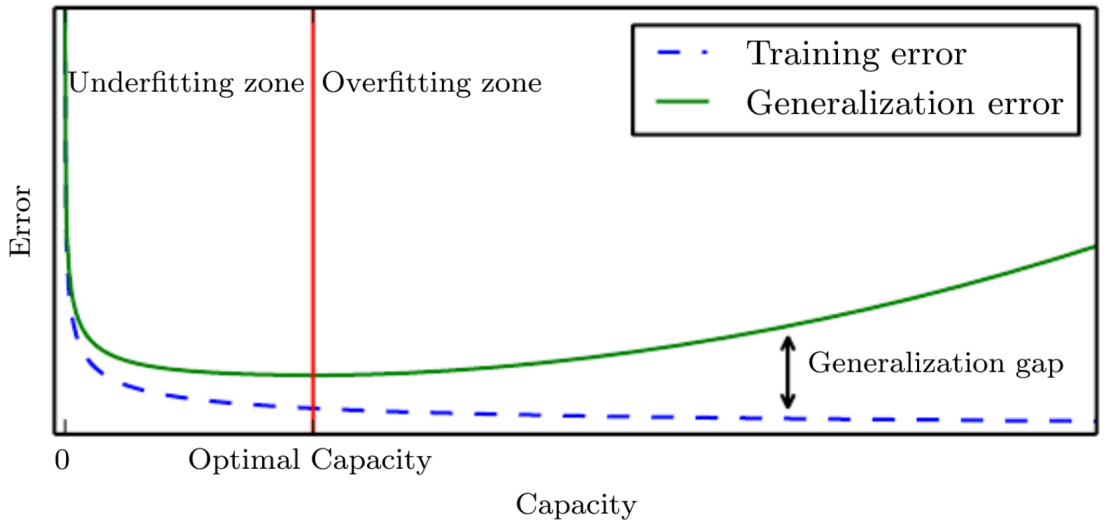


图 10.7 模型容量与训练误差、泛化误差之间的示意关系。横轴是容量，不是训练轮次。

训练轮次衡量同一训练过程推进了多久，模型容量描述结构能够表达多复杂的关系。前一幅曲线用于理解训练停止位置，后一幅曲线用于理解容量选择。两者都需要关注训练数据之外的表现，但对应的可调对象不同。

10.6.3 从权重图像理解学习到的特征

一层直接连接图像像素的网络，其某个单元具有 784 个输入权重。按照像素展开时的顺序，把这些权重重新排列成 28×28 的网格，就可以观察这个单元如何对不同像素位置加权。这种可视化把权重向量重新映射回输入的空间结构。

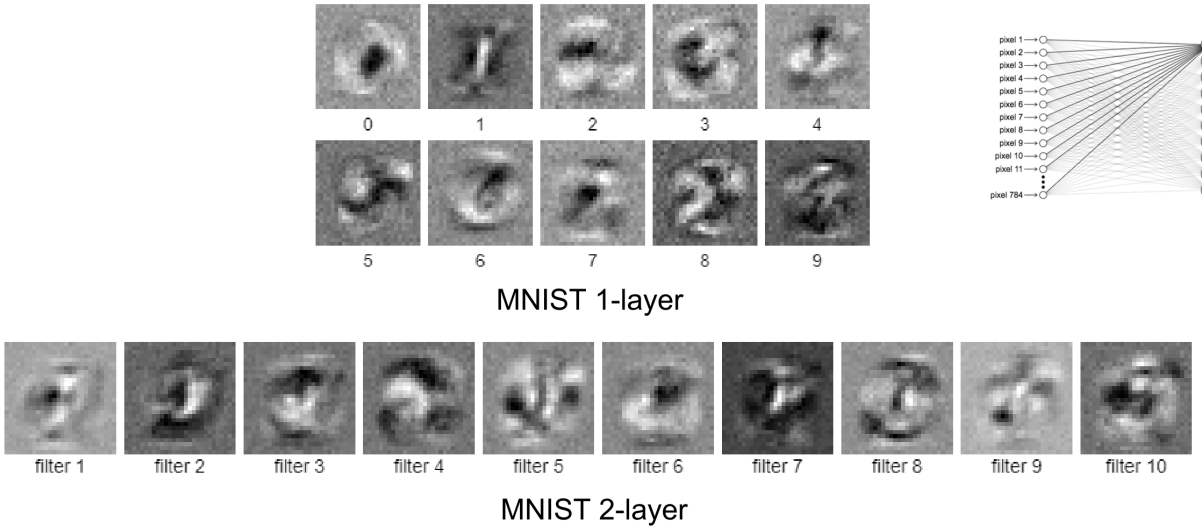


图 10.8 单层 MNIST 网络的类别权重图与两层网络的特征权重图。上方以数字 0–9 标注，下方以特征编号标注。

直接面向类别的权重图可以呈现与相应数字外观有关的结构。两层网络的隐藏单元则提供后续输出层要组合的中间特征，因此图中用特征编号而非数字类别为它们命名。这个区别与前面的表示学习一致：输出判断可以建立在多个中间特征的组合上。

10.6.4 无免费午餐定理的任务层面含义

无免费午餐定理（no free lunch theorem）讨论算法在广泛可能问题上的平均表现。一种概括性表述是：当对所有可能的输入生成分布取平均时，各种分类算法在未观测输入上的平均错误率相同。这里的关键限定是*对所有可能分布取平均*。

针对一个具体任务，模型面对的是具有特定结构的数据。网络结构、特征表示和训练设置都需要与这种结构相适配。因而，“从数据中学习”仍然包含任务相关的模型设计与选择；泛化评价用于检验这些选择是否在新输入上有效。平均性能结论依赖所选问题空间及平均方式，并不意味着各算法在某一个具体数据集上的表现相同。

10.7 关键公式与结构关系

10.7.1 从输入到参数更新

一个全连接层先计算 $\mathrm{fc} = Wx + b$，再由激活函数产生下一层输入。多个这样的层组合后，网络获得逐层变换数据表示的能力。分类输出经 softmax 归一化为概率，损失函数再把概率与标签联系起来。

本讲两层网络的计算链

设 σ 为 sigmoid，T 为数字标签，$q[j] = (j - T)^2$。前向计算为

$$\begin{aligned} \mathrm{fc}_1 &= W_1 x + b_1, & y &= \sigma(\mathrm{fc}_1), \\ \mathrm{fc}_2 &= W_2 y + b_2, & z &= \sigma(\mathrm{fc}_2), \\ k &= \mathrm{softmax}(z), & E &= k^{\mathrm{T}} q. \end{aligned}$$

反向传播按相反顺序传递导数：

$$\begin{aligned} g_z &= k \odot (q - E\mathbf{1}), \\ \delta_2 &= g_z \odot z \odot (\mathbf{1} - z), \\ \delta_1 &= (W_2^{\mathrm{T}} \delta_2) \odot y \odot (\mathbf{1} - y). \end{aligned}$$

因此 $\nabla_{W_2} E = \delta_2 y^{\mathrm{T}}$、$\nabla_{W_1} E = \delta_1 x^{\mathrm{T}}$，偏置梯度分别为 δ_2 与 δ_1。参数按 $\Theta \leftarrow \Theta - \epsilon \nabla_\Theta E$ 更新。

10.7.2 全连接层的三个梯度

对于 $W \in \mathbb{R}^{M \times N}$、$x \in \mathbb{R}^N$ 和上游梯度 $\delta = \partial E / \partial \mathrm{fc} \in \mathbb{R}^M$，三个基本关系为

$$\boxed{\nabla_W E = \delta x^{\mathrm{T}}}, \qquad \boxed{\nabla_b E = \delta}, \qquad \boxed{\nabla_x E = W^{\mathrm{T}} \delta}.$$

权重梯度把前向输入与反向导数组合成外积；输入梯度把误差信息传回上一层；偏置梯度直接接收该层导数。对一批样本求和后，外积累加成为矩阵乘法，偏置梯度成为沿样本维的求和。

表示学习建立输入与输出的关系；反向传播计算导数，梯度下降据此更新参数，小批量规定每次更新的样本范围。验证与测试用于评估模型对新输入的泛化能力。

第 11 讲

Computation in Convolutional Neural Networks

11.1 卷积网络的结构与局部计算

卷积神经网络（Convolutional Neural Network，CNN）用局部加权求和提取特征，再通过非线性变换和空间下采样构成多层计算。实现这类网络时，需要同时确定三个相互联系的对象：每个输出值依赖哪些输入，张量中的元素如何存放，以及哪些输出可以交给不同线程独立计算。卷积层的数学定义决定数据依赖，而线程映射和存储组织决定这些依赖如何在图形处理器（Graphics Processing Unit，GPU）上得到高效执行。

11.1.1 从全连接计算到局部连接

多层感知机（Multi-Layer Perceptron，MLP）的全连接层把输入写成向量 $\mathbf{x} \in \mathbb{R}^n$。若该层有 m 个输出，则第 j 个输出为 $a_j = \phi(\sum_{i=0}^{n-1} V_{j,i} x_i + \beta_j)$，其中 $V \in \mathbb{R}^{m \times n}$ 是权重矩阵，β_j 是偏置，ϕ 是逐元素激活函数。一个输出连接全部 n 个输入，因此该层有 mn 个权重和 m 个偏置。

例如，把一幅 28×28 的灰度图像展平为 784 维向量，先连接到 10 个隐藏神经元，再连接到 10 个输出神经元，总参数量为 $(784 \times 10 + 10) + (10 \times 10 + 10) = 7\,960$。当图像增大到 250×250，输入维数为 $62\,500$；若隐藏层也有 $62\,500$ 个神经元，仅第一层就需要 $62\,500^2 = 3\,906\,250\,000$ 个权重。按每个权重占 4 字节计算，权重存储约为 15.625 GB，还没有计入偏置、激活值以及训练时的其他数组。

图像中的局部边缘、纹理等模式通常可以通过一个较小区域内的像素关系描述。卷积层因此让每个输出只连接输入中的局部窗口，形成稀疏交互（sparse interaction）。一个 $K \times K$ 的窗口在单个输入通道上只需要 K^2 次对应元素相乘，而不需要访问整幅图像。这个窗口是该输出相对于上一层的感受野（receptive field）。

局部连接与多层组合可以同时存在。在窗口完整落入有效输入范围的位置，若每层都使用步幅为 1 的 3×3 卷积，第一层输出依赖输入中的 3×3 区域；第二层把第一层相邻的 3×3

个位置组合起来，这些位置对应原输入中的 5×5 区域。继续堆叠 ℓ 层时，沿任一空间轴的感受野宽度为 $r_\ell = 1+\ell(K-1)$。这个关系假设各层核宽均为 K、没有空洞且步幅均为 1；它说明局部运算可以逐层建立较大范围的数据依赖。

11.1.2 参数共享与平移等变性

卷积层还把同一组权重应用到不同空间位置，这称为参数共享（parameter sharing）。对于固定的输入通道和输出通道，窗口向右或向下移动时，卷积核系数保持不变。输出位置改变的是取用的输入数据，而不改变模型参数。若输入有 C 个通道、输出有 M 个通道、每个二维核为 $K\times K$，则权重总数为 MCK^2；每个输出通道再有一个偏置时，参数总数为 MCK^2+M。这里没有乘输出的空间位置数。

例如，单通道图像经过 16 个 5×5 核产生 16 张特征图，只需 $16\times1\times25=400$ 个权重，含偏置时为 416 个参数。只要输入仍有一个通道，这组权重就能作用于不同高度和宽度的图像，输出特征图的尺寸随之改变。若后续把全部空间位置展平再送入一个固定大小的全连接层，则该全连接层的输入维数仍然固定；因此，卷积核可复用于不同图像尺寸，与整个网络能否接收任意尺寸，是两个需要分别检查的条件。

参数共享还产生平移等变性（translation equivariance）：输入中的局部模式平移后，对应响应的位置随之平移。以单通道、步幅为 1 的二维卷积为例，记其算子为 $\mathcal{C}_F$，令 Δ_h,Δ_w 为整数。二维平移算子满足 $(T_\Delta X)[h,w]=X[h-\Delta_h,w-\Delta_w]$，卷积核不随位置改变，且所比较位置的取值和边界处理均有效，则

$$\mathcal{C}_F(T_\Delta X)[h,w]=\sum_{p,q}F[p,q]X[h+p-\Delta_h,w+q-\Delta_w]=(T_\Delta\mathcal{C}_F(X))[h,w]. \tag{11.1}$$

左右两边使用完全相同的乘积项，只是描述坐标不同。等变性保留特征出现的位置；如果需要输入平移后最终分类结果保持不变，还需要后续的汇聚和分类过程。有限图像的裁剪、填充方式，以及步幅大于 1 的下采样，会限制这种逐像素平移对应关系。

11.1.3 空间维度、通道与特征图

空间维度描述卷积窗口移动的方向，通道（channel）描述同一位置上并列的不同信号。一幅彩色图像有高度和宽度两个空间轴，在每个位置有红、绿、蓝三个颜色通道。三通道二维卷积的窗口沿两个空间轴移动，并在三个通道之间求和；它不会因为存在三个通道就变成沿三个空间轴移动的三维卷积。

表 11.1 空间轴与通道的不同含义

数据对象	卷积移动的轴	通道的含义
音频波形	一维时间轴	单路或多路音频信号
骨架动作序列	一维时间轴	不同关节的角度或坐标分量
彩色图像	二维高度、宽度	同一像素的红、绿、蓝分量
时频表示	二维时间、频率	同一时频位置的不同测量分量
体数据	三维空间坐标	同一体素的一个或多个测量分量
彩色视频	二维空间与一维时间	同一时空位置的颜色分量

输出通道也称输出特征图（feature map）。每个输出通道有一整组覆盖所有输入通道的权重：对各输入通道分别计算局部加权和，再把这些部分结果相加，得到一个输出值。因而 C 个输入通道与 M 个输出通道之间共有 $C \times M$ 个二维卷积核，但最终输出是 M 张二维特征图。不同的输出通道拥有不同权重，可以提取不同模式；同一输出通道的权重在所有图像和空间位置上共享。

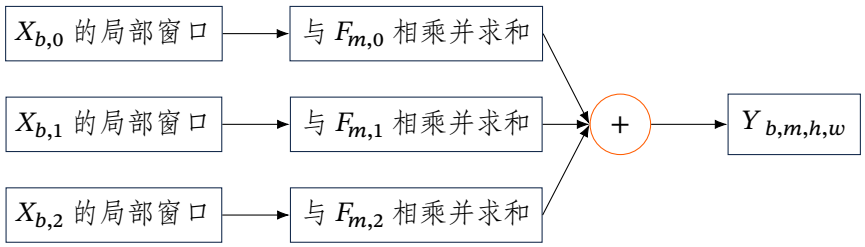


图 11.1 三个输入通道的贡献相加，生成固定输出通道中的一个空间位置。对每个新的输出通道，都需要另一组覆盖三个输入通道的卷积核。

11.1.4 卷积、池化与分类层的组合

卷积层产生包含空间位置信息的特征图，激活函数对其中每个标量执行非线性变换，池化（pooling）则把局部区域汇聚为较少的输出值。多次组合这些操作后，网络可以将较大感受野中的信息交给全连接层或其他分类结构。每个模块都接收和产生张量，上一层输出的通道数与空间尺寸直接决定下一层计算的输入形状。

经典卷积网络（LeNet-5）的一个形状序列是：32×32 单通道图像经过卷积得到 6×28×28，下采样为 $6\times14\times14$，再卷积为 $16\times10\times10$，再下采样为 $16\times5\times5$，之后形成 120 维、84 维的表示并产生 10 类输出。空间尺寸的收缩来自有效卷积和下采样，通道数的变化来自输出卷积核组数。较深的网络可以采用更多卷积层，也可以引入跨层连接或上采样分支；这些网络结构改变了张量之间的连接方式，卷积层内部仍可从同一套局部求和与通道组合规则出发实现。

卷积的局部连接主要减少每个输出所需的输入范围，参数共享主要减少独立权重数量，池化主要减少空间位置数。这三种机制作用于不同对象，因此需要分别分析参数量、计算量和中间张量存储量。

11.2 卷积与池化的前向计算

11.2.1 张量形状与求和维度

设一个批次（batch）包含 B 幅图像，每幅图像有 C 个输入通道，每个通道高 H、宽 W。卷积层产生 M 个输出通道，每个输出通道使用一组 C 个 $K\times K$ 的二维核。输入、权重和卷积输出分别记为

$$X\in\mathbb{R}^{B\times C\times H\times W},\qquad F\in\mathbb{R}^{M\times C\times K\times K},\qquad Y\in\mathbb{R}^{B\times M\times H_{\mathrm{o}}\times W_{\mathrm{o}}}. \tag{11.2}$$

宽度始终使用 W，权重始终使用 F，从而避免同一个字母同时表示两种对象。所有数组下标均从 0 开始。表11.2给出了各索引及其合法范围。

先考虑步幅（stride）为 1、无填充（padding）、无空洞、每个输出通道组合全部输入通道的情形，并把偏置与激活留到后面单独处理。固定输出坐标 (b,m,h,w) 后，卷积计算为

$$Y_{b,m,h,w}=\sum_{c=0}^{C-1}\sum_{p=0}^{K-1}\sum_{q=0}^{K-1}F_{m,c,p,q}\,X_{b,c,h+p,w+q}. \tag{11.3}$$

表 11.2 卷积前向计算的索引约定

索引	含义	范围
b	批次中的图像编号	$0 \le b < B$
c	输入通道编号	$0 \le c < C$
m	输出通道编号	$0 \le m < M$
h, w	输出的行、列坐标	$0 \le h < H_{\mathrm{o}},\ 0 \le w < W_{\mathrm{o}}$
p, q	核内的行、列偏移	$0 \le p < K,\ 0 \le q < K$

这里的 h, w 确定窗口左上角；p, q 在窗口内部移动；c 遍历同一位置上的全部输入通道。一个输出值是长度为 CK^2 的内积。改变 m 时，输入窗口不变而权重组改变；改变 h, w 时，权重组不变而输入窗口移动；改变 b 时，处理另一幅图像而继续使用同一组参数。

式(11.3)在信号处理的严格命名中属于互相关（cross-correlation），因为核内的两个索引没有翻转。若采用翻转核的卷积约定，同一个窗口内应使用 $F_{m,c,K-1-p,K-1-q}$。CNN 的前向计算通常把式(11.3)直接称为卷积。实现时应保持公式中的索引顺序；核不对称时，额外翻转会改变输出。例如，核 $\left(\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right)$ 翻转后为 $\left(\begin{smallmatrix}4&3\\2&1\end{smallmatrix}\right)$，这两组权重与同一输入窗口的内积一般不同。

11.2.2 有效卷积的输出尺寸

有效卷积（valid convolution）只计算卷积核完全位于输入内部的位置。对于高度方向，窗口左上角从 $h = 0$ 开始，最下面一个核元素的行坐标是 $h + K - 1$。要求该坐标不超过 $H - 1$，就得到 $0 \le h \le H - K$，因此共有 $H - K + 1$ 个合法起点。宽度方向同理，故

$$H_{\mathrm{o}} = H - K + 1, \qquad W_{\mathrm{o}} = W - K + 1, \qquad H \ge K,\ W \ge K. \tag{11.4}$$

这个计数包含起点 0 和终点 $H - K$，所以必须加 1。输出数组使用紧凑坐标：$Y_{b,m,0,0}$ 对应输入左上角的第一个完整窗口。若把输出响应画在核中心所对应的原图位置，两者的坐标原点不同，但它们代表相同的局部计算。

例如，7×7 输入与 5×5 核产生 3×3 输出；3×3 输入与 2×2 核产生 2×2 输出；86×86 输入与 7×7 核产生 80×80 输出。增加输出通道数只改变输出张量中的 M，不会改变这两个空间尺寸。步幅、填充或核内采样间隔发生变化时，需要重新按窗口合法起点计数，不能直接沿用式(11.4)。

对于高为 K_h、宽为 K_w 的矩形核，同样的计数分别给出 $H_{\mathrm{o}} = H - K_h + 1$ 和 $W_{\mathrm{o}} = W - K_w + 1$，每个输出的内积长度相应为 CK_hK_w。下面的推导与代码统一使用方形核，以一个 K 表示两个方向的大小。

11.2.3 步幅与零填充的尺寸关系

设矩形核的高、宽分别为 K_h, K_w，两个方向的步幅为正整数 S_h, S_w，输入上下各补 P_h 行零、左右各补 P_w 列零，其中填充量为非负整数。输出位置 (h, w) 的核内偏移 (p, q) 对应原输入坐标

$$i = hS_h - P_h + p, \qquad j = wS_w - P_w + q. \tag{11.5}$$

当 i, j 位于原输入范围内时读取 $X_{b,c,i,j}$，否则该项使用零。填充后的高度为 $H + 2P_h$，最后一个完整窗口的起点必须满足 $hS_h + K_h \le H + 2P_h$。在两个方向均至少容纳一个完整窗口时，输出尺寸为

$$H_o = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \qquad W_o = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1. \tag{11.6}$$

例如，$H = 6, W = 8$，使用 3×3 核、两个方向步幅均为 2，四侧各填充 1，输出为 3×4。左上输出的窗口从原输入坐标 $(-1, -1)$ 开始；右下输出的窗口从 $(3, 5)$ 开始。步幅限定了实际采样的起点，因此填充后末端仍可能有未被最后一个窗口覆盖的位置。

这些公式采用核内相邻元素连续采样的约定。以下算例、张量展开和代码继续使用步幅 1、无填充、无空洞的方形核；将它们推广到其他步幅或填充时，必须同时修改输出尺寸和输入坐标映射。

11.2.4 逐通道累加的完整算例

考虑一幅三通道图像，$C = 3$、$H = W = 4$，使用 $K = 3$ 的核组产生一个输出通道，即 $M = 1$。固定批次编号后，把三个输入通道简写为 X_0, X_1, X_2，相应核片简写为 F_0, F_1, F_2。同时固定唯一的输出通道 $m = 0$，并简写 $Y_{h,w} = Y_{b,0,h,w}$。输入与核组为：

$$X_0 = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 1 & 1 & 3 & 2 \\ 0 & 2 & 2 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix}, \quad X_1 = \begin{bmatrix} 0 & 2 & 1 & 0 \\ 0 & 3 & 2 & 1 \\ 1 & 1 & 0 & 2 \\ 2 & 1 & 0 & 3 \end{bmatrix}, \quad X_2 = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 2 \\ 3 & 3 & 2 & 0 \\ 1 & 3 & 2 & 0 \end{bmatrix},$$

$$F_0 = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 3 \\ 2 & 1 & 0 \end{bmatrix}, \qquad F_1 = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix}, \qquad F_2 = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 2 \\ 1 & 2 & 1 \end{bmatrix}. \tag{11.7}$$

输出高和宽均为 $4-3+1=2$。先求左上角的 $Y_{0,0}$，每个通道都取输入左上角的 3×3 窗口。按照窗口的行分别相乘再相加，可以直接看见每个通道如何贡献到同一个标量：

$$\begin{aligned} v_0 &= (1\cdot 1+2\cdot 1+0\cdot 1)+(1\cdot 2+1\cdot 2+3\cdot 3)+(0\cdot 2+2\cdot 1+2\cdot 0)\\ &= 3+13+2=18,\\ v_1 &= (0\cdot 1+2\cdot 2+1\cdot 3)+(0\cdot 1+3\cdot 1+2\cdot 0)+(1\cdot 3+1\cdot 0+0\cdot 1)\\ &= 7+3+3=13,\\ v_2 &= (1\cdot 0+2\cdot 1+1\cdot 1)+(0\cdot 1+1\cdot 0+3\cdot 2)+(3\cdot 1+3\cdot 2+2\cdot 1)\\ &= 3+6+11=20. \end{aligned} \tag{11.8}$$

三个部分和合并后，$Y_{0,0}=18+13+20=51$。累加变量在开始计算这个输出前初始化一次为 0；处理完第一个通道后为 18，处理完第二个通道后为 31，处理完第三个通道后才写回最终值 51。如果在每个通道开始时清零并覆盖输出，就会丢失先前通道的贡献。

将窗口移到其余三个合法起点，三个通道各自贡献的 2×2 矩阵以及最终结果为

$$Y=\begin{bmatrix}18 & 23\\ 20 & 16\end{bmatrix}+\begin{bmatrix}13 & 14\\ 20 & 17\end{bmatrix}+\begin{bmatrix}20 & 13\\ 20 & 15\end{bmatrix}=\begin{bmatrix}51 & 50\\ 60 & 48\end{bmatrix}. \tag{11.9}$$

这四个输出使用相同的核组。它们的输入窗口彼此重叠，但输出地址不同；这种输入可复用、输出可独立写入的结构，正是后续并行实现的依据。

11.2.5 参数量、乘加次数与张量存储量

一次乘加（multiply-accumulate，MAC）表示将一个乘积累加到已有的和。式(11.3)中的每个输出需要 CK^2 次乘加，总输出元素数为 $BMH_{\mathrm{o}}W_{\mathrm{o}}$。如果把一次乘加按一次乘法和一次加法合计为两次浮点运算（floating-point operations，FLOPs），则

$$N_{\mathrm{MAC}}=BMH_{\mathrm{o}}W_{\mathrm{o}}CK^2,\qquad N_{\mathrm{FLOP}}=2BMH_{\mathrm{o}}W_{\mathrm{o}}CK^2. \tag{11.10}$$

这里采用每次累加均计一次加法的惯例，与代码中从 0 开始执行乘加的形式对应。即使硬件把乘法和加法融合成一条指令，按照这个统计口径仍记为两次浮点运算。偏置、激活和池化需要另行计数。

参数存储与这些运算量具有不同的缩放关系。权重只有 MCK^2 个，与批次大小和输出空间位置数无关；输入有 $BCHW$ 个元素，输出有 $BMH_{\mathrm{o}}W_{\mathrm{o}}$ 个元素。使用 32 位浮点数时，每

个数组的字节数等于元素数乘 4。增加 B 会增加输入、输出和卷积总运算量，而不会增加模型权重数量；增加图像尺寸会增加输出位置数，却仍复用同一组核参数。

在上面的三通道算例中，只有 $1\times3\times3^2=27$ 个权重，却需要为四个输出执行 $4\times27=108$ 次乘加，即 216 次浮点运算。若处理一百幅同样尺寸的图像，权重仍为 27 个，卷积运算量则增为 21 600 次浮点运算。

11.2.6 局部池化与非线性变换

池化把一个局部窗口内的多个数值汇总为一个输出。窗口沿空间轴移动，输入通道之间保持分离，因此常规空间池化保留批次大小和通道数量，同时改变每个通道的空间尺寸。常见汇总函数包括平均值、最大值、L^2 范数和加权平均。

设一个窗口包含 N^2 个值，按任意固定顺序记为 $z_1,\dots,z_{N^2}$。平均池化输出 $N^{-2}\sum_i z_i$，最大池化输出 $\max_i z_i$，L^2 池化输出 $(\sum_i z_i^2)^{1/2}$；带非负权重 a_i 且 $\sum_i a_i=1$ 的加权平均输出 $\sum_i a_i z_i$。这些选择保留的局部信息不同：平均值反映整体响应，最大值保留最强响应，L^2 范数反映窗口中响应的合成幅度。若将平方和先除以 N^2 再开平方，得到的是均方根形式，需要与未归一化的 L^2 范数区分。

对卷积输出 Y 使用 $N\times N$ 窗口、步幅为 N、无填充的不重叠平均池化，空间输出尺寸为 $H_\mathrm{s}=\lfloor H_\mathrm{o}/N\rfloor$、$W_\mathrm{s}=\lfloor W_\mathrm{o}/N\rfloor$。池化输出坐标 (x,y) 所需窗口的左上角是 (Nx,Ny)，因此窗口内元素为 $Y_{b,m,Nx+p,Ny+q}$，其中 $0\le p,q<N$。若尺寸无法整除 N，最右侧或最下侧不足一个完整窗口的元素不参与这一约定下的池化。

进一步对平均值加上输出通道偏置 β_m，再施加 S 形激活函数（sigmoid），得到

$$
\begin{aligned}
S_{b,m,x,y} &= \sigma\!\left(\frac{1}{N^2}\sum_{p=0}^{N-1}\sum_{q=0}^{N-1} Y_{b,m,Nx+p,Ny+q}+\beta_m\right),\\
\sigma(z) &= \frac{1}{1+\exp(-z)}, \qquad S\in\mathbb{R}^{B\times M\times H_\mathrm{s}\times W_\mathrm{s}}.
\end{aligned}
\tag{11.11}
$$

偏置 β_m 在所有图像和同一输出通道的全部空间位置上共享，所以这个偏置向量只包含 M 个数。平均池化本身没有需要学习的权重；附加偏置以后，这个组合模块具有 M 个可学习参数。

式(11.11)的运算顺序是先求平均、再加偏置、最后激活。非线性函数通常不能与平均运算交换。例如，窗口值为 $\{0,0,2,2\}$ 且偏置为 0 时，先平均再激活得到 $\sigma(1)\approx0.7311$；先激活再平均则得到 $[\sigma(0)+\sigma(2)]/2\approx0.6904$。两者表示不同的层结构。固定通道偏置与平均可以交换，是因为 $\mathrm{avg}(Y+\beta_m)=\mathrm{avg}(Y)+\beta_m$；这项线性关系不能用来交换后面的非线性激活。

激活函数还可以使用整流线性单元（Rectified Linear Unit，ReLU），定义为 $\phi(z) = \max(0, z)$。它与 S 形函数都逐元素作用，不改变张量形状。具体实现应按照网络给定的函数和顺序执行，不能因为两个模块都称为激活层就互相替换。

11.2.7 池化的数值解释与独立性

对式(11.9)中的整个 2×2 输出做一次平均池化，得到 $(51+50+60+48)/4 = 52.25$。若偏置取 -52，则激活输入为 0.25，最终输出为 $\sigma(0.25) \approx 0.5622$。若只做最大池化，输出为 60；若做未归一化的 L^2 池化，输出为 $\sqrt{51^2+50^2+60^2+48^2} = \sqrt{11\,005}$。同一个窗口的不同汇总规则会产生不同结果，不能只根据输出尺寸识别池化操作。

池化可以降低对局部位置变化的敏感性，但这种效果取决于窗口边界与汇总方式。以一维最大池化作为简化例子，窗口宽度和步幅均为 2：输入 $[9,0,0,0]$ 和 $[0,9,0,0]$ 都得到 $[9,0]$；若响应继续右移一个位置，输入变为 $[0,0,9,0]$，输出就变为 $[0,9]$。因此，在一个窗口内部移动的特征可能给出相同结果，跨过窗口边界后则可能改变输出。空间下采样本身也不保证尺度不变性。

从并行计算角度看，(b,m,x,y) 不同的池化输出可以独立计算。一个线程可以读取所需的 N^2 个值，在寄存器中累加，除以 N^2，再执行偏置与激活，最后写回一个输出。这个方案与卷积中的单输出线程具有相同的组织原则：输出由线程唯一负责，局部窗口内的求和由该线程完成。

卷积与池化之间还有一项可利用的数据复用。如果池化所需的卷积结果已由同一线程块产生，并且在写出前仍位于寄存器或共享内存中，就可以直接完成局部汇总，减少中间张量的全局写回和随后重读。合并实现时，必须保留式(11.11)的准确顺序，并确保每个池化窗口所需的全部卷积结果已经可用。窗口跨越线程块边界时，块内同步不能使另一个块的中间结果自动变得可用，需要重新安排分工或保留独立执行阶段。

11.3 直接卷积的并行实现与数据复用

11.3.1 张量地址与顺序参考实现

数学上的四维下标需要转换为实际内存地址。假设数组连续存储，最右侧的宽度索引变化最快，其次为高度、通道和批次，这就是这里采用的行主序（row-major order）布局。输入中的一个通道占 HW 个元素，一幅图像占 CHW 个元素，因此输入元素 $X_{b,c,i,j}$ 的线性偏移为 $bCHW + cHW + iW + j$。权重和输出使用同样的逐维展开规则：

$$\begin{aligned} \mathrm{off}_X(b,c,i,j) &= ((bC + c)H + i)W + j, \\ \mathrm{off}_F(m,c,p,q) &= ((mC + c)K + p)K + q, \\ \mathrm{off}_Y(b,m,h,w) &= ((bM + m)H_{\mathrm{o}} + h)W_{\mathrm{o}} + w. \end{aligned} \tag{11.12}$$

例如，输入宽度为 W 时，同一行中相邻像素的地址相差 1，同一列中相邻两行的地址相差 W。这既是正确索引的依据，也决定相邻线程的访存是否连续。

对于类型为 `float*` 的指针，`X[b,c,h,w]` 在 C++ 中包含逗号表达式，不能实现四维数组访问。代码需要使用真正的线性偏移。下面的辅助函数把四个索引映射到一个偏移，后三个参数依次给出后三维的长度。先把第一个索引转换为 `std::size_t`，可使后续的地址乘法使用适合数组大小的无符号整数类型。

代码 11.1 四维数组的行主序地址计算

```
#pragma once
#include <cstddef>

#ifdef __CUDACC__
#define LEC11_HD __host__ __device__
#else
#define LEC11_HD
#endif

LEC11_HD inline std::size_t off4(
    int i0, int i1, int i2, int i3,
    int d1, int d2, int d3) {
  return ((static_cast<std::size_t>(i0) * d1 + i1)
          * d2 + i2) * d3 + i3;
}

#undef LEC11_HD
```

辅助函数在普通 C++ 中是主机函数，在 CUDA 编译中同时提供主机端和设备端版本。其调用约定完全相同：例如 `off4(b,c,i,j,C,H,W)` 返回输入偏移，而 `off4(m,c,p,q,C,K,K)` 返回权重偏移。

顺序卷积直接对应式(11.3)：外层四个循环枚举输出，内层三个循环完成一个输出的求

和。以下函数假设 B, M, C, K 均为正数，$H, W \geq K$，三个数组已按式(11.2)分配，且输出数组不与输入或权重数组重叠。它只执行卷积，不在其中混入偏置或激活。

代码 11.2 逐输出累加的顺序卷积

```
#include "tensor_index.h"

void conv_forward_cpu(
    int B, int M, int C, int H, int W, int K,
    const float* X, const float* filter, float* Y) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  for (int b = 0; b < B; ++b)
    for (int m = 0; m < M; ++m)
      for (int h = 0; h < Hout; ++h)
        for (int w = 0; w < Wout; ++w) {
          float acc = 0.0f;
          for (int c = 0; c < C; ++c)
            for (int p = 0; p < K; ++p)
              for (int q = 0; q < K; ++q) {
                const auto xi = off4(b, c, h+p, w+q, C, H, W);
                const auto fi = off4(m, c, p, q, C, K, K);
                acc += X[xi] * filter[fi];
              }
          Y[off4(b, m, h, w, M, Hout, Wout)] = acc;
        }
}
```

每次进入新的 (b, m, h, w) 时创建并清零一个累加变量；全部输入通道和核位置处理完之后，写回一次输出。这个变量的生命周期明确了计算的归约边界：同一个输出的所有乘积必须合并在一起。连续处理不同输出时，重复读取的输入和权重来自运算自身的数据共享关系。

平均池化的顺序实现把每个输出窗口的起点改为 (Nx, Ny)，求和后除以窗口面积。其输入参数 `Hout` 和 `Wout` 表示前一卷积层的输出尺寸，`Hs` 和 `Ws` 才是当前池化结果的尺寸。假设 $N > 0$ 且至少存在一个完整池化窗口，整数除法直接给出前述向下取整。

代码 11.3 平均池化、通道偏置与 S 形激活

```
#include <cmath>
#include "tensor_index.h"

static float sigmoid_cpu(float z) {
  const float e = std::exp(-std::fabs(z));
  return z >= 0.0f ? 1.0f / (1.0f + e) : e / (1.0f + e);
}

void pool_forward_cpu(
    int B, int M, int Hout, int Wout, int N,
    const float* Y, const float* bias, float* S) {
  const int Hs = Hout / N;
  const int Ws = Wout / N;
  for (int b = 0; b < B; ++b)
    for (int m = 0; m < M; ++m)
      for (int x = 0; x < Hs; ++x)
        for (int y = 0; y < Ws; ++y) {
          float acc = 0.0f;
          for (int p = 0; p < N; ++p)
            for (int q = 0; q < N; ++q)
              acc += Y[off4(b, m, N*x+p, N*y+q,
                            M, Hout, Wout)];
          const float mean = acc / (static_cast<float>(N) * N);
          S[off4(b, m, x, y, M, Hs, Ws)] =
              sigmoid_cpu(mean + bias[m]);
        }
}
```

激活函数使用了与 $1/(1+\exp(-z))$ 等价的分支形式。$z \geq 0$ 时计算 $1/(1+\exp(-z))$，$z < 0$ 时计算 $\exp(z)/(1+\exp(z))$；两种分支中指数的自变量都不大于 0，从而避免为一个最终接近 0 的激活值先计算很大的正指数。

11.3.2 输出并行与归约维度

卷积中的并行性可以直接从输出下标和求和下标区分。批次索引 b、输出通道索引 m、输出坐标 h, w 共同确定一个唯一的输出地址。不同四元组之间没有结果依赖，因此可以并行执行。只要每个四元组分配给唯一线程，各线程就能独立读输入、读权重并写自己的输出。

输入通道 c 和核内坐标 p, q 则属于同一个输出的归约维度（reduction dimension）。它们对应许多需要相加的部分乘积。基础方案让一个线程顺序完成全部 CK^2 项，因而不需要原子操作；如果把这些求和项分给不同线程，就需要归约或原子累加来合并部分结果，并处理累加初值与最终写回的时序。

这一区分也解释了为什么只把不同输出通道交给不同线程可能并行度不足：M 有时只

有几个，而全部输出位置数为 $BMH_{\mathrm{o}}W_{\mathrm{o}}$。利用空间位置和批次中的独立图像，可以同时暴露更多任务。较深层的特征图可能更小，但通道数和单个输出的累加长度可能更大；线程粒度与共享数据的组织因此需要结合具体形状选择。

11.3.3 空间分块与线程索引

令每个线程块计算固定图像、固定输出通道中的一个 $T \times T$ 输出分块。块内配置 $T \times T$ 个线程，线程的横坐标对应输出列，纵坐标对应输出行。对于一张大小为 $H_{\mathrm{o}} \times W_{\mathrm{o}}$ 的特征图，横向分块数为 $n_w = \lceil W_{\mathrm{o}}/T \rceil$，纵向分块数为 $n_h = \lceil H_{\mathrm{o}}/T \rceil$。向上取整保证最右侧和最下侧剩余的位置也有对应线程。

将空间分块按行展开，记其编号为 t，则 $0 \le t < n_h n_w$。分块所在的行、列分别为 $\lfloor t/n_w \rfloor$ 与 $t \bmod n_w$，所以该块左上角的输出坐标为 $(T\lfloor t/n_w \rfloor,\, T(t \bmod n_w))$。使用以下网格维度，就能把输出通道、空间分块和图像批次分别放在三个网格轴上：

$$
\begin{aligned}
&\text{线程块维度} = (T, T, 1), && \text{网格维度} = (M, n_h n_w, B),\\
&m = \texttt{blockIdx.x}, && b = \texttt{blockIdx.z},\\
&h = T\left\lfloor \frac{\texttt{blockIdx.y}}{n_w} \right\rfloor + \texttt{threadIdx.y},\\
&w = T(\texttt{blockIdx.y} \bmod n_w) + \texttt{threadIdx.x}.
\end{aligned}
\tag{11.13}
$$

这里 CUDA 网格的 x 维代表输出通道，图像水平方向的分块数却是 n_w。图像的空间方向与网格轴的名称不必相同，关键是恢复坐标时使用一致的对应关系。

例如，$M = 4$、$H_{\mathrm{o}} = W_{\mathrm{o}} = 8$、$T = 4$ 时，每个输出通道需要 $2 \times 2 = 4$ 个空间分块，单幅图像的网格大小为 $(4, 4, 1)$，共有 16 个线程块。空间分块 $t = 0, 1$ 分别位于上排的左、右两侧，$t = 2, 3$ 分别位于下排的左、右两侧。固定 $m = 2$、$t = 3$，再取块内坐标 $\texttt{threadIdx.x} = 1$、$\texttt{threadIdx.y} = 2$，就得到 $h = 6, w = 5$，因此该线程负责 $Y_{0,2,6,5}$。

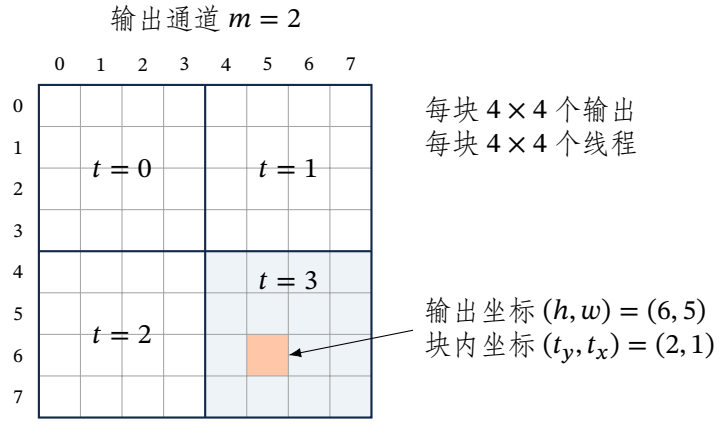


图 11.2 一张 8×8 特征图的空间分块。浅蓝区域由分块 $t = 3$ 负责，橙色单元由该块中纵坐标为 2、横坐标为 1 的线程计算。

11.3.4 基础卷积核与边界处理

基础设备函数用式(11.13)替代顺序程序的四个输出循环，其余的 c, p, q 三重累加保持不变。模板参数 T 是编译时确定的分块边长，启动时的线程块必须与它一致；`tiles_w` 对应上面的 n_w。输入、权重和输出指针都指向设备内存。

代码 11.4 每个线程计算一个卷积输出的设备函数

```
#include <cuda_runtime.h>
#include "tensor_index.h"

template<int T>
__global__ void conv_forward_kernel(
    int M, int C, int H, int W, int K, int tiles_w,
    const float* X, const float* filter, float* Y) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const int b = blockIdx.z;
  const int m = blockIdx.x;
  const int h = (blockIdx.y / tiles_w) * T + threadIdx.y;
  const int w = (blockIdx.y % tiles_w) * T + threadIdx.x;
  if (h >= Hout || w >= Wout) return;

  float acc = 0.0f;
  for (int c = 0; c < C; ++c)
    for (int p = 0; p < K; ++p)
      for (int q = 0; q < K; ++q) {
        const auto xi = off4(b, c, h+p, w+q, C, H, W);
        const auto fi = off4(m, c, p, q, C, K, K);
        acc += X[xi] * filter[fi];
      }
  Y[off4(b, m, h, w, M, Hout, Wout)] = acc;
}
```

使用向上取整后，一些线程会落在输出范围之外。对于这个没有块内同步的基础核，可以直接让这些线程返回。对剩余线程，有 $h \le H-K$、$p \le K-1$，从而 $h+p \le H-1$；宽度方向同理。这说明，只要输出坐标合法，整个有效卷积窗口中的输入地址就合法，无须在内层循环中再对每个输入元素重复判断。

相应主机函数计算网格并启动设备函数。下面取 $T=16$，每块为 256 个线程。数组应提前分配，且网格维度与数组大小需要位于所用设备和索引类型的支持范围内。

代码 11.5 包含批次维度与边缘分块的卷积启动函数

```
cudaError_t launch_conv_forward(
    int B, int M, int C, int H, int W, int K,
    const float* dX, const float* dFilter, float* dY) {
  if (B <= 0 || M <= 0 || C <= 0 || K <= 0 || H < K || W < K)
    return cudaErrorInvalidValue;
  constexpr int T = 16;
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const int tiles_w = (Wout + T - 1) / T;
  const int tiles_h = (Hout + T - 1) / T;
  const dim3 block(T, T, 1);
  const dim3 grid(M, tiles_h * tiles_w, B);
  conv_forward_kernel<T><<<grid, block>>>(
      M, C, H, W, K, tiles_w, dX, dFilter, dY);
  return cudaGetLastError();
}
```

启动函数返回的是内核启动阶段的错误状态。内核执行通常相对于主机异步进行；主机读取计算结果前，还需要在执行流程中完成相应的同步或阻塞式结果复制。这个时序安排与卷积的数值公式分开处理。

输出尺寸为 34×34、$T=16$ 时，必须启动 3×3 个空间分块。若直接用整数除法计算 $34/16=2$，就只覆盖前 32 行和前 32 列。向上取整加输出边界检查使同一个核能够同时处理可整除和不可整除的形状。

平均池化也可以沿用同一输出映射，把 $H_{\mathrm{o}}, W_{\mathrm{o}}$ 换成 $H_{\mathrm{s}}, W_{\mathrm{s}}$，让每个线程负责一个 $S_{b,m,x,y}$，再按式(11.11)访问 $(Nx+p, Ny+q)$。这时改变的是线程内部的输入地址与汇总运算，批次、通道和空间分块的组织方式仍然适用。

基础核中的输入全部从全局内存读取。把输出分给二维线程块，已经解决了并行任务分配；要进一步让同一块中的线程复用输入数据，还需要显式引入共享内存。

11.3.5 共享内存中的输入复用

相邻输出的输入窗口大量重叠。对于一个完整的 $T \times T$ 输出分块，固定一个输入通道，如果每个线程都从全局内存独立读取自己的 $K \times K$ 窗口，共发出 T^2K^2 次输入读取请求。其中，同一个输入元素可能被块内多个线程重复请求。

求这些窗口的联合覆盖范围，就能确定需要协作加载的数据。设输出块左上角为 (h_0, w_0)，一个线程需要的输入行是 $h_0 + t_y + p$，其中 $0 \le t_y < T$、$0 \le p < K$。它的最小值为 h_0，最大值为 $h_0 + T + K - 2$，因此联合输入区域在高度方向包含 $T + K - 1$ 个位置。宽度方向相同，所以输入分块大小为 $(T + K - 1) \times (T + K - 1)$。

输出分块之外、为完成边缘输出而额外加载的输入区域称为晕区 (halo)。将整个联合输入区域加载到共享内存，每个输入元素在这个块内只从全局内存取一次，后面的所有局部乘加都可读取共享缓冲区。对于单个输入通道，完整分块的读取请求减少倍数为

$$R_{\text{load}} = \frac{\text{独立读取各窗口的次数}}{\text{协作加载联合输入区域的次数}} = \frac{T^2K^2}{(T + K - 1)^2}. \tag{11.14}$$

例如，$T = 16, K = 7$ 时，独立读取需要 $256 \times 49 = 12\,544$ 次输入请求；联合输入块只有 $22^2 = 484$ 个元素，于是 $R_{\text{load}} = 12\,544/484 \approx 25.92$。若逐个处理 C 个输入通道，分子和分母都乘 C，该比值保持不变。

这个比值刻画指定条件下请求的全局输入读取数。共享内存加载、同步、乘加和输出写回仍然需要时间，缓存也可能已服务了基础实现的一部分重复读取，因此不能把 25.92 直接当作运行时间的加速倍数。相邻线程块的晕区仍可能重叠，同一输入区域也可能由不同输出通道对应的线程块重复读取；式(11.14)只计算一个块内部的协作收益。

11.3.6 协作加载、同步与权重复用

输入分块有 $(T + K - 1)^2$ 个元素，往往多于块内的 T^2 个线程，因此每个线程只装入一个元素并不足够。令 $Q = T + K - 1$，块内线性线程号为 $u = t_yT + t_x$。让线程 u 负责所有满足 $a = u + jT^2 < Q^2$ 的输入线性位置，其中 $j = 0, 1, 2, \ldots$。每个合法 a 恰好具有一个模 T^2 的余数，因此所有线程的加载范围不重叠，并且覆盖整个输入分块。由 a 恢复共享缓冲区坐标为 $r = \lfloor a/Q \rfloor$、$s = a \bmod Q$，对应的原输入位置是 $(h_0 + r, w_0 + s)$。

使用同一共享缓冲区依次处理不同输入通道时，每个通道需要遵守三个阶段的时序：

1. 所有线程协作把该通道所需的 $Q \times Q$ 输入区域放入共享内存。边缘块中，超出原输入的加载位置填入 0；合法输出所需要的输入仍全部来自有效窗口。

2. 全块执行同步。随后，负责合法输出的线程读取共享缓冲区中的 $K \times K$ 窗口，把本通道贡献累加到各自的寄存器中。

3. 在下一通道覆盖这块共享缓冲区之前再次全块同步，确保当前通道的数据已经被所有需要它的线程读完。

第一道同步防止读取尚未加载完的数据，第二道同步防止较快线程过早覆盖较慢线程仍在使用的数据。输出范围外的线程虽然不产生结果，仍可能承担晕区加载并参与同步。因此，代码11.4中的越界提前返回只适用于当前没有块内同步的基础核；引入共享缓冲区以后，应保留全块加载与同步，只对累加和最终输出写回设置有效性条件。

按通道复用缓冲区时，共享内存大小约为 $4Q^2$ 字节，而不是 $4CQ^2$ 字节；累加变量跨通道保存在每个线程的寄存器中。一次装入所有通道也能实现相同计算，但共享内存占用随 C 增长。如何选择取决于可用容量、同步成本以及一个计算单元上能够同时驻留多少个线程块。

权重还有另一种复用方式。在固定输出通道的基础映射中，同一线程束（warp）内的线程处理不同空间位置，却在相同的 c, p, q 循环迭代读取相同的 $F_{m,c,p,q}$。当核组能够放入常量内存（constant memory）且访问保持这种同址特征时，可以利用广播让多个线程共享一次权重取值。若另一种线程映射让同一线程束同时处理不同输出通道，权重地址就可能不同，需要重新分析访问行为。代码11.4使用普通权重指针，常量内存属于可在该基础上采用的存储优化。

在图像输入方面，固定的 c, p, q 下，相邻横向线程读取相邻列，行主序布局因而有利于连续访问。不同输出通道的线程块还会访问同一幅图像中的相同区域；这些块有可能利用缓存中的输入，但实际复用取决于调度和缓存容量。若希望在一次协作计算中显式共享跨输出通道的输入，可以把多个输出通道一起纳入分块，或者利用下面的矩阵乘法表示。

11.4 卷积的矩阵表示与输入展开

11.4.1 从局部内积到矩阵乘法

固定一幅图像和一个输出位置 (h, w)，把该位置所需的全部输入通道窗口连接成一个长度为 CK^2 的向量。每个输出通道都将自己的核组与这个向量做内积，因此可以把不同输出通道的权重排成矩阵的各行，把不同输出空间位置所需的输入向量排成另一个矩阵的各列。矩阵乘法中的一个行列内积就生成一个卷积输出。

输入展开（image-to-column，im2col）是按照上述方式重新组织输入窗口的操作。令 $D = CK^2$ 为一个输出所需内积的长度，$L = H_{\mathrm{o}} W_{\mathrm{o}}$ 为每个输出通道的位置数。对固定图像 b，定义核矩阵 $A \in \mathbb{R}^{M \times D}$、展开矩阵 $U_b \in \mathbb{R}^{D \times L}$，以及矩阵形式的输出 $V_b \in \mathbb{R}^{M \times L}$。

首先固定展开顺序。核内列坐标 q 变化最快，其次为核内行坐标 p，最后为输入通道 c，所以展开行号为 $r = cK^2 + pK + q$。输出列坐标 w 变化最快，其次为输出行坐标 h，所以展开列号为 $\ell = hW_{\mathrm{o}} + w$。由这两个编号得到

$$A_{m,r} = F_{m,c,p,q}, \qquad (U_b)_{r,\ell} = X_{b,c,h+p,w+q}, \qquad V_b = A U_b. \tag{11.15}$$

展开矩阵的一列包含一个输出位置所需的全部 C 个窗口，列内每连续 K^2 行属于一个输入通道。展开矩阵的一行则固定了一个输入通道及一个核内偏移，让输出窗口的左上角遍历全部空间位置。

矩阵乘法与卷积的等价性可以逐元素验证。对 $\ell = hW_{\mathrm{o}} + w$，用 r 与 (c, p, q) 的一一对应关系重写内积，得到

$$\begin{aligned} (V_b)_{m,\ell} &= \sum_{r=0}^{D-1} A_{m,r} (U_b)_{r,\ell} \\ &= \sum_{c=0}^{C-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} F_{m,c,p,q} X_{b,c,h+p,w+q} = Y_{b,m,h,w}. \end{aligned} \tag{11.16}$$

恢复卷积输出时，只需把 V_b 的每一行按 $H_{\mathrm{o}} \times W_{\mathrm{o}}$ 解释。若 V_b 按行连续存储，其内部顺序与对应图像的 $Y_{b,:,:,:}$ 已经一致，这一步不需要再搬运元素。

这套组织方式使同一展开列可被所有 M 个输出通道使用，也使同一权重行可用于全部 L 个空间位置。随后可以用分块矩阵乘法，把这两种复用显式放入共享内存和寄存器。

11.4.2 三通道、双输出的完整矩阵算例

取 $B=1$、$C=3$、$M=2$、$H=W=3$、$K=2$。输出尺寸为 $H_{\mathrm{o}}=W_{\mathrm{o}}=2$，因而 $D=12$、$L=4$。固定唯一图像后，三个输入通道为

$$X_0=\begin{bmatrix}1&2&0\\1&1&3\\0&2&2\end{bmatrix},\qquad X_1=\begin{bmatrix}0&2&1\\0&3&2\\1&1&0\end{bmatrix},\qquad X_2=\begin{bmatrix}1&2&1\\0&1&3\\3&3&2\end{bmatrix}. \tag{11.17}$$

将两组卷积权重写成下面的 2×12 矩阵。每一行对应一个输出通道，每连续四列对应一个输入通道上的 2×2 核，竖线仅用于标出通道之间的分界：

$$A=\left[\begin{array}{cccc|cccc|cccc}1&1&2&2&1&1&1&1&0&1&1&0\\1&0&0&1&2&1&2&1&1&2&2&0\end{array}\right]. \tag{11.18}$$

例如，第一行的前四个数 $[1,1,2,2]$ 对应 $F_{0,0}=\left(\begin{smallmatrix}1&1\\2&2\end{smallmatrix}\right)$，下一组 $[1,1,1,1]$ 对应 $F_{0,1}$。把每组连续四个数按两行两列恢复，就能得到全部六个核片。

四个输出位置按照 $(0,0),(0,1),(1,0),(1,1)$ 排列为四列。依次取各位置上的输入窗口，并按通道、核行、核列顺序展开，得到

$$U_0=\left[\begin{array}{cccc}1&2&1&1\\2&0&1&3\\1&1&0&2\\1&3&2&2\\\hline0&2&0&3\\2&1&3&2\\0&3&1&1\\3&2&1&0\\\hline1&2&0&1\\2&1&1&3\\0&1&3&3\\1&3&3&2\end{array}\right]. \tag{11.19}$$

上、中、下三个四行区域分别来自输入通道 $0,1,2$。例如，第三列对应 $(h,w)=(1,0)$，其最后四个数必须来自 X_2 中的窗口 $\left(\begin{smallmatrix}0&1\\3&3\end{smallmatrix}\right)$，所以这一段为 $[0,1,3,3]^{\mathrm{T}}$。核组和输入窗口必须按相同的 (c,p,q) 顺序展开；只改变一侧的排列通常会改变内积。

第一个输出通道的左上值由 A 第一行与 U_0 第一列相乘得到。把十二项按三个输入通道分组，计算为

$$\begin{aligned}(V_0)_{0,0} &= (1\cdot 1 + 1\cdot 2 + 2\cdot 1 + 2\cdot 1)\\ &\quad + (1\cdot 0 + 1\cdot 2 + 1\cdot 0 + 1\cdot 3)\\ &\quad + (0\cdot 1 + 1\cdot 2 + 1\cdot 0 + 0\cdot 1) = 7 + 5 + 2 = 14.\end{aligned} \tag{11.20}$$

对两行和四列完成全部内积后，得到

$$V_0 = AU_0 = \begin{bmatrix} 14 & 20 & 15 & 24 \\ 12 & 24 & 17 & 26 \end{bmatrix}, \qquad Y_{0,0} = \begin{bmatrix} 14 & 20 \\ 15 & 24 \end{bmatrix}, \quad Y_{0,1} = \begin{bmatrix} 12 & 24 \\ 17 & 26 \end{bmatrix}. \tag{11.21}$$

这里 $Y_{0,0}$ 与 $Y_{0,1}$ 分别固定图像编号和输出通道编号，保留下来的矩阵行、列对应输出的两个空间坐标。矩阵乘法只是重新排列了乘积项的组织方式，输出数值与逐通道卷积一致。

11.4.3 展开索引与实际数组实现

权重张量的线性偏移可以写成 $mD+r$，与 $M\times D$ 行主序矩阵的偏移相同，因此在既定布局下，核矩阵 A 可以直接使用原来的权重数组。输入窗口存在重叠，展开矩阵则需要把每个窗口所需的值放入指定行列。显式存放全部展开矩阵时，数组形状为 $B\times D\times L$，元素 $(U_b)_{r,\ell}$ 的偏移为 $(bD+r)L+\ell$。

顺序展开代码按图像、输入通道和核内偏移确定矩阵行，再让输出位置遍历矩阵列。两个输出位置即使需要相同的原输入元素，也分别写入自己的展开位置。

代码 11.6 按批次保存展开矩阵的顺序实现

```cpp
#include "tensor_index.h"

void unroll_cpu(
    int B, int C, int H, int W, int K,
    const float* X, float* U) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const std::size_t D = static_cast<std::size_t>(C) * K * K;
  const std::size_t L = static_cast<std::size_t>(Hout) * Wout;
  for (int b = 0; b < B; ++b)
    for (int c = 0; c < C; ++c)
      for (int p = 0; p < K; ++p)
        for (int q = 0; q < K; ++q) {
          const std::size_t r =
              (static_cast<std::size_t>(c) * K + p) * K + q;
          for (int h = 0; h < Hout; ++h)
            for (int w = 0; w < Wout; ++w) {
              const auto ell = static_cast<std::size_t>(h) * Wout + w;
              U[(static_cast<std::size_t>(b) * D + r) * L + ell] =
                  X[off4(b, c, h+p, w+q, C, H, W)];
            }
        }
}
```

另一种组织方式是直接枚举展开矩阵的位置，再反求应当读取的原输入。给定 r 和 ℓ，由于 r 先按通道分成长度为 K^2 的区段，再在每段中按行展开，所以逆映射为

$$\begin{aligned} c &= \lfloor r/K^2 \rfloor, \quad p = \lfloor (r \bmod K^2)/K \rfloor, \qquad q = r \bmod K, \\ h &= \lfloor \ell/W_{\mathrm{o}} \rfloor, \quad w = \ell \bmod W_{\mathrm{o}}, \qquad (U_b)_{r,\ell} = X_{b,c,h+p,w+q}. \end{aligned} \tag{11.22}$$

例如，$C=3, K=2$ 时，$r=9$ 对应 $(c,p,q)=(2,0,1)$；若 $W_{\mathrm{o}}=2$ 且 $\ell=2$，则 $(h,w)=(1,0)$，因此展开位置 $(9,2)$ 读取第三输入通道中的 $(1,1)$。

设备实现可以使用一维网格，把完整 $B\times D\times L$ 数组的线性位置分给各线程。下面的网格步进循环（grid-stride loop）让每个线程在处理完自己的起始位置后，跨过整个网格的线程总数继续处理。所有合法位置都被恰好一个线程负责，即使数组大于一次启动的线程总数，也能够覆盖全部元素。

代码 11.7 从展开位置反求输入地址的设备实现

```
#include <cuda_runtime.h>
#include "tensor_index.h"

__global__ void unroll_kernel(
    int B, int C, int H, int W, int K,
    const float* X, float* U) {
  const int Hout = H - K + 1;
  const int Wout = W - K + 1;
  const std::size_t D = static_cast<std::size_t>(C) * K * K;
  const std::size_t L = static_cast<std::size_t>(Hout) * Wout;
  const std::size_t total = static_cast<std::size_t>(B) * D * L;
  const std::size_t step =
      static_cast<std::size_t>(blockDim.x) * gridDim.x;

  for (std::size_t idx =
           static_cast<std::size_t>(blockIdx.x) * blockDim.x +
   threadIdx.x;
       idx < total; idx += step) {
    const std::size_t ell = idx % L;
    const std::size_t r = (idx / L) % D;
    const int b = idx / (D * L);
    const int c = r / K / K;
    const int p = (r / K) % K;
    const int q = r % K;
    const int h = ell / Wout;
    const int w = ell % Wout;
    U[idx] = X[off4(b, c, h+p, w+q, C, H, W)];
  }
}
```

该设备函数使用一维线程块和一维网格，块数及每块线程数均应为所用设备支持的正值。循环中的 `idx` 就是展开数组的目标偏移：先用余数恢复 ℓ，再恢复 r 与图像编号 b。同一批相邻线程写入连续展开元素；当它们仍位于同一展开行和同一输出空间行时，对原输入的读取也沿宽度方向连续。跨输出行、跨核偏移或跨输入通道时，输入地址会相应跳转。

展开操作的输出位置没有重叠写入，因而不需要原子操作。重复出现在不同窗口中的输入是重复读取的数据，它们进入展开矩阵中的不同地址。这种复制正是显式展开产生额外存储需求的原因。

11.4.4 批次组织与输出内存顺序

逐图计算 $V_b = AU_b$ 时，U 按 $B \times D \times L$ 存储，每个 V_b 写到该图像对应的 $M \times L$ 区域，自然得到 $B \times M \times L$ 的输出顺序。核矩阵 A 在整个批次中共享，不需要为每幅图像复制一份权重。批量矩阵乘法也可以使用相同的数学组织。

也可以把所有图像的展开列连接成 $U_{\text{all}} \in \mathbb{R}^{D \times (BL)}$，再一次计算 $AU_{\text{all}} \in \mathbb{R}^{M \times (BL)}$。具体地，$(U_{\text{all}})_{r,bL+\ell} = (U_b)_{r,\ell}$，其行主序偏移为 $r(BL) + bL + \ell$，与原数组的 $(bD + r)L + \ell$ 不同。因此，拼列需要按新顺序组织输入，不能仅改变原数组的形状解释。

在这种表示中，每个输出通道的一整行包含全部图像的位置，因此矩阵乘积的行主序布局对应 $M \times B \times L$。其地址与原来按批次优先存储的输出地址分别为

$$\text{off}_{\text{concat}} = m(BL) + bL + \ell, \qquad \text{off}_{\text{batch}} = (bM + m)L + \ell. \tag{11.23}$$

例如，$B = 2, M = 2, L = 4$ 时，图像 $b = 1$、输出通道 $m = 0$ 的第一个元素，在拼列乘积中的偏移为 4，在批次优先输出中的偏移为 8。因此，选择拼列计算时，展开数组本身就要采用对应布局，乘积也需要按后续层期望的顺序重排，或让后续计算显式接收这种新布局。矩阵形状相容只保证数学乘法成立，实际数组还必须使用相应的元素顺序。

11.4.5 展开存储与矩阵分块复用

显式展开把窗口重叠转化为数组中的重复元素。全部输入共有 $BCHW$ 个元素，全部展开矩阵共有 $BDL = BCK^2H_{\text{o}}W_{\text{o}}$ 个元素，因此展开数组相对于输入数组的大小比为

$$\rho_{\text{expand}} = \frac{BCK^2H_{\text{o}}W_{\text{o}}}{BCHW} = \frac{K^2(H - K + 1)(W - K + 1)}{HW}. \tag{11.24}$$

式(11.19)中的展开矩阵有 $12 \times 4 = 48$ 个元素，原输入有 $3 \times 3 \times 3 = 27$ 个元素，大小比为 $48/27 = 16/9 \approx 1.78$。当 H, W 远大于 K 时，$H_{\text{o}}W_{\text{o}}$ 接近 HW，该比值趋近 K^2。这不是每个输入元素都复制相同次数：内部元素可能属于许多窗口，边界元素属于较少窗口；式(11.24)比较的是整个数组的大小。

若每个元素占 s 字节，完整展开工作区需要 $sBDL$ 字节。代码11.6或11.7对每个展开位置读取一次原输入并写入一次展开数组，因而产生 BDL 次输入读取请求和 BDL 次展开写入；随后矩阵乘法还要读取展开数组。缓存、事务合并和矩阵分块会影响实际显存传输量，因此这里的元素请求计数不能直接等同于硬件上的总传输字节数。

展开并没有减少卷积的乘加次数。对每幅图像计算一个 $M \times D$ 与 $D \times L$ 的矩阵乘积，仍

需 MDL 次乘加，整个批次为 $BMDL$ 次。其优势在于把输入、权重和输出之间的复用关系组织成适合通用矩阵乘法（general matrix multiplication，GEMM）的规则形式。

考虑矩阵乘法中的一个输出块，它覆盖 T_m 个输出通道和 T_n 个展开列；一次沿求和维度处理 T_k 个元素。这里两个输出方向是通道 m 和线性化空间位置 ℓ，并非直接卷积空间块中的高度和宽度。在这一阶段，线程块将 $T_m \times T_k$ 的核矩阵块与 $T_k \times T_n$ 的展开矩阵块放入共享内存，再计算它们对输出块的贡献。对于三个方向均完整有效的分块，加载元素数与乘加数分别为

$$N_{\text{load}} = T_m T_k + T_k T_n, \qquad N_{\text{MAC}} = T_m T_n T_k. \tag{11.25}$$

每个核元素可用于 T_n 个输出位置，每个展开元素可用于 T_m 个输出通道。相比只让一个线程块负责单一输出通道的空间块，矩阵分块还能够在块内显式复用不同输出通道需要的同一输入窗口。完成全部求和块后，累加结果才写回对应的输出区域。

输出块更大通常带来更多块内复用，也需要更多共享内存、累加寄存器和线程工作量。边缘块中不足 T_m, T_n, T_k 的部分还会产生无效位置。选择块尺寸时，需要同时考虑有效工作比例和可驻留的线程块数量，不能只追求单个块的复用次数。

矩阵乘法的效率还取决于三个维度 M, D, L 各自的大小。一个维度很大，并不能补偿另一个维度中的大量无效分块位置。例如，第一层卷积的矩阵形状为 4×49 与 49×6400：空间位置很多，但输出通道只有 4 个。若沿输出通道方向固定使用 $T_m = 16$ 的分块，则该方向只有四分之一的位置有效；若求和分块宽度为 $T_k = 16$，最后一个求和块也只有一个有效元素。调整分块形状或让线程承担不同数量的输出，可以减少这些损失，但会改变资源占用和访存组织。较小的矩阵还可能使内核启动开销占据较大比例。因此，将卷积写成 GEMM 提供了规则的复用结构，实际性能仍需结合矩阵形状、边缘利用率和完整执行阶段衡量。

11.4.6 隐式展开与共享内存、寄存器联合分块

隐式展开（implicit unrolling）保留 $V_b = AU_b$ 的矩阵计算方式，但不在全局内存中存放完整 U_b。当分块矩阵乘法准备加载展开矩阵中的某个元素时，直接利用式(11.22)算出它在原输入中的地址，将原输入值送入当前共享内存块。展开矩阵只作为索引关系存在。

固定图像 b，设当前展开矩阵块的左上位置为 (r_0, ℓ_0)，块内位置为 (k, j)。首先得到 $r = r_0 + k$、$\ell = \ell_0 + j$；若 $r < D$ 且 $\ell < L$，就从 (r, ℓ) 恢复 c, p, q, h, w，再读取 $X_{b,c,h+p,w+q}$。核矩阵块中的元素仍是 A_{m_0+i, r_0+k}。两块就绪后，输出块的每个元素累加

$$\text{acc}_{i,j} \mathrel{+}= \sum_{k=0}^{T_k-1} A_{m_0+i, r_0+k} \, (U_b)_{r_0+k, \ell_0+j}. \tag{11.26}$$

超出 M, D, L 范围的分块位置填零，最终只写回合法输出。这里的零用于补齐矩阵边缘块，并未给原输入增加卷积填充；合法输出仍然使用原输入内部的完整窗口。共享内存块的加载、使用和下一轮覆盖之间，同样需要正确的块内同步。

把展开地址计算并入矩阵乘法，是核函数融合（kernel fusion）的一种形式。它省去了完整展开数组的全局写入和独立展开阶段的启动开销；原先读取 U 的动作改为按需读取 X，无需再从全局内存读取物化的展开数组，同时也省去了 *sBDL* 字节的完整展开工作区。共享内存中的局部块与输出累加寄存器仍然需要空间。原输入的一个元素可能因属于多个窗口或多个线程块而再次被读取，隐式展开不会自动消除所有重叠读取。它还增加了加载路径中的取整、取余和地址计算，因此是否更快取决于减少的存储开销与新增的寻址成本。

在共享内存分块之上，还可以让每个线程保存一个 $R_m \times R_n$ 的输出子块。对一个求和位置，该线程从共享内存取出 R_m 个权重和 R_n 个输入值，组成两组短向量；它们的所有两两乘积更新 $R_m R_n$ 个寄存器累加器。这样，一次取到的权重可在本线程内用于 R_n 个输出位置，一次取到的输入可用于 R_m 个输出通道。线程块之间的分工、共享内存块和线程内寄存器子块由此形成两级复用。

寄存器子块增大后，每个线程要保存更多累加器和临时值，可能减少同时驻留的线程，过高的寄存器需求还可能造成溢出存储。因此，共享内存与寄存器联合分块应平衡数据复用和执行资源占用。四种实现的主要区别可归纳为表11.3；联合分块既可用于显式展开后的矩阵乘法，也可用于隐式展开。

表 11.3 卷积实现方式与主要数据组织

实现方式	数据复用方式	主要代价或约束
直接空间分块	协作装入带晕区的输入；适合时广播权重	晕区及不同块的重复读取；通道间同步
显式展开后乘法	完整保存窗口列，再用矩阵分块共享输入和权重	展开工作区、展开读写与独立执行阶段
隐式展开乘法	装入矩阵块时从原输入取值	地址计算；输入访问仍可能重复
共享内存与寄存器联合分块	块内共享数据，线程内复用短向量	累加寄存器与共享内存占用

这些实现计算的是同一组实数乘积与求和，但浮点加法不满足任意重排下的结合律。分块、并行归约以及融合乘加指令可能改变舍入位置或累加顺序，因此数值核验应同时检查布局、边界和合理的浮点误差，不能把所有非零差异都判定为索引错误。

11.5 工程规模与综合例题

11.5.1 两层卷积的形状与运算规模

一个用于服饰图像数据集 Fashion MNIST 分类的修改版 LeNet-5 网络，在 Mini-DNN 这一 C++ 神经网络框架中实现，优化对象是使用 CUDA 执行的卷积层前向计算。两次卷积调用分别使用 86×86 和 40×40 的输入空间尺寸，批次包含 100 至 10 000 幅图像，卷积核均为 7×7。这些尺寸指实际传给相应卷积层的张量空间大小。

表11.4使用步幅 1、无填充的定义，列出每幅图像的形状与计算量。参数统计只包含卷积权重，运算统计只包含卷积乘加，均不包含偏置、激活或池化。

表 11.4 两次卷积调用的单幅图像规模

项目	第一次调用	第二次调用
输入形状 $C\times H\times W$	$1\times86\times86$	$4\times40\times40$
输出通道数 M	4	16
核大小 $K\times K$	7×7	7×7
输出形状 $M\times H_o\times W_o$	$4\times80\times80$	$16\times34\times34$
卷积权重数 MCK^2	196	3 136
输出元素数 MH_oW_o	25 600	18 496
每个输出的乘加数 CK^2	49	196
每幅图像的乘加数	1 254 400	3 625 216
每幅图像的浮点运算数	2 508 800	7 250 432

第一次卷积的空间输出为 $86-7+1=80$，第二次为 $40-7+1=34$。如果两次卷积后均采用 2×2、步幅 2 的池化，则空间尺寸分别从 80 降到 40、从 34 降到 17；这个条件下，第一层池化输出的空间尺寸与第二次卷积输入相容。卷积本身的尺寸和计算量不依赖是否随后采用这一池化配置。

第二次调用的输出元素数仅为第一次的 $18\,496/25\,600=0.7225$，但每个输出需要处理四倍的输入通道数，因此每图计算量比为

$$\frac{16\cdot34^2\cdot4\cdot7^2}{4\cdot80^2\cdot1\cdot7^2}=0.7225\times4=2.89. \tag{11.27}$$

空间分辨率下降并不保证下一层计算量下降；输入通道数、输出通道数和窗口大小共同决定总量。两次卷积每图合计 4 879 616 次乘加，即 9 759 232 FLOPs。批次为 B 时，这些计算量乘以 B，卷积权重数量则保持不变。

11.5.2 展开工作区与优化评估

将上述两层改写为矩阵乘法时，第一次的 $D = 49, L = 6400$，第二次的 $D = 196, L = 1156$。表11.5给出显式展开的空间需求，其中单精度浮点格式（FP32）中每个浮点数占 4 字节。

表 11.5 显式展开数组的空间需求

项目	第一次调用	第二次调用
每幅图像的展开矩阵 $D \times L$	49×6400	196×1156
每幅图像的展开元素数	313 600	226 576
每幅图像的 FP32 展开字节数	1 254 400	906 304
展开数组与原输入的大小比	≈ 42.40	35.4025
$B = 100$ 的完整展开字节数	125 440 000	90 630 400
$B = 10\,000$ 的完整展开字节数	12 544 000 000	9 063 040 000

采用十进制单位 $1\,\text{GB} = 10^9$ 字节时，一次展开全部 10 000 幅图像，两个调用分别需要约 12.544 GB 和 9.06304 GB 的展开工作区。这些数字仅包含展开数组，不包括输入、权重、输出及其他临时数组。两层按顺序执行并复用工作区时，容量只需覆盖其中较大的需求，不必将二者相加。

完整批次展开是一种执行选择。逐图展开、相乘并复用缓冲区时，工作区可缩小为单幅图像的 $4DL$ 字节；每次处理 B_c 幅图像的小批次时，则为 $4B_cDL$ 字节。小批次降低峰值存储占用，但可能增加执行阶段数量，并改变矩阵乘法的规模与并行度。隐式展开进一步移除了完整展开数组，直接从原输入装入当前计算块。工作区约束因此会影响算法组织，而不仅是最后的内存分配。

实现优化可以按相互依赖的三个技术阶段组织：先建立 CPU 参考结果和 GPU 基础卷积，再通过性能分析定位主要开销并实现矩阵展开及其融合，最后改进卷积与展开中的分块、寻址和存储复用。性能分析（profiling）需要区分实际执行时间、访存开销和资源占用，才能判断某项改动解决了哪一类瓶颈。

比较显式与隐式展开时，应统计完成同一卷积结果所需的全部设备计算阶段。显式方案至少包含展开和矩阵乘法；若另需布局重排，也应计入。只记录矩阵乘法内核的时间，会遗漏显式方案生成输入矩阵的成本。包含或排除主机与设备之间的数据传输也应保持一致。由于内核启动是异步的，计时终点需要对应设备工作实际完成的位置。

正确性核验应覆盖不同批次、多个输入与输出通道、非方形空间尺寸以及不能被线程块尺寸整除的边缘区域。可以先用小整数输入核对展开矩阵和输出的确切值，再用浮点输入比较不同实现的结果。每次改变线程分工、批次布局或求和组织后，都需要重新确认对应的索引关系；性能提升只有在保持预期输出和统计范围一致时才有意义。

11.5.3 三维卷积的存储读取比

考虑步幅为 1 的三维卷积，核的三个维度为 $5\times5\times7$，权重存于常量内存。一个线程块负责 $16\times16\times16$ 的逻辑输出块，并先将完整的联合输入区域装入共享内存，其中每个输入元素只从全局内存读取一次。卷积阶段，对每个输出的每个核位置执行一次共享内存标量读取。要求内部输出块的全局读取次数与共享读取次数之比，忽略权重访问和输出写入。

设输出块的三个维度为 T_x, T_y, T_z，卷积核的对应维度为 K_x, K_y, K_z。沿每个轴，相邻输出窗口的联合范围都比输出范围多出 $K_i - 1$ 个输入位置。因此，装入共享内存的联合输入区域包含

$$G = (T_x + K_x - 1)(T_y + K_y - 1)(T_z + K_z - 1) \tag{11.28}$$

个元素。每个元素只装入一次，故这也是全局读取次数。一个输出需要读取 $K_xK_yK_z$ 个共享输入，输出总数为 $T_xT_yT_z$，所以共享读取次数为

$$S = T_xT_yT_zK_xK_yK_z. \tag{11.29}$$

代入本题参数，输入区域尺寸为 $(16+4)\times(16+4)\times(16+6) = 20\times20\times22$，故 $G = 8\,800$。逻辑输出数为 $16^3 = 4\,096$，每个输出读取 $5\times5\times7 = 175$ 个共享输入，故 $S = 4\,096\times175 = 716\,800$。所求比值为

$$G : S = 8\,800 : 716\,800 = 11 : 896 \approx \boxed{1 : 81.45}. \tag{11.30}$$

这里比较的是规定计算方式下的标量读取次数。共享内存读取具有不同于全局内存读取的成本，实际实现还包含同步、地址计算和算术操作，因此该比值不是运行时间的加速比。

逻辑输出块的尺寸也不等于线程块必须采用的尺寸。16^3 个输出可以由较少线程分担，一个线程计算多个输出；实际线程块必须满足设备的线程数和资源限制。输出的分工方式应与上面采用的读取计数模型保持一致。

同样的方法适用于长方体输出块。例如，若第二个输出维度为 161，即输出块为 $16\times161\times16$，输入区域变为 $20\times165\times22$。此时

$$\begin{aligned} G &= 72\,600, \\ S &= 16\cdot161\cdot16\cdot175 = 7\,212\,800, \\ G : S &= 72\,600 : 7\,212\,800 \approx 1 : 99.35. \end{aligned} \tag{11.31}$$

输出块的具体尺寸决定了晕区开销，不能在读取比计算中省略任一轴的长度。

11.5.4 多通道卷积的浮点运算总数

一幅 100×200 的彩色图像含 3 个输入通道。卷积层使用 9×9 核生成 10 个输出特征图，每个输出特征图都结合全部 3 个输入通道，步幅为 1，不进行填充。一次乘加计为 2 次浮点运算，仅统计卷积，忽略偏置、激活、池化和其他操作。

首先计算有效输出范围。高度方向的窗口左上角可从 0 移动到 $100 - 9 = 91$，因此共有 92 个位置；宽度方向可从 0 移动到 $200 - 9 = 191$，因此共有 192 个位置。每个输出特征图有 $92 \times 192 = 17\,664$ 个值，全部十个特征图共有 $176\,640$ 个输出值。

每个输出值分别与三个通道中的 9×9 窗口做乘积并累加，需要 $3 \times 9 \times 9 = 243$ 次乘加。总乘加数为

$$N_{\mathrm{MAC}} = 10 \cdot 92 \cdot 192 \cdot 3 \cdot 9^2 = 42\,923\,520. \tag{11.32}$$

按一次乘加计 2 FLOPs 的约定，总浮点运算数为

$$N_{\mathrm{FLOP}} = 2 \cdot 42\,923\,520 = \boxed{85\,847\,040\ \mathrm{FLOPs}}. \tag{11.33}$$

若错用输入的 100×200 个空间位置，就会把不属于有效卷积输出的边界位置也计入；若漏掉输入通道因子 3，则只计算了一个输入通道的贡献。输出通道数 M 决定需要多少组核，输入通道数 C 决定每个输出内积的长度，这两个因子承担不同作用。

该层只有 $10 \times 3 \times 9^2 = 2\,430$ 个卷积权重，却执行了四千多万次乘加，因为每个权重都在 92×192 个输出位置复用。权重共享降低参数量，同时保留了对所有输出位置逐一完成计算的要求。

11.5.5 计算组织的统一关系

卷积的数学依赖可以分为两组索引：(b, m, h, w) 确定独立输出，(c, p, q) 确定该输出内部的乘加归约。直接实现保留这组张量索引；矩阵实现把 (c, p, q) 合并为 r，把 (h, w) 合并为 ℓ。两种表示的核心量对应为

$$D = CK^2, \qquad L = H_{\mathrm{o}} W_{\mathrm{o}}, \qquad N_{\mathrm{output}} = BML, \qquad N_{\mathrm{MAC}} = BMDL. \tag{11.34}$$

共享内存复用相邻输出的重叠输入，矩阵分块沿输出通道复用输入，寄存器子块在单个线程内复用操作数。显式与隐式展开保持上述索引和输出含义；融合卷积、池化、偏置与激活时，还必须保持各阶段的运算次序、布局和同步条件。

第 12 讲

Computation in Large Language Models

12.1 自回归生成与模型输入

12.1.1 从条件分布到逐步生成

大语言模型（Large Language Model，LLM）把文本生成表示为一个序列概率问题。分词器（tokenizer）先把文本转换为词元（token）的整数编号序列。词元可以对应一个字、一个子词、一段标点或若干字节；词元边界由分词规则决定，通常与自然语言中的词边界并不完全一致。设词表大小为 V_{voc}，一个长度为 N 的已知序列写为 $t_0, t_1, \dots, t_{N-1}$，其中 $t_i \in \{0, \dots, V_{\mathrm{voc}} - 1\}$。

自回归（autoregressive）模型根据已有前缀预测下一个词元。对于一个完整序列，其联合概率可以按概率的链式法则分解为

$$p(t_0, \dots, t_{T-1}) = \prod_{i=0}^{T-1} p(t_i \mid t_0, \dots, t_{i-1}). \tag{12.1}$$

当 $i = 0$ 时，条件为空，或由约定的起始标记提供。模型中的可训练参数共同确定这些条件分布。推理时固定参数，把系统输入、用户输入以及已经生成的词元共同作为上下文（context），反复计算下一词元的分布。

设完成最终归一化后，位置 i 的表示向量为 $h_i \in \mathbb{R}^D$，D 为模型宽度。词表投影把它变为未归一化分数（logits）$\ell_i = h_i W_U + b_U$，其中 $W_U \in \mathbb{R}^{D \times V_{\mathrm{voc}}}$，$b_U \in \mathbb{R}^{V_{\mathrm{voc}}}$ 为偏置向量。再经过软最大归一化（softmax）得到

$$p(t_{i+1} = v \mid t_0, \dots, t_i) = \frac{\exp(\ell_{i,v})}{\sum_{u=0}^{V_{\mathrm{voc}}-1} \exp(\ell_{i,u})}. \tag{12.2}$$

这里位置 i 的输出对应位置 $i+1$ 的预测。这个错开一位的关系解释了因果注意力为什么允许位置 i 读取它自身的输入。

生成过程还需要一个解码规则。贪心解码取分数最大的词元；随机采样则按照归一化后的分布选择词元。若采用温度 $\tau > 0$，先把分数替换为 ℓ_i / τ，再计算分布。选出的词元追加到

前缀后，进入下一次推理；遇到序列结束标记（end-of-sequence，EOS）、达到长度限制或满足其他既定停止条件时终止。词表分数的最大值位置在软最大归一化前后相同，因此只需贪心选择时可以直接对分数求最大值。

训练使用已知文本，可以同时构造许多位置的下一词元预测。推理时未来词元尚未确定，生成下一步的输入依赖当前步的选择。因此，同一已知前缀内部可以并行计算多个位置，连续的新词元生成仍存在步骤之间的依赖。

12.1.2 分词、嵌入与位置信息

分词器处理离散的符号边界与编号。字节对编码（Byte Pair Encoding，BPE）从较小的符号单元出发，根据语料中的共现统计反复合并高频相邻单元，形成词表和合并次序。推理时按照固定的规则把输入编码成编号。GPT-2 使用字节层面的 BPE；对同一段文字，具体编号必须与预训练模型使用的词表一致，不能通过临时给词语编号来替代。

词元嵌入（token embedding）把编号映射到连续向量。设嵌入矩阵为 $E \in \mathbb{R}^{V_{\text{voc}} \times D}$，编号 t_i 的嵌入就是第 t_i 行 $E_{t_i,:}$。这等价于用独热向量左乘 E，实现时直接查表即可。编号本身只标识表中的行，编号之差不表示语义距离。嵌入向量之间的方向或内积关系可以携带训练得到的相似性，但一个坐标不必对应一个可独立命名的语义属性。

词表和嵌入表承担不同的任务。分词规则及合并次序通常在分词器构建阶段确定，随后保持固定；浮点嵌入参数通过语言模型的训练目标学习。推理既不重新学习词表，也不更新嵌入权重。GPT-2 小型模型的嵌入宽度为 $D = 768$，所以一个整数编号被查表为一个含 768 个浮点分量的向量。

词元嵌入本身不区分同一词元出现在哪个位置。位置信息需要另外提供。采用可训练的绝对位置嵌入时，设位置表为 $E^{\text{pos}} \in \mathbb{R}^{N_{\max} \times D}$，其中 $N_{\max}$ 为模型支持的最大位置数。模型的初始输入为

$$X^{(0)}_{i,:} = E_{t_i,:} + E^{\text{pos}}_{i,:}, \qquad 0 \le i < N \le N_{\max}. \tag{12.3}$$

GPT-2 采用这种相加方式。嵌入矩阵 $X^{(0)} \in \mathbb{R}^{N \times D}$ 的行保留序列顺序，列表示特征。位置编码还可以采用其他形式；对这里的计算分析，关键条件是每个词元的位置在前缀扩展后保持一致。

查表与相加对每个位置读取并写入 D 个分量，工作量为 $\Theta(ND)$。后续线性层常需 $\Theta(ND^2)$ 的算术工作，因此嵌入通常不是主体计算。不过，词表投影需要把宽度 D 的表示映射到整个词表，涉及矩阵乘法，其开销应与输入查表区分。

12.1.3 解码器结构与张量约定

变换器（Transformer）架构利用注意力在不同位置之间传递信息。编码器把完整输入序列转换为上下文相关表示；在编码器与解码器组合的结构中，解码器还可以通过交叉注意力（cross-attention）读取编码器输出，并根据已生成前缀产生目标序列。仅含解码器的变换器（decoder-only Transformer）把输入提示和生成结果置于同一个序列中，通过因果自注意力（causal self-attention）建立依赖。GPT 系列的生成计算属于后一种结构。

设网络包含 L 个变换器层。每层接收 $N \times D$ 的表示，输出形状保持为 $N \times D$，从而能够逐层堆叠。层内的多头注意力负责不同位置之间的信息混合，逐位置前馈网络负责每个位置内部的特征变换；残差连接和层归一化使这些变换能够稳定地组合。最终归一化和词表投影把最后一层表示转成下一词元分布。

表 12.1 张量与计数符号。矩阵公式默认省略批量维，向量按行参与乘法。

符号	含义	形状或关系
B, N, L	批量大小、上下文长度、层数	均为正整数
D, h, d	模型宽度、注意力头数、单头维度	$D = hd$
X	某一层或子层的输入	$N \times D$
$W_Q^{(a)}, W_K^{(a)}, W_V^{(a)}$	第 a 个头的投影矩阵	每个为 $D \times d$
$Q^{(a)}, K^{(a)}, V^{(a)}$	单头的查询、键和值	每个为 $N \times d$
$S^{(a)}, P^{(a)}$	单头分数矩阵、注意力权重	每个为 $N \times N$
$O^{(a)}$	单头输出	$N \times d$
D_{ff}	前馈网络的中间宽度	常取为 D 的若干倍
V_{voc}	词表大小	与值矩阵 $V^{(a)}$ 区分

例如，$N = 4096$、$D = 4096$、$h = 32$ 时，$d = 128$。输入仍然具有 4096 个特征；每个注意力头把这 4096 个特征投影到 128 维空间。所有头的输出拼接后恢复到 $32 \times 128 = 4096$ 维。上下文长度和模型宽度恰好相等只是这个例子的取值，二者在定义和计算作用上独立。

12.2 因果多头注意力与层内计算

12.2.1 查询、键和值的投影

自注意力（self-attention）根据当前输入决定不同位置之间的混合权重。对一个注意力头，设输入为 $X \in \mathbb{R}^{N\times D}$。通过三个独立的可训练投影得到查询（query）、键（key）和值（value）：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad W_Q, W_K, W_V \in \mathbb{R}^{D\times d}. \tag{12.4}$$

这里暂时省略头编号，并省略偏置项，以突出矩阵乘法；实际模型若含有投影偏置，仍需按行相加。每个输出矩阵均为 $N \times d$。写到元素层面，$Q_{i,r} = \sum_{c=0}^{D-1} X_{i,c}(W_Q)_{c,r}$，所以投影只对同一位置的特征做线性组合，还没有混合不同位置的信息。

查询 $q_i = Q_{i,:}$ 决定位置 i 如何评价其他位置；键 $k_j = K_{j,:}$ 提供位置 j 用于匹配的特征；值 $v_j = V_{j,:}$ 则提供匹配之后要组合的内容。三者由同一序列产生，所以称为自注意力。在交叉注意力中，查询和键值可以来自两个不同的表示序列。

查询与键的内积 $q_i k_j^{\mathrm{T}}$ 给出有方向的匹配分数。由于 W_Q 与 W_K 一般不同，$q_i k_j^{\mathrm{T}}$ 与 $q_j k_i^{\mathrm{T}}$ 不要求相等。因此，注意力矩阵的第 i 行描述“位置 i 如何读取其他位置”，不应理解成一个必然对称的相似度矩阵。值向量随后按这些权重被组合；最终加权的对象是 V 中的行。

卷积通常在固定局部范围内，使用训练后固定且跨位置共享的核系数来混合输入。自注意力的混合权重由当前输入重新计算，可以覆盖掩码允许的全部位置。两者都可以处理变化的序列长度，其主要差别在于连接范围以及混合系数如何确定。

12.2.2 缩放、因果掩码与加权聚合

先计算缩放分数 $S = QK^{\mathrm{T}}/\sqrt{d} \in \mathbb{R}^{N\times N}$。缩放因子由单头维度 d 决定。为理解其作用，暂设查询和键的各分量独立，均值为零、方差为一，并且不同维度的乘积互不相关。于是每项 $q_{i,r}k_{j,r}$ 的方差为一，包含 d 项的点积方差为 d，标准差为 $\sqrt{d}$。除以 $\sqrt{d}$ 后，分数尺度不再仅因头维度增加而增长。

这个估算说明了缩放的动机，不要求训练后的分量严格满足独立分布。若分数尺度过大，指数归一化容易把概率集中到少数位置，其他位置的梯度也会变小。$1/\sqrt{d}$ 作用于每个点积，与后续归一化中减去行最大值分别解决尺度和数值稳定性问题。

因果掩码（causal mask）限制位置 i 只能读取 $0, \ldots, i$。用加性掩码 M 表示这个约束，单

头的完整计算为

$$
\begin{aligned}
M_{ij} &= \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} & P &= \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^{\mathrm{T}}}{\sqrt{d}} + M\right), \\
O &= PV, & O_{i,:} &= \sum_{j=0}^{i} P_{ij} v_j .
\end{aligned}
\tag{12.5}
$$

下标 row 表示独立归一化每一行。对于有效位置 $j \le i$，$P_{ij} \ge 0$，且 $\sum_{j=0}^{i} P_{ij} = 1$；对于 $j > i$，$\exp(-\infty) = 0$，因此权重为零。P 是包含对角线的下三角矩阵。

对角线保留是因为当前位置的词元已经是输入。例如，已知 t_0, t_1, t_2 时，位置 2 的输出用来预测 t_3，读取 t_2 不会读取到预测目标。若把对角线也屏蔽，第一行将没有任何有效位置，同时改变模型原本的条件分布。

从形状上看，第一项乘法 $(N \times d)(d \times N)$ 把特征维上的匹配变成位置之间的关系；第二项乘法 $(N \times N)(N \times d)$ 再把这些关系应用到值向量上。结果 $O \in \mathbb{R}^{N \times d}$ 的每一行是可见值向量的凸组合。多个头的拼接、输出投影和残差相加继续变换这个结果，所以完整层的输出不受这个单头凸组合性质直接约束。

12.2.3 多个注意力头与特征融合

多头注意力（multi-head attention）包含 h 个头，编号为 $a = 0, \ldots, h-1$。每个头分别配置查询、键和值的投影矩阵 $W_Q^{(a)}$、$W_K^{(a)}$ 和 $W_V^{(a)}$。各头从相同的输入构造不同的查询、键和值，在不同的投影空间中形成各自的权重分布。它们可以学习不同位置关系和特征关系；这些关系由训练形成，不需要给每个头指定一种分类任务。

各头在给定输入后可以独立计算。令第 a 个头的输出为 $O^{(a)} \in \mathbb{R}^{N \times d}$，沿特征方向拼接，再经过输出投影：

$$
Z = \mathrm{Concat}\big(O^{(0)}, \ldots, O^{(h-1)}\big) \in \mathbb{R}^{N \times hd}, \qquad \mathrm{MHA}(X) = ZW_O + b_O, \tag{12.6}
$$

其中 $W_O \in \mathbb{R}^{hd \times D}$、$b_O \in \mathbb{R}^{D}$，偏置按行广播。标准设置令 $hd = D$，所以输出形状回到 $N \times D$，能够与输入残差相加。拼接保留不同头的通道，W_O 再把它们线性混合。

在存储层面，可以把所有头的查询投影横向拼成 $\overline{W}_Q \in \mathbb{R}^{D \times D}$，一次计算 $X\overline{W}_Q \in \mathbb{R}^{N \times D}$，再将每行的 D 个分量分成 h 组、每组 d 个分量。键和值可以同样处理；进一步把三组权重拼为 $W_{QKV} \in \mathbb{R}^{D \times 3D}$，就能用一次矩阵乘法得到全部投影。逻辑上的多头独立与物理实现中的融合计算彼此兼容。

12.2.4 残差连接、层归一化与前馈网络

注意力只是一个完整层中的子层。残差连接（residual connection）把子层变换的结果与其输入逐元素相加，例如 $X + F(X)$。这样，原有表示可以直接传递，同时由 F 提供更新量；相加的两个矩阵必须具有相同形状。归一化与残差的先后顺序会改变实际函数，因此实现时必须固定顺序。

GPT-2 使用前置层归一化（pre-layer normalization）。若 $X^{(\ell)}$ 是第 ℓ 层输入，则完整层写为

$$\begin{aligned} U^{(\ell)} &= X^{(\ell)} + \mathrm{MHA}_{\ell}\big(\mathrm{LN}_{\ell,1}(X^{(\ell)})\big), \\ X^{(\ell+1)} &= U^{(\ell)} + \mathrm{FFN}_{\ell}\big(\mathrm{LN}_{\ell,2}(U^{(\ell)})\big). \end{aligned} \tag{12.7}$$

这里每个层归一化算子具有自己的一组参数。在式 (12.4) 中，X 表示送入注意力投影的矩阵；对应到式 (12.7)，它就是归一化后的 $\mathrm{LN}_{\ell,1}(X^{(\ell)})$。另一种常见的后置形式把子层写为 $\mathrm{LN}(X + F(X))$。两种放置方式的张量形状相同，但数值结果不同。

层归一化（layer normalization）独立处理每个位置的特征。对一行 $x_i \in \mathbb{R}^D$，定义

$$\begin{aligned} \mu_i &= \frac{1}{D}\sum_{c=0}^{D-1} x_{i,c}, \qquad \sigma_i^2 = \frac{1}{D}\sum_{c=0}^{D-1}(x_{i,c} - \mu_i)^2, \\ \mathrm{LN}(x_i)_c &= \gamma_c \frac{x_{i,c} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} + \beta_c. \end{aligned} \tag{12.8}$$

其中 $\gamma, \beta \in \mathbb{R}^D$ 是可训练的逐特征缩放和平移参数，$\varepsilon > 0$ 用于稳定分母。求均值和方差的范围是同一位置的 D 个特征，因此对新位置做归一化不需要重新计算旧位置的统计量。并行实现包括行内归约，以及随后独立的逐元素变换。

逐位置前馈网络（position-wise feed-forward network，FFN）采用两次线性变换，中间加入非线性：

$$\mathrm{FFN}(H) = \phi(HW_1 + b_1)W_2 + b_2, \quad W_1 \in \mathbb{R}^{D \times D_{\mathrm{ff}}}, \quad W_2 \in \mathbb{R}^{D_{\mathrm{ff}} \times D}. \tag{12.9}$$

其中偏置分别按行广播。所有位置共享相同的 W_1, W_2，但各行的计算彼此独立。第一层把特征扩展到中间宽度，非线性改变特征组合的表达能力，第二层恢复到 D 维。若去掉非线性，两次线性变换可以合并为一次，无法表达这种非线性映射。跨位置的信息已经由注意力汇入当前行，前馈网络进一步变换该行的特征。

GPT-2 取 $D_{\mathrm{ff}} = 4D$，并采用高斯误差线性单元（Gaussian Error Linear Unit，GELU）。其

定义为 $\mathrm{GELU}(u) = u\Phi(u)$，其中 Φ 是标准正态分布的累积分布函数；常用的计算近似为

$$\mathrm{GELU}(u) \approx \frac{u}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(u + 0.044715u^3)\right)\right]. \tag{12.10}$$

复现一个既定模型时，需要匹配其使用的激活函数及近似表达。完成全部 L 层后，再对 $X^{(L)}$ 做最终层归一化，得到用于词表投影的 H。GPT-2 的词表投影与输入嵌入共享权重，即 $W_U = E^{\mathrm{T}}$，且 $b_U = 0$，从而将宽度 D 的表示映射到 V_{voc} 个分数。

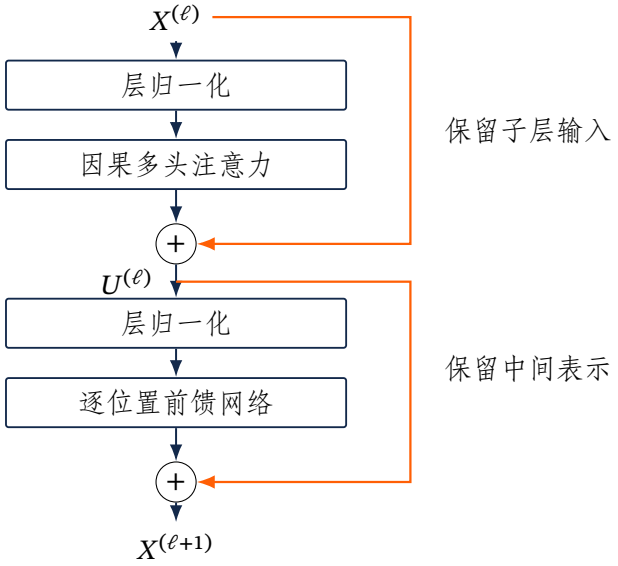


图 12.1 前置层归一化结构。每个子层先对输入归一化，再计算更新量，并通过残差路径加回未归一化的子层输入。所有残差相加均保持 $N \times D$ 的形状。

12.2.5 三个词元的注意力算例

考虑单头、三个位置、头维度 $d = 2$。为直接检查注意力内部的计算，给定已经投影得到的三个矩阵：

$$Q = \begin{bmatrix} \sqrt{2} & 0 \\ 0 & \sqrt{2} \\ \sqrt{2} & \sqrt{2} \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}. \tag{12.11}$$

先计算每一行查询与各行键的内积，并除以 $\sqrt{2}$，得到

$$S = \frac{QK^{\mathrm{T}}}{\sqrt{2}} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \qquad S + M = \begin{bmatrix} 1 & -\infty & -\infty \\ 0 & 1 & -\infty \\ 1 & 1 & 2 \end{bmatrix}. \tag{12.12}$$

第一行只有位置 0 可见，因此权重恰为 $(1,0,0)$。第二行有效分数为 $(0,1)$，相应权重为 $(1/(1+e), e/(1+e), 0)$。第三行分数为 $(1,1,2)$，约去共同因子 e 后，权重为 $(1/(2+e), 1/(2+e), e/(2+e))$。因此

$$P \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.268941 & 0.731059 & 0 \\ 0.211942 & 0.211942 & 0.576117 \end{bmatrix}, \qquad O = PV \approx \begin{bmatrix} 1 & 0 \\ 0.268941 & 1.462117 \\ 1.940292 & 1 \end{bmatrix}. \tag{12.13}$$

例如第三个输出是 $0.211942(1,0) + 0.211942(0,2) + 0.576117(3,1)$。其第二个分量严格等于 $(2+e)/(2+e) = 1$，第一分量严格等于 $(1+3e)/(2+e)$。每行权重和为一，上三角为零，输出形状仍为 3×2。这些关系可以同时检查索引方向、掩码边界、归一化范围和值聚合是否正确。

12.3 注意力的 CUDA 实现

12.3.1 矩阵乘法流水线与存储布局

注意力中的投影、分数计算和值聚合都能归结为通用矩阵乘法（General Matrix Multiplication，GEMM）。可以调用 CUDA 基础线性代数库（cuBLAS），也可以用分块矩阵乘法核函数实现。逐行的指数归一化穿插在两次注意力矩阵乘法之间，需要额外处理归约与因果性。

表 12.2 单序列注意力的基线计算步骤。多头分数和输出按各头分别计算。

步骤	运算与形状	GPU 上的主要操作
三个投影	$(N \times D)(D \times 3D) \to N \times 3D$	可融合的矩阵乘法
分数计算	$(N \times d)(d \times N) \to N \times N$	矩阵乘法与 $1/\sqrt{d}$ 缩放
因果归一化	$S \to P$，保持 $N \times N$	行内最大值与求和归约
值聚合	$(N \times N)(N \times d) \to N \times d$	矩阵乘法
头合并与投影	$(N \times D)(D \times D) \to N \times D$	重排与矩阵乘法

缩放可以作为分数矩阵乘法的标量系数完成，从而避免再单独遍历 N^2 个元素。数学上的 K^{T} 也不必先物化成一个新矩阵；支持转置操作的矩阵乘法接口可以按照原缓冲区的布局读取。但行主序与列主序、转置标志以及前导维度必须相互一致，才能实现式 (12.5) 指定的乘法。

加入批量维后，输入常表示为 $[B, N, D]$。把融合投影的最后一维拆成 $[3, h, d]$，得到

$[B, N, 3, h, d]$。抽取查询部分后得到 $[B, N, h, d]$，再交换头轴与序列轴，形成 $[B, h, N, d]$。分数矩阵则是 $[B, h, N, N]$。维度重塑（reshape）可以只改变同一线性存储的解释；轴置换是否需要数据搬运，取决于后续核函数是否能直接处理相应的步长。布局选择会影响合并访存和矩阵乘法接口的使用方式。

以下核函数只处理一个头的一张分数矩阵。假定它按行主序连续存储，因此 S_{ij} 的地址偏移是 $iN + j$。输入 S 已经包含 $1/\sqrt{d}$ 缩放，但尚未归一化。输出 P 使用另一块同形状缓冲区，并将掩码位置明确置零，供后续稠密矩阵乘法使用。

12.3.2 数值稳定的因果归一化

直接计算 $\exp(S_{ij})$ 可能溢出。对同一行减去一个常数 c_i 不改变归一化结果，因为分子和分母中的公共因子 $\exp(c_i)$ 可以约去：

$$\frac{\exp(S_{ij})}{\sum_{k=0}^{i} \exp(S_{ik})} = \frac{\exp(S_{ij} - c_i)}{\sum_{k=0}^{i} \exp(S_{ik} - c_i)}. \tag{12.14}$$

取有效列中的最大值 $m_i = \max_{0 \le j \le i} S_{ij}$，所有参与计算的指数输入都不大于零，至少有一项指数等于一。定义行分母 s_i，得到

$$\begin{aligned} m_i &= \max_{0 \le j \le i} S_{ij}, \qquad s_i = \sum_{j=0}^{i} \exp(S_{ij} - m_i), \\ P_{ij} &= \begin{cases} \exp(S_{ij} - m_i)/s_i, & 0 \le j \le i, \\ 0, & i < j < N. \end{cases} \end{aligned} \tag{12.15}$$

在有效分数有限的条件下，$1 \le s_i \le i + 1$。这既避免指数上溢，也使分母具有明确的正下界。非常小的权重仍可能发生浮点下溢，但它们对加权和的影响相应较小。

最大值与分母的范围必须遵守同一掩码。例如，在行 $i = 1$，假设三个分数依次为 1000、1001 和 9000。只有前两列可见，正确计算取 $m_1 = 1001$，使用 $(e^{-1}, 1)$ 归一化，得到约 $(0.268941, 0.731059, 0)$。若错误地把未来列的 9000 纳入最大值，再只累加前两列，两项指数都可能下溢成零。若把未来列也纳入分母，则直接改变了因果分布。实现时遍历有效列，可以同时避免这两类问题。

这一步的并行结构不同于逐元素激活：同一行的每个输出都依赖该行共同的最大值和分母。因此必须先在行内归约，再把归约结果提供给所有负责该行的线程。

12.3.3 一行一个线程块的计算组织

令线程块包含 B_t 个线程，块编号 i = `blockIdx.x` 对应行号，线程编号 u = `threadIdx.x`。线程 u 按步长 B_t 访问 $j = u, u + B_t, u + 2B_t, \ldots$。前两次遍历只处理 $j \le i$；最后一次遍历覆盖 $j < N$，以写出完整的一行。

计算先由每个线程求自己负责部分的最大值，再在线程块内归约得到 m_i；随后，每个线程累加自己的 $\exp(S_{ij} - m_i)$，再次归约得到 s_i；最后，各线程独立写出概率。没有有效列的线程在最大值归约中贡献 $-\infty$，在求和归约中贡献零，仍然参与全部同步。这样，即使第一行只有一个有效元素，整个线程块的控制流程也保持一致。

例如取 $B_t = 256$，处理行 $i = 700$ 时，可见列为 $0, \ldots, 700$。线程 0 负责列 $0, 256, 512$；线程 189 负责 $189, 445$，下一项 701 已越过可见边界。因此，一行的长度可以大于线程块大小；增加长度表现为每个线程循环更多次。线程块数对应行数，也需要满足设备的网格维度限制。若把多个批次和多个头的行展平，则总行号 r 对应的因果位置应取 $i = r \bmod N$，不能直接把 r 当作掩码边界。

下面采用共享内存上的二叉树归约。设 B_t 为二的幂，线程先把局部结果写入共享数组；归约步长依次取 $B_t/2, B_t/4, \ldots, 1$。每一步中，前半部分线程组合两个位置的结果，再由块内屏障保证下一步读到已完成的写入。归约结束后，共享数组的第零项包含整行需要的统计量。

代码 12.1 线程块内的最大值或求和归约

```cpp
#include <cuda_runtime.h>
#include <math_constants.h>
#include <cmath>
#include <cstddef>
#include <limits>

// Contract: blockDim == (BT, 1, 1); every thread calls this helper.
// The final barrier lets the caller safely reuse the same scratch array.
template <int BT, bool ReduceMax>
__device__ __forceinline__ float block_reduce(float value, float* scratch
    ) {
    static_assert(BT > 0 && (BT & (BT - 1)) == 0,
                  "BT must be a positive power of two");
    const unsigned int tid = threadIdx.x;
    scratch[tid] = value;
    __syncthreads();

    for (unsigned int step = BT / 2; step > 0; step >>= 1) {
        if (tid < step) {
            const float other = scratch[tid + step];
            scratch[tid] = ReduceMax ? fmaxf(scratch[tid], other)
                                     : scratch[tid] + other;
        }
        __syncthreads();
    }
    const float result = scratch[0];
    __syncthreads();
    return result;
}
```

模板参数 ReduceMax 在编译时确定归约操作。每个线程都调用这个函数，包括局部结果为单位元的线程。循环结束时的屏障保证归约已经完成；随后先把 scratch[0] 读入每个线程自己的寄存器变量，再做一次屏障。这最后一次同步保证所有线程都已读取旧结果，后面的求和归约才可以安全地复用同一块共享内存。

核函数采用三次按行遍历。第二次遍历将指数临时写入输出缓冲区 P，第三次遍历再除以公共分母；这样每个有效位置只需计算一次指数。最后一次遍历使用相同的线程分工，因此每个线程归一化自己刚才写入的位置。P 的上三角部分尚未初始化，条件表达式会直接写零，不读取其中的旧内容。

代码 12.2 单头、单序列的因果归一化核函数

```
// One block per row. S already contains the 1/sqrt(d) scaling.
// S and P are disjoint row-major N-by-N device buffers; N >= 1.
// Only S[row, j] with j <= row must be finite; masked entries are not read.
template <int BT>
__global__ void causal_softmax_kernel(const float* __restrict__ S,
                                      float* __restrict__ P, int N) {
    __shared__ float scratch[BT];
    const std::size_t row = blockIdx.x;
    const std::size_t n = static_cast<std::size_t>(N);
    const std::size_t base = row * n;
    const std::size_t tid = threadIdx.x;

    // Inactive threads contribute the identity for a maximum reduction.
    float local_max = -CUDART_INF_F;
    for (std::size_t j = tid; j <= row; j += BT) {
        local_max = fmaxf(local_max, S[base + j]);
    }
    const float row_max = block_reduce<BT, true>(local_max, scratch);

    // P temporarily stores exponentials, owned by the same thread in both loops.
    // Every row includes its diagonal, so at least one exponential equals one.
    float local_sum = 0.0f;
    for (std::size_t j = tid; j <= row; j += BT) {
        const float value = expf(S[base + j] - row_max);
        P[base + j] = value;
        local_sum += value;
    }
    const float row_sum = block_reduce<BT, false>(local_sum, scratch);

    // The conditional avoids reading an uninitialized masked entry of P.
    for (std::size_t j = tid; j < n; j += BT) {
        P[base + j] = (j <= row) ? P[base + j] / row_sum : 0.0f;
    }
}
```

这里使用 `std::size_t` 计算行偏移，使乘积 iN 在较宽的无符号索引类型中形成。输入与输出缓冲区需要各自至少容纳 N^2 个单精度数，且二者的存储区间互不重叠。有效输入分数应为有限值；仅用于表示未来位置的上三角值不会被读取。

暂存指数和重新计算指数是两种保持同一数学结果的实现方式。暂存版本减少指数函数调用，同时增加一次中间结果写入；重新计算版本可以在最后一轮再次读取 S 并求指数。哪一种更快，取决于指数运算吞吐、显存带宽和缓存行为，需要按目标设备测量。上面的版本主要用于清楚展示归约与同步关系。

12.3.4 启动条件与实现检查

核函数通过一维网格启动 N 个线程块，每块固定为 256 个线程。启动之前应验证设备允许相应的网格大小、线程块大小和共享内存需求。共享数组只需 $256 \times 4 = 1024$ 字节，大小与 N 无关。较长上下文增加线程内循环次数和总行数，并不要求共享内存容纳整行。

代码 12.3 主机端的参数检查与异步启动

```
// The caller supplies disjoint buffers of at least N*N floats on this device.
// Returns argument/device/launch errors; does not wait for kernel completion.
cudaError_t launch_causal_softmax(const float* S, float* P, int N,
                                  cudaStream_t stream = nullptr) {
    constexpr int BT = 256;
    if (S == nullptr || P == nullptr || S == P || N <= 0)
        return cudaErrorInvalidValue;
    const std::size_t n = static_cast<std::size_t>(N);
    if (n > std::numeric_limits<std::size_t>::max() / sizeof(float) / n)
        return cudaErrorInvalidValue;

    int device = 0;
    cudaError_t status = cudaGetDevice(&device);
    if (status != cudaSuccess) return status;
    cudaDeviceProp prop{};
    status = cudaGetDeviceProperties(&prop, device);
    if (status != cudaSuccess) return status;
    if (N > prop.maxGridSize[0] || BT > prop.maxThreadsPerBlock ||
        BT > prop.maxThreadsDim[0] ||
        BT * sizeof(float) > prop.sharedMemPerBlock)
        return cudaErrorInvalidConfiguration;

    causal_softmax_kernel<BT><<<static_cast<unsigned int>(N), BT, 0, stream>>>(
        S, P, N);
    return cudaGetLastError();
}
```

这个辅助函数返回参数、设备查询或启动阶段发现的错误码，不等待核函数执行结束。同一流中的后续矩阵乘法按提交顺序读取 P；需要确认执行完成或测量耗时时，还应在合适的位置等待该流或记录设备事件。函数无法从裸指针确定缓冲区的实际容量，调用者仍需满足前述容量与不重叠条件。

正确性检查可以直接使用第 12.2.5 节的分数矩阵。预期输出为式 (12.13) 中的 P；随后计算 PV 应得到对应的 O。还应覆盖 $N = 1$、长度小于线程数、长度不等于线程数的整数倍以及长度大于线程数的情况。检查的对象包括有效行和接近一、上三角严格为零、输出有限，以及与高精度逐行参考计算的一致性。单精度归约改变加法顺序，通常会产生很小的舍入差

异，应根据数据类型比较数值误差。

性能上，一个线程块处理一行使行内通信局限于共享内存和块内屏障。相邻线程在同一次循环中访问相邻列，便于合并访存；早期行只有少量有效元素，部分线程参与同步但没有实际数据工作。更大的线程块可以减少长行的循环次数，也会增加线程和寄存器资源占用。线程数的选择需要同时考虑行长、活跃线程比例和设备并发能力。

12.4 两阶段推理与键值缓存

12.4.1 预填充与增量解码的时序

处理一个新的文本输入时，初始的多个词元都已知。预填充（prefill）把整个初始前缀转换为输入矩阵，并一次经过全部网络层。各位置虽然只能访问因果掩码允许的前缀，但可以一起进行投影、矩阵乘法和逐行归一化。最后一个位置的输出分布用于选择第一个新词元。

后续的增量解码（incremental decoding）每次只把刚生成的一个词元作为新的输入位置送入网络，同时复用历史位置已计算的状态。设初始前缀为 $t_0, \dots, t_{N-1}$。预填充完成后，每层都已经得到这 N 个位置的键和值；根据位置 $N-1$ 的最终输出，选择词元 t_N。此时词元 t_N 刚被选出，还没有经过网络，它的各层键和值尚未形成。

下一次前向传播把 t_N 放在绝对位置 N，逐层计算该新位置的表示和键值，并预测 t_{N+1}。所以“得到词元编号”与“形成该词元的各层缓存”发生在相邻的两个计算阶段。词元编号已经由词表选择直接得到，可以马上用于嵌入查表；生成回路不需要先把编号转为文本再分词。

表 12.3 初始前缀含 N 个词元时的前三次前向传播。缓存长度只统计已经输入网络的词元。

阶段	本次送入网络的词元	缓存行数	本次新词元
预填充	$t_0, \dots, t_{N-1}$	N	t_N
增量第 1 步	t_N，位置为 N	$N+1$	t_{N+1}
增量第 2 步	t_{N+1}，位置为 $N+1$	$N+2$	t_{N+2}

预填充对给定上下文进行表示计算，这里的阶段名称不表示模型必须执行文本摘要任务。聊天、续写、翻译等生成任务都可以采用同样的预填充与解码流程。

12.4.2 历史状态为什么可以复用

键值缓存（key-value cache，KV cache）保存已经计算出的历史键和值。它的正确性来自因果依赖，以及层内其他运算的逐位置性质。在模型参数固定、位置编号保持不变的确定性推理中，把未来词元追加到序列末尾，不会改变已有位置的表示。

这个结论可以按网络层数证明。首先，位置 i 的初始向量由 t_i 和位置 i 的嵌入相加得到。扩展序列不改变这两个量，所以旧位置的 $X^{(0)}_{i,:}$ 保持不变。假设第 ℓ 层所有旧位置的输入已经保持不变，考察该层位置 i 的计算。

层归一化只读取位置 i 的特征，因此它的结果不变，随后得到的 q_i, k_i, v_i 也不变。因果注意力仅使用位置 $0, \ldots, i$ 的键和值；新增的未来位置被屏蔽，因此第 i 行有效分数、归一化分母和加权输出都不变。输出投影、残差相加和前馈网络继续独立处理这一行，使第 $\ell+1$ 层的旧位置输入仍然不变。由此逐层归纳，所有层中历史位置的键和值都可以保存后复用。

缓存必须按层保存，因为第 ℓ 层的键和值由该层自己的输入与投影权重产生。只保存初始嵌入无法代替中间层的键值，只保存最后一层也不能支持前面各层的注意力。标准多头结构中，每层保存 $K^{(\ell)}, V^{(\ell)} \in \mathbb{R}^{B\times h\times N\times d}$；不同层的数值不同，尽管形状相同。

未来位置使用的是新的查询，它需要与历史键进行匹配，再对历史值做加权和。旧查询只用于生成旧位置的输出，因此标准增量解码不需要缓存历史 Q。同样，也不需要保存全部历史注意力概率矩阵。缓存 K/V 保留了后续新查询所需的历史信息，同时避免存储会随位置数平方增长的概率状态。

如果修改已有前缀、改变已有位置编号或替换模型参数，上述不变性条件就发生变化，受影响的缓存需要重新建立。这一条件也说明：缓存的有效性取决于生成时的实际计算依赖。

12.4.3 一个新增位置在各层中的计算

设某层、某头已经缓存 N 个位置。对于新位置 N，令 $x_N \in \mathbb{R}^{1\times D}$ 表示送入该头投影的归一化输入。只对这一行做三个投影，并将新键和值追加到对应的历史缓存：

$$
\begin{aligned}
&q_N = x_N W_Q, \qquad k_N = x_N W_K, \qquad v_N = x_N W_V,\\
&\widehat{K} = \begin{bmatrix} K_{0:N,:} \\ k_N \end{bmatrix}, \qquad \widehat{V} = \begin{bmatrix} V_{0:N,:} \\ v_N \end{bmatrix},\\
&o_N = \mathrm{softmax}\left(\frac{q_N \widehat{K}^{\mathrm{T}}}{\sqrt{d}}\right)\widehat{V}.
\end{aligned}
\tag{12.16}
$$

切片 $0:N$ 表示包含 0、不包含 N，即原来的 N 行。新查询的形状为 $1\times d$，扩展后的缓存为 $(N+1)\times d$，分数和权重为 $1\times(N+1)$，输出为 $1\times d$。当前新位置自己的键和值已经包含在缓存中，因此对角线也被正确保留。

当一次只新增一个位置，且缓存仅含当前与过去位置时，缓存中的全部列都满足因果条件，不再需要显式构造一个三角掩码。若存在填充位置或其他可见性限制，仍需按照这些限制屏蔽相应列。若一次追加多个位置，则每个新增位置的可见列数不同，仍需要带历史长度偏移的因果掩码。

第 12.3 节的核函数使用方阵中的局部行号作为因果边界。单词元解码的分数矩阵只有一行，但这一行对应的绝对位置是 N，其可见范围是 $0,\dots,N$。因此，把这个长向量作为局部第零行直接套用方阵核函数，只会错误地保留第一列。解码核函数需要使用实际可见长度，或者显式加入历史长度偏移。

所有头只计算新行之后，拼接它们的输出，再经过 W_O、残差、归一化和前馈网络，得到下一层的新行。下一层使用自己保存的 K/V 重复相同过程。完成全部层后，最后归一化与词表投影只需处理这个新位置，从而得到再下一个词元的分布。

第 12.2.5 节可以直接展示这种复用。先处理前两个位置，缓存键 $(1,0),(0,1)$ 和值 $(1,0),(0,2)$。追加第三个位置时，只需新的 $q_2=(\sqrt{2},\sqrt{2})$、$k_2=(1,1)$、$v_2=(3,1)$。新查询与三行键的缩放分数为 $(1,1,2)$，因此新输出为 $(1.940292,1)$，与一次处理全部三行的结果相同。前两个输出仍然是 $(1,0)$ 和 $(0.268941,1.462117)$，无需重新计算。

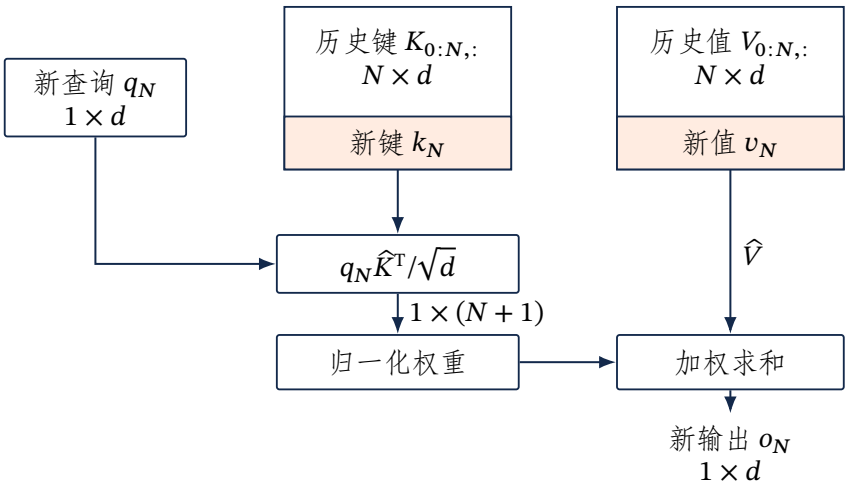


图 12.2 单层单头的增量注意力。历史键值保留，新键值追加为最后一行；只用新查询计算一个长度为 $N+1$ 的权重向量，再形成新输出。

12.4.4 节省的计算与增加的状态容量

比较两种实现处理到相同总长度 N 时，产生下一词元分布所做的工作。直接重算方式把全部 N 个位置重新输入网络；缓存方式只处理新增的一行，并访问已有键值。单头的主要差别如表 12.4 所示。

表 12.4 单头注意力在总长度 N 时的计算与持久状态。省略不影响阶数的常数。

对象	重算完整前缀	增量解码
查询、键和值投影	$\Theta(NDd)$	$\Theta(Dd)$
分数与值聚合	$\Theta(N^2d)$	$\Theta(Nd)$
注意力权重的计算行数	全部 N 行	新增的 1 行
持久注意力状态	不保存历史状态	$2Nd$ 个键值元素

缓存把重复计算历史位置的工作去掉了，但新查询仍要访问全部可见键值。因此，每步的注意力计算和缓存读取量仍随 N 线性增长。所有头的 $hd = D$，所以每层的新键值投影为 $\Theta(D^2)$ 量级，注意力匹配与聚合为 $\Theta(ND)$ 量级。输出投影与前馈网络也只计算新行，不再对历史行重复执行。

缓存容量由两个矩阵、批量、层数、上下文长度和特征宽度共同决定。在标准多头注意力中，若每个缓存元素占 b_{elem} 字节，则

$$n_{\text{KV}} = 2BLhNd = 2BLND, \qquad M_{\text{KV}} = 2BLND\, b_{\text{elem}} \text{ 字节}. \tag{12.17}$$

这个公式假定每个查询头都有对应的独立键和值头。它只统计已经使用的缓存元素；按最大长度预分配的实现，应把 N 替换为分配容量对应的长度。模型权重、其他激活和临时工作区还会额外占用显存。

例如，取 $B = 1$、$L = 32$、$D = 4096$、$h = 32$、$d = 128$、$N = 4096$，并用 16 位数保存缓存，即每个元素占 2 字节，则

$$M_{\text{KV}} = 2 \times 1 \times 32 \times 4096 \times 4096 \times 2 = 2\,147\,483\,648 \text{ 字节} = 2 \text{ GiB}. \tag{12.18}$$

这里 $1 \text{ GiB} = 2^{30}$ 字节。这是一组说明容量关系的标准多头配置。保持其他参数不变时，上下文长度或并行序列数加倍，已使用的缓存容量也加倍；网络层数增加同样线性增加缓存。

工程实现可以为每层预分配足够的键值空间，并用当前有效长度指示已写入部分。追加时只写新行，避免每步重新分配并复制全部历史缓存。若底层采用分块或分页存储，其逻辑内容仍然是式 (12.16) 中按位置排列的历史键值。

12.4.5 变长序列与键值头共享

一个批量中的不同序列通常具有不同的有效长度。设第 b 个序列的长度为 N_b，则标准多头结构实际保存的键值数据为

$$M_{\mathrm{KV}} = 2Lhd\, b_{\mathrm{elem}} \sum_{b=1}^{B} N_b. \tag{12.19}$$

如果为每个序列按统一上限 N_{cap} 预分配，则分配量为 $2BLhN_{\mathrm{cap}}d\, b_{\mathrm{elem}}$，可能大于有效数据量。按块分配可以减少未使用空间；块表、对齐以及最后一块中的空余位置仍会产生额外开销。不同请求的历史词元和中间表示通常不同，因此不能把一个请求的缓存直接用作另一个请求的缓存。

标准多头注意力为每个查询头配置独立的键值头。也可以让多个查询头共用一组键和值。设查询头数为 h_q、键值头数为 h_{kv}，每个头的维度为 d，模型宽度仍为 $D = h_q d$。等长批量的缓存容量统一写为

$$M_{\mathrm{KV}} = 2BLNh_{\mathrm{kv}}d\, b_{\mathrm{elem}}. \tag{12.20}$$

多头注意力（MHA）取 $h_{\mathrm{kv}} = h_q$；分组查询注意力（grouped-query attention，GQA）取 $1 < h_{\mathrm{kv}} < h_q$，通常让 h_{kv} 整除 h_q，每组查询头共享一个键值头；多查询注意力（multi-query attention，MQA）取 $h_{\mathrm{kv}} = 1$，所有查询头共享键和值。各查询头仍保留自己的查询，计算各自的匹配分数和输出。

在式 (12.18) 的配置中，32 个查询头若对应 8 个键值头，缓存由 2 GiB 降至 0.5 GiB。同一组内的查询还可以复用读入的键值数据，实际复用效果取决于核函数组织。这些结构使用不同形状的投影参数，必须与模型及预训练权重匹配；既定 GPT-2 权重的推理仍采用其原有多头结构。

12.5 计算规模与端到端性能

12.5.1 矩阵形状如何决定运算量

浮点运算次数（floating-point operations，FLOP）的计数需要先固定约定。把一次实数乘法和一次实数加法各计为一次运算，则一次融合乘加（fused multiply-add，FMA）计为 2 FLOP。对于 $(m \times k)(k \times n)$ 的稠密矩阵乘法，主导运算量按 $2mkn$ 计算。下面只统计主要矩阵乘法，暂不计偏置、归一化、激活和采样中的低阶或特殊函数开销。

在标准多头结构 $hd = D$ 下，所有头的三个输入投影合计为 $6ND^2$ FLOP，输出投影为 $2ND^2$ FLOP。单头 QK^{T} 需要约 $2N^2d$ FLOP，所有头合计 $2N^2D$；PV 同样需要 $2N^2D$。前馈网络的两次乘法合计 $4NDD_{\mathrm{ff}}$。由此得到表 12.5。

表 12.5 单序列、单层的主要矩阵乘法开销。预填充列采用完整稠密矩阵乘法的计数，解码列的可见长度为 N。

计算部分	长度 N 的预填充	一个位置的增量解码
全部查询、键和值投影	$6ND^2$	$6D^2$
分数矩阵 QK^{T}	$2N^2D$	$2ND$
值聚合 PV	$2N^2D$	$2ND$
多头输出投影	$2ND^2$	$2D^2$
前馈两次投影	$4NDD_{\mathrm{ff}}$	$4DD_{\mathrm{ff}}$

因此，一层预填充的主要算量为 $8ND^2 + 4N^2D + 4NDD_{\mathrm{ff}}$。取 $D_{\mathrm{ff}} = 4D$，可写成

$$F_{\mathrm{prefill}} \approx 24ND^2 + 4N^2D, \qquad F_{\mathrm{decode}} \approx 24D^2 + 4ND. \tag{12.21}$$

对包含 L 层的网络，层内部分再乘 L。最终词表投影需要另外计入：若计算全部位置的分布，需要约 $2NDV_{\mathrm{voc}}$ FLOP；纯生成的预填充只需最后一个位置的词表分布，可以把这部分降为约 $2DV_{\mathrm{voc}}$，而网络内部仍需计算各层的前缀键值。

式 (12.21) 的二次项来自位置两两匹配与聚合。若实现真正跳过上三角，则 QK^{T} 与 PV 的有效计算合计约为 $2N(N+1)D$，常数约减半；直接调用完整的稠密矩阵乘法时，上三角即使在归一化后为零，也不会自动省掉乘法。两种实现仍具有 $\Theta(N^2D)$ 的预填充算术阶数。

模型宽度也决定主导开销。举例而言，若 $N = D$ 且 $D_{\mathrm{ff}} = 4D$，完整稠密基线中位置匹配与聚合占主要矩阵乘法算量的 $4/(24+4) = 1/7$。长度增加时这一占比上升。因此，判断耗时需要同时考虑 N、D、前馈宽度和实际核函数效率，不能只根据某一步含有 N^2 就决定优化顺序。

12.5.2 大规模投影的参数与容量算例

以 GPT-3 的大规模配置中的投影尺寸为例，取 $D = 12\,288$、$h = 96$、$d = 128$、$L = 96$。这里只统计每层三个注意力输入投影及一个输出投影，忽略偏置。单头的一个输入投影有 $Dd = 12\,288 \times 128 = 1\,572\,864$ 个参数；全部头的三个投影合计 $3hDd = 3D^2$。输出矩阵有 D^2 个参数，因而 L 层共有

$$n_{\text{proj}} = L(3D^2 + D^2) = 4LD^2 = 57\,982\,058\,496. \tag{12.22}$$

它约为 58.0×10^9 个注意力投影参数。每个新位置都要使用这些投影，按每次乘加 2 FLOP 计，其主要算量为 $2n_{\text{proj}} \approx 115.964 \times 10^9$ FLOP。

表 12.6 给定宽度和层数下的投影规模。参数数为精确整数，运算量按乘加计数后取近似显示。

对象	参数数	16 位容量	单位置算量
单头的一种输入投影	1 572 864	3 MiB	3.146 MFLOP
单层全部输入投影	452 984 832	864 MiB	0.906 GFLOP
单层输出投影	150 994 944	288 MiB	0.302 GFLOP
96 层四类投影	57 982 058 496	108 GiB	115.964 GFLOP

容量按每个参数 2 字节计算，其中 $1\ \text{MiB} = 2^{20}$ 字节、$1\ \text{GiB} = 2^{30}$ 字节；运算量前缀采用十进制，即 $1\ \text{MFLOP} = 10^6$ FLOP、$1\ \text{GFLOP} = 10^9$ FLOP。显存容量的二进制单位与运算计数的十进制前缀需要分别处理。

这些结果仅来自四类投影矩阵。上下文相关的匹配与聚合、前馈网络、词表投影以及各类中间状态都还未计入，因此 108 GiB 不是完整模型的总显存需求，115.964 GFLOP 也不是生成一个词元的全部计算量。这个算例的用途是把矩阵尺寸、参数存储和单位置算量对应起来。

12.5.3 预填充与解码的资源需求

预填充中的投影具有许多输入行，同一权重块可以服务多个位置，适合用分块矩阵乘法增加数据复用。单序列增量解码只有一行输入，投影退化为向量与矩阵乘法，单次读入的权重参与的计算更少。批量处理多个序列可以增加同时参与计算的行数，但同时也会增加各序列的缓存容量和键值读取量。

从算术强度（arithmetic intensity）可以看出这个差别。假设一个投影权重矩阵含 Dd 个元素，本次需要把它们全部从显存读取，每元素占 b_{elem} 字节。只处理一个新位置时，主要算量为 $2Dd$，仅对权重读取量计算的算术强度约为 $2/b_{\text{elem}}$ FLOP/字节。若以 16 位保存权重，

这个比值约为 1 FLOP/字节。多个输入行共同使用权重块时，可以把这一次读取摊到更多乘加操作上。

若同时统计输入读取和输出写入，可进一步量化批量带来的复用。设一个线性层的权重形状为 $m \times n$，处理 B 行输入。假定输入、权重各读取一次，输出写入一次，每个元素占 b_{elem} 字节，忽略偏置和其他操作，则

$$AI_{\text{linear}} \approx \frac{2Bmn}{b_{\text{elem}}(mn + Bm + Bn)}. \tag{12.23}$$

取 $m = 1024$、$n = 4096$、$b_{\text{elem}} = 2$，$B = 1$ 时约为 0.999 FLOP/字节；$B = 512$ 时约为 315.08 FLOP/字节。输入行数增多使权重读取被更多计算复用，而输入和输出的数据量也随之增长。因此，这个完整传输估计与前面只统计权重读取的估计具有不同分母。真实实现若重复读取数据，算术强度还会降低。

不同请求可以使用同一组投影权重，但通常必须读取各自的键值缓存。增加请求批量不会自动减少每个查询所需的历史键值读取，也不会自动提高这一部分按缓存读取计算的算术强度；它仍可能通过增加并行工作改善设备利用率。批量扩大时，还需同时检查缓存容量、序列长度差异及调度开销。

注意力读取缓存也具有相似的限制。单头一个新查询的匹配与聚合共约 $4Nd$ FLOP，需要读取约 $2Nd$ 个历史键值元素；只考虑这些读取时，算术强度同样约为 $2/b_{\text{elem}}$。这解释了为什么缓存虽然减少计算，较长上下文的逐词元解码仍可能受显存带宽限制。实际缓存命中、并行批量、数据类型和核函数融合都会影响最终表现。

设某段运算需要 F FLOP、传输 M 字节，设备在相应数据类型上的峰值算术吞吐为 R_{comp}、显存带宽为 R_{mem}，则理想时间至少满足

$$T \geq \max\left(\frac{F}{R_{\text{comp}}}, \frac{M}{R_{\text{mem}}}\right). \tag{12.24}$$

真实时间还包含无法完全重叠的访存、同步、启动和布局转换等开销。因此，算量减少多少与最终时间减少多少之间，还隔着硬件资源利用和实现方式。

12.5.4 分块融合如何减少中间矩阵读写

基线预填充通常显式生成分数矩阵 S 和权重矩阵 P。每个头每张矩阵包含 N^2 个元素，矩阵乘法、归一化和值聚合在相继的核函数中对它们写入和读取。例如 $B = 1$、$h = 32$、$N = 4096$ 时，一张所有头合计的矩阵有 $32 \times 4096^2 = 536\,870\,912$ 个元素；若以 32 位保存，仅这一张矩阵就占 2 GiB，同时保留 S 和 P 则需要 4 GiB。这部分是单层的临时状态，与跨层持久保存的 KV 缓存承担不同的作用。

闪电注意力（FlashAttention）通过分块与核函数融合，在片上保存局部查询、键值和中间统计量，使完整的 $N \times N$ 分数与概率矩阵不必写回显存。对一组查询，逐块读取键和值，计算局部分数、更新归一化统计量，并累积输出。它改善的是数据搬运和中间存储；完整稠密注意力需要处理的位置关系仍然是二次规模。

分块计算要保持相同的归一化结果，需要在读取后续块时正确调整已经累积的量。对某一个固定查询，设已经处理的位置集合为 A，分数为 s_j。保存最大值 $m = \max_{j \in A} s_j$、指数和 $\lambda = \sum_{j \in A} e^{s_j - m}$，以及未归一化输出向量 $u = \sum_{j \in A} e^{s_j - m} v_j$。当前输出就是 u/λ。

假设随后读入另一个非空可见块 J，其最大分数为 m_J。新的公共最大值取 $m' = \max(m, m_J)$。由于 $e^{s_j - m'} = e^{m - m'} e^{s_j - m}$，旧的累计量只需乘同一个校正因子，就能与新块合并：

$$
\begin{aligned}
m' &= \max(m, m_J), \qquad \alpha = \exp(m - m'), \\
\lambda' &= \alpha \lambda + \sum_{j \in J} \exp(s_j - m'), \\
u' &= \alpha u + \sum_{j \in J} \exp(s_j - m') v_j .
\end{aligned}
\tag{12.25}
$$

先用第一个非空可见块初始化这些量，再反复使用此递推；因果掩码下没有可见元素的块直接跳过。读完全部可见位置后，输出 u/λ，即与整行归一化后加权求和相同的数学结果。有限精度下，运算顺序的改变可能带来舍入差异。这个递推说明了为什么只保留行统计量与一个长度为 d 的累加向量，就能避免存储整行的全部概率。

KV 缓存复用不同生成步骤之间已经计算的历史键值，分块融合则减少同一次注意力计算内部的显存往返。这两个方向可以共同使用：预填充高效地建立各层缓存，后续增量解码继续访问这些缓存，并使用适合单查询或小批量形状的核函数。

12.5.5 从核函数性能到完整生成时间

完整推理包括初始分词和嵌入、预填充、重复的增量解码、词表选择以及必要的数据传输。首个词元的延迟主要涉及初始输入处理和预填充；后续逐词元延迟取决于增量路径。测量时应明确输入长度、输出长度、批量和数据类型，并将这两个阶段分别观察，再统计完整生成任务的时间。

局部加速对整体性能的影响由它在总时间中的占比决定。若某部分原先占总时间的比例为 f，优化后加速 r 倍，其余部分耗时保持不变，则整体加速比为

$$R_{\text{total}} = \frac{1}{(1-f) + f/r}. \tag{12.26}$$

例如某部分占原时间的 60%，即使该部分加速 4 倍，整体加速也只有 $1/(0.4 + 0.6/4) \approx 1.82$ 倍。这个假设算例说明，优化时应结合实际耗时占比评估收益；核函数之间多出的缓冲区转换或同步也可能抵消局部收益。

给定预训练权重的 GPT-2 推理项目可以围绕这条计算链展开。第一阶段把已有 CPU 参考实现对应的基本运算移到 GPU，并建立数值一致性；第二阶段分析单个核函数和完整推理流程的耗时，提出并实现相应优化；第三阶段继续优化并分析完整生成任务的性能。预训练参数保持固定，工作重点是正确实现和加速前向推理。

从计算依赖来看，投影和前馈主要改变特征表示，因果注意力把历史信息汇入当前位置，逐行归一化把分数变为稳定权重，键值缓存让已完成的位置计算得到复用。对应到实现，矩阵乘法的数据复用、归约的同步组织、缓存的按层维护以及中间结果的读写，共同决定完整生成过程的性能。

第 13 讲

Atomic Operations and Histogramming

13.1 并发更新与原子操作

13.1.1 读—改—写与更新丢失

共享计数器的更新通常包含读取旧值、计算新值和写回三个步骤，称为读—改—写（read-modify-write）。例如，`++bins[b]` 在逻辑上执行“读出第 b 个计数器，将它加一，再写回”。即使源程序只写了一条语句，也不能据此认为其他线程无法插入执行。

设共享计数器初值为 0，线程 A 与线程 B 各执行一次加一。每个线程分别持有局部变量 r_A、r_B；这些变量只记录该线程读取到的旧值。若 A 完成整个更新后 B 才读取，则 A 读到 0、写入 1，B 随后读到 1、写入 2。交换两个线程的先后顺序，最终也得到 2。然而，表13.1中的交错执行会丢失一次更新。

表 13.1 两个线程各自完成加一，最终计数却只有一

时刻	线程 A	线程 B	共享值
1	读取旧值，$r_A = 0$		0
2	计算 $r_A + 1 = 1$		0
3		读取旧值，$r_B = 0$	0
4	写回 1		1
5		计算 $r_B + 1 = 1$	1
6		写回 1	1

两个线程都完成了自己的指令，但它们根据同一个旧值计算，第二次写回覆盖了第一次更新。问题出在共享状态的访问没有形成正确的先后关系。多个线程并发访问同一内存位置、至少一次为写入，并且这些冲突访问缺少适当的同步或原子保证时，就存在数据竞争（data race）。上表只展示一种直观的错误交错；含数据竞争的程序不能依靠这几种手工列举的结果来保证行为。

类似的问题也出现在状态占用中。若用 0 表示座位空闲、1 表示已占用，两个请求都可能先读到 0，再分别写入 1，于是它们都认为自己取得了同一个座位。计数需要保护“读取后累加”的整体，占用操作需要保护“检查条件后修改”的整体。两者都要求把依赖旧状态

的判断或计算与相应写回联系起来。

是否发生冲突由最终访问的地址决定。两个线程读取不同输入，并不保证它们写入不同输出。例如，不同键可能散列到同一个表项，不同顶点也可能向共同邻居累加贡献。直方图正具有这种结构：输入位置可以独立划分，而输入的值可能把多个线程映射到同一个计数器。

13.1.2 互斥、线程束与执行进度

互斥（mutual exclusion）要求同一时刻至多有一个参与者执行受保护的更新。将读取、计算和写回放入临界区（critical section），并在进入前获取锁、退出后释放锁，可以使整个更新按某种顺序完成。例如，对同一把互斥锁，只有 A 释放后 B 才能进入，或者只有 B 释放后 A 才能进入。两种顺序都避免了上面的覆盖问题。

这种方法依赖一个进度条件：获得锁的线程必须能够继续执行并最终释放锁。在传统的线程束（warp）共同推进模型中，同一束中的一个线程可能获得锁，其余线程则停留在等待锁的循环中。若执行调度一直停留在等待分支，使持锁线程的临界区与释放语句无法得到执行，就出现循环等待：等待者需要锁被释放，持锁者又无法越过当前执行路径去释放锁。这是把简单自旋锁直接搬入这种执行模型时可能发生死锁的原因。

上述死锁分析以线程束内的具体调度模型为前提。独立线程调度（independent thread scheduling）改变了线程束内线程推进的方式，因此分析锁时仍需同时检查参与线程的执行方式、内存可见性以及持锁者的进度。对于单个计数器加一这样的操作，可以直接使用硬件支持的原子更新，避免手工管理锁住和释放的临界区。

13.1.3 原子更新的保证与边界

原子操作（atomic operation）将指定地址上的读—改—写作为一个不可分割的操作完成。对于同一地址上、作用域相互覆盖的冲突原子操作，每次更新都必须看到某个完整的先前状态；硬件不会让另一原子更新插入这次操作的中间阶段。因此，同地址的冲突更新在效果上形成一个串行顺序。

若计数器初值为 h_0，所有 m 次更新都通过原子加一完成，而且没有溢出或其他未同步访问，那么最终值就是 $h_0 + m$。执行顺序可以未知，结果仍然确定，因为整数加法满足交换律和结合律。对于其他更新，顺序仍可能影响结果。例如，对同一位置分别执行原子交换，写入 3 和写入 7，最终值取决于哪个交换最后完成。

原子保证只覆盖该操作指定的状态转移。对两个不同计数器先后执行两次原子加法，并不会自动把二者合成一个不可分割的事务；另一个线程可能观察到其中一次已经完成、另一

次尚未完成。对单个计数器的原子加法也不提供“所有线程都已完成”的通知。后面在共享直方图中使用的块内同步，负责保证一个阶段全部结束后才开始下一阶段；原子加法则负责该阶段内每次冲突更新的正确性。

原子性与顺序

同一地址的原子更新不会互相覆盖掉未完成的中间结果，但线程之间谁先更新通常没有预定顺序。计数结果的确定性还依赖更新运算本身；完整结果应在所有更新结束后读取。

13.1.4 CUDA 原子接口与比较交换

CUDA 提供由编译器识别的内建原子函数。`atomicAdd` 接受目标地址和增量两个参数，原子地读出旧值、加上增量、写回结果，并返回更新前的旧值。例如，一次调用向计数器加入 3，若原子动作读到 4，它就写入 7，并向调用者返回 4。多个线程争用同一地址时，返回值也服从实际形成的串行顺序。

`atomicAdd` 有不同数据类型的重载，常见类型包括整数与浮点数；具体重载的支持取决于设备和数据类型。计数问题通常使用整数。其他原子原语包括减法、最小值、最大值、按位与、按位或、按位异或、交换以及比较交换。对于可交换的整数累加，原子操作保持计数正确；浮点加法还受到舍入和累加顺序的影响。

比较交换（compare-and-swap，CAS）把“检验当前位置是否仍为预期值”与“条件成立时写入新值”合成一个原子动作。用 v 表示操作开始时该地址的值，e 表示预期值，d 表示希望写入的值，其状态变化为

$$v_{\text{after}} = \begin{cases} d, & v = e, \\ v, & v \neq e. \end{cases} \tag{13.1}$$

在抽象描述中，可以让 CAS 返回“成功”或“失败”。CUDA 的 `atomicCAS` 接口依次接受目标地址、预期值和目标新值三个参数。无论替换是否成功，它都返回操作之前观察到的旧值。因此，返回旧值等于预期值，才是替换成功的判断条件。

这一区别会直接影响程序逻辑。如果旧值为 0、预期值也为 0，替换成功时返回 0；直接将返回值当作布尔量，会把这次成功当成失败。反过来，如果当前值为 5、预期值为 0，替换失败时返回 5；将其转换为布尔量又会得到真。普通的条件判断加赋值本身也无法实现式(13.1)要求的不可分割性，真正的保证来自原子原语。

对座位占用状态，可以执行 `old = atomicCAS(&seat, 0u, 1u)`。若返回 0，说明本次调用取得了此前空闲的座位；若返回 1，说明调用时座位已被占用。两个并发请求都使用这一原语时，只有一个能够把同一次空闲状态成功改为占用状态。

无后缀的传统 CUDA 原子接口采用设备作用域；对于全局计数器，这允许同一设备中不同线程块共同原子更新。它们不会自动建立其他地址的读写顺序。共享内存对象则属于各自线程块，只有该块中的线程参与访问。地址必须有效并满足类型对齐要求，初始化需要先于并发更新，所有冲突访问也必须遵守相容的原子与同步规则。

13.1.5 用比较交换构造原子加法

CAS 可以构造“根据旧值计算新值”的更新。线程先取得一个估计值，计算目标值，然后尝试仅在共享值仍与估计相等时写入。如果比较时共享值与估计值不同，CAS 会失败并返回新的观测值；当前线程据此重新计算和重试。这样，线程私有的计算可以被打断，最终提交更新的动作仍然是原子的。

代码13.1用无符号整数实现这一过程。函数名与 CUDA 内建的 `atomicAdd` 区分开，且整个实现中的共享访问都使用 CAS。初始调用 `atomicCAS(address, 0u, 0u)` 只用于取得当前值：当前值为 0 时写回相同的 0，否则保持原值。

代码 13.1 由 CAS 构造无符号整数原子加法

```
__device__ unsigned int atomic_add_cas(
    unsigned int* address, unsigned int value)
{
    unsigned int old = atomicCAS(address, 0u, 0u);
    while (true) {
        unsigned int assumed = old;
        old = atomicCAS(address, assumed, assumed + value);
        if (old == assumed) {
            return assumed;
        }
    }
}
```

变量 `assumed` 保存本轮准备提交时的预期旧值；`old` 在 CAS 返回后变为该原子动作实际观察到的值。若二者相同，说明比较时共享值确实等于 `assumed`，写入的正是这个当前值加上 `value`，函数返回此次更新前的值。若二者不同，本次 CAS 没有修改共享状态，下一轮直接使用返回值作为新估计。

例如，初始计数为 0，A、B 各加 1，两者都先估计为 0。A 的 CAS 将 0 改成 1，返回 0，因此成功。B 随后尝试把 0 改成 1，但当前值已经是 1，于是 CAS 返回 1 且不写入。B 下一轮改为尝试把 1 改成 2，成功后返回 1。最终计数为 2。

正确性的关键是成功 CAS 的时刻。在线程准备估计值或计算加法期间，其他线程都可以继续更新；只有当提交时的共享值满足比较条件，当前计算才会生效。因此，函数整体可以被看作在成功 CAS 时刻完成了一次加法。竞争越激烈，失败重试可能越多，不能为每个线

程保证固定的重试次数或取得更新机会的顺序。已有合适的内建原子加法时，应直接使用内建函数；这个构造用于解释 CAS 如何提供更一般的更新机制。

无符号整数在溢出时按其位宽回绕。若把它作为通常意义上的计数器，仍须保证实际计数不超过可表示范围。原子性保证更新不会相互丢失，计数容量则决定最终数值能否完整表示真实数量。

13.2 直方图与并行数据划分

13.2.1 桶映射与计数不变量

直方图（histogram）统计数据落入不同类别或区间的次数。设输入为 $x_0, x_1, \dots, x_{N-1}$，共有 B 个桶（bin），桶映射 $b(x)$ 把每个有效输入映射到 $\{0, 1, \dots, B-1\}$。第 j 个桶的计数为

$$H_j = \sum_{i=0}^{N-1} \mathbf{1}\{b(x_i) = j\}, \qquad j = 0, \dots, B-1, \tag{13.2}$$

其中指示函数 $\mathbf{1}\{\mathcal{E}\}$ 在条件 $\mathcal{E}$ 成立时取 1，否则取 0。这里每个输入恰好进入一个桶，因此 $\sum_{j=0}^{B-1} H_j = N$。这一总计数不变量适合检查输入遗漏、重复处理或更新丢失，不过总数相等还不足以保证各桶都正确。

对于一个字节表示的灰度值，$x_i \in \{0, \dots, 255\}$，可以取 $B = 256$、$b(x_i) = x_i$。若图像宽度为 W_{img}、高度为 H_{img}，输入长度为 $N = W_{\text{img}} H_{\text{img}}$；随后只需按一维数组遍历。以下代码使用 `unsigned char` 表示输入字节，避免普通 `char` 的符号性使较大字节值变成负下标。

桶也可以表示数值区间。例如，将 256 种灰度值分成四个宽度为 64 的区间，可以取 $b(x) = \lfloor x/64 \rfloor$。此时 0 与 63 进入桶 0，64 进入桶 1，255 进入桶 3。映射规则改变了输出桶数，但每个有效像素仍恰好贡献一次计数。

字母频率也是直方图。对输入字符串

```
Programming Massively Parallel Processors
```

若统一大小写、跳过空格，并令字母 A 到 Z 对应 26 个桶，则共有 38 个有效字母。表13.2列出所有非零计数；其余字母计数为 0。这里“跳过空格”是桶映射前的数据筛选规则，所以总计数应等于有效字母数。

直方图可以概括图像灰度、类别频数或数值区间分布。不同应用的差别主要在桶映射，而并行计算面对的共同问题是：每个输入贡献一个增量，多个增量可能同时指向同一个桶。

表 13.2 字母直方图的完整非零计数

字母	A	C	E	G	I	L	M
次数	4	1	3	2	2	4	3
字母	N	O	P	R	S	V	Y
次数	1	3	3	5	5	1	1

13.2.2 输入划分与输出地址的关系

将输入分给不同线程，可以保证每个输入位置只被处理一次，但输出桶仍由输入值决定。例如，四个线程当前读到的字母分别为 P、M、M、E，则这轮需要给 P 和 E 各加 1，给 M 加 2。处理两个 M 的线程即使读取了不同位置，也会对同一个输出地址形成竞争。

最直接的并行方案让每个线程读入一个或多个数据，并对相应桶执行原子加一。原子操作解决输出的冲突，输入读取方式则影响全局内存访问效率。这两个方面可以分别设计：相邻线程可以读取相邻数据，但它们所更新的桶仍可能离散或高度集中。

设网格含 G 个线程块，每块有 T 个线程，总线程数为 $P = GT$。为便于比较，暂设每个线程恰好处理 c 个输入，即 $N = Pc$。一种划分方式给全局编号为 g 的线程分配一个连续的长度为 c 的区间，第 k 轮读取位置 $gc + k$。虽然单个线程连续读取，但是同一轮中相邻线程的地址间隔为 c，会使一个线程束的访问分散。

另一种方式让线程 g 在第 k 轮读取 $g + kP$。于是同一轮中线程 g、$g + 1$ 读取相邻位置，而每个线程在不同轮之间跨过 P 个元素。表13.3用 $P = 4$、$c = 4$ 给出完整地址序列。

表 13.3 四个线程处理十六个输入的两种分配

线程	每线程连续区间	每轮相邻线程访问相邻位置
0	0, 1, 2, 3	0, 4, 8, 12
1	4, 5, 6, 7	1, 5, 9, 13
2	8, 9, 10, 11	2, 6, 10, 14
3	12, 13, 14, 15	3, 7, 11, 15

在第一轮中，两种方案的线程组分别访问 0, 4, 8, 12 和 0, 1, 2, 3。合并访存（coalesced memory access）关注的是同一条访存指令下线程束内各线程地址的组合。后者让访问落在连续地址范围，更容易用少量内存事务完成；实际事务数量还取决于访问宽度、对齐和硬件粒度。四线程表只是缩小的索引示例，相同原则适用于完整线程束。

13.2.3 网格跨步核函数与完整启动条件

将上面的第二种划分写成循环，得到网格跨步循环（grid-stride loop）。设线程块编号为 r、块内线程编号为 t，则线程的全局编号为 $g = rT + t$，跨步长度为 $P = GT$，依次处理 $g, g+P, g+2P, \ldots$ 中小于 N 的索引。代码13.2直接对公共直方图做全局原子加法。

代码 13.2 统计 256 种无符号字节值的基本核函数

```
__global__ void histogram_global(
    const unsigned char* input, std::size_t n,
    unsigned int* hist)
{
    std::size_t i = std::size_t(blockIdx.x) * blockDim.x
                  + threadIdx.x;
    std::size_t stride = std::size_t(gridDim.x) * blockDim.x;

    for (; i < n; i += stride) {
        unsigned int bin = input[i];
        atomicAdd(&hist[bin], 1u);
    }
}
```

长度、索引和跨步长度均使用 `std::size_t`。乘法之前先提升类型，避免先以较窄整数计算乘积，再将已经溢出的结果赋给宽类型。代码需要包含 `<cstddef>` 和 CUDA 运行时头文件。这里的核函数采用一维线程块与一维网格，输入与输出数组互不重叠，输出恰有 256 个计数器。

跨步划分的完整性可以直接证明。任取 $i \in \{0, \ldots, N-1\}$，带余除法给出唯一分解 $i = kP + g$，其中 $0 \le g < P$。因此索引 i 恰由线程 g 在第 k 轮处理，既不会遗漏，也不会重复。末尾不足一轮时，条件 $i < N$ 使无对应输入的线程停止循环。网格线程数可以小于输入数，也可以大于输入数。

核函数只累加，因而一次独立统计开始前必须先将公共直方图清零。若希望跨多批输入累积同一结果，则只在第一批之前清零，后续各批需要遵守同样的访问顺序。对于 32 位无符号计数器，每个桶最终计数应不超过 $2^{32} - 1$；这项要求约束实际计数，与输入索引使用宽整数是两个不同问题。

代码13.3给出一个完整的异步启动函数。输入已经位于设备内存，调用者提供合法的线程块数和每块线程数。清零与核函数提交到同一个流（stream），使核函数在清零完成后才开始使用输出。

代码 13.3 同一流中先清零再启动统计

```
cudaError_t launch_histogram_global(
    const unsigned char* input, std::size_t n,
    unsigned int* hist, unsigned int blocks,
    unsigned int threads, cudaStream_t stream)
{
    if (blocks == 0 || threads == 0) {
        return cudaErrorInvalidConfiguration;
    }
    cudaError_t status = cudaMemsetAsync(
        hist, 0, 256 * sizeof(unsigned int), stream);
    if (status != cudaSuccess || n == 0) {
        return status;
    }
    histogram_global<<<blocks, threads, 0, stream>>>(
        input, n, hist);
    return cudaGetLastError();
}
```

返回成功表示清零和启动请求已按上述规则提交，异步核函数此时可能仍在执行。主机消费结果之前，需要等待相应流完成并检查完成状态；若之后在同一流中安排设备上的后续计算，则由流内顺序保证先后关系。该函数在 $N = 0$ 时仍把输出清零，然后跳过核函数启动。

基本算法总共读取 N 个输入元素，执行 N 次全局原子加。不同输入值决定这些更新如何分布到桶上：分布均匀时，许多桶可以同时接受更新；若大量输入具有相同值，同一个计数器就成为热点。原子操作保证后一种情况下仍得到正确结果，但其性能需要进一步分析。

13.3 原子操作的延迟与吞吐

13.3.1 同一地址上的串行服务

吞吐率（throughput）表示单位时间完成多少次操作，延迟（latency）表示一次操作从发起到完成经历多久。对普通内存访问，多个独立请求可以重叠，因此一个请求延迟较长，并不必然使总吞吐率等于该延迟的倒数。原子热点的特殊之处在于，很多请求依赖同一个地址上前一次更新的结果。

考虑一个简化模型：同一地址每次原子更新独占服务时间为 τ，下一次冲突更新必须在前一次服务完成后才能开始。对持续饱和的请求序列，这个地址的最高更新速率约为

$$R_{\text{addr}} \le \frac{1}{\tau}. \tag{13.3}$$

这里 τ 是模型中的持续服务间隔，不是某个线程排队后测到的总等待时间。若同时到达 q 个更新，最后一个请求可能需要等待约 $q\tau$ 才完成，但设备在这段时间内仍以每 τ 完成一次更新的速率工作。

假设这个服务过程需要超过 1000 个时钟周期，那么同一热点地址每周期完成的更新少于 1/1000 次。相对于“每周期一次更新”的理想基准，速率降到了不足千分之一。这个比较的对象是单个热点地址的更新能力；若多个地址可以独立服务，它们仍可能并行贡献总吞吐。

令桶 j 一共接收 H_j 次更新。若采用相同服务时间 τ，即使忽略读取输入、其他桶和所有调度开销，完成统计的时间仍受到最长串行链约束：$T_{\text{hot}} \geq \max_j H_j\tau$。所有输入进入同一个桶时，这条链长为 N；输入接近均匀分布时，每个桶大约只承担 N/B 次更新。这个下界说明了输入分布如何影响竞争，但多个桶仍可能共享内部执行资源，因此它本身不是完整的设备性能预测。

更多线程有助于提出更多独立请求，但不会让同一地址上必须依次生效的更新同时完成。如果原子热点已经饱和，继续增加等待线程主要会增加排队长度。优化可以从两方面着手：缩短服务时间，或者减少需要到达同一个公共地址的更新数量。

13.3.2 从显存到二级缓存和共享内存

若原子读—改—写必须经过动态随机存取存储器（DRAM），一次更新需要取得旧值并提交新值，长距离数据传输和存储访问会使完整服务时间较大。在以完整读、写往返计时的模型中，这包含读延迟、写延迟以及内部路由和传输开销。对同地址冲突操作，后续更新还必须等待当前更新的完整结果。

二级缓存（L2 cache）可以缩短这一路径。目标位置驻留在 L2 时，支持相应原子处理的硬件可以在更近的位置完成更新，避免完整的 DRAM 往返。不同线程块仍然访问同一公共全局对象；如果它们都更新同一桶，冲突仍存在，只是每次服务可能更快。

共享内存（shared memory）属于线程块，存储距离更近，原子更新通常具有更短的服务路径。它不能让所有线程块直接访问同一个共享数组。要用共享内存统计整份输入，需要让各块各自维护局部直方图，随后把局部结果合并到全局输出。因此，从 L2 的硬件加速进一步走向共享内存，需要算法配合。

这里的层次比较说明机制，不为某一设备指定固定延迟。实际性能还依赖原子数据类型、目标分布、缓存状态和存储系统实现。无论在哪个层次，同一个计数器仍必须表现为一串正确的原子更新。

表 13.4 三种原子更新位置及其作用

更新位置	延迟来源与改进	仍需处理的限制
DRAM 路径	读、写往返与内部传输构成较长服务过程	同地址排队，完整服务时间较长
L2 路径	目标驻留缓存时可避免 DRAM 往返	全部线程块仍可能争用同一公共桶
共享内存	块内就近更新局部桶	块内同桶更新仍串行，需要跨块合并

13.3.3 性能优化的两个方向

缩短服务时间时，更新次数可以完全不变。例如，把一个块的 n_r 次更新转移到共享内存后，这些输入仍各贡献一次加一，但每次操作经过较短的局部路径。减少公共更新次数时，则先把多次相同目标的贡献合成局部计数，再向公共桶提交一个较大的增量。

设某线程块处理的输入中有 400 个值等于 7。直接算法会对公共桶 7 做 400 次原子加一；局部统计可以先在该块中得到计数 400，最后只对公共桶 7 做一次原子加 400。每个输入的贡献仍然保留，而公共地址只接收一次更新。共享内存私有化同时利用了更近的局部存储和较少的公共更新。

评价这种变换时，要计入新增的局部数组清零、阶段同步和结果合并。它是否划算取决于每个块能把多少输入更新聚合成多少个非零桶。下一节以完整算法和操作数量明确这些条件。

13.4 线程块私有化与共享内存

13.4.1 局部统计与全局结果的等价性

私有化（privatization）为不同参与者维护独立的输出副本，先在副本中积累局部结果，再把这些结果合并到公共输出。在直方图中，适合让每个线程块维护一份共享内存直方图。这里的“私有”是相对于其他线程块而言；同一块内的线程仍共同访问该副本。

令第 r 个线程块处理的输入索引集合为 $\mathcal{I}_r$，其中 $r = 0, \dots, G-1$。这些集合互不相交，且并集包含全部输入。该块的局部直方图记为 $H_j^{(r)}$，则

$$H_j^{(r)} = \sum_{i \in \mathcal{I}_r} \mathbf{1}\{b(x_i) = j\}, \qquad H_j = \sum_{r=0}^{G-1} H_j^{(r)}. \tag{13.4}$$

第二个等式来自对输入求和的重新分组：每个输入只属于一个集合，所以它对相应桶的那一次贡献只出现一次。局部累加改变了加法的分组方式，不同块完成和合并的顺序也可能不同。

整数计数在不溢出的条件下满足结合律和交换律，因此这两种变化保持结果不变。

更一般地，如果要任意重新分组并调整局部结果的提交顺序，所用合并运算需要满足相应的结合与交换性质。若某个任务要求保留更新的先后顺序，就需要另外保存和执行该顺序。对当前的非负整数计数，逐块合并可以直接使用原子加法。

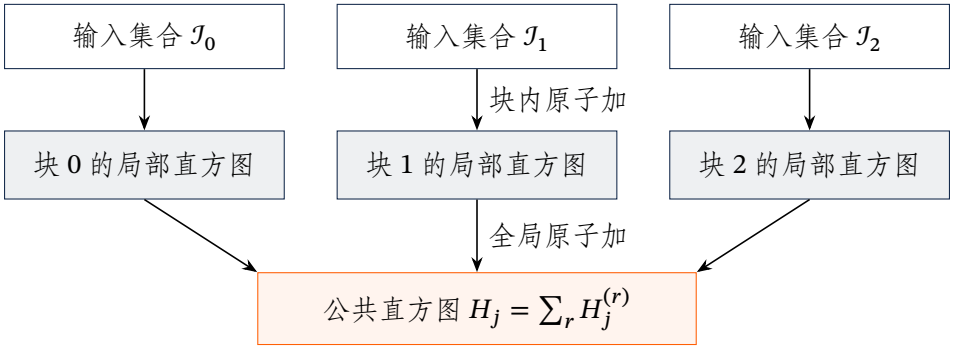


图 13.1 三个线程块先独立形成局部计数，再逐桶合并到公共结果。

在网格跨步划分下，$\mathcal{I}_r$ 通常由多个起点相隔 GT 的连续小段组成，不必是输入中的一个连续大区间。式(13.4)只要求集合构成无重复、无遗漏的划分；存储地址上是否连续不影响求和恒等式。

13.4.2 初始化、局部统计与全局合并

每个线程块执行三个阶段。首先将自己的共享数组清零；然后处理分配到的输入，对该数组中的目标桶做原子加；最后将每个非零局部桶的计数原子加入全局输出。代码13.4给出完整实现，输入仍为 256 种无符号字节值。

代码 13.4 共享内存私有化的完整直方图核函数

```
__global__ void histogram_private(
    const unsigned char* input, std::size_t n,
    unsigned int* hist)
{
    __shared__ unsigned int shared_hist[256];

    for (unsigned int b = threadIdx.x; b < 256;
         b += blockDim.x) {
        shared_hist[b] = 0u;
    }
    __syncthreads();

    std::size_t i = std::size_t(blockIdx.x) * blockDim.x
                  + threadIdx.x;
    std::size_t stride = std::size_t(gridDim.x) * blockDim.x;
    for (; i < n; i += stride) {
        atomicAdd(&shared_hist[input[i]], 1u);
    }
    __syncthreads();

    for (unsigned int b = threadIdx.x; b < 256;
         b += blockDim.x) {
        unsigned int count = shared_hist[b];
        if (count != 0u) {
            atomicAdd(&hist[b], count);
        }
    }
}
```

清零阶段按块内线程数跨步分配桶。任一桶 b 都可唯一写成 $b = qT + t$，其中 $0 \le t < T$，因此恰由线程 t 清零。若每块只有 128 个线程，线程 t 依次清零桶 t 和 $t+128$；若每块有 512 个线程，只有前 256 个线程需要写零。这个循环支持任意合法的一维线程块大小。

第一处 `__syncthreads()` 等待块内线程完成全部清零，并使随后阶段能看到这些写入。没有这处同步，快线程可能已经向某个桶加一，负责该桶初始化的慢线程随后再写入零，从而覆盖已有贡献。每个桶只由一个线程清零，也仍然需要阶段间同步，因为清零者与后续更新者可能不同。

局部统计阶段使用共享内存原子加。不同线程块的副本已经分开，因此块 0 对自己桶 7 的更新不会与块 1 的局部桶 7 争用同一地址；但是，同一块内多个线程仍可能同时更新本块的桶 7。共享内存的可见性和地址独立性并不替代块内冲突更新所需的原子性。

第二处 `__syncthreads()` 保证本块所有局部更新完成，随后才读取桶值并提交公共计数。若提前合并，一个线程读到的可能只是局部统计的中间值，后续增量就不会被提交到全局。该屏障位于整个输入循环之后，循环次数不同的线程最终都会到达同一个阶段边界。

13.4.3 合并阶段的并发与同步边界

第二次同步后，块内不再修改局部直方图。各桶由一个线程读取并负责提交，因此读取局部计数不需要原子操作。不同块却可能同时向同一个全局桶提交结果，所以全局合并仍使用 `atomicAdd`。把局部原子加换成普通加法会在块内丢失更新，把全局原子加换成普通加法则会在块之间丢失更新。

零计数没有贡献，代码通过 `count != 0u` 跳过这类全局更新。提交之后不再复用局部数组，也没有下一阶段需要块内共同读取提交结果，因此无需在核函数末尾再加一次块屏障。不同线程块可以各自完成、各自合并；所有块的更新都结束时，公共直方图才是完整结果。

对这一核函数，不能因为某个线程的初始输入索引超过 $N-1$ 就在开头提前返回。即使它没有输入需要统计，也可能承担某些桶的清零或归并工作，并且需要参与块内的阶段同步。正确写法是让边界条件只控制输入循环的工作量，全部线程继续经过两处屏障。

主机端仍需要在核函数开始前清零全局输出。代码13.3的清零和完成处理原则可以直接沿用，只需把启动目标改为 `histogram_private`。不能由某个普通线程块在同一个核函数开始处负责清零公共数组、其他块立即累加，因为普通的块内屏障无法保证其他块等到全局清零完成。

算法的正确性由三部分组成：网格跨步划分使每个输入只处理一次；两次屏障和共享原子加使各块准确得到式(13.4)中的局部计数；全局原子加将全部局部计数完整累加。初始化、阶段边界和原子更新分别保证不同环节。

13.4.4 原子操作数量与资源开销

令 U_r 表示线程块 r 的非零桶数。直接算法为每个输入执行一次全局原子加，因此全局原子更新总数为 N。私有化算法则执行 N 次共享内存原子加，并在合并阶段执行

$$A_{\text{global}} = \sum_{r=0}^{G-1} U_r \le \min\{N, GB\} \tag{13.5}$$

次全局原子加。每块最多有 B 个非零桶，所以有上界 GB；每个非零桶至少需要一个输入贡献，而不同块的输入互不重叠，所以同时有上界 N。代码若不跳过零桶，合并阶段的全局原子加次数就固定为 GB。

当各块几乎覆盖全部桶时，可以取 $U_r \approx B$，从而全局原子操作次数由 N 降到约 GB。令每块平均处理的输入数为 $n_{\text{block}} = N/G$，则次数缩减倍数约为 $N/(GB) = n_{\text{block}}/B$。每块处理的数据远多于桶数时，可以把很多局部增量聚合成少量公共更新。

表 13.5 基本算法与块级私有化的工作量比较

操作或资源	基本全局算法	共享内存私有化
输入元素读取	N	N
共享内存原子加	0	N
全局原子加	N	$\sum_r U_r \le \min\{N, GB\}$
私有桶清零	0	GB
每块屏障次数	0	2
每块共享计数器容量	0	B 个计数器

若各块只处理很少的输入，局部非零桶数可能接近局部输入数，此时 $\sum_r U_r$ 接近 N，而清零、共享原子加和屏障已经引入额外开销。若输入集中于一个桶，只要每个块都有有效输入，就有 $U_r = 1$，全局原子更新数量从 N 降为 G；同时各块仍各自承担局部同桶更新的串行链。

表13.5统计的是操作数量，不能把不同位置的原子加按相同单价换算成运行时间。是否获得加速还取决于共享内存服务能力、L2 中公共计数器的处理能力、输入分布和设备并行度。次数分析提供的是优化原因与规模关系。

使用 32 位无符号计数器时，每个局部直方图占用 $4B$ 字节；$B = 256$ 时为 1024 字节，即 1 KiB。若某流式多处理器（streaming multiprocessor，SM）可供驻留线程块使用的共享内存为 S_{SM}，仅考虑这一资源时，驻留块数至多为 $\lfloor S_{\mathrm{SM}}/(4B) \rfloor$。实际还受每块和每个 SM 的线程数、寄存器及块数限制，因此局部数组增大可能降低驻留并行度。

13.4.5 桶数超过共享内存容量时

共享内存私有化要求局部数组能容纳在设备允许的每块共享内存中，并保留足够的并行执行资源。桶数很大时，可以只为一部分桶建立局部数组，按桶区间分批统计。每一批都扫描输入，仅对当前桶区间内的元素更新。这样减少了单批的局部容量，但会重复读取输入；若共有 K 批且每批都完整扫描，输入读取量变为 KN。

另一个选择是把各块的局部直方图放在全局内存，使用不同的行保存不同块的结果。这样仍把原来所有块对同一地址的竞争分散到不同地址，但局部更新不再享受共享内存的低延迟。第一阶段结束后，第二个核函数按桶求各行之和；若每个最终桶由唯一线程或唯一协作组负责输出，最终写入可以不使用全局原子加。其代价是约 GB 个额外计数器、局部结果的全局读写和第二次核函数启动。

这两种方法分别用更多输入扫描或更多全局存储换取较小的共享内存需求。选择时应结合桶数、输入长度和冲突程度；它们保持的数学关系仍然是式(13.4)。

13.5 线程粗化与性能权衡

13.5.1 增加每个线程处理的输入数

线程粗化（thread coarsening）让一个线程承担多个原先可以分配给不同线程的输入元素。在已经使用网格跨步循环的直方图中，可以保持每块线程数 T 不变，减小网格块数 G，让每个线程自然执行更多轮循环。每个块仍只建立一份私有直方图，结束时也只合并这一份结果。

设希望每个线程大致处理 c 个元素，一种网格选择为

$$G_c = \left\lceil \frac{N}{cT} \right\rceil, \qquad P_c = G_c T. \tag{13.6}$$

该公式用于选择工作划分；末尾不足一轮由边界条件处理。只要 $N > 0$，线程 g 继续按 $i = g + kP_c$ 遍历即可。每一轮中相邻线程仍读取相邻地址，因此这种粗化保留了输入访问的合并模式。

在固定输入长度下，增加 c 使私有直方图数量由约 N/T 减为约 $N/(cT)$。清零量和不跳过零桶时的归并量从约 NB/T 降为 $NB/(cT)$。若跳过零桶，确切归并量仍是 $\sum_r U_r$，但它的 $G_c B$ 上界随块数减少。

这种收益来自实际减少线程块数量。仅把循环展开、让代码一次写出多次更新，而启动的块数保持不变，并不会减少局部副本数量及每块合并开销。另一方面，循环变长也不会减少总输入读取和总共享原子加：它们仍各为 N 次。

13.5.2 粗化的数量例子与并行度代价

设 $N = 2^{20} = 1\,048\,576$ 个输入，$B = 256$ 个桶，每块 $T = 256$ 个线程。表13.6比较 $c = 1$ 与 $c = 8$ 的网格设置。两种设置都沿用代码13.4，没有改变桶数或局部数组类型。

表 13.6 固定输入规模下减少块数的效果

数量	$c = 1$	$c = 8$
线程块数 G_c	4096	512
每线程处理的输入数	1	8
平均每块输入数	256	2048
私有桶清零次数 $G_c B$	1 048 576	131 072
全局合并次数的 $G_c B$ 上界	1 048 576	131 072
共享内存原子加次数	1 048 576	1 048 576
每块共享直方图容量	1 KiB	1 KiB

清零工作量和全局合并上界下降为原来的八分之一，每个块能用同一份共享数组聚合更多贡献。对于全部输入集中于一个桶的极端情况，实际全局归并次数分别为 4096 和 512。对于分散输入，实际次数由各块观察到的非零桶数决定，不能把表中的上界直接当作测得次数。

增加粗化程度也会让单个块持续更久。如果所有输入集中于一个桶，每块内部对该桶的串行链随局部输入数增加；若整个网格只剩很少几个块，一些 SM 可能没有足够工作，设备无法利用全部并行资源。因而不能仅凭全局归并次数最少，就断定最快的设置应当只使用一个线程块。

通过延长同一个跨步循环实现粗化时，不必然需要更多寄存器，因为循环中可以反复使用同一组索引和临时变量。若具体实现同时保留多组输入、中间结果或展开后的独立状态，才可能增加寄存器压力。粗化的资源代价需要对应到实际代码，而它减少私有副本数量的机制由网格规模直接决定。

13.5.3 线程级局部聚合

粗化使一个线程依次处理多个输入，也提供了在线程内部合并更新的机会。若为每个线程保存完整直方图，需要每线程 B 个计数器；桶数较大时，寄存器或局部存储开销可能过高。一个更小的状态只保存最近访问的桶编号和该桶连续出现的次数。线程再次读到同一桶时只增加私有计数，读到不同桶时才把上一段计数原子加入块内直方图。

设线程 g 实际读取的桶序列为 $b_{g,0}, \dots, b_{g,m_g-1}$。将其中连续相同的桶划为一段，段数为

$$R_g = \begin{cases} 0, & m_g = 0, \\ 1 + \displaystyle\sum_{k=1}^{m_g-1} \mathbf{1}\{b_{g,k} \neq b_{g,k-1}\}, & m_g > 0. \end{cases} \tag{13.7}$$

每一段只提交一次，因此共享内存原子加的次数由 N 变为 $\sum_g R_g \le N$。每个桶的贡献总量不变，块内直方图及其非零桶数也不变，所以最后的全局合并次数仍为 $\sum_r U_r$。

以下伪代码替换共享直方图核函数中的输入统计循环；前面的清零与第一处屏障，以及后面的第二处屏障与全局合并，均保持不变。`valid` 表示是否存在尚未提交的一段，不能用某个合法桶编号表示空状态。

代码 13.5 线程内合并连续同桶更新的伪代码

```
valid = false
count = 0
for each valid input index assigned to this thread:
    bin = input[index]
    if valid and bin == last_bin:
        count = count + 1
    else:
        if valid:
            atomicAdd(shared_hist[last_bin], count)
        last_bin = bin
        count = 1
        valid = true
if valid:
    atomicAdd(shared_hist[last_bin], count)
// All threads then reach the block barrier.
```

线程读到的序列若为 $(7,7,2,2,2,7)$，就依次提交 $(7,2)$、$(2,3)$ 和 $(7,1)$ 三个“桶、增量”对，将六次共享原子加减少为三次。最后一对在循环结束后提交；省略这一步会丢失末段。空线程不提交任何计数，但仍参加块内屏障；只处理一个元素的线程则通过末段提交完成唯一一次更新。私有段计数和共享计数器同样必须能表示实际累计数量。

这里的连续性取决于线程的访问顺序。网格跨步循环使同一线程依次读取 $g, g+P, g+2P, \ldots$，原数组中相邻像素的相似性不保证这些位置也相似。例如，输入 $(0,0,1,1,0,0,1,1)$ 由 $P=2$ 个线程跨步读取时，每个线程都看到 $(0,1,0,1)$，八次原子更新一次也不能省略；若改成 $(0,1,0,1,0,1,0,1)$，两个线程分别看到四个 0 和四个 1，只需两次提交。输入的桶频数在这两个例子中相同，聚合收益却不同。

当线程所见序列包含较长的同桶段时，少量私有状态能够明显减少热点桶上的共享原子操作。若相邻访问很少命中同桶，R_g 接近 m_g，原子操作数量几乎不变，而比较、分支与状态维护仍有成本。因此，线程级聚合应结合实际访问顺序和输入分布评估，不能仅凭直方图中存在高频桶就断定会加速。

13.5.4 访存、分支和复用的整体判断

输入数组通常只被读取一次，每个元素只用于决定自己的桶，因此算法没有需要专门缓存的跨线程输入复用。私有直方图则不同：同一个块内的许多输入可能反复更新同一组桶，输出状态具有复用。将这组状态放进共享内存，正是把访问频繁的数据保留在块内。

输入跨步循环的边界条件主要影响最后一轮不满的线程束，或者没有输入任务的线程。合并阶段的非零判断也可能使一部分线程提交更新、另一部分跳过；这种分支换来了对零贡献原子操作的省略。相比之下，很多线程同时命中同一桶造成的是原子地址竞争：即使所有线程执行完全相同的指令，也可能因为目标地址相同而排队。

表 13.7 私有化直方图的主要性能因素

观察对象	形成的开销	对应调整
同轮输入地址	地址分散时需要更多内存事务	使相邻线程读取相邻输入
公共桶竞争	多块贡献排成同地址更新序列	先在块内聚合，再提交局部计数
块内热点桶	共享内存同址原子更新仍排队	结合局部输入量与数据分布评估
副本数量	每块都需清零和逐桶合并	适度增加每块工作量、减少块数
驻留与并行度	资源需求大或网格过小会限制并行	同时考虑桶容量、线程数和块数

直接全局原子、共享内存私有化和线程粗化分别改变不同环节。基本算法提供简单的正确实现；私有化改变竞争发生的位置及公共更新次数；粗化进一步摊薄每份私有副本的固定开销。比较这些方案时，应固定输入长度、桶映射和数据分布，再结合操作数量与实际运行时间判断收益。

13.6 例题与完整推导

13.6.1 从输入划分到最终直方图

设输入长度为 $N = 16$，数据依次为

$$x = (0, 2, 1, 2,\ 3, 2, 0, 1,\ 2, 3, 3, 1,\ 0, 2, 3, 2).$$

使用两个线程块，每块四个线程，即 $G = 2$、$T = 4$、$P = 8$，运行代码13.4。桶数仍为 256，但只有桶 0 到 3 可能非零。要求写出各线程负责的输入、每块的局部结果、全局结果，并计算原子操作次数。

线程块 r 中线程 t 的全局编号为 $g = 4r + t$，处理索引 g 与 $g + 8$。表13.8逐一列出两个轮次，其中 $i \mapsto x_i$ 表示“输入索引映射到该位置的值”。

表 13.8 两块、每块四线程的完整输入分配

块编号 r	块内线程 t	第一轮 $i \mapsto x_i$	第二轮 $i \mapsto x_i$
0	0	$0 \mapsto 0$	$8 \mapsto 2$
0	1	$1 \mapsto 2$	$9 \mapsto 3$
0	2	$2 \mapsto 1$	$10 \mapsto 3$
0	3	$3 \mapsto 2$	$11 \mapsto 1$
1	0	$4 \mapsto 3$	$12 \mapsto 0$
1	1	$5 \mapsto 2$	$13 \mapsto 2$
1	2	$6 \mapsto 0$	$14 \mapsto 3$
1	3	$7 \mapsto 1$	$15 \mapsto 2$

块 0 处理的八个值为 $0, 2, 1, 2, 2, 3, 3, 1$，所以桶 0 到 3 的局部计数为 $H^{(0)} = (1, 2, 3, 2)$。块 1 处理的值为 $3, 2, 0, 1, 0, 2, 3, 2$，局部计数为 $H^{(1)} = (2, 1, 3, 2)$。每个局部向量的元素之和都是 8，符合各块分配的输入数。两块分别完成第二次同步后，逐桶合并得到

$$H = H^{(0)} + H^{(1)} = (1, 2, 3, 2) + (2, 1, 3, 2) = (3, 3, 6, 4). \tag{13.8}$$

这里为了简洁只列出前四个桶，其余 252 个桶为零。总和 $3 + 3 + 6 + 4 = 16$，且逐桶结果与直接计数相同。

直接算法需要 16 次全局原子加。私有化算法需要 16 次共享内存原子加，每块有 4 个非零桶，所以全局合并只做 $4 + 4 = 8$ 次原子加。由于代码仍为每块清零 256 个桶，局部清零总数却是 $2 \times 256 = 512$ 次；这个小例子说明，全局原子次数减少并不足以保证整体加速，私有

化还需要足够工作量来摊薄固定开销。

如果输入只保留前 14 个元素，索引 14 和 15 不再处理。块 1 的线程 2、3 在第二轮没有有效输入，但仍参与局部初始化和两次同步。最终前四桶为 $(3,3,5,3)$，总计数为 14。这一边界变化只减少两次输入贡献，不改变线程块各阶段的参与规则。

13.6.2 由浮点吞吐率反推 L2 原子操作比例

设一次原子操作在 L2 完成时耗时 5 ns，在 DRAM 完成时耗时 120 ns；一次浮点运算耗时 1 ns。每个线程执行 5 次原子操作和 100 次浮点运算，因此每次原子操作对应 20 次浮点运算。给定有效浮点吞吐率为 0.2424 GFLOPS，其中 1 GFLOPS 表示每秒 10^9 次浮点运算。求在 L2 完成的原子操作所占比例。

采用等效单条串行执行流的简化计时模型：一个线程所代表的工作量中，L2 原子操作、DRAM 原子操作和浮点运算的用时直接相加；给定吞吐率按这一等效工作流换算。令 $p \in [0,1]$ 表示原子操作在 L2 完成的平均比例，则一次原子操作的平均耗时为 $5p+120(1-p)$ 纳秒。一份线程工作量的平均耗时为

$$t(p) = 5\big[5p + 120(1-p)\big] + 100 = 700 - 575p \quad \text{ns}. \tag{13.9}$$

式中最外侧的 5 是每线程原子操作次数，括号里的 5 是一次 L2 原子操作的纳秒耗时；最后的 100 是全部浮点运算的纳秒耗时。

将给定吞吐率换算为每秒完成的线程工作量：每份工作量含 100 次浮点运算，所以速率为 $(0.2424 \times 10^9)/100 = 2.424 \times 10^6$ 份工作量每秒。取倒数得到平均用时

$$t_{\text{obs}} = \frac{100}{0.2424 \times 10^9}\ \text{s} \approx 412.5413\ \text{ns}. \tag{13.10}$$

在这里，一次浮点运算每纳秒等于 1 GFLOPS，也可以直接计算 100/0.2424，并将所得时间单位写为纳秒。

令式(13.9)等于式(13.10)，可得

$$p = \frac{700 - 412.5413}{575} \approx 0.49993 \approx 50\%. \tag{13.11}$$

给定吞吐率只有四位小数，因此答案写作约 50%。若取 $p = 1/2$ 反向核验，则平均原子操作耗时为 $(5+120)/2 = 62.5$ ns；每份工作量耗时 $5 \times 62.5 + 100 = 412.5$ ns，对应 $100/412.5 = 0.242424\ldots$ GFLOPS，四舍五入即为 0.2424。

还可以检查两个端点：若全部原子操作在 L2 完成，平均耗时为 125 ns，吞吐率为 0.8 GFLOPS；若全部在 DRAM 完成，平均耗时为 700 ns，吞吐率约为 0.142857 GFLOPS。给定值位于二者之间，与求得的 $p \in [0,1]$ 一致。50% 是许多线程或多次执行中原子操作的平均比例，不要求每个含五次原子操作的线程恰好有同样的 L2 次数。

这个反推依赖已明确的串行等效计时口径。只有“单线程内部三类操作不重叠”，还不足以把真实多线程设备的总吞吐直接换算成单线程延迟；线程之间可能重叠执行，也可能共享原子服务资源并排队。若用于真实内核测量，需要额外确定并行执行与吞吐统计的关系，才能由总吞吐唯一反推存储层级比例。

13.6.3 实现检查与关键关系

对直方图实现，可以围绕完整的贡献路径检查正确性：每个输入是否恰好被处理一次，桶下标是否有效，每次共享更新是否被保留，以及结果读取是否发生在相关更新完成之后。表13.9列出四种具体错误及其后果。

表 13.9 典型实现错误及其修正依据

错误写法或条件	产生的问题	修正依据
每块 128 线程，仅用小于 256 的线程编号清零同编号桶	只有桶 0 到 127 被清零，后半数组初值未知	按块内线程数跨步遍历全部桶
共享数组上的累加改成普通加一	同块多个线程命中同一桶时丢失更新	局部冲突仍需共享原子加
第二次同步前读取局部计数并合并	局部统计尚未结束，后续贡献可能遗漏	所有局部更新结束后再提交结果
多个块用普通加法合并到同一公共桶	不同块的局部总数可能互相覆盖	使用全局原子加，或另设独占输出的归并阶段

数学结果始终由 $H_j = \sum_i \mathbf{1}\{b(x_i) = j\}$ 定义。网格跨步划分通过 $i = g + kGT$ 保证输入覆盖；块级私有化通过 $H_j = \sum_r H_j^{(r)}$ 保持结果；跳过零桶时，全局更新次数为 $\sum_r U_r \le \min\{N, GB\}$。这些关系分别连接输入分配、统计正确性和性能开销。

合并访存改善同一轮输入读取，原子操作保护冲突的读—改—写，私有化把许多公共更新聚合为局部总数，线程粗化则用更长的局部工作摊薄初始化与合并成本。将这几个机制放回各自负责的环节，才能解释为什么一个实现既正确、又在特定数据规模和分布下更高效。

第 14 讲

Profiling on Nvidia GPU

14.1 性能分析与 GPU 硬件

14.1.1 性能数据所回答的问题

性能分析(*profiling*)是在程序实际执行时，收集运行时间、事件时间线和硬件计数器，再把这些观察与源代码联系起来的过程。**性能分析器**(*profiler*)可以观察处理器何时工作、数据经过哪些存储层级、指令为何等待，以及哪些代码占用了主要执行时间。程序的数学表达式给出了要完成的运算；执行性能还取决于线程组织、访存布局、资源分配和具体硬件。

优化需要先建立正确的程序，再测量其开销，并针对主要限制修改实现。每次修改之后都要重新检查结果和运行时间，因为减少一种开销可能增加另一种开销。例如，共享内存分块能减少重复的全局读取，同时也会引入共享读取、块内同步和片上资源占用；这些成本共同决定最终收益。

分析尺度可以分为应用和内核两层。应用层关心主机准备数据、设备传输、内核计算与同步之间的时间关系；内核层关心某次内核执行期间的指令发射、计算流水线、缓存及存储系统。系统级分析工具（Nsight Systems）适合先定位应用中的主要耗时区间，内核级分析工具（Nsight Compute）再用于解释目标内核内部的硬件行为。一次内核执行变快以后，还应回到应用层确认总时间是否下降。

设目标内核原先占应用总时间的比例为 f，对它的加速比为 s，其余部分的时间保持不变。将原总时间归一化为 1，优化后的时间为 $(1-f)+f/s$，所以应用加速比为

$$S_{\text{app}} = \frac{1}{(1-f)+f/s}. \tag{14.1}$$

例如，某内核只占总时间的 20%，即使它达到 4 倍加速，应用也只有 $1/(0.8+0.2/4)\approx 1.18$ 倍加速。这个关系说明了为什么应先确定耗时位置，再选择值得深入分析的内核。

14.1.2 线程组织与硬件执行层级

在统一计算设备架构（CUDA）的执行模型中，一次内核启动产生一个**网格**(*grid*)，网格由**线程块**(*thread block*)组成，线程块内包含线程。GPU 的主要通用执行单元是**流式多处理器**(*streaming multiprocessor*，*SM*)。一个普通线程块的线程在同一个 SM 上执行，因此能够通过该 SM 的共享内存交换数据并进行块内同步；一个 SM 在资源允许时可以同时驻留多个线程块。

线程块内部以 32 个线程组成一个**线程束**(*warp*)。线程束中的线程共享指令发射过程，各线程仍保留自己的寄存器状态和数据索引。调度器从驻留线程束中选择已经具备执行条件的线程束，并向相应流水线发射指令。于是，线程束等待某次访存时，调度器可以发射其他就绪线程束的指令，这构成了**延迟隐藏**(*latency hiding*)的基础。

一个 SM 通常进一步划分为若干子分区（SM subpartition，SMSP），其中包含线程束调度器、寄存器资源和执行流水线。芯片级还可能采用图形处理簇（graphics processing cluster，GPC）、纹理处理簇（texture processing cluster，TPC）等组织层级。这些结构帮助解释硬件图，但程序中的线程块与这些簇没有固定的一一对应关系。线程块到 SM 的分配、每个 SM 可驻留的线程束数，以及每周期能够发射的指令，都要结合所分析的 GPU 判断。

不同指令会使用不同执行资源：普通浮点乘加使用浮点流水线，地址计算通常需要整数指令，读取与写入使用加载 / 存储流水线，某些数学函数使用特殊函数单元（special function unit，SFU）。张量核心（Tensor Core）承担特定矩阵运算。一个内核是否有效利用硬件，必须对应到它实际执行的指令类别；某种流水线很忙，无法直接推出全部计算资源都达到了峰值。

并行数量与执行能力

驻留线程束提供可供调度的工作，就绪线程束提供当前可发射的工作，执行流水线决定当前能接收什么指令。这三个层次共同限制吞吐率。增加线程数只有在改善其中的实际限制时才会带来收益。

14.1.3 存储空间、物理位置与缓存路径

CUDA 的存储空间规定数据的可见范围和访问方式；物理存储层级决定数据放在哪里、如何被缓存，以及访问经过哪些硬件。理解性能指标时，需要同时掌握这两种含义。表14.1给出与本讲相关的对应关系。

表 14.1 存储空间与典型硬件实现

存储空间	典型物理位置或访问路径	分析时的关注点
全局内存	设备内存，通常经过 L2 与 L1 / 纹理缓存路径	访问合并、缓存复用和各层实际流量
共享内存	SM 上显式管理的片上存储	线程块内复用、存储体冲突和读取指令压力
常量内存	设备内存及相应常量缓存路径	同一线程束访问相同地址时的广播
纹理 / 表面内存	设备内存及纹理相关缓存路径	专用访问方式及局部性
局部内存	线程私有地址空间，数据可能位于设备内存并经过缓存	大型私有数组和寄存器溢出带来的访存
寄存器	SM 的寄存器文件；按线程保存状态，并受分配粒度约束	寄存器用量、依赖链和可驻留线程束数

一级缓存（L1 cache）靠近 SM，二级缓存（L2 cache）位于更外层并由多个 SM 共享。许多架构让 L1 与共享内存共用一部分物理片上容量，但两者的管理方式不同：共享内存由程序显式写入、读取和同步，L1 则依照缓存机制自动保存访问过的数据。共享内存容量的配置可能影响可用缓存容量，也可能限制同时驻留的线程块数。

局部内存（*local memory*）中的“局部”表示线程私有的可见性。它不能作为低延迟片上存储的保证。当编译器无法把某些线程私有变量保留在寄存器中时，这些变量可能被放到局部内存，从而增加实际加载和存储。寄存器数量因此存在两方面作用：更多寄存器可以保存更多可复用数据和独立累加器，但每线程用量过高又可能压低占用率，甚至导致**寄存器溢出**（*register spilling*）。

缓存命中（*cache hit*）表示所需数据已经位于当前检查的缓存层；**缓存未命中**（*cache miss*）则需要继续向下一级请求数据。**时间局部性**（*temporal locality*）来自一段时间内反复访问同一数据，**空间局部性**（*spatial locality*）来自访问相邻地址。缓存容量有限，旧数据会因替换而被移出，因此同一个全局地址的多次访问不一定每次都需要访问设备内存，也不一定一直能够命中同一层缓存。

把一次加载简单理解为“L1 未命中后读取 L2，L2 未命中后读取设备内存”，能够解释数据来源；要进一步理解吞吐率，还必须考虑请求队列、地址标签检查、数据返回和执行依赖。图14.1展示了这种更具体的路径。共享请求与全局请求可以经过相关的入口和数据通路，因此即使设备内存流量不大，片上请求处理也可能形成限制。

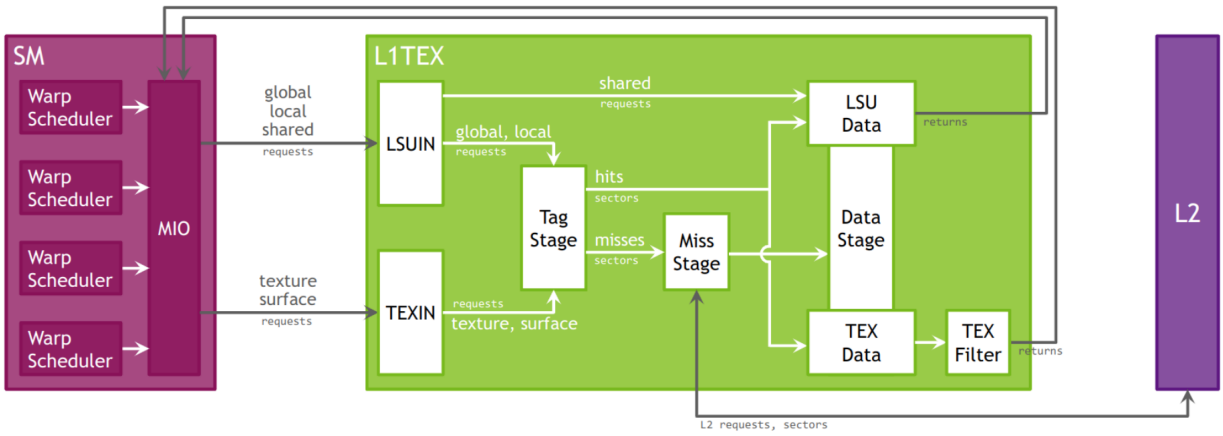


图 14.1 线程束调度器到 L1 / 纹理单元及 L2 的请求与返回路径。共享访问需要经过片上的请求处理和数据通路；全局访问还涉及缓存标签检查与未命中处理。

这些层级还解释了为何同一份 CUDA 代码在不同 GPU 上呈现不同的瓶颈。缓存容量、每个 SM 的执行资源、可驻留线程束数和设备内存带宽都可能改变。比较硬件时，应保留问题规模、数据类型、编译选项和计时范围，再依据对应设备的计数器解释差异。

14.2 矩阵乘法基准与可信测量

14.2.1 统一问题规模与运算量口径

通用矩阵乘法（general matrix multiplication，GEMM）提供了一个便于比较不同数据复用方式的基准。设 $A \in \mathbb{R}^{M\times K}$、$B \in \mathbb{R}^{K\times N}$，计算 $C = AB \in \mathbb{R}^{M\times N}$。采用从 0 开始的索引，则 $C_{ij} = \sum_{k=0}^{K-1} A_{ik}B_{kj}$。以下实现均使用**行主序**（*row-major order*）：A_{ik} 的元素偏移为 $iK + k$，B_{kj} 为 $kN + j$，C_{ij} 为 $iN + j$。代码假定 M, N, K 为正整数，设备数组容量与矩阵尺寸匹配。

每个输出需要 K 次乘法和累加。性能统计通常把一次**融合乘加**（*fused multiply-add*，*FMA*）计为两次浮点运算，即使它在机器层只对应一条指令。因此，统一采用总运算量 $F = 2MNK$。这里的浮点运算量（FLOP）与每秒浮点运算数（FLOP/s）含义不同：前者是工作量，后者是吞吐率。若耗时为 t_{ms} 毫秒，则

$$P_{\text{G}} = \frac{2MNK}{10^{6} t_{\text{ms}}} \ \text{GFLOP/s}, \qquad S = \frac{t_{\text{base}}}{t}, \qquad \eta = \frac{P}{P_{\text{peak}}}. \tag{14.2}$$

其中 S 是相对基准的加速比，η 是采用指定理论峰值计算的效率。峰值应与浮点精度、执行单元类型和时钟条件一致；用不同精度或张量核心的峰值评价普通单精度内核，会改变效率分母的含义。$2MNK$ 统计有效工作量；分块边界处执行的填零乘加也可能计入硬件计数，因

此报告按实际指令统计的吞吐率与这里的有效吞吐率可能略有差异。

14.2.2 简单内核与共享内存分块

简单内核让每个线程负责一个输出。为了把输出空间排成一维网格，可令线程索引 q 对应 $i = \lfloor q/N \rfloor$、$j = q \bmod N$。这使相邻线程优先处理同一行的相邻列。下面的内核用 `size_t` 计算线性地址，并对输出边界进行检查。

代码 14.1 每线程计算一个输出的简单矩阵乘法

```
__global__ void gemm_simple(const float* A, const float* B,
                            float* C, int M, int N, int K) {
  size_t q = size_t(blockIdx.x) * blockDim.x + threadIdx.x;
  if (q >= size_t(M) * N) return;
  int row = int(q / N);
  int col = int(q % N);
  float sum = 0.0f;
  for (int k = 0; k < K; ++k) {
    sum += A[size_t(row) * K + k] * B[size_t(k) * N + col];
  }
  C[q] = sum;
}
```

从源代码看，每个输出执行 $2K$ 次全局加载。同一行的多个输出都要使用同一组 A 元素，同一列的多个输出也会重复使用 B 的元素。这些重复访问可能命中缓存，因此源代码加载次数并不直接等于设备内存事务次数。不过，程序仍然需要发射相关加载指令并处理地址与依赖，这些开销即使在缓存命中时也存在。

共享内存分块把 $T \times T$ 个输出交给一个包含 $T \times T$ 个线程的线程块。每次沿 K 方向处理长度为 T 的区间时，线程块先协作读入 A 的一个 $T \times T$ 子块和 B 的一个 $T \times T$ 子块，再让各线程从共享内存取数并更新自己的累加器。一个加载进来的 A 元素供同一输出行中的 T 个线程复用，一个 B 元素则供同一输出列中的 T 个线程复用。

对一个不含边界的完整分块阶段，简单实现需要约 $2T^3$ 次全局标量加载，共享分块只需 $2T^2$ 次；两者都完成 $2T^3$ 次浮点运算。因此，全局加载的逻辑需求约缩小到原来的 $1/T$。实现还需要共享内存加载和同步，其实际加速比应通过测量确定。

代码 14.2 带边界处理的共享内存分块内核

```
template<int T>
__global__ void gemm_shared(const float* A, const float* B,
                            float* C, int M, int N, int K) {
  __shared__ float As[T][T];
  __shared__ float Bs[T][T];
  int tx = threadIdx.x, ty = threadIdx.y;
  int row = blockIdx.y * T + ty;
  int col = blockIdx.x * T + tx;
  float sum = 0.0f;
  for (int k0 = 0; k0 < K; k0 += T) {
    As[ty][tx] = (row < M && k0 + tx < K)
        ? A[size_t(row) * K + k0 + tx] : 0.0f;
    Bs[ty][tx] = (k0 + ty < K && col < N)
        ? B[size_t(k0 + ty) * N + col] : 0.0f;
    __syncthreads();
    for (int k = 0; k < T; ++k) {
      sum += As[ty][k] * Bs[k][tx];
    }
    __syncthreads();
  }
  if (row < M && col < N) C[size_t(row) * N + col] = sum;
}
```

代码14.2要求以 `dim3(T,T)` 作为线程块维度，以 `dim3((N+T-1)/T,(M+T-1)/T)` 作为网格维度。合法的 T 还须满足设备对线程块线程数和共享内存的限制。第一个同步保证整个子块已经加载完成；第二个同步保证所有线程读取完当前子块后，才会有人覆盖下一阶段的数据。边界线程仍参与加载、补零与两个同步，不能在进入循环之前因输出越界而提前返回。

这一实现也给出了两种不同的连续性。全局加载时，相邻 `threadIdx.x` 访问 A 或 B 的相邻列；累加时，固定线程沿 k 依次读取 `As[ty][k]`，这些地址连续，而 `Bs[k][tx]` 的相邻读取之间相隔一整行。前一种连续性关系到线程束的全局访存合并，后一种连续性决定单个线程是否方便把多次共享读取合成宽指令。

14.2.3 寄存器与共享内存联合分块

线程粗化(*thread coarsening*)让一个线程负责多个输出。设一个线程计算某行上的 r 个相邻输出，则它需要 r 个累加器 $c_0, \dots, c_{r-1}$。对某个归约索引 k，只需把 $a = A_{ik}$ 加载到一个寄存器中，就能用它更新 r 个累加器：$c_j \leftarrow c_j + aB_{k,j_0+j}$。$A$ 的这次加载于是在线程内部得到复用。

联合分块可以让一个线程块的 b_m 个线程分别处理 b_m 行，每线程负责 b_n 列输出，形成 $b_m \times b_n$ 的输出子块。各线程使用的 A 行不同，适合把当前 A 数据保留在线程自己的寄存器中；所有线程会重复使用同一组 B 数据，适合将 B 子块放入共享内存。这将复用分别安排在线程内部和线程块内部。

若每个归约阶段的长度为 b_k，理想情况下需要加载 $b_m b_k$ 个 A 元素和 $b_k b_n$ 个 B 元素，完成 $2b_m b_n b_k$ 次浮点运算。线程同时持有多个输出累加器，还能提供多条相对独立的乘加依赖链。代价是寄存器需求增大、线程块配置改变，边界处理和数据加载的分工也更复杂。合理的分块形状需要兼顾复用、访存布局与资源限制。

高性能线性代数库（cuBLAS）提供 GEMM 的优化实现，可以作为参考结果和性能对照。比较时应统一矩阵尺寸、行列主序的解释、输入与累加精度，以及是否允许采用张量核心。以下实验表采用普通单精度计算口径，不将张量核心峰值混入效率分母。库函数的函数名本身不足以确定全部执行路径，数学模式和实际指令也属于实验配置。

14.2.4 正确性检查与性能构建

性能比较成立的前提是各实现完成同一个计算。对矩阵乘法，可以先使用可手工核算的小矩阵，再使用中央处理器（CPU）参考实现或可信的 GPU 实现比较全部输出。全 1 输入能快速检查数量级，因为每个输出都应等于 K；非方阵、非均匀数据和不能被分块宽度整除的尺寸则能检查索引与边界。仅检查少数输出可能漏掉线程映射错误。

浮点加法不满足严格结合律，不同分块或乘加融合方式会改变舍入路径。设参考结果为 C_{ij}^{ref}，一种逐元素判据是

$$|C_{ij} - C_{ij}^{\mathrm{ref}}| \le \varepsilon_{\mathrm{abs}} + \varepsilon_{\mathrm{rel}} |C_{ij}^{\mathrm{ref}}|. \tag{14.3}$$

绝对容差照顾接近零的结果，相对容差照顾数值尺度较大的结果。容差应依据数据类型、输入尺度和累加长度选定，同时单独检查非有限值；不能因为更宽松的容差能通过，就认定一个越界或竞争问题已经解决。

计算检查器（Compute Sanitizer）提供几种互补检查。`memcheck` 检查非法或未对齐的内存访问，并可检查泄漏；`racecheck` 检查共享内存访问竞争；`initcheck` 检查未初始化

的全局内存读取；synccheck 检查同步原语的使用错误。对代码14.2，缺少前一个同步可能让线程读取尚未写入的共享元素，缺少后一个同步则可能使下一阶段覆盖仍在被读取的数据。

代码 14.3 构建待测程序并分别运行正确性工具

```
nvcc -O3 -lineinfo gemm_demo.cu -o gemm_demo
compute-sanitizer --tool memcheck ./gemm_demo
compute-sanitizer --tool racecheck ./gemm_demo
compute-sanitizer --tool initcheck ./gemm_demo
compute-sanitizer --tool synccheck ./gemm_demo
```

这里的 gemm_demo.cu 和 gemm_demo 分别表示待测程序的源文件与可执行文件；构建时还应选择与目标 GPU 匹配的架构。检查工具会改变运行开销，它们的执行时间不用于比较正常性能。不同工具通过以后，仍要确认数值输出，因为内存访问合法并不能保证公式或索引正确。

性能构建应保留正常优化，并加入 -lineinfo，以便把设备指令映射回源代码行。表14.2区分容易混用的选项。

表 14.2 与性能分析相关的编译选项

选项	作用	对性能分析的意义
-O3	启用主机编译的优化级别	与正常发布构建保持一致；设备调试选项需另行检查
-lineinfo	生成设备代码行号信息	便于把热点定位到源代码，并保留性能优化
-g	生成主机调试信息	本身不等于关闭设备代码优化；不是设备行号信息的替代品
-G	生成设备调试信息	通常会关闭设备优化，适合调试；一般性能测量避免使用
-pg	为传统主机分析器生成插桩	不属于此处的 GPU 性能分析构建选项

14.2.5 流、事件与异步执行的计时范围

CUDA **流**（*stream*）是按顺序执行操作的队列，类型为 cudaStream_t。同一流中的内核、传输和事件遵循该流的顺序；不同流之间可以在资源和依赖允许时重叠。未显式给出流时使用默认流，其跨流同步语义还受默认流模式影响。使用明确创建的流，能让计时范围及依赖更容易检查。

一次内核启动通常先将工作提交给 GPU，主机随后继续执行。因此，主机在内核启动语句前后立即读取时钟，可能主要测到提交开销。CUDA **事件**（*event*）是插入某个流中的标记，类型为 cudaEvent_t；事件时间戳在设备执行推进到该标记时记录。同一流中依次插入开

始事件、待测内核和结束事件，再等待结束事件完成，就能测量这一段设备时间。

以下函数接受一个 launch 可调用对象，它将待测内核启动到参数指定的同一条流中。头文件提供运行时类型和异常类型；check 把失败的 CUDA 调用转成异常，避免把未正常执行的内核计入有效结果。

代码 14.4 带预热与重复执行的 CUDA 事件计时

```
#include <cuda_runtime.h>
#include <stdexcept>

inline void check(cudaError_t status) {
  if (status != cudaSuccess)
    throw std::runtime_error(cudaGetErrorString(status));
}

template<class Launch>
float average_kernel_ms(Launch launch, cudaStream_t stream,
                        int warmup, int repeats) {
  if (warmup < 0 || repeats <= 0)
    throw std::invalid_argument("invalid iteration count");
  for (int i = 0; i < warmup; ++i) launch();
  check(cudaGetLastError());
  check(cudaStreamSynchronize(stream));
  cudaEvent_t start, stop;
  check(cudaEventCreate(&start));
  check(cudaEventCreate(&stop));
  check(cudaEventRecord(start, stream));
  for (int i = 0; i < repeats; ++i) launch();
  check(cudaGetLastError());
  check(cudaEventRecord(stop, stream));
  check(cudaEventSynchronize(stop));
  float elapsed_ms = 0.0f;
  check(cudaEventElapsedTime(&elapsed_ms, start, stop));
  check(cudaEventDestroy(start));
  check(cudaEventDestroy(stop));
  return elapsed_ms / repeats;
}
```

例如，若 A、B、C 已经指向设备数组，并已创建 stream，可使用以下调用。这里的 5 次预热和 20 次重复只是计时示例参数。

代码 14.5 将计时对象显式绑定到同一条流

```
dim3 block(16, 16);
dim3 grid((N + 15) / 16, (M + 15) / 16);
auto launch = [&]() {
  gemm_shared<16><<<grid, block, 0, stream>>>(
      A, B, C, M, N, K);
};
float ms = average_kernel_ms(launch, stream, 5, 20);
```

分配、数据上传和结果校验位于计时区间之外，就得到内核计算口径；若要比较应用端到端时间，则需要另行包含这些工作，并在主机计时结束前等待对应的设备工作完成。

预热可以排除首次上下文初始化、延迟加载及起始时钟状态的一部分影响，但预热也改变缓存状态。重复运行同一矩阵得到的是该重复执行场景的性能，应与实际应用的输入复用方式相符。对很短的内核，两事件间还可能存在主机未及时提交后续启动导致的空隙；因而重复平均时间仍需结合时间线理解。

流间可以通过事件记录函数 `cudaEventRecord` 与流等待函数 `cudaStreamWaitEvent` 建立“生产完成后再消费”的依赖。先在生产流记录完成事件，再让消费流等待该事件，就只建立了所需的先后关系，并不要求主机一直阻塞。若要测量多条流共同完成的总设备区间，需要让结束标记依赖所有相关流的完成事件；随意在两条无依赖关系的流上分别记录开始和结束事件，不能定义可靠的整体工作范围。

主机与设备传输能否和计算重叠，还取决于复制引擎、数据依赖、流配置，以及主机缓冲区是否适合异步传输。多条流提供表达并发的机会，实际是否重叠必须在时间线上确认。相同原则也适用于频率：记录 GPU、工具版本、时钟策略和测量范围，才能解释不同测试之间的差异。

14.3 系统时间线与内核指标采集

14.3.1 在统一时间轴上定位耗时

系统级分析将 CPU 活动、CUDA 应用程序接口（application programming interface，API）调用、GPU 内核和数据传输标在统一的时间轴上。它能区分几种表面上都表现为“程序很慢”的情况：主机尚未提交工作，GPU 正在搬运数据，目标内核本身运行较久，或者某个同步点正在等待前面的工作完成。统计视图给出调用次数、累计时间、平均时间等信息，时间线则展示这些事件之间的重叠和先后关系。

图14.2中，主机上的流同步调用 `cudaStreamSynchronize` 与 GPU 上连续执行的矩阵乘法重叠。该调用的长区间表示主机在等待指定流完成；沿同一时间轴观察 GPU，才能判断等待期间发生了什么。若直接把主机同步累计时间与 GPU 内核累计时间相加，同一段真实经过的时间就可能被重复计算。

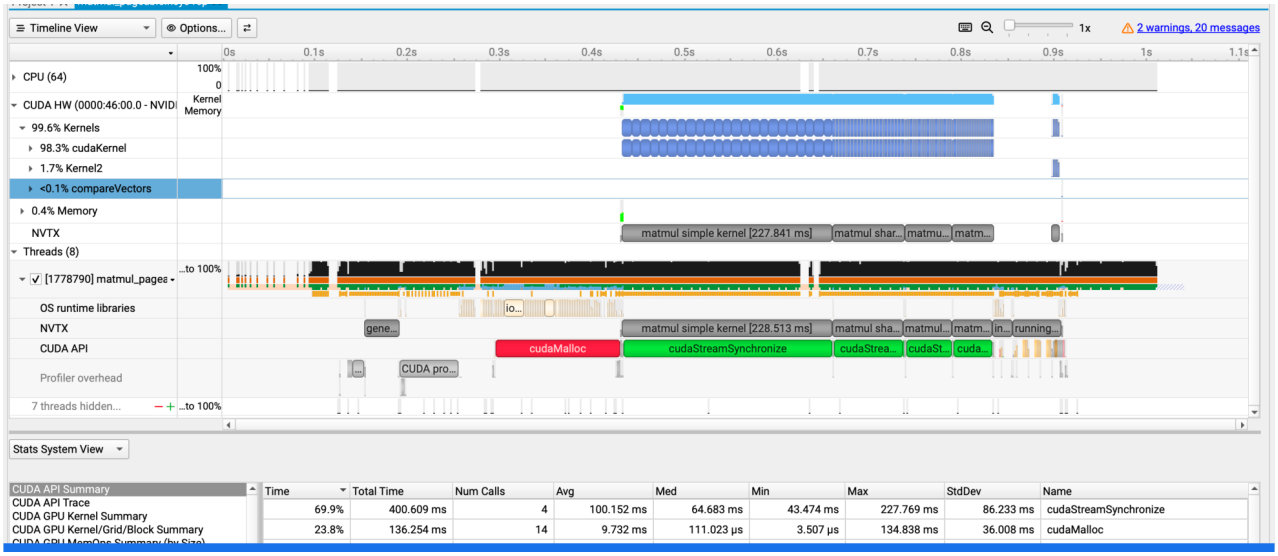


图 14.2 主机、CUDA 接口和 GPU 活动的统一时间线。同步函数在主机侧持续等待的同时，设备侧可以连续执行多个内核；区间长度需要结合所属轨道解释。

分析时可以先观察最长区间，再检查其中的结构。若多个短内核之间有明显空隙，应查看主机提交、同步和数据准备；若一段传输之后才开始计算，应检查依赖是否允许重叠；若单个内核占据主要时间，就适合进入内核级分析。内存分配也可能是显著的一次性开销，反复测试时应明确它是否属于目标应用的正常路径。

厂商工具扩展接口（NVIDIA Tools Extension，NVTX）允许程序为主机代码区间命名。命名区间能把“初始化”“矩阵乘法”“结果校验”等阶段对应到时间线。主机区间的结束通常只说明该段主机代码已执行完；如果其中只提交了异步 GPU 工作，设备执行可能延续到

区间之后。命名区间、GPU 上对应活动及同步事件需要共同解释。

14.3.2 系统级采集命令与报告

远端采集与本地查看可以分开完成：在具有目标 GPU 的机器上运行命令行采集器 nsys，生成 .nsys-rep 报告，再用图形客户端 nsys-ui 打开。查看已有报告不需要在本地重新执行同一个 GPU 内核。一次普通采集可写成：

代码 14.6 系统级采集与报告查看

```
nsys profile --trace=cuda,nvtx,osrt \
  --output=gemm_system --force-overwrite=true ./gemm_demo
nsys-ui gemm_system.nsys-rep
```

其中 profile 启动待测程序并按指定范围采集。--trace 选择 CUDA、命名区间和操作系统运行时等事件；--output 指定报告文件名的主体；--force-overwrite=true 允许覆盖同名报告。命令行中的续行符必须位于物理行末，后面不应再接注释。

表 14.3 系统级采集的常用控制选项

选项	用途
-o、--output	设置输出报告名称
-f、--force-overwrite	控制是否覆盖已有报告
-e、--env-var	为被分析程序设置环境变量
-y、--delay	延迟指定秒数后开始采集，适合略过已知的启动阶段
--capture-range=nvtx	用指定的 NVTX 区间控制采集范围
-p、--nvtx-capture	选择 NVTX 区间；需要同时启用相应范围控制
--stats=true	采集后生成统计摘要；数据库导出和统计会增加分析时间

对较复杂的程序，交互模式把启动和采样控制分开：launch 在适合分析的环境中启动程序，start 与 stop 控制采样开始和结束，shutdown 结束分析会话。提供 cancel 的版本可以用它取消采集并丢弃尚未保留的数据。停止采样、结束会话和终止目标进程属于不同动作，目标进程的后续状态还受会话配置影响。某些 CUDA 事件需要在程序启动时注入分析支持，不能假设对任意已运行进程开始采样都能补齐此前的接口跟踪。

14.3.3 内核级采集、筛选与源码范围

内核级分析以一次或若干次选定的内核启动为对象，读取相关硬件计数器，并组织成吞吐率、存储工作负载、调度、占用率及源码热点等报告。命令行工具 `ncu` 直接接收选项和目标程序，不采用 `nsys profile` 那样的子命令形式。其报告扩展名为 `.ncu-rep`，图形客户端为 `ncu-ui`。

代码 14.7 筛选预热之后的一次目标内核启动

```
ncu --set full --kernel-name regex:gemm_shared \
  --launch-skip 5 --launch-count 1 \
  --export gemm_kernel --force-overwrite ./gemm_demo
ncu-ui gemm_kernel.ncu-rep
```

这个例子先按内核名称筛选，再跳过前 5 次匹配的启动，只分析接下来 1 次匹配启动。若被测程序使用其他内核名称，就应相应替换过滤表达式。跳过次数按匹配结果计数；若程序中穿插了初始化或校验内核，它们不会仅因位置较早就自动消耗这里的匹配次数。

表 14.4 内核级采集的常用选项

选项	用途
`-o`、`--export`	导出报告；`-f` 允许覆盖已有输出
`--set basic`、`--set full`	选择基本指标或完整指标集；`--list-sets` 列出可用集合
`-k`、`--kernel-name`	按内核名称或匹配表达式筛选
`--kernel-id`	用上下文、流、内核名称或启动编号等条件精确定位
`-s`、`--launch-skip`	跳过指定数量的匹配启动
`-c`、`--launch-count`	限制采集的匹配启动次数
`--profile-from-start off`	由程序中的性能分析控制接口决定开始与结束
`--nvtx-include`	配合 `--nvtx`，只分析匹配 NVTX 区间内启动的内核

程序也可以显式圈定采集范围。下面的函数接收与计时函数相同形式的启动对象，要求它把内核提交到给定流，并假定目标上下文中没有其他流并发提交工作。先排空指定流此前的工作，再开启采样、执行目标并等待其完成，最后关闭采样，使范围边界清楚。控制接口作用于当前上下文，因此若有其他流在范围内提交内核，这些工作也可能被采集。

代码 14.8 以运行时接口圈定目标内核

```
#include <cuda_profiler_api.h>

template<class Launch>
void profile_one(Launch launch, cudaStream_t stream) {
  check(cudaStreamSynchronize(stream));
  check(cudaProfilerStart());
  launch();
  check(cudaGetLastError());
  check(cudaStreamSynchronize(stream));
  check(cudaProfilerStop());
}
```

代码 14.9 配合程序内部范围进行采集

```
ncu --profile-from-start off --set full \
  --export gemm_range ./gemm_demo
```

代码14.8复用代码14.4中的错误检查函数。启动与停止接口分别为 `cudaProfilerStart` 和 `cudaProfilerStop`。若命令在可执行文件名之后出现 `--profile` 一类参数，该参数属于待测程序自身，只有程序显式实现了相应逻辑才有效；它与分析器的 `--profile-from-start` 属于不同参数层级。

14.3.4 采集扰动与可比较的报告

硬件能够同时读取的计数器数量有限，完整指标集往往需要**重放**(*replay*)：分析器将一次内核启动划分为若干采集轮次，在必要时保存和恢复相关内存状态。采集还可能改变缓存状态、时钟控制和原本可重叠的内核执行关系。因此，整条 `ncu` 命令的墙钟耗时包含采集与数据处理开销，不能拿来充当正常的单次内核时间。

正常 CUDA 事件计时回答“在给定执行场景下需要多久”，分析报告则提供“为何如此”的硬件证据。比较不同内核的报告时，应尽量固定输入、启动次数、指标集、缓存与时钟控制，并选择相同阶段。开启完整指标前，可以先采集基本报告确定目标；待有明确假设后再补充相应细节，以减少无关数据和重放次数。

图形界面的**基准比较**(*baseline comparison*)可以把一个报告中的目标内核设为参考，并展示另一内核各项指标的相对变化。若基准值为 x_0、新值为 x_1，相对变化为 $(x_1 - x_0)/x_0$。命中率从 98% 降到 52% 是降低 46 个百分点，相对下降约 46.9%；报告显示的相对百分比应与这两种口径区分。对于基准中原本为零的指标，相对百分比通常没有有用含义，应直接查看新出现的绝对数值。

14.4 吞吐率、屋顶线与存储流量

14.4.1 理论峰值与资源利用指标

硬件的理论吞吐能力常被称为峰值上限（Speed of Light，SoL）。内核报告中的计算和存储汇总指标，用相对于对应资源峰值的比例描述当前活动强度。由于一个汇总项包含多个组成资源，应先检查它由哪些计数器形成，再判断接近饱和的是浮点流水线、加载 / 存储流水线、缓存入口还是设备内存接口。

设某个汇总指标的组成资源利用比例为 $u_1, \dots, u_q$，相关吞吐指标采用最大组成比例，即 $u_{\text{group}} = \max_j u_j$。例如，加载 / 存储通路达到约 98%，而浮点流水线只有较低活动度，汇总项仍可能接近 98%。它描述的是最忙组成资源的状态，不能直接用来计算全芯片的总体浮点效率。

这与式(14.2)的效率 $\eta = P/P_{\text{peak}}$ 有不同的用途。η 由选定的浮点工作量和耗时计算，回答目标运算相对于对应算力峰值达到了多少；资源汇总及其分解则帮助判断什么硬件正在承受最大压力。若两者相差很大，通常应继续检查计数口径、精度、时钟，以及非算术指令的开销。

14.4.2 算术强度与屋顶线模型

算术强度（*arithmetic intensity*）表示每传输一个字节所完成的浮点运算量。设某内核工作量为 F，在指定存储层级 ℓ 实际读取和写入的总字节数为 Q_ℓ，则 $I_\ell = F/Q_\ell$，单位为 FLOP/byte。存储层级必须明确：设备内存的读写量、L2 与 L1 之间的流量、共享内存访问量，各自对应不同的强度和带宽上限。

先考虑一个只有计算峰值 P_{peak} 与指定层级带宽峰值 B_ℓ 的模型。完成全部计算至少需要 F/P_{peak} 时间，完成全部数据传输至少需要 Q_ℓ/B_ℓ 时间。即使二者理想重叠，总时间也不能小于其中的较大者。因此

$$t \geq \max\left(\frac{F}{P_{\text{peak}}}, \frac{Q_\ell}{B_\ell}\right) \quad \Longrightarrow \quad P = \frac{F}{t} \leq \min\left(P_{\text{peak}}, B_\ell I_\ell\right). \tag{14.4}$$

这个上界称为**屋顶线模型**（*roofline model*）。在双对数坐标下，$B_\ell I_\ell$ 是斜线，P_{peak} 是水平线；二者相交处的算术强度为 $I_\ell^* = P_{\text{peak}}/B_\ell$，也称转折点。

当 $I_\ell < I_\ell^*$ 时，带宽项更小；当 $I_\ell > I_\ell^*$ 时，计算项更小。在内核具有足够并行度、能够充分驱动相关资源的条件下，二者分别对应**带宽受限**（*memory-bound*）和**计算受限**（*compute-*

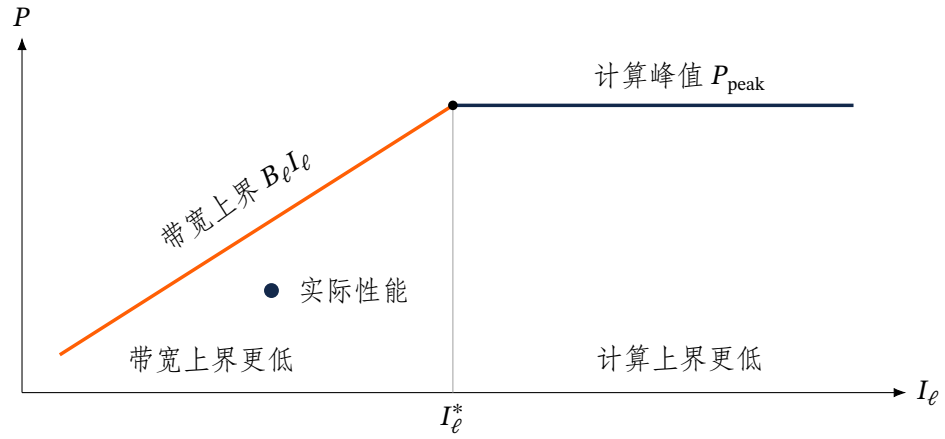


图 14.3 屋顶线模型的双对数示意。上界由算力和指定层级带宽中更严格的一项确定，实际内核还可能受到延迟、调度或指令通路限制。

bound) 的典型区域。若测量点离两条上界都很远，则需要进一步查找延迟、依赖、指令发射或其他片上通路的限制；仅比较横坐标与转折点不足以给出完整诊断。

例如，设某 GPU 对当前精度的峰值为 20 TFLOP/s，对应层级带宽为 1 TB/s，则转折点为 20 FLOP/byte。算术强度为 4 的内核，其模型上界为 4 TFLOP/s；若实测只有 0.5 TFLOP/s，就仅达到该上界的 12.5%，还需检查为什么带宽没有被有效使用。若强度提高到 80，带宽项达到 80 TFLOP/s，模型上界则由 20 TFLOP/s 的计算峰值决定。这些数字用于说明计算步骤，并非某一实验 GPU 的规格。

不同 GPU 的算力和带宽比例不同，同一个内核的横坐标也可能因缓存行为改变。因此，换用计算峰值更高的 GPU 后，内核可能更接近带宽限制；反过来，带宽提高较多的设备也可能让计算上界成为更严格的约束。使用多个精度的屋顶线时，还需要选择与实际执行精度相符的水平峰值线。

14.4.3 矩阵乘法中不同层级的复用

算术强度的推导必须说明数据流量假设。对单精度 $C = AB$，若 A 和 B 各从设备内存读入一次，C 写回一次，则理想流量为 $Q_{\text{once}} = 4(MK + KN + MN)$ 字节，理想强度为

$$I_{\text{once}} = \frac{2MNK}{4(MK + KN + MN)}. \tag{14.5}$$

对边长为 n 的方阵，这给出 $I_{\text{once}} = n/6$。这个结果描述一次读入、一次写回的理想流量；真实分块中，不同线程块可能重复读取输入子块，能否由缓存承接这些重复访问取决于实际执行。

为了比较源代码层面的加载需求，可暂时忽略缓存，并令 n 能被分块宽度 T 整除。简单内核有约 $2n^3$ 次单精度输入加载和 n^2 次输出写入，因此逻辑字节数约为 $8n^3+4n^2$，强度在 n 较大时趋近 1/4。共享分块将输入加载减少到约 $2n^3/T$ 个元素，逻辑字节数变为 $8n^3/T+4n^2$，相应强度趋近 $T/4$。例如 $T=16$ 时，这个逻辑加载模型给出约 4 FLOP/byte。

这里的 1/4、$T/4$ 与 $n/6$ 采用了不同的数据流量假设。前两者比较内核发出的逻辑全局访问需求，后者是理想的一次读写边界。若使用设备内存屋顶线，就应以实测设备内存字节数计算 I_{DRAM}；缓存命中的重复加载不会每次都计入这部分字节。因此，简单内核在实测设备内存屋顶线中的强度完全可能高于共享内核，同时运行得更慢。

在带宽上界主导且其他资源仍有余量时，提高每份已加载数据上的有效计算量，例如复用数据或融合需要相同输入的运算，可以提高强度。新增计算仍要满足寄存器、同步和指令通路的限制；人为加入无用运算，只会改变统计工作量，不能构成有意义的应用加速。对已经接近计算峰值的内核，则应考虑减少必要运算、改善指令组织，或在精度要求允许时使用更适合的计算路径。低精度路径的峰值提升幅度取决于具体架构和指令，不采用固定倍数作为通用结论。

14.4.4 从存储工作负载图读取实际数据流

存储工作负载图（Memory Workload Analysis）把左侧的逻辑存储空间与右侧的物理缓存层级联系起来。图14.4中，左侧记录指令活动，中间记录向存储单元发出的请求，右侧箭头则显示 L1、L2 与设备内存之间的数据流量。不同位置的数字描述不同对象，不能因为都与“内存访问”有关就直接相互替代。

指令（*instruction*）是实际执行的加载或存储操作；**请求**（*request*）是指令执行时向对应硬件单元提出的访问需求；**扇区**（*sector*）是缓存传输的计量粒度；**波前**（*wavefront*）表示硬件为了完成请求而进行的一轮服务。一个线程束的访存指令可以涉及多个地址、多个扇区或多个服务波前。报告中不少指令和请求指标以线程束为计数单位，而字节流量按实际传输计数，使用时还应核对单位。

在本讲的 L1 / L2 访存分析中，一个扇区为 32 字节。若同一请求的 32 个线程分别读取 4 字节，全部地址连续且与 32 字节边界对齐，则共需 128 字节，对应 4 个扇区。地址分散时，硬件可能要搬入更多扇区，虽然线程最终使用的数据量没有增加。部分线程不活跃、广播或每线程访问宽度不同时，理想扇区数也会随之改变。

缓存命中率回答按指定统计单位计算的访问命中比例；本图中的 L1 / L2 命中率按扇区统计，不能与请求命中率混用。吞吐率回答单位时间处理了多少数据或请求。高命中率与低延迟通常有利，但大量命中请求仍可能让片上入口、加载流水线或请求队列饱和。反过来，

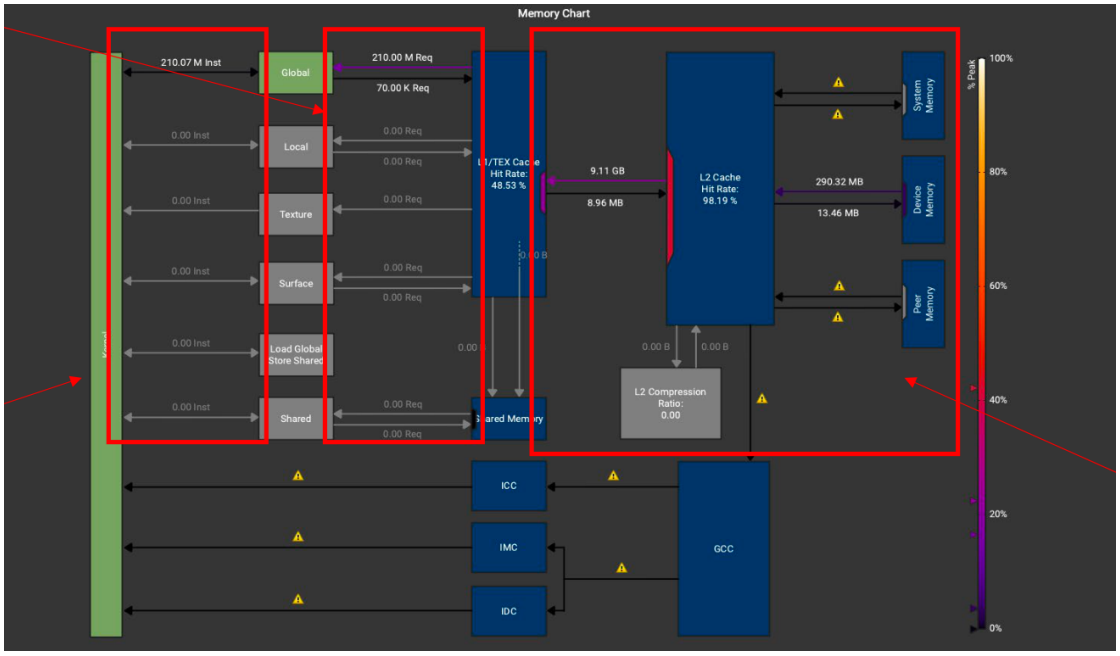


图 14.4 存储工作负载中的指令、请求与字节流量。读取可以在不同缓存层级得到满足，缓存命中与读写策略共同决定层级之间的流量。

一次优化减少了请求数，却把访问顺序改变得更不利于复用，外层流量也可能上升。判断效果需要联合检查指令数、请求数、每请求扇区数、各层命中率和运行时间。

14.4.5 全局访存合并与多余扇区

合并访存（*coalesced memory access*）利用同一线程束各线程地址之间的空间关系，尽可能用较少的存储事务服务访问。表14.5以 32 个活跃线程、每线程读取一个单精度数为例，令 $j = 0, \dots, 31$，a_j 为第 j 个线程的字节地址，并假定基地址为 0。各线程读取的数据互不重叠，实际需要的有用字节总数均为 128。

表 14.5 地址布局对所需扇区数量的影响

地址形式	覆盖扇区数	扇区总字节	解释
$a_j = 4j$	4	128	连续且对齐，全部传输字节均有用
$a_j = 4j + 4$	5	160	连续访问跨过额外扇区边界
$a_j = 8j$	8	256	每隔一个单精度元素读取，利用一半字节
$a_j = 32j$	32	1024	每线程落入不同扇区，只使用其中 4 字节

这个例子中的扇区利用比例可写为 $\eta_{\text{sector}} = 128/(32n_s)$，其中 n_s 是实际覆盖的扇区数，四种布局分别得到 100%、80%、50% 和 12.5%。这些是对应请求的传输效率；后续缓存命中可能再次改变设备内存实际流量，因此不能把该比例直接当成整机带宽效率。

多余扇区指标把某种理想访问需求与实际需要的扇区比较，用于定位可能存在的合并不足。它指出值得检查的代码位置，随后仍需结合每线程字节数、活跃线程数、对齐与请求类型解释。通常可以通过让相邻线程访问相邻元素、调整线程到输出的映射、修正行跨度或采用更合适的加载分工来改善。

对二维线程块，还应记住线程线性编号优先沿 x 方向增长。例如 16×16 的线程块中，一个线程束跨越两行、每行 16 个线程；是否高效取决于这两个半行分别覆盖多少扇区，以及行起点是否对齐。线程束跨两行本身不会必然导致浪费，必须代入实际地址分析。

14.4.6 共享内存存储体与宽访问

共享内存按**存储体**(*bank*)组织。在本讲使用的基本模型中，共有 32 个存储体，连续的 32 位字映射到连续存储体，每个存储体每周期提供一个 32 位字的带宽。若字节地址为 a，则存储体编号为 $b(a)=\lfloor a/4\rfloor \bmod 32$。访问无需按地址连续排列；只要同一次服务中的不同地址能分散到不同存储体，就可以并行服务。

存储体冲突(*bank conflict*)发生在同一服务阶段中，多个不同的 32 位字映射到同一存储体，导致额外串行服务。多个线程读取同一个 32 位字时，可以由一次读取及广播满足，包括该字内较窄的读取；这与读取同一存储体中的不同字有不同的代价。同地址并发写入则还涉及数据竞争，不能用读取广播来说明其正确性。

考虑完整线程束中的标量读取，每线程访问字索引 $w_j=w_0+sj$，其中步长 s 是正整数，32 个地址互不相同。存储体编号为 $(w_0+sj) \bmod 32$。若两个线程间隔 Δj 后落入同一存储体，就有 $s\Delta j\equiv 0 \pmod{32}$。令 $d=\gcd(s,32)$，最小正间隔是 $32/d$，所以访问覆盖 $32/d$ 个存储体，每个存储体需要服务 d 个不同地址。在这个标量模型下，冲突程度就是 d。

因此，$s=1$ 访问全部 32 个存储体，没有额外冲突；$s=2$ 只访问 16 个存储体，形成两路冲突；$s=32$ 的 32 个地址都落入同一存储体，形成 32 路冲突；$s=33$ 又能覆盖全部存储体。二维共享数组宽度为 32 时，沿列读取对应步长 32；将物理行宽填充到 33，就能让相邻行的同一列落入不同存储体。这个推导针对标量、互异地址；同地址广播和宽访问需要分别处理。

宽共享读取还要求区分每线程宽度和整个线程束的数据量。一条 `LDS.128` 指令中的 128 表示每线程 128 位，即 16 字节，可以装入四个单精度数。若 32 个线程读取互不相同的连续 16 字节区域，总请求为 512 字节；在常见的每轮 128 字节服务模型下，需要四个理想波前。这些必需的波前属于访问宽度带来的工作量，不能仅因同一存储体在不同波前中重复出现就判为冲突。

共享访问的多余波前指标考察实际服务波前超过理想需求的部分，能够帮助定位冲突。

向量化可能减少指令和请求数量，同时保留相同的传输字节数；若布局和对齐合适，它就能缓解指令队列压力。至于具体共享读取是否被编译为宽指令，以及额外波前是否下降，还需要结合机器指令与报告验证。

14.5 线程束调度、占用率与指令定位

14.5.1 资源预算如何限制占用率

占用率(*occupancy*)描述一个 SM 中驻留线程束数相对于硬件最大线程束数的比例。**理论占用率**(*theoretical occupancy*)由内核资源需求和启动配置限制，**实际占用率**(*achieved occupancy*)则由执行过程中测得的平均驻留情况得到。驻留表示线程束已经占有相应执行状态，并不要求它当前已经就绪。

设每个线程块有 T_b 个线程，对应 $w_b = \lceil T_b/32 \rceil$ 个线程束；一个 SM 允许的最大线程数、线程束数和线程块数分别为 $T_{\max}$、$W_{\max}$、$B_{\max}$。设寄存器容量为 R_{SM}，共享内存容量为 S_{SM}，每块实际占用量为 R_b 和 S_b，包含需要计入的驱动保留与分配取整。在简化资源预算中，可驻留线程块数 B_{res} 满足

$$B_{\mathrm{res}} \le \min\left\{B_{\max}, \left\lfloor \frac{T_{\max}}{T_b} \right\rfloor, \left\lfloor \frac{W_{\max}}{w_b} \right\rfloor, \left\lfloor \frac{R_{\mathrm{SM}}}{R_b} \right\rfloor, \left\lfloor \frac{S_{\mathrm{SM}}}{S_b} \right\rfloor\right\}, \qquad O_{\mathrm{theory}} = \frac{B_{\mathrm{res}} w_b}{W_{\max}}. \tag{14.6}$$

当实际分配量 $S_b = 0$ 时，应略去共享内存这一限制项。寄存器需求不能总是直接用每线程寄存器数乘线程数，因为真实分配会按线程束、线程块或子分区等硬件粒度取整；更详细的资源限制还要由目标设备的占用率报告确认。

考虑一个明确的预算例子：每个 SM 最多驻留 1536 个线程、48 个线程束和 16 个线程块，具有 65536 个 32 位寄存器及 100 KiB 可用共享内存。内核每块 256 个线程，即 8 个线程束；每线程需要 32 个寄存器，共享内存预算中的每块总占用量为 16 KiB。暂时忽略寄存器的额外分配取整，则每块占 8192 个寄存器，五个限制依次为 $16, 6, 6, 8, 6$，最多可驻留 6 块，对应 48 个线程束和 100% 理论占用率。

若只把寄存器用量提高到每线程 64 个，每块需要 16384 个寄存器，寄存器上限就变成 4 块，于是最多驻留 32 个线程束，占用率降至 $32/48 \approx 66.7\%$。这并不能单独判断粗化后的内核是否更慢：新增寄存器也可能减少重复读取、保存更多独立累加器，从而改善每个线程束的执行效率。这个预算例子中的硬件数量用于演示计算，不作为所有 GPU 的通用规格。

实际运行还受工作量分布影响。线程块太少、不同块执行时间不一致，以及最后一轮只剩少量工作，都可能使实际占用率低于理论值。若每轮能容纳 $B_{\mathrm{res}} N_{\mathrm{SM}}$ 个线程块，但总块数

不能被该数量整除，末轮便无法像满负载阶段那样填满设备。报告中的各项平均值还可能使用不同的活跃周期口径，因此需要同时检查 SM 活动和整个时间线。即使线程束槽位已占满，边界线程束中的部分线程也可能没有有效输出。

例如，若设备同时能容纳 264 个线程块，总工作量原本包含 1584 块，则可以形成 6 轮满负载执行。若粗化后块数变为 396，就形成一轮完整工作和一轮只有 132 块的尾部工作。假定各块耗时相同、分配理想，按两个完整轮次的容量计算，槽位利用比例仅为 $396/(2\times264) = 75\%$。这个比例描述网格规模与驻留容量不匹配带来的尾部损失；粗化还会改变每块耗时和资源需求，因此它不能直接预测最终加速比。

14.5.2 从驻留到就绪，再到实际发射

线程束调度器维护一组驻留线程束。每周期先检查哪些线程束的下一条指令已具备执行条件，再从这些就绪线程束中选择可发射者。因此，调度报告中的**驻留线程束**（*active warps*）、**就绪线程束**（*eligible warps*）和**已发射线程束**（*issued warps*）分别对应候选集合、当前具备条件的子集和实际发射行为。

图14.5给出的例子中，每调度器的理论上限为 12 个线程束，平均实际驻留 11.61 个，平均就绪却只有 0.75 个，平均每周期发射约 0.46 个线程束。没有就绪线程束的周期约占 53.89%。驻留状态已经接近满额，主要问题是很多线程束同时在等待，调度器缺少可以立即发射的工作。

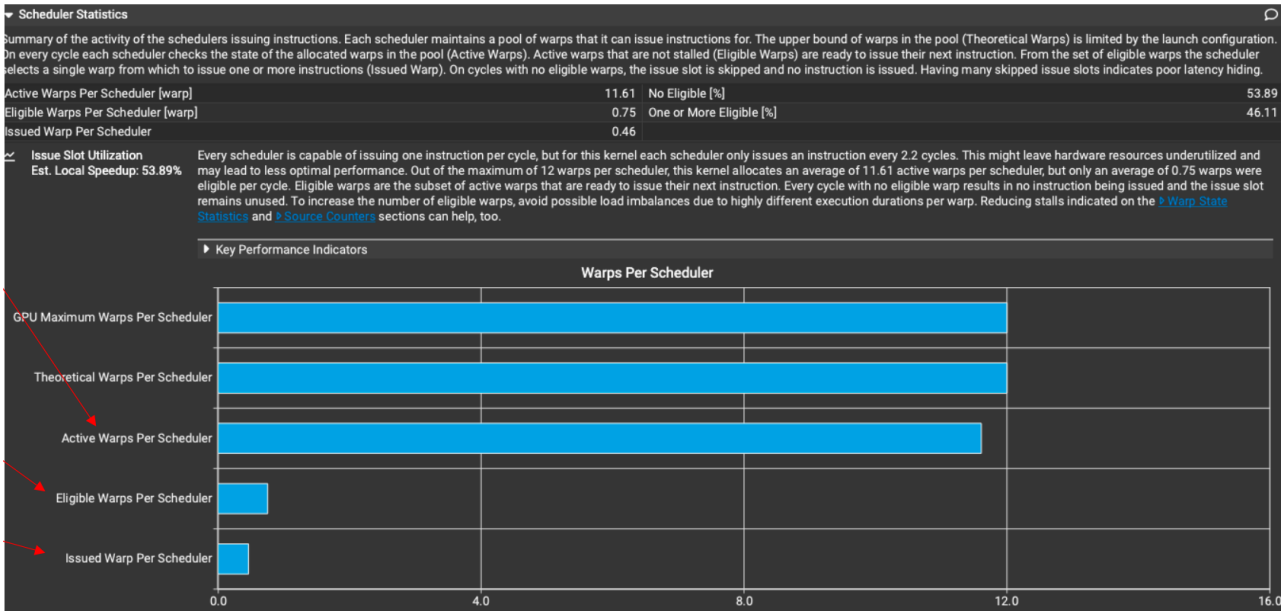


图 14.5 每调度器的理论、驻留、就绪和发射线程束统计。平均驻留数量接近上限，而就绪与发射数量明显较低，说明需要进一步检查等待原因。

平均每周期发射 0.46 个线程束，可理解为调度器平均约每 1/0.46 ≈ 2.17 周期发射一次。报告中的“每条发射指令对应的线程束周期”还会把各驻留线程束经历的周期汇总，示例值约为 25.18，其中包含被选中、就绪但未被选中以及停顿等状态。两者的分母不同：前者看调度器整体的发射节奏，后者反映每次指令发射所对应的线程束周期。粗略地，11.61/0.46 ≈ 25.2，可以帮助理解两个数字为何处于不同数量级。

如果驻留数量很低，应先检查线程块规模、寄存器、共享内存和网格大小；如果驻留充足而就绪很少，就应检查数据依赖、存储等待与同步；如果就绪线程束较多而目标流水线已经持续忙碌，则要考虑其接收能力和指令组合。这一顺序能把占用率与真正的发射瓶颈联系起来。

14.5.3 线程束停顿与延迟隐藏

线程束状态统计（Warp State Statistics）把平均线程束周期分配到被选中、就绪但未被选中以及各类停顿状态。**记分板**（*scoreboard*）跟踪指令依赖是否已经满足：如果下一条指令需要某个尚未返回的加载结果，该线程束就必须等待。其他线程束仍可继续执行，因此某类停顿存在，并不自动说明整个 SM 同时空闲；应先结合调度报告确认是否有大量未使用的发射机会。

表14.6列出与矩阵乘法诊断最相关的几类状态。对应硬件通路的名称较具体，尤其需要区分等待数据返回与等待队列空位。

表 14.6 常见线程束等待状态及其解释

状态	正在等待什么	可进一步检查的内容
长记分板等待（Long Scoreboard）	L1 / 纹理路径操作产生的数据依赖，包括全局、局部、纹理或表面访问	请求合并、缓存命中、加载到使用之间的独立工作
短记分板等待（Short Scoreboard）	非 L1 / 纹理路径的相关操作依赖；共享读取是常见原因	共享布局、存储体冲突及共享数据的消费位置
存储输入输出节流（MIO Throttle）	对应指令队列尚无空位	共享加载数量、可否使用较宽加载，以及相关通路压力
数学流水线节流（Math Pipe Throttle）	所需数学流水线暂时不能接收下一条指令	指令是否集中在特定流水线，有无冗余运算或可调整的指令组合
同步屏障等待（Barrier）	同步要求的其他线程束尚未到达	块内工作是否均衡、加载与计算分工、同步的位置
未被选中（Not Selected）	线程束已经就绪，但本周期选择了其他线程束	结合实际发射率判断；存在这种状态说明调度器有可选工作

长记分板等待涉及的加载可能命中缓存，因而不能直接解释成全部时间都在等待显存。短记分板和存储输入输出通路还可能涉及特殊数学或动态分支等操作，具体应以热点指令为准。存储输入输出节流侧重队列接收能力；如果大量窄共享加载不断挤入同一入口，减少加

载指令数量就可能有效，即使共享数据的总字节数保持不变。

在前述简单内核的示例中，每条发射指令对应约 25.18 个线程束周期，其中长记分板等待约占 20.6 周期，即约 81.8%。结合接近满额的驻留数量和较低的就绪数量，可以把重点放在加载依赖及其延迟隐藏上。增加更多驻留线程束受到硬件上限限制；提高数据复用、让加载更高效，或在等待结果期间提供其他独立计算，都可能改善当前状态。

同一线程束中的单一累加器也会形成连续依赖：下一次乘加需要前一次乘加的结果。使用多个输出累加器，可以提供多条独立链，让编译器有机会交错安排指令。这与线程束之间的切换共同隐藏延迟，但会增加寄存器占用。诊断时需要观察最终发射率与总时间，不能把某个占用率或某类停顿的单项变化作为唯一目标。

停顿统计还常以每条发射指令或采样次数作为分母。如果优化删除了很多指令，某种剩余等待的比例可能上升；执行时间仍可能下降，因为总工作量和主要队列压力已经减少。比较时应保持分母意识，同时观察绝对指令数量、运行时间和多个相关指标。

14.5.4 从源代码行追踪到机器指令

源码热点视图将设备代码行号与编译后的指令联系起来。虚拟指令集（Parallel Thread Execution，PTX）提供中间表示，目标架构机器指令（SASS）则反映具体 GPU 上使用的加载、存储、乘加和控制指令。源代码中的一行表达式可能展开为地址计算、加载、乘加与循环控制等多条指令；编译器还会进行展开、重排和寄存器分配，所以源码行与机器指令通常不一一对应。

使用 `-lineinfo` 编译后，可以按源码行查看执行指令数量、寄存器活跃情况与停顿采样，再沿 PTX 和 SASS 确认实际执行路径。例如，`LDG` 表示全局读取，`LDS` 表示共享读取，`STS` 表示共享写入，`FFMA` 表示单精度融合乘加。指令后缀还可能说明访问宽度或其他执行语义。

一个典型依赖链可以写成下面的机器指令结构。第一条从全局内存加载到寄存器 `R2`，第二条乘加使用 `R2`；地址寄存器和其他源寄存器在此前已经准备好。

代码 14.10 加载结果到消费指令的依赖示意

```
LDG.E R2, [R4];
FFMA  R8, R2, R6, R8;
```

若 `R2` 尚未准备好，停顿可能记录在等待使用它的乘加指令上。因此，热点落在乘加行并不说明乘加流水线本身一定是瓶颈；需要沿寄存器依赖回溯到生产该数据的加载。类似地，一条共享存储也可能因为等待前面的全局加载结果而呈现长记分板等待。

存活寄存器（*live registers*）表示在某个程序位置仍保存着后续需要使用的值的寄存器。它

帮助理解哪些临时值同时占据状态，与整个内核最终分配的每线程寄存器数量相关，但两者不是逐行相同的数值。查看向量化或线程粗化后的代码时，应同时确认宽读取是否生成、寄存器压力是否增大，以及占用率与局部内存访问是否发生变化。

一条完整的定位过程由这些观察逐步收敛：时间线确定主要耗时内核，吞吐与屋顶线提供性能上界，存储和调度报告定位繁忙通路或等待类型，源码热点与机器指令再确定可以修改的代码。修改之后，预期受影响的计数器应出现对应变化，并由正常计时验证实际收益。

14.6 矩阵乘法的比较与优化

14.6.1 三组设备上的测量结果

表14.7比较简单内核、共享分块、共享改进版、寄存器与共享内存联合分块，以及 cuBLAS。三组软件环境分别为 A40 上的 CUDA 12.4、L40S 上的 CUDA 12.2 和 B200 上的 CUDA 12.8。各组使用相同目标工作量，组内加速比以简单内核为基准。

表 14.7 矩阵乘法的独立计时、吞吐率与峰值效率

设备	实现	时间（ms）	GFLOP/s	加速比	效率
A40	简单内核	9.92	677	1.00	1.81%
	共享分块	3.34	2 011	2.97	5.38%
	共享改进版	2.29	2 935	4.33	7.85%
	联合分块	1.90	3 545	5.22	9.48%
	cuBLAS	0.30	22 026	33	58.90%
L40S	简单内核	3.47	1 937	1.00	2.96%
	共享分块	1.92	3 498	1.81	5.35%
	共享改进版	1.31	5 138	2.65	7.85%
	联合分块	1.06	6 354	3.27	9.71%
	cuBLAS	0.18	36 631	19.28	55.98%
B200	简单内核	4.42	1 520	1.00	3.21%
	共享分块	1.35	4 980	3.27	10.53%
	共享改进版	1.07	6 256	4.13	13.22%
	联合分块	1.03	6 514	4.30	13.77%
	cuBLAS	0.17	39 907	26	84.35%

时间、吞吐率和加速比各列经过独立舍入，因而用显示的两位小数时间重新计算，可能与其他列略有出入。将表中各行时间与吞吐率相乘，可以回推出共同目标工作量约为 $F = 6.72 \times 10^9$ 次浮点运算。这个工作量适合用来检查理论时间；仅由表格和启动配置，尚不足以唯一确定 M, N, K。

表14.8按 $t_{\min} = F / P_{\text{peak}}$ 统一计算纯计算下界。A40 的峰值分母采用提升时钟条件，未必

等于整个测量区间内可持续达到的频率；L40S 与 B200 的实验分别将图形时钟锁定为 1800 MHz 和 1400 MHz。真实性能还受访存及指令通路约束，因此计算下界只用于建立参照。

表 14.8 指定峰值口径下的计算时间下界

设备	峰值（GFLOP/s）	峰值与实验时钟口径	$t_{\min}$（ms）
A40	37 400	提升时钟条件下的理论峰值	0.180
L40S	65 434	实验锁定 1800 MHz	0.103
B200	47 309	实验锁定 1400 MHz	0.142

例如，L40S 的计算过程为 $6.72 \times 10^9/(65\,434 \times 10^9)$ 秒，即约 0.1027 毫秒。相同数据下，简单内核相对于该纯计算下界的比值约为 $3.47/0.1027 \approx 33.8$。这样可以同时核对运算量、时间单位和乘加计数，避免把一次乘加在不同位置分别算作一次和两次浮点运算。

时钟控制命令使用设备编号选择目标 GPU，`-lgc` 的两个数值分别限定最小和最大图形时钟。以下示例将设备 0 锁定为 1800 MHz，最后一条恢复默认图形时钟控制。设置需要相应系统权限；B200 对应实验将数值换为 1400, 1400。

代码 14.11 实验中的图形时钟控制示例

```
sudo nvidia-smi -i 0 -lgc 1800,1800
sudo nvidia-smi -i 0 -rgc
```

共享改进版在三组设备上都快于原共享版本，但收益幅度不同；联合分块又继续改善了时间。cuBLAS 仍明显更快，说明减少全局读取只是优化链条中的一部分，还需要兼顾分块形状、寄存器复用、指令调度和各流水线的工作分配。

跨设备效率比较还受到分母影响。B200 上 cuBLAS 的吞吐率约为 39.91 TFLOP/s，略高于 L40S 的 36.63 TFLOP/s；但两组使用的峰值分母分别约为 47.31 与 65.43 TFLOP/s，因此效率分别为 84.35% 与 55.98%。效率差异同时反映实现与所选峰值口径，不能只据此概括设备的所有性能特征。简单内核在 L40S 上又比 B200 更快，也说明具体实现与资源匹配决定最终次序。

14.6.2 简单内核到共享分块：瓶颈如何改变

图14.6展示不同矩阵乘法实现的屋顶线位置。简单内核与共享分块的横坐标次序，反映的是报告所用存储层级上的实测算术强度。共享分块的运行更快，却可能因为设备内存读取增加而向左移动；联合分块和库实现则在不同程度上改善复用与执行效率。图中不同水平峰值线对应不同精度口径，判断单精度点时应使用匹配的上界。

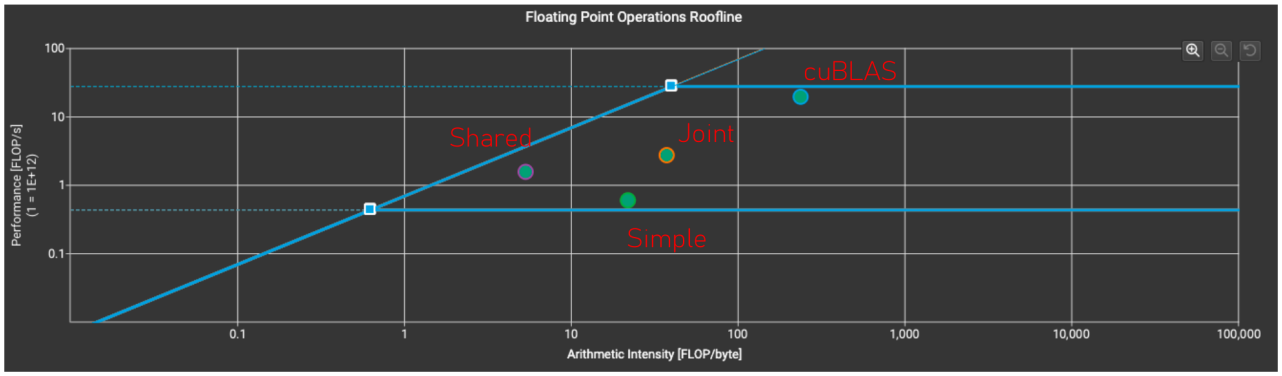


图 14.6 简单、共享、联合分块与库实现的屋顶线位置。纵坐标为浮点吞吐率，横坐标采用报告层级的实测算术强度；性能提升并不要求横坐标始终增大。

表14.9给出简单与共享内核的几项报告数据。全局加载请求由约 210.00 百万降至 13.22 百万，接近 16 倍的减少，与 $T = 16$ 分块的复用机制相符。与此同时，共享读取请求达到约 211.76 百万，说明原有的全局读取需求有很大部分转成了片上读取工作。

表 14.9 简单内核与共享分块的存储和占用率比较

指标	简单内核	共享分块
全局存储空间指令（百万）	210.07	13.29
全局加载请求（百万）	210.00	13.22
共享加载请求（百万）	0	211.76
L1 / 纹理缓存命中率	48.53%	11.21%
L2 命中率	98.19%	52.08%
L2 向 L1 返回的数据（GB）	9.11	1.69
设备内存读取量	290.32 MB	1.24 GB
理论占用率	100%	100%
实际占用率	96.86%	97.90%

简单内核的重复请求很多，但大量请求命中 L2；共享分块的逻辑请求减少后，访问顺序和复用在缓存中的表现也发生改变，L2 命中率降至约 52%。最终，L2 向 L1 返回的数据仍减少，设备内存读取却从约 290 MB 增加到约 1.24 GB。这些数字分别描述不同层级，因此能够同时成立。

共享内核的平均每请求扇区数约为 4.40，报告还显示额外扇区，提示部分全局访问有进

一步检查的空间。分析时要代入活跃线程数、每线程宽度、行跨度与对齐；不同内核的请求可能具有不同的广播与空间分布，单独比较平均扇区数并不足以确定具体代码问题。

两种内核的理论占用率都为 100%，实际占用率也都接近 97%，所以共享版本的收益并非来自显著增加驻留线程束。更明显的变化是等待从大量全局加载依赖转向共享访问和存储输入输出队列压力。共享版本的汇总资源利用率接近 98% 时，主要贡献还可以来自加载 / 存储相关通路；这正是前面区分资源汇总与浮点峰值效率的实际例子。

这些报告中的时钟与采集条件可能与表14.7的独立计时不同。报告比较用于验证硬件机制，最终加速幅度则回到同一正常计时条件下测量。

14.6.3 以向量读取缓解共享指令队列压力

共享内核的累加循环每步执行 `As[ty][k]` 与 `Bs[k][tx]` 两次共享读取，再完成一次乘加。对固定线程，连续四个 k 对应的 A 数据位于同一行相邻位置；相应的 B 数据沿列分布，相邻地址间隔 T 个元素。因此，可以把每线程连续的四个 A 元素读成一个向量，B 仍保留四次标量读取。

为了明确数据布局和对齐条件，可将 A 的共享子块表示为自然 16 字节对齐的 `float4` 数组。令 `As4[y][g]` 的四个分量依次保存原来的 A 子块元素 $A^s_{y,4g},\dots,A^s_{y,4g+3}$，并要求 T 为正的 4 的倍数。加载阶段使用对应的全局标量地址填入这些分量，在全部线程完成块内同步后，再执行代码14.12所示的累加函数。该函数只表达消费端的向量读取机制，外层分块加载、边界补零及覆盖前同步仍按共享内核的规则处理。

代码 14.12 对自然对齐的共享四元组进行向量读取

```
template<int T>
__device__ float accumulate_vector_tile(
    const float4 (&As4)[T][T / 4],
    const float (&Bs)[T][T],
    int ty, int tx, float sum) {
  static_assert(T > 0 && T % 4 == 0,
                "T must be a positive multiple of four");
  #pragma unroll
  for (int g = 0; g < T / 4; ++g) {
    float4 a = As4[ty][g];
    int k = 4 * g;
    sum += a.x * Bs[k + 0][tx];
    sum += a.y * Bs[k + 1][tx];
    sum += a.z * Bs[k + 2][tx];
    sum += a.w * Bs[k + 3][tx];
  }
  return sum;
}
```

这个表示让向量对象的类型和对齐条件明确，避免把任意未对齐地址直接解释成宽向量。每个线程仍需要相同的输入数值，并按相同的 k 顺序累加；改变的是这些值进入寄存器时使用的读取指令宽度。是否最终生成一条宽共享加载，仍由编译结果决定，需要检查机器指令。

```
55 00007feb 47264960       LDS R30, [R18.X4+0x400]
56 00007feb 47264970       LDS.128 R12, [R24]
57 00007feb 47264980       LDS R29, [R18.X4+0x440]
58 00007feb 47264990       LDS R34, [R18.X4+0x480]
59 00007feb 472649a0       LDS R32, [R18.X4+0x4c0]
60 00007feb 472649b0       LDS R31, [R18.X4+0x500]
61 00007feb 472649c0       LDS.128 R4, [R24+0x10]
62 00007feb 472649d0       LDS R28, [R18.X4+0x540]
63 00007feb 472649e0       LDS R25, [R18.X4+0x580]
64 00007feb 472649f0       LDS R26, [R18.X4+0x5c0]
65 00007feb 47264a00       LDS R27, [R18.X4+0x600]
```

图 14.7 共享向量读取对应的机器指令片段。宽指令 LDS.128 与保留的标量 LDS 同时存在，对应向量化的连续读取和其他标量读取。

以连续四次乘加为一个计数单位，原实现需要四次 A 标量读取与四次 B 标量读取，共 8 次共享加载；向量化后需要一次 A 宽读取与四次 B 标量读取，共 5 次。因此，理想共享加载请求的下降比例为

$$r_{\text{load}} = \frac{8-5}{8} = 37.5\%. \tag{14.7}$$

单看 A 的读取指令，数量减少到四分之一；把 A、B 都计入，整体减少到八分之五。四次乘加本身保持不变，所需标量数据量也保持不变。LDS.128 中的 128 位是每线程的读取宽度，不能与一轮共享服务的 128 字节混用。

14.6.4 由指标变化验证优化机制

表14.10给出原共享内核和改进版的比较。共享加载从约 211.76 百万降到 132.35 百万，下降 37.50%，与式(14.7)一致。共享存储请求保持约 13.24 百万，因此计入写入后的共享总指令下降比例较小，为约 35.29%。

表 14.10 共享读取向量化前后的报告变化

指标	共享分块	共享改进版	相对变化
共享加载请求（百万）	211.76	132.35	−37.50%
共享存储请求（百万）	13.24	13.24	0%
共享总指令（百万）	225.00	145.59	−35.29%
全局存储空间指令（百万）	13.29	13.29	0%
设备内存读取量（GB）	1.24	1.23	约 −1.34%

表中的数据量经过舍入，相对变化可能由更高精度的报告计数计算。例如 1.24 GB 与 1.23 GB 的显示值不足以重现 −1.34% 的报告变化，应将其解释为小幅下降。

全局加载请求与全局指令基本保持不变，设备内存读取量也只有小幅波动，这与局部修改共享读取方式的范围相符。图14.8的指令组合显示共享加载显著减少，浮点乘加数量保持不变；总执行指令下降约 15.32%。因此，这次改进的主要证据是减少共享加载指令与队列压力，而非显著改变设备内存流量。

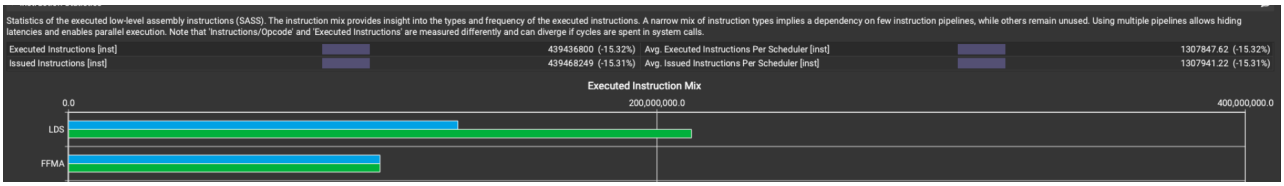


图 14.8 共享改进版与原共享版本的指令组合比较。上方共享加载条带缩短，下方单精度乘加条带基本重合，说明算术工作保留而读取指令减少。

线程束状态比较也与这个机制相符。图14.9中，存储输入输出节流的平均周期约从 24.4 降到 16.0；部分其他等待没有同步下降，长记分板等待还略有增加。这并不破坏当前解释，因为优化针对的是主要共享指令压力，总执行指令和主要等待已经减少。

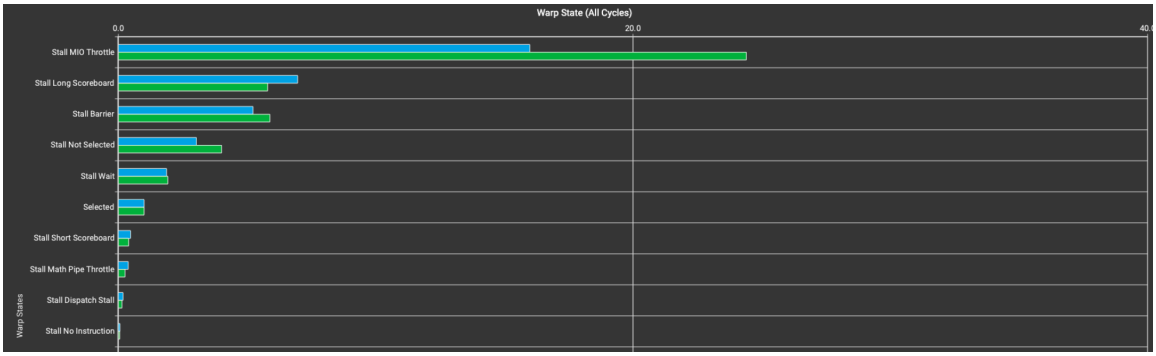


图 14.9 共享改进版与原共享版本的线程束状态比较。蓝色对应改进版，绿色对应原共享版本；主要变化是存储输入输出节流降低，其他等待不必按相同比例变化。

最后回到正常 CUDA 事件计时。A40 上耗时从 3.34 毫秒降至 2.29 毫秒，相对原共享版本达到约 1.46 倍加速；L40S 上从 1.92 降至 1.31 毫秒，约为 1.47 倍；B200 上从 1.35 降至 1.07 毫秒，约为 1.26 倍。总时间没有按共享加载请求的比例直接下降，因为全局读取、共享写入、乘加、同步及其他指令仍然存在，减少某一类请求也不等于消除了所有等待。

进一步优化时，多余扇区提示检查线程映射与对齐，片上加载繁忙提示考虑寄存器复用，寄存器用量压低驻留数量则提示调整粗化程度或分块形状。修改后应核对相关计数器、数值结果及同条件计时。

性能诊断的关键关系

诊断应同时联系工作量、指令数、各层流量、驻留资源与发射状态，并用总时间判断修改的收益。

第 15 讲

Parallel Computation Patterns: Reduction Trees

15.1 归约的运算结构与并行代价

15.1.1 二元运算、单位元与结果的含义

归约（reduction）把一组输入通过同一种二元运算合并成一个结果。设输入包含 N 个值，依次为 $x_0, x_1, \dots, x_{N-1}$，二元运算记为 $\circ$，则结果为 $r = x_0 \circ x_1 \circ \cdots \circ x_{N-1}$。求和、求积、最小值和最大值都具有这种结构。与归约相关的前缀扫描（prefix scan）会输出各个前缀的累计结果，例如求和扫描在位置 k 输出 $\sum_{i=0}^{k} x_i$。两者共享规则化的合并结构，归约只保留最终的一个值。

并行化需要明确运算允许怎样重排。**结合律** $(a \circ b) \circ c = a \circ (b \circ c)$ 允许改变括号结构，从而把一条依赖链改写成一棵树；**交换律** $a \circ b = b \circ a$ 允许交换元素顺序。后面按两半配对、再按线程束合并的求和实现都会改变分组，某些映射还会改变合并次序，因此采用满足这两条性质的理想运算模型。一般而言，保持原有顺序的归约树只依赖结合律；要允许任意顺序重排，还需要交换律。

单位元 e 满足 $e \circ a = a \circ e = a$。它既用于初始化累计量，也用于把不足一个完整数据段的位置补齐。常用运算与单位元如表15.1 所示。对于浮点最小值和最大值，这里只讨论普通有序数值，特殊非数值的处理需要另行指定。

表 15.1 常见归约运算及其单位元

运算	二元合并	单位元	补齐位置的含义
求和	$a + b$	0	不增加总和
求积	ab	1	不改变已有乘积
最小值	$\min(a, b)$	$+\infty$ 或类型上界	不成为更小的候选
最大值	$\max(a, b)$	$-\infty$ 或类型下界	不成为更大的候选

顺序归约可以先令 $a = e$，再对每个输入更新 $a \leftarrow a \circ x_i$。这需要 N 次更新；若 $N \geq 1$，

也可以直接用 x_0 初始化，只合并余下的 $N-1$ 个值。两种写法的时间复杂度均为 $\Theta(N)$，后者便于与树上必需的 $N-1$ 次二元合并比较。空输入则按单位元约定返回 e。

整数求和也需要明确数值范围。数学整数的结合律不意味着固定宽度类型可以忽略溢出：有符号整数的某个中间部分和溢出时，C++ 运算不再具有预期语义，即使最终数学结果可表示也不足以保证安全。使用无符号类型时，加法按位宽取模；只有所有累计值均在计数范围内，模运算结果才等于通常的整数总和。

对实数加法成立的结合律，在有限精度浮点运算中通常只能近似成立。以单精度数值为例，$(10^{20}+(-10^{20}))+1$ 可以得到 1，而 $10^{20}+((-10^{20})+1)$ 可以得到 0，因为小量 1 在与大幅值负数相加时被舍入掉。归约树改变括号结构，块间原子累加还可能改变执行次序，因此正确的并行求和也可能与顺序求和产生数值差异；存在严重消去时，这种差异可能明显增大。验证时应依据数据尺度比较绝对误差和相对误差；若任务要求固定的浮点结果，则还需要固定跨块合并次序及运算实现。

15.1.2 从顺序依赖链到归约树

以下并行代价讨论取 $N \geq 2$。对于 $N = 2^L$ 个输入，平衡二叉归约树在第一级完成 $N/2$ 次两两合并，第二级完成 $N/4$ 次合并，直到最后一级只合并两个部分结果。同一级的各个操作互不依赖，后一层只等待自己的两个子结果。图15.1 用八个数的求和说明这一过程。

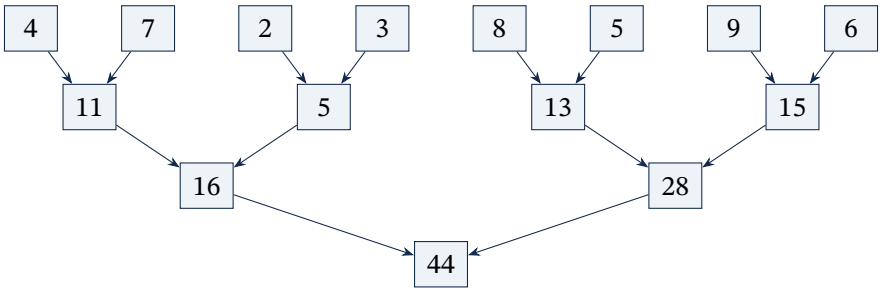


图 15.1 八个输入通过三级合并得到总和。每层分别执行四次、两次和一次加法。

用**工作量**（work）$W(N)$ 表示所有线程执行的必要二元合并总数，用**跨度**（span）$D(N)$ 表示最长依赖路径上的合并数。在单位合并时间的抽象模型中，归约树满足

$$W(N)=\sum_{k=1}^{L}\frac{N}{2^k}=N-1,\qquad D(N)=L=\log_2 N. \tag{15.1}$$

工作量与优化过的顺序算法相同，因此这棵树是工作高效的（work-efficient）。这里比较的是必要合并数；索引计算、数据搬移、同步和后面线程束代码中的冗余通道运算仍然具有实际开销。

若 N 不是二的幂，可以补单位元到 $\widehat{N} = 2^{\lceil \log_2 N \rceil}$。规则树的深度为 $\lceil \log_2 N \rceil$，工作量为 $\widehat{N} - 1 < 2N - 1$；若只合并实际存在的值，仍只需要 $N - 1$ 次合并。后面的分块实现通过边界判断补零，无需真的建立一份全局填充数组。

平均可用并行度为 W/D。当 N 是二的幂时，它等于 $(N-1)/\log_2 N$，峰值出现在第一级，为 $N/2$。例如，对 $N = 1{,}000{,}000$ 个实际输入建立平衡归约树时，深度为 $\lceil \log_2 N \rceil = 20$，工作量为 $N - 1$，平均并行度约为 50,000，第一级包含 500,000 次独立合并。这些数量说明，要把树深度直接兑现成运行时间，需要非常多的执行资源。

若硬件同一时刻最多执行 P 次合并，则 $T_P \geq \max\{W/P, D\}$；忽略通信等开销的理想调度时间量级为 $\Theta(W/P + D)$。因此，$\Theta(\log N)$ 描述依赖深度，并不能单独给出实际图形处理器（GPU）的运行时间。树越接近根，活跃操作越少，后期必然出现并行度收缩。一个核函数启动后线程块的线程数固定，停止做加法的线程也不会自动变成新线程去处理下一批数据。

15.1.3 原子累加与局部合并

最直接的并行求和方式，是让每个线程读取一个输入，再用原子操作（atomic operation）把它加到同一个全局输出上。原子加法把一次读、加和写作为不可分割的更新，保证同时到达的线程不会因覆盖彼此的写入而丢失贡献。

代码 15.1 逐元素原子求和：同一个输出地址接收全部更新

```
__global__ void reduce_atomic(const float* input,
                              size_t n, float* sum) {
    size_t i = size_t(blockIdx.x) * blockDim.x
             + threadIdx.x;
    if (i < n) {
        atomicAdd(sum, input[i]);
    }
}
```

这段核函数要求输出地址有效，并且在核函数执行前将 `*sum` 设为 0。输出不能与仍待读取的输入区域重叠。即使所有输入读取都能并行，同一地址上的 N 次原子更新仍然相互竞争；这些更新必须以某种顺序生效，难以获得归约树同一级那样的独立合并能力。原子操作保证更新正确，并不会自动消除这个共享地址形成的瓶颈。

一种直接的改进是先让每个线程块完成局部归约，只由该块的一个线程原子累加块和。若每块处理 $2B$ 个元素，块数为 $G = \lceil N/(2B) \rceil$，则全局原子更新数从 N 降为 G。例如 $N = 1{,}048{,}576$、$B = 256$ 时，只需 2048 次块结果更新。局部树负责把大量独立加法分散到各块中，最终的原子操作只连接这些块的结果。

另一种块间合并方法是先把 G 个块和写入独立数组，再启动后续归约核函数，直到只剩

一个值。同一执行流中的后续核函数会在先前核函数完成后使用这些部分和。这两种方案共享相同的块内算法，区别集中在最终结果如何跨块汇合；后面将分别给出它们的使用条件。

15.2 分块归约与内存访问

15.2.1 数据分段与同步边界

归约树相邻两层之间存在数据依赖：后一层要读取前一层写出的部分和。线程块内的屏障 `__syncthreads()` 使块内参与线程在该点会合，并使屏障前的共享内存或全局内存访问对块内后续访问正确可见。普通核函数中的这个屏障只作用于本块，不能等待其他线程块完成它们的树层。

因此，大输入通常按块分段。统一记总输入长度为 N，每块线程数为 B，块号为 b，块内线程号为 t。未粗化时每块处理 $2B$ 个元素，段首为 $s_b = 2Bb$，块数为 $G = \lceil N/(2B) \rceil$。各块在自己的数据段内部完成归约，再把块和交给原子累加或后续核函数。这里将一次全局归约拆成块级局部归约，各段最终仍合成一个结果。通常所说的分段归约（segmented reduction）则为每个给定分段分别输出一个结果，输出含义与本讲任务不同。

局部树的所有线程都必须遵守同一层间同步结构。即使某个线程已经不再执行加法，只要它仍在这一固定块内算法中，就继续经过对应的块屏障。因此，`__syncthreads()` 放在活动线程的条件分支之后。把屏障移入 `if (t < stride)` 内部，会使仍在执行的同块线程走向不一致的同步位置。

下面先用完整数据段比较两种线程映射。两段原地代码均要求 B 为二的幂、启动时恰好使用 B 个线程，且输入缓冲区具有完整的 $2BG$ 个元素；若 $N < 2BG$，应预先在缓冲区尾部补 0。它们会覆盖输入，并使用单独的块结果数组。之后的共享内存实现将显式检查 N，从而直接处理未补齐的尾块。

15.2.2 递增与递减步长的线程映射

第一种映射让线程 t 从相邻的两个位置 $s_b + 2t$ 和 $s_b + 2t + 1$ 开始。第一次合并后，部分和存放在偶数下标；下一次再隔开两个位置合并。步长 s 按 $1, 2, 4, \ldots, B$ 递增，参与条件为 $t \bmod s = 0$。固定的写入位置 $i = s_b + 2t$ 与逐渐增大的第二操作数位置 $i + s$ 共同形成相邻子树的合并。

代码 15.2 递增步长：在全局输入数组中原地合并

```
template<int B>
__global__ void reduce_interleaved(float* data,
                                   float* partial) {
    unsigned int t = threadIdx.x;
    size_t base = size_t(2) * B * blockIdx.x;
    size_t i = base + 2 * t;
    for (unsigned int s = 1; s <= B; s *= 2) {
        if (t % s == 0) {
            data[i] += data[i + s];
        }
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = data[base];
}
```

当步长为 s 的一层结束时，每个保留下来的位置保存连续 $2s$ 个原始元素之和。下一层把两个相邻的长度 $2s$ 的区间合并成长度 $4s$ 的区间，因此可以逐层推出最终段首保存全部 $2B$ 个元素之和。图15.2 给出 $B = 8$ 时的下标变化。

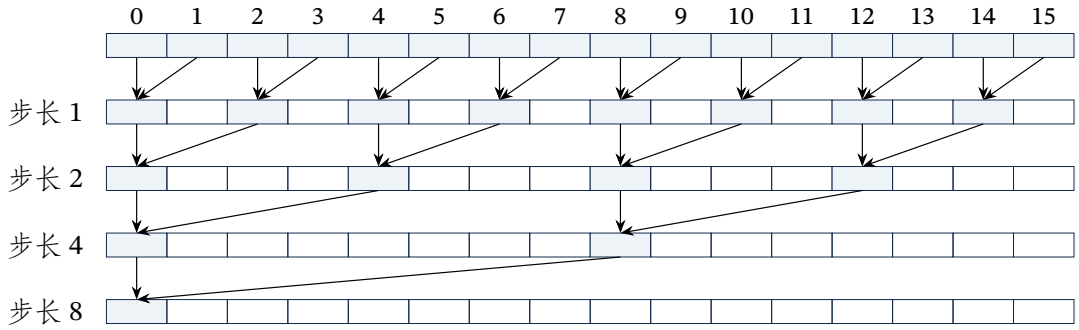


图 15.2 递增步长归约。蓝色位置保存仍会被后续层使用的部分和，存放位置逐层变得稀疏。

这套映射有两个相互独立的性能问题。首先，合并访问 (coalesced access) 关注*同一条内存指令下，相邻线程访问的地址*。在第一层，线程 $0, 1, 2, \ldots$ 的第一操作数分别位于 $0, 2, 4, \ldots$，第二操作数分别位于 $1, 3, 5, \ldots$。每个线程读取的两个位置虽然相邻，每条读指令在整个线程束上的地址却带有空隙，通常需要更多内存事务才能服务相同数量的有效数据。

其次，一个线程束 (warp) 由连续的 32 个线程组成。步长增大后，活动线程变为 $0, 2, 4, \ldots$，

再变为 $0, 4, 8, \ldots$，使许多线程束内同时存在执行加法和跳过加法的线程。这种控制分歧（control divergence）降低了每条指令的有效通道利用率。直到较晚阶段，部分线程束才会整体停止算术工作。

第二种映射让线程 t 首先合并 $s_b + t$ 与 $s_b + t + B$，即将整个数据段的左右两半对应相加。随后步长按 $B, B/2, \ldots, 1$ 递减，活动线程始终是连续前缀 $0 \le t < s$。

代码 15.3 递减步长：使活动线程和保存位置保持连续

```
template<int B>
__global__ void reduce_contiguous(float* data,
                                  float* partial) {
    unsigned int t = threadIdx.x;
    size_t base = size_t(2) * B * blockIdx.x;
    size_t i = base + t;
    for (unsigned int s = B; s > 0; s /= 2) {
        if (t < s) {
            data[i] += data[i + s];
        }
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = data[base];
}
```

每一层把长度 $2s$ 的有效前缀缩短为长度 s 的前缀。两半是互不重叠的部分和集合，合并后仍包含相同的一组原始输入，因而最终总和保持不变。完整的 $2B$ 元素归约一共有 $1 + \log_2 B$ 层；只有在首次 $2B \to B$ 的合并已经完成之后，剩余层数才是 $\log_2 B$。

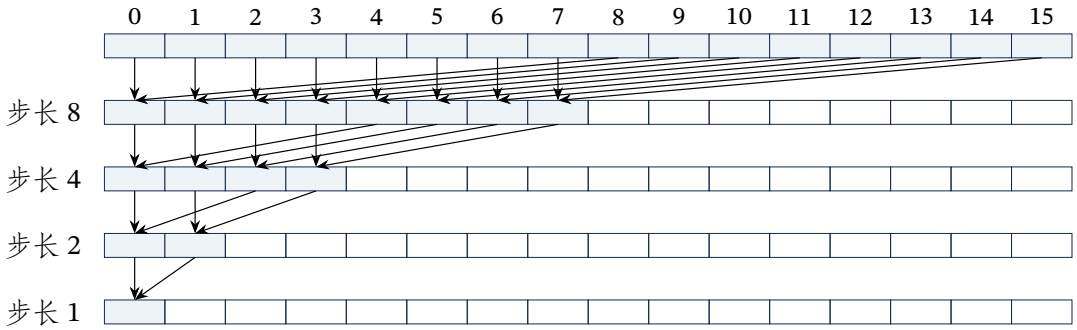


图 15.3 递减步长归约。每层把右半部分并入左半部分，有效部分和始终紧凑地位于前缀中。

在相同的读取指令下，活动线程的地址现在依次是 $s_b + t$ 或 $s_b + t + s$，两组访问各自连续。只要起始地址满足相应对齐条件，这种映射有利于合并访存。在活动线程数至少为 32 且为 32 的倍数时，活动范围包含若干完整线程束，因此每个线程束内部的分支结果一致；到最后不足 32 个线程工作时，分歧才集中出现在第一个线程束中。

例如 $B = 128$ 时，两种算法各层的活动线程数都依次为 $128, 64, 32, 16, 8, 4, 2, 1$，但含有活动线程的线程束数不同。表15.2 只统计执行加法的线程束，未参与加法的线程仍会按代码

经过块屏障。

表 15.2 每块 128 个线程时，两种映射的算术活动分布

树层序号	0	1	2	3	4	5	6	7
活动线程数	128	64	32	16	8	4	2	1
递增步长的活动线程束数	4	4	4	4	4	4	2	1
递减步长的活动线程束数	4	2	1	1	1	1	1	1

还可以把表中的活动线程束数换算成算术通道利用率。只考虑这些加法指令，将每个含有活动线程的线程束按 32 个通道计费，则递增步长占用的通道槽位为 $32(4+4+4+4+4+4+2+1)=864$，递减步长为 $32(4+2+1+1+1+1+1+1)=384$。两者均完成 $256-1=255$ 次必要加法，因此利用率分别为 $255/864\approx 29.5\%$ 和 $255/384\approx 66.4\%$。这项指标量化了线程映射造成的有效通道比例，不计访存、屏障和调度，也不是 SM 占用率或实测加速比。

两种树具有相同的必要加法次数和依赖深度。性能差别来自这些操作如何分配给线程、每次访存服务多少有效数据，以及线程束内多少通道实际做了有用工作。因此，单看 $O(N)$ 工作量与 $O(\log N)$ 深度，还无法区分这两段代码的性能。

15.2.3 在共享内存中保存中间结果

归约过程会不断读写存放部分和的位置。虽然数值每层都在变化，这些中间存储位置会被反复使用，因此适合放入由同一线程块访问的共享内存（shared memory）。让全局内存只提供原始输入并接收块结果，就可以减少归约树内部对全局地址的读写，也能保留原输入以供后续计算使用。

首次加载可以直接完成 $2B\to B$ 的合并：线程 t 读取两个原始值，在寄存器中相加，再把结果写入共享数组的第 t 项。这样共享数组只需要 B 项，而无需保存 $2B$ 个原始输入。所有槽位写入并同步后，再对这 B 项运行递减步长归约。

代码 15.4 带边界检查的共享内存归约：每块输出一个部分和

```
template<int B>
__global__ void reduce_shared(const float* input,
                              size_t n, float* partial) {
    static_assert(B > 0 && (B & (B - 1)) == 0,
                  "B must be a power of two");
    __shared__ float values[B];
    unsigned int t = threadIdx.x;
    size_t i = size_t(2) * B * blockIdx.x + t;

    float a = (i < n) ? input[i] : 0.0f;
    float b = (i + B < n) ? input[i + B] : 0.0f;
    values[t] = a + b;
    __syncthreads();

    for (unsigned int s = B / 2; s > 0; s /= 2) {
        if (t < s) values[t] += values[t + s];
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = values[0];
}
```

该核函数使用一维网格和一维线程块，并要求实际启动线程数等于模板参数 B，同时符合设备的块大小限制。输出数组至少容纳 G 项，且与输入区间不重叠；例如另一块尚未完成读取时，不能把紧凑块结果写回输入数组前部。共享数组中每个槽位都由对应线程初始化；两个输入地址独立进行边界判断，保证尾块中的有效输入不会被遗漏。

以 $B = 128$、某一尾块仅剩 150 个有效输入为例，线程 0 至 21 各读取两个有效值，线程 22 至 127 各读取一个有效值并配上一个 0。整个块仍初始化 128 个共享槽位，并沿同一棵树归约。线程是否读到真实输入，与线程是否参与后续集体计算是两件事；补零使缺少输入的槽位仍具有定义良好的值，也保持了固定的同步结构。

初始化后的第一道屏障保证任何线程读取别人的共享槽位时，该槽位已经写好。循环中，每层的写入目标是前半个有效区间，第二操作数来自后半个有效区间；不同活跃线程的写入位置互不相同，且不会覆盖同层其他线程所需的第二操作数。层末屏障又保证后一层看到已经完成的前一层结果。这个不变量同时给出正确性依据和同步位置的依据。

对一个完整块，按源码中的标量操作计数，原地全局内存树有 $2B - 1$ 次必要加法，每次涉及两次读和一次写；共享内存版本只从全局输入读取 $2B$ 个值，再向块结果数组写一个值。缓存、寄存器保留和内存事务合并会影响实际访存次数，因此这种源码级比较用于识别访问结构，不能直接当作显存总线上的字节测量。

若输入为四字节单精度数，全部 N 个输入各读取一次、G 个块结果各写入一次，对应 $4N + 4G$ 字节的有效全局数据。所做加法与 N 同阶，所以数据搬移、同步以及指令利用率容

易主导耗时。共享内存优化解决了中间结果的存放问题，树后半段的块级同步和活动通道减少仍然存在。

在上面的直接写法中，每块执行一次初始化屏障和 $\log_2 B$ 次循环屏障，共 $1+\log_2 B$ 次。最后一层之后的屏障对只由线程 0 读取自身结果而言可以省略，这里保留统一的循环形式，便于后续比较。进一步减少同步，需要利用线程束内部更小的通信范围。

15.3 线程束内通信与两阶段归约

15.3.1 线程束内的寄存器交换

共享内存归约到后几层时，真正执行加法的线程已经集中到第一个线程束。此时，所需通信范围只有同一线程束内的线程，可以使用线程束交换（warp shuffle）直接传递寄存器中的数值。与先写共享内存、同步、再读取相比，这种方式减少了中间存储访问，也使协调范围缩小到参与交换的线程束。

记线程在束内的通道编号（lane ID）为 $\ell = t \bmod 32$，其寄存器值为 v_ℓ。交换指令让线程获得另一个通道在相应调用中提供的值，而不用把数值先写到共享数组。表15.3 按宽度为 32 的一个线程束给出四种常用寻址方式，d 为偏移或异或掩码，k 为指定的源通道，取 $0 \le k < 32$。

表 15.3 线程束交换原语的寻址关系

原语	源通道	作用
`__shfl_sync`	k	从指定通道取值，可用于广播
`__shfl_up_sync`	$\ell - d$	从编号更小的通道取值
`__shfl_down_sync`	$\ell + d$	从编号更大的通道取值，适合向第 0 通道汇合
`__shfl_xor_sync`	$\ell \oplus d$	按编号的按位异或关系交换

在向上或向下交换时，源编号越过当前宽度的边界会返回调用线程自身提供的值，编号不会环绕。交换宽度可以设为不超过 32 的二的幂，用于把线程束划分为连续的小组；后面的正式代码统一采用默认宽度 32。

交换调用的参与掩码（mask）描述*当前线程束*中哪些通道参与协调。`0xffffffffu` 的 32 位全为 1，对应完整线程束。下面统一要求一个线程束内 32 个线程都执行相同位置的交换调用，并使用相同掩码；被读取的源通道也必须实际参与相应调用。掩码不会把不存在的输入变成 0，也不会把稀疏的通道重新排成连续编号。因此，尾部输入的补零在交换之前完成。

代码 15.5 完整线程束内求和：仅第 0 通道的返回值用于输出

```
__device__ __forceinline__ float warp_sum(float value) {
    for (int offset = 16; offset > 0; offset /= 2) {
        value += __shfl_down_sync(0xffffffffu,
                                  value, offset);
    }
    return value;
}
```

这个函数依次使用偏移 $16, 8, 4, 2, 1$，共五轮交换和加法。第一次使低编号的前 16 个通道各合并两个值，第二次使前 8 个通道各覆盖四个原始值，依此类推，最终第 0 通道覆盖全部 32 个输入。所有通道都执行代码，函数只保证第 0 通道的返回值是整束总和；其他通道的值不能统一当作完整和或一般后缀和使用。

用一个完整的八通道子组可以手算同样的递减偏移规则。令初值依次为 $4, 7, 2, 3, 8, 5, 9, 6$，子组宽度设为 8，偏移依次为 $4, 2, 1$。表15.4 只列出仍会沿归约树进入第 0 通道最终结果的部分和，所有子组成员仍参与每轮交换。

表 15.4 八通道子组的交换归约手算

阶段	与最终结果相关的部分和	第 0 通道的值
初始化	$4, 7, 2, 3, 8, 5, 9, 6$	4
偏移 4	$12, 12, 11, 9$	$4 + 8 = 12$
偏移 2	$23, 21$	$(4 + 8) + (2 + 9) = 23$
偏移 1	44	$23 + 21 = 44$

同一轮交换读取的是其他通道为该次调用提供的寄存器值，随后各线程更新自己的局部变量。程序因此无需用共享内存模拟这些寄存器交换。交换调用本身不提供共享内存或全局内存访问的排序保证；跨线程束传递共享数组中的结果时，仍须使用相应的块级同步。

15.3.2 用线程束交换完成归约树的末段

一种渐进优化是保留共享内存树的前半部分，等到只剩较少的部分和时，再由第一个线程束完成归约。以下取 $B \geq 64$ 且 B 为二的幂，沿用上一节共享内存核函数的输入加载、共享数组 `values` 及其后的第一道屏障，把归约循环和输出部分替换为下面的代码。

代码 15.6 共享内存树结束后，把最后 64 个部分和交给一个线程束

```
for (unsigned int s = B / 2; s > 32; s /= 2) {
    if (t < s) values[t] += values[t + s];
    __syncthreads();
}

if (t < 32) {
    float value = values[t] + values[t + 32];
    value = warp_sum(value);
    if (t == 0) partial[blockIdx.x] = value;
}
```

循环条件 $s > 32$ 的含义是，最后一次共享内存层使用 $s = 64$，将 128 个部分和压缩为 64 个；对于 $B = 64$，初始加载后已经只有 64 个部分和，循环可以直接跳过。随后前 32 个线程先执行 `values[t] + values[t+32]`，完成必要的 $64 \to 32$ 合并，再用五轮交换完成 $32 \to 1$。若直接只读取前 32 项而省略这次加法，就会漏掉另外 32 个部分和。

当 $B = 256$ 时，初始加载先将 512 个输入变为 256 个共享值，随后共享内存层按 $s = 128, 64$ 运行两次；前 32 个线程接收最后 64 个部分和。块屏障数由直接共享树的 $1+\log_2 256 = 9$ 次降到 3 次：一次初始加载屏障，以及两次共享树层末屏障。

分支 `if (t < 32)` 只选择完整的第一个线程束，其他线程束不调用 `warp_sum`。这符合完整掩码的约定，因为掩码的范围是各自的线程束。需要严格保留的是所选线程束的全部 32 个参与线程，以及读取共享数组前已经完成的块屏障。当前片段还依赖 `values[t+32]` 有效，因此不能直接用于仅 32 个线程的块；这种配置可在初始加载后直接对各线程自己的值做一次线程束归约。

15.3.3 显式线程组与协作组

协作组（Cooperative Groups）接口把参与某次协作的线程集合表示为一个组对象。组可以对应整个网格、线程块簇、一个线程块，以及由线程块划分出的多线程束、单线程束或更小的线程组。这里使用由连续 32 个线程组成的分块组，使组内交换的参与范围直接体现在函数参数中。

使用协作组时，需要包含对应头文件，并由线程块成员共同取得块句柄和创建分组。下面的分组语句放在核函数中、任何只允许部分线程进入的分支之前；实际块大小取 32 的倍数。

代码 15.7 在核函数内由整个线程块共同建立 32 线程分组

```
// File scope:
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

// Inside the kernel, reached by every block thread:
cg::thread_block block = cg::this_thread_block();
cg::thread_block_tile<32> tile =
    cg::tiled_partition<32>(block);
```

这里每个线程得到的 `tile` 表示自己所属的 32 线程组。`tile.thread_rank()` 给出该线程在组内的编号，`tile.size()` 返回组大小，`tile.shfl_down(value, offset)` 则对应在组内向较大编号通道取值。原先的完整线程束辅助函数可以写成下面的形式。

代码 15.8 把参与交换的线程组显式传入归约函数

```
__device__ __forceinline__
float warp_sum_group(const cg::thread_block_tile<32>& tile,
                     float value) {
    for (int offset = tile.size() / 2;
         offset > 0; offset /= 2) {
        value += tile.shfl_down(value, offset);
    }
    return value;
}
```

分组操作 `tiled_partition<32>(block)` 是父组上的集体操作，必须由整个父线程块共同调用。因此，应先完成分组，再把 `tile` 传入归约函数。若把整块分组的创建语句放进一个只由第一个线程束调用的函数，就破坏了父组全员参与的条件。对于已经创建的 32 线程子组，后续归约只要求该子组的成员共同参与；其他子组可以执行自己的归约或别的工作。

协作组表达了通信范围，算法仍然需要正确地决定何时使用组内交换、何时进行跨线程束的共享内存同步。网格级或线程块簇级协作还有对应的执行条件；这里的普通块内实现只使用线程块与 32 线程子组。

15.3.4 先在各线程束内归约，再合并线程束结果

只优化最后一个线程束，仍会在共享内存树的前几层执行多次块屏障。两阶段线程束归约可以把更多的计算留在寄存器中：第一阶段，各线程束独立完成自己的局部归约；第二阶段，由第一个线程束归约各束的结果。跨线程束的数据交换只发生在这两个阶段之间。

令每块有 $Q = B/32$ 个线程束，束号为 $q = \lfloor t/32 \rfloor$，通道号为 $\ell = t \bmod 32$。首先，每个线程在寄存器中形成 $v_t = x_{s_b+t} + x_{s_b+t+B}$。线程束 q 对自己的 32 个 v_t 求和，由其第 0 通道把结果 u_q 写入共享数组。块屏障之后，第一个线程束的前 Q 个通道读入 $u_0, \dots, u_{Q-1}$，其他通道取 0，再完成一次完整线程束归约。

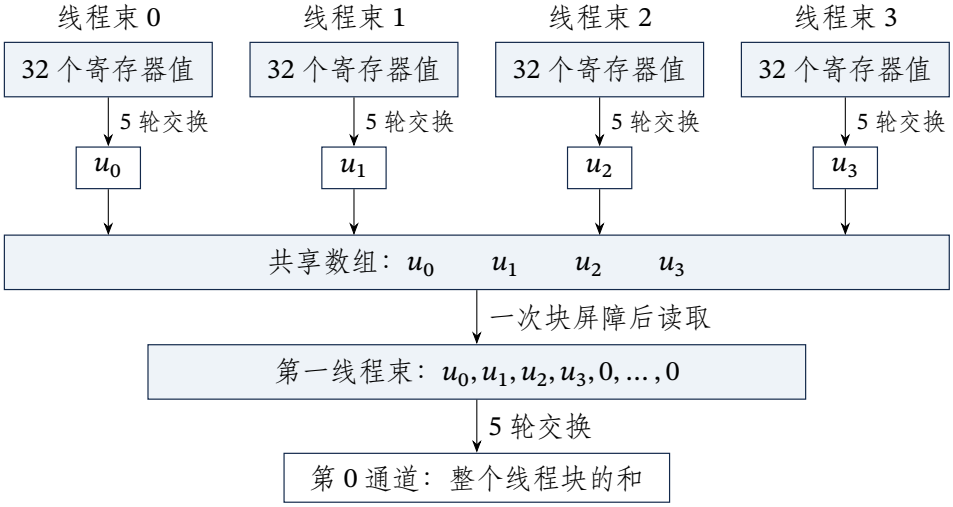


图 15.4 每块四个线程束的两阶段归约。各线程束先独立产生一个结果，只在共享数组交接处需要一次块屏障；第二阶段用零补齐到 32 个通道。

代码 15.9 两阶段线程束归约：一个块屏障与正确的第二阶段补零

```
template<int B>
__global__ void reduce_two_stage(const float* input,
                                 size_t n, float* partial) {
    static_assert(B >= 32 && B <= 1024 && B % 32 == 0,
                  "B must contain complete warps");
    constexpr int Q = B / 32;
    __shared__ float warp_totals[Q];
    unsigned int t = threadIdx.x;
    size_t i = size_t(2) * B * blockIdx.x + t;

    float a = (i < n) ? input[i] : 0.0f;
    float b = (i + B < n) ? input[i + B] : 0.0f;
    float value = warp_sum(a + b);
    if (t % 32 == 0) warp_totals[t / 32] = value;
    __syncthreads();

    if (t < 32) {
        value = (t < Q) ? warp_totals[t] : 0.0f;
        value = warp_sum(value);
        if (t == 0) partial[blockIdx.x] = value;
    }
}
```

与前面的核函数一样，实际启动线程数必须等于 B，满足设备限制，且输出数组与输入不重叠。这里要求 B 是 32 的倍数，使第一阶段所有线程束完整；再要求 $B \le 1024$，使 $Q \le 32$，全部束结果可以交给一个线程束。由于第二阶段显式补零，Q 不必是二的幂，因此 $B = 96$ 这样的三个完整线程束也可以使用这套算法。

第二阶段的边界判断十分关键。例如 $B = 128$ 时，共享数组只有 $Q = 4$ 项；前 32 个线程如果无条件读取 `warp_totals[t]`，线程 4 至 31 就会越界。正确结构是先让四个通道读取真实部分和，其余通道设为 0，然后让完整的第一个线程束共同调用 `warp_sum`。只让前四个线程进入原有完整掩码函数，同样不能满足该函数约定。

唯一的块屏障保证所有 u_q 已写入共享数组，再由第一个线程束统一读取。最终写出只由线程 0 执行，各块写到自己的输出位置。对 $B = 128$，共享数组由直接共享树中的 128 个浮点数减到 4 个，即由 512 字节降到 16 字节；直接共享树的八次块屏障也减为一次。

这套代码中，各线程束都分别经历五轮交换，后面一些通道的运算结果不再进入第 0 通道的最终总和；第二阶段还包含对补零通道的运算。因而，按线程累计的标量加法次数会多于理想树的 $2B - 1$ 次必要合并。代码通过减少共享内存访问和块屏障来交换这些额外运算；其工作量仍随输入规模线性增长。这里应区分有效贡献通道减少与控制分歧：上述交换循环由全部通道无条件执行，通道利用率下降并不等同于源代码出现了新的按通道分支。

使用协作组时，核函数开始处先由全块建立子组句柄。每次束内归约均通过辅助函数

`warp_sum_group` 执行，将该句柄与当前部分和作为两个实参传入。两阶段之间保留块屏障，第二阶段由完整的第一个子组处理补齐后的束结果。

15.4 线程粗化与完整归约实现

15.4.1 增加每个线程承担的局部工作

前面的未粗化实现让每个线程先处理两个输入，再进入块内归约。对很大的输入，线程块数量可能远多于硬件能同时驻留的块数，块会分批执行。每批数据都会再次经历归约树从高并行度逐渐收缩到低并行度的过程，并支付相应的块内通信代价。

线程粗化（thread coarsening）让同一个线程先顺序合并更多输入，再执行一次块内归约。设粗化因子为正整数 C，则每个线程负责 $2C$ 个输入，每个块处理 $2CB$ 个输入，段首改为 $s_b = 2CBb$。每个线程在寄存器中形成

$$v_t = \sum_{j=0}^{2C-1} \tilde{x}_{s_b+t+jB}, \qquad \tilde{x}_i = \begin{cases} x_i, & i < N, \\ 0, & i \geq N. \end{cases} \tag{15.2}$$

这里 $\tilde{x}_i$ 表示逻辑补零后的输入，无需真的分配填充数组。完成局部累加后，只需再对 B 个 v_t 使用前面任一种块内归约结构。

地址 $s_b + t + jB$ 使同一轮 j 中相邻线程访问连续位置。以 $B = 128$、$C = 2$、$b = 0$ 为例，线程 5 读取下标 $5, 133, 261, 389$，线程 6 读取 $6, 134, 262, 390$。每个线程沿自己的序列顺序累加，而各线程的同一轮访问仍组成连续地址区间。图15.5 用较小的线程数展示这种结构。

块数随之变为 $G_C = \lceil N/(2CB) \rceil$。与未粗化相比，局部顺序工作增多，而需要启动和归约的线程块减少，块间原子更新数或写出的块结果数也随之减少。这个取舍在原本块数充足、硬件不得不分批处理时尤其有意义。

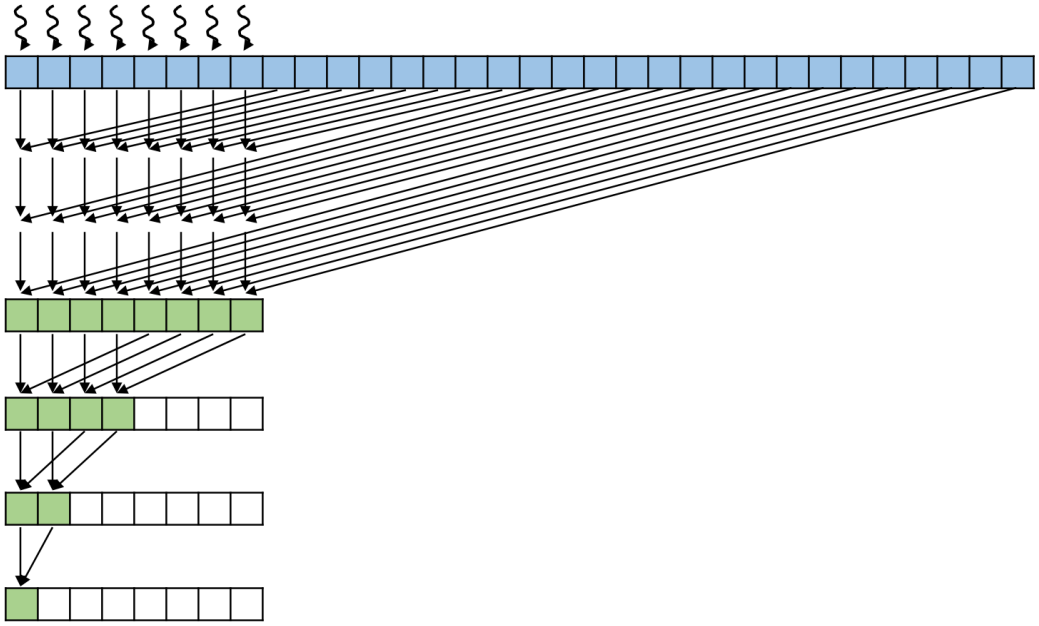


图 15.5 图示采用八个线程，每个线程先累加四个输入，再对八个局部部分和进行树形归约。上方的多轮读取保持跨线程的连续地址关系。

15.4.2 工作量、阶段数与同步次数的比较

为分析粗化如何改变树尾的重复开销，令原始每块输入数为 $M_0 = 2B$，并取 B 为二的幂。原始块内树深度为 $L_B = \log_2 M_0 = 1 + \log_2 B$。以下先使用单位加法时间、忽略数据传输细节的理想模型，比较处理同样 $2CB$ 个输入时的局部阶段数。

若未粗化的 C 个块必须由硬件依次执行，则需要约 CL_B 个阶段。粗化后，每个线程先把 $2C$ 个值顺序合为一个值，沿该线程的依赖链需要 $2C-1$ 次加法；之后再运行深度为 $\log_2 B$ 的块内树。因此，粗化后的跨度和必要工作量为

$$D_C = (2C-1) + \log_2 B = 2(C-1) + L_B, \tag{15.3}$$

$$W_C = B(2C-1) + (B-1) = 2CB - 1. \tag{15.4}$$

工作量仍恰好等于把 $2CB$ 个值合并成一个值所需的次数。未粗化的 C 个块内部一共做 $C(2B-1)$ 次必要加法，若再把这 C 个块和合成一个结果，还要做 $C-1$ 次，因此总数同样是 $2CB-1$。两种方案主要改变工作在线程内部、块内部和块之间的分配。

上述 CL_B 只计原有数据段依次完成块级输出的阶段，得到的是 C 个块和；粗化后的一个块直接输出覆盖同样数据的一个块和。实际总耗时还应考虑最后的块间合并。若原来的 C 个块本就能同时执行，则它们局部归约的并行深度只有 L_B，粗化反而增加了线程内部的顺序依赖，因此粗化因子增大不保证获得更短的时间。

以 $B = 128$、$C = 4$ 为例，原始每块有 $M_0 = 256$ 个输入，$L_B = 8$。四个块顺次执行的局部阶段数为 $4 \times 8 = 32$；粗化后，一个块处理 1024 个输入，每线程合并八个值需要七次加法，再经过七层块内树，共 $7 + 7 = 14$ 个阶段。这与式(15.3) 的 $2(4-1)+8 = 14$ 一致。$32/14 \approx 2.29$ 是这一理想模型中的阶段数比，并非实测加速比。

同步次数必须按实际块内实现分别计算。若采用每个树层末尾均保留块屏障的经典结构，C 个原始块共有 CL_B 次块屏障调用，粗化后只需对 B 个局部值执行一次归约，连同共享值初始化后的屏障，共 L_B 次。对于 $B = 128, C = 4$，计数从 32 次降至 8 次。若已经采用两阶段线程束归约，则每块原本只有一次块屏障，粗化把对应的 C 次降为一次，即这里的 $4 \to 1$。

因此，选择 C 时需要同时观察局部顺序依赖、块数和同步开销。较大的 C 能减少重复的块内归约及块结果写出，但也会减少可调度的块数；输入规模较小时，剩余块数可能不足以利用所有计算资源。循环展开及更多同时保留的中间值还可能提高寄存器占用。实际参数应依据固定输入规模下的运行时间比较，并保持相同的计时范围和数值正确性标准。

式(15.3) 使用的是理想二叉块内树。若采用固定 32 通道的两阶段交换代码，补零通道及不再进入最终结果的通道仍会执行部分指令，实际指令路径不必恰好等于这个理想层数。阶段数模型解释了粗化的来源，最终性能仍由所用代码与硬件共同决定。

15.4.3 完整核函数与块间合并

下面把每线程局部累加、两阶段线程束归约和最终输出连接起来。实现采用 C++17，使用前面定义的 `warp_sum` 辅助函数；源文件包含 `<cuda_runtime.h>` 和 `<cstddef>`。模板参数 B 表示每块线程数，C 表示粗化因子，编译期布尔参数 `AtomicOutput` 选择直接原子累加总和，或写出每个块的部分和。

对一个完整块，初始化直接读取每个线程的第一个输入，后面的循环进行 $2C-1$ 次加法，与前面的局部跨度计数一致。尾块的读取分别检查边界，缺少输入的线程保留零或已有部分和，并继续参加完整的线程束归约。输入索引在乘法之前转换为 `size_t`，避免先用较窄整数计算大数组地址。

输出选择只在每块的第 0 个线程执行。采用原子输出时，输出只需一个浮点数，在所有块开始累加前初始化一次为 0；采用部分和输出时，输出需要 G_C 个浮点数，由各块分别覆盖自己的位置。两种模式均要求输出区间不与当前输入重叠。

代码 15.10 带线程粗化的完整归约核函数

```
template<int B, int C, bool AtomicOutput>
__global__ void reduce_coarsened(const float* input,
                                 size_t n, float* output) {
    static_assert(B >= 32 && B <= 1024 && B % 32 == 0,
                  "B must contain complete warps");
    static_assert(C >= 1, "C must be positive");
    constexpr int Q = B / 32;
    __shared__ float warp_totals[Q];
    unsigned int t = threadIdx.x;
    size_t i = size_t(2) * C * B * blockIdx.x + t;

    float value = (i < n) ? input[i] : 0.0f;
    for (size_t tile = 1; tile < size_t(2) * C; ++tile) {
        size_t j = i + tile * B;
        if (j < n) value += input[j];
    }

    value = warp_sum(value);
    if (t % 32 == 0) warp_totals[t / 32] = value;
    __syncthreads();

    if (t < 32) {
        value = (t < Q) ? warp_totals[t] : 0.0f;
        value = warp_sum(value);
        if (t == 0) {
            if constexpr (AtomicOutput) {
                atomicAdd(output, value);
            } else {
                output[blockIdx.x] = value;
            }
        }
    }
}
```

原子模式可以使用下面的主机端启动函数。输入和输出指针均指向已经分配好的设备内存，块数应在设备允许的网格范围内。初始化与核函数在同一执行流（stream）中提交，使清零先于所有块的累加执行。

代码 15.11 在同一执行流中初始化输出并启动归约

```
template<int B, int C>
cudaError_t launch_atomic_sum(const float* input,
                              size_t n, float* sum,
                              cudaStream_t stream = nullptr) {
    cudaError_t status =
        cudaMemsetAsync(sum, 0, sizeof(float), stream);
    if (status != cudaSuccess || n == 0) return status;

    constexpr size_t chunk = size_t(2) * B * C;
    size_t blocks = n / chunk + (n % chunk != 0);
    reduce_coarsened<B, C, true>
        <<<static_cast<unsigned int>(blocks), B, 0, stream>>>
        (input, n, sum);
    return cudaGetLastError();
}
```

以 $B = 256, C = 4$ 为例，每块处理 2048 个输入。调用时将 `launch_atomic_sum` 的两个模板实参依次设为 256 和 4。空输入只执行输出清零，不启动零个线程块的核函数。启动函数是异步提交：返回的状态用于检查提交和启动错误；主机要读取数值时，还需等待该执行流完成并检查执行错误，再把设备结果复制到主机。

采用多轮部分和归约时，把 `AtomicOutput` 设为 `false`。空输入直接把最终结果设为单位元 0，单元素输入可以直接返回或复制该值，均无需进入多轮过程。以下递推取 $N \geq 2$，设当前输入长度为 n_k，每块容量为 $M = 2BC$，则下一轮的输入长度为

$$n_{k+1} = \left\lceil \frac{n_k}{M} \right\rceil, \qquad n_0 = N. \tag{15.5}$$

第一轮从原始输入写入临时数组，后续在两个互不重叠的临时数组之间交替读写，直到只剩一个部分和。每轮在同一执行流中依次提交，核函数边界为下一轮提供完整的块间阶段顺序。部分和输出模式会覆盖本轮的全部有效输出位置，因此这些输出槽位不需要像原子总和那样预先清零。

两种块间合并方法的主要差别是，原子模式用一次核函数完成所有块的局部计算，并竞争同一个最终地址；多轮模式把块间合并继续表示成较小的归约任务，需要额外的核函数启动和临时结果读写。输入规模、块数和设备特性共同决定其性能，比较时应包含所选方案完成一次归约实际需要的全部阶段。

15.5 综合算例与关键结论

15.5.1 非整块输入的完整归约过程

考虑 $N = 1000$ 个全部等于 1 的输入，取 $B = 128$、$C = 2$，使用两阶段线程束归约。每个块处理 $2CB = 512$ 个逻辑位置，所以需要 $\lceil 1000/512 \rceil = 2$ 个块。第 0 块处理下标 0 至 511，全部位置有效；第 1 块从下标 512 开始，只有 488 个真实输入，其余 24 个逻辑位置补零。

尾块线程 t 依次尝试读取 $512+t$、$640+t$、$768+t$ 和 $896+t$。前三次读取对所有 $0 \le t < 128$ 都有效；第四次读取要求 $896 + t < 1000$，即 $t \le 103$。因此线程 0 至 103 的寄存器部分和为 4，线程 104 至 127 的部分和为 3。边界处，线程 103 读取到最后一个有效下标 999，线程 104 的最后一次读取被边界判断跳过。

前三个线程束分别包含 32 个局部值 4，各得到 128。最后一个线程束包含线程 96 至 127，其中八个局部值为 4，二十四个局部值为 3，因此得到 $8 \times 4 + 24 \times 3 = 104$。与完整块相比，两块的中间结果如表15.5 所示。

表 15.5 1000 个全 1 输入在两个线程块中的归约结果

阶段或数量	第 0 块	第 1 块
有效输入数	512	488
逻辑补零数	0	24
四个线程束的和	128, 128, 128, 128	128, 128, 128, 104
第一个线程束第二次归约	512	488

在两个块的第二阶段，第一线程束均先读取四个共享部分和，再用 28 个零补齐为 32 项。它们共同产生块和 512 与 488。若使用原子输出，两个块分别向已经清零的总和地址执行一次原子加法，最终得到 1000。

若使用部分和输出，第一轮的结果数组是 $[512, 488]$，第二轮再对这个长度为 2 的数组归约，长度变化为 $1000 \to 2 \to 1$。第二轮仍可启动 128 个线程，只有线程 0、1 的首次输入读取有效，其余逻辑输入均取 0；所有线程继续遵守原有线程束与块屏障结构。第一次线程束归约得到一个 1000 和三个 0，第二次线程束归约得到最终的 1000。

这个算例说明，输入长度不足一个块或不足一个线程束时，仍可以保持固定的线程组织，只在数据加载处改变有效值与单位元的分配。有效输入完整参与、共享数组初始化以及完整线程束参与由此能够同时成立。

15.5.2 大规模输入需要多少轮核函数

考虑 $N = 1{,}000{,}000$，取 $B = 256$、$C = 4$，每块容量为 $M = 2048$。由于 $1{,}000{,}000 = 488 \times 2048 + 576$，第一轮有 488 个完整块和一个尾块，共产生 489 个部分和。尾块的段首为 999424，包含 576 个有效输入，其余 1472 个逻辑位置补零。

若继续用同样的配置归约部分和数组，第二轮只需一个块，因为 $489 < 2048$。完整的长度变化和块数如表15.6 所示。

表 15.6 百万元素的多轮分块归约

轮次	输入长度	每块容量	启动块数	最后一个块的有效输入数
第一轮	1,000,000	2048	489	576
第二轮	489	2048	1	489

若输入也全部为 1，第一轮前 488 个块各输出 2048，尾块输出 576；第二轮将这些值合为 $488 \times 2048 + 576 = 1{,}000{,}000$。这个输入规模在理想二叉树中具有约 20 层依赖，但多轮实现只启动两次核函数，因为每次核函数内部已经完成了每线程局部累加和块内的多级归约。核函数轮次、树的算术深度和块内屏障阶段数分别描述不同层次的工作。

若改用原子输出，同样的第一轮仍需 489 个块，但每块直接贡献一次原子加法，从而在一次核函数中完成全局结果。此时原子更新数量为 489，而逐元素原子方案需要 1,000,000 次。是否减少了一次核函数启动、是否保留最终同地址竞争，应一起计入方案比较。

15.5.3 归约结构与开销的联系

为统一比较，表15.7 取 $B \geq 64$ 且为二的幂，统计每块的同步阶段数；一次块屏障由参与线程共同执行，表中没有把它按线程重复累计。直接树保留前面代码中的末次屏障。两阶段方法本身只要求由完整线程束组成，表中的二的幂条件用于与其他树形实现采用相同配置。

表 15.7 不同块内实现的存储与同步开销

实现	中间存储	每块屏障阶段数	主要变化
递增步长全局树	全局数组	$1 + \log_2 B$	活动线程及地址逐层稀疏
递减步长全局树	全局数组	$1 + \log_2 B$	连续地址与连续活动前缀
共享内存树	B 个共享值	$1 + \log_2 B$	中间读写留在共享内存
共享树与末段交换	B 个共享值及寄存器	$\log_2 B - 5$	最后 64 项交给一个线程束
两阶段线程束归约	$B/32$ 个共享值及寄存器	1	各束独立归约，只在交接处同步
粗化与两阶段归约	$B/32$ 个共享值及寄存器	1	每线程先处理 $2C$ 个输入，减少块数

归约优化涉及三个相互联系的选择：怎样把二元合并分配给线程、在哪一级存储中传递部分和，以及让哪些线程共同同步。递减步长改善线程与地址的布局；共享内存和线程束交换逐步降低中间通信代价；线程粗化把更多工作放到单线程内部，以减少重复的块内归约。它们分别作用于不同的开销，因此可以组合使用。

正确性可以沿输入覆盖和阶段依赖追踪：每个有效输入进入一个且仅一个局部和，跨线程读取发生在所需结果已可见之后，每个块结果被全局归约恰好使用一次。改变存放位置、分组和同步范围时，这些关系需要同时保留。浮点输入还应按所要求的误差标准验证数值结果，因为不同的合法归约顺序可能产生不同的舍入误差。